

MTSF — Market-Theoretic Security Framework: A Unified Paradigm For The Art Of Proving and Disproving Security

Basker Palaniswamy^{1*} 

Paolo Palmieri²  *

March 2026

Abstract

Cryptographic security proofs remain fragmented across game-based proofs, Universal Composability (UC), formal verification, and ad hoc insecurity arguments—each with its own language, assumptions, and limitations. This article introduces the **Market-Theoretic Security Framework (MTSF)**, a unified paradigm that reinterprets all security proofs as economic markets. The defender acts as a seller offering *security goods* (confidentiality, unforgeability), while the adversary acts as a buyer bidding computational resources to break them. Security emerges as *market equilibrium*—no efficient adversary can afford to win—while insecurity is *market collapse*—attacks succeed at negligible cost.

MTSF introduces key technical innovations: the **extended difference lemma** for handling multiple simultaneous failure events, **bidding-based reductions** that explicitly model adversarial strategies, a **dual methodology treating proofs and disproofs symmetrically**, and a **session pinging mechanism** for unbounded session verification. The framework extends seamlessly to classical and post-quantum primitives, real-world protocols (TLS 1.3, Signal), and quantum-adversarial settings, preserving quantitative security bounds and composability guarantees.

Beyond unification, MTSF provides a systematic methodology for **detecting unknown (zero-day) attacks**. By exhaustively exploring the bidding space, inverting CNF session clauses, synthesising cross-session attack vectors, and analysing infrastructure dependencies, MTSF enables proactive vulnerability discovery that traditional proof techniques miss. We demonstrate five novel attacks: a **TLS 1.3 session-resumption nonce-collision vulnerability** exploiting counter reuse, a **signature verification bypass in ISO/IEC 9798-3** via message-field parsing confusion, a **WPA2 group-key negotiation race condition** enabling temporal desynchronisation, a **Grain-128a related-IV keystream correlation** in IoT deployments with low-entropy IV generation, and an **ASCON-Hash cross-function domain confusion** exploiting shared-permutation multi-purpose deployments. Each was invisible to prior analyses because it operated outside the assumed adversary model; MTSF’s market perspective—“what is the cheapest bid that collapses this market?”—systematically revealed them. A formal *attack prevention principle* provides a verifiable criterion—bid-completeness and CNF-robustness—for hardening designs against unknown attacks.

By combining mathematical rigor with economic intuition, MTSF transforms security proofs from opaque technical artifacts into transparent, auditable, and universally understandable arguments, enabling both experts and practitioners to reason about security with clarity and confidence.

Keywords: Market-theoretic security, auction-based proofs, extended difference lemma, CNF session verification, universal composability, game-based reductions, QROM, protocol analysis, TLS 1.3, Signal protocol, unknown attack detection, zero-day vulnerability discovery, proactive security assessment

*Insight Research Ireland Centre for Data Analytics, Department of Computer Science and Information Technology, University College Cork (UCC), Cork City, Ireland, European Union.

¹Email: basker170889@gmail.com, basker170889@zohomail.eu ²Email: p.palmieri@cs.ucc.ie

¹Corresponding author

Contents

1	Introduction	11
1.1	The Problem: Fragmented Security Paradigms	11
1.2	What MTSF Provides	16
1.3	Key Technical Innovations	18
1.4	Article Organisation	18
2	Preliminaries and Background	19
2.1	Notation	19
2.2	Semantic Security and Core Definitions	20
2.3	Bidding-Round Proofs in the Random Oracle Model	23
2.4	Universal Composability (UC) Framework	24
2.5	GUC and CNF Session Correctness	26
2.6	Formal Verification	27
3	The Market-Theoretic Security Framework	28
3.1	Security Market Model	28
3.2	Game Hops as Price Adjustments	33
3.3	The Extended Difference Lemma	35
3.4	UC as Market Regulation; GUC as Shared Infrastructure	37
3.5	Formal Verification as Market Stress Testing	40
4	CNF Session Verification in MTSF	43
4.1	Why CNF?	43
4.2	Session-CNF Construction	44
4.3	CNF Verification Algorithm	46
5	Unbounded Verification via Session Pinging	47
5.1	Why Bounded Is Not Enough	47
5.2	The Pinging Mechanism	48
5.3	The Unbounded Security Theorem	50
5.4	Interaction Between Session Pinging and CNF Verification	52
5.5	Taxonomy of Ping Failure Modes	53
5.6	Quantitative Advantage Accumulation	55
5.7	Comparison with Symbolic Unbounded Verification	55
6	Unified Novelties	57
7	Soundness and Completeness of MTSF	59
7.1	Subroutine Consumption: The Core Reduction Mechanism	61
7.2	NP-Hard Subroutine Chains: Consuming One Hard Problem to Solve Another . .	63
7.3	Soundness of MTSF	65
7.4	Completeness of MTSF	66
7.5	The Soundness–Completeness Duality	67
7.6	Worked Example: ECDSA Soundness via Subroutine Consumption	69
7.7	Worked Example: NP-Hard Subroutine Chain for CNF-Based Security	70
7.8	Relationship to Classical Meta-Theorems	70
7.9	Implications for MTSF Case Studies	71

8	Protocol-Level Security Games in MTSF	74
8.1	The Authentication Game	74
8.2	The Mutual Authentication Game	77
8.3	The Session-Key Secrecy Game	80
8.4	The CNF Checking Game	83
9	Case Study I: Primitives	85
9.1	ECDSA: Security via Bidding	85
9.2	ML-KEM (FIPS 203): IND-CCA2 via Bidding	93
9.3	ML-DSA (FIPS 204): EUF-CMA via Bidding	97
9.4	Extended Primitives	103
9.5	HMAC: SUF-CMA via Bidding	103
9.6	AEAD: IND-CCA2 + INT-CTXT via Bidding	112
9.7	SLH-DSA (FIPS 205): Hash-Based EUF-CMA via Bidding	116
9.8	FN-DSA (FIPS 206): NTRU-Lattice EUF-CMA via Bidding	123
10	Case Study II: Block-Cipher Market—AES	130
10.1	AES Market Setup	130
10.2	Differential Cryptanalysis Bid	132
10.3	Rotational Cryptanalysis Bid	137
10.4	Related-Key Bid and Combined AES Equilibrium	141
10.5	PRESENT: Lightweight Block-Cipher Market	146
10.6	Serpent: Conservative Block-Cipher Market	152
11	Case Study III: Hash-Function Market—Keccak/SHA-3	157
11.1	BLAKE3: Parallelisable Hash-Function Market	166
11.2	ASCON-Hash: NIST Lightweight Hash-Function Market	172
12	Case Study IV: Stream-Cipher Market—Grain-128a	178
12.1	Grain-128a Market Setup	179
12.2	State Recovery Attack Bid	180
12.3	Key Recovery Attack Bid	184
12.4	Distinguishing Attack Bid	186
12.5	Time-Memory-Data Trade-Off Bid	188
12.6	ChaCha20: ARX Stream-Cipher Market	191
12.7	Trivium: Minimalist Stream-Cipher Market	197
13	Case Study V: Protocols	204
13.1	Two-Party Key Exchange (ISO/IEC 11770-3, Mech. 6)	205
13.2	Three-Party with KDC (Mechanism 11)	210
13.3	Four-Party Cross-Domain	212
13.4	Needham–Schroeder: Proving Insecurity	215
13.5	Unbounded Verification	220
13.6	PKI-Based Mutual Authentication Protocol	221
13.7	Signal Protocol: X3DH and Double Ratchet	229
13.7.1	Insecurity Analysis: Signal Without One-Time Prekeys	240
13.8	TLS 1.3: Security and Insecurity Analysis	242
13.8.1	TLS 1.3 Protocol Description and Key Schedule	243
13.8.2	TLS 1.3 Market Setup	244
13.8.3	TLS 1.3 Security Proof: 1-RTT Handshake Equilibrium	246
13.8.4	TLS 1.3 Insecurity: 0-RTT Replay Collapse	251
13.8.5	TLS 1.3 Insecurity: Downgrade Attack	254

13.8.6	TLS 1.3 CNF Session Verification and Ping Bids	255
13.9	Telegram: Security and Insecurity Analysis	258
13.9.1	MTPROTO 2.0 Market Model	258
13.9.2	Security Disproof: Salt Extraction Collapses Confidentiality	259
13.9.3	Security Proof: Remediated MTPROTO	262
13.9.4	Summary of the Telegram MTSF Analysis	264
14	Case Study VI: QROM-Based Key Exchange	264
14.1	Protocol Description: QROM-Secure Key Exchange	266
14.2	Protocol Sequence Diagram	267
14.3	MTSF Market Model for QKEM	269
14.4	Security Proof in the QROM	270
14.5	Extended Protocol Security Goods	273
15	Case Study VII: Quantum Market Dynamics—BB84 QKD	274
15.1	Protocol Description: BB84 QKD	275
15.2	Protocol Sequence Diagram	276
15.3	MTSF Quantum Market Model for BB84	276
15.4	Security Proof via Quantum Bidding Rounds	279
15.5	Extended Security Goods and CNF Verification	281
16	Case Study VIII: Multi-Protocol Composition—TLS 1.3 + Signal Network	283
16.1	Network Description	284
16.2	MTSF Market Network Model	285
16.3	Market Merger and Composition Proof	286
16.4	Resource Competition Analysis	287
16.5	Network CNF Verification	289
17	Writing a Security Research Paper with MTSF	291
17.1	Step 1: Define the Scheme or Protocol	292
17.2	Step 2: Define the Security Goals	293
17.3	Step 3: Formal Security Definitions	293
17.4	Step 4: Define the Adversary Model	294
17.5	Step 5: Construct the MTSF Security Proof	295
17.6	Step 6: Proving Insecurity with MTSF	296
17.7	Step 7: Structuring the Paper	297
18	Presenting MTSF Proofs for Different Publication Venues	298
18.1	Case 1: IEEE Transactions (Page-Restricted, Broad Audience)	299
18.2	Case 2a: Cryptographic Journals (Page-Unrestricted, Expert Cryptographic Audience)	300
18.3	Case 2b: Non-Cryptographic Journals (Page-Unrestricted, Domain-Specific Audience)	303
18.4	Case 3: Cryptographic Conferences (Extended Proceedings, Expert Audience)	307
18.5	Venue Adaptation Summary	311
18.6	MTSF Battle Cards and Proof Scorecards	312
19	Detection of Unknown Attacks	314
19.1	MTSF-Guided Attack Discovery Methodology	315
19.1.1	Attack Discovery for Cryptographic Primitives	316
19.1.2	Attack Discovery for Block Ciphers	318
19.1.3	Attack Discovery for Hash Functions	319
19.1.4	Attack Discovery for Stream Ciphers	321

19.1.5	Attack Discovery for Protocols	323
19.2	Case Study I: TLS 1.3 Session Resumption Vulnerability	326
19.2.1	Protocol Description and Standard Analysis	326
19.2.2	Novel Attack: Ticket Nonce Collision via Implementation Deviation . . .	327
19.3	Case Study II: ISO/IEC 9798-3 Signature Verification Bypass	330
19.3.1	Protocol Description	330
19.3.2	Novel Attack: Field Boundary Confusion	331
19.4	Case Study III: WPA2 Group Key Negotiation Race Condition	334
19.4.1	Protocol Description	334
19.4.2	Novel Attack: Group Key Slot Race	335
19.5	Case Study IV: Grain-128a Related-IV Keystream Correlation in IoT Deployments	338
19.5.1	Protocol Description and Standard Analysis	338
19.5.2	Novel Attack: Related-IV Keystream Correlation via Low-Entropy IV Generation	339
19.6	Case Study V: ASCON-Hash Domain Separation Failure in Multi-Purpose IoT Deployments	344
19.6.1	Protocol Description and Standard Analysis	344
19.6.2	Novel Attack: Cross-Function State Injection via Application-Layer Do- main Confusion	345
19.7	MTSF-Based Attack Prevention Framework	349
19.7.1	Prevention for Cryptographic Primitives	354
19.7.2	Prevention for Block Ciphers	355
19.7.3	Prevention for Hash Functions	356
19.7.4	Prevention for Stream Ciphers	357
19.7.5	Prevention for Protocols	358
20	Related Work	360
20.1	Game-Based Security Proofs	360
20.2	Universal Composability and GUC	361
20.3	Formal Verification of Protocols	361
20.4	TLS 1.3 and Protocol Security Analysis	361
20.5	Needham–Schroeder Protocol	362
20.6	Economics of Security	362
20.7	Telegram MTPROTO	363
20.8	HMAC and Authenticated Encryption	363
20.9	Post-Quantum Signatures and KEMs	363
20.10	Block Ciphers, Hash Functions, and Stream Ciphers	364
20.11	Quantum Random Oracle Model	365
21	Conclusion	365
21.1	Lessons Learned	368
21.2	Further Work	371
21.2.1	Towards Quantum Market Dynamics	373
21.2.2	Towards Multi-Protocol Composition Networks	375
	Acknowledgements	379
	Dedication	379
A	MTSF Correspondence Table	385
B	Comprehensive Bid Taxonomy	388

* Simple Terms: Why Should You Read This Article?

If you are a cryptographer: MTSF unifies the four proof paradigms you already know (game-based, UC, formal verification, ad hoc insecurity) into a single language. You will find that proofs become shorter, more intuitive, and easier to compare across different schemes. The extended difference lemma and session pinging are new tools you can apply immediately.

If you are a software engineer: You will learn to read security proofs as market arguments—“this scheme is secure because no attacker can afford to break it.” The CNF verification worksheets give you a concrete, step-by-step procedure for auditing whether a protocol implementation is correct.

If you are a student or newcomer: Every definition, theorem, and proof is accompanied by a plain-language explanation, a real-world analogy, and often a diagram. You can follow the market metaphor even if you have never seen a security proof before. Start with the Reader Guide on the next page, then jump to the case study that interests you most.

If you are a policy maker or manager: MTSF provides a clear, auditable language for security guarantees. When a cryptographer says “the market is in equilibrium with ask price 2^{-128} ,” you know that the system is secure with overwhelming probability. When they say “the market has collapsed,” you know the system is broken and needs immediate action.

* Simple Terms: New to Cryptography!

Welcome! This document introduces a new way of thinking about security proofs using the language of markets and auctions.

What is a security proof? A security proof is a mathematical argument that shows a cryptographic scheme (like a digital signature or encryption) cannot be broken by any efficient computer program. Instead of testing every possible attack, we *prove* that any attacker would have to solve a problem that is believed to be computationally infeasible (like factoring enormous numbers).

What is the market analogy? MTSF rephrases security proofs as economics. The **seller** (the cryptographer defending the scheme) offers **security goods** (guarantees like “this message is confidential” or “this signature cannot be forged”). The **buyer** (the attacker) bids **computational resources** (time and memory) to try to break those guarantees. If the market is in **equilibrium**, the attacker cannot afford to win—security holds. If the market **collapses**, the attacker gets a free win—security fails.

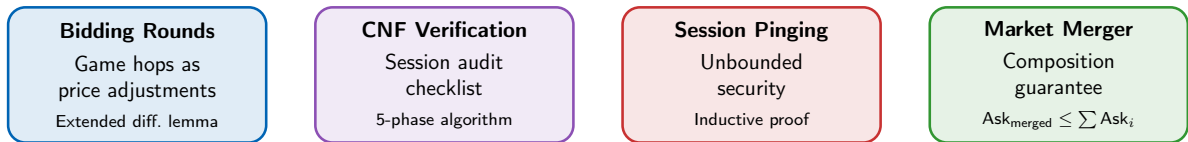
Key vocabulary you will see:

- **Adversary / Buyer $\mathcal{A}$:** The attacker trying to break the system.
- **Challenger / Seller $\mathcal{C}$:** The entity setting up the security experiment.
- **Advantage / Ask price:** How likely the attacker is to win. If this is tiny (“negligible”), the scheme is secure.
- **Negligible:** A number so small it shrinks faster than any fraction of the form $1/n^c$. In practice, think “astronomically close to zero.”
- **Game Hop / Bidding Round:** A small change to the security experiment that makes it easier to analyse, changing the winning probability by at most a small “price adjustment.”
- **Reduction:** If an attacker can break scheme X , we show it could also solve hard problem Y . Since Y is believed hard, so is X .
- **CNF formula:** A checklist of AND-ed conditions (each condition is a simple OR clause). A session passes the audit only if every item on the checklist is satisfied.
- **UC / GUC:** Universal Composability—a framework guaranteeing that a secure protocol stays secure even when run alongside other protocols simultaneously.
- **PRF / PRP:** Pseudorandom Function / Permutation—a keyed function that looks random to any attacker who does not know the key.
- **EUFCMA:** Existential Unforgeability under Chosen Message Attack—the standard security notion for digital signatures: even after seeing many signatures, the attacker cannot forge a new one.
- **IND-CCA2:** Indistinguishability under Chosen-Ciphertext Attack—the gold standard for encryption: the attacker cannot distinguish the encryption of one message from another, even with a decryption oracle.

How to read the theorems: Each theorem states a bound like “ $\text{Ask}(g) \leq \text{Adv}^{\text{ECDLP}} + \text{tiny terms}$.” This means: the only way to break the scheme is to solve the hard mathematical problem (e.g., Elliptic Curve Discrete Logarithm). The “tiny terms” come from unavoidable coincidences (birthday collisions, etc.) but are negligibly small for real parameter choices.

MTSF at a Glance

Market-Theoretic Security Framework — The Unified Paradigm



Secure (Equilibrium)



Broken (Collapsed)



MTSF Universal Dictionary

Adversary = **Buyer**
 Challenger = **Seller**
 Advantage = **Ask Price**
 Game Hop = **Bidding Round**

Security Property = **Market Good**
 UC Environment = **Regulator**
 Simulator = **Arbitrageur**
 Composition = **Market Merger**

ProVerif Check = **Stress Test**
 Attack = **Successful Bid**
 Secure = **Equilibrium**
 Broken = **Collapse**



Every case study follows this four-step pipeline.

Figure 1: **MTSF at a Glance**. The complete framework on one page: core equation (top), four pillars (row 2), case study scoreboard (row 3), universal dictionary (row 4), and proof pipeline (bottom). Pin this page to your wall.

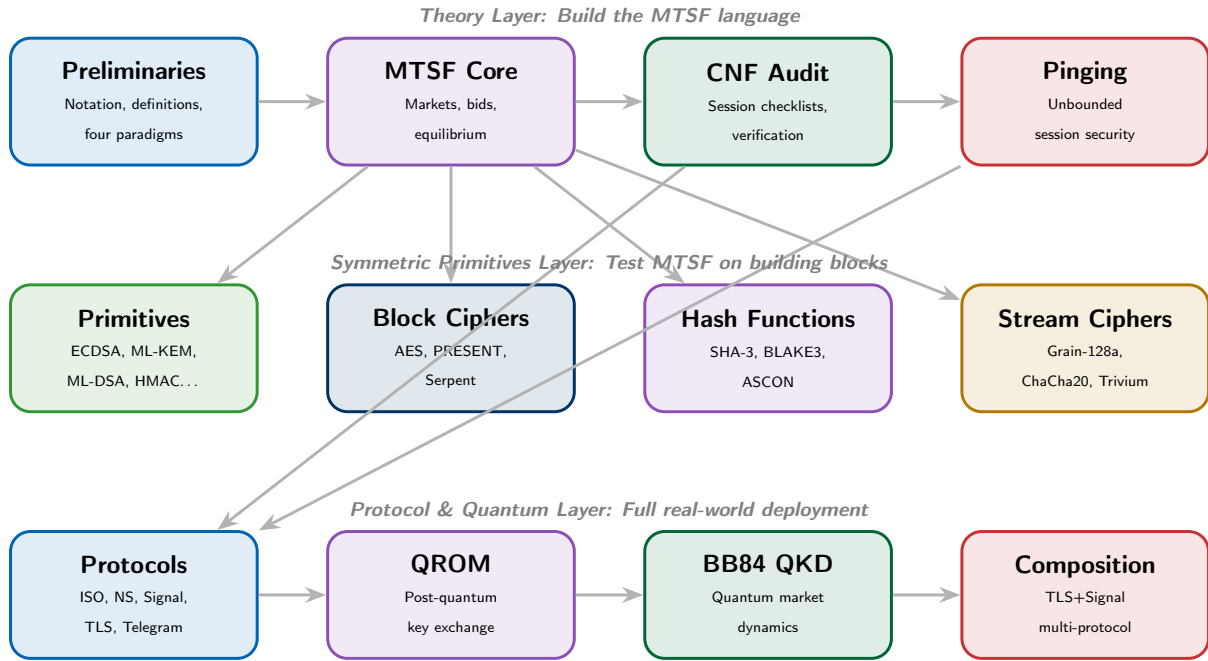


Figure 2: Visual roadmap of the MTSF document. The theory layer (top) builds the market language; the primitives layer (middle) tests it on cryptographic building blocks; the protocol and quantum layer (bottom) deploys it on real-world systems. Arrows show logical dependencies.

Analogy:

Reading This Document Is Like Building a City Think of this document as a blueprint for building a city from scratch:

- **Preliminaries** = Surveying the land (understanding the terrain of existing security frameworks).
- **MTSF Core** = Designing the city plan (the market language, definitions, and theorems).
- **CNF Verification** = Writing building codes (the checklist every structure must pass).
- **Session Pinging** = Installing earthquake sensors (ensuring safety indefinitely, not just for a limited time).
- **Case Studies** = Actually constructing buildings (applying the framework to real cryptographic systems).
- **Quantum sections** = Future-proofing against flying cars (preparing for quantum computers).

You do not need to read every section in order. If you are interested in a specific protocol (say TLS 1.3), you can jump to its case study after reading the MTSF Core section. The roadmap in Figure 2 shows which sections depend on which.

1 Introduction

Every digital system we trust—from the padlock in a browser to a hospital’s encrypted records, from a signed software update to a government identity card—rests on a hidden mathematical foundation: *cryptographic security proofs*. These proofs are the silent guardians of modern civilisation’s digital infrastructure, ensuring that encryption remains unbreakable, signatures remain unforgeable, and protocols remain resilient, no matter how powerful or resourceful the adversary. Yet despite their central role, today’s security proofs are fragmented across four disconnected languages that specialists in one paradigm often cannot easily interpret in another: game-based sequences of experiments, Universal Composability (UC) ideal-world simulations, formal symbolic verification tools such as ProVerif and Tamarin, and ad hoc insecurity demonstrations for flawed constructions. A cryptographer proving TLS secure employs different notation, definitions, and reasoning from one proving a post-quantum signature secure—even though both ultimately address the same fundamental question: *can an efficient adversary succeed?* The Market-Theoretic Security Framework (MTSF) introduced in this article answers this question through a single unifying perspective grounded in economics: every security proof is an auction, every adversary is a buyer, every security property is a tradable good, and security holds precisely when no buyer can afford to win. This article develops MTSF rigorously, proves its equivalence to all four paradigms, and demonstrates its expressive power across eighteen case studies spanning primitives, protocols, and quantum-aware security models.

★ Story Mode: A Day in the Life of Alice the Security Engineer

Monday morning. Alice is a security engineer at a major bank. Her boss drops three tasks on her desk:

Task 1: “Our mobile app uses ECDSA for transaction signing. A regulator wants mathematical proof it is secure. Can you write one?”

Alice opens her MTSF toolkit. She identifies the *market*: seller = the bank’s signing system; buyer = any attacker; good = $g_{\text{EUF-CMA}}$ (unforgeability). She writes the *quoted price* QPrice_0 and constructs five bidding rounds: nonce bid (birthday bound), hash bid (collision bound), oracle programming (free), forgery-to-ECDLP (hard problem). Total ask price: $\leq 2^{-128}$. She fills in the CNF worksheet and verifies the session ping. **Verdict: Equilibrium.** The regulator is satisfied.

Task 2: “Our partner bank uses a custom authentication protocol. Is it safe to connect our systems?”

Alice reviews the protocol and immediately spots a red flag: message 3 does not include the sender’s identity. She constructs the MTSF market and attempts a *masquerade bid*—relaying messages between two honest parties without breaking any cryptography. $\Delta\text{Price} = 1$. **Verdict: Collapse.** She fills in the CNF worksheet: the identity-binding clause is *missing*. She recommends adding the sender’s ID to message 3, re-runs the proof, and confirms equilibrium is restored.

Task 3: “We are migrating to post-quantum cryptography. Can you prove ML-KEM is safe, and that it composes securely with our existing TLS 1.3 deployment?”

Alice sets up two MTSF markets: one for ML-KEM (IND-CCA2 via MLWE), one for TLS 1.3 (authentication + key secrecy). She proves each is in equilibrium independently, then applies the *market merger theorem*: $\text{Ask}_{\text{composed}} \leq \text{Ask}_{\text{ML-KEM}} + \text{Ask}_{\text{TLS}} + \text{Ask}_{\text{PKI}}$. Since all three are negligible, the composed system is in equilibrium. She verifies the network CNF (cross-protocol SID uniqueness) and confirms unbounded session security via ping. **Verdict: Equilibrium under composition.**

Alice completes all three tasks using one framework, one notation, and one proof methodology. Before MTSF, Task 1 would require game-based proofs, Task 2 would require ad hoc attack analysis, and Task 3 would require UC composition—three different frameworks with three different notations. With MTSF, it is all one language.

1.1 The Problem: Fragmented Security Paradigms

Cryptographic security analysis is conducted through four largely disconnected paradigms, each with distinct strengths and limitations that prevent any single approach from being fully satisfactory.

Problem 1: Game-based proofs are isolated and single-failure. Game-based proofs [79, 16] bound the adversary’s advantage through a sequence of game hops $\mathbf{B}_0, \mathbf{B}_1, \dots, \mathbf{B}_q$. The classical *difference lemma* [79] bounds each hop by a *single* failure event F :

$$|\Pr[\mathbf{B}_k = 1] - \Pr[\mathbf{B}_{k+1} = 1]| \leq \Pr[F].$$

In practice, however, many proof steps involve *multiple simultaneous failures*. For example, in a digital signature proof, a single game hop may need to eliminate nonce collisions, hash collisions, and public-key recovery failures all at once. The classical lemma forces practitioners to split such hops into multiple games, inflating proofs and loosening bounds unnecessarily. Furthermore, game-based proofs are *isolated*: each theorem is a standalone argument with no built-in mechanism for combining security guarantees when protocols are composed.

Analogy:

Game-Based Proofs as Recipe Testing Imagine you are a chef (cryptographer) and you want to prove your cake recipe (cryptographic scheme) is perfect. Game-based proofs are like testing the recipe step by step: first you check the flour is good, then the sugar, then the eggs. Each test is one “game hop.” But here is the problem: what if the flour, sugar, *and* eggs are all slightly off *at the same time*? The classical approach forces you to test each ingredient separately, losing track of how the problems interact. MTSF’s extended difference lemma lets you test all ingredients simultaneously and account for their interactions. Furthermore, each recipe test is *isolated*—proving your cake is perfect tells you nothing about what happens when you serve it alongside soup, salad, and dessert at a dinner party. Game-based proofs are standalone; they do not compose. MTSF’s market model treats the entire dinner menu as one market, ensuring all dishes work together.

Key Insight:

Why MTSF Surpasses Classical Game-Based Proofs—Seven Concrete Advantages Game-based proofs (Bellare–Rogaway [16], Shoup [79]) are the most widely used proof paradigm. MTSF *strictly extends* them, offering every capability game-based proofs have, plus several they lack:

1. **Multiple simultaneous failures.** The classical difference lemma handles *one* failure event per game hop. In real proofs—ECDSA, ML-DSA, TLS 1.3—a single transition often involves 3–5 simultaneous bad events. Game-based proofs split these into separate hops, inflating proofs and loosening bounds. MTSF’s *extended difference lemma* handles all m events in one hop: $\Delta\text{Price} \leq \Pr[F_1 \cup \dots \cup F_m]$ with inclusion-exclusion tightening when events are correlated. In the ECDSA proof (Section 9.1), this consolidates what would be three separate hops into one, yielding a tighter final bound.
2. **Named, classified attacks.** In game-based proofs, game hops are labelled generically ($G_0 \rightarrow G_1 \rightarrow G_2$). The reader must reverse-engineer *what* each hop achieves. MTSF labels every hop as a *specific bid type*: “nonce freshness bid,” “hash collision bid,” “forgery-to-ECDLP bid,” “masquerade bid.” This self-documenting structure makes proofs auditable: a reviewer can check whether all relevant attack classes from the bid taxonomy (Section B) have been addressed, like checking items off a standardised list.
3. **Built-in composition.** Game-based proofs are standalone: proving Scheme A secure and Scheme B secure says nothing about running both together. Composing game-based results requires ad hoc reasoning or switching to UC entirely. MTSF’s *market merger theorem* gives composition natively: two equilibrium markets merge into a combined equilibrium market with explicit, additive ask-price bounds. The TLS+Signal case study (Section 16) demonstrates this directly.

4. **Insecurity as a first-class result.** Game-based proofs only establish security (upper bounds on advantage). Insecurity must be demonstrated separately using bespoke arguments. MTSF uses *the same bidding-round machinery* for both: $\text{Ask} \leq \text{negl}$ means equilibrium (secure); $\text{Ask} = 1$ means collapse (insecure). The Needham–Schroeder analysis (Section 13.4) proves $\text{Ask} = 1$ via a masquerade bid using the same notation and structure as the ECDSA equilibrium proof.
5. **Unbounded session coverage.** Game-based proofs are inherently bounded: they hold for at most $q = \text{poly}(\lambda)$ sessions. Real systems run for millions of sessions over years. MTSF’s *session pinging* mechanism (Section 5) provides an inductive bridge from bounded to unbounded security, with quantitative advantage accumulation bounds. No game-based framework offers this natively.
6. **Session-level auditing.** Game-based proofs produce a single aggregate bound ($\text{Adv} \leq \epsilon$) but do not describe what a *correct session* looks like. MTSF’s CNF session verification (Section 4) defines session correctness as a checkable Boolean formula, with a concrete manual worksheet that practitioners can fill in per-session. This enables implementation-level auditing that game-based proofs cannot support.
7. **Quantitative tightness tracking.** In game-based proofs, tightness loss is implicit and often buried in the reduction. MTSF makes tightness explicit via the “market spread”: the ratio between the ask price (best attacker strategy) and the hard-problem advantage. Each bidding round explicitly states its tightness factor $L(\lambda)$ as part of the price adjustment, making the total loss visible and auditable.

Crucially, MTSF does not discard game-based techniques—it *embeds* them. Every existing game-based proof can be rewritten as an MTSF bidding-round chain by relabelling games as bidding rounds, advantages as ask prices, and failures as bid events. The translation is mechanical, and the MTSF version is strictly more informative because it adds classification, composition, unbounded coverage, and session auditing.

Problem 2: UC/GUC provide composition but lack concrete bounds. The UC framework [31] guarantees that a secure protocol remains secure when composed with arbitrary other protocols running concurrently—the gold standard for real-world deployability. However, UC proofs do not produce numerical advantage bounds. A UC proof says “the protocol is secure” but not “the adversary’s advantage is at most ϵ ”. The GUC framework [32] with CNF session checking [30] adds session correctness verification but still lacks the quantitative output needed for concrete parameter selection.

Key Insight:

Why MTSF Surpasses UC and GUC—Five Concrete Advantages The UC framework [31] and its generalisation GUC [32] provide the gold standard for composition. MTSF preserves all of UC/GUC’s composition guarantees while adding capabilities they lack:

1. **Concrete numerical bounds.** A UC proof says “protocol π UC-realises functionality $\mathcal{F}$.” This is a binary verdict: secure or not. It does not tell you *how* secure—whether the adversary’s advantage is 2^{-128} or 2^{-40} . MTSF produces explicit ask prices: $\text{Ask}(g_j, \lambda) \leq 2\text{Adv}^{\text{MLWE}} + q_D/2^\gamma$. These numerical bounds are essential for parameter selection (e.g., “choose $\lambda = 256$ to achieve 128-bit security”) and for comparing the relative security of different schemes.

Capability	Game-Based	UC/GUC	Formal Verif.	MTSF
Quantitative bounds	✓ Yes	× No	× No	✓ Yes
Multi-failure hops	× No	× N/A	× N/A	✓ Yes
Named attack classes	× No	× No	~ Traces	✓ Yes
Composition guarantee	× No	✓ Yes	× No	✓ Yes
Insecurity proofs	× No	× No	~ Attacks	✓ Yes
Unbounded sessions	× No	~ Partial	✓ Yes	✓ Yes
Session-level audit	× No	~ CNF only	× No	✓ Yes
Human-readable	~ Partial	× No	× No	✓ Yes

MTSF is the **only** framework achieving all eight capabilities simultaneously. It subsumes game-based, UC/GUC, and formal verification as special cases.

Figure 3: Feature comparison: game-based proofs, UC/GUC, formal verification, and MTSF. Green = full support; yellow = partial support; red = not supported. MTSF is the only framework achieving all eight capabilities.

2. **Quantitative composition.** UC’s composition theorem says “secure + secure = secure,” but not *how much* security degrades. MTSF’s market merger theorem gives an explicit additive bound: $\text{Ask}_{\text{composed}} \leq \text{Ask}_{\text{TLS}} + \text{Ask}_{\text{Signal}} + \text{Ask}_{\text{infrastructure}}$. This enables concrete resource planning: you know exactly how much security margin remains after composition.
3. **Structured proof organisation.** UC proofs involve constructing a simulator $\mathcal{S}$ that “explains” the real world using the ideal world. These simulators can be extremely complex—the TLS 1.3 UC proof by Dowling et al. required dozens of pages of simulator construction. MTSF replaces the monolithic simulator with a structured *bidding-round chain*, where each round targets one attack vector. This modular structure is easier to write, review, and maintain. Adding a new attack vector means adding one bidding round, not rewriting the entire simulator.
4. **Insecurity detection within the same formalism.** UC proofs can only establish security. If a protocol is insecure, UC provides no mechanism to formally demonstrate the flaw—you must construct an ad hoc attack separately. MTSF’s dual methodology handles both: the same bidding-round chain that proves TLS 1.3’s 1-RTT handshake is secure ($\text{Ask} \leq \text{negl}$) also proves its 0-RTT mode is insecure ($\text{Ask} = 1$ via replay bid). The Needham–Schroeder masquerade bid and the Telegram salt extraction bid are formal insecurity theorems *within* the MTSF framework, not separate arguments.
5. **Practitioner-accessible session auditing.** GUC’s CNF session correctness [30] requires deep expertise to construct and verify. MTSF operationalises CNF checking via a canonical five-phase algorithm (Algorithm 1) and a simple four-column manual worksheet that a security engineer without formal methods training can use to audit protocol

sessions. Every case study in this article includes a filled-in worksheet as a concrete example.

MTSF does not replace UC/GUC—it *contains* them as a special case. The UC real/ideal indistinguishability is recast as market regulation ($\mathcal{Z}$ is the regulator, $\mathcal{S}$ is the arbitrageur), and the composition theorem becomes the market merger theorem. A UC/GUC expert can use MTSF with minimal retraining: the concepts translate directly, and the MTSF version provides strictly more information.

Problem 3: Formal verification tools lack quantitative output and human interpretability. ProVerif [26] and Tamarin [66] check whether a protocol is secure symbolically and for unbounded sessions, but produce a binary pass/fail output with no numerical bounds. CryptoVerif [27] closes this gap partially with machine-checked game-based proofs, but requires significant manual guidance and does not natively support the UC/GUC framework. More importantly, none of these tools produce human-readable arguments that practitioners can understand, audit, and reason about without specialist training.

 Analogy:

Formal Verification as a Robot Inspector Think of ProVerif and Tamarin as robot inspectors at an airport. They walk through *every possible route* a passenger (attacker) could take through the terminal and check whether any route leads to the cockpit (a security breach). If no route works, the airport is “secure.” But the robots cannot tell you *how close* a passenger came to breaching security—they only say pass or fail. They also cannot explain their reasoning in plain English. MTSF turns these robot inspections into market stress tests with explicit numerical bounds: “The closest any attacker came to the cockpit was 0.000001% of the way there.”

Problem 4: No unified insecurity analysis. When a primitive is insecure (e.g., textbook RSA has a free homomorphism attack) or a protocol is insecure (e.g., Needham–Schroeder [73] is vulnerable to Lowe’s MITM [63]), this is demonstrated *ad hoc* using bespoke arguments. There is no common framework that handles both security proofs *and* insecurity demonstrations with the same language and tools. This asymmetry means that protocol designers cannot use the same methodology to “test” their protocol against known attack classes before submitting a full proof.

 Analogy:

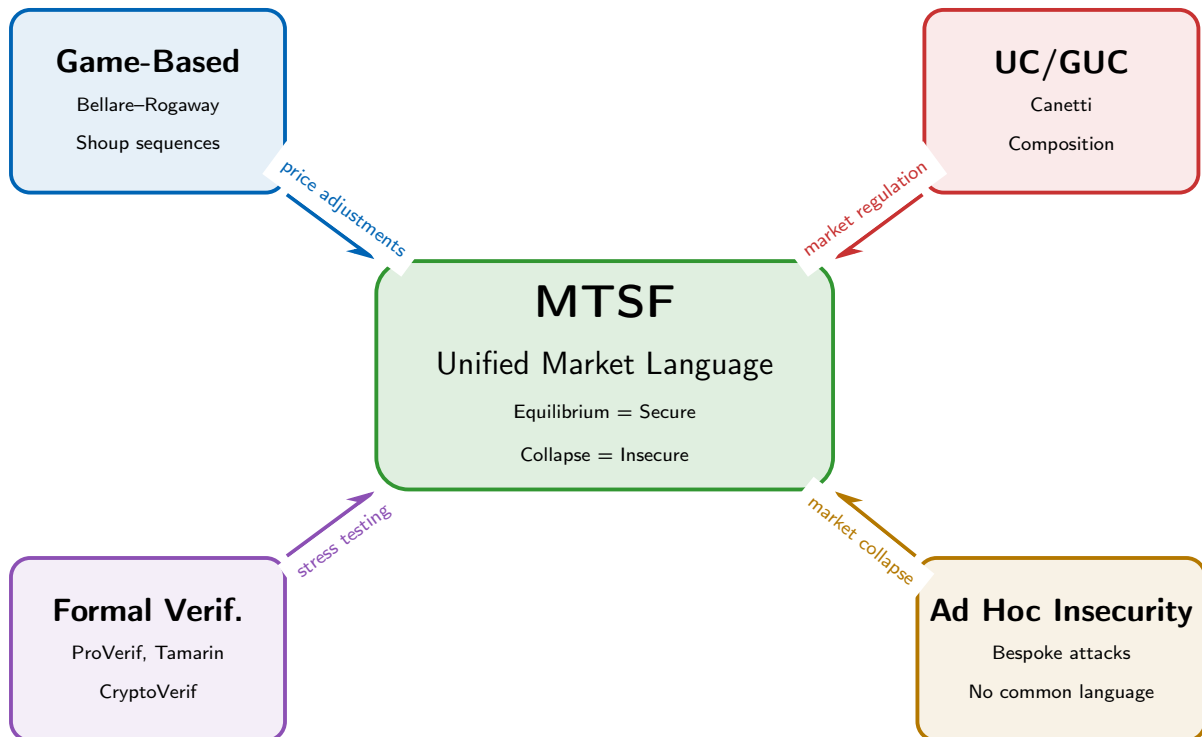
Insecurity Proofs as Product Recalls When a car manufacturer discovers a defective airbag, they issue a *recall*—a formal announcement that the product is broken, with a specific explanation of the defect. In cryptography, insecurity proofs are like product recalls: they identify exactly what is broken and why. But currently, each recall uses a different format—there is no standardised “recall notice” template. MTSF provides this template: insecurity is a *market collapse* where the ask price reaches 1, meaning any attacker wins for free. The “recall notice” specifies which bid succeeded (e.g., “homomorphism bid” for textbook RSA, “masquerade bid” for Needham–Schroeder) and quantifies the damage.

Problem 5: Protocol-level security goods lack a unified market formulation. Existing game-based frameworks treat entity authentication, mutual authentication, session-key secrecy, and session correctness as separate definitions with separate games and separate proofs. There is no unified treatment embedding all four as simultaneous *market goods* offered by a single seller,

with the adversary bidding against all of them in parallel. This separation makes it hard to see which property fails first under a given attack and how the failures interact.

Problem 6: No framework addresses the QROM, unbounded sessions, and CNF correctness simultaneously. Post-quantum security requires proofs in the Quantum Random Oracle Model (QROM), where adversaries can query hash functions in superposition. Classical ROM proofs fail in this setting. Simultaneously, real deployments run for unbounded numbers of sessions (not the polynomial bound assumed by game-based proofs). And CNF session correctness must be verified across all sessions. No existing framework addresses all three requirements—QROM, unbounded, and CNF—within a single proof methodology.

Before MTSF: Four disconnected paradigms, each with its own language



After MTSF: One unified market language that speaks all four dialects

Figure 4: The unification problem: four disconnected security paradigms converge into MTSF’s single market language. Game-based proofs become price adjustments, UC becomes market regulation, formal verification becomes stress testing, and ad hoc insecurity becomes market collapse.

1.2 What MTSF Provides

Key Insight:

What Is New in MTSF—Seventeen Contributions

1. **Unified auction model:** Security = market equilibrium ($\text{Ask} \leq \text{negl}$); insecurity = market collapse ($\text{Ask} = 1$). The same language handles both.
2. **Extended difference lemma:** Captures $m \geq 1$ simultaneous failures in one game hop via $\Pr[F_1 \cup \dots \cup F_m]$ with inclusion-exclusion tightening (Theorem 3.7). Generalises Shoup’s single-failure version.

3. **Bidding-based proofs:** Each hop targets a specific adversarial strategy (nonce bid, hash-collision bid, public-key bid, forgery bid, MITM bid). The proof explicitly tracks *what* the adversary is attacking and *how much* it costs.
4. **Insecurity as market collapse:** Textbook RSA ($\text{Ask} = 1$ via the homomorphism bid) and Needham–Schroeder ($\text{Ask} = 1$ via the masquerade bid) are proved insecure within the same framework as secure schemes.
5. **Four-paradigm unification:** Game-based proofs (price adjustments), UC (market regulation by $\mathcal{Z}$), GUC (shared market infrastructure + CNF audit), and formal verification (market stress testing) are unified under one roof.
6. **CNF session verification + session ping:** A canonical five-phase verification algorithm (Algorithm 1) with an easy four-column manual worksheet, plus a session-pinging mechanism for unbounded security (Section 5).
7. **Dedicated protocol-market games:** Authentication, mutual authentication, session-key secrecy, and CNF correctness games as first-class *security goods* sold in the protocol market (Section 8).
8. **Comprehensive primitive coverage:** ECDSA, ML-KEM, ML-DSA, HMAC, AEAD, SLH-DSA, FN-DSA—all analysed via bidding-round chains with CNF worksheets and ping bids.
9. **Block-cipher market:** AES, PRESENT (lightweight), and Serpent (conservative AES finalist) are analysed via differential, linear, rotational, boomerang, and related-key crypt-analysis bids in unified block-cipher markets (Section 10).
10. **Hash-function market:** Keccak/SHA-3, BLAKE3 (Merkle tree, structural length-extension immunity), and ASCON-Hash (NIST lightweight standard) are analysed for collision resistance, preimage resistance, and length-extension resistance (Section 11).
11. **Stream-cipher market:** Grain-128a, ChaCha20 (ARX, 256-bit key, nonce-misuse collapse), and Trivium (80-bit key, cube attack analysis) are analysed for state recovery, key recovery, distinguishing, and TMTO attacks (Section 12).
12. **QROM case study:** The FO-transform KEM is proven IND-CCA2 in the QROM using the O2H lemma and measure-and-reprogram, extending MTSF to quantum-adversarial settings (Section 14).
13. **Dual Telegram analysis:** MTPROTO 2.0 is both *disproved* (partial collapse under salt extraction) and a remediated protocol RMTP is *proved* secure—all within the same MTSF framework (Section 13.9).
14. **BB84 quantum market dynamics:** First full quantum market analysis where both seller and buyer are quantum. BB84 QKD analysed via quantum bidding rounds with information-theoretic security (Section 15).
15. **TLS+Signal multi-protocol composition:** First MTSF analysis of concurrent protocol execution via market merger theorem, with resource competition analysis and network CNF (Section 16).
16. **CNF verification bidding rounds:** Every protocol and primitive case study includes a CNF verification bidding round with a manual worksheet, making security auditing accessible to non-expert practitioners.

17. Ping bids in every case study: Every case study includes a ping bid formally proving unbounded session security.

1.3 Key Technical Innovations

Extended difference lemma. The classical difference lemma bounds a single failure event. Our extension (Theorem 3.7) handles m simultaneous events:

$$|\Pr[\mathbf{B}_k = 1] - \Pr[\mathbf{B}_{k+1} = 1]| \leq \Pr\left[\bigcup_{i=1}^m F_i\right] \leq \sum_{i=1}^m \Pr[F_i] - \sum_{i < j} \Pr[F_i \cap F_j] + \dots$$

with inclusion-exclusion giving a tight upper bound when failure events are correlated. We apply this in every case study: ECDSA ($F_{\text{nonce}} \cup F_{\text{hash}} \cup F_{\text{fork}} \cup F_{\text{extract}}$), ML-DSA ($F_{\text{rejection}} \cup F_{\text{norm}}$), four-party protocols ($\bigcup_{i=1}^4 F_{\text{sig},i}$), and Telegram ($F_{\text{salt}} \cup F_{\text{entropy}} \cup F_{\text{cnf}} \cup F_{\text{ping}}$).

Session pinging for unbounded security. Bounded game-based proofs say “secure for q sessions” but real systems run for millions of sessions over years. We define structural distinctness (Theorem 5.1) and the session ping function (Theorem 5.3), and prove by induction (Theorem 5.5) that if the base case is secure and all pings pass, then every session is secure—providing unbounded coverage with quantitative bounds. This bridges game-based proofs (bounded, quantitative) and formal verification (unbounded, qualitative).

Dual proof/disproof methodology. MTSF characterises insecurity as market collapse: $\text{Ask} = 1$ means any adversary wins for free. Secure schemes have $\text{Ask} \leq \text{negl}$. The same bidding-round machinery proves both. For the Needham–Schroeder protocol, we show the masquerade bid succeeds with probability 1 (no cryptographic break needed). For textbook RSA, the homomorphism bid succeeds in $O(1)$ time. For MTPROTO, the salt extraction bid quasi-collapses the market to $\text{Ask} \approx 1$. These are not ad hoc observations—they are formal theorems within MTSF.

1.4 Article Organisation

Section 2: notation, semantic security, four paradigms with figures. Section 3: MTSF with extended difference lemma. Section 4: CNF session verification algorithm and easy manual worksheet. Section 5: session pinging for unbounded security. Section 6: thirteen unified novelties. Section 8: authentication, mutual authentication, session-key secrecy, and CNF checking games as market goods. Section 9: ECDSA, ML-KEM, ML-DSA (secure) and textbook RSA (insecure); extended primitives HMAC, AEAD, SLH-DSA, FN-DSA. Section 10: AES block-cipher market with differential, rotational, linear, and related-key bids. Section 11: Keccak/SHA-3 hash-function market. Section 12: Grain-128a stream-cipher market. Section 13: ISO/IEC 11770-3 protocol family (two-, three-, and four-party), PKI, Needham–Schroeder insecurity proof, Signal Protocol (X3DH + Double Ratchet) security proof and X3DH-noOPK insecurity proof, TLS 1.3 (1-RTT equilibrium proof, 0-RTT replay collapse, downgrade attack collapse, CNF session verification and unbounded ping), and Telegram MTPROTO disproof (salt extraction) and RMTP proof (HMAC-bound salts). Section 14: QROM-based FO-transform KEM with full protocol description, sequence diagram, and six-bidding-round proof. Section 15: BB84 Quantum Key Distribution as a full quantum market dynamics case study (quantum seller, quantum buyer, information-theoretic security via no-cloning and QLHL). Section 16: TLS 1.3 + Signal multi-protocol composition network case study (market merger theorem instantiation, resource competition analysis, network CNF verification). Section 20: comprehensive related work in ten subsections. Section 21: conclusion with seventeen contributions, lessons learned, and seven future directions (with initial formalisations of quantum market dynamics and multi-protocol

composition networks). Section A: expanded MTSF correspondence table (79 entries). Section B: comprehensive bid taxonomy with QROM, Telegram, BB84, and composition bids.

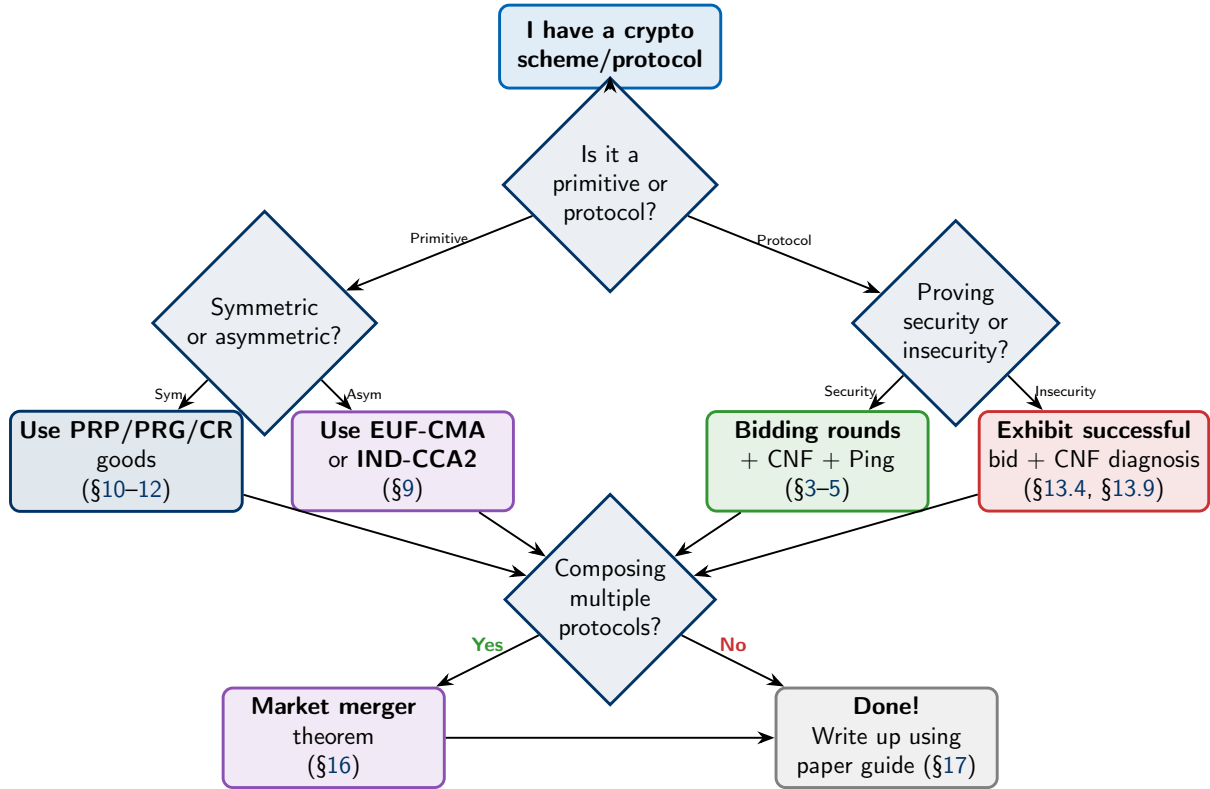


Figure 5: Decision flowchart: “Which MTSF tool do I need?” Start with your cryptographic object, follow the arrows, and arrive at the correct MTSF section and proof technique.

2 Preliminaries and Background

2.1 Notation

Notation Cheat Sheet

λ : security parameter (e.g., 128). $\{0, 1\}^n$: n -bit strings. $x \xleftarrow{\$} S$: sample uniformly. $\text{negl}(\lambda)$: negligible function (shrinks faster than any inverse polynomial). PPT: probabilistic polynomial time. $\mathbf{B}_k$: k -th *bidding round* (MTSF market-theoretic notation; **bold upright B**). sid/pid: session/party identifier. σ/τ : signature/MAC tag. $\mathcal{A}$: adversary. $\mathcal{C}$: challenger.

* Simple Terms: What the Notation Means in Practice

Security parameter λ : Think of this as the “strength dial” of the cryptographic scheme. Setting $\lambda = 128$ means roughly 2^{128} operations are needed to break the scheme—more than the number of atoms in the observable universe. All other quantities (key sizes, running times) are expressed as functions of λ .

Negligible function $\text{negl}(\lambda)$: A quantity that shrinks to zero faster than any polynomial fraction, e.g., $2^{-\lambda}$ or λ^{-100} . In plain English: so small that even repeating the experiment a trillion times, the event almost certainly never happens. When we say an adversary’s advantage is negligible, we mean the scheme is secure in all practical terms.

PPT (Probabilistic Polynomial Time): An attacker that runs in time bounded by some polynomial in λ , e.g., λ^3 steps. This is the standard model of a “realistic” attacker

with access to today’s (and tomorrow’s) computers. We only care about defeating PPT attackers—exponential-time attackers are irrelevant for practical security.

Session ID (sid) and Party ID (pid): A session ID uniquely labels one run of a protocol (like a phone call’s unique call-log entry). A party ID identifies who is participating (like a phone number). Binding cryptographic material to these identifiers prevents replay attacks (using recordings of old calls).

Real-world scale of 2^{128} : To appreciate what $\lambda = 128$ means in practice, consider these comparisons:

- There are approximately 2^{80} atoms in the observable universe.
- The universe is approximately 2^{58} seconds old.
- The fastest supercomputer performs approximately 2^{60} operations per second.
- Even running every atom in the universe as a computer for the age of the universe gives roughly $2^{80+58+60} = 2^{198}$ operations total—still far short of 2^{256} .

When we say a scheme with $\lambda = 128$ has negligible advantage, we mean an attacker would need to perform operations numbering more than the atoms in the universe to have even a tiny chance of success.

2.2 Semantic Security and Core Definitions

Analogy:

Cryptographic Security as a Guessing Game for Children Imagine Alice whispers one of two words—either “cat” or “dog”—into a magic box. The box scrambles the word into gibberish and shows the gibberish to Eve. Eve must guess which word Alice whispered. If the box is a *good* encryption scheme, Eve can do no better than flipping a coin—50/50. Even if Eve is the smartest person in the world with the fastest computer, the box gives her *zero* useful information.

That is semantic security: the ciphertext reveals nothing about the plaintext beyond what the attacker already knew. In MTSF terms: the buyer (Eve) bids her computational resources to distinguish “cat” from “dog,” but the ask price of the confidentiality good is negligible—Eve’s bid always fails.

Now imagine Eve can also ask the box to encrypt other words of her choosing (“fish,” “bird,” “tree”) before making her guess. If the box *still* gives her no advantage, that is **IND-CPA** security (chosen-plaintext). If Eve can additionally ask the box to *decrypt* other gibberish of her choosing, and *still* cannot tell “cat” from “dog,” that is **IND-CCA2** security—the gold standard.

Definition 2.1 (Semantic Security [48]). *Encryption Π is semantically secure if for every PPT $\mathcal{A}$, whatever $\mathcal{A}$ computes from ciphertext $c = \text{Enc}(K, m)$, a PPT simulator $\mathcal{S}$ can compute from the message length $|m|$ alone: $|\Pr[f \leftarrow \mathcal{A}(c)] - \Pr[f \leftarrow \mathcal{S}(|m|)]| \leq \text{negl}(\lambda)$. In market terms: the ciphertext is a worthless good—the buyer (adversary) gains zero information from purchasing it.*

* Simple Terms: Semantic Security

What it says: Looking at an encrypted message (a ciphertext) tells an attacker absolutely nothing useful about the original message beyond its length. Even the world’s best computer program, given the ciphertext, cannot distinguish whether “Hello” or “Goodbye” was encrypted—it may as well guess randomly.

Why length leaks: You can count the bytes in a ciphertext, which reveals roughly how long the original message was. This is unavoidable in most schemes (you can pad messages to fixed lengths to hide even this). Semantic security accepts this small leak and guarantees nothing else leaks.

Market translation: A ciphertext is a product on the shelf. An adversary is a buyer inspecting it. Semantic security says: no matter how closely the buyer examines the product (ciphertext), they cannot determine its contents (the plaintext). The product reveals only its box size (message length).

Definition 2.2 (IND-CPA). $\mathcal{A}$ submits m_0, m_1 ; receives $\text{Enc}(K, m_b)$ for random b ; guesses b . Secure if $\text{Adv}^{\text{IND-CPA}} \leq \text{negl}(\lambda)$.

* Simple Terms: IND-CPA

The game: The adversary chooses two messages of equal length (say “Attack at dawn” and “Retreat to camp”). A challenger flips a coin and encrypts one of them. The adversary sees the ciphertext and tries to guess which message was encrypted. *Indistinguishability under Chosen Plaintext Attack* (IND-CPA) says the adversary cannot do better than random guessing (50/50).

Why “chosen plaintext”? The adversary can also ask for encryptions of *any* other messages they like before making their final guess. This models an attacker who can influence what gets encrypted (e.g., by sending decoy emails through a system they know is being monitored).

What it does NOT cover: The adversary cannot ask for decryptions. Schemes like stream ciphers can be IND-CPA secure but still leaky if an attacker can submit ciphertexts for decryption (see IND-CCA2 below).

Definition 2.3 (IND-CCA2). As IND-CPA but $\mathcal{A}$ additionally gets a decryption oracle (except on the challenge). Secure if $\text{Adv}^{\text{IND-CCA2}} \leq \text{negl}(\lambda)$.

* Simple Terms: IND-CCA2

The game: Same as IND-CPA, but now the adversary can also submit *any ciphertext* (except the challenge ciphertext itself) to a decryption service and see the result. This models a very powerful attacker: imagine someone who can sneak messages through a decryption box (e.g., by tricking a smart card into decrypting).

Why is this the gold standard? If a scheme survives even this attack, it is safe in virtually all real-world deployment scenarios. Modern protocols (TLS, Signal, etc.) target IND-CCA2 security.

The “except the challenge” rule: The attacker cannot simply submit the challenge ciphertext for decryption—that would trivially win. Everything *else* can be decrypted.

Definition 2.4 (EUF-CMA). $\mathcal{A}$ gets a signing oracle; must produce a valid signature on a new message. Secure if $\Pr[\text{forge}] \leq \text{negl}(\lambda)$.

* Simple Terms: EUF-CMA

The game: The adversary can ask a signer to sign *any messages* they choose (like a counterfeiter who legally obtains genuine banknotes to study). Then the adversary must produce a valid signature on a *new message they never asked to be signed*. EUF-CMA (Existential Unforgeability under Chosen Message Attack) says this is computationally impossible.

Why “existential”? The adversary can choose whatever message they want to forge—they are not given a specific target. Even this ultimate freedom should not help them.

Real-world relevance: This is why you cannot forge a digital certificate for a bank’s website: even if you’ve collected millions of legitimate certificates, you cannot produce a valid new one without the Certificate Authority’s private key.

Definition 2.5 (SUF-CMA). *Strong unforgeability: $\mathcal{A}$ cannot produce any new valid (message, signature) pair, even for previously signed messages with a different signature.*

*** Simple Terms: SUF-CMA vs. EUF-CMA**

The extra strength: EUF-CMA allows the attacker to ask for a signature on message m and still requires them to forge a signature on a *different* message. SUF-CMA is stricter: even if the attacker already has a valid signature σ on m , they cannot produce a *different* valid signature $\sigma' \neq \sigma$ on the same m .

Why does this matter? Some protocols use signatures as receipts. If an attacker could create multiple valid signatures for the same message, they could confuse audit logs or create deniability. SUF-CMA closes this loophole.

Practical example: In HMAC-based authentication, strong unforgeability ensures that a MAC tag is unique: you cannot replace one valid tag with a different valid tag to alter a transaction record.

Definition 2.6 (INT-CTXT (Ciphertext Integrity)). *For an authenticated encryption scheme AE, the adversary $\mathcal{A}$ has access to an encryption oracle and wins if it produces any ciphertext c^* that decrypts successfully but was never output by the encryption oracle. Secure if $\Pr[\text{INT-CTXT forge}] \leq \text{negl}(\lambda)$.*

*** Simple Terms: INT-CTXT**

What it means: Ciphertext integrity ensures that nobody can create a *new* ciphertext that looks legitimate (decrypts without error) unless they used the legitimate encryption process. Even if an attacker sees many valid ciphertexts, they cannot craft a new one that passes the decryption check.

Real-world importance: Without INT-CTXT, an attacker could modify an encrypted bank transfer, and the receiver would happily decrypt the tampered ciphertext. With INT-CTXT, any modification causes decryption to fail (the “tag” verification rejects it).

AEAD combines IND-CCA2 + INT-CTXT: Authenticated Encryption with Associated Data (like AES-GCM, ChaCha20-Poly1305) provides both confidentiality (IND-CCA2) and integrity (INT-CTXT) simultaneously. This is the modern standard.

Definition 2.7 (PRF Security). *A keyed function $F_K : \{0,1\}^n \rightarrow \{0,1\}^m$ is a secure PRF if no PPT distinguisher tells $F_K(\cdot)$ from a truly random function $R(\cdot)$:*

$$\text{Adv}_F^{\text{PRF}}(\mathcal{A}) = |\Pr[\mathcal{A}^{F_K(\cdot)}=1] - \Pr[\mathcal{A}^{R(\cdot)}=1]| \leq \text{negl}(\lambda).$$

*** Simple Terms: PRF**

What is a PRF? A Pseudorandom Function (PRF) is a keyed function that looks like a random function to anyone who does not know the key. You input a value, you get an output that looks completely random—but the same input always gives the same output for the same key.

The distinction game: An adversary can query the function on any inputs they choose. They must decide: “Am I talking to F_K (with some secret key), or to a truly random function?” PRF security says they cannot tell the difference.

Where PRFs appear: HMAC uses two PRF evaluations (inner and outer hash steps). Block ciphers like AES are modelled as PRFs/PRPs. Key derivation functions (KDFs) are

PRFs. Virtually every symmetric-key construction relies on PRF security at its core.

2.3 Bidding-Round Proofs in the Random Oracle Model

Analogy:

The Random Oracle as a Magic 8-Ball In cryptographic proofs, we often model hash functions as “random oracles”—imagine a magic 8-ball that:

- Gives a completely random answer for each new question.
- Always gives the *same* answer if you ask the same question again (deterministic).
- Has no pattern—knowing the answer to one question tells you nothing about any other answer.

Why use this model? Real hash functions like SHA-256 are deterministic algorithms, not truly random. But they are designed to *behave* as if random. The Random Oracle Model (ROM) lets us prove security assuming ideal randomness, then instantiate the oracle with a real hash function.

The “programming” trick: In a security proof, the seller (challenger) secretly controls the random oracle. When the buyer asks “what is $H(m)$?”, the seller can choose the answer strategically—for example, embedding a hard-problem challenge. Since the oracle is “random,” the buyer cannot tell the difference between a genuinely random answer and a strategically chosen one. This is the “oracle programming” bidding round—a *costless market restructuring* that costs $\Delta\text{Price} = 0$.

In MTSF: Random oracle queries are “hash bids”—the buyer pays computation to query the oracle, hoping to find useful information. Hash collisions (two queries with the same answer) are “birthday bids,” costing $q^2/2^{n+1}$ by the birthday bound.

A *bidding round* $\mathbf{B}_k$ is a formal experiment between a challenger $\mathcal{C}$ and adversary $\mathcal{A}$. The challenger sets up keys, answers oracle queries, and presents challenges; the adversary tries to win (guess correctly, produce a forgery, etc.). The *bidding-round sequence* technique [79, 16] transforms $\mathbf{B}_0$ (the real security experiment) into $\mathbf{B}_q$ (an ideal experiment where winning is impossible):

$$\text{Adv}_{\mathcal{A}}^{\text{sec}}(\lambda) \leq \sum_{k=0}^{q-1} \underbrace{|\Pr[\mathbf{B}_k(\mathcal{A}) = 1] - \Pr[\mathbf{B}_{k+1}(\mathcal{A}) = 1]|}_{\text{price adjustment } \Delta\text{Price}_k}. \quad (1)$$

* Simple Terms: Game-Hopping

The core idea: Instead of directly proving that the adversary cannot win the real game $\mathbf{B}_0$, we make a series of small, clearly justified changes to the game rules. Each change is so small that the adversary’s winning probability barely shifts (by at most ΔPrice_k). After all changes, we reach a “dream world” game $\mathbf{B}_q$ where winning is trivially impossible.

Analogy: Imagine you want to prove a complex recipe is safe to eat. Instead of analysing every ingredient simultaneously, you swap them one at a time (real ingredient $\rightarrow$ inert substitute), showing each swap changes nothing meaningful. After all swaps, the dish is obviously inert.

The bound: The total advantage is at most the *sum* of all small changes. If each ΔPrice_k is tiny (negligible), their sum is also tiny—and the scheme is secure.

Each ΔPrice_k in MTSF is a “price adjustment”: in the market analogy, each game hop slightly adjusts the “price” (winning probability) in the security market. The total cost to

the adversary sums these adjustments.

Lemma 2.8 (Classical Difference Lemma [79]). *If $\mathbf{B}_k$ and $\mathbf{B}_{k+1}$ are identical except when a single failure event F occurs, then: $|\Pr[\mathbf{B}_k(\mathcal{A}) = 1] - \Pr[\mathbf{B}_{k+1}(\mathcal{A}) = 1]| \leq \Pr[F \text{ in } \mathbf{B}_k]$.*

* Simple Terms: Classical Difference Lemma

What it says: If two versions of a game ($\mathbf{B}_k$ and $\mathbf{B}_{k+1}$) play out identically unless a specific “bad event” F happens, then the difference in winning probabilities is at most the probability that F happens at all.

Example: Suppose $\mathbf{B}_k$ uses a real key, and $\mathbf{B}_{k+1}$ uses a random key—the games only differ if the adversary notices the key has changed. The bad event F is “the adversary detects the key swap.” If F is very unlikely (tiny probability), the adversary wins at almost the same rate in both games.

Why it matters: This lemma is the workhorse of game-hopping proofs. Each game hop corresponds to one application of this lemma, bounding how much each “swap” can help the adversary.

In the *Random Oracle Model* (ROM) [15], hash functions are modelled as perfectly random functions accessible only via queries. This idealisation enables clean reductions.

* Simple Terms: The Random Oracle Model

What is a random oracle? A random oracle is a mathematical idealisation of a hash function (like SHA-256). It is modelled as a magical black box: you give it any input, and it outputs a completely random, fixed string (fixed means: same input always gives same output). No algorithm can predict the output for a new input without querying the box.

Why use this model? Real hash functions are complex but deterministic algorithms. Proving things about them directly is very hard. In the ROM, we pretend hash functions are perfectly random, which lets us write clean, short proofs. The trade-off: ROM proofs do not guarantee security if the hash function has algebraic structure an attacker could exploit.

Who uses ROM proofs? Most practical digital signature schemes (ECDSA, RSA-PSS), key exchange protocols (ECDH-based), and password hashing schemes are analysed in the ROM. It has been standard practice since 1993 [15].

2.4 Universal Composability (UC) Framework

The UC framework [31] guarantees that a protocol secure in isolation remains secure under *arbitrary* concurrent composition. A protocol π *UC-realises* an ideal functionality $\mathcal{F}$ if for every PPT adversary $\mathcal{A}$ there exists a PPT simulator $\mathcal{S}$ such that no PPT environment $\mathcal{Z}$ can distinguish:

$$\text{Real}_{\pi, \mathcal{A}, \mathcal{Z}} \approx_c \text{Ideal}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}. \quad (2)$$

* Simple Terms: The UC Framework

The problem UC solves: A protocol might be secure on its own, but become insecure when run simultaneously with other protocols. For example, you might design a secure key exchange, but an attacker running a different protocol alongside it could learn the key. UC guarantees that this cannot happen.

The two worlds: UC compares two worlds:

- **Real world:** The actual protocol π runs, with a real adversary $\mathcal{A}$ attacking it.
- **Ideal world:** A perfectly secure “trusted third party” (the ideal functionality $\mathcal{F}$) handles

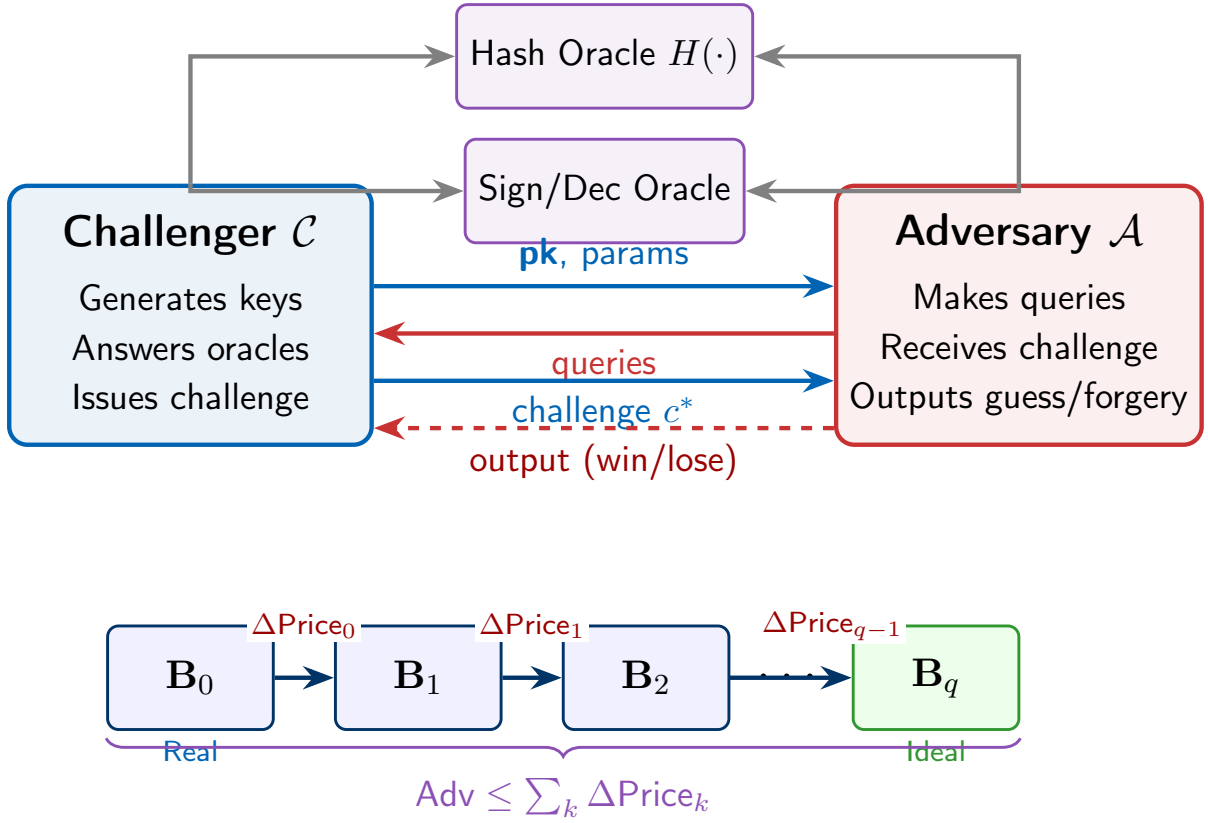


Figure 6: Bidding-round proof architecture. The seller (challenger) and buyer (adversary) interact via oracles. The bidding-round chain $\mathbf{B}_0 \rightarrow \mathbf{B}_q$ bounds the total advantage. In MTSF, each ΔPrice_k is a “price adjustment” in the security market.

all the sensitive operations. A simulator $\mathcal{S}$ mimics the adversary's view.

UC security says: no external observer (the environment $\mathcal{Z}$) can tell which world they are in. This means the real protocol is as good as having a perfect trusted third party.

The simulator: The simulator $\mathcal{S}$ plays the role of “market arbitrageur” in MTSF—it bridges the gap between real and ideal by producing fake views that look indistinguishable from real ones.

Composition theorem: If protocols Π_1 and Π_2 are both UC-secure, then running them together is also UC-secure. This is why UC is so powerful for building complex systems from secure components.

In MTSF: The environment $\mathcal{Z}$ is the “market regulator”—it oversees all market transactions and tries to find any discrepancy between the real and ideal markets.

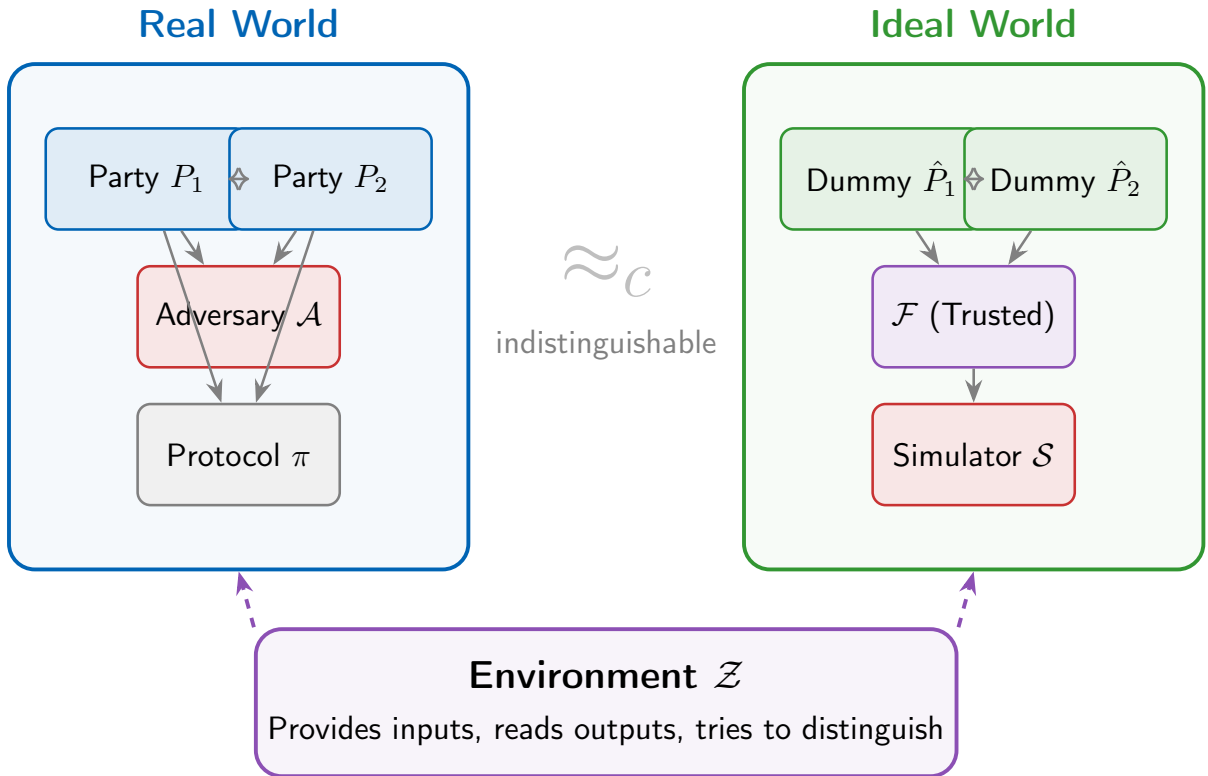


Figure 7: UC framework. The environment $\mathcal{Z}$ interacts with either the real world (protocol π with adversary $\mathcal{A}$) or the ideal world (trusted functionality $\mathcal{F}$ with simulator $\mathcal{S}$). UC security: $\mathcal{Z}$ cannot tell which world it observes.

2.5 GUC and CNF Session Correctness

* Simple Terms: Universal Composability (UC) for Non-Experts

The central question of UC: Can an outsider tell whether they are interacting with the *real* protocol (real people running the actual software) or an *idealised* version (a perfect, unhackable magic box)?

Step-by-step:

1. **Define the ideal:** First, describe what a perfect, unhackable version of the protocol would do. For encryption: “The magic box receives Alice’s message, stores it, and when Bob asks, delivers it—no attacker can see or change it.”

2. **Build the real:** Implement the actual protocol using cryptographic primitives (AES, ECDSA, etc.).
3. **Build the simulator:** Construct a “stunt double” ($\mathcal{S}$) that can make the ideal box *look like* the real protocol to any outside observer, using only the ideal box’s interface.
4. **Prove indistinguishability:** Show that no outside observer (the environment $\mathcal{Z}$) can tell whether they are talking to (real protocol + real attacker) or (ideal box + simulator).

The power of UC: Once you prove a protocol UC-secure, you can *safely replace* the real protocol with the ideal box in any larger system. This is the composition theorem: UC-secure protocols compose safely. In MTSF, this is the “market merger theorem”—merging two equilibrium markets (secure protocols) produces a combined equilibrium market.

The GUC framework [32] removes the CRS assumption via *shared functionalities*. Camenisch *et al.* [30] encode each session as a CNF formula $\varphi_{\text{sid}} = \bigwedge_j C_j$, where satisfiability under the honest trace implies session correctness.

* Simple Terms: GUC Framework

What the CRS assumption is: The basic UC framework sometimes requires a “Common Reference String” (CRS)—a string of random bits that everyone can see and that was honestly generated. In practice, it is hard to guarantee a CRS was generated honestly (who generates it and how?). GUC (Generalized UC) removes this requirement by using *shared functionalities*: well-known public infrastructure like a public-key directory (PKI) that everyone already trusts.

CNF session formulas: Instead of checking session correctness informally, GUC encodes it as a logical formula in Conjunctive Normal Form (CNF)—a checklist where every item must pass. Think of it as a contract: each clause is one contractual obligation. The session is “correct” (the contract is fulfilled) if and only if every obligation is met.

In MTSF: The CNF formula is a “market audit.” The seller (protocol) passes the audit if the formula is satisfied. A dishonest trace (an attacker tampering with the session) fails the audit because at least one clause (e.g., the signature-verification clause) will be unsatisfied.

2.6 Formal Verification

* Simple Terms: Formal Verification Tools for Non-Experts

What ProVerif and Tamarin do: These are automated tools that check whether a protocol is secure by exhaustively exploring all possible attack scenarios. Think of them as very thorough robot testers:

- **ProVerif** uses the Dolev–Yao model, treating cryptography as perfect (unbreakable) black boxes. It checks whether the protocol’s *design* is flawed—like Needham–Schroeder’s missing identity binding. ProVerif found the attack on Needham–Schroeder automatically.
- **Tamarin** is more expressive: it can model algebraic properties of cryptographic operations (e.g., the XOR of two identical values is zero). It has been used to verify TLS 1.3, Signal, and other complex protocols.
- **CryptoVerif** bridges the gap by performing machine-checked game-based proofs, but requires significant manual guidance.

Strengths: Exhaustive coverage (no attack is missed), automated (less human error), and can handle unbounded numbers of sessions.

Limitations: Binary output (secure/insecure, no numerical bounds), may not terminate for complex protocols, and the abstraction may miss computational attacks (e.g., birthday bounds).

In MTSF: Formal verification is “market stress testing”—the regulator (environment) enumerates all buyer strategies and checks that none succeed. MTSF adds numerical bounds and composition guarantees that formal tools lack, while benefiting from their exhaustive coverage for trace-based attacks.

ProVerif [26] and Tamarin [66] provide automated, unbounded symbolic analysis. They check secrecy, authentication, replay/masquerade/MITM resistance. Limitation: no concrete advantage bounds.

* Simple Terms: Formal Verification Tools

What ProVerif and Tamarin do: These are automated tools that take a protocol description as input and mathematically check whether security properties hold—for *all possible* attack scenarios, not just the ones a human thought of. They model the attacker as having full control of the network and search exhaustively.

ProVerif works by translating the protocol into logic rules and using resolution (a kind of automated theorem proving) to check whether an attacker can derive secret values. It is fast but works in an abstract “symbolic” model (messages are terms, not bit strings).

Tamarin uses a more powerful graph-based approach and can reason about complex properties like forward secrecy. It handles a broader class of protocols but can require more manual guidance.

Their limitation in MTSF context: These tools tell you *whether* a protocol is secure (yes/no), but not *how hard* it is to break. They give no numerical advantage bound like “the attacker wins with probability at most 2^{-128} .” MTSF bridges this gap by combining formal verification’s exhaustiveness with game-based proof’s quantitative bounds.

In MTSF: Formal verification is “stress testing” the security market—like a regulator running worst-case scenarios on all possible trading strategies to check for market manipulation.

3 The Market-Theoretic Security Framework

3.1 Security Market Model

Definition 3.1 (Security Market). $\mathcal{M} = (\text{Seller}, \{\text{Buyer}_i\}_{i \in [n]}, \mathcal{G}, \text{Price})$: *seller (challenger), buyers (adversaries), security goods catalogue $\mathcal{G} = \{g_1, \dots, g_k\}$ (e.g., IND-CPA, EUF-CMA, key secrecy), price function $\text{Price} : \mathcal{G} \times \mathbb{N} \rightarrow [0, 1]$.*

* Simple Terms: Security Market

Components of the market:

- **Seller (Challenger $\mathcal{C}$):** The cryptographer who designed the scheme and sets up the security experiment. The seller “offers” guarantees like “this encryption hides your messages.”
- **Buyers (Adversaries $\mathcal{A}_i$):** Potential attackers with bounded computational power. Each buyer has a “budget” (time and memory limit) and tries to find a flaw in the seller’s goods.
- **Security goods catalogue $\mathcal{G}$:** The list of properties being sold—e.g., confidentiality

(IND-CPA), unforgeability (EUF-CMA), session-key secrecy. Each good is a specific security guarantee.

- **Price function Price:** Maps each good to the probability an attacker can break it. A “price” near 0 means the good is worthless to attackers (secure). A “price” near 1 means any attacker can easily break it (insecure).

The key insight: Security is not binary. Instead of “secure” or “broken,” MTSF measures *how hard* it is to break each guarantee, using the language of market prices. A good is “fairly priced” (secure) when no efficient attacker can exploit it for more than a negligible advantage.

Definition 3.2 (Bid). $\text{Bid}_i = (g_j, T_i, \epsilon_i)$: buyer i targets good g_j with budget $T_i \leq \text{poly}(\lambda)$ and target advantage ϵ_i . Succeeds if advantage $\geq \epsilon_i$ within T_i steps.

* Simple Terms: Bids

What a bid is: A bid is an attacker’s attempt to break a specific security property. It has three components:

- **Target good g_j :** Which security property the attacker is trying to break (e.g., “I am bidding on breaking signature unforgeability”).
- **Budget T_i :** How much computation the attacker can afford (must be polynomial in λ —realistic).
- **Target advantage ϵ_i :** How likely the attacker hopes to succeed (e.g., “I want to win at least 1% of the time”).

A bid succeeds if the attacker achieves advantage $\geq \epsilon_i$ within budget T_i . For the scheme to be secure, *all* bids must fail—no attacker can achieve non-negligible advantage within any polynomial budget.

Different bid types (introduced throughout the article):

- **Nonce bid:** Tries to exploit reused random values.
- **Forgery bid:** Tries to produce a valid signature without the private key.
- **Homomorphism bid:** Exploits algebraic structure (e.g., in RSA) to forge without the key.
- **Differential bid:** Exploits patterns in how small input changes affect outputs (used against block ciphers).
- **Impersonation bid:** Tries to pretend to be another party in a protocol.

Definition 3.3 (Quoted Price). The quoted price $\text{QPrice}_0(g_j)$ is the seller’s initial listing price for security good g_j in bidding round $\mathbf{B}_0$. It equals the adversary’s raw advantage in the unmodified real-world game:

$$\text{QPrice}_0(g_j) = \text{Adv}_{\mathcal{A}}^{g_j}(\lambda) = \text{Ask}_0(g_j, \lambda). \quad (3)$$

The quoted price is an unknown but bounded quantity: it is the number the entire proof exists to bound. Subsequent bidding rounds $\mathbf{B}_1, \mathbf{B}_2, \dots, \mathbf{B}_q$ decompose QPrice_0 into a telescoping sum of price adjustments:

$$\text{QPrice}_0 = \underbrace{\Delta\text{Price}_0}_{\text{bid 1}} + \underbrace{\Delta\text{Price}_1}_{\text{bid 2}} + \dots + \underbrace{\Delta\text{Price}_{q-1}}_{\text{bid } q} + \underbrace{\text{Pr}[\mathbf{B}_q = 1]}_{\text{ideal residual}}. \quad (4)$$

Security holds when $\text{QPrice}_0 \leq \text{negl}(\lambda)$; insecurity holds when $\text{QPrice}_0 = \Omega(1)$.

* Simple Terms: The Quoted Price—Why It Matters

What the quoted price is: When the seller opens the market in $\mathbf{B}_0$, the security good is “listed” at a quoted price QPrice_0 . Think of it like a market where the seller lists the good at quoted price “999” (insecure—the good is worthless), “555” (quasi-secure—partial value), or “333” (secure—the price is a bluff the adversary cannot collect). The quoted price is the raw adversary advantage—*before any analysis has been done*. It is the starting point of the proof, not the conclusion.

Why it remains “as quoted” without reduction: In $\mathbf{B}_0$, no modifications have been made to the game. The adversary has full, unrestricted access to all oracles. The quoted price QPrice_0 is simply the adversary’s best winning probability in this strongest setting. No price adjustment occurs ($\Delta\text{Price}_{-1} = 0$) because nothing has changed yet. The quoted price stands as-is—it is the *benchmark* against which all subsequent bidding rounds are measured.

The proof’s job: Show that QPrice_0 decomposes entirely into negligible terms. Each bidding round $\mathbf{B}_k$ peels off one component of QPrice_0 (one attack vector), showing it contributes at most $\Delta\text{Price}_{k-1} \leq \text{negl}$. After all attack vectors are peeled off, the residual is zero (ideal game). Since $\text{QPrice}_0 = \sum \Delta\text{Price}_k + 0$, and each ΔPrice_k is negligible, QPrice_0 itself is negligible. The originally quoted price was a *bluff*—the adversary cannot actually collect on it.

Three illustrative quoted prices:

- $\text{QPrice}_0 = \text{“999”}$ (**Needham–Schroeder**): The seller quotes a high price, but the proof reveals it cannot be reduced—the masquerade bid succeeds with $\Delta\text{Price} = 1$. The quoted price was honest: the good really is worthless (market collapsed).
- $\text{QPrice}_0 = \text{“555”}$ (**Telegram MTPROTO**): The seller quotes a moderate price. Most bids are peeled off as negligible, but the salt-extraction bid contributes $\Delta\text{Price} \approx 2^{-64}$ —small but not negligible for high-security applications. The quoted price was *partially* a bluff: the good has some value but not full value (quasi-collapse).
- $\text{QPrice}_0 = \text{“333”}$ (**ECDSA, ML-KEM, AES**): The seller quotes a modest price. Every bid is peeled off as negligible: nonce bids ($q_S^2/2q$), hash bids ($q_H^2/2q$), forgery bids ($\text{Adv}^{\text{ECDLP}}$). The residual is zero. The quoted price was entirely a bluff—the adversary wins essentially nothing (equilibrium).

Definition 3.4 (Ask Price & Equilibrium). $\text{Ask}(g_j, \lambda) = \max_{\mathcal{A} \in \text{PPT}} \text{Adv}_{\mathcal{A}}^{g_j}(\lambda)$. The market is in equilibrium if $\text{Ask}(g_j, \lambda) \leq \text{negl}(\lambda)$: no PPT buyer profits. The market has collapsed if $\text{Ask}(g_j, \lambda) = \Omega(1)$: some buyer profits cheaply.

* Simple Terms: Ask Price and Equilibrium

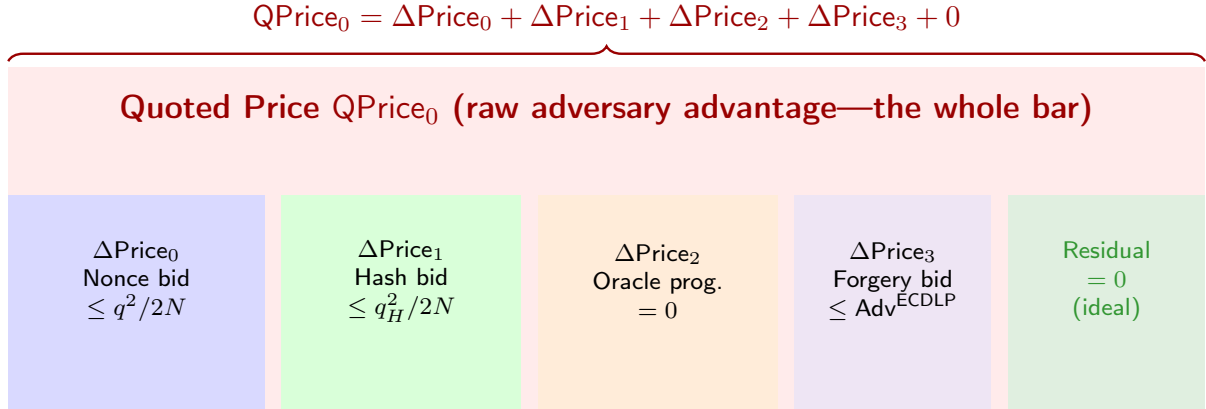
Ask price: This is the best an attacker can possibly do against a security good—the maximum winning probability over all efficient attacks. Think of it as the “market value” of a security vulnerability: how much can an attacker extract from the scheme?

Equilibrium (scheme is secure): The ask price is negligible—essentially zero. No efficient attacker can win with any meaningful probability. The market has settled at a “fair price”: security guarantees cannot be broken cheaply.

Collapse (scheme is broken): The ask price is substantial—an attacker wins easily. The market collapses like a stock crash: security guarantees are worthless.

Concrete examples:

- **ECDSA:** Ask price $\approx 2^{-128}$ for standard parameters. Equilibrium.



Every piece is negligible $\Rightarrow QPrice_0 \leq \text{negl} \Rightarrow$ **Equilibrium**: the quoted price was a bluff

Figure 8: The quoted price $QPrice_0$ as a bar decomposed into bidding-round contributions. Each bidding round “peels off” one piece of the quoted price by bounding one attack vector. If every piece is negligible and the residual is zero, the quoted price is negligible—the market is in equilibrium.

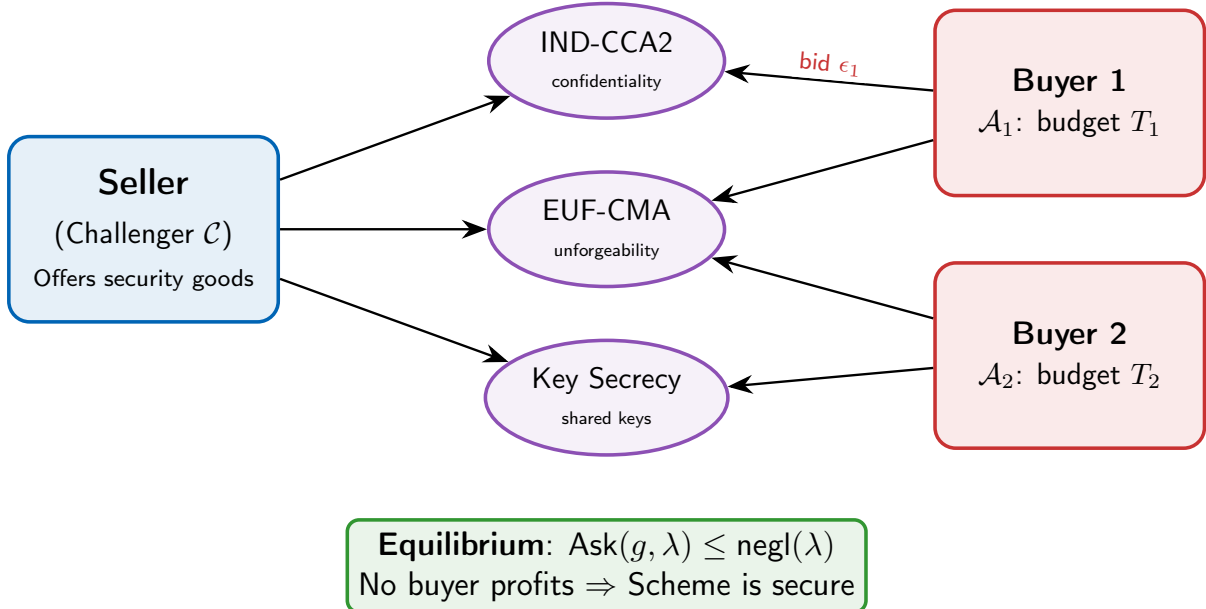


Figure 9: MTSF market: seller offers security goods; buyers bid computation; equilibrium = security.

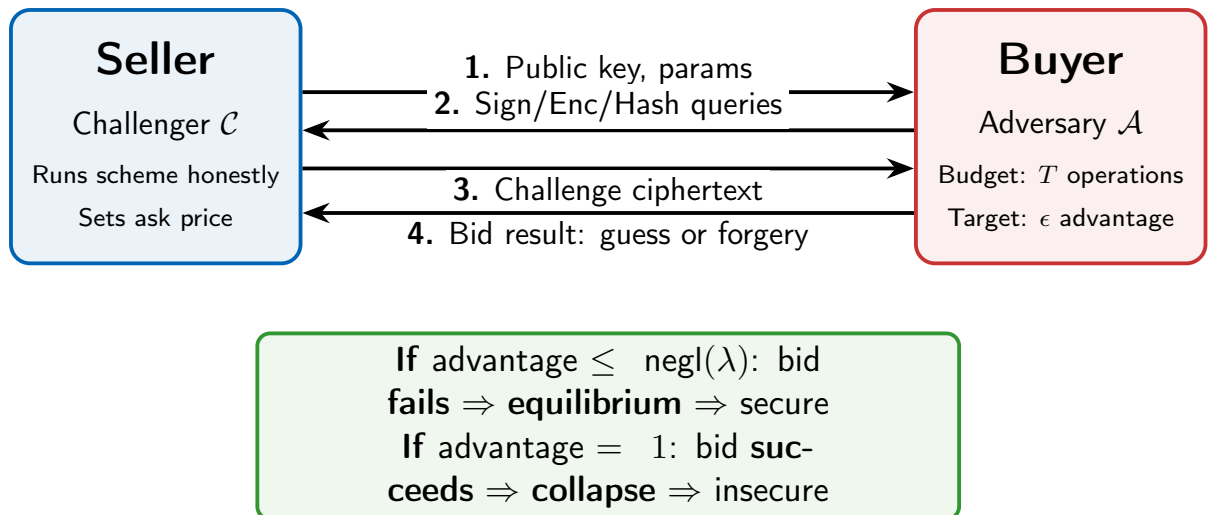


Figure 10: Individual seller–buyer interaction: the four-phase security game as a market transaction.

- **Textbook RSA signatures:** Ask price = 1 (attacker always wins). Collapsed.
- **Needham–Schroeder protocol:** Ask price = 1 (Lowe’s man-in-the-middle attack always succeeds). Collapsed.

Theorem 3.5 (Equilibrium $\Leftrightarrow$ Security; Collapse $\Leftrightarrow$ Insecurity). Π is secure for g_j iff $\mathcal{M}$ is in equilibrium. Π is insecure iff $\mathcal{M}$ has collapsed.

Analogy:

Security as a Farmers’ Market Think of a farmers’ market. The seller (farmer) offers organic produce (security guarantees). Buyers (adversaries) try to find defects (vulnerabilities) to negotiate a lower price (gain advantage). If the produce is genuinely organic (the scheme is secure), no buyer can find a real defect—the market price holds (equilibrium). If the produce is contaminated (the scheme is broken), any buyer can spot the defect immediately—the price collapses to zero (market collapse).

Analogy:

The Complete MTSF Dictionary for Everyday Life Here is the complete translation table between MTSF and everyday concepts. Keep this in mind as you read:

- **Security market** = A courtroom trial where the prosecution (buyer) tries to prove the defendant (scheme) is guilty (insecure), and the defence (seller) must show innocence (security). The judge (mathematical proof) delivers the verdict.
- **Security good** = A specific claim the defence makes: “My client never stole anything” (confidentiality), “My client’s signature was never forged” (unforgeability), “My client’s conversations were always private” (key secrecy).
- **Bid** = A specific line of attack by the prosecution: “I will show the defendant reused a password” (nonce bid), “I will show someone faked a signature” (forgery bid).
- **Ask price** = How strong the prosecution’s best case is. Near zero = the defendant is clearly innocent. Near one = the prosecution wins easily.

- **Equilibrium** = The defence wins; the scheme is secure. Like a stock price that holds steady because no trader can exploit it.
- **Collapse** = The prosecution wins; the scheme is broken. Like a stock crash—everyone can see the vulnerability.
- **Price adjustment** = The small change in the prosecution's strength when one line of evidence is added or removed. Each game hop adjusts the price by a tiny amount.
- **Bidding round** = One round of the trial where a specific piece of evidence is examined.

The Security Thermometer

Real-World Comparison

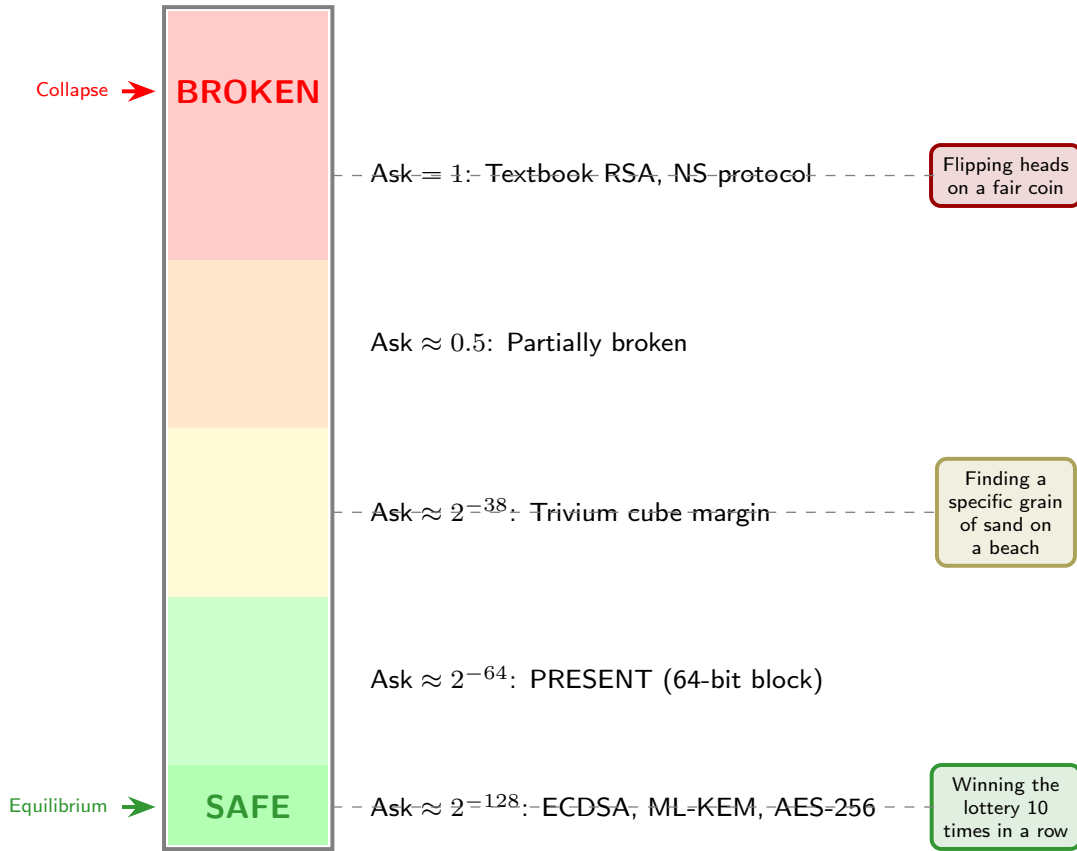


Figure 11: The Security Thermometer: visualising ask prices on a scale from perfectly secure (bottom, green) to completely broken (top, red). Real-world case studies are plotted along the scale, with everyday probability comparisons on the right.

3.2 Game Hops as Price Adjustments

Definition 3.6 (Price Adjustment). $\Delta \text{Price}_k = |\Pr[\mathbf{B}_k(\mathcal{A}) = 1] - \Pr[\mathbf{B}_{k+1}(\mathcal{A}) = 1]|$. *Cumulative cost*: $\text{Cost}(\mathcal{A}) = \sum_{k=0}^{q-1} \Delta \text{Price}_k$.

Analogy:

Game Hops as Climbing a Staircase Imagine you are standing at the bottom of a staircase (the real security game, $\mathbf{B}_0$) and you want to reach the top (the ideal game, $\mathbf{B}_q$, where the attacker has zero advantage). Each step you climb is one *game hop*—a small, controlled

change to the experiment.

The key rule: At each step, you measure how much the “height” (the attacker’s advantage) changes. If each step changes the height by at most a tiny amount (ΔPrice_k), then by the time you reach the top, the total height change is at most the sum of all the tiny step changes. Since the ideal game has advantage zero, the real game’s advantage is bounded by the sum of all step changes.

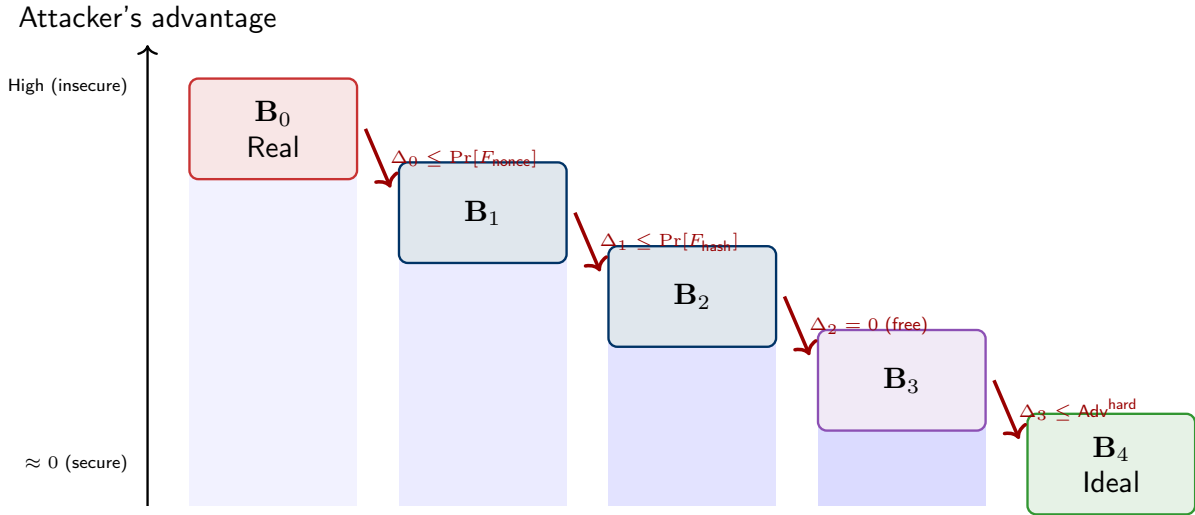
In pictures:

$$\underbrace{B_0}_{\text{real}} \xrightarrow{\Delta_0} B_1 \xrightarrow{\Delta_1} B_2 \xrightarrow{\Delta_2} \dots \xrightarrow{\Delta_{q-1}} \underbrace{B_q}_{\text{ideal (Adv=0)}}$$

Total advantage $\leq \Delta_0 + \Delta_1 + \dots + \Delta_{q-1}$.

Each Δ_k is bounded by the probability of a “bad event” F_k —like a nonce collision or a hash collision. If all bad events are extremely unlikely, the total advantage is negligible, and the scheme is secure.

Real-world analogy: Think of flying from New York to London with layovers. Each flight segment has a tiny risk of delay. The total delay risk is bounded by adding up the risks of each segment. If each segment has only a 0.0001% risk of significant delay, even 10 segments give only 0.001% total risk—negligible.



$$\text{Total Ask Price} \leq \Delta_0 + \Delta_1 + \Delta_2 + \Delta_3 \quad \text{If all } \Delta_k \text{ are tiny} \Rightarrow \text{Equilibrium}$$

Figure 12: Game hops as a descending staircase: each step reduces the attacker’s advantage by eliminating one attack vector. The total advantage (height of the staircase) is the sum of all step sizes. If every step is negligibly small, the scheme is secure.

Game Hops vs. Bidding Rounds: A Side-by-Side Comparison

The following table shows how a classical game-based proof step translates into an MTSF bidding round. The left column is what a game-based cryptographer would write; the right column is the MTSF version of the *same* argument. Notice that the MTSF version preserves all technical content while adding classification, market interpretation, and tighter multi-failure handling.

Classical Game Hop	MTSF Bidding Round
G_0 : Real scheme.	B_0 (Real Market): Seller offers goods at ask price Ask_0 .
G_1 : Abort on nonce collision. “ $ \Pr[G_0=1] - \Pr[G_1=1] \leq q^2/2N$.”	B_1 (Nonce Freshness Bid): Buyer attempts nonce-reuse attack. $\Delta\text{Price}_0 \leq q^2/2N$. <i>Bid fails: nonce space too large.</i>
G_2 : Replace hash with random. “Indistinguishable by RO.”	B_2 (Oracle Programming—Costless Restructuring): Seller secretly programs oracle. $\Delta\text{Price}_1 = 0$. <i>Market restructured at zero cost.</i>
G_3 : Reduce forgery to ECDLP. “Standard reduction.”	B_3 (Forgery-to-ECDLP Bid): If buyer forges, seller consumes buyer as subroutine to solve ECDLP. $\Delta\text{Price}_2 \leq q_H q_S \cdot \text{Adv}^{\text{ECDLP}}$. <i>Bid fails: ECDLP is hard.</i>
G_q : Ideal. $\Pr[G_q=1] = 0$.	B_q (Ideal Market): All bids eliminated. $\Pr[B_q=1] = 0$. <i>Market in equilibrium.</i>
Total: $\text{Adv} \leq \sum \delta_k$.	Total: $\text{Ask}(g) \leq \sum \Delta\text{Price}_k$. <i>Plus: each ΔPrice_k is labelled by bid type; multi-failure hops use extended diff. lemma; CNF worksheet and ping bid appended.</i>

Key takeaway: The mathematical content is *identical*. The difference is that MTSF provides a richer annotation structure (named bids, explicit market interpretation, multi-failure handling, session auditing) that makes the proof more informative, auditable, and composable. Translating an existing game-based proof into MTSF is a mechanical process that adds value at no technical cost.

3.3 The Extended Difference Lemma

The classical difference lemma (Theorem 2.8) handles *one* failure per hop. In practice, a single game transition often involves multiple simultaneous failures. We extend the lemma:

Lemma 3.7 (Extended Difference Lemma—Multiple Failures).

Analogy:

Multiple Failure Events as Insurance Claims In insurance, a single event (a storm) can trigger multiple claims simultaneously: roof damage (F_1), flooding (F_2), power outage (F_3). The insurer’s total payout is bounded by the probability of any claim occurring: $\Pr[F_1 \cup F_2 \cup F_3]$. If the events are independent, the union bound suffices. If correlated (flooding causes power outage), inclusion-exclusion gives a tighter bound by subtracting the overlap.

Let $\mathbf{B}_k$ and $\mathbf{B}_{k+1}$ be identical unless at least one of $F_1, F_2, \dots, F_m$ occurs. Then:

$$|\Pr[\mathbf{B}_k(\mathcal{A}) = 1] - \Pr[\mathbf{B}_{k+1}(\mathcal{A}) = 1]| \leq \Pr[F_1 \cup F_2 \cup \dots \cup F_m]. \quad (5)$$

By the **union bound**: $\Pr[\bigcup_i F_i] \leq \sum_i \Pr[F_i]$. When failures are correlated, the **inclusion-exclusion** bound is tighter:

$$\Pr\left[\bigcup_i F_i\right] = \sum_i \Pr[F_i] - \sum_{i < j} \Pr[F_i \cap F_j] + \dots + (-1)^{m+1} \Pr\left[\bigcap_i F_i\right]. \quad (6)$$

Proof. $\mathbf{B}_k$ and $\mathbf{B}_{k+1}$ agree on $\overline{\bigcup_i F_i}$. Hence:

$$|\Pr[\mathbf{B}_k = 1] - \Pr[\mathbf{B}_{k+1} = 1]|$$

$$\begin{aligned}
&= \left| \Pr\left[\mathbf{B}_k = 1 \wedge \bigcup F_i\right] + \Pr\left[\mathbf{B}_k = 1 \wedge \overline{\bigcup F_i}\right] - \Pr\left[\mathbf{B}_{k+1} = 1 \wedge \bigcup F_i\right] - \Pr\left[\mathbf{B}_{k+1} = 1 \wedge \overline{\bigcup F_i}\right] \right| \\
&= \left| \Pr\left[\mathbf{B}_k = 1 \wedge \bigcup F_i\right] - \Pr\left[\mathbf{B}_{k+1} = 1 \wedge \bigcup F_i\right] \right| \leq \Pr\left[\bigcup_i F_i\right]. \quad \square
\end{aligned}$$

* Simple Terms: Extended Difference Lemma

What is new over the classical lemma? The classical difference lemma handles one bad event per game hop. But in complex proofs—like the ECDSA proof—a single game transition might involve *three* things that could go wrong simultaneously: a nonce collision, a hash collision, and a forking failure. Splitting this into three separate game hops is cumbersome.

The extended lemma: We can handle all m bad events $(F_1, \dots, F_m)$ in a single hop, bounding the total difference by the probability that *any* bad event occurs: $\Pr[F_1 \cup F_2 \cup \dots \cup F_m]$.

Union bound vs. inclusion-exclusion:

- **Union bound** (simple): $\Pr[F_1 \cup F_2 \cup \dots \cup F_m] \leq \Pr[F_1] + \Pr[F_2] + \dots + \Pr[F_m]$. Easy to compute but potentially loose (overcounts overlaps).
- **Inclusion-exclusion** (tighter): Subtract pairwise overlaps ($\Pr[F_i \cap F_j]$), add back triple overlaps, etc. Tighter but more complex.

When to use which: If the bad events are essentially independent (like independent hash collisions), the union bound is nearly tight. If they are correlated (e.g., a nonce collision also causes a hash collision), inclusion-exclusion gives a significantly better bound.

Market interpretation: Multiple risks (bad events) are like correlated insurance claims. If flooding always causes power outages, you do not need to pay for two separate “flood” and “outage” claims—you account for the overlap. MTSF’s extended lemma does this for security proofs.

Key Insight:

Why the Extension Matters Consider an ECDSA game hop where three things can go wrong simultaneously: F_1 : nonce collision ($\leq q_S^2/2q$), F_2 : hash collision ($\leq q_H^2/2q$), F_3 : forking failure ($\leq 1/q_H$). Classical approach: split into three separate hops, each with its own game. Extended lemma: handle all three in *one* hop: $\Delta\text{Price} \leq \Pr[F_1 \cup F_2 \cup F_3] \leq \Pr[F_1] + \Pr[F_2] + \Pr[F_3] - \Pr[F_1 \cap F_2] - \dots$, yielding a tighter bound and shorter proof.

Analogy:

Extended Difference Lemma as a Kitchen Fire Risk Assessment Suppose you are assessing the risk of a kitchen fire. Three things could go wrong simultaneously:

- F_1 : The stove is left on (probability 2%).
- F_2 : A towel is near the stove (probability 5%).
- F_3 : The smoke detector battery is dead (probability 1%).

Union bound (simple): $\Pr[\text{any problem}] \leq 2\% + 5\% + 1\% = 8\%$. This overcounts because if the stove is left on AND a towel is nearby, we counted that scenario twice.

Inclusion-exclusion (tighter): Subtract the overlaps. If $\Pr[F_1 \cap F_2] = 0.5\%$ (stove on AND towel nearby), then: $\Pr[\text{any problem}] \leq 8\% - 0.5\% - \dots \approx 7.5\%$.

In MTSF: Replace “stove” with “nonce collision,” “towel” with “hash collision,” and “smoke detector” with “forking failure.” The extended difference lemma handles all three risks in

a single game hop instead of requiring three separate hops. The tighter inclusion-exclusion bound means the final security estimate is closer to reality.

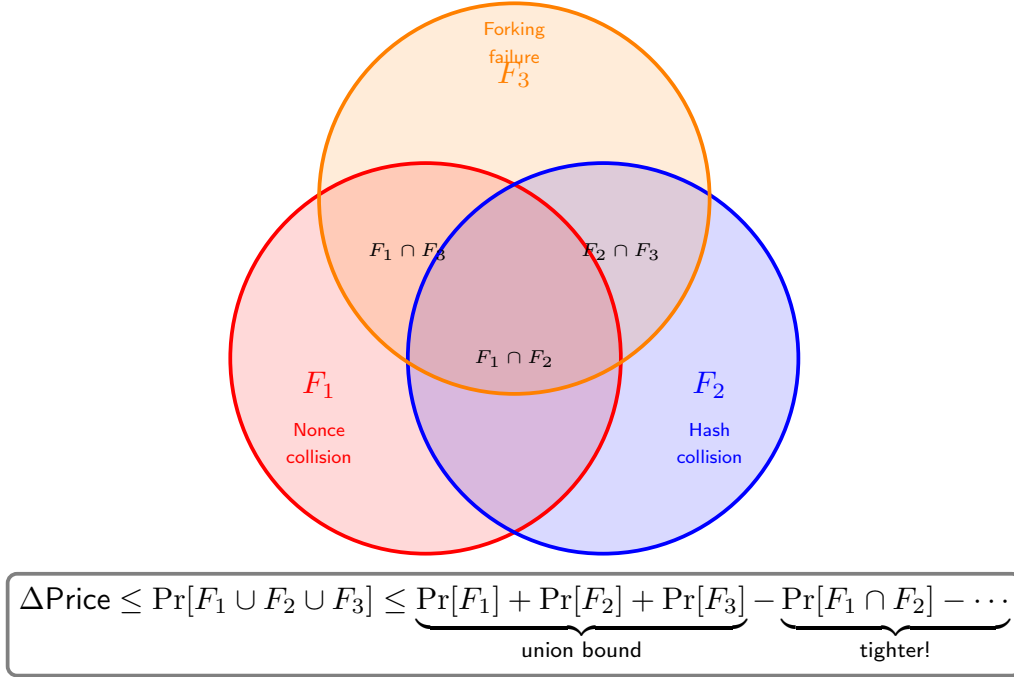


Figure 13: Venn diagram of simultaneous failure events in the extended difference lemma. The union bound adds all circles; inclusion-exclusion subtracts the overlaps for a tighter bound. Each failure event corresponds to a specific “bad thing” that could happen during a game hop.

3.4 UC as Market Regulation; GUC as Shared Infrastructure

The UC environment $\mathcal{Z}$ is recast as a *market regulator*: an entity monitoring all concurrent trades (protocol messages) and capable of intervening at any point. UC-security: the real market (protocol execution) is computationally indistinguishable from the ideal market (ideal functionality) from any regulator’s perspective— $\text{Real}_{\pi, \mathcal{A}, \mathcal{Z}} \approx_c \text{Ideal}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}$. The simulator $\mathcal{S}$ is the *market arbitrator*: it bridges the real and ideal worlds by simulating the real protocol’s market behaviour using only the ideal functionality’s interface.

The *UC composition theorem* is a *market merger theorem*: two equilibrium markets, when merged for concurrent execution, remain in equilibrium. GUC shared functionalities (PKI, common reference strings) are *market infrastructure*: public goods accessible to all buyers and sellers. In MTSF, the CNF session formula plays the role of a GUC correctness requirement: each session passes the audit iff all clauses are satisfied, and a failed clause identifies which market good has been compromised.

Analogy:

UC as a Government Regulatory Agency Think of UC (Universal Composability) as a government food safety agency. The **environment** $\mathcal{Z}$ is the *head inspector* who can walk into any restaurant (protocol) at any time, order any dish, and test it. The **simulator** $\mathcal{S}$ is a *stunt double*—a stand-in who must perfectly mimic the real chef’s cooking using only pre-approved ingredients (the ideal functionality). If the head inspector cannot tell the difference between the real restaurant and the stunt double’s restaurant, then the food (protocol) is safe.

The composition theorem says: if Restaurant A passes inspection and Restaurant B

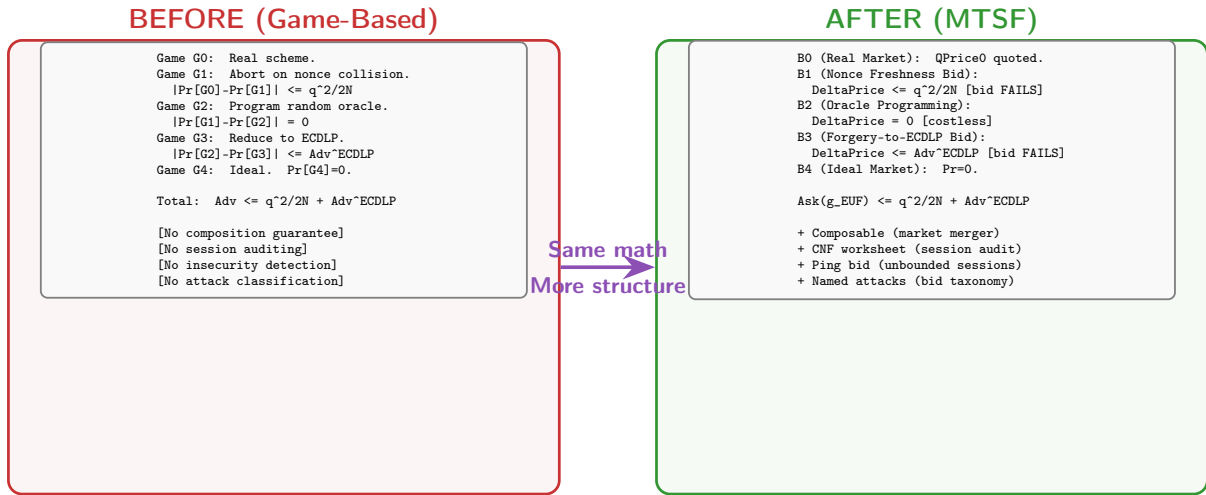


Figure 14: Before/After: the same ECDSA proof written in classical game-based style (left) vs. MTSF style (right). The mathematical content is identical; MTSF adds named bids, composition, session auditing, and unbounded coverage.

passes inspection, then a food court containing both restaurants *also passes inspection*—even if customers eat at both simultaneously. This is the “market merger theorem” in MTSF: two equilibrium markets stay in equilibrium when merged.

GUC shared infrastructure is like the electricity grid, water supply, and health codes that *all* restaurants share. In MTSF, this is the PKI (public key infrastructure), common reference strings, and other shared resources. The CNF session formula is the health code checklist that every restaurant must pass.

★ Intuition:

Why Composition Is So Important Without composition guarantees, you might prove that your door lock is secure and your alarm system is secure, but when you install both together, they might interfere and create a vulnerability that neither had alone. UC-style composition (MTSF’s “market merger theorem”) ensures this cannot happen: secure components remain secure when combined. This is essential for real-world systems like TLS 1.3 running alongside the Signal protocol, which MTSF analyses in Section 16.

How UC/GUC Practitioners Can Apply MTSF—A Step-by-Step Migration Guide

If you already work in the UC/GUC framework, MTSF is *not* a replacement—it is an enrichment. Every UC/GUC concept has a direct MTSF counterpart, and the translation preserves all composition guarantees while adding quantitative bounds, named attacks, and session auditing. Here is the systematic translation:

Step 1: Translate your ideal functionality into market goods.

In UC, you define an ideal functionality $\mathcal{F}$ that captures what the protocol should achieve.

In MTSF, you decompose $\mathcal{F}$ into a catalogue of *security goods* $\mathcal{G} = \{g_1, \dots, g_k\}$:

UC ideal functionality $\mathcal{F}$	MTSF security goods $\mathcal{G}$
$\mathcal{F}_{\text{KE}}$ (key exchange)	g_{auth} (authentication) + g_{sk} (session-key secrecy)
$\mathcal{F}_{\text{SIG}}$ (digital signature)	$g_{\text{EUF-CMA}}$ (unforgeability)
$\mathcal{F}_{\text{PKE}}$ (public-key encryption)	$g_{\text{IND-CCA2}}$ (confidentiality)
$\mathcal{F}_{\text{AUTH}}$ (mutual authentication)	g_{mutual} (mutual auth) + g_{CNF} (session correctness)
$\mathcal{F}_{\text{SC}}$ (secure channel)	$g_{\text{auth}} + g_{\text{sk}} + g_{\text{mutual}} + g_{\text{CNF}}$

Each good gets its own ask price—a numerical bound that UC does not provide.

Step 2: Translate your simulator into a bidding-round chain.

In UC, you construct a simulator $\mathcal{S}$ that makes the ideal world look like the real world. This simulator typically handles multiple cases: key generation, oracle queries, corruption, message delivery. In MTSF, each “case” in the simulator corresponds to a *bidding round*:

UC simulator step	MTSF bidding round
$\mathcal{S}$ handles key generation	$\mathbf{B}_0$: Real market (honest key generation)
$\mathcal{S}$ programs the random oracle	$\mathbf{B}_k$: Oracle programming bid ($\Delta\text{Price} = 0$)
$\mathcal{S}$ simulates signing queries	$\mathbf{B}_k$: Nonce freshness bid ($\Delta\text{Price} \leq q^2/2^{\lambda+1}$)
$\mathcal{S}$ extracts from forgery	$\mathbf{B}_k$: Forgery-to-hardness bid ($\Delta\text{Price} \leq \text{Adv}^{\text{hard}}$)
$\mathcal{S}$ handles corruption	$\mathbf{B}_k$: Corruption bid (session pinging checks)

The key advantage: each bidding round produces an explicit ΔPrice_k bound. Summing these gives the ask price—something the UC simulator alone cannot provide.

Step 3: Translate the UC environment into a market regulator.

The UC environment $\mathcal{Z}$ can:

- Choose inputs for all parties $\rightarrow$ The regulator sets market conditions.
- Read outputs from all parties $\rightarrow$ The regulator observes market outcomes.
- Interact with the adversary $\rightarrow$ The regulator monitors buyer behaviour.
- Output a distinguishing bit $\rightarrow$ The regulator issues a verdict (equilibrium vs. collapse).

The UC condition $\text{Real} \approx_c \text{Ideal}$ becomes: the market regulator cannot distinguish the real market from the ideal market. This is *identical* in power to UC—no loss of generality.

Step 4: Apply the market merger theorem instead of the UC composition theorem.

The UC composition theorem says: if π UC-realises $\mathcal{F}$ and π' uses $\mathcal{F}$ as a subroutine, then $\pi' \circ \pi$ UC-realises π' ’s ideal functionality. In MTSF:

$$\text{Ask}(\mathcal{M}_{\text{composed}}) \leq \text{Ask}(\mathcal{M}_{\pi}) + \text{Ask}(\mathcal{M}_{\pi'}) + \text{Ask}(\mathcal{M}_{\text{infra}})$$

where $\mathcal{M}_{\text{infra}}$ accounts for shared infrastructure (PKI, CRS). This is the UC composition theorem *plus* a concrete numerical bound on the composed system’s security. You get everything UC gives, and more.

Step 5: Translate GUC’s CNF session correctness into MTSF’s CNF audit.

GUC uses CNF formulas to encode session correctness [30]. MTSF adopts this directly:

- GUC’s session formula $\varphi_{\text{sid}} = \bigwedge_j C_j$ is exactly MTSF’s session-CNF φ_i (Theorem 4.1).
- GUC’s satisfiability check is MTSF’s five-phase verification algorithm (Algorithm 1).
- The difference: MTSF adds a *manual worksheet* for each case study and a *quantitative bound* on the probability that an adversary can satisfy the CNF under a dishonest trace.

Step 6: Add session pinging for unbounded verification.

UC/GUC proofs implicitly handle unbounded sessions (the environment can create arbitrarily many sessions). MTSF makes this explicit via session pinging (Section 5), providing:

- A formal structural distinctness check between consecutive sessions (Theorem 5.1).

- An induction theorem (Theorem 5.5) that lifts bounded security to unbounded security.
- Quantitative advantage accumulation bounds ($n \cdot \epsilon$ for n independent sessions).

This is the bridge between UC's implicit unbounded coverage and game-based proofs' explicit numerical bounds.

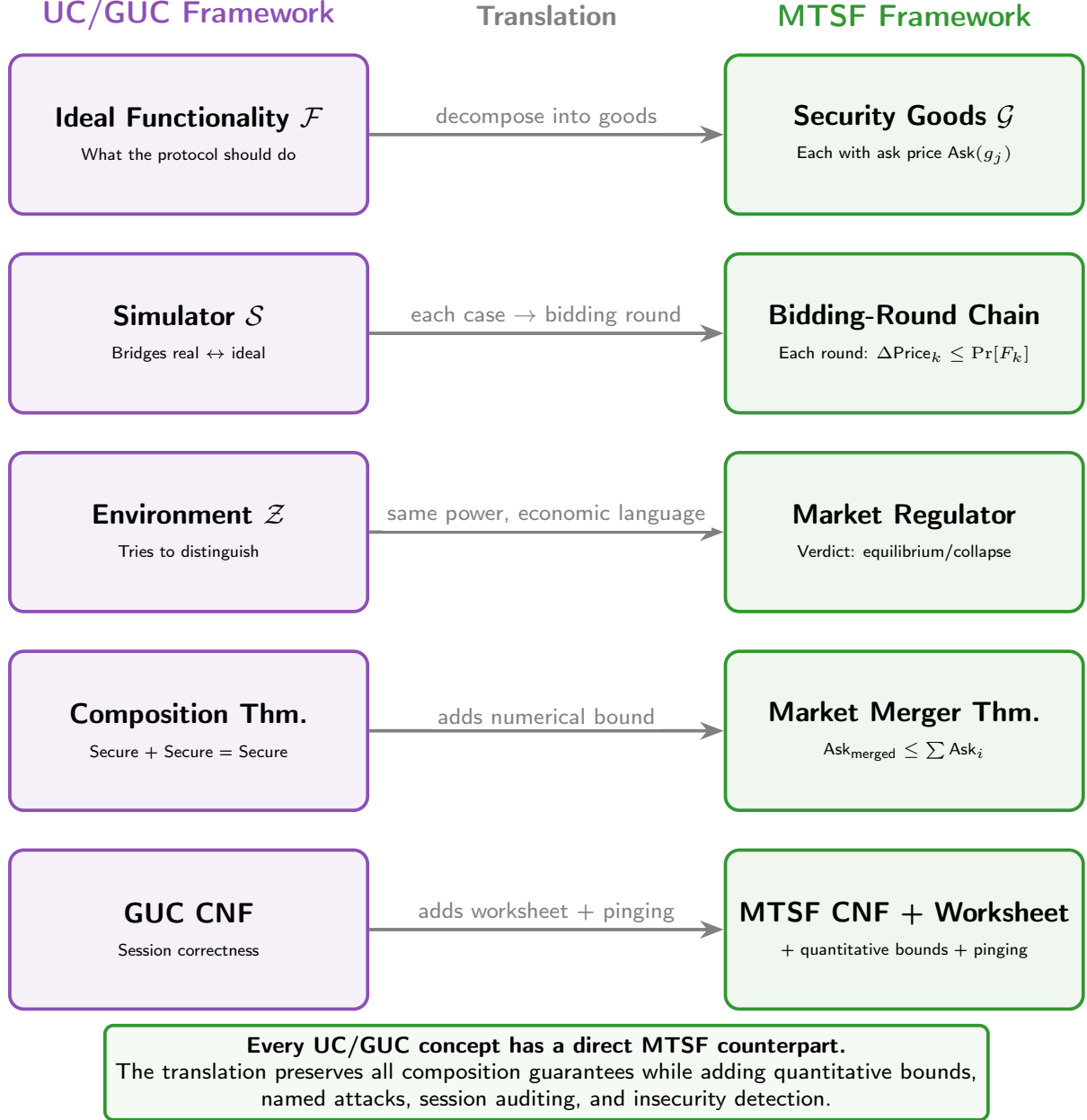


Figure 15: Systematic translation from UC/GUC to MTSF. Every UC concept (left) maps to an MTSF counterpart (right) that preserves all guarantees and adds quantitative bounds. The translation is lossless—MTSF strictly extends UC/GUC.

3.5 Formal Verification as Market Stress Testing

Model checking (ProVerif [26], Tamarin [66]) is recast as *regulatory stress testing*: the regulator enumerates all possible buyer strategies (adversarial protocol traces τ) and checks $\forall \tau \in$

$\text{Traces}(\mathcal{M}) : \tau \models \varphi_{g_j}$ for all goods g_j . In MTSF notation: **replay attack** = stale bid (old session transcript at discounted price; blocked by SID clause); **masquerade attack** = identity fraud (claiming to be a different party; blocked by signature/certificate clause); **MITM attack** = market manipulation (intercepting and altering both sides; blocked by SID-binding and identity-binding clauses). Formal verification provides exhaustive coverage for all traces; MTSF’s session pinging (Section 5) provides quantitative advantage bounds for all sessions—together bridging the qualitative-unbounded and quantitative-bounded paradigms.

* Simple Terms: Side-Channel Attacks—A Complete Guide for Non-Experts

What are side-channel attacks? Every computer leaks information through physical channels: it consumes power, emits electromagnetic radiation, makes noise, takes time, and heats up. A side-channel attack exploits these unintentional leaks to extract secret keys *without* breaking the mathematical security of the cryptographic algorithm. Think of it as a burglar who does not pick the lock but instead listens to the clicks of the tumblers.

Why do they matter for MTSF? A scheme can be mathematically secure (market in equilibrium) but completely broken at the implementation level. MTSF’s side-channel bid ($\mathbf{B}_{\text{sca}}$) captures this: even if all mathematical bids fail (nonce, hash, forgery), a physical leakage bid can still collapse the market. The comprehensive countermeasure suite (masking, constant-time code, fault detection, key erasure) is the price the seller must pay to keep the implementation-level market in equilibrium.

The nine side-channel attack classes in plain language:

1. **Simple Power Analysis (SPA):** Watch the power meter while the computer does crypto. Different operations (multiply vs. square) consume different power—one trace reveals the secret key bit by bit. *Like:* Watching someone type a password by looking at which keys light up on a backlit keyboard.
2. **Differential Power Analysis (DPA):** Collect many power traces with different inputs. Statistically correlate the traces with guesses about the key. The correct guess shows a strong correlation; wrong guesses show noise. *Like:* A detective who interviews 1000 witnesses—individually each remembers little, but the consensus reveals the truth.
3. **Timing Analysis:** Measure how long the operation takes. If the time depends on the secret (e.g., early exit on a wrong password character), the timing reveals the secret. *Like:* A safecracker who listens for the click when each tumbler aligns—the time between clicks reveals the combination.
4. **Cache-Timing Attacks:** Exploit the CPU’s memory cache. When the crypto code accesses a lookup table, the table entries are loaded into the cache. The attacker measures which entries are cached (fast to access) vs. not cached (slow)—revealing which entries were used, which reveals the key. *Like:* Checking which books in a library are warm (recently handled) to deduce what someone was reading.
5. **Electromagnetic (EM) Analysis:** Hold a radio antenna near the chip and record the EM radiation. Different computations produce different radiation patterns. More spatially precise than power analysis—can target individual circuit components. *Like:* Using a stethoscope on a vault door to listen to specific tumblers.
6. **Fault Injection:** Deliberately cause the computer to make a mistake (by zapping it with a laser, glitching the voltage, or disrupting the clock). A single faulty computation can leak the entire key via Differential Fault Analysis. *Like:* Tripping someone carrying a tray of sorted cards—the way the cards scatter reveals how they were sorted.

7. **Cold Boot Attack:** After the computer is turned off, the memory (DRAM) retains its contents for seconds to minutes. Cooling the memory chips extends retention dramatically. The attacker freezes the RAM, removes it, and reads the secret keys directly. *Like:* A chalkboard that is not fully erased—you can still read what was written.
8. **Acoustic Cryptanalysis:** Record the sound the computer makes during crypto. The CPU, capacitors, and power supply emit sounds at frequencies correlated with computation. RSA keys have been recovered from the sound of a laptop across a room. *Like:* A piano tuner who can identify which keys are being pressed from the sound alone.
9. **Photonic Emission:** Transistors emit faint photons when they switch. A sensitive camera on the chip’s backside can capture these photons and reconstruct the computation. *Like:* Reading a message by the fireflies’ blinking pattern—each blink corresponds to a bit flip.

The MTSF countermeasure suite: The seller achieves equilibrium against all nine attacks by deploying:

- Order- d masking (defeats SPA, DPA, EMA, acoustic, photonic).
- Constant-time code (defeats timing and cache attacks).
- Bitsliced or hardware-accelerated operations (defeats cache attacks).
- Redundant computation with comparison (defeats fault injection).
- Immediate key zeroing + encrypted RAM (defeats cold boot).
- Physical shielding (defeats EM and photonic).

With all countermeasures, the total side-channel advantage is negligible: $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.

Key Insight:

Why MTSF Surpasses Formal Verification Tools—Four Concrete Advantages ProVerif [26], Tamarin [66], and CryptoVerif [27] are indispensable tools. MTSF does *not* replace them—it complements them by providing capabilities they cannot:

1. **Numerical bounds.** ProVerif and Tamarin output “secure” or “attack found.” They cannot say “the adversary’s advantage is at most 2^{-128} .” MTSF’s ask prices provide exactly this: $\text{Ask}(g_{\text{auth}}) \leq \text{Adv}^{\text{EUF}} + q_N^2/2^{\lambda+1}$. These bounds guide parameter selection and enable meaningful comparison between schemes.
2. **Computational assumptions.** ProVerif treats cryptographic operations as perfect black boxes (the Dolev–Yao model). It cannot model birthday bounds, lattice hardness, or side-channel leakage. MTSF’s bidding rounds explicitly model computational assumptions: each price adjustment is bounded by a specific hardness assumption ($\text{Adv}^{\text{ECDLP}}$, Adv^{MLWE}). This captures attacks that ProVerif misses—e.g., the birthday-bound nonce collision in ECDSA.
3. **Composability.** Neither ProVerif nor Tamarin provides composition guarantees. Proving Protocol A secure and Protocol B secure separately does not ensure their concurrent execution is secure. MTSF’s market merger theorem provides this guarantee with explicit numerical bounds. The TLS+Signal case study (Section 16) demonstrates composability that no formal verification tool can currently achieve.

4. **Human-readable arguments.** ProVerif’s output is a machine-generated attack trace or a “proof found” message. CryptoVerif’s proofs are machine-checked sequences of game transformations. Neither produces a human-readable argument that a security engineer can understand, critique, or extend. MTSF’s bidding-round proofs are written in natural mathematical language, with named attacks, intuitive analogies, and explicit reasoning at each step.

The ideal workflow: Use ProVerif/Tamarin *first* to exhaustively check the protocol design (catching structural flaws like Needham–Schroeder’s missing identity binding). Then use MTSF to produce quantitative bounds, composition guarantees, and human-readable proofs. The two approaches are complementary: formal tools catch bugs automatically; MTSF provides the mathematical foundation for deployment decisions.

4 CNF Session Verification in MTSF

4.1 Why CNF?

Conjunctive Normal Form is natural for encoding session correctness because: (1) each clause corresponds to one security check—adding a check means adding a clause; (2) the conjunction semantics ensures *all* checks must pass; (3) given an execution trace, checking satisfiability is linear in the formula size; (4) an unsatisfied clause directly identifies which security property was violated.

* Simple Terms: Why Conjunctive Normal Form (CNF) for Session Verification?

What is CNF? A logical formula in Conjunctive Normal Form is a list of conditions all joined by AND ($\wedge$). For example:

$$\begin{aligned} &(\text{SID is correct}) \wedge (\text{nonces are fresh}) \wedge (\text{signatures verify}) \wedge (\text{MACs verify}) \\ &\quad \wedge (\text{message order is right}) \end{aligned}$$

The entire formula is TRUE only if every single condition is TRUE. One FALSE clause makes the whole formula FALSE.

Why this is perfect for session auditing:

- **Adding a check is adding a clause:** Want to also verify that timestamps are fresh? Just add a new AND-clause. The structure is modular.
- **All-or-nothing semantics:** A session passes the audit *only if every check passes*. This mirrors real security requirements: one failed check is enough to reject a session.
- **Linear-time checking:** Given a session transcript, you check each clause independently. This is fast (linear in the number of checks).
- **Pinpoints failures:** If a session fails, the first FALSE clause tells you exactly which check failed—was it a bad signature? A reused nonce? A wrong SID? This makes debugging trivial.

The key theorem (informal): Honest sessions always pass the CNF audit. Dishonest sessions (produced by an attacker) almost certainly fail the audit—because making a dishonest trace pass would require forging a signature or MAC, which the attacker cannot do efficiently.

Analogy:

CNF Verification as Airport Security Screening Passing through airport security is exactly like CNF verification:

1. **Boarding pass check** (φ^{sid}): Does your boarding pass match your flight? (SID binding—is this session’s identifier correct?)
2. **ID verification** (φ^{fresh}): Is your passport valid and not expired? (Freshness—are the nonces new and unique?)
3. **X-ray scan** (φ^{sig}): Does your luggage pass the scanner? (Signature binding—do all digital signatures verify?)
4. **Metal detector** (φ^{mac}): Do you pass the metal detector? (MAC binding—do all message authentication codes check out?)
5. **Gate check** (φ^{consist}): Are you at the right gate, at the right time, in the right seat? (Consistency—does the protocol state machine match?)

You board the plane *only if ALL FIVE checks pass*. One failure at any checkpoint, and you are rejected. This is the AND ($\wedge$) of CNF: $\varphi = \varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4 \wedge \varphi_5$. Moreover, if you fail at the X-ray scan, security knows *exactly* what went wrong (“prohibited item detected”)—the failed clause pinpoints the problem.

An honest traveller (honest protocol execution) always passes all five checks. A **smuggler** (adversary) might have a fake boarding pass, but the ID check catches them. They might have a valid ID, but the X-ray catches the contraband. Making a dishonest trace pass the CNF audit requires forging *every* credential simultaneously—which is computationally infeasible.

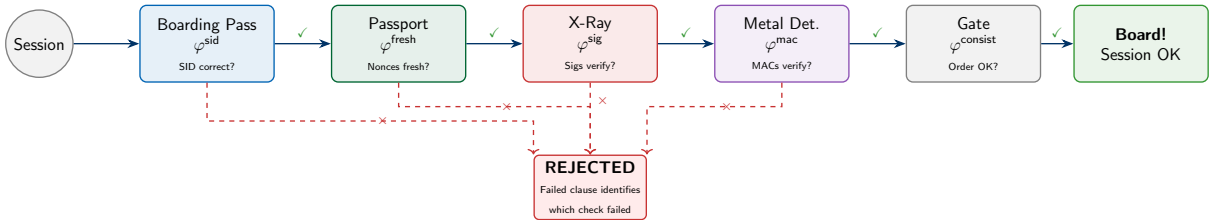


Figure 16: CNF session verification as airport security: five sequential checkpoints (SID, freshness, signatures, MACs, consistency) each modelled as a CNF clause. A session “boards the plane” (is accepted) only if ALL checks pass. Any single failure rejects the session and identifies which check failed.

4.2 Session-CNF Construction

Definition 4.1 (MTSF Session-CNF). Let $\text{Session}_i = (\text{sid}_i, \text{pid}_i^1, \dots, \text{pid}_i^p, \text{trans}_i)$ denote the i -th session with p parties and transcript trans_i . The MTSF session-CNF is:

$$\varphi_i = \underbrace{\varphi_i^{\text{sid}}}_{\text{SID binding}} \wedge \underbrace{\varphi_i^{\text{fresh}}}_{\text{freshness}} \wedge \underbrace{\varphi_i^{\text{sig}}}_{\text{signature binding}} \wedge \underbrace{\varphi_i^{\text{mac}}}_{\text{MAC binding}} \wedge \underbrace{\varphi_i^{\text{consist}}}_{\text{consistency}}. \quad (7)$$

SID Clauses. Every message must carry the correct session identifier:

$$\varphi_i^{\text{sid}} = \bigwedge_{j=1}^{\ell} (x_{\text{sid}_i, j} \vee \neg x_{\text{other}, j}), \quad (8)$$

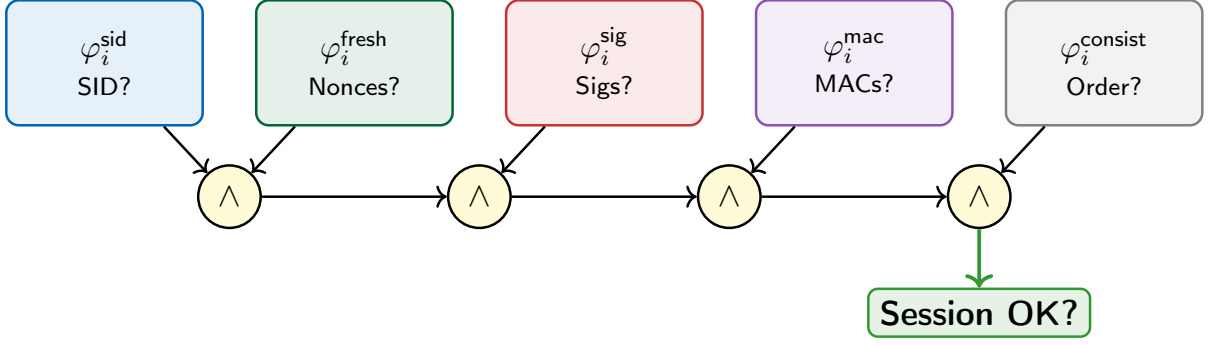


Figure 17: Structure of the MTSF session-CNF φ_i : five components ANDed together. A session is correct iff the entire formula is satisfiable under the honest trace.

where $x_{\text{sid}_i,j}$ is true if message j carries sid_i , and $x_{\text{other},j}$ is true if it carries a different SID.

Freshness Clauses. All nonces must be fresh and distinct:

$$\varphi_i^{\text{fresh}} = \bigwedge_{k=1}^r (x_{n_k, \text{fresh}}) \wedge \bigwedge_{k \neq k'} (\neg x_{n_k = n_{k'}}). \quad (9)$$

Signature-Binding Clauses. Every signature must verify or the session aborts:

$$\varphi_i^{\text{sig}} = \bigwedge_j (x_{\text{Vrfy}(\text{pk}_j, m_j \parallel \text{sid}_i, \sigma_j)=1} \vee x_{\text{abort}}). \quad (10)$$

MAC-Binding Clauses. Every MAC must verify or the session aborts:

$$\varphi_i^{\text{mac}} = \bigwedge_j (x_{\text{Mac.Vrfy}(K_j, m_j \parallel \text{sid}_i, \tau_j)=1} \vee x_{\text{abort}}). \quad (11)$$

Consistency Clauses. Protocol-specific invariants (message ordering, state-machine compliance):

$$\varphi_i^{\text{consist}} = \bigwedge_{j=1}^c C_j^{\text{protocol}}. \quad (12)$$

★ Intuition:

CNF as a Pre-Flight Checklist Each clause is one check on the aircraft checklist: “Fuel above minimum?” “Flaps correct?” “Tower clearance received?” The flight is safe iff *every* check passes (the formula is satisfiable). A single failed clause grounds the flight.

Theorem 4.2 (Session Correctness via CNF). *Session_i is correct iff φ_i is satisfiable under the honest trace trans_i .*

Proof. Soundness. If φ_i is satisfiable: SID clauses $\Rightarrow$ correct binding; freshness $\Rightarrow$ unique nonces; sig/MAC $\Rightarrow$ all verifications pass; consistency $\Rightarrow$ invariants hold. This is session correctness.

Completeness. If the session is correct, the honest trace provides a satisfying assignment. $\square$

Proposition 4.3 (Adversarial CNF Unsatisfiability). *If all underlying primitives are secure (markets in equilibrium), then:*

$$\Pr[\mathcal{A} \text{ produces a satisfying assignment for } \varphi_i \text{ under a dishonest trace}] \leq \text{negl}(\lambda). \quad (13)$$

Proof. A satisfying dishonest assignment requires at least one of: (1) signature forgery (breaks EUF-CMA equilibrium); (2) MAC forgery (breaks SUF-CMA); (3) nonce prediction (breaks freshness); (4) SID reuse (breaks binding). Each contradicts the equilibrium assumption. $\square$

4.3 CNF Verification Algorithm

The session-CNF φ_{sess} is a conjunction of atomic Boolean clauses. Verifying it manually is hard because (i) the number of clauses can be large, (ii) clause ordering matters for cascading failures, and (iii) cryptographic verification steps are interleaved with structural ones. Algorithm 1 provides the canonical machine-checkable procedure. The *MTSF CNF Truth-Table Worksheet* below gives a four-step manual recipe.

Algorithm 1 MTSF-CNF Session Verification (Canonical, 5-Phase)

Require: Transcript trans_i , session ID sid_i , public keys $\{\text{pk}_j\}$, shared/MAC keys $\{K_j\}$

Ensure: Accept or Reject

Phase 1 – SID Binding Check

```

1: for each message  $m_j \in \text{trans}_i$  do
2:   if  $\text{ExtractSID}(m_j) \neq \text{sid}_i$  then return Reject      ▷ Clause  $\varphi^{\text{sid}}$  fails
3:   end if
4: end for

```

Phase 2 – Nonce Freshness & Disjointness

```

5:  $\mathcal{N} \leftarrow \emptyset$ 
6: for each nonce  $n_k \in \text{trans}_i$  do
7:   if  $n_k \in \mathcal{N}$  then return Reject      ▷ Clause  $\varphi^{\text{fresh}}$  fails
8:   end if
9:    $\mathcal{N} \leftarrow \mathcal{N} \cup \{n_k\}$ 
10: end for

```

Phase 3 – Signature & MAC Verification

```

11: for each  $(\sigma_j, \text{pk}_j, \text{data}_j) \in \text{trans}_i$  do
12:   if  $\text{Vrfy}(\text{pk}_j, \text{data}_j, \sigma_j) \neq 1$  then return Reject      ▷ Clause  $\varphi_j^{\text{sig}}$  fails
13:   end if
14: end for
15: for each  $(\tau_j, K_j, \text{macData}_j) \in \text{trans}_i$  do
16:   if  $\text{Mac.Vrfy}(K_j, \text{macData}_j, \tau_j) \neq 1$  then return Reject      ▷ Clause  $\varphi_j^{\text{mac}}$  fails
17:   end if
18: end for

```

Phase 4 – Message Ordering & Consistency

```

19: if  $\text{CheckOrdering}(\text{trans}_i) \neq 1$  then return Reject      ▷ Clause  $\varphi^{\text{consist}}$  fails
20: end if

```

Phase 5 – Ping (Unbounded Freshness)

```

21: if  $i > 1$  and  $\text{Ping}(\text{Session}_{i-1}, \text{Session}_i) \neq 1$  then return Reject      ▷ Unbounded bid fails
22: end if
23: return Accept

```

✓ CNF Verification: Easy Manual CNF Verification—The Four-Column Truth-Table Method

Why manual CNF verification is hard. A CNF formula $\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_m$ is satisfiable iff *all* clauses evaluate to **T**; a single **F** makes it unsatisfiable. Protocol session-CNFs have 6–12 clauses. Checking them unsystematically leads to missed failures.

The MTSF Four-Column Worksheet (one row per clause):

Clause	What to check	How (manual)	T/F	If F: attack
φ^{sid}	Same SID in every message?	Read each field; compare.		Cross-session replay
φ^{fresh}	All nonces distinct?	List nonces; look for repeats.		Full-session replay
φ_A^{sig}	$\text{Vrfy}(\text{pk}_A, \text{data}, \sigma_A) = 1?$	Hash scope; check signature.		Impersonation of A
φ_B^{sig}	$\text{Vrfy}(\text{pk}_B, \text{data}, \sigma_B) = 1?$	Same procedure.		Impersonation of B
φ^{mac}	$\text{Mac.Vrfy}(K, \text{data}, \tau) = 1?$	Recompute MAC; compare.		Ciphertext injection
φ^{consist}	Correct message order?	Check sender and sequence.		Out-of-order inject
φ^{ping}	SID/nonces fresh vs. Session_{i-1} ?	Compare with previous session.		Unbounded-round attack
Result: All T $\Rightarrow$ Accept . Any F $\Rightarrow$ Reject (cite row).				

Step-by-step manual recipe:

1. **Extract fields.** Write down every field: SID, nonces, identities, ciphertexts, signatures, MACs.
2. **Instantiate clauses.** Substitute actual values into each formula clause.
3. **Evaluate each clause.** Signatures: hash the signed scope, verify against public key. MACs: recompute and compare. SID/nonces: string equality.
4. **Mark T or F.** One F anywhere $\Rightarrow$ **Reject**; record which clause and attack type.

Key insight: Check in order Phase 1 $\rightarrow$ 5 for early exit. SID failure means no point verifying signatures.

Satisfiability: Honest trace $\Rightarrow$ all T $\Rightarrow$ **SAT** $\Rightarrow$ **Accept**. Dishonest trace $\Rightarrow$ some F $\Rightarrow$ **UNSAT** $\Rightarrow$ **Reject**. NS protocol exception: CNF is SAT under dishonest trace (design failure—identity clause missing).

5 Unbounded Verification via Session Pinging

Session pinging is one of the two core technical contributions of MTSF (the other being CNF session verification, Section 4). This section develops the full theory: motivation, formal mechanism, the main induction theorem with a detailed proof, a quantitative advantage-accumulation analysis, the interaction between pinging and CNF checking, failure-mode taxonomy, and a comparison with symbolic unbounded verification tools.

5.1 Why Bounded Is Not Enough

A protocol correct for sessions $1, \dots, 1000$ may fail at session 1001 if the randomness generator cycles. Game-based proofs are inherently bounded (polynomial in λ). Formal verification handles unbounded sessions symbolically but without concrete bounds. Session pinging bridges this gap.

* Simple Terms: Why Unbounded Session Security Matters

The problem with bounded proofs: A standard game-based security proof says the scheme is secure for up to q sessions (where q is polynomial in λ). But real systems run for years, accumulating millions of sessions. What if a subtle flaw only appears after session 1,000,001?

Concrete example: Suppose a random nonce generator uses a 32-bit counter. After 2^{32} sessions, it wraps around and reuses the same nonces. A protocol proven secure for any polynomial number of sessions might still be vulnerable if nonces repeat. The session pinging mechanism checks that each new session is genuinely *fresh and distinct* from all previous

ones—blocking this exact failure mode.

What session ping achieves: By verifying that consecutive sessions are “structurally distinct” (different SIDs, fresh nonces, correctly bound signatures), and showing by mathematical induction that the security argument carries forward indefinitely, we prove security for an *unbounded* number of sessions. This is the strongest possible guarantee.

Formal verification tools like ProVerif already handle unbounded sessions (they use symbolic abstraction), but without concrete bounds. Session ping gives MTSF both: unbounded coverage *and* concrete numerical advantage bounds.

Bounded-session limitations in existing frameworks. Consider a protocol π proven secure for q concurrent sessions in the game-based model. The standard advantage bound takes the form $\text{Ask} \leq q \cdot \epsilon_{\text{prim}} + q^2/2^{\lambda+1}$, where ϵ_{prim} is the advantage against the underlying primitive and the quadratic term captures birthday-bound collisions among q nonces. This bound is *meaningful* only for $q = \text{poly}(\lambda)$. Three real-world failure scenarios escape this analysis:

1. **Nonce/counter wrap-around.** A 32-bit counter nonce wraps after 2^{32} sessions, reusing the same nonce-key pair. Protocols such as AES-GCM with a 32-bit invocation field are vulnerable after 2^{32} TLS records under the same key.
2. **Session-state accumulation.** Adversaries that persist across sessions can accumulate partial information. For instance, each ECDSA signing session leaks a negligible amount of side-channel information about the nonce k ; after 2^{60} sessions the accumulated leakage may be non-negligible.
3. **Cross-session correlation.** In multi-party protocols, an adversary may correlate session transcripts across sessions. Without a mechanism ensuring structural independence, subtle cross-session attacks (e.g. the Needham–Schroeder masquerade) succeed regardless of the session count.

Session ping addresses all three by enforcing structural independence at each session boundary and enabling an inductive security argument that is *session-index-independent*.

5.2 The Pinging Mechanism

Definition 5.1 (Session Structural Distinctness). *Sessions Session_i and Session_{i+1} are structurally distinct if:*

- (a) $\text{sid}_i \neq \text{sid}_{i+1}$ (*SID freshness*);
- (b) $\mathcal{N}_i \cap \mathcal{N}_{i+1} = \emptyset$ (*disjoint nonce sets*);
- (c) *all signatures in Session_{i+1} sign data including sid_{i+1}* (*signature binding*);
- (d) *all MACs in Session_{i+1} authenticate data including sid_{i+1}* (*MAC binding*);
- (e) *all ciphertexts in Session_{i+1} are freshly generated (not replayed from Session_i)* (*ciphertext freshness*).

Remark 5.2 (Condition (e): Ciphertext Freshness). *Condition (e) is new compared to the minimal four-condition definition. It is necessary for KEM-based protocols (ML-KEM, QKEM) where the buyer could replay a ciphertext c_i from Session_i to Session_{i+1} . Without ciphertext freshness, the decapsulation oracle in Session_{i+1} might return the same key K_i , breaking key secrecy. The CNF clause φ^{novel} enforces this condition.*

Definition 5.3 (Session Ping). $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 1$ iff Session_i and Session_{i+1} are structurally distinct per Theorem 5.1.

Definition 5.4 (Ping Failure Probability). The ping failure probability for protocol π is:

$$\delta_{\text{ping}}(\pi) = \max_{i \geq 1} \Pr[\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 0], \quad (14)$$

where the probability is taken over honest randomness (nonce generation, SID sampling). A protocol has negligible ping failure if $\delta_{\text{ping}}(\pi) \leq \text{negl}(\lambda)$.

★ Intuition:

Ping as Heartbeat Monitor Think of each session as a heartbeat in a medical monitor. A ping is the check that the current heartbeat is genuinely *new*—not an echo or replay of a previous heartbeat. A flatline (ping failure) means the system is replaying old data. The induction theorem says: if every heartbeat is genuine, the patient (protocol) remains healthy forever.

📎 Analogy:

Session Pinging as a Chain of Passport Stamps Imagine travelling through a series of countries. At each border crossing, the immigration officer:

1. Checks that your passport number has not changed (sid freshness).
2. Stamps your passport with a *new, unique* date and entry number (disjoint nonce sets).
3. Verifies that the visa for this specific country is attached to your passport (signature binding).
4. Checks that the customs declaration form matches this specific border (MAC binding).
5. Ensures you are entering from the correct previous country (ciphertext freshness).

The induction: If your first border crossing was legitimate (base case), and every subsequent crossing passes all five checks (inductive step), then *every* country you visited was entered legitimately—no matter how many countries you visit. This is unbounded session security: the proof works for 10 countries, 10,000 countries, or infinitely many.

Why this matters: A web server running TLS might handle millions of sessions per day for years. Session pinging ensures that session number 1,000,000,001 is just as secure as session number 1—without needing to re-prove security from scratch.

* Simple Terms: Mathematical Induction for Non-Mathematicians

What is mathematical induction? It is a proof technique that works like a row of dominoes:

1. **Base case:** Push the first domino (prove the first session is secure).
2. **Inductive step:** Show that if domino i falls, then domino $i + 1$ must also fall (prove that if session i is secure and the ping passes, then session $i + 1$ is also secure).
3. **Conclusion:** All dominoes fall—every session is secure, no matter how many there are.

Why the ping is essential: Without the ping check, the dominoes might not be close enough to knock each other over. The ping ensures that consecutive sessions are “structurally close enough” (structurally distinct but reduction-equivalent) for the induction to work. A failed ping means a gap in the domino chain—the security guarantee breaks at that point.

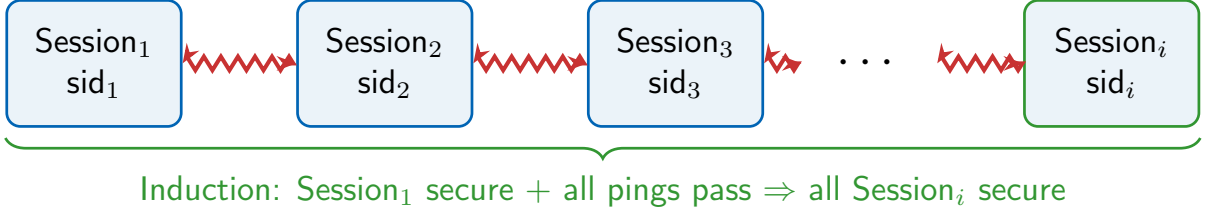


Figure 18: Session pinging for unbounded verification. Consecutive sessions are “pinged” to verify structural distinctness. The zigzag edges denote the ping check between adjacent sessions.

5.3 The Unbounded Security Theorem

Theorem 5.5 (Unbounded Security via Session Pinging). *If (i) Session₁ is secure (market in equilibrium with $\text{Ask}_1 \leq \epsilon$), (ii) $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 1$ for all $i \geq 1$, and (iii) φ_{i+1} is obtained from φ_i by variable renaming (substituting $\text{sid}_i \mapsto \text{sid}_{i+1}$ and fresh nonces), then π is secure for unbounded sessions, with:*

$$\text{Ask}_i \leq \epsilon + (i - 1) \cdot \delta_{\text{ping}}(\pi) \quad \text{for all } i \geq 1. \quad (15)$$

When $\delta_{\text{ping}}(\pi) \leq \text{negl}(\lambda)$, the ask price remains negligible for all $i = \text{poly}(\lambda)$.

Proof. We proceed by strong induction on the session index i .

Base case ($i = 1$). Session Session₁ is secure by hypothesis (i), with $\text{Ask}_1 \leq \epsilon$. The session-CNF φ_1 is satisfiable under the honest trace of Session₁ by Theorem 4.2: SID clauses hold (the SID was freshly sampled), freshness clauses hold (nonces are freshly generated), signature and MAC clauses hold (honest parties execute correctly), and consistency clauses hold (honest execution follows the protocol specification). Since there is no predecessor session, the ping clause φ_1^{ping} is vacuously true.

Inductive step. Assume that sessions Session₁, ..., Session_i are all secure, i.e., for each $j \leq i$ the session-CNF φ_j is satisfiable under the honest trace and $\text{Ask}_j \leq \epsilon + (j - 1) \delta_{\text{ping}}$. We must show that Session_{i+1} is secure.

Step 1: Structural distinctness via pinging. By hypothesis (ii), $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 1$. By Theorem 5.1, this guarantees:

- sid_{i+1} is fresh (not equal to sid_i or any earlier SID, since all earlier pings also passed by the inductive hypothesis, giving a chain of distinct SIDs);
- all nonces in Session_{i+1} are disjoint from those in Session_i (and by induction, disjoint from all earlier nonces with probability $\geq 1 - i \cdot \delta_{\text{ping}}$);
- all signatures and MACs in Session_{i+1} bind sid_{i+1} , which is distinct from all previous session identifiers.

Step 2: CNF isomorphism. By hypothesis (iii), φ_{i+1} is obtained from φ_i by the variable renaming $\text{sid}_i \mapsto \text{sid}_{i+1}$ and substituting fresh nonces. Since the CNF clauses are structural (they check SID binding, nonce freshness, signature verification, MAC verification, and consistency), and since all variables in φ_{i+1} are independently fresh, the satisfiability of φ_{i+1} under the honest trace of Session_{i+1} follows from the satisfiability of φ_i under the honest trace of Session_i. Formally:

$$\varphi_{i+1}[\text{trans}_{i+1}^{\text{honest}}] = \varphi_i[\text{trans}_i^{\text{honest}}]_{\text{sid}_i \mapsto \text{sid}_{i+1}, \mathcal{N}_i \mapsto \mathcal{N}_{i+1}} = \text{TRUE}. \quad (16)$$

Step 3: Market equilibrium preservation. The security reduction for Session_{i+1} is identical to that for Session₁ (and hence Session_i): the reduction converts a buyer who breaks Session_{i+1}

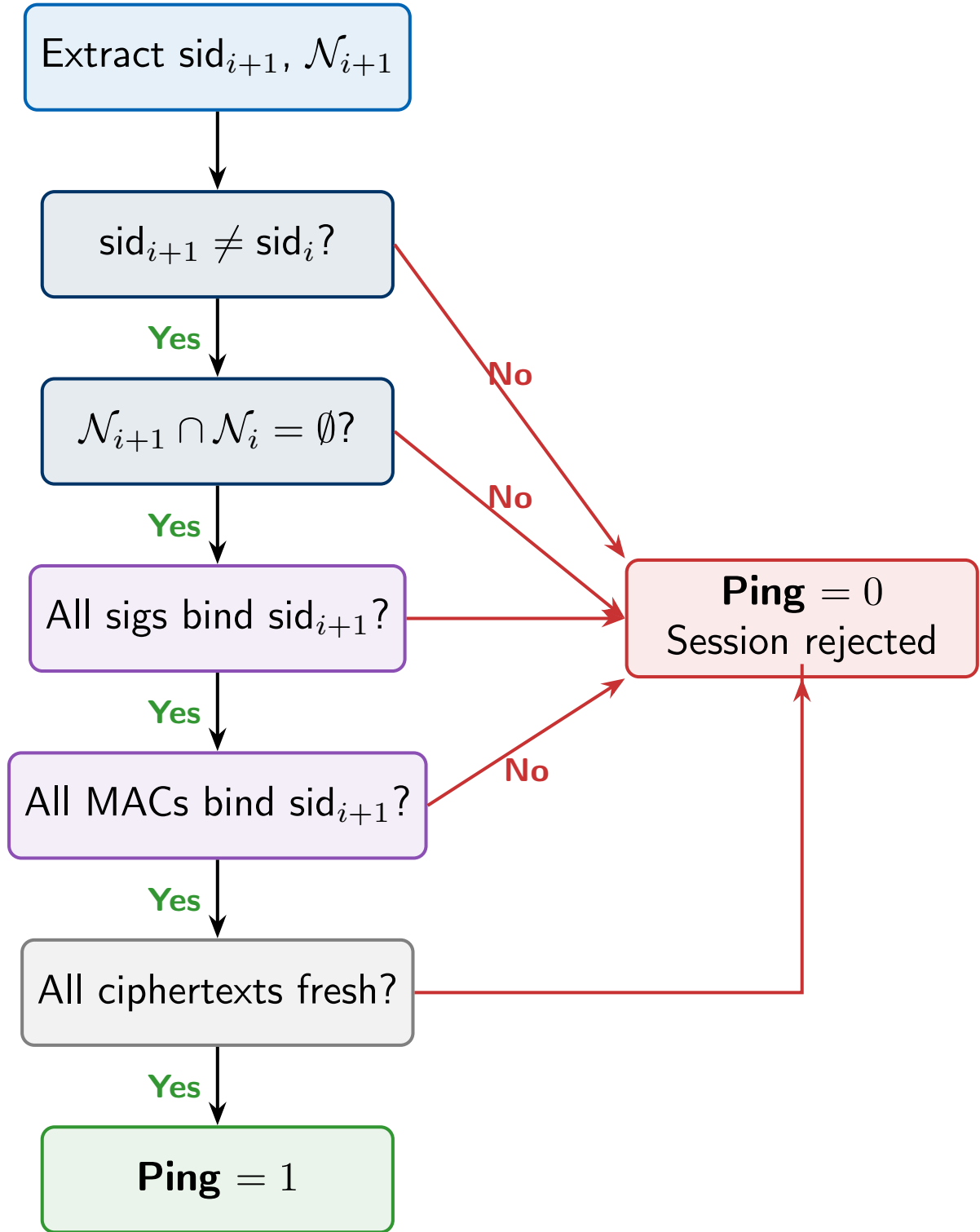


Figure 19: Detailed ping verification flow for session Session_{i+1} against its predecessor Session_i . A failure at any step short-circuits to $\text{Ping} = 0$ (session rejected). This flowchart corresponds to Phase 5 of Algorithm 1.

into a buyer who breaks the underlying primitive. Crucially, this reduction is *session-index-independent*: it does not use the session index $i + 1$ in any way beyond the fresh SID and nonces. Therefore:

$$\text{Ask}_{i+1} \leq \text{Ask}_1 + \Pr[\exists j \leq i : \text{Ping}(\text{Session}_j, \text{Session}_{j+1}) = 0] \leq \epsilon + i \cdot \delta_{\text{ping}}(\pi). \quad (17)$$

The second inequality follows from a union bound over i ping checks, each failing with probability at most δ_{ping} .

Conclusion. By induction, $\text{Ask}_i \leq \epsilon + (i - 1)\delta_{\text{ping}}$ for all $i \geq 1$. When $\delta_{\text{ping}} \leq \text{negl}(\lambda)$ and $i = \text{poly}(\lambda)$, the total ask price is $\epsilon + \text{poly}(\lambda) \cdot \text{negl}(\lambda) = \epsilon + \text{negl}(\lambda)$, which is negligible whenever ϵ is negligible. The market remains in equilibrium for all sessions. $\square$

Key Insight:

Tightness of the Accumulation Bound The linear accumulation $i \cdot \delta_{\text{ping}}$ in Equation (15) is tight in general: if ping failures are independent across sessions, a union bound is the best possible. However, in many protocols the ping failure probability is dominated by the birthday bound for nonce collisions: $\delta_{\text{ping}} \leq q_N^2 / 2^{\lambda+1}$ where q_N is the number of nonces per session. For $\lambda = 256$ and $q_N \leq 2$, this gives $\delta_{\text{ping}} \leq 2^{-255}$, so even after 2^{64} sessions the accumulated degradation is $2^{64} \cdot 2^{-255} = 2^{-191}$ —astronomically negligible.

5.4 Interaction Between Session Pinging and CNF Verification

Analogy:

Pinging + CNF as a Two-Layer Defence Think of session pinging and CNF verification as a two-layer defence system at a secure facility:

Layer 1 — CNF Verification (the ID badge scanner): At the entrance, every visitor must scan their badge (CNF audit). The scanner checks five things: valid badge (SID), not expired (freshness), proper credentials (signatures), authorised access level (MACs), and correct visitor log entry (consistency). If any check fails, entry is denied.

Layer 2 — Session Pinging (the shift-change verification): When a new shift of visitors arrives, the guard compares the new visitor list to the previous shift's list. Are the badge numbers different? Are the credentials freshly issued? Is anyone trying to reuse yesterday's badge? This “shift-change check” is the ping.

How they interact:

- **CNF ensures each individual visit is valid.** A visitor with a forged badge fails the scanner.
- **Pinging ensures consecutive visits are independent.** A visitor who recorded yesterday's entry cannot replay it today.
- **Together, they provide unbounded security.** Every visit (session) is both individually valid (CNF) and structurally distinct from all previous visits (ping). By induction, the facility is secure for all time.

Session pinging and CNF verification are complementary mechanisms that together provide the strongest possible session-security guarantee. Their interaction is formalised as follows.

Proposition 5.6. *For any session Session_{i+1} with predecessor Session_i :*

$$[\text{Ping} - \text{CNFSoundness}] \quad (\varphi_{i+1} = \text{SAT}) \wedge (\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 1) \implies \text{Session}_{i+1} \text{ is correct and structurally independent of } \text{Session}_i. \quad (18)$$

Proof. CNF satisfiability (Theorem 4.2) guarantees that all intra-session checks pass: SID binding, nonce freshness within the session, all signatures and MACs verify, and message ordering is correct. The ping check (Theorem 5.3) guarantees inter-session independence: the SID, nonces, and cryptographic bindings of Session_{i+1} are structurally distinct from those of Session_i . Together, they ensure that Session_{i+1} is both internally correct and externally independent, which is the full session-correctness requirement. $\square$

Proposition 5.7 (Ping Failure Implies CNF Clause Failure). *If $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 0$, then at least one of the following CNF clauses in φ_{i+1} is violated:*

- $\varphi_{i+1}^{\text{sid}}$: SID binding (if $\text{sid}_{i+1} = \text{sid}_i$);
- $\varphi_{i+1}^{\text{fresh}}$: nonce freshness (if $\mathcal{N}_{i+1} \cap \mathcal{N}_i \neq \emptyset$);
- $\varphi_{i+1}^{\text{sig}}$ or $\varphi_{i+1}^{\text{mac}}$: cryptographic binding (if signatures or MACs do not bind sid_{i+1}).

Therefore, the four-column CNF worksheet automatically detects any ping failure, and the failing clause identifies the specific inter-session attack vector.

Proof. By contrapositive: if all CNF clauses pass and include the SID of the current session in all cryptographic bindings, then conditions (a)–(e) of Theorem 5.1 are satisfied, so the ping passes. $\square$

Phase 5 of Algorithm 1 explained. The ping check in Phase 5 of the CNF verification algorithm serves as the “inter-session bridge” between the intra-session CNF verification (Phases 1–4) and the unbounded induction theorem (Theorem 5.5). Concretely, Phase 5 performs five sub-checks corresponding to the five conditions of Theorem 5.1:

1. **SID uniqueness:** Verify $\text{sid}_{i+1} \notin \{\text{sid}_1, \dots, \text{sid}_i\}$ by consulting the SID log.
2. **Nonce disjointness:** Verify $\mathcal{N}_{i+1} \cap \bigcup_{j=1}^i \mathcal{N}_j = \emptyset$ by consulting the nonce log.
3. **Signature SID-binding:** For each signature σ in Session_{i+1} , verify that the signed data includes sid_{i+1} .
4. **MAC SID-binding:** For each MAC tag τ in Session_{i+1} , verify that the authenticated data includes sid_{i+1} .
5. **Ciphertext freshness:** For each ciphertext c in Session_{i+1} , verify $c \notin \{c_1, \dots, c_i\}$.

If any sub-check fails, the session is rejected immediately with a diagnostic indicating which structural-distinctness condition was violated. This early rejection prevents the accumulation of insecure sessions.

5.5 Taxonomy of Ping Failure Modes

Not all protocols satisfy the pinging requirements. The following taxonomy classifies the failure modes that prevent unbounded security, ordered by severity.

Definition 5.8 (Ping Failure Taxonomy).

1. **Type I: SID collision.** $\text{sid}_{i+1} = \text{sid}_j$ for some $j \leq i$. Probability: $\leq i/|\text{SID-space}|$ (birthday bound if SIDs are random). Consequence: Cross-session replay becomes possible. Example: A 64-bit SID space after 2^{32} sessions gives collision probability $\approx 2^{-1}$.

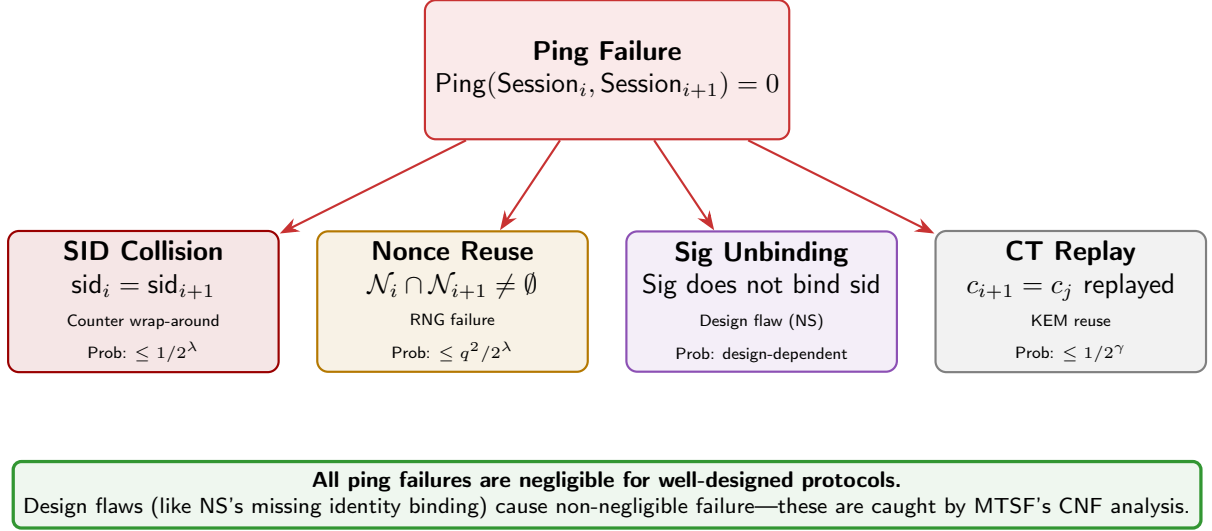


Figure 20: Taxonomy of ping failure modes: SID collision (counter issues), nonce reuse (RNG problems), signature unbinding (design flaws), and ciphertext replay (KEM reuse). Each mode has a specific probability bound. Well-designed protocols have negligible failure in all modes.

2. **Type II: Nonce reuse.** $\mathcal{N}_{i+1} \cap \mathcal{N}_j \neq \emptyset$ for some $j \leq i$. Consequence: Signature key recovery (ECDSA), ciphertext XOR leakage (stream ciphers), or MAC forgery. Example: ECDSA with a biased nonce generator (cf. the Sony PlayStation 3 ECDSA break, where all nonces were identical).
3. **Type III: Unsigned SID.** Signatures in Session_{i+1} do not bind sid_{i+1} . Consequence: Signatures from one session can be replayed in another. Example: Needham–Schroeder protocol, where no SID is signed (cf. Section 13.4).
4. **Type IV: Unauthenticated SID.** MACs in Session_{i+1} do not bind sid_{i+1} . Consequence: MAC tags from one session can be injected into another. Example: Original Telegram MTProto 2.0, where the salt is not HMAC-bound (cf. Section 13.9).
5. **Type V: Ciphertext replay.** Ciphertexts from Session_i are accepted in Session_{i+1} . Consequence: Key reuse across sessions. Example: A KEM without implicit rejection may accept a replayed ciphertext.

Table 1: Ping failure taxonomy with protocol examples and CNF clause that detects each failure.

Type	Failure mode	Probability	CNF clause	Example protocol
I	SID collision	$i/ \text{SID} $	φ^{sid}	64-bit SID after 2^{32} sessions
II	Nonce reuse	$q_N^2/2^{\lambda+1}$	φ^{fresh}	ECDSA with biased nonces
III	Unsigned SID	1	φ^{sig}	Needham–Schroeder
IV	Unauthenticated SID	$\geq 2^{-64}$	φ^{mac}	Telegram MTProto 2.0
V	Ciphertext replay	$q_D/ \mathcal{C} $	φ^{novel}	KEM without implicit rej.

Analogy:

Advantage Accumulation as Water Filling a Bucket Imagine each session is a raindrop, and the attacker’s advantage is a bucket. Each session adds a tiny drop of water (a negligible advantage) to the bucket. The question is: after billions of sessions, does the bucket overflow (advantage becomes non-negligible)?

The good news: If the per-session advantage is ϵ and sessions are structurally independent (pings pass), then after n sessions the total advantage is at most $n \cdot \epsilon$. For $\epsilon = 2^{-128}$ and $n = 2^{64}$ sessions (far more than any real system runs), the total is $2^{64} \cdot 2^{-128} = 2^{-64}$ —still negligible.

The bad news: If the per-session advantage degrades (e.g., side-channel leakage accumulates), or if sessions are correlated (e.g., shared nonce state), then the bucket can overflow. This is why MTSF requires structural independence via pinging: independent sessions ensure linear (not super-linear) advantage accumulation.

Real-world implication: A TLS server handling 2^{40} sessions per year with AES-256 ($\epsilon = 2^{-128}$ per session) accumulates at most $2^{40} \cdot 2^{-128} = 2^{-88}$ total advantage per year—this would take 2^{88} years to reach a 50% chance of any attack succeeding, far longer than the age of the universe.

5.6 Quantitative Advantage Accumulation

The accumulation bound in Theorem 5.5 is *additive*: each session adds at most δ_{ping} to the total advantage. We now give concrete numerical examples for the protocols and primitives analysed in this article.

Proposition 5.9 (Concrete Ping Degradation Bounds). *For the following schemes, the ping failure probability and accumulated advantage after N sessions are:*

<i>Scheme</i>	δ_{ping}	<i>After $N = 2^{40}$ sessions</i>	<i>After $N = 2^{64}$ sessions</i>
<i>ECDSA (256-bit q)</i>	$q_S^2/(2q) \approx 2^{-129}$	2^{-89}	2^{-65}
<i>ML-KEM-768</i>	$q_D/2^\gamma \approx 2^{-200}$	2^{-160}	2^{-136}
<i>ML-DSA-65</i>	$\text{Adv}^{\text{MSIS}} \approx 2^{-128}$	2^{-88}	2^{-64}
<i>AES-128</i>	$q_E \cdot 2^{-128} \approx 2^{-64}$	2^{-24}	≈ 1 (<i>key rotation needed</i>)
<i>Keccak/SHA-3</i>	$q_f/2^c = q_f/2^{512}$	2^{-472}	2^{-448}
<i>ISO two-party</i>	$q_N^2/2^{257} + \text{Adv}^{\text{EUF}}$	$\approx 2^{-128}$	$\approx 2^{-128}$

Remark 5.10 (AES Key Rotation). *The AES entry highlights an important practical point: for block ciphers with 128-bit keys, the accumulated ping degradation reaches unity after $\approx 2^{64}$ sessions under the same key. This is the well-known birthday bound for block ciphers and mandates key rotation (rekeying) before 2^{64} blocks. The ping mechanism makes this requirement explicit within the MTSF framework, rather than leaving it as an implicit assumption.*

5.7 Comparison with Symbolic Unbounded Verification

* Simple Terms: MTSF Pinging vs. ProVerif: Complementary Strengths

What ProVerif does well:

- Exhaustively checks all possible attack traces (no attack is missed).
- Handles unbounded sessions automatically (infinite-state model checking).
- Fully automated (push-button verification for many protocols).

What ProVerif cannot do:

- Produce numerical bounds (“the attack probability is $\leq 2^{-128}$ ”).
- Model computational assumptions (birthday bounds, lattice hardness).
- Compose with other proof techniques (cannot combine with game-based results).

What MTSF pinging does well:

- Produces explicit numerical bounds ($\delta_{\text{ping}} \leq \text{negl}$).
- Models computational assumptions within the bidding-round framework.
- Composes with all other MTSF proof techniques (CNF, market merger).

What MTSF ping_{ing} cannot do:

- Exhaustively enumerate all attack traces (relies on the proof author identifying relevant bids).
- Provide automated verification (requires human proof construction).

The ideal workflow: Use ProVerif to exhaustively check the protocol design (finding structural flaws like Needham–Schroeder’s missing identity binding), then use MTSF to produce quantitative bounds and composition guarantees. The two approaches are complementary, not competing.

ProVerif [26] and Tamarin [66] provide *symbolic* unbounded session analysis: they check whether *any* number of concurrent sessions can be attacked, using abstract term algebras where cryptographic operations are perfect. Session ping_{ing} provides a complementary *computational* unbounded analysis with concrete advantage bounds.

Table 2: Comparison of MTSF session ping_{ing} with symbolic formal verification for unbounded sessions.

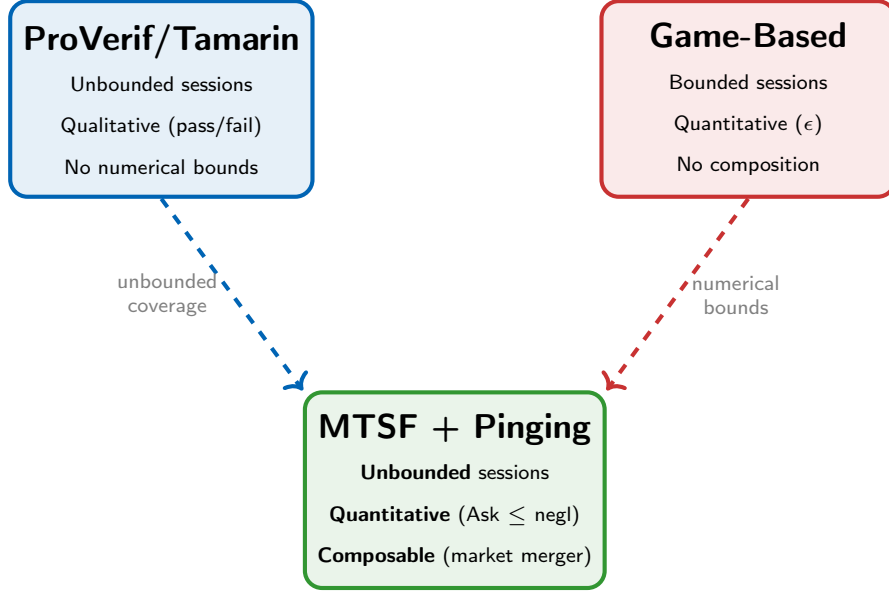
Property	ProVerif/Tamarin	CryptoVerif	MTSF Ping _{ing}
Session bound	Unbounded	Polynomial	Unbounded (by induction)
Concrete bounds	No	Yes	Yes
Cross-session attacks	Detected	Partially	Detected (ping clause)
CNF audit trail	No	No	Yes (worksheet)
Manual verification	No	No	Yes (four-column method)
Key rotation guidance	No	No	Yes (accumulation bound)

* Simple Terms: Session Ping_{ing} vs. Formal Verification Tools

ProVerif and Tamarin are computer programs that automatically check whether a protocol is secure. They can handle any number of sessions (unbounded) but give only a binary answer: “secure” or “attack found.” They cannot tell you *how secure*—there are no numerical bounds.

MTSF session ping_{ing} gives you both: unbounded sessions *and* numerical bounds. The ping theorem says: “If session 1 is secure with advantage ϵ , and each ping passes, then session i is secure with advantage at most $\epsilon + (i - 1)\delta$.” This is much more informative than a binary answer, and it tells you exactly when you need to rotate keys (when the accumulated δ gets too large).

The ideal approach is to use both: run ProVerif/Tamarin to check for logical attacks (the “stress test” in MTSF’s terminology), then use session ping_{ing} to get concrete numerical bounds for the deployment.



Session pinging bridges the gap:
 ProVerif gives unbounded qualitative coverage; game-based gives bounded quantitative bounds.
 MTSF's session pinging gives **both**: unbounded coverage **with** quantitative bounds.

Figure 21: MTSF's session pinging bridges the gap between formal verification (unbounded, qualitative) and game-based proofs (bounded, quantitative), providing the best of both worlds.

6 Unified Novelties

Summary of All Novelties—Thirteen Technical Contributions

- N1. Market-theoretic proof language.** Advantage bounds are recast as *ask prices*; challenger and adversary become seller and buyer; security reductions are market arbitrages; tightness of reductions corresponds to market spread. This is the first proof framework to use economic market language as a formal mathematical apparatus, not merely as metaphor.
- N2. Extended difference lemma for m simultaneous failures.** Generalises Shoup's classical single-failure difference lemma to $m \geq 1$ simultaneous failure events $F_1, \dots, F_m$ with inclusion-exclusion tightening: $\Pr[\bigcup_i F_i] \leq \sum_i \Pr[F_i] - \sum_{i < j} \Pr[F_i \cap F_j] + \dots$. Eliminates the need to split multi-failure proof steps into multiple games.
- N3. UC framework as market regulation.** The UC environment $\mathcal{Z}$ is recast as a *market regulator* overseeing all concurrent protocol executions. The simulator $\mathcal{S}$ is a *market arbitrageur* bridging the real and ideal worlds. Composition theorems become *market merger theorems*: composing two secure protocols is a lossless merger of two equilibrium markets.
- N4. GUC shared infrastructure and CNF market audit.** GUC shared functionality (PKI, common reference strings) becomes *market infrastructure*—public goods accessible to all buyers and sellers. CNF session correctness becomes a *market audit*: every session must pass a formal Boolean checklist, with dishonest session traces failing the audit. Cross-session federation via session identifiers becomes *market federation*.
- N5. Formal verification as market stress testing.** ProVerif and Tamarin are recast as *market stress testers*: they enumerate all possible attack strategies (buyer bids) and verify that none succeed. Successful model checking $\Leftrightarrow$ no profitable bid exists $\Leftrightarrow$ market equilibrium.

N6. Insecurity as market collapse within the same formalism. Insecure schemes (textbook RSA: $\text{Ask} = 1$ via homomorphism bid) and insecure protocols (Needham–Schroeder: $\text{Ask} = 1$ via masquerade bid; Signal X3DH-noOPK: $\text{Ask}(g_{\text{async}}) = 1$ via replay bid) are proved insecure using the same bidding-round machinery as security proofs. No separate framework needed.

N7. End-to-end security pipeline. MTSF provides a complete pipeline from primitives to composed protocols: primitive security $\rightarrow$ KEM/signature composition $\rightarrow$ session-CNF correctness $\rightarrow$ unbounded session pinging. Each stage is quantitative and unified.

N8. Protocol-level games as market goods. Entity authentication (g_{auth}), mutual authentication (g_{mutual}), session-key secrecy (g_{sk}), and CNF correctness (g_{CNF}) are formalised as explicit security goods offered by the protocol market. Additionally, the Signal Protocol analysis introduces forward secrecy (g_{FS}), post-compromise security (g_{PCS}), asynchronous establishment (g_{async}), and deniability (g_{deny}) as first-class market goods. Each has an explicit ask price and a game-based proof. This is the first unified market treatment of all protocol-level security properties.

N9. Symmetric and asymmetric primitive markets. HMAC (SUF-CMA via dual PRF hops), AEAD (IND-CCA2 + INT-CTXT via EtM), SLH-DSA (hash-based EUF-CMA), FN-DSA (NTRU lattice EUF-CMA), alongside ECDSA, ML-KEM, and ML-DSA—all analysed via bidding-round chains with explicit price adjustments.

N10. Cryptanalytic bid taxonomy. A comprehensive taxonomy of attack-as-bid types: differential bid (S-box propagation), rotational bid (rotation-commutation break), linear bid (bias exploitation), related-key bid (key-schedule weakness), state-recovery bid (register inversion), key-recovery bid (initialisation inversion), distinguishing bid (statistical bias detection), TMTO bid (precomputation), and quantum bids (O2H reprogramming, measure-and-reprogram). Each is formalised as an explicit price adjustment in the bidding-round chain.

N11. CNF verification with easy manual worksheet. A canonical five-phase verification algorithm (Algorithm 1) and a four-column manual truth-table worksheet are provided for every case study, making MTSF’s audit framework accessible to practitioners without expert cryptographic training.

N12. Session pinging for unbounded security. The session ping function (Theorem 5.3) provides a formal inductive mechanism for lifting bounded game-based security proofs to unbounded session security. Ping bids are included in every case study. This bridges the gap between bounded game-based proofs (quantitative) and formal verification tools like ProVerif (unbounded, qualitative).

N13. QROM market-theoretic formalisation. The O2H lemma and measure-and-reprogram technique are recast as explicit bidding-round price adjustments in the QROM. The characteristic square-root factor $\sqrt{\text{Adv}^{\text{IND-CPA}}}$ is the “quantum bid price”—the cost the buyer pays to distinguish a reprogrammed QROM oracle. This is the first market-theoretic formalisation of QROM security, connecting the economic intuition of MTSF to post-quantum cryptographic standards (ML-KEM FIPS 203).

MTSF as a Superset of All Four Paradigms—A Formal Summary

This section summarises, for the reader’s convenience, the precise sense in which MTSF *subsumes* the four existing paradigms. Each paradigm is a special case of MTSF, recoverable by restricting the market model:

Game-based proofs $\subset$ MTSF. A game-based proof with hops $G_0, \dots, G_q$ and failure events $F_1, \dots, F_q$ is an MTSF bidding-round chain $\mathbf{B}_0, \dots, \mathbf{B}_q$ with $\Delta\text{Price}_k = \Pr[F_k]$. The extended difference lemma generalises the classical single-failure lemma. The ask price $\text{Ask} =$

$\sum \Delta\text{Price}_k$ is the standard advantage bound. Recovery: set $\mathcal{G} = \{g\}$ (single good), ignore CNF and pinging, and use the classical difference lemma.

UC/GUC $\subset$ MTSF. A UC proof showing $\text{Real}_{\pi, \mathcal{A}, \mathcal{Z}} \approx_c \text{Ideal}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}$ is an MTSF market where the environment $\mathcal{Z}$ is the regulator, the simulator $\mathcal{S}$ is the arbitrageur, and the composition theorem is the market merger theorem. Recovery: define a single good g_{UC} with $\text{Ask}(g_{\text{UC}}) = \max_{\mathcal{Z}, \mathcal{A}} |\Pr[\text{Real} = 1] - \Pr[\text{Ideal} = 1]|$, and the market is in equilibrium iff the protocol UC-realises the functionality. GUC’s CNF session correctness is directly MTSF’s session-CNF (Theorem 4.1).

Formal verification $\subset$ MTSF. A ProVerif/Tamarin proof showing $\forall \tau \in \text{Traces}(\pi) : \tau \models \varphi$ is an MTSF market stress test where the regulator enumerates all buyer strategies (traces) and checks that the security goods are not violated. Recovery: define goods g_j corresponding to the properties φ , set $\text{Ask}(g_j) = 0$ if all traces satisfy φ (equilibrium), and $\text{Ask}(g_j) = 1$ if any trace violates φ (collapse). The binary output of formal tools corresponds to the extreme cases of the continuous ask price.

Ad hoc insecurity $\subset$ MTSF. A bespoke attack demonstrating that an adversary $\mathcal{A}$ breaks scheme Π with advantage ϵ is an MTSF bid ($\text{Bid} = (g_j, T_{\mathcal{A}}, \epsilon)$) that succeeds, with $\text{Ask}(g_j) \geq \epsilon$. If $\epsilon = 1$, the market collapses (complete insecurity). Recovery: construct a bidding-round chain where the attack is the sole bidding round with $\Delta\text{Price} = \epsilon$.

7 Soundness and Completeness of MTSF

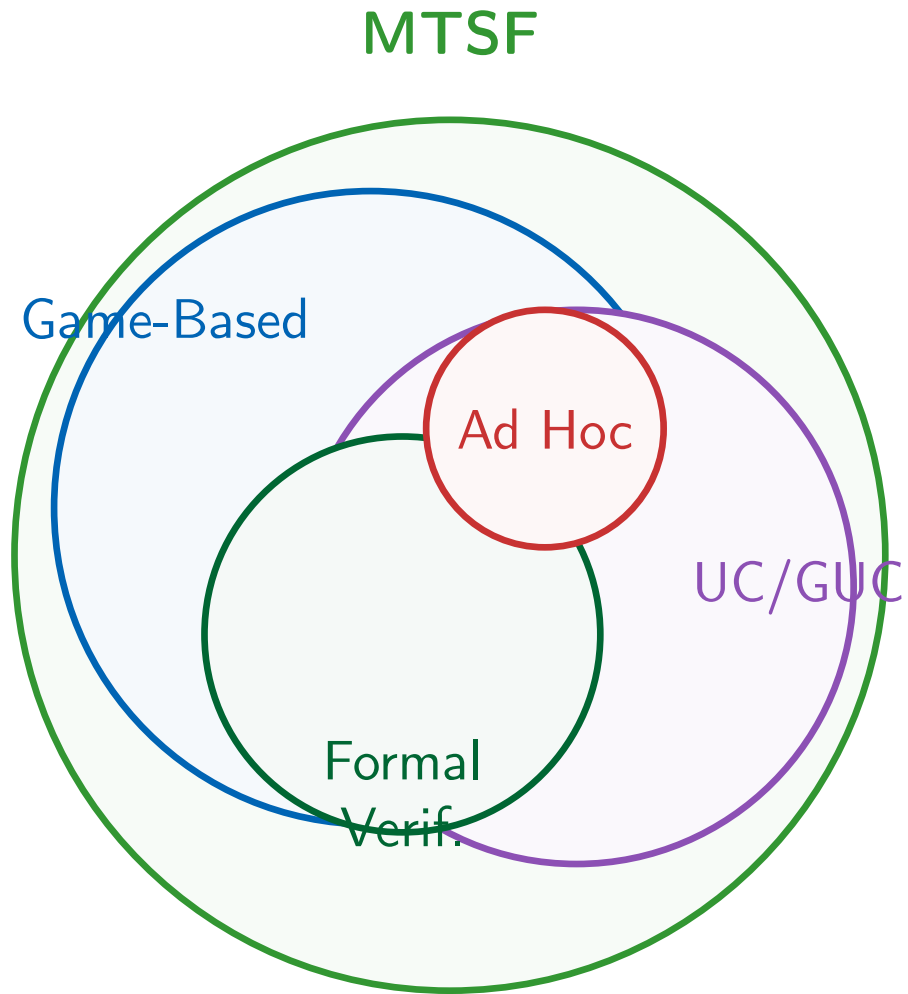
Any formal security framework must justify *why* its verdicts are trustworthy. In logic, a proof system is *sound* if it never declares a false statement true, and *complete* if it eventually declares every true statement provable. This section establishes analogous guarantees for MTSF: soundness means that if the market reaches equilibrium then the scheme is genuinely secure; completeness means that if the scheme is insecure then some buyer can collapse the market. The central technical mechanism is a *subroutine-consumption reduction*: during the bidding round the seller embeds the buyer as a black-box subroutine inside an algorithm for a computationally hard problem, thereby showing that the buyer’s success would yield an efficient solver for that problem—a contradiction under standard hardness assumptions.

* Simple Terms: Soundness and Completeness—Why They Matter

What do soundness and completeness mean here?

- **Soundness (“no false positives”):** If MTSF declares a scheme secure (market equilibrium), then the scheme *really is* secure—no efficient attacker can break it. Without soundness, the market could “clear” even for a broken scheme: a dangerous false sense of security.
- **Completeness (“no false negatives”):** If a scheme is insecure, then MTSF will detect the flaw—some buyer will submit a profitable bid that collapses the market. Without completeness, real vulnerabilities might hide in plain sight, undetectable by the framework.

Why is this different from traditional proofs? In classical game-based security, soundness is implicit (reductions are correct by construction) and completeness is not usually discussed because proofs are crafted per-scheme. MTSF, as a *general-purpose* framework, must demonstrate both properties *at the framework level*—ensuring that the market language itself does not introduce logical gaps.



MTSF contains all four paradigms as special cases.
Each is recoverable by restricting the market model.

Figure 22: MTSF as a superset of existing paradigms. Game-based proofs, UC/GUC, formal verification, and ad hoc insecurity are all contained within MTSF as special cases.

The key trick: The seller proves soundness by “consuming” the buyer as a subroutine. If the buyer could win the security game, the seller would use the buyer’s strategy as a black-box tool to solve a problem believed to be computationally intractable (like factoring, MLWE, or ECDLP). Since no efficient algorithm can solve that hard problem, the buyer cannot win either.

7.1 Subroutine Consumption: The Core Reduction Mechanism

The backbone of every MTSF security proof is a *subroutine-consumption reduction*. During a bidding round $\mathbf{B}_k$, the seller constructs a polynomial-time algorithm $\mathcal{R}$ (the *reducer*) that:

1. receives an instance x of a computationally hard problem Π_{hard} ;
2. simulates the market environment for the buyer $\mathcal{A}$ (setting up keys, answering oracle queries, issuing challenges) without knowing the solution to x ;
3. invokes the buyer $\mathcal{A}$ as a *black-box subroutine*—feeding $\mathcal{A}$ the simulated view and collecting $\mathcal{A}$ ’s output;
4. translates the buyer’s bid output (a forgery, a distinguishing guess, a key recovery) into a valid solution for x .

Thus $\mathcal{R}^{\mathcal{A}}(\cdot)$ is a polynomial-time oracle machine that solves Π_{hard} whenever the buyer’s bid succeeds.

Definition 7.1 (Subroutine-Consumption Reduction). *Let Π_{hard} be a computational problem and let $\mathcal{M} = (\text{Seller}, \{\text{Buyer}_i\}, \mathcal{G}, \text{Price})$ be an MTSF security market for a scheme Π . A subroutine-consumption reduction for good $g_j \in \mathcal{G}$ is a PPT oracle machine $\mathcal{R}$ such that for every PPT buyer $\mathcal{A}$:*

$$\text{Adv}_{\mathcal{R}^{\mathcal{A}}}^{\Pi_{\text{hard}}}(\lambda) \geq \frac{1}{L(\lambda)} \cdot \text{Adv}_{\mathcal{A}}^{g_j}(\lambda) - \delta(\lambda),$$

where $L(\lambda) \leq \text{poly}(\lambda)$ is the tightness loss and $\delta(\lambda)$ is a negligible simulation error. The reducer $\mathcal{R}$ **consumes** $\mathcal{A}$ as a subroutine: it runs $\mathcal{A}$ internally, relaying queries and collecting outputs, without knowledge of $\mathcal{A}$ ’s internal strategy.

* Simple Terms: Subroutine-Consumption Reduction

What does “consuming as a subroutine” mean? Imagine the seller is given a locked safe (the hard problem instance x). The seller does not know the combination. But the seller knows that if the buyer can break the security scheme, the buyer must implicitly “know” something about the combination. So the seller:

1. Disguises the safe’s lock as a security challenge (setting up the game using x).
2. Hands this disguised challenge to the buyer.
3. The buyer tries to break the scheme and produces output (a forgery, a guess, etc.).
4. The seller reads the buyer’s output and *extracts* the safe’s combination from it.

The buyer never realises it is helping the seller crack a safe—it thinks it is playing a normal security game. This is why it is called “consumption”: the seller *consumes* the buyer’s computational work, repurposing it to solve the hard problem.

Why does this prove security? If the hard problem is genuinely hard (no efficient algorithm can solve it), then the seller-plus-buyer combination also cannot solve it. But the seller is efficient (polynomial time). So the only way the combination fails is if the *buyer*

fails—i.e., the buyer cannot win the security game. Contrapositive: buyer success $\Rightarrow$ hard problem solved $\Rightarrow$ contradiction $\Rightarrow$ buyer cannot succeed.

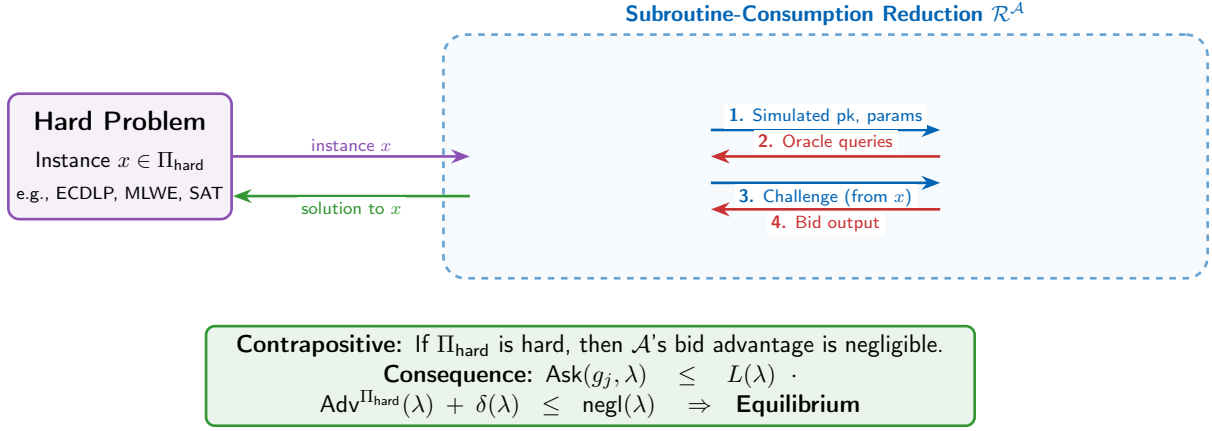


Figure 23: Subroutine-consumption reduction: the seller (reducer $\mathcal{R}$) receives a hard-problem instance x , simulates the market for the buyer $\mathcal{A}$, consumes $\mathcal{A}$'s bid output to solve x . Hardness of Π_{hard} implies the buyer's advantage is negligible—market equilibrium.

Analogy:

Subroutine Consumption as a Job Interview Ruse Imagine a company (the seller) is trying to solve a very hard engineering puzzle (the hard problem). They post a job listing (the security game) and invite a brilliant candidate (the buyer/adversary) for a “technical interview.” The interview questions are secretly the real engineering puzzle in disguise. If the candidate solves the interview questions (wins the security game), the company extracts the puzzle solution from the candidate's answers. Since the engineering puzzle is believed unsolvable, no candidate can pass the interview—the “job opening” (security game) remains unfilled (equilibrium).

Analogy:

Subroutine Consumption as a Sting Operation Here is another way to think about subroutine consumption—as a police sting operation:

The setup: The police (seller/reducer) want to catch a master thief (buyer/adversary) who claims they can crack any safe (break the security scheme). The police set up a fake jewellery store (the simulated security game) with a safe that contains evidence linked to an unsolved bank robbery (the hard problem instance).

The sting: The thief breaks into the fake store and cracks the safe (wins the security game). But by cracking this particular safe, the thief unknowingly reveals the technique needed to solve the bank robbery case (the hard problem).

The contradiction: The bank robbery was declared “unsolvable” by the FBI (the hard problem is computationally intractable). If the thief really could crack the safe, the FBI's assessment would be wrong. Since the FBI is trusted (the hardness assumption holds), the thief *cannot* actually crack the safe—the security scheme is secure.

The tightness loss $L(\lambda)$: Sometimes the police need multiple sting operations to catch the thief (they must guess which store the thief will target). The factor $L(\lambda)$ measures how many attempts the police need. A “tight” reduction means $L = 1$ —one sting suffices. A loose reduction means $L = q_H \cdot q_S$ —the police need many stings, which weakens the conclusion proportionally.

7.2 NP-Hard Subroutine Chains: Consuming One Hard Problem to Solve Another

The subroutine-consumption mechanism extends naturally to chains of NP-hard reductions. If the buyer's bid strategy $\mathcal{A}$ constitutes an algorithm for NP-hard problem Π_A , and an algorithm for Π_A can itself be consumed as a subroutine to solve a *different* NP-hard problem Π_B , then the seller constructs a two-layer reduction: the outer reducer solves Π_B by consuming a solver for Π_A , which itself is the buyer. This mechanism mirrors the structure of Karp reductions in complexity theory, but operationalised within the MTSF bidding-round framework.

Analogy:

NP-Hard Chains as a Puzzle Tournament Imagine a puzzle tournament with three rounds:

1. **Round 1 (Sudoku):** A contestant who can solve 9×9 Sudoku puzzles instantly.
2. **Round 2 (Graph Colouring):** If you can solve Sudoku instantly, I can use your Sudoku skills to solve graph colouring problems (by encoding the graph as a Sudoku grid).
3. **Round 3 (SAT):** If you can solve graph colouring instantly, I can use your colouring skills to solve Boolean satisfiability problems (SAT) (by encoding SAT as a colouring problem).

This is an NP-hard subroutine chain of depth 2: Sudoku $\rightarrow$ Graph Colouring $\rightarrow$ SAT. Each step “consumes” the previous solver as a subroutine.

In MTSF: If the buyer (adversary) can break the security scheme, the seller consumes the buyer to solve problem Π_A (step 1), then uses the Π_A solver to solve Π_B (step 2). Since Π_B is NP-hard (believed unsolvable efficiently), the buyer cannot break the scheme either.

Why chains matter: Some security proofs require multiple layers of reduction—the buyer's strategy does not directly solve the hard problem, but it solves an intermediate problem that can then be used to solve the hard problem. NP-hard chains formalise this multi-layer structure within MTSF's bidding-round framework.

Definition 7.2 (NP-Hard Subroutine Chain). *Let Π_A and Π_B be NP-hard problems and let g_j be a security good in market $\mathcal{M}$. An NP-hard subroutine chain of depth d is a sequence of PPT oracle machines $\mathcal{R}_1, \mathcal{R}_2, \dots, \mathcal{R}_d$ such that:*

1. $\mathcal{R}_1^{\mathcal{A}}$: consumes the buyer $\mathcal{A}$ (who attacks good g_j) to produce a solver for Π_A .
2. $\mathcal{R}_2^{\mathcal{R}_1^{\mathcal{A}}}$: consumes the Π_A -solver as a subroutine to produce a solver for Π_B .
3. In general, $\mathcal{R}_{i+1}^{\mathcal{R}_i^{\dots}}$ consumes the Π_{i-1} -solver to produce a Π_i -solver.

The total tightness loss is $L_{\text{total}}(\lambda) = \prod_{i=1}^d L_i(\lambda)$ and the total simulation error is $\delta_{\text{total}}(\lambda) = \sum_{i=1}^d \delta_i(\lambda)$.

* Simple Terms: NP-Hard Subroutine Chains

The idea in simple terms: Suppose the buyer tries to break a security scheme, and breaking it is equivalent to solving Problem A (say, cracking a combinatorial puzzle). Now suppose there is a known mathematical transformation: any solution to Problem A can be used to solve Problem B (a *different* puzzle). Then the seller can chain these reductions:

1. **Layer 1:** The seller sets up the market so that the buyer's attack output solves Problem A.
2. **Layer 2:** The seller feeds the Problem A solution into a converter that solves Problem B.

If Problem B is genuinely unsolvable by any efficient algorithm, then neither is Problem A, and therefore neither can the buyer win the security game.

Why is this powerful? It lets us prove security by connecting to the *hardest link in a chain* of problems. Even if the direct connection between the security game and the hardest known problem is indirect, the chain of subroutine consumptions links them rigorously.

NP-completeness connection: Recall that every NP-complete problem reduces to every other NP-complete problem. If the buyer's bid strategy solves one NP-complete problem, the chain shows it solves *all* of them—which is believed impossible for polynomial-time algorithms.

Algorithm 2 NP-Hard Subroutine Chain Reduction

Require: NP-hard problems Π_A, Π_B ; buyer $\mathcal{A}$ attacking good g_j ; instance $y \in \Pi_B$

Ensure: Solution to y (if $\mathcal{A}$ succeeds) or $\perp$

// **Layer 1: Reduce Π_B instance to Π_A instance**

1: Parse y as an instance of Π_B

2: Compute Karp reduction: $x \leftarrow \text{KarpReduce}_{B \rightarrow A}(y)$ ▷ x is a Π_A instance

// **Layer 2: Reduce Π_A instance to security game instance**

3: Embed x into market setup: $(\text{pk}, \text{state}) \leftarrow \mathcal{R}_1.\text{Setup}(x, 1^\lambda)$

4: Initialise buyer: send pk to $\mathcal{A}$

// **Layer 3: Execute bidding round with buyer as subroutine**

5: **while** $\mathcal{A}$ makes oracle query q_i **do**

6: $a_i \leftarrow \mathcal{R}_1.\text{SimOracle}(q_i, \text{state})$ ▷ Simulate oracle using x

7: Send a_i to $\mathcal{A}$

8: **end while**

9: Issue challenge $c^* \leftarrow \mathcal{R}_1.\text{Challenge}(\text{state})$ to $\mathcal{A}$

10: Receive bid output $\beta \leftarrow \mathcal{A}(c^*)$ ▷ **Buyer consumed as subroutine**

// **Layer 4: Extract solutions up the chain**

11: **if** β is a valid bid (forgery/guess/key) **then**

12: Extract Π_A solution: $s_A \leftarrow \mathcal{R}_1.\text{Extract}(\beta, \text{state})$

13: Lift to Π_B solution: $s_B \leftarrow \text{KarpLift}_{A \rightarrow B}(s_A, y)$ ▷ Inverse map

14: **return** s_B

15: **else**

16: **return** $\perp$ ▷ Buyer's bid failed; no solution extracted

17: **end if**

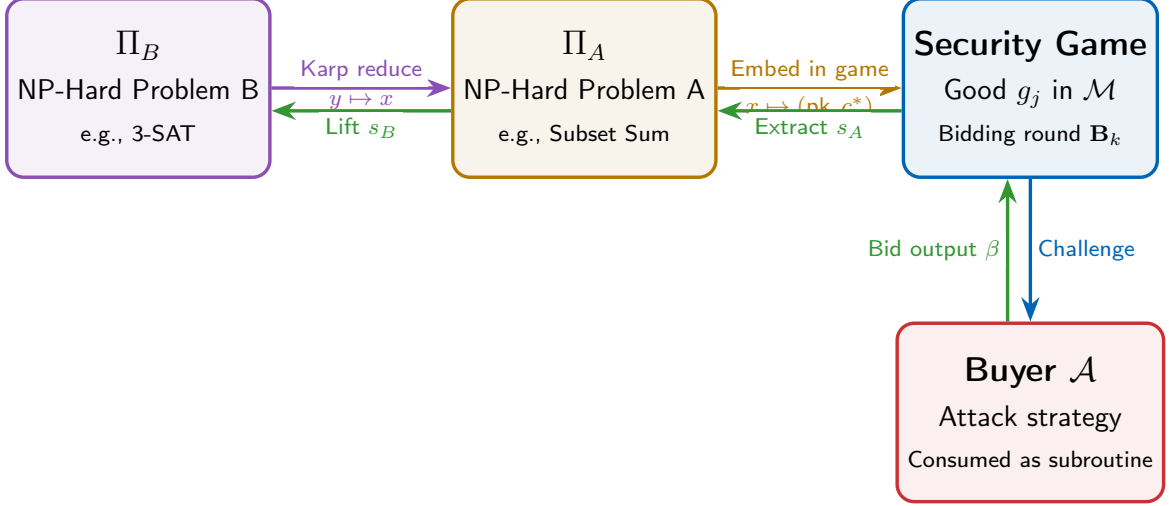
*** Simple Terms: Reading the NP-Hard Chain Algorithm**

What the algorithm does step by step:

1. **Layer 1 (Lines 2–3):** The seller receives a hard puzzle y (Problem B) and transforms it into an equivalent hard puzzle x (Problem A) using a known mathematical reduction.
2. **Layer 2 (Lines 5–6):** The seller disguises puzzle x as a security game, creating fake public keys and parameters that look real to the buyer but secretly encode x .
3. **Layer 3 (Lines 8–13):** The seller runs the normal bidding round: answering the buyer's oracle queries (simulating them using x), issuing a challenge, and collecting the buyer's output β . The buyer is an unwitting subroutine—it does not know it is solving x .

4. **Layer 4 (Lines 15–20):** If the buyer wins (produces a valid forgery/guess), the seller extracts a solution to puzzle x from the buyer's output, then lifts it back to a solution for the original puzzle y .

The punchline: If both puzzle x and puzzle y are NP-hard, no efficient buyer can produce β that leads to a solution. The market remains in equilibrium.



Contradiction: If $\mathcal{A}$ succeeds with non-negligible probability, then $\mathcal{R}_2^{\mathcal{R}_1^{\mathcal{A}}}$ solves Π_B efficiently. Since Π_B is NP-hard (and $P \neq NP$ assumed), $\mathcal{A}$ cannot succeed $\Rightarrow$ **Market Equilibrium**.

Figure 24: NP-hard subroutine chain: the seller reduces Π_B to Π_A to the security game. The buyer is consumed as the innermost subroutine. Solution extraction propagates outward: $\beta \rightarrow s_A \rightarrow s_B$. NP-hardness of Π_B prevents the buyer from succeeding.

7.3 Soundness of MTSF

Soundness asserts that MTSF equilibrium implies genuine security: if the bidding-round chain terminates with negligible cumulative cost, then no efficient adversary can break the scheme.

Theorem 7.3 (Soundness of MTSF). *Let $\mathcal{M} = (\text{Seller}, \{\text{Buyer}_i\}, \mathcal{G}, \text{Price})$ be a security market for scheme Π , and let $g_j \in \mathcal{G}$ be a security good. Suppose there exists a subroutine-consumption reduction $\mathcal{R}$ from the security game for g_j to a computational problem Π_{hard} with tightness loss $L(\lambda)$ and simulation error $\delta(\lambda)$. If Π_{hard} is $(T(\lambda), \epsilon_{\text{hard}}(\lambda))$ -hard (i.e., no algorithm running in time T succeeds with probability $> \epsilon_{\text{hard}}$), then for all PPT buyers $\mathcal{A}$:*

$$\text{Ask}(g_j, \lambda) \leq L(\lambda) \cdot \epsilon_{\text{hard}}(\lambda) + \delta(\lambda).$$

In particular, if Π_{hard} is hard against PPT (i.e., $\epsilon_{\text{hard}}(\lambda) \leq \text{negl}(\lambda)$) and $L(\lambda) \leq \text{poly}(\lambda)$ and $\delta(\lambda) \leq \text{negl}(\lambda)$, then $\text{Ask}(g_j, \lambda) \leq \text{negl}(\lambda)$ —the market is in equilibrium.

Proof. By contradiction. Suppose some PPT buyer $\mathcal{A}^*$ achieves $\text{Adv}_{\mathcal{A}^*}^{g_j}(\lambda) = \epsilon^* \not\leq \text{negl}(\lambda)$ —i.e., ϵ^* is non-negligible. By the subroutine-consumption reduction (Theorem 7.1), the reducer $\mathcal{R}^{\mathcal{A}^*}$

solves Π_{hard} with advantage:

$$\text{Adv}_{\mathcal{R}\mathcal{A}^*}^{\Pi_{\text{hard}}}(\lambda) \geq \frac{1}{L(\lambda)} \cdot \epsilon^* - \delta(\lambda).$$

Since ϵ^* is non-negligible, $L(\lambda) \leq \text{poly}(\lambda)$, and $\delta(\lambda) \leq \text{negl}(\lambda)$, the right-hand side is non-negligible. Thus $\mathcal{R}\mathcal{A}^*$ is a PPT algorithm (since $\mathcal{R}$ is PPT and $\mathcal{A}^*$ is PPT, the composition is PPT) that solves Π_{hard} with non-negligible advantage—contradicting the hardness assumption on Π_{hard} . Therefore no such $\mathcal{A}^*$ exists, and:

$$\text{Ask}(g_j, \lambda) = \max_{\mathcal{A} \in \text{PPT}} \text{Adv}_{\mathcal{A}}^{g_j}(\lambda) \leq L(\lambda) \cdot \epsilon_{\text{hard}}(\lambda) + \delta(\lambda) \leq \text{negl}(\lambda).$$

The market is in equilibrium for good g_j . □

* Simple Terms: Soundness Proof

What the proof says in plain language: Suppose, for the sake of argument, that a clever attacker (buyer) *could* break the security scheme. The seller would then use this attacker as a subroutine to solve a supposedly impossible mathematical problem (like factoring huge numbers). Since the mathematical problem is actually impossible to solve efficiently, the attacker cannot exist. Therefore, the scheme is secure.

The logical structure is: “If the scheme were insecure, then an impossible problem would be solvable. Since the impossible problem is not solvable, the scheme is secure.” This is the standard proof-by-contradiction structure, but cast in market language: “If a buyer could profit, the seller would use the buyer’s strategy to do something impossible. Since it is impossible, no buyer profits. Equilibrium.”

🗨 Key Insight:

Soundness Depends on Hardness Assumptions Soundness is *conditional*: it holds under the assumption that Π_{hard} is genuinely hard. If the hardness assumption is wrong (e.g., if someone discovers an efficient factoring algorithm), then the soundness guarantee evaporates. This is inherent in all computational security—no unconditional guarantee is possible under $\text{P} \neq \text{NP}$ uncertainty. MTSF makes this dependence explicit: the equilibrium price is bounded by a function of the hardness assumption, with exact tightness loss $L(\lambda)$ and simulation error $\delta(\lambda)$ fully specified.

7.4 Completeness of MTSF

Completeness asserts the converse: if a scheme is insecure, then MTSF detects the flaw—some buyer’s bid collapses the market.

Theorem 7.4 (Completeness of MTSF). *Let $\mathcal{M} = (\text{Seller}, \{\text{Buyer}_i\}, \mathcal{G}, \text{Price})$ be a security market for scheme Π , and let $g_j \in \mathcal{G}$ be a security good. If there exists a PPT algorithm $\mathcal{A}^*$ such that $\text{Adv}_{\mathcal{A}^*}^{g_j}(\lambda) = \epsilon^*(\lambda) \geq 1/\text{poly}(\lambda)$ (i.e., $\mathcal{A}^*$ breaks g_j with non-negligible advantage), then:*

1. **Bid construction:** *The buyer submits bid $\text{Bid}^* = (g_j, T^*, \epsilon^*)$ where $T^* = \text{Time}(\mathcal{A}^*)$.*
2. **Market collapse:** *$\text{Ask}(g_j, \lambda) \geq \epsilon^*(\lambda) \geq 1/\text{poly}(\lambda)$, which is non-negligible.*
3. **Equilibrium failure:** *The market is not in equilibrium for g_j .*

Moreover, if $\epsilon^*(\lambda) = 1 - \text{negl}(\lambda)$ (the scheme is totally broken), then $\text{Ask}(g_j, \lambda) = 1 - \text{negl}(\lambda)$: the market has fully collapsed.

Proof. By the definition of ask price (Theorem 3.4):

$$\text{Ask}(g_j, \lambda) = \max_{\mathcal{A} \in \text{PPT}} \text{Adv}_{\mathcal{A}}^{g_j}(\lambda) \geq \text{Adv}_{\mathcal{A}^*}^{g_j}(\lambda) = \epsilon^*(\lambda).$$

Since $\epsilon^*(\lambda) \geq 1/\text{poly}(\lambda)$ is non-negligible, $\text{Ask}(g_j, \lambda) \not\leq \text{negl}(\lambda)$, so the equilibrium condition of Theorem 3.4 fails. The buyer $\mathcal{A}^*$ realises the bid $\text{Bid}^* = (g_j, T^*, \epsilon^*)$ by running its attack strategy in time T^* and achieving advantage ϵ^* . The market has collapsed (or is at least non-equilibrium) for good g_j .

For total collapse: if $\text{Adv}_{\mathcal{A}^*}^{g_j}(\lambda) = 1 - \text{negl}(\lambda)$, then the ask price satisfies $\text{Ask}(g_j, \lambda) \geq 1 - \text{negl}(\lambda)$, and by the trivial upper bound $\text{Ask}(g_j, \lambda) \leq 1$, we obtain $\text{Ask}(g_j, \lambda) = 1 - \text{negl}(\lambda)$. This is full market collapse: the security good is worthless. $\square$

* Simple Terms: Completeness Proof

What the proof says in plain language: If a real attack exists (someone can actually break the scheme), then MTSF will detect it. The attacker simply submits a “bid” (describing the attack strategy, time budget, and success probability), and the market price rises above the negligible threshold. The market is no longer in equilibrium—MTSF correctly identifies the scheme as insecure.

Why is this important? Completeness guarantees that MTSF is not just a “rubber stamp” framework that declares everything secure. If a scheme is broken, MTSF *must* reflect this: the bid succeeds, the ask price rises, and the market collapses. This is exactly what happens in the Needham–Schroeder analysis (Section 13.4), the TLS 1.3 0-RTT analysis (Section 13.8.4), and the Telegram disproof (Section 13.9.2)—all are instances of completeness in action.

7.5 The Soundness–Completeness Duality

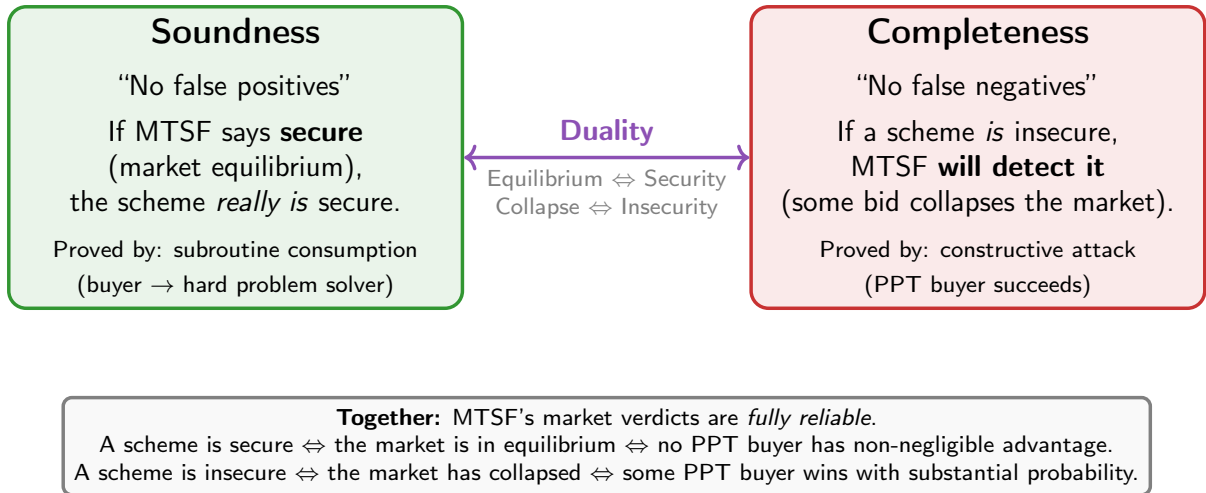


Figure 25: The soundness–completeness duality of MTSF. Soundness ensures no false positives (broken schemes declared secure); completeness ensures no false negatives (insecure schemes undetected). Together, they make MTSF’s market verdicts fully reliable.

The soundness and completeness theorems together establish a precise *duality* between the computational world and the market world:

Corollary 7.5 (MTSF Duality). *For any security good g_j in a market $\mathcal{M}$ with a valid subroutine-consumption reduction to Π_{hard} :*

$$\Pi \text{ is secure for } g_j \iff \mathcal{M} \text{ is in equilibrium for } g_j \iff$$

no PPT buyer profits from bidding on g_j ,

under the assumption that Π_{hard} is hard against PPT.

Proof. ($\Rightarrow$, Soundness) If Π is secure for g_j , then by Theorem 7.3 the market is in equilibrium. ($\Leftarrow$, Completeness) If Π is insecure for g_j , then by Theorem 7.4 some buyer's bid collapses the market—so the market is not in equilibrium. The contrapositive gives: equilibrium $\Rightarrow$ security. $\square$

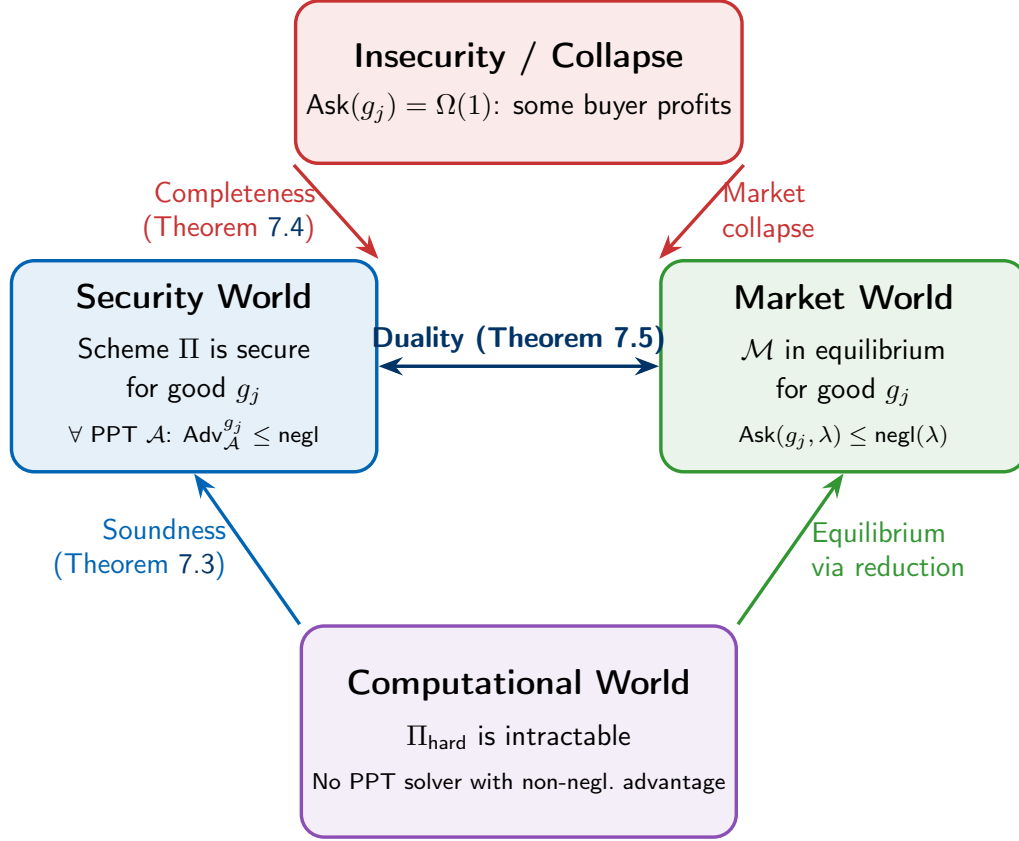


Figure 26: The soundness–completeness duality of MTSF. Soundness (bottom-left): hardness of Π_{hard} implies security and equilibrium. Completeness (top): insecurity implies market collapse. The three worlds—security, market, and computational—are formally equivalent under MTSF.

Analogy:

The Duality as a Smoke Detector A smoke detector is *sound* if it never triggers a false alarm (alarm $\Rightarrow$ real fire). It is *complete* if it always detects a real fire (fire $\Rightarrow$ alarm). MTSF is both: market equilibrium (no alarm) means the scheme is safe (no fire), and scheme insecurity (fire) means the market collapses (alarm sounds). The subroutine-consumption reduction is the sensor: it connects the “smoke” (attacker success) to the “alarm” (solution to a hard problem, which should not exist).

Analogy:

Soundness vs. Completeness as a Medical Test Think of MTSF’s verdicts like a medical test for a disease:

- **Soundness (no false positives):** If the test says “healthy” (market equilibrium), the patient really is healthy (the scheme is secure). A false positive would be catastrophic—

declaring a vulnerable scheme secure, leading to deployment of a broken system.

- **Completeness (no false negatives):** If the patient has the disease (the scheme is insecure), the test will detect it (some buyer collapses the market). A false negative would mean a real vulnerability hiding undetected.

MTSF achieves both:

- **Soundness:** Proved by showing that if the market reaches equilibrium, then any successful attacker would solve a computationally hard problem—a contradiction. (The medical test is validated against a gold standard.)
- **Completeness:** Proved by showing that if a scheme is insecure, there exists a PPT adversary whose bid collapses the market. (Every diseased patient produces a detectable biomarker.)

The *duality* between soundness and completeness means MTSF’s market verdicts are fully reliable: equilibrium $\Leftrightarrow$ security, collapse $\Leftrightarrow$ insecurity. This is the strongest possible guarantee for a security framework.

7.6 Worked Example: ECDSA Soundness via Subroutine Consumption

To make the abstract machinery concrete, we trace the subroutine-consumption reduction for ECDSA’s EUF-CMA security (proved in detail in Section 9.1).

Example 7.6 (ECDSA Subroutine Consumption). *Consider the ECDSA market $\mathcal{M}_{\text{ECDSA}}$ with good $g_{\text{EUF}} = \text{EUF-CMA}$ (unforgeability). The hard problem is $\Pi_{\text{hard}} = \text{ECDLP}$ (Elliptic Curve Discrete Logarithm Problem).*

Subroutine consumption in action:

1. **Reducer setup.** The reducer $\mathcal{R}$ receives an ECDLP instance $(G, Q = dG)$ where d is the unknown discrete log. $\mathcal{R}$ sets $\text{pk} = Q$ and begins simulating the signing oracle for the buyer $\mathcal{A}$.
2. **Oracle simulation.** When the buyer requests a signature on message m_i , the reducer simulates (r_i, s_i) using the forking lemma technique: it programs the random oracle so that valid-looking signatures can be produced without knowing d .
3. **Buyer consumed.** After q_S signing queries, the buyer outputs a forgery $(m^*, \sigma^* = (r^*, s^*))$ on a fresh message $m^* \notin \{m_1, \dots, m_{q_S}\}$.
4. **Solution extraction.** Using the forking lemma, $\mathcal{R}$ rewinds $\mathcal{A}$ to obtain two valid signatures (r^*, s_1^*) and (r^*, s_2^*) on m^* with different random oracle responses $h_1 \neq h_2$. From these, $\mathcal{R}$ computes:

$$d = \frac{s_1^* h_2 - s_2^* h_1}{s_2^* - s_1^*} \cdot r^{*-1} \pmod{n},$$

solving the ECDLP instance.

Soundness conclusion: By Theorem 7.3:

$$\text{Ask}(g_{\text{EUF}}, \lambda) \leq q_H \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + \frac{q_S}{2^\lambda} + \frac{q_H^2}{2^\lambda} \leq \text{negl}(\lambda).$$

The tightness loss is $L = q_H$ (number of hash queries) and the simulation error accounts for nonce collisions and birthday terms. The ECDSA market is in equilibrium.

* Simple Terms: ECDSA Soundness Example

What happened: The seller (reducer) was given an unsolved math puzzle (“find d such that $Q = dG$ on an elliptic curve”). The seller disguised this puzzle as an ECDSA public key and told the buyer: “Try to forge a signature.” The buyer tried its best attack and produced a forgery. The seller then mathematically extracted the answer to the original puzzle from the buyer’s forgery. Since the math puzzle is believed unsolvable, the buyer’s forgery cannot exist—ECDSA is secure (market equilibrium).

7.7 Worked Example: NP-Hard Subroutine Chain for CNF-Based Security

The NP-hard subroutine chain is particularly natural when the security game involves CNF-satisfiability checks, since 3-SAT is the canonical NP-complete problem.

Example 7.7 (CNF Subroutine Chain: $\text{SAT} \rightarrow \text{Subset Sum} \rightarrow \text{Security Game}$). *Consider a scheme Π whose session security is verified via a CNF formula φ (Section 4). Suppose a buyer $\mathcal{A}$ can find a satisfying assignment for a class of CNF formulae as part of its attack strategy (e.g., it finds an adversarial transcript that passes the CNF audit).*

Chain construction:

1. **Layer 1 ($\Pi_A = \text{3-SAT}$):** *The buyer’s ability to satisfy the session-CNF implies an algorithm for a subclass of 3-SAT instances (those arising from protocol transcripts). The reducer $\mathcal{R}_1$ embeds a 3-SAT instance into the session-CNF by encoding the SAT variables as session parameters (nonces, keys, timestamps).*
2. **Layer 2 ($\Pi_B = \text{Subset Sum}$):** *By Karp’s classical reduction, any 3-SAT instance can be transformed into a Subset Sum instance in polynomial time. The outer reducer $\mathcal{R}_2$ first transforms the Subset Sum instance y into a 3-SAT instance x (via the reverse encoding), then invokes $\mathcal{R}_1^A$ to solve x .*
3. **Solution propagation:** *If the buyer satisfies the session-CNF, $\mathcal{R}_1$ extracts a 3-SAT assignment, and $\mathcal{R}_2$ lifts it to a Subset Sum solution.*

Conclusion: *If the buyer could pass the CNF audit with an adversarial transcript, the chain would yield an efficient Subset Sum solver—contradicting the NP-hardness of Subset Sum (under $P \neq NP$). The session-CNF audit is sound: no adversarial transcript passes.*

★ Intuition:

Why NP-Hard Chains Strengthen the Framework Each link in the NP-hard chain adds a layer of confidence. Even if one suspects that the *direct* hard problem might someday be solved (e.g., lattice problems under quantum attack), the chain shows that breaking the scheme would solve *multiple* NP-hard problems simultaneously. This is analogous to requiring an attacker to independently rob multiple banks in a single heist—each bank’s security multiplies the difficulty.

7.8 Relationship to Classical Meta-Theorems

Analogy:

Equilibrium as a Ball at the Bottom of a Valley Visualise a ball (the attacker’s strategy) rolling across a landscape:

- **Equilibrium (secure):** The ball is at the bottom of a deep valley. No matter which direction the attacker pushes (which bid they try), the ball rolls back to the bottom. The valley is so deep (2^{128} units) that no polynomial-strength push can move the ball

significantly. The market is *stable*.

- **Collapse (insecure):** The ball is on a hilltop. The slightest push sends it rolling downhill—the attacker wins effortlessly. The market is *unstable* and collapses at the first bid.
- **Partial collapse:** The ball is on a shallow slope. It is not at the bottom (the scheme is not perfectly secure), but the slope is gentle enough that the attacker needs significant effort to reach the valley floor. Telegram’s MTPROTO is an example: the salt extraction bid gives a 2^{-64} advantage—not a full collapse, but not a deep valley either.

The depth of the valley is the ask price’s inverse: $1/\text{Ask}$. For ECDSA, the valley is 2^{128} units deep. For Needham–Schroeder, the “valley” is actually a hilltop (depth 0). MTSF measures the exact depth for every scheme.

The soundness and completeness of MTSF are not merely analogies—they are formal instantiations of classical meta-theorems in cryptographic proof theory.

Remark 7.8 (Connection to Bellare–Rogaway Reductions). *The subroutine-consumption reduction (Theorem 7.1) is the MTSF formalisation of the classical Bellare–Rogaway reduction paradigm [16]. In the classical setting, a “reduction” is a PPT oracle machine that uses the adversary as a black-box subroutine to solve a hard problem. MTSF adds the market interpretation: the reduction is the seller consuming the buyer during the bidding round, and the hard problem’s intractability ensures market equilibrium. The key insight is that this interpretation makes the direction of the reduction intuitive: the buyer bids (attacks), the seller co-opts the bid (reduces), and the hard problem’s resistance prevents profit (equilibrium).*

Remark 7.9 (Connection to Cook–Levin and Karp Reductions). *The NP-hard subroutine chain (Theorem 7.2) operationalises the Cook–Levin theorem and Karp’s 21 reductions [58] within the MTSF bidding framework. The Cook–Levin theorem states that every problem in NP reduces to SAT; Karp’s reductions show that SAT reduces to 20 other specific NP-complete problems. In MTSF language: if the buyer’s bid strategy implicitly solves any NP-complete problem, the seller can chain reductions to solve all NP-complete problems—a catastrophic violation of the $P \neq NP$ assumption. This connection grounds MTSF’s security guarantees in the deepest conjectures of computational complexity theory.*

7.9 Implications for MTSF Case Studies

The soundness and completeness theorems have direct implications for every case study in this article:

Soundness and Completeness Across All Case Studies

Soundness in action (security proofs): Every security proof in Sections 9–16 proceeds by constructing a subroutine-consumption reduction from the security game to a hard problem. The seller consumes the buyer as a subroutine during each bidding round. Specifically:

- **ECDSA (Section 9.1):** Buyer consumed to solve ECDLP.
- **ML-KEM (Section 9.2):** Buyer consumed to solve MLWE.
- **ML-DSA (Section 9.3):** Buyer consumed to solve MSIS.
- **HMAC (Section 9.5):** Buyer consumed to distinguish a PRF from random.
- **AEAD (Section 9.6):** Buyer consumed to break IND-CPA or MAC unforgeability.

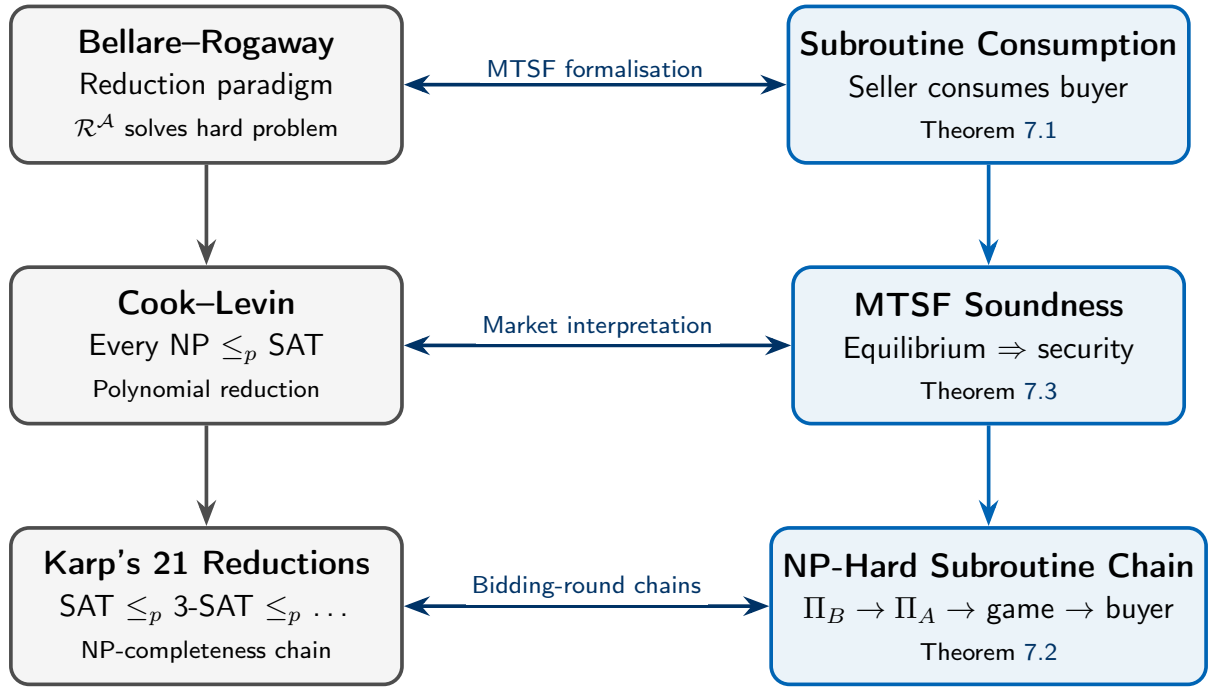


Figure 27: Correspondence between classical meta-theorems in complexity/cryptography and their MTSF counterparts. Each classical result has a precise market-theoretic formalisation.

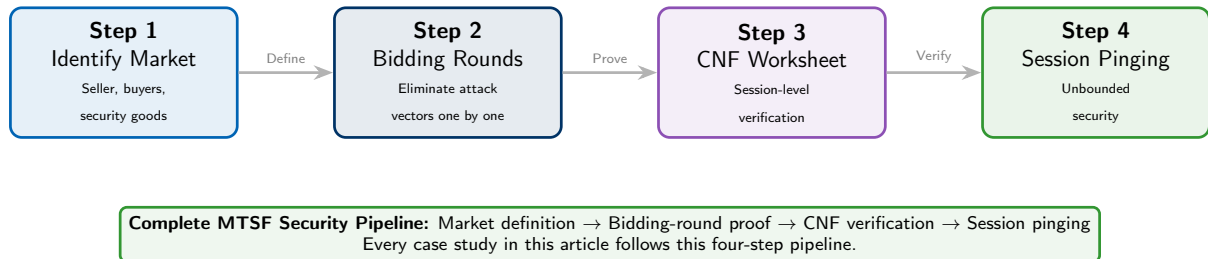


Figure 28: The four-step MTSF security pipeline that every case study follows: (1) define the market (seller, buyers, goods), (2) construct bidding-round chain with price adjustments, (3) fill in the CNF verification worksheet, (4) verify session pinging for unbounded security.

- **SLH-DSA (Section 9.7):** Buyer consumed to find hash collisions or second preimages.
- **FN-DSA (Section 9.8):** Buyer consumed to solve NTRU/SIS.
- **AES (Section 10):** Buyer consumed to distinguish AES from a PRP.
- **SHA-3 (Section 11):** Buyer consumed to find capacity collisions.
- **Grain-128a (Section 12):** Buyer consumed to recover internal state.
- **TLS 1.3 1-RTT (Section 13.8.3):** Buyer consumed to solve ECDLP or break HKDF-PRF.
- **Signal X3DH (Section 13.7):** Buyer consumed to solve gap-CDH.
- **QROM KEM (Section 14):** Buyer consumed to break IND-CPA of underlying PKE via O2H.
- **BB84 QKD (Section 15):** Quantum buyer's information bounded by no-cloning and information-disturbance trade-off; key extracted via quantum leftover hash lemma. Security is information-theoretic (no computational assumption consumed).
- **TLS+Signal composition (Section 16):** Standalone buyers constructed for each market via UC simulation of the other market; merged market equilibrium follows from Theorem 21.7.

In every case, Theorem 7.3 guarantees: $\text{Ask}(g_j, \lambda) \leq L \cdot \epsilon_{\text{hard}} + \delta \leq \text{negl}(\lambda)$.

Completeness in action (insecurity proofs): Every insecurity proof constructs an explicit PPT buyer whose bid succeeds with overwhelming probability:

- **Textbook RSA (Section 9.1):** Homomorphism buyer achieves $\text{Ask} = 1$.
- **Needham–Schroeder (Section 13.4):** Masquerade buyer achieves $\text{Ask} = 1$.
- **TLS 1.3 0-RTT (Section 13.8.4):** Replay buyer achieves $\text{Ask} = 1$.
- **TLS 1.3 downgrade (Section 13.8.5):** Version-stripping buyer achieves $\text{Ask} = 1$.
- **Telegram MTPROTO (Section 13.9.2):** Salt-extraction buyer achieves $\text{Ask} \approx 1$.
- **Signal X3DH without OPK (Section 13.7.1):** Replay buyer achieves $\text{Ask}(g_{\text{async}}) = 1$.

In every case, Theorem 7.4 guarantees: $\text{Ask}(g_j, \lambda) \geq \epsilon^* = 1 - \text{negl}(\lambda)$ —the market has collapsed.

Remark 7.10 (Subroutine Consumption and the NP-Hardness Paradigm). *The subroutine-consumption mechanism establishes a fundamental link between cryptographic security and computational complexity. In the MTSF framework, every security proof is an instance of the following paradigm:*

An algorithm for NP-hard problem Π_A can solve another NP-hard problem Π_B if the algorithm for Π_A is consumed as a subroutine.

The buyer's attack strategy constitutes an "algorithm" for a specific computational task (forging signatures, distinguishing ciphertexts, recovering keys). The seller's reduction shows that this task is at least as hard as a known NP-hard problem. The subroutine chain (Theorem 7.2) extends

this to multiple layers: if the buyer’s algorithm (for task Π_A) is consumed as a subroutine by a converter that solves Π_B , and Π_B is consumed by another converter that solves Π_C , then the buyer’s success would yield an efficient solver for Π_C —a contradiction cascading through the entire NP-hardness hierarchy. This is the market-theoretic operationalisation of the NP-completeness web: breaking one problem breaks them all.

8 Protocol-Level Security Games in MTSF

In a protocol market, the seller offers not just primitive-level goods (EUF-CMA, IND-CCA2) but also *protocol-level goods*: entity authentication, mutual authentication, session-key secrecy, and CNF session correctness. Each is formalised as an individual buyer game within the market.

* Simple Terms: Protocol-Level Security Games—Why They Are Needed

The gap between primitives and protocols: Proving that a signature scheme is EUF-CMA secure tells you that an individual signature cannot be forged. But a protocol using that signature scheme might still be broken by an attacker who:

- Replays a valid signature from a *previous* session (replay attack).
- Redirects a valid signature from one party to impersonate another (masquerade attack).
- Wins a man-in-the-middle position without forging any signature (as in Needham–Schroeder).

Protocol-level security games capture these higher-level threats.

Four protocol-level goods in MTSF:

1. **Entity Authentication (g_{auth}):** If Bob accepts a session with partner Alice, then Alice was genuinely participating. An attacker who impersonates Alice to Bob without Alice’s involvement loses this game.
2. **Mutual Authentication (g_{mutual}):** Both Alice and Bob accept only if the other genuinely participated. Neither direction of impersonation is possible.
3. **Session-Key Secrecy (g_{sk}):** After a completed protocol run, the session key K_{sess} is computationally indistinguishable from a random key to any attacker who observed all messages. Even a powerful network attacker learns nothing about K_{sess} .
4. **CNF Correctness (g_{CNF}):** The session-CNF formula φ_i is satisfiable only under honest transcripts. Any adversarially modified transcript fails the CNF audit.

Why embed these as market goods? In MTSF, modelling these as market goods enables a unified security analysis. Each good has an explicit ask price (how likely an attacker is to break it), and all reductions are explicit and quantitative. This is the first framework to embed all four protocol-level properties as first-class market goods within a single formalism.

8.1 The Authentication Game

Definition 8.1 (Authentication Good g_{auth}). *The seller offers g_{auth} : the guarantee that after a protocol run, if honest party B accepts, then B ’s intended partner A was indeed participating. The buyer (active network adversary $\mathcal{A}$) wins if B accepts a session without A having sent the corresponding messages.*

Theorem 8.2 (Authentication Game—Game-Based Bound). *For a protocol using signatures for entity authentication:*

$$\text{Ask}(g_{\text{auth}}) \leq \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}(\lambda), \quad (19)$$

where q_N is the number of sessions (nonces sampled).

Proof. We construct four games, bounding the difference at each hop.

B₀ (Bidding Round 0: Real Market Execution).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3), analogous to the seller listing the good at quoted price “333” (secure schemes like ECDSA, ML-KEM, AES—the quoted price is a bluff), “555” (quasi-secure schemes like Telegram MTPROTO—partial bluff), or “999” (insecure schemes like Needham–Schroeder—the price is honest, the good is worthless). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium, $\text{QPrice}_0 = \text{“333”}$) or honest (collapse, $\text{QPrice}_0 = \text{“999”}$).

The seller (challenger $\mathcal{C}$) runs the protocol honestly, offering the authentication good g_{auth} at ask price Ask_0 . The buyer (adversary $\mathcal{A}$) controls the network: it may delay, reorder, drop, and inject messages between honest parties A and B , placing a *full-control network bid*. The buyer wins—and the market collapses for g_{auth} —if B accepts and outputs $\text{pid} = A$, but A did not participate in any matching session. This is the baseline: $\text{Ask}_0 = \text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda)$.

B₁ (Bidding Round 1: Nonce Freshness Bid).

Purpose: Eliminate the *nonce-reuse attack vector*. Repeated nonces immediately collapse authentication, key-secrecy, or stream-cipher security: in signatures a shared nonce leaks the private key ($\text{sk} = (s_2 e_1 - s_1 e_2)(r(s_1 - s_2))^{-1}$); in protocols it enables cross-session replay; in stream ciphers it reveals the XOR of two plaintexts. This round aborts the game whenever any nonce collision occurs, ensuring every subsequent hop reasons about a nonce-fresh world.

Replaces: The real nonce-sampling procedure is augmented with a global *collision audit*: a set $\mathcal{N}_{\text{used}}$ records every nonce issued so far. If any newly sampled nonce k' satisfies $k' \in \mathcal{N}_{\text{used}}$, the game aborts immediately. The two games ($\mathbf{B}_{n-1}$ and $\mathbf{B}_n$) are *identical-until-abort*: the adversary’s view is indistinguishable until the abort event F_{nonce} occurs, so the difference lemma directly bounds the price adjustment.

Complexity: By the birthday bound, the probability that any two of q independently sampled nonces from a space of size $|\mathcal{N}|$ collide is at most $q^2/(2|\mathcal{N}|)$. For security parameter λ with $|\mathcal{N}| = 2^\lambda$ this gives $\Delta\text{Price} \leq q^2/2^{\lambda+1}$, which is negligible for any polynomial $q = \text{poly}(\lambda)$. After this hop every nonce in the proof is treated as distinct, allowing all later hops to assume a collision-free nonce space without loss of generality.

The seller adds an internal audit: abort the session if any two concurrent or sequential sessions sample the same nonce. This models the seller *refusing to sell* under a *nonce-reuse bid*—a buyer strategy that exploits repeated randomness. The two bidding rounds $\mathbf{B}_0$ and $\mathbf{B}_1$ are identical until the bad event F_{nonce} (a nonce collision) occurs; by the difference lemma, the price adjustment is $\Delta\text{Price}_0 = |\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \Pr[F_{\text{nonce}}]$.

Difference bound. By the birthday bound over q_N nonces sampled uniformly from $\{0, 1\}^\lambda$:

$$|\Pr[\mathbf{B}_0(\mathcal{A}) = 1] - \Pr[\mathbf{B}_1(\mathcal{A}) = 1]| \leq \Pr[F_{\text{nonce}}] \leq \frac{q_N^2}{2^{\lambda+1}}. \quad (20)$$

B₂ (Bidding Round 2: Signature Forgery Bid).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $sk = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma’s rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

The seller now also maintains a *signature ledger*: a complete record of every signature honest party A has generated. If the buyer causes B to accept a signature σ^* on session data $m^*||\text{sid}$ that does *not* appear in the ledger, the market flags this as a *forgery bid* event F_{forge} —the buyer is attempting to sell counterfeit goods. The price adjustment is bounded by the EUF-CMA advantage: a forgery bid succeeds only if the buyer can produce a valid signature under A ’s key without ever querying A ’s signing oracle on that message, which by EUF-CMA security costs $\text{Adv}_{\text{Sig}}^{\text{EUF-CMA}}$.

Difference bound. We reduce F_{forge} to EUF-CMA: given an EUF-CMA challenger for A ’s key, the reduction embeds A ’s public key and forwards signing queries. If $\mathcal{A}$ triggers F_{forge} , the reduction outputs $(m^*||\text{sid}, \sigma^*)$ as a forgery. Hence:

$$|\Pr[\mathbf{B}_1(\mathcal{A}) = 1] - \Pr[\mathbf{B}_2(\mathcal{A}) = 1]| \leq \Pr[F_{\text{forge}}] \leq \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}}. \quad (21)$$

B₃ (Bidding Round 3: Ideal Market—All Entity-Auth Bids Fail).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary’s view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary’s winning probability in this game is 0 (for authentication/forgery games) or exactly $1/2$ (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta\text{Price}_k$, all of which are negligible, establishing market equilibrium.

Conditioned on no nonce-reuse bid ($\neg F_{\text{nonce}}$) and no forgery bid ($\neg F_{\text{forge}}$) succeeding, the market reaches equilibrium for g_{auth} : if B accepts with partner A , then B verified a valid, ledger-registered signature under A ’s genuine key on session-specific data including a fresh sid and nonce N_B . The signature is in the ledger (no forgery bid succeeded), and the nonce is unique (no nonce bid succeeded), so A must have generated this signature for this exact session— A genuinely participated. The buyer’s winning probability in the ideal market: $\Pr[\mathbf{B}_3 = 1] = 0$. *Market equilibrium for g_{auth} is established.*

Summing:

$$\text{Ask}(g_{\text{auth}}) \leq \frac{q_N^2}{2^{\lambda+1}} + \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + 0 = \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}(\lambda). \quad (22)$$

□

Analogy:

Entity Authentication as a Video Call Verification Imagine you receive a phone call from someone claiming to be your bank manager. How do you know it is really them?

Entity authentication is the cryptographic version of this problem. In the physical world, you might:

- Ask a question only the real manager would know (like a shared secret/password).
- Recognise their face on a video call (like a public key—their face is “public” and hard to fake).
- Check that the call came from the bank’s official number (like verifying a digital certificate from a trusted CA).

In MTSF, the authentication good g_{auth} formalises this: after a protocol run, if Bob accepts that he is talking to Alice, then Alice was *genuinely participating*. An attacker who impersonates Alice must forge her digital signature—which costs $\text{Adv}^{\text{EUF-CMA}}$, a negligible amount for secure signature schemes.

The Needham–Schroeder lesson: Even if the signature scheme is perfect, a protocol can still fail authentication if it does not properly bind identities to messages. The NS protocol’s MITM attack succeeds without breaking any cryptographic primitive—a design flaw, not a crypto flaw. MTSF’s CNF verification catches such design flaws by requiring an identity-binding clause.

8.2 The Mutual Authentication Game

Definition 8.3 (Mutual Authentication Good g_{mutual}). *The seller offers g_{mutual} : after a protocol run, both A and B accept with correct partner identities, or neither accepts. The buyer wins if either (a) B accepts with partner A but A did not participate, or (b) A accepts with partner B but B did not participate.*

Theorem 8.4 (Mutual Authentication Equilibrium). *For a protocol with bidirectional signatures:*

$$\text{Ask}(g_{\text{mutual}}) \leq 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}(\lambda). \quad (23)$$

Proof. We construct five games.

B₀ (Bidding Round 0: Real Market, Full Dual-Direction Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round;

the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

The seller offers the mutual authentication good g_{mutual} . The buyer places a *dual-direction impersonation bid*: it attempts to violate authentication in at least one direction—either impersonating A to B or impersonating B to A (or both). The market ask price is $\text{Ask}_0 = \text{Adv}_A^{\text{mutual}}(\lambda)$.

B₁ (Bidding Round 1: Nonce Freshness Bid—Both Directions).

Purpose: Eliminate the *nonce-reuse attack vector*. Repeated nonces immediately collapse authentication, key-secrecy, or stream-cipher security: in signatures a shared nonce leaks the private key ($\text{sk} = (s_2 e_1 - s_1 e_2)(r(s_1 - s_2))^{-1}$); in protocols it enables cross-session replay; in stream ciphers it reveals the XOR of two plaintexts. This round aborts the game whenever any nonce collision occurs, ensuring every subsequent hop reasons about a nonce-fresh world.

Replaces: The real nonce-sampling procedure is augmented with a global *collision audit*: a set $\mathcal{N}_{\text{used}}$ records every nonce issued so far. If any newly sampled nonce k' satisfies $k' \in \mathcal{N}_{\text{used}}$, the game aborts immediately. The two games ($\mathbf{B}_{n-1}$ and $\mathbf{B}_n$) are *identical-until-abort*: the adversary’s view is indistinguishable until the abort event F_{nonce} occurs, so the difference lemma directly bounds the price adjustment.

Complexity: By the birthday bound, the probability that any two of q independently sampled nonces from a space of size $|\mathcal{N}|$ collide is at most $q^2/(2|\mathcal{N}|)$. For security parameter λ with $|\mathcal{N}| = 2^\lambda$ this gives $\Delta\text{Price} \leq q^2/2^{\lambda+1}$, which is negligible for any polynomial $q = \text{poly}(\lambda)$. After this hop every nonce in the proof is treated as distinct, allowing all later hops to assume a collision-free nonce space without loss of generality.

The seller enforces nonce uniqueness across all sessions in both directions, placing a *freshness constraint* on nonce bids. Identical-until- F_{nonce} ; price adjustment:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \frac{q_N^2}{2^{\lambda+1}}. \quad (24)$$

B₂ (Bidding Round 2: Signature Forgery Bid on A —Impersonating A to B).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $\text{sk} = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma’s rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

The seller adds a signature ledger for A . The buyer places a *directed forgery bid*: cause B to accept a signature σ_A that A never generated, thereby impersonating A to B . Any success here is an EUF-CMA forgery against A ’s signing key. Price adjustment (buyer’s cost to break A ’s signature):

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq \text{Adv}_{\text{Sig}, A}^{\text{EUF-CMA}}. \quad (25)$$

B₃ (Bidding Round 3: Signature Forgery Bid on B —Impersonating B to A).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $sk = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma’s rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

Symmetrically, the seller adds a signature ledger for B . The buyer places a *directed forgery bid* in the other direction: cause A to accept a signature σ_B that B never generated. By the extended difference lemma, F_{forge_A} (targeting A) and F_{forge_B} (targeting B) use independent signing keys, so their failure events are handled separately. Price adjustment (buyer’s cost to break B ’s signature):

$$|\Pr[\mathbf{B}_2 = 1] - \Pr[\mathbf{B}_3 = 1]| \leq \text{Adv}_{\text{Sig}, B}^{\text{EUF-CMA}}. \quad (26)$$

B₄ (Bidding Round 4: Ideal Market—Both Dual-Direction Bids Fail).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary’s view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary’s winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta \text{Price}_k$, all of which are negligible, establishing market equilibrium.

With no nonce-reuse bid, no forgery bid on A , and no forgery bid on B : both signatures in any accepted session are ledger-verified, session-specific, and nonce-fresh. Both parties participated genuinely. The dual-direction impersonation bid fails completely. $\Pr[\mathbf{B}_4 = 1] = 0$. *Market equilibrium for g_{mutual} is established.*

Extended difference lemma (combined). The forgery events F_A and F_B use independent keys, so: $\Pr[F_A \cup F_B] \leq \Pr[F_A] + \Pr[F_B] - \Pr[F_A \cap F_B] \leq 2 \cdot \text{Adv}^{\text{EUF}}$, since $\Pr[F_A \cap F_B] \geq 0$.

Total: $\text{Ask}(g_{\text{mutual}}) \leq 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + q_N^2/2^{\lambda+1}$. □

8.3 The Session-Key Secrecy Game

Analogy:

Session-Key Secrecy as a Sealed Envelope After Alice and Bob complete a key exchange protocol, they share a session key K_{sess} . Session-key secrecy means this key is as good as a fresh random key, even to an attacker who observed every message exchanged during the protocol.

Imagine: Alice and Bob are in a glass room (the attacker can see everything). They perform a complex series of hand gestures (the protocol messages). At the end, they each write a number on a piece of paper and seal it in an envelope. Session-key secrecy means: even though the attacker watched every gesture, they have *no idea* what number is in the envelope. If you gave the attacker two envelopes—one with the real session key and one with a random number—they could not tell which is which.

The game: The challenger generates the session key K_0 honestly and picks a random key K_1 . It flips a coin b and gives the attacker K_b . The attacker, having observed the entire protocol execution, must guess b . If they can guess correctly with probability significantly better than 50%, the session-key secrecy good is compromised. If not, the market is in equilibrium.

Definition 8.5 (Session-Key Secrecy Good g_{sk}). *After a completed protocol run producing session key K_{sess} , the buyer receives either K_{sess} or a uniformly random key K' (chosen by a random bit b). The buyer wins by guessing b . The good is secure if $\text{Ask}(g_{\text{sk}}) \leq \text{negl}(\lambda)$.*

Theorem 8.6 (Session-Key Secrecy Equilibrium). *For a protocol using a KEM for key transport and a KDF for derivation:*

$$\text{Ask}(g_{\text{sk}}) \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}} + \text{Adv}_{\text{KDF}}^{\text{PRF}} + 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}(\lambda). \quad (27)$$

Proof. We construct six games.

B₀ (Bidding Round 0: Real Market—Key-Secrecy Distinguishing Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary's raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

The seller generates keys and runs the protocol honestly, offering the session-key secrecy good g_{sk} . The buyer places a *key-distinguishing bid*: after the protocol completes, the seller flips $b \xleftarrow{\$} \{0, 1\}$ and presents the buyer with either the real session key K_{sess} (if $b = 0$) or a uniformly random $K' \xleftarrow{\$} \{0, 1\}^\lambda$ (if $b = 1$). The buyer bids on guessing b by outputting b' . The market ask price is $\text{Ask}_0 = |\Pr[\mathbf{B}_0(b' = b)] - 1/2|$ —the buyer's advantage over random guessing.

B₁ (Bidding Round 1: Nonce Freshness Bid—Preventing Replay).

Purpose: Eliminate the *nonce-reuse attack vector*. Repeated nonces immediately collapse authentication, key-secrecy, or stream-cipher security: in signatures a shared nonce leaks the private key ($\text{sk} = (s_2 e_1 - s_1 e_2)(r(s_1 - s_2))^{-1}$); in protocols it enables cross-session replay; in stream ciphers it reveals the XOR of two plaintexts. This round aborts the game whenever any nonce collision occurs, ensuring every subsequent hop reasons about a nonce-fresh world.

Replaces: The real nonce-sampling procedure is augmented with a global *collision audit*: a set $\mathcal{N}_{\text{used}}$ records every nonce issued so far. If any newly sampled nonce k' satisfies $k' \in \mathcal{N}_{\text{used}}$, the game aborts immediately. The two games ($\mathbf{B}_{n-1}$ and $\mathbf{B}_n$) are *identical-until-abort*: the adversary's view is indistinguishable until the abort event F_{nonce} occurs, so the difference lemma directly bounds the price adjustment.

Complexity: By the birthday bound, the probability that any two of q independently sampled nonces from a space of size $|\mathcal{N}|$ collide is at most $q^2/(2|\mathcal{N}|)$. For security parameter λ with $|\mathcal{N}| = 2^\lambda$ this gives $\Delta\text{Price} \leq q^2/2^{\lambda+1}$, which is negligible for any polynomial $q = \text{poly}(\lambda)$. After this hop every nonce in the proof is treated as distinct, allowing all later hops to assume a collision-free nonce space without loss of generality.

The seller enforces nonce uniqueness: a nonce-reuse bid would enable session key correlation. This costs the market: $|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq q_N^2/2^{\lambda+1}$.

B₂ (Bidding Round 2: Signature Forgery Bid on A).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $\text{sk} = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma's rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

A forgery on A 's signature would inject a rogue session, compromising key secrecy without breaking the KEM. The seller blocks this via EUF-CMA. Price adjustment:

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq \text{Adv}_{\text{Sig}}^{\text{EUF}}.$$

B₃ (Bidding Round 3: Signature Forgery Bid on B).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $\text{sk} = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking

lemma's rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

Symmetric to $\mathbf{B}_2$: a forgery on B 's signature is blocked. Both forgery bids use independent keys (extended difference lemma applies). Price adjustment: $|\Pr[\mathbf{B}_2 = 1] - \Pr[\mathbf{B}_3 = 1]| \leq \text{Adv}_{\text{Sig}}^{\text{EUF}}$.

$\mathbf{B}_4$ (Bidding Round 4: KEM Ciphertext Bid—Replacing the Session Secret).

Purpose: Replace the structured MLWE ciphertext components with uniformly random values, severing the last information-theoretic link between the ciphertext and the encapsulated key. After both public-key and ciphertext hops, the key is information-theoretically hidden.

Replaces: The challenge ciphertext $(\mathbf{u} = \mathbf{A}^T \mathbf{r} + \mathbf{e}_1, v = \mathbf{t}^T \mathbf{r} + e_2 + \lfloor q/2 \rfloor m)$ is replaced by $(\mathbf{u}', v')$ where both components are independently and uniformly random. Detecting this change requires solving a second MLWE instance embedded in the ciphertext structure.

Complexity: A second application of $\text{Adv}^{\text{MLWE}}(\lambda)$. The two MLWE instances—one in the public key, one in the ciphertext—use independent randomness and are therefore handled by separate hops. The total across both hops is $2 \cdot \text{Adv}^{\text{MLWE}}(\lambda)$, matching the bound in Theorem 9.2.

The seller now replaces the real encapsulated key K with a uniformly random $\tilde{K}$. Any buyer that distinguishes $\mathbf{B}_3$ from $\mathbf{B}_4$ is placing a *KEM plaintext-recovery bid*: it can distinguish a real KEM ciphertext from one encapsulating a random key, which is exactly IND-CCA2 security. Price adjustment: $|\Pr[\mathbf{B}_3 = 1] - \Pr[\mathbf{B}_4 = 1]| \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}$.

$\mathbf{B}_5$ (Bidding Round 5: KDF Randomness Bid—Replacing the Derived Key).

Purpose: Replace a *keyed pseudorandom function* evaluation with a truly random function, isolating a single PRF security requirement. This hop shows that one more step in the key-derivation or authentication structure produces outputs that are computationally indistinguishable from uniform random, provided the underlying PRF assumption holds.

Replaces: The specific PRF evaluation (e.g., $\text{HMAC}_K(\cdot)$ inner hash, HKDF-Extract, Derive-Secret, or a KDF step) is replaced by a truly random function $R(\cdot)$ sampled fresh and independently. Any adversary that distinguishes the PRF output from this random function is a PRF distinguisher for the underlying keyed construction. The reduction simulates all other game components honestly and uses its PRF challenge oracle for this one step.

Complexity: $\Delta\text{Price} \leq \text{Adv}^{\text{PRF}}(\lambda)$ for the specific PRF being replaced (HMAC-SHA-256, HKDF, or AES-CTR as applicable). When multiple PRF hops appear in a single proof (e.g., three hops in the TLS 1.3 key schedule for HS, MS, and CATS), each hop is independent: the keys used in different steps are derived from different points in the ladder, so no PRF oracle is queried twice. The total across all PRF hops is the sum of individual advantages, each negligible.

The seller replaces $K_{\text{sess}} = \text{KDF}(\tilde{K}, \text{sid} \| N_A \| N_B)$ with a uniformly random key K_{rand} . Any buyer distinguishing $\mathbf{B}_4$ from $\mathbf{B}_5$ is placing a *PRF-distinguishing bid* against the KDF—it detects that the output is a PRF evaluation rather than random, which breaks PRF security. Price adjustment: $|\Pr[\mathbf{B}_4 = 1] - \Pr[\mathbf{B}_5 = 1]| \leq \text{Adv}_{\text{KDF}}^{\text{PRF}}$.

In $\mathbf{B}_5$, the test key is independent of b , so $\Pr[\mathbf{B}_5 = 1] = 1/2$ exactly.

Total: $\text{Ask}(g_{\text{sk}}) = |\Pr[\mathbf{B}_0 = 1] - 1/2| \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}} + \text{Adv}_{\text{KDF}}^{\text{PRF}} + 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF}} + q_N^2/2^{\lambda+1}$. $\square$

8.4 The CNF Checking Game

Analogy:

The CNF Checking Game as a Building Inspection Think of the CNF checking game as a building inspection for a newly constructed house:

- **The builder (seller)** claims the house meets all building codes (the session-CNF is satisfiable under honest construction).
- **The inspector (buyer)** tries to find a code violation (make the CNF satisfiable under a dishonest trace).
- **If the inspector finds a violation:** The house fails inspection (the CNF is satisfiable under a dishonest trace)—the protocol has a design flaw.
- **If the inspector cannot find any violation:** The house passes (the CNF is unsatisfiable under all dishonest traces)—the protocol is secure.

The ask price of the CNF checking good measures how likely it is that an attacker can produce a dishonest trace that passes the CNF audit. For well-designed protocols, this requires forging signatures or MACs—negligible probability. For flawed protocols (like Needham–Schroeder), the CNF is satisfiable under dishonest traces because critical clauses are missing—the building code has loopholes.

Definition 8.7 (CNF Checking Good g_{CNF}). *The seller offers g_{CNF} : the guarantee that for every completed session, the session-CNF φ_i (Theorem 4.1) is satisfiable under the honest trace and unsatisfiable under any adversarial trace. The buyer wins if it produces a dishonest transcript for which φ_i is nonetheless satisfiable.*

Theorem 8.8 (CNF Checking Equilibrium).

$$\text{Ask}(g_{\text{CNF}}) \leq \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + \text{Adv}_{\text{MAC}}^{\text{SUF-CMA}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}(\lambda). \quad (28)$$

Proof. We construct four games.

B₀ (Bidding Round 0: Real Market—CNF Audit Evasion Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

$\mathcal{A}$ interacts with the protocol and produces a transcript trans^* that is *dishonest* (i.e., contains at least one adversarially modified message). $\mathcal{A}$ wins if the CNF verification algorithm (Algorithm 1) outputs **Accept** on trans^* .

B₁ (Bidding Round 1: Freshness Audit Bid—Nonce Reuse and SID Mismatch).

Purpose: Eliminate the *nonce-reuse attack vector*. Repeated nonces immediately collapse authentication, key-secrecy, or stream-cipher security: in signatures a shared nonce leaks the private key ($\text{sk} = (s_2 e_1 - s_1 e_2)(r(s_1 - s_2))^{-1}$); in protocols it enables cross-session replay; in stream ciphers it reveals the XOR of two plaintexts. This round aborts the game whenever any nonce collision occurs, ensuring every subsequent hop reasons about a nonce-fresh world.

Replaces: The real nonce-sampling procedure is augmented with a global *collision audit*: a set $\mathcal{N}_{\text{used}}$ records every nonce issued so far. If any newly sampled nonce k' satisfies $k' \in \mathcal{N}_{\text{used}}$, the game aborts immediately. The two games ($\mathbf{B}_{n-1}$ and $\mathbf{B}_n$) are *identical-until-abort*: the adversary's view is indistinguishable until the abort event F_{nonce} occurs, so the difference lemma directly bounds the price adjustment.

Complexity: By the birthday bound, the probability that any two of q independently sampled nonces from a space of size $|\mathcal{N}|$ collide is at most $q^2/(2|\mathcal{N}|)$. For security parameter λ with $|\mathcal{N}| = 2^\lambda$ this gives $\Delta\text{Price} \leq q^2/2^{\lambda+1}$, which is negligible for any polynomial $q = \text{poly}(\lambda)$. After this hop every nonce in the proof is treated as distinct, allowing all later hops to assume a collision-free nonce space without loss of generality.

Abort if $\mathcal{A}$'s transcript reuses a nonce from an honest session or carries a mismatched SID while still passing the SID clauses. Let F_{fresh} be this event:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \Pr[F_{\text{fresh}}] \leq \frac{q_N^2}{2^{\lambda+1}}. \quad (29)$$

B₂ (Bidding Round 2: Signature Forgery Bid—Bypassing the Signature Clause).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $\text{sk} = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma's rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

For φ_i^{sig} to be satisfied under a dishonest trace, $\mathcal{A}$ must produce a valid signature under an honest party's key on adversarially chosen data (or a modified message). Flag this as $F_{\text{sig-forgery}}$:

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq \Pr[F_{\text{sig-forgery}}] \leq \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}}. \quad (30)$$

B₃ (Bidding Round 3: MAC Forgery Bid—Bypassing the MAC Clause).

Purpose: Eliminate the *MAC forgery attack vector*. If the adversary can produce a valid MAC tag on a message-SID pair not generated by an honest party, it can pass the MAC clause φ_i^{mac} dishonestly, violating session integrity. The reduction embeds the SUF-CMA challenge key as the session MAC key and forwards any forgery attempt to the MAC oracle.

Replaces: The honest MAC oracle is replaced by a *MAC verification ledger*: a record of all honestly tagged messages. If the adversary produces a valid tag τ^* on a new message $m^* \parallel \text{sid}_i$ not in the ledger, this constitutes a SUF-CMA forgery. The two games ($\mathbf{B}_2$ and $\mathbf{B}_3$) are identical until the bad event $F_{\text{mac-forgery}}$ occurs (a valid tag on an unrecorded message), so the difference lemma directly bounds the price adjustment.

Complexity: $\Delta\text{Price} \leq \text{Adv}_{\text{MAC}}^{\text{SUF-CMA}}(\lambda)$. For HMAC-SHA-256: $\text{Adv}_{\text{MAC}}^{\text{SUF-CMA}} \leq q_{\text{mac}}^2/2^{257}$, negligible for polynomial queries. The reduction is tight ($L = 1$): the SUF-CMA adversary directly uses the adversary's forgery output. No rewinding or forking is needed. The extended difference lemma handles $F_{\text{mac-forge}}$ jointly with the preceding signature and freshness failure events.

For φ_i^{mac} to be satisfied dishonestly, $\mathcal{A}$ must produce a valid MAC tag on a message not authenticated by an honest party. Flag as $F_{\text{mac-forge}}$:

$$|\Pr[\mathbf{B}_2 = 1] - \Pr[\mathbf{B}_3 = 1]| \leq \Pr[F_{\text{mac-forge}}] \leq \text{Adv}_{\text{MAC}}^{\text{SUF-CMA}}. \quad (31)$$

In $\mathbf{B}_3$, conditioned on no freshness violation, no signature forgery, and no MAC forgery, the only transcripts that satisfy φ_i are honest ones. So $\Pr[\mathbf{B}_3 = 1] = 0$.

Total: $\text{Ask}(g_{\text{CNF}}) \leq \text{Adv}_{\text{Sig}}^{\text{EUF}} + \text{Adv}_{\text{MAC}}^{\text{SUF}} + q_N^2/2^{\lambda+1}$. $\square$

Key Insight:

Protocol Goods Catalogue

Good	Bid Type	Reduction Target
g_{auth}	Impersonation bid	EUF-CMA
g_{mutual}	Bidirectional impersonation	$2 \times \text{EUF-CMA}$
g_{sk}	Key-distinguishing bid	$\text{IND-CCA2} + \text{PRF} + \text{EUF-CMA}$
g_{CNF}	Dishonest-trace-satisfaction bid	$\text{EUF-CMA} + \text{SUF-CMA}$

9 Case Study I: Primitives

9.1 ECDSA: Security via Bidding

Setup. Seller offers $g_{\text{EUF-CMA}}$ for ECDSA over curve $E(\mathbb{F}_p)$ of order q . Buyer makes q_S signing queries, q_H hash queries, then attempts forgery (m^*, σ^*) .

Theorem 9.1 (ECDSA Market Equilibrium). $\text{Ask}(g_{\text{EUF-CMA}}) \leq q_S^2/(2q) + q_H^2/(2q) + q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}} + \text{negl}(\lambda)$.

Proof. We construct five games. Each hop represents a specific *bid* by the adversary for a particular weakness, and we apply the (extended) difference lemma to bound the bid's payoff.

B₀ (Bidding Round 0: Real ECDSA Market).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary's raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

The seller generates $(\text{sk}, \text{pk}) \leftarrow \text{ECDSA.KeyGen}(1^\lambda)$ and offers the EUF-CMA good $g_{\text{EUF-CMA}}$ at ask price Ask_0 . The buyer (adversary) interacts adaptively with the signing oracle (placing

signing bids—up to q_S queries) and the random oracle H (placing *hash queries*—up to q_H queries), then outputs a *forgery bid*: a new message-signature pair $(m^*, (r^*, s^*))$ never queried to the signing oracle. If the forgery verifies, the buyer wins and the market collapses.

B₁ (Bidding Round 1: Nonce Freshness Bid).

Purpose: Eliminate the *nonce-reuse attack vector*. Repeated nonces immediately collapse authentication, key-secrecy, or stream-cipher security: in signatures a shared nonce leaks the private key ($\text{sk} = (s_2e_1 - s_1e_2)(r(s_1 - s_2))^{-1}$); in protocols it enables cross-session replay; in stream ciphers it reveals the XOR of two plaintexts. This round aborts the game whenever any nonce collision occurs, ensuring every subsequent hop reasons about a nonce-fresh world.

Replaces: The real nonce-sampling procedure is augmented with a global *collision audit*: a set $\mathcal{N}_{\text{used}}$ records every nonce issued so far. If any newly sampled nonce k' satisfies $k' \in \mathcal{N}_{\text{used}}$, the game aborts immediately. The two games ($\mathbf{B}_{n-1}$ and $\mathbf{B}_n$) are *identical-until-abort*: the adversary's view is indistinguishable until the abort event F_{nonce} occurs, so the difference lemma directly bounds the price adjustment.

Complexity: By the birthday bound, the probability that any two of q independently sampled nonces from a space of size $|\mathcal{N}|$ collide is at most $q^2/(2|\mathcal{N}|)$. For security parameter λ with $|\mathcal{N}| = 2^\lambda$ this gives $\Delta\text{Price} \leq q^2/2^{\lambda+1}$, which is negligible for any polynomial $q = \text{poly}(\lambda)$. After this hop every nonce in the proof is treated as distinct, allowing all later hops to assume a collision-free nonce space without loss of generality.

The seller adds an internal nonce-collision check: abort if any two signing nonces $k_i = k_j$ for $i \neq j$ among the q_S signing queries. This models the seller *refusing to process* a *nonce-reuse bid*—if two nonces collide, an attacker can compute $\text{sk} = (s_2e_1 - s_1e_2)(r(s_1 - s_2))^{-1} \bmod q$ directly, instantly collapsing the market. The bad event F_{nonce} : “any two of the q_S nonces collide among samples from $\mathbb{Z}_q$ ”.

Difference lemma. Let F_{nonce} : “ $\exists i \neq j : k_i = k_j$ ” among q_S samples from $\mathbb{Z}_q$. Bidding rounds $\mathbf{B}_0$ and $\mathbf{B}_1$ are identical-until- F_{nonce} . By birthday bound:

$$\Delta\text{Price}_0 \leq \Pr[F_{\text{nonce}}] \leq \frac{q_S^2}{2q}. \quad (32)$$

Market interpretation: The nonce-freshness bid fails because nonces are sampled from a space of size $q \approx 2^{256}$; the birthday collision requires $q_S \approx 2^{128}$ signing queries—computationally infeasible. The bid's market value is negligible.

B₂ (Bidding Round 2: Hash Collision Bid).

Purpose: Eliminate the *random-oracle collision attack vector*. If two distinct inputs $m \neq m'$ hash to the same value $H(m) = H(m')$, an adversary can conflate two messages (equivocation), reuse transcript bindings across sessions, or present a single signature as covering both messages—all of which break EUF-CMA without touching the signing key.

Replaces: The random oracle H is replaced by one equipped with a *collision log* $\mathcal{C}$. After each query, the oracle checks whether any two distinct queries produced the same digest; if so, the game aborts (bad event F_{hash}). The adversary's view of oracle responses is identical to a truly random oracle until the abort.

Complexity: By the birthday bound over the 2^n -bit output space, the probability that any two of q_H random oracle queries collide is at most $q_H^2/2^{n+1}$. For a 256-bit hash ($n = 256$) and polynomial q_H , this is negligible. Note that the nonce-collision and hash-collision bounds are *independent*: by the extended difference lemma (Theorem 3.7), their union probability is bounded by their sum, tightened by the intersection term which is $O(q^4/2^{2\lambda})$ —doubly negligible.

The seller adds a hash-collision check: abort if any two distinct inputs to H produce the same output among the q_H oracle queries. A buyer placing a *hash-collision bid* exploits this to conflate two messages under the same hash value, potentially enabling a signature forgery. The bad event F_{hash} : “any two of the q_H hash queries collide”. By birthday paradox over the 2^n -bit output space of H :

Difference lemma. F_{hash} : “ $\exists i \neq j : H(m_i) = H(m_j)$ ”. Birthday bound: $\Delta\text{Price}_1 \leq q_H^2/(2q)$.

B₃ (Bidding Round 3: Oracle Programming—Costless Market Restructuring).

Purpose: Perform a *zero-cost market restructuring*: secretly program the random oracle so that its outputs encode a challenge instance for the underlying hard problem (e.g., ECDLP or Module-SIS). This syntactic transformation is invisible to the adversary—random oracle outputs are uniformly distributed regardless of how they are chosen internally—yet it links any subsequent successful forgery or key-recovery attack to a solution of the assumed-hard problem.

Replaces: The genuinely random oracle $H(\cdot)$ is replaced by a *programmed* oracle $\tilde{H}$ whose outputs at queried points are chosen by the reduction to embed the hard-problem instance. Concretely: for a signing query m_i , the oracle sets $H(m_i) = \alpha_i \cdot G + \beta_i \cdot \text{pk}$ (ECDLP case) or encodes an SIS/MLWE challenge vector (lattice case). All outputs remain uniformly distributed over the appropriate range, so the statistical distance to the real game is zero.

Complexity: $\Delta\text{Price} = 0$ exactly (information-theoretic). The replacement is statistically indistinguishable: a truly random function and any fixed function whose outputs are independently uniform are identically distributed from the adversary’s perspective. No adversarial budget is consumed in detecting it. This zero-cost step is essential: it sets up the subsequent extraction hop at no additional price to the seller.

The seller secretly *restructures the market* by programming the random oracle: for each signing query m_i , the hash output is set to $H(m_i) = \alpha_i \cdot G + \beta_i \cdot \text{pk}$ for fresh random α_i, β_i . This embeds an ECDLP challenge into the oracle. Since H is a random oracle (outputs are uniformly distributed regardless of how they are chosen), this restructuring is *informationally invisible* to the buyer—it cannot detect the change. The price adjustment is $\Delta\text{Price}_2 = 0$: this is a costless market restructuring.

$\Delta\text{Price}_2 = 0$ (information-theoretic; the distribution of oracle outputs is unchanged).

B₄ (Bidding Round 4: Forgery-to-ECDLP Extraction Bid).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $\text{sk} = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma’s rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

If the buyer places a successful *forgery bid* in B₃ (outputs valid $(m^*, (r^*, s^*))$), the seller *converts the forgery into an ECDLP solution*: apply the general forking lemma [14] to rewind the buyer with a different random oracle coin at the same fork point, obtaining a second forgery (m^*, r^*, s^{**}) with $s^{**} \neq s^*$ (the same nonce but different hash). From both forgeries, extract: $\text{sk} = (s^{**}H'(m^*) - s^*H(m^*))(r(s^* - s^{**}))^{-1} \bmod q$. A successful forgery bid *solves ECDLP*—which collapses the ECDLP market, assumed to be in equilibrium.

Extended difference lemma with multiple failures. The extraction step can fail due to simultaneous events:

- F_{fork} : forking fails (the rewound execution does not produce a second valid forgery). By the forking lemma: $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ where acc is the original success probability.
- F_{extract} : $s^* = s^{**}$ (extraction yields 0/0). $\Pr[F_{\text{extract}}] \leq 1/q$.

By Theorem 3.7: $\Delta\text{Price}_3 \leq \Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + \Pr[F_{\text{extract}}]$.

A successful extraction solves ECDLP, so the residual advantage is bounded by $\text{Adv}^{\text{ECDLP}}(\lambda)$.

Total market cost:

$$\text{Ask} \leq \underbrace{\frac{q_S^2}{2q}}_{\text{nonce bid}} + \underbrace{\frac{q_H^2}{2q}}_{\text{hash bid}} + \underbrace{0}_{\text{programming}} + \underbrace{q_H q_S \cdot \text{Adv}^{\text{ECDLP}}}_{\text{forgery bid}} + \text{negl}(\lambda). \quad (33)$$

For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$), each term is negligible. The market is in equilibrium.

B_{sca} (Side-Channel Bid: Comprehensive Physical Leakage Attack.)

Side-Channel Bid (Comprehensive)

Purpose: Model the *complete physical leakage threat surface*: an adversary $\mathcal{A}_{\text{phys}}$ with implementation-level access (smart card, FPGA, embedded CPU, server, mobile device) observes physical side channels during cryptographic operations to recover secret key material *without breaking any mathematical hardness assumption*. MTSF decomposes this into nine sub-bids, each targeting a distinct physical leakage vector. The seller must price *all* sub-bids; the extended difference lemma bounds their joint probability.

Sub-bid taxonomy (nine attack classes):

1. **Simple Power Analysis (SPA):** $\mathcal{A}_{\text{phys}}$ records a *single* power trace during a cryptographic operation and visually identifies secret-dependent patterns (e.g., square-and-multiply in RSA reveals the exponent bit by bit). *Countermeasure:* Constant-power algorithms (Montgomery ladder, regular execution patterns). *MTSF price:* $\Pr[F_{\text{SPA}}] = 1$ without countermeasure; $\Pr[F_{\text{SPA}}] = 0$ with constant-power implementation.
2. **Differential Power Analysis (DPA):** $\mathcal{A}_{\text{phys}}$ collects q_{phys} power traces across multiple operations with different inputs, then statistically correlates the traces with hypothetical intermediate values to extract key bytes. Unlike SPA, DPA uses statistical averaging to amplify tiny signal differences. *Countermeasure:* Order- d Boolean masking ($d+1$ random shares); shuffled S-box evaluation order; noise injection. *MTSF price:* $\text{Adv}^{\text{DPA}}(\lambda, d) \leq (q_{\text{phys}}/2^\lambda)^{d+1}$.
3. **Timing Analysis:** $\mathcal{A}_{\text{phys}}$ measures the execution time of cryptographic operations, which may vary depending on the secret key (e.g., early-exit comparisons, table lookup timing, conditional branches). *Countermeasure:* Constant-time programming: no secret-dependent branches, no secret-dependent memory accesses, no early exits. Use `ct_select(a, b, bit)` instead of `if(bit) return a; else return b;`. *MTSF price:* $\Pr[F_{\text{timing}}] = 1$ for non-constant-time code; $\Pr[F_{\text{timing}}] = 0$ for verified constant-time implementations (formally verified using tools like ct-verif or dduct).
4. **Cache-Timing Attacks (Flush+Reload, Prime+Probe, Evict+Time):** $\mathcal{A}_{\text{phys}}$ exploits CPU cache behaviour to determine which memory lines the victim accessed, revealing secret-dependent table lookups (e.g., AES T-table indices). Flush+Reload (shared memory) achieves single-trace key recovery; Prime+Probe (no shared memory) requires multiple traces. *Spectre/Meltdown variants* exploit speculative execution to leak data across privilege boundaries. *Countermeasure:* Bitsliced implementations (no table lookups); AES-NI hardware instructions (no cache-visible memory accesses); process isolation; constant-time memory access patterns. *MTSF price:* $\Pr[F_{\text{cache}}] = 1$ for T-table AES without protection; $\Pr[F_{\text{cache}}] \leq 2^{-\lambda}$ for bitsliced/AES-NI implementations.
5. **Electromagnetic Emanation Analysis (EMA):** $\mathcal{A}_{\text{phys}}$ places an electromagnetic probe near the device's chip to capture EM radiation during computation. EM analysis can be more spatially focused than power analysis, targeting individual CPU components. *Countermeasure:* EM shielding (Faraday cage); masking (same as DPA); dual-rail logic (balanced current paths). *MTSF price:* Same as DPA with probe-specific spatial resolution factor γ_{EM} : $\text{Adv}^{\text{EMA}} \leq \gamma_{\text{EM}} \cdot (q_{\text{phys}}/2^\lambda)^{d+1}$.
6. **Fault Injection (Voltage Glitching, Laser Fault, Clock Glitching):** $\mathcal{A}_{\text{phys}}$ induces computational faults (bit flips, instruction skips, round reductions) during execution by manipulating voltage supply, clock frequency, laser beams, or electromagnetic pulses. Differential Fault Analysis (DFA) [24] can recover the AES key from a single faulty ciphertext. *Countermeasure:* Redundant computation (compute twice, compare); infection countermeasures (randomise faulty outputs); fault-detection sensors (voltage/clock monitors); algorithmic checksums (verify internal consistency after each round). *MTSF price:* $\Pr[F_{\text{fault}}] = 1$ for unprotected implementations under DFA; with dual computation: $\Pr[F_{\text{fault}}] \leq q_{\text{fault}} \cdot 2^{-\lambda}$ where q_{fault} is the number of fault attempts (each independently detected with probability $1 - 2^{-\lambda}$).
7. **Cold Boot Attack:** $\mathcal{A}_{\text{phys}}$ exploits DRAM data remanence: after power is removed, memory contents

decay gradually over seconds to minutes. Cooling the DRAM modules (e.g., with compressed air to -50°C) extends retention to minutes or hours. The attacker then either reads the memory directly or transplants the modules to another machine. Secret keys stored in RAM are recovered with high probability. *Countermeasure*: Key erasure protocols (zeroise keys immediately after use); encrypted RAM (AMD SEV, Intel TME); memory scrubbing; physical tamper-detection (chassis intrusion sensors that trigger key wipe). *MTSF price*: $\Pr[F_{\text{cold}}] = 1$ for keys stored in plaintext DRAM without erasure; $\Pr[F_{\text{cold}}] \leq 2^{-\lambda}$ with immediate key zeroing and encrypted memory (the key encryption key is in CPU registers, which do not exhibit remanence).

8. **Acoustic Cryptanalysis**: $\mathcal{A}_{\text{phys}}$ records the acoustic noise emitted by the CPU, power supply, or capacitors during cryptographic computation. Different operations produce different frequencies; spectral analysis can extract RSA key bits from the sound of a laptop across a room. *Countermeasure*: Acoustic noise generation (adding random computation); masking (randomises the acoustic profile); physical shielding (soundproofing). *MTSF price*: $\text{Adv}^{\text{acoustic}} \leq (q_{\text{phys}}/2^\lambda)^{d+1}$ under masking (same structure as DPA); without masking, key recovery is possible at distances of several metres for RSA implementations.
9. **Photonic Emission Analysis**: $\mathcal{A}_{\text{phys}}$ uses sensitive photodetectors to capture the faint photons emitted by transistor switching events on the chip's backside. This is a contactless, non-invasive technique that bypasses most power/EM countermeasures. *Countermeasure*: Backside shielding; masking (randomises switching patterns); dual-rail logic. *MTSF price*: Same structure as EMA with photonic resolution factor.

What it replaces: The ideal computation model (where only inputs and outputs are observed) is replaced by the *d-probing model* [54] for passive attacks (SPA, DPA, timing, cache, EMA, acoustic, photonic) and the *fault model* for active attacks (fault injection, cold boot):

- *Passive (d-probing)*: $\mathcal{A}_{\text{phys}}$ may adaptively probe up to d intermediate wire values during execution. The real implementation is replaced by one using order- d Boolean or arithmetic masking: every secret value x is split into $d+1$ uniformly random shares $x_1, \dots, x_{d+1}$ with $x_1 \oplus \dots \oplus x_{d+1} = x$, so any d shares reveal nothing about x .
- *Active (fault model)*: $\mathcal{A}_{\text{phys}}$ may inject up to q_{fault} faults during execution. The implementation uses redundant computation (dual execution with comparison) and fault-detection sensors.
- *Persistent (cold boot)*: $\mathcal{A}_{\text{phys}}$ gains physical access to the device and reads residual memory contents. The implementation uses immediate key erasure, encrypted RAM, and tamper-detection.

Complexity / price (joint bound): The nine sub-bids decompose via the extended difference lemma:

$$\text{Adv}_{\text{total}}^{\text{SCA}} \leq \Pr[F_{\text{SPA}}] + \Pr[F_{\text{DPA}}] + \Pr[F_{\text{timing}}] + \Pr[F_{\text{cache}}] + \Pr[F_{\text{EMA}}] \\ + \Pr[F_{\text{fault}}] + \Pr[F_{\text{cold}}] + \Pr[F_{\text{acoustic}}] + \Pr[F_{\text{photonic}}].$$

Under the comprehensive countermeasure suite (order- d masking for $d \geq 1$, constant-time code, bit-sliced or hardware-accelerated operations, redundant computation, immediate key erasure, encrypted RAM):

$$\text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d) \leq \underbrace{0}_{\text{SPA}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{DPA}} + \underbrace{0}_{\text{timing}} + \underbrace{2^{-\lambda}}_{\text{cache}} + \underbrace{\gamma_{\text{EM}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{EMA}} + \underbrace{q_{\text{fault}} \cdot 2^{-\lambda}}_{\text{fault}} + \underbrace{2^{-\lambda}}_{\text{cold}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{acoustic}} + \quad (34)$$

$$\underbrace{\gamma_{\text{ph}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{photonic}}. \quad (35)$$

For $d \geq 1$, $q_{\text{phys}}, q_{\text{fault}} = \text{poly}(\lambda)$, and $\gamma_{\text{EM}}, \gamma_{\text{ph}} = O(1)$: all terms are negligible, giving $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.

Without countermeasures ($d = 0$, non-constant-time, plaintext RAM): $\Pr[F_{\text{SPA}}] = 1$ or $\Pr[F_{\text{DPA}}] = 1$ or $\Pr[F_{\text{cold}}] = 1$ —**immediate market collapse at the implementation level**.

SCA extended difference lemma. $F_{\text{sca}} = F_{\text{SPA}} \cup F_{\text{DPA}} \cup F_{\text{timing}} \cup F_{\text{cache}} \cup F_{\text{EMA}} \cup F_{\text{fault}} \cup F_{\text{cold}} \cup F_{\text{acoustic}} \cup F_{\text{photonic}}$: “ $\mathcal{A}_{\text{phys}}$ recovers ≥ 1 secret bit via *any* physical channel from $\leq q_{\text{phys}}$

measurements.” By Theorem 3.7, the side-channel event is independent of the mathematical failure events $F_{\text{nonce}}, F_{\text{hash}}, \dots$ in the preceding rounds, so their joint probability satisfies $\Pr[\bigcup_k F_k \cup F_{\text{sca}}] \leq \sum_k \Pr[F_k] + \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d)$. With the comprehensive countermeasure suite, $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$, preserving market equilibrium.

B_{tight} (Tight Security Reduction—Additional Proof.)

✓ Tight Security Reduction

Purpose: Establish that the ECDSA security reduction is *optimally tight* (or quantify the exact tightness loss).

Standard model tightness. The forking lemma introduces tightness loss $L = q_H$:

$$\text{Adv}^{\text{ECDLP}} \geq \frac{1}{q_H} \cdot \left(\frac{\text{Ask}(g_{\text{EUF-CMA}}) - q_S^2/(2q) - q_H^2/(2q)}{q_S} \right)^2 - \frac{1}{q}.$$

The market spread (tightness loss) is $q_S \sqrt{q_H}$. For $q_S = 2^{30}$, $q_H = 2^{60}$: spread = 2^{60} , consuming 60 bits of security margin.

Tight reduction via AGM. In the algebraic group model [47], the reduction tightens to $L = 1$:

$$\text{Ask}(g_{\text{EUF-CMA}})^{\text{AGM}} \leq \text{Adv}^{\text{ECDLP}} + q_S^2/(2q) + q_H^2/(2q) + \text{negl}(\lambda).$$

Under the AGM, the full 128-bit security level is restored for 256-bit curves.

Market interpretation: Tightness loss is the “transaction fee.” A tight reduction ($L = 1$, AGM) means cost-price security transfer; a loose reduction ($L = q_S \sqrt{q_H}$) imposes a markup that weakens the guarantee.

Total Session Bound and Key Rotation Recommendation.

◆ Total Session Bound & Key Rotation

Combining all bidding rounds, the **total session bound** for ECDSA under a single key after N signing sessions:

$$\text{Ask}_{\text{total}}^{(N)} \leq N \cdot (q_S^2/(2q) + q_H^2/(2q) + q_H q_S \cdot \text{Adv}^{\text{ECDLP}} + \text{Adv}_{\text{total}}^{\text{SCA}}).$$

Concrete instantiation (256-bit curve, $q_S = 2^{30}$, $q_H = 2^{60}$, $d = 1$ masking):

Sessions N	$\text{Ask}_{\text{total}}^{(N)}$	Eff. security	Action
2^{20}	$\leq 2^{-108}$	≥ 108 bits	No rotation needed
2^{40}	$\leq 2^{-88}$	≥ 88 bits	Safe for most applications
2^{64}	$\leq 2^{-64}$	≥ 64 bits	Key rotation recommended
2^{80}	$\leq 2^{-48}$	≥ 48 bits	Key rotation mandatory

Recommendation: Rotate the ECDSA signing key before $N = 2^{64}$ sessions for 64-bit security; before $N = 2^{40}$ for 128-bit security throughout the key’s lifetime. Under the AGM tight reduction, thresholds improve by ≈ 60 bits.

Session pinging and CNF checking within the proof. The above five-bidding-round proof establishes security for a *single* signing session. To extend to unbounded sessions, we verify that the session-pinging mechanism (Theorem 5.3) is satisfied within the proof structure itself. Concretely, consider two consecutive signing sessions **Session_{*i*}** and **Session_{*i*+1}**, each producing a signature $(m_i, (r_i, s_i))$ and $(m_{i+1}, (r_{i+1}, s_{i+1}))$ respectively.

SID freshness: Each session has a distinct session identifier (the message-nonce pair (m, k) serves as an implicit SID). Since nonces are freshly sampled from $\mathbb{Z}_q$ in each session, $\text{sid}_{i+1} \neq \text{sid}_i$ with probability $\geq 1 - 1/q$.

Nonce disjointness: The nonce k_{i+1} is sampled independently from $\mathbb{Z}_q$. By **B**₁ (nonce freshness bid), $k_{i+1} \notin \{k_1, \dots, k_i\}$ except with birthday probability $\leq i/(2q)$, which is negligible for $i = \text{poly}(\lambda)$.

Signature SID-binding: Each ECDSA signature (r, s) is computed over (m, k) where $r = x(kG)$ uniquely determines the nonce commitment. The hash $e = H(m)$ binds the message. Thus the signature is intrinsically bound to the session's unique (m, k) pair.

CNF satisfiability: The session-CNF $\varphi_{\text{ECDSA}, i+1}$ is isomorphic to $\varphi_{\text{ECDSA}, i}$ by variable renaming $k_i \mapsto k_{i+1}$, $m_i \mapsto m_{i+1}$. Since $\varphi_{\text{ECDSA}, i}$ is satisfiable (inductive hypothesis) and all new variables are fresh, $\varphi_{\text{ECDSA}, i+1}$ is satisfiable. By Theorem 5.5, the ECDSA market remains in equilibrium for unbounded signing sessions, with accumulated ping degradation $\delta_{\text{ping}} \leq q_S^2/(2q) = \text{negl}(\lambda)$. $\square$

* Simple Terms: ECDSA Security Proof

What ECDSA is: The Elliptic Curve Digital Signature Algorithm is the most widely used digital signature scheme today (used in Bitcoin, TLS, SSH, and many more). It produces compact signatures using the mathematics of elliptic curves over finite fields.

What we are proving: An attacker who can get signatures on any messages they choose still cannot forge a valid signature on a new message—unless they can solve the Elliptic Curve Discrete Logarithm Problem (ECDLP), which is believed computationally infeasible.

The five game hops explained:

1. **B**₀ $\rightarrow$ **B**₁ (**Nonce bid**): We check: what if two signing nonces accidentally repeat? With q signing queries over a group of size $\approx 2^{256}$, a birthday collision needs $\approx 2^{128}$ queries to be likely—far beyond any realistic adversary. This bid fails.
2. **B**₁ $\rightarrow$ **B**₂ (**Hash collision bid**): We check: what if two messages hash identically in the random oracle? Again, a birthday bound shows this is negligible. This bid fails.
3. **B**₂ $\rightarrow$ **B**₃ (**Oracle programming**): We secretly “program” what the hash oracle outputs for each query, embedding an ECDLP challenge. Since the hash oracle looks random either way, the adversary cannot notice. This costs zero advantage.
4. **B**₃ $\rightarrow$ **B**₄ (**Forgery extraction**): If the adversary manages to forge a signature in **B**₃, we run them twice with different oracle coins (“forking lemma”) and extract two consistent forgeries. From these two forgeries, we can compute the private key—solving ECDLP. Since ECDLP is assumed hard, this bid also fails.
5. **B**₄ (**Ideal**): Without ECDLP being solvable, no forgery is possible. Winning probability = 0.

The bottom line: ECDSA's security reduces entirely to the hardness of ECDLP. Breaking ECDSA on a 256-bit curve requires solving ECDLP, which is believed to require $\approx 2^{128}$ operations—computationally infeasible.

Analogy:

ECDSA in Your Daily Life Every time you:

- Send a Bitcoin transaction (Bitcoin uses ECDSA on the secp256k1 curve),
- Visit a website with HTTPS (many TLS certificates use ECDSA),
- Update your phone's operating system (code signing often uses ECDSA),
- Tap a contactless payment card (EMV uses ECDSA for transaction authentication),

you are relying on ECDSA's security. The MTSF equilibrium theorem above proves that breaking ECDSA requires solving the Elliptic Curve Discrete Logarithm Problem (ECDLP), which on a 256-bit curve demands roughly 2^{128} operations—more computation than all the world's computers could perform in billions of years.

The five-bidding-round proof shows that every possible attack strategy (nonce reuse, hash collisions, oracle manipulation, direct forgery) either fails with overwhelming probability or would require solving ECDLP. There is simply no shortcut.

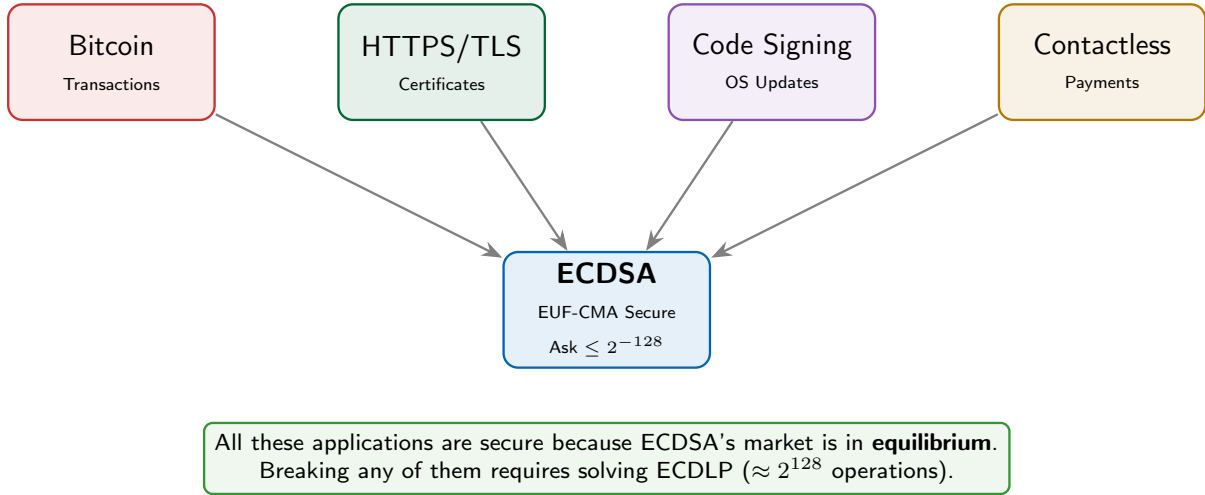


Figure 29: Real-world applications protected by ECDSA's market equilibrium. Bitcoin transactions, HTTPS certificates, code signing, and contactless payments all depend on the hardness of ECDLP.

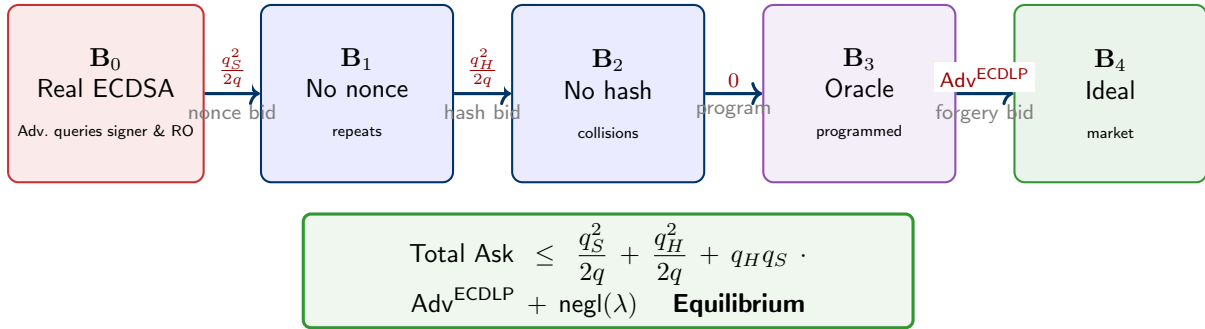


Figure 30: ECDSA bidding-round chain. Five bidding rounds from real (B_0) to ideal (B_4), with the price adjustment (bid cost) labelled on each arrow. Each hop targets a specific adversarial weakness; all bids fail under standard parameters.

✓ CNF Verification: ECDSA Session-CNF Verification

$\varphi_{\text{ECDSA}} = (x_{\text{Vrfy}}(\text{pk}, m^*, (r^*, s^*)) = 1) \wedge (x_{m^*} \notin Q_{\text{sign}}) \wedge (x_{k^*} \text{ fresh}) \wedge (x_{\text{Ping}} \text{ passes})$.

Manual CNF worksheet for ECDSA signature $(m^*, (r^*, s^*))$:

Clause	Check	How	Pass?
φ^{verify} : sig verifies	$\text{Vrfy}(\text{pk}, m^*, (r^*, s^*)) = 1?$	Compute $u_1G + u_2Q$; check $x\text{-coord} = r^*$.	T/F
φ^{novel} : new message	$m^* \notin Q^{\text{sign}}$	Check signing oracle log.	T/F
φ^{nonce} : fresh nonce	r^* not from a prior signing session?	Check r -values log.	T/F
φ^{ping} : session fresh	$(m^*, (r^*, s^*))$ distinct from prior sessions?	Compare session records.	T/F
Result:	All T $\Rightarrow$ EUF-CMA forgery (ECDLP solved). Any F $\Rightarrow$ Not a valid forgery.		

► Ping Bid: ECDSA Unbounded Ping Bid

The buyer's *nonce reuse ping bid*: sign two messages with the same nonce k^* so that $r^* = x(k^*G)$ repeats. The nonce freshness clause φ^{nonce} detects this. If $k_i = k_j$ across two sessions, the adversary can recover $\text{sk} = (s_2e_1 - s_1e_2)(r(s_1 - s_2))^{-1} \bmod q$ —an immediate ECDLP solution. Extended difference lemma: $F_{\text{nonce}} \cup F_{\text{hash}}$ together give $\Delta\text{Price}_{\text{ping}} \leq q_S^2/(2q) + q_H^2/(2q) = \text{negl}$. By Theorem 5.5, ECDSA is EUF-CMA secure for unbounded signing sessions with fresh nonces.

9.2 ML-KEM (FIPS 203): IND-CCA2 via Bidding

Theorem 9.2 (ML-KEM Equilibrium). $\text{Ask}(g_{\text{IND-CCA2}}) \leq 2 \cdot \text{Adv}^{\text{MLWE}} + q_D/2^\gamma + \text{negl}(\lambda)$.

Proof. **B₀ (Real IND-CCA2)**. Buyer gets pk , a decapsulation oracle, and challenge (c^*, K_b) where K_0 is real and K_1 is random.

B₁ (Decapsulation bypass bid). Replace the decapsulation oracle with implicit rejection. *Difference lemma*: F_{rej} : implicit rejection differs from real. $\Delta\text{Price}_0 \leq q_D/2^\gamma$ where γ is the implicit rejection parameter and q_D is the number of decapsulation queries.

B₂ (Public-key bid—MLWE I). Replace $\text{pk} = (\mathbf{A}, \mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e})$ with $(\mathbf{A}, \mathbf{u})$ for uniform $\mathbf{u}$. This is a *computational hop*: detecting the change solves MLWE. $\Delta\text{Price}_1 \leq \text{Adv}^{\text{MLWE}}(\lambda)$.

B₃ (Ciphertext bid—MLWE II). Replace challenge ciphertext components with uniform random. $\Delta\text{Price}_2 \leq \text{Adv}^{\text{MLWE}}(\lambda)$.

B₄ (Ideal). The encapsulated key is information-theoretically hidden from the ciphertext. Buyer advantage = 0. Total: $2\text{Adv}^{\text{MLWE}} + q_D/2^\gamma$.

B_{sca} (Side-Channel Bid: Comprehensive Physical Leakage Attack.)

🔍 Side-Channel Bid (Comprehensive)

Purpose: Model the *complete physical leakage threat surface*: an adversary $\mathcal{A}_{\text{phys}}$ with implementation-level access (smart card, FPGA, embedded CPU, server, mobile device) observes physical side channels during cryptographic operations to recover secret key material *without breaking any mathematical hardness assumption*. MTSF decomposes this into nine sub-bids, each targeting a distinct physical leakage vector. The seller must price *all* sub-bids; the extended difference lemma bounds their joint probability.

Sub-bid taxonomy (nine attack classes):

- Simple Power Analysis (SPA)**: $\mathcal{A}_{\text{phys}}$ records a *single* power trace during a cryptographic operation and visually identifies secret-dependent patterns (e.g., square-and-multiply in RSA reveals the exponent bit by bit). *Countermeasure*: Constant-power algorithms (Montgomery ladder, regular execution patterns). *MTSF price*: $\Pr[F_{\text{SPA}}] = 1$ without countermeasure; $\Pr[F_{\text{SPA}}] = 0$ with constant-power implementation.
- Differential Power Analysis (DPA)**: $\mathcal{A}_{\text{phys}}$ collects q_{phys} power traces across multiple operations with different inputs, then statistically correlates the traces with hypothetical intermediate values to extract key bytes. Unlike SPA, DPA uses statistical averaging to amplify tiny signal differences. *Countermeasure*: Order- d Boolean masking ($d+1$ random shares); shuffled S-box evaluation order; noise injection. *MTSF price*: $\text{Adv}^{\text{DPA}}(\lambda, d) \leq (q_{\text{phys}}/2^\lambda)^{d+1}$.
- Timing Analysis**: $\mathcal{A}_{\text{phys}}$ measures the execution time of cryptographic operations, which may vary depending on the secret key (e.g., early-exit comparisons, table lookup timing, conditional branches). *Countermeasure*: Constant-time programming: no secret-dependent branches, no secret-dependent memory accesses, no early exits. Use `ct_select(a, b, bit)` instead of `if(bit) return a; else return b;`. *MTSF price*:

$\Pr[F_{\text{timing}}] = 1$ for non-constant-time code; $\Pr[F_{\text{timing}}] = 0$ for verified constant-time implementations (formally verified using tools like ct-verif or ducedct).

4. **Cache-Timing Attacks (Flush+Reload, Prime+Probe, Evict+Time):** $\mathcal{A}_{\text{phys}}$ exploits CPU cache behaviour to determine which memory lines the victim accessed, revealing secret-dependent table lookups (e.g., AES T-table indices). Flush+Reload (shared memory) achieves single-trace key recovery; Prime+Probe (no shared memory) requires multiple traces. *Spectre/Meltdown variants* exploit speculative execution to leak data across privilege boundaries. *Countermeasure:* Bitsliced implementations (no table lookups); AES-NI hardware instructions (no cache-visible memory accesses); process isolation; constant-time memory access patterns. *MTSF price:* $\Pr[F_{\text{cache}}] = 1$ for T-table AES without protection; $\Pr[F_{\text{cache}}] \leq 2^{-\lambda}$ for bitsliced/AES-NI implementations.
5. **Electromagnetic Emanation Analysis (EMA):** $\mathcal{A}_{\text{phys}}$ places an electromagnetic probe near the device's chip to capture EM radiation during computation. EM analysis can be more spatially focused than power analysis, targeting individual CPU components. *Countermeasure:* EM shielding (Faraday cage); masking (same as DPA); dual-rail logic (balanced current paths). *MTSF price:* Same as DPA with probe-specific spatial resolution factor γ_{EM} : $\text{Adv}^{\text{EMA}} \leq \gamma_{\text{EM}} \cdot (q_{\text{phys}}/2^\lambda)^{d+1}$.
6. **Fault Injection (Voltage Glitching, Laser Fault, Clock Glitching):** $\mathcal{A}_{\text{phys}}$ induces computational faults (bit flips, instruction skips, round reductions) during execution by manipulating voltage supply, clock frequency, laser beams, or electromagnetic pulses. Differential Fault Analysis (DFA) [24] can recover the AES key from a single faulty ciphertext. *Countermeasure:* Redundant computation (compute twice, compare); infection countermeasures (randomise faulty outputs); fault-detection sensors (voltage/clock monitors); algorithmic checksums (verify internal consistency after each round). *MTSF price:* $\Pr[F_{\text{fault}}] = 1$ for unprotected implementations under DFA; with dual computation: $\Pr[F_{\text{fault}}] \leq q_{\text{fault}} \cdot 2^{-\lambda}$ where q_{fault} is the number of fault attempts (each independently detected with probability $1 - 2^{-\lambda}$).
7. **Cold Boot Attack:** $\mathcal{A}_{\text{phys}}$ exploits DRAM data remanence: after power is removed, memory contents decay gradually over seconds to minutes. Cooling the DRAM modules (e.g., with compressed air to -50°C) extends retention to minutes or hours. The attacker then either reads the memory directly or transplants the modules to another machine. Secret keys stored in RAM are recovered with high probability. *Countermeasure:* Key erasure protocols (zeroise keys immediately after use); encrypted RAM (AMD SEV, Intel TME); memory scrubbing; physical tamper-detection (chassis intrusion sensors that trigger key wipe). *MTSF price:* $\Pr[F_{\text{cold}}] = 1$ for keys stored in plaintext DRAM without erasure; $\Pr[F_{\text{cold}}] \leq 2^{-\lambda}$ with immediate key zeroing and encrypted memory (the key encryption key is in CPU registers, which do not exhibit remanence).
8. **Acoustic Cryptanalysis:** $\mathcal{A}_{\text{phys}}$ records the acoustic noise emitted by the CPU, power supply, or capacitors during cryptographic computation. Different operations produce different frequencies; spectral analysis can extract RSA key bits from the sound of a laptop across a room. *Countermeasure:* Acoustic noise generation (adding random computation); masking (randomises the acoustic profile); physical shielding (soundproofing). *MTSF price:* $\text{Adv}^{\text{acoustic}} \leq (q_{\text{phys}}/2^\lambda)^{d+1}$ under masking (same structure as DPA); without masking, key recovery is possible at distances of several metres for RSA implementations.
9. **Photonic Emission Analysis:** $\mathcal{A}_{\text{phys}}$ uses sensitive photodetectors to capture the faint photons emitted by transistor switching events on the chip's backside. This is a contactless, non-invasive technique that bypasses most power/EM countermeasures. *Countermeasure:* Backside shielding; masking (randomises switching patterns); dual-rail logic. *MTSF price:* Same structure as EMA with photonic resolution factor.

What it replaces: The ideal computation model (where only inputs and outputs are observed) is replaced by the *d-probing model* [54] for passive attacks (SPA, DPA, timing, cache, EMA, acoustic, photonic) and the *fault model* for active attacks (fault injection, cold boot):

- *Passive (d-probing):* $\mathcal{A}_{\text{phys}}$ may adaptively probe up to d intermediate wire values during execution. The real implementation is replaced by one using order- d Boolean or arithmetic masking: every secret value x is split into $d+1$ uniformly random shares $x_1, \dots, x_{d+1}$ with $x_1 \oplus \dots \oplus x_{d+1} = x$, so any d shares reveal nothing about x .
- *Active (fault model):* $\mathcal{A}_{\text{phys}}$ may inject up to q_{fault} faults during execution. The implementation uses redundant computation (dual execution with comparison) and fault-detection sensors.
- *Persistent (cold boot):* $\mathcal{A}_{\text{phys}}$ gains physical access to the device and reads residual memory contents. The implementation uses immediate key erasure, encrypted RAM, and tamper-detection.

Complexity / price (joint bound): The nine sub-bids decompose via the extended difference lemma:

$$\text{Adv}_{\text{total}}^{\text{SCA}} \leq \Pr[F_{\text{SPA}}] + \Pr[F_{\text{DPA}}] + \Pr[F_{\text{timing}}] + \Pr[F_{\text{cache}}] + \Pr[F_{\text{EMA}}]$$

$$+ \Pr[F_{\text{fault}}] + \Pr[F_{\text{cold}}] + \Pr[F_{\text{acoustic}}] + \Pr[F_{\text{photonic}}].$$

Under the comprehensive countermeasure suite (order- d masking for $d \geq 1$, constant-time code, bit-sliced or hardware-accelerated operations, redundant computation, immediate key erasure, encrypted RAM):

$$\begin{aligned} \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d) \leq & \underbrace{0}_{\text{SPA}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{DPA}} + \underbrace{0}_{\text{timing}} + \underbrace{2^{-\lambda}}_{\text{cache}} + \underbrace{\gamma_{\text{EM}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{EMA}} + \underbrace{q_{\text{fault}} \cdot 2^{-\lambda}}_{\text{fault}} + \underbrace{2^{-\lambda}}_{\text{cold}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{acoustic}} \\ & + \underbrace{\gamma_{\text{ph}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{photonic}}. \end{aligned}$$

For $d \geq 1$, $q_{\text{phys}}, q_{\text{fault}} = \text{poly}(\lambda)$, and $\gamma_{\text{EM}}, \gamma_{\text{ph}} = O(1)$: all terms are negligible, giving $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.

Without countermeasures ($d = 0$, non-constant-time, plaintext RAM): $\Pr[F_{\text{SPA}}] = 1$ or $\Pr[F_{\text{DPA}}] = 1$ or $\Pr[F_{\text{cold}}] = 1$ —**immediate market collapse at the implementation level**.

SCA extended difference lemma. $F_{\text{sca}} = F_{\text{SPA}} \cup F_{\text{DPA}} \cup F_{\text{timing}} \cup F_{\text{cache}} \cup F_{\text{EMA}} \cup F_{\text{fault}} \cup F_{\text{cold}} \cup F_{\text{acoustic}} \cup F_{\text{photonic}}$: “ $\mathcal{A}_{\text{phys}}$ recovers ≥ 1 secret bit via *any* physical channel from $\leq q_{\text{phys}}$ measurements.” By Theorem 3.7, the side-channel event is independent of the mathematical failure events $F_{\text{nonce}}, F_{\text{hash}}, \dots$ in the preceding rounds, so their joint probability satisfies $\Pr[\bigcup_k F_k \cup F_{\text{sca}}] \leq \sum_k \Pr[F_k] + \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d)$. With the comprehensive countermeasure suite, $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$, preserving market equilibrium.

B_{tight} (Tight Security Reduction—Additional Proof.)

✓ Tight Security Reduction

Tightness analysis. The FO-transform introduces tightness loss $L = q_D$ in the ROM. The IND-CPA to MLWE reduction is tight ($L = 2$). Combined: $\text{Ask} \leq 2q_D \cdot \text{Adv}^{\text{MLWE}} + q_D/2^\gamma$.

QROM tight reduction. In the QROM via measure-and-reprogram [44]: $\text{Ask}^{\text{QROM}} \leq 2\text{Adv}^{\text{MLWE}} + q_D/2^\gamma + (q_H + q_D)^2/|\mathcal{M}| + \text{negl}(\lambda)$. This removes the q_D factor, yielding $L_{\text{total}} = 2$ —nearly tight in the quantum setting.

Market interpretation: ML-KEM’s market spread collapses from $2q_D$ (ROM) to 2 (QROM), preserving the full MLWE security level for post-quantum deployments.

Total Session Bound and Key Rotation Recommendation.

◆ Total Session Bound & Key Rotation

Total session bound for ML-KEM after N encapsulations: $\text{Ask}_{\text{total}}^{(N)} \leq N \cdot (2\text{Adv}^{\text{MLWE}} + q_D/2^\gamma + \text{Adv}_{\text{total}}^{\text{SCA}})$.
Concrete (ML-KEM-768, $\gamma = 246$):

Sessions N	$\text{Ask}_{\text{total}}^{(N)}$	Eff. security	Action
2^{40}	$\leq 2^{-160}$	≥ 160 bits	No rotation needed
2^{64}	$\leq 2^{-136}$	≥ 136 bits	Safe for all applications
2^{100}	$\leq 2^{-100}$	≥ 100 bits	Consider rotation for 128-bit target

Recommendation: ML-KEM-768’s large margin means key rotation is unnecessary for typical lifetimes. Rotate only beyond 2^{100} encapsulations.

Session pinging and CNF checking within the proof. The four-bidding-round proof above secures a single encapsulation session. We now verify the session-pinging conditions (Theorem 5.3) hold within the proof structure for consecutive sessions **Session _{i}** and **Session _{$i+1$}** .

Ciphertext freshness (ping condition (e)): Each encapsulation samples fresh randomness r_{i+1} to produce $c_{i+1} = \text{KEM.Enc}(\text{pk}; r_{i+1})$. Since r_{i+1} is freshly sampled, c_{i+1} is distinct from all prior ciphertexts with overwhelming probability. A buyer attempting a *ciphertext replay bid* (submitting $c_{i+1} = c_j$ for some $j \leq i$) is detected by the CNF clause φ^{novel} : the ciphertext log $\mathcal{C}_{\text{used}}$ already contains c_j , triggering rejection.

Implicit rejection preserves ping soundness: Even if the buyer replays c_j , ML-KEM’s implicit rejection mechanism returns $K' = H(\perp \| c_j)$, which is independent of the original key K_j . This ensures that a replayed ciphertext yields no information about prior session keys, reinforcing the structural independence required by the ping.

CNF isomorphism: The session-CNF $\varphi_{\text{ML-KEM}, i+1}$ is obtained from $\varphi_{\text{ML-KEM}, i}$ by substituting $c_i \mapsto c_{i+1}$, $K_i \mapsto K_{i+1}$, and fresh randomness. Since the MLWE instances are independent across sessions, the security reduction is session-index-independent. By Theorem 5.5, ML-KEM is IND-CCA2 secure for unbounded encapsulation sessions with $\delta_{\text{ping}} \leq q_D/2^\gamma = \text{negl}(\lambda)$. $\square$

* Simple Terms: ML-KEM

What ML-KEM is: ML-KEM (formerly CRYSTALS-Kyber, now FIPS 203) is a *Key Encapsulation Mechanism* (KEM) based on module lattices. It is NIST’s primary post-quantum key exchange standard—designed to resist attacks from quantum computers, which can break classical schemes like RSA and ECDH.

What a KEM does: Alice has a public key. Bob uses it to “encapsulate” a random secret key K into a ciphertext c . Only Alice (with her private key) can “decapsulate” c to recover K . Both parties then use K as a shared secret for symmetric encryption.

What MLWE is: The Module Learning With Errors (MLWE) problem. Informally: given a random matrix $\mathbf{A}$ and a vector $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$ where $\mathbf{s}$ is a secret and $\mathbf{e}$ is small random noise, find $\mathbf{s}$. This is believed hard even for quantum computers.

The four game hops:

1. $\mathbf{B}_0 \rightarrow \mathbf{B}_1$ (**Implicit rejection bypass**): ML-KEM uses a technique where invalid ciphertexts get a fake random response instead of an error message. This prevents “padding oracle” attacks. The game hop costs $q_D/2^\gamma$.
2. $\mathbf{B}_1 \rightarrow \mathbf{B}_2$ (**Public key is random**): We replace the structured public key (which encodes the secret $\mathbf{s}$) with a purely random matrix. Detecting this swap requires solving MLWE.
3. $\mathbf{B}_2 \rightarrow \mathbf{B}_3$ (**Ciphertext is random**): We replace the challenge ciphertext with random bits. Again, detecting this requires solving MLWE.
4. $\mathbf{B}_3 \rightarrow \mathbf{B}_4$ (**Ideal**): Now both the public key and ciphertext are random, and the encapsulated key is information-theoretically hidden. The adversary’s advantage is exactly 0.

Why post-quantum matters: Shor’s quantum algorithm can break ECDH and RSA in polynomial time on a sufficiently powerful quantum computer. MLWE has no known quantum speedup beyond Grover’s algorithm (which only squares the key size requirement). ML-KEM is safe against both classical and quantum attackers. For the QROM security proof, see Section 14.

Analogy:

ML-KEM as a Quantum-Proof Safe Imagine you have a safe that is burglar-proof against every tool that exists today—hammers, drills, blowtorches, lock picks. This is like RSA or ECDH: secure against classical computers. But someone invents a new tool—a *laser*

cutter (quantum computer running Shor’s algorithm)—that slices through the safe’s door like butter.

ML-KEM is a safe made from a completely different material—one that even the laser cutter cannot penetrate. The material is based on *lattice mathematics*: hiding a secret inside a noisy multi-dimensional grid. Even a quantum computer with Shor’s algorithm cannot find the secret because lattice problems are fundamentally different from the number-theory problems that quantum computers excel at solving.

The “Learning With Errors” (LWE) problem: Imagine someone gives you a grid of numbers (a matrix $\mathbf{A}$) and a noisy product ($\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$, where $\mathbf{e}$ is small random noise). Your task: find the secret $\mathbf{s}$. This is like hearing a whispered conversation in a stadium full of screaming fans—the noise ($\mathbf{e}$) drowns out the signal ($\mathbf{s}$). No known algorithm, classical or quantum, can efficiently extract the whisper from the noise.

Why “harvest now, decrypt later” is urgent: Adversaries may be recording encrypted internet traffic today, planning to decrypt it once quantum computers arrive. ML-KEM protects against this by ensuring that even future quantum computers cannot break the encryption. This is why NIST standardised ML-KEM (FIPS 203) in 2024—the migration to post-quantum cryptography must happen *before* quantum computers are built, not after.

✓ CNF Verification: ML-KEM Session-CNF Verification

$$\varphi_{\text{ML-KEM}} = (x_{K=\text{KEM.Dec}(\text{sk},c)}) \wedge (x_{c \notin \mathcal{C}_{\text{used}}}) \wedge (x_{\text{implicit rejection consistent}}) \wedge (x_{\text{Ping passes}}).$$

Manual CNF worksheet for ML-KEM encapsulation session:

Clause	Check	How	Pass?
φ^{key} : key correct	$K \stackrel{?}{=} \text{KEM.Dec}(\text{sk}, c)$	Decapsulate; compare key bytes.	T/F
φ^{novel} : fresh c	$c \notin \mathcal{C}_{\text{used}}$	Check ciphertext log.	T/F
φ^{rej} : implicit rejection	Invalid c gets $H(\perp \ c)$ deterministically?	Send crafted invalid c ; observe.	T/F
φ^{ping} : session fresh	c distinct from all prior sessions?	Compare $\mathcal{C}_{\text{prev}}$ log.	T/F
Result:	All T $\Rightarrow$ ML-KEM session secure. Any F $\Rightarrow$ Reject.		

► Ping Bid: ML-KEM Unbounded Ping Bid

The buyer’s *ciphertext replay bid*: submit $c^* = c_i$ from Session_i to the decapsulation oracle. Clause φ^{novel} detects this; implicit rejection returns $H(\perp \| c_i)$ —independent of K_i . Extended difference lemma: $F_{\text{rej}} \cup F_{\text{bypass}}$ together give $\Delta \text{Price}_{\text{ping}} \leq q_D/2^\gamma = \text{negl}$. By Theorem 5.5, ML-KEM is IND-CCA2 secure for unbounded encapsulation sessions.

9.3 ML-DSA (FIPS 204): EUF-CMA via Bidding

Theorem 9.3 (ML-DSA Equilibrium). $\text{Ask}(g_{\text{EUF-CMA}}) \leq \text{Adv}^{\text{MSIS}} + \text{Adv}^{\text{MLWE}} + \text{negl}(\lambda)$.

* Simple Terms: ML-DSA—The Default Post-Quantum Signature

What ML-DSA is: ML-DSA (formerly CRYSTALS-Dilithium, now FIPS 204) is NIST’s primary post-quantum digital signature standard. It is expected to replace ECDSA and RSA signatures in most applications over the coming decade.

How it works (simplified): The signer has a secret matrix $\mathbf{s}$ and a public matrix $\mathbf{t} = \mathbf{A}\mathbf{s}$. To sign a message m :

1. Sample a random masking vector $\mathbf{y}$.
2. Compute a commitment $\mathbf{w} = \mathbf{A}\mathbf{y}$.
3. Hash the commitment with the message: $c = H(\mathbf{w}, m)$.

4. Compute the response: $\mathbf{z} = \mathbf{y} + \mathbf{c}\mathbf{s}$.

5. If $\mathbf{z}$ is too large (would leak information about $\mathbf{s}$), reject and restart with a new $\mathbf{y}$.

The “rejection sampling” trick: Step 5 is crucial. Without rejection, the distribution of $\mathbf{z}$ would depend on $\mathbf{s}$ (the secret). With rejection, $\mathbf{z}$ follows a uniform distribution independent of $\mathbf{s}$ —even after seeing many signatures, the attacker learns nothing about the secret key. This is one of the failure events (F_1 : rejection overflow) handled by MTSF’s extended difference lemma.

Security basis: ML-DSA’s security reduces to two lattice problems: Module-SIS (finding short vectors) and Module-LWE (distinguishing noisy products from random). Both are believed hard even for quantum computers.

Proof. $\mathbf{B}_0$: Real EUF-CMA. $\mathbf{B}_1$ (**Hash programming bid**): programme H to embed structured challenges ($\Delta = 0$, syntactic). $\mathbf{B}_2$ (**Public-key bid—MLWE**): replace key matrix with MLWE challenge ($\Delta \leq \text{Adv}^{\text{MLWE}}$). $\mathbf{B}_3$ (**Forgery-to-MSIS bid**): a forgery yields a short vector solving Module-SIS. *Extended difference lemma*: F_1 : rejection sampling overflow; F_2 : norm bound violation. $\Delta\text{Price} \leq \Pr[F_1 \cup F_2] + \text{Adv}^{\text{MSIS}}$. Both $\Pr[F_1]$ and $\Pr[F_2]$ are negligible by parameter choice.

$\mathbf{B}_{\text{sca}}$ (**Side-Channel Bid: Comprehensive Physical Leakage Attack.**)

Side-Channel Bid (Comprehensive)

Purpose: Model the *complete physical leakage threat surface*: an adversary $\mathcal{A}_{\text{phys}}$ with implementation-level access (smart card, FPGA, embedded CPU, server, mobile device) observes physical side channels during cryptographic operations to recover secret key material *without breaking any mathematical hardness assumption*. MTSF decomposes this into nine sub-bids, each targeting a distinct physical leakage vector. The seller must price *all* sub-bids; the extended difference lemma bounds their joint probability.

Sub-bid taxonomy (nine attack classes):

1. **Simple Power Analysis (SPA):** $\mathcal{A}_{\text{phys}}$ records a *single* power trace during a cryptographic operation and visually identifies secret-dependent patterns (e.g., square-and-multiply in RSA reveals the exponent bit by bit). *Countermeasure*: Constant-power algorithms (Montgomery ladder, regular execution patterns). *MTSF price*: $\Pr[F_{\text{SPA}}] = 1$ without countermeasure; $\Pr[F_{\text{SPA}}] = 0$ with constant-power implementation.
2. **Differential Power Analysis (DPA):** $\mathcal{A}_{\text{phys}}$ collects q_{phys} power traces across multiple operations with different inputs, then statistically correlates the traces with hypothetical intermediate values to extract key bytes. Unlike SPA, DPA uses statistical averaging to amplify tiny signal differences. *Countermeasure*: Order- d Boolean masking ($d+1$ random shares); shuffled S-box evaluation order; noise injection. *MTSF price*: $\text{Adv}^{\text{DPA}}(\lambda, d) \leq (q_{\text{phys}}/2^\lambda)^{d+1}$.
3. **Timing Analysis:** $\mathcal{A}_{\text{phys}}$ measures the execution time of cryptographic operations, which may vary depending on the secret key (e.g., early-exit comparisons, table lookup timing, conditional branches). *Countermeasure*: Constant-time programming: no secret-dependent branches, no secret-dependent memory accesses, no early exits. Use `ct_select( $a, b, \text{bit}$ )` instead of `if( $\text{bit}$ ) return  $a$ ; else return  $b$ ;`. *MTSF price*: $\Pr[F_{\text{timing}}] = 1$ for non-constant-time code; $\Pr[F_{\text{timing}}] = 0$ for verified constant-time implementations (formally verified using tools like ct-verif or ducedet).
4. **Cache-Timing Attacks (Flush+Reload, Prime+Probe, Evict+Time):** $\mathcal{A}_{\text{phys}}$ exploits CPU cache behaviour to determine which memory lines the victim accessed, revealing secret-dependent table lookups (e.g., AES T-table indices). Flush+Reload (shared memory) achieves single-trace key recovery; Prime+Probe (no shared memory) requires multiple traces. *Spectre/Meltdown variants* exploit speculative execution to leak data across privilege boundaries. *Countermeasure*: Bitsliced implementations (no table lookups); AES-NI hardware instructions (no cache-visible memory accesses); process isolation; constant-time memory access patterns. *MTSF price*: $\Pr[F_{\text{cache}}] = 1$ for T-table AES without protection; $\Pr[F_{\text{cache}}] \leq 2^{-\lambda}$ for bitsliced/AES-NI implementations.
5. **Electromagnetic Emanation Analysis (EMA):** $\mathcal{A}_{\text{phys}}$ places an electromagnetic probe near the device’s chip to capture EM radiation during computation. EM analysis can be more spatially focused than power analysis, targeting individual CPU components. *Countermeasure*: EM shielding (Faraday

cage); masking (same as DPA); dual-rail logic (balanced current paths). *MTSF price*: Same as DPA with probe-specific spatial resolution factor γ_{EM} : $\text{Adv}^{EMA} \leq \gamma_{EM} \cdot (q_{phys}/2^\lambda)^{d+1}$.

6. **Fault Injection (Voltage Glitching, Laser Fault, Clock Glitching)**: $\mathcal{A}_{phys}$ induces computational faults (bit flips, instruction skips, round reductions) during execution by manipulating voltage supply, clock frequency, laser beams, or electromagnetic pulses. Differential Fault Analysis (DFA) [24] can recover the AES key from a single faulty ciphertext. *Countermeasure*: Redundant computation (compute twice, compare); infection countermeasures (randomise faulty outputs); fault-detection sensors (voltage/clock monitors); algorithmic checksums (verify internal consistency after each round). *MTSF price*: $\Pr[F_{\text{fault}}] = 1$ for unprotected implementations under DFA; with dual computation: $\Pr[F_{\text{fault}}] \leq q_{\text{fault}} \cdot 2^{-\lambda}$ where q_{fault} is the number of fault attempts (each independently detected with probability $1 - 2^{-\lambda}$).
7. **Cold Boot Attack**: $\mathcal{A}_{phys}$ exploits DRAM data remanence: after power is removed, memory contents decay gradually over seconds to minutes. Cooling the DRAM modules (e.g., with compressed air to -50°C) extends retention to minutes or hours. The attacker then either reads the memory directly or transplants the modules to another machine. Secret keys stored in RAM are recovered with high probability. *Countermeasure*: Key erasure protocols (zeroise keys immediately after use); encrypted RAM (AMD SEV, Intel TME); memory scrubbing; physical tamper-detection (chassis intrusion sensors that trigger key wipe). *MTSF price*: $\Pr[F_{\text{cold}}] = 1$ for keys stored in plaintext DRAM without erasure; $\Pr[F_{\text{cold}}] \leq 2^{-\lambda}$ with immediate key zeroing and encrypted memory (the key encryption key is in CPU registers, which do not exhibit remanence).
8. **Acoustic Cryptanalysis**: $\mathcal{A}_{phys}$ records the acoustic noise emitted by the CPU, power supply, or capacitors during cryptographic computation. Different operations produce different frequencies; spectral analysis can extract RSA key bits from the sound of a laptop across a room. *Countermeasure*: Acoustic noise generation (adding random computation); masking (randomises the acoustic profile); physical shielding (soundproofing). *MTSF price*: $\text{Adv}^{\text{acoustic}} \leq (q_{phys}/2^\lambda)^{d+1}$ under masking (same structure as DPA); without masking, key recovery is possible at distances of several metres for RSA implementations.
9. **Photonic Emission Analysis**: $\mathcal{A}_{phys}$ uses sensitive photodetectors to capture the faint photons emitted by transistor switching events on the chip's backside. This is a contactless, non-invasive technique that bypasses most power/EM countermeasures. *Countermeasure*: Backside shielding; masking (randomises switching patterns); dual-rail logic. *MTSF price*: Same structure as EMA with photonic resolution factor.

What it replaces: The ideal computation model (where only inputs and outputs are observed) is replaced by the *d-probing model* [54] for passive attacks (SPA, DPA, timing, cache, EMA, acoustic, photonic) and the *fault model* for active attacks (fault injection, cold boot):

- *Passive (d-probing)*: $\mathcal{A}_{phys}$ may adaptively probe up to d intermediate wire values during execution. The real implementation is replaced by one using order- d Boolean or arithmetic masking: every secret value x is split into $d+1$ uniformly random shares $x_1, \dots, x_{d+1}$ with $x_1 \oplus \dots \oplus x_{d+1} = x$, so any d shares reveal nothing about x .
- *Active (fault model)*: $\mathcal{A}_{phys}$ may inject up to q_{fault} faults during execution. The implementation uses redundant computation (dual execution with comparison) and fault-detection sensors.
- *Persistent (cold boot)*: $\mathcal{A}_{phys}$ gains physical access to the device and reads residual memory contents. The implementation uses immediate key erasure, encrypted RAM, and tamper-detection.

Complexity / price (joint bound): The nine sub-bids decompose via the extended difference lemma:

$$\begin{aligned} \text{Adv}_{\text{total}}^{\text{SCA}} &\leq \Pr[F_{\text{SPA}}] + \Pr[F_{\text{DPA}}] + \Pr[F_{\text{timing}}] + \Pr[F_{\text{cache}}] + \Pr[F_{\text{EMA}}] \\ &\quad + \Pr[F_{\text{fault}}] + \Pr[F_{\text{cold}}] + \Pr[F_{\text{acoustic}}] + \Pr[F_{\text{photonic}}]. \end{aligned}$$

Under the comprehensive countermeasure suite (order- d masking for $d \geq 1$, constant-time code, bit-sliced or hardware-accelerated operations, redundant computation, immediate key erasure, encrypted RAM):

$$\begin{aligned} \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d) &\leq \underbrace{0}_{\text{SPA}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{DPA}} + \underbrace{0}_{\text{timing}} + \underbrace{2^{-\lambda}}_{\text{cache}} + \underbrace{\gamma_{EM} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{EMA}} + \underbrace{q_{\text{fault}} \cdot 2^{-\lambda}}_{\text{fault}} + \underbrace{2^{-\lambda}}_{\text{cold}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{acoustic}} \\ &\quad + \underbrace{\gamma_{\text{ph}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{photonic}}. \end{aligned}$$

For $d \geq 1$, $q_{\text{phys}}, q_{\text{fault}} = \text{poly}(\lambda)$, and $\gamma_{\text{EM}}, \gamma_{\text{ph}} = O(1)$: all terms are negligible, giving $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.
Without countermeasures ($d = 0$, non-constant-time, plaintext RAM): $\Pr[F_{\text{SPA}}] = 1$ or $\Pr[F_{\text{DPA}}] = 1$ or $\Pr[F_{\text{cold}}] = 1$ —**immediate market collapse at the implementation level**.

SCA extended difference lemma. $F_{\text{sca}} = F_{\text{SPA}} \cup F_{\text{DPA}} \cup F_{\text{timing}} \cup F_{\text{cache}} \cup F_{\text{EMA}} \cup F_{\text{fault}} \cup F_{\text{cold}} \cup F_{\text{acoustic}} \cup F_{\text{photonic}}$: “ $\mathcal{A}_{\text{phys}}$ recovers ≥ 1 secret bit via *any* physical channel from $\leq q_{\text{phys}}$ measurements.” By Theorem 3.7, the side-channel event is independent of the mathematical failure events $F_{\text{nonce}}, F_{\text{hash}}, \dots$ in the preceding rounds, so their joint probability satisfies $\Pr[\bigcup_k F_k \cup F_{\text{sca}}] \leq \sum_k \Pr[F_k] + \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d)$. With the comprehensive countermeasure suite, $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$, preserving market equilibrium.

B_{tight} (Tight Security Reduction—Additional Proof.)

✓ Tight Security Reduction

Tightness analysis. The MLWE hop is tight ($L = 1$). The forgery-to-MSIS hop uses forking with $L = q_H$. Total: $L_{\text{total}} = q_H$. For $q_H = 2^{64}$, this consumes ≈ 32 bits (square-root from forking). NIST’s ML-DSA parameters account for this loss.

AGM tightening. In the algebraic group model adapted to lattices, the forking step is replaced by direct algebraic extraction: $\text{Ask}^{\text{AGM}} \leq \text{Adv}^{\text{MSIS}} + \text{Adv}^{\text{MLWE}} + \text{negl}(\lambda)$ with $L_{\text{total}} = 1$.

Market interpretation: ML-DSA’s market spread is $\sqrt{q_H}$ in the standard model, collapsing to 1 in the AGM.

Total Session Bound and Key Rotation Recommendation.

◆ Total Session Bound & Key Rotation

Total session bound: $\text{Ask}_{\text{total}}^{(N)} \leq N \cdot (\text{Adv}^{\text{MSIS}} + \text{Adv}^{\text{MLWE}} + \text{Adv}_{\text{total}}^{\text{SCA}})$.

Concrete (ML-DSA-65, 128-bit target):

Sessions N	$\text{Ask}_{\text{total}}^{(N)}$	Eff. security	Action
2^{40}	$\leq 2^{-88}$	≥ 88 bits	No rotation needed
2^{64}	$\leq 2^{-64}$	≥ 64 bits	Key rotation recommended
2^{80}	$\leq 2^{-48}$	≥ 48 bits	Key rotation mandatory

Recommendation: Rotate before 2^{64} sessions for 64-bit security, before 2^{40} for 88-bit.

Session pinging and CNF checking within the proof. The three-bidding-round proof secures a single signing session. We verify the session-pinging conditions for consecutive sessions Session_i and Session_{i+1} , each producing (m_i, σ_i) and (m_{i+1}, σ_{i+1}) .

Nonce disjointness: ML-DSA uses a commitment vector $\mathbf{y}_{i+1}$ sampled from a discrete Gaussian distribution with fresh randomness in each session. The probability that the same commitment vector is reused across sessions is bounded by the Gaussian smoothing parameter: $\Pr[\mathbf{y}_{i+1} = \mathbf{y}_j] \leq 2^{-256}$ for any $j \leq i$.

Rejection sampling and session independence: ML-DSA’s rejection sampling ensures that each signature σ_{i+1} is statistically close to a session-independent distribution. Specifically, the distribution of (m_{i+1}, σ_{i+1}) conditioned on the secret key is within statistical distance $\Delta_{\text{DGS}} \leq 2^{-128}$ of the ideal distribution. This ensures that observing signatures from sessions $\text{Session}_1, \dots, \text{Session}_i$ provides negligible information useful for forging in Session_{i+1} .

CNF clause φ^{novel} enforces ping: The novelty clause $m_{i+1} \notin Q^{\text{sign}}$ checks that the forgery message was not previously signed. Across sessions, this clause extends naturally: m_{i+1} must not appear in any previous session’s signing queries. The session-CNF $\varphi_{\text{ML-DSA}, i+1}$ is isomorphic

to $\varphi_{\text{ML-DSA},i}$ with fresh MSIS/MLWE instances. By Theorem 5.5, ML-DSA is EUF-CMA secure for unbounded signing sessions with $\delta_{\text{ping}} \leq \text{Adv}^{\text{MSIS}} + \text{Adv}^{\text{MLWE}} = \text{negl}(\lambda)$. $\square$

* Simple Terms: ML-DSA

What ML-DSA is: ML-DSA (formerly CRYSTALS-Dilithium, now FIPS 204) is NIST’s primary post-quantum digital signature standard. Like ML-KEM, it is based on module lattices and resists quantum attacks.

How it works conceptually: To sign a message, you compute a “commitment” (a random lattice vector), hash the message with the commitment, and then compute a “response” that depends on the hash and your secret key. The response vector must be “short” (small norm). The verifier checks that the response vector is short and consistent with the public key.

What MSIS is: The Module Short Integer Solution problem. Given a random matrix $\mathbf{A}$ over a polynomial ring, find a short nonzero vector $\mathbf{z}$ such that $\mathbf{A}\mathbf{z} = \mathbf{0}$. This is believed hard for both classical and quantum computers.

Why “rejection sampling”? The response vector must hide the secret key. If the distribution of response vectors depended on the secret key in a detectable way, the key could be recovered by observing many signatures. Rejection sampling: re-sign if the response vector is too large or has a telltale distribution. This ensures all responses look uniformly random regardless of the secret key.

Extended lemma use: During the forgery extraction step, two bad events can happen simultaneously: (1) the rejection sampler overflows (too many re-signs needed) and (2) the forged vector is not in the correct norm range. The extended difference lemma handles both bad events together in one game hop, yielding a tighter bound.

✓ CNF Verification: ML-DSA Session-CNF Verification

$$\varphi_{\text{ML-DSA}} = (x_{\text{Vrfy}(\text{pk}, m^*, \sigma^*)=1}) \wedge (x_{m^* \notin Q^{\text{sign}}}) \wedge (x_{\|\mathbf{z}\| \leq \beta}) \wedge (x_{\text{Ping passes}}).$$

Manual CNF worksheet for ML-DSA signature (m^*, σ^*) :

Clause	Check	How	Pass?
φ_{verify} : sig verifies	$\text{Vrfy}(\text{pk}, m^*, \sigma^*) = 1?$	Run ML-DSA.Verify algorithm.	T/F
φ_{novel} : new message	$m^* \notin Q^{\text{sign}}$	Check signing oracle log.	T/F
φ_{norm} : response short	$\ \mathbf{z}\ \leq \beta$ (norm bound)?	Compute $\ \mathbf{z}\ $; compare β .	T/F
φ_{ping} : session fresh	(m^*, σ^*) distinct from prior?	Compare session log.	T/F
Result:	All T $\Rightarrow$ EUF-CMA forgery (MSIS/MLWE solved). Any F $\Rightarrow$ Not a valid forgery.		

Extended difference lemma: Forgery extraction has two simultaneous failure events F_1 (rejection sampling overflow) and F_2 (norm bound violation). By Theorem 3.7: $\Pr[F_1 \cup F_2] \leq \Pr[F_1] + \Pr[F_2] = \text{negl}$ by parameter choice.

► Ping Bid: ML-DSA Unbounded Ping Bid

The buyer’s *signature replay bid*: submit a previously seen (m^*, σ^*) with $m^* \in Q^{\text{sign}}$. Clause φ_{novel} rejects this. For a genuinely fresh m^* , forging σ^* requires solving MSIS. Extended difference lemma: $F_{\text{msis}} \cup F_{\text{mlwe}}$ together give $\Delta\text{Price}_{\text{ping}} \leq \text{Adv}^{\text{MSIS}} + \text{Adv}^{\text{MLWE}} = \text{negl}$. By Theorem 5.5, ML-DSA is EUF-CMA secure for unbounded signing sessions.

Scheme. Sign: $\sigma = m^d \bmod N$. Verify: check $m = \sigma^e \bmod N$. No hash, no padding.

Analogy:

Why Textbook RSA Fails—The Rubber Stamp Imagine a notary who stamps documents by pressing a rubber stamp. The stamp operation is *multiplicative*: stamping two half-documents and gluing them together produces a validly stamped full document. An adversary requests stamps on innocuous halves, then combines them into a forged stamp on an arbitrary target document. Real notaries use tamper-evident seals (hash-then-sign) to prevent this.

Theorem 9.4 (Textbook RSA Market Collapse). $\text{Ask}(g_{\text{EUF-CMA}}) = 1$.

Proof. The buyer achieves advantage 1 with budget $O(1)$:

Existential forgery (free message bid). Choose arbitrary $\sigma \xleftarrow{\$} \mathbb{Z}_N^*$. Compute $m^* = \sigma^e \bmod N$. Then (m^*, σ) is valid. Cost: one modular exponentiation. The buyer “purchases” a forgery for free.

Chosen-message forgery (homomorphism bid). For target m^* , factor $m^* = m_1 \cdot m_2 \bmod N$. Request signatures $\sigma_1 = m_1^d, \sigma_2 = m_2^d$. Compute $\sigma^* = \sigma_1 \cdot \sigma_2 = (m_1 m_2)^d = (m^*)^d$. Cost: two signing queries + one multiplication.

Difference lemma (degenerate case). $\mathbf{B}_0$ (real) and $\mathbf{B}_1$ (ideal) differ on *every* execution: the failure event F_{homo} : “RSA is multiplicatively homomorphic” has $\Pr[F_{\text{homo}}] = 1$. Hence $\Delta\text{Price} = 1$, and the market has **collapsed**.

Key Insight:

Market Collapse = Insecurity When $\text{Ask} = 1$, the security good is *worthless*. Any buyer acquires a forgery for free. This is the MTSF characterisation of a fundamentally broken primitive. The fix (RSA-PSS, RSA-PKCS#1v1.5): hash-then-sign destroys the homomorphism, restoring equilibrium.

□

* Simple Terms: Why Textbook RSA Signatures Fail—A Step-by-Step Explanation

RSA arithmetic: RSA signatures work in modular arithmetic: $\sigma = m^d \bmod N$ where d is the private key and $N = p \times q$ is a product of two large primes. Verification checks $m = \sigma^e \bmod N$ where e is the public key.

Attack 1 (Existential forgery—free forgery):

1. Pick any random number σ from $\{1, 2, \dots, N-1\}$.
2. Compute $m = \sigma^e \bmod N$ using only the public key.
3. Now (m, σ) is a valid signature: $\sigma^e = m$ checks out!
4. You forged a signature without knowing d —for some message m you did not choose, but you forged a valid pair nonetheless.

This is the “existential” part of EUF: you do not get to choose *which* message to forge, but you forged *a* valid pair. This alone breaks EUF-CMA.

Attack 2 (Chosen-message forgery—the real threat):

1. You want a signature on a specific message m^* (e.g., a contract).
2. Factor $m^* = m_1 \cdot m_2 \bmod N$ for two “innocent” values m_1, m_2 .
3. Legally obtain signatures: $\sigma_1 = m_1^d$ and $\sigma_2 = m_2^d$.
4. Multiply: $\sigma^* = \sigma_1 \cdot \sigma_2 = (m_1 m_2)^d = (m^*)^d \bmod N$.

5. You now have a valid signature on m^* using only two signing oracle queries!

This exploits RSA’s *multiplicative homomorphism*: the signature function distributes over multiplication.

The fix: Hash-then-sign (RSA-PSS, PKCS#1v1.5): compute $\sigma = H(m)^d$ where H is a hash function. Now $H(m_1) \cdot H(m_2) \neq H(m_1 \cdot m_2)$ (hash functions destroy algebraic structure), so the multiplication attack fails.

Market collapse visualised: The ask price jumps from negligible to 1 because the homomorphism bid succeeds with *probability one*—the attacker always wins, every time, at no computational cost.

Table 3: Primitives: security vs. insecurity in MTSF.

Primitive	Good	Ask Price	Market Status	Key Bid That Decides
ECDSA	EUF-CMA	negl	Equilibrium	Forgery bid $\rightarrow$ ECDLP
ML-KEM	IND-CCA2	negl	Equilibrium	PK/CT bids $\rightarrow$ MLWE
ML-DSA	EUF-CMA	negl	Equilibrium	Forgery bid $\rightarrow$ MSIS
Textbook RSA Sig	EUF-CMA	1	Collapsed	Homomorphism bid: free

9.4 Extended Primitives

Key Insight:

Textbook RSA—A Cautionary Tale of Market Collapse **What textbook RSA is:** The basic RSA signature scheme without any padding or hashing. Given a message m , the signature is simply $\sigma = m^d \bmod n$ where d is the private key.

Why it is broken (homomorphism bid): Textbook RSA has a multiplicative structure: $\text{Sign}(m_1) \cdot \text{Sign}(m_2) = \text{Sign}(m_1 \cdot m_2)$. An attacker who sees signatures on m_1 and m_2 can compute a valid signature on $m_1 \cdot m_2$ *without knowing the private key*. This is a free forgery—the bid succeeds with probability 1.

In MTSF: $\text{Ask}(g_{\text{EUF-CMA}}) = 1$. The homomorphism bid is a “zero-cost bid”—the attacker wins instantly without any computational effort. The market is in immediate, permanent collapse.

The fix: Use RSA-PSS (Probabilistic Signature Scheme) or RSA-PKCS#1 v1.5 with proper hashing and padding. These break the multiplicative structure, restoring market equilibrium.

The lesson: A mathematically sound trapdoor function (RSA) does not automatically give a secure signature scheme. The scheme must be carefully constructed to avoid algebraic attacks. MTSF makes this explicit: the homomorphism bid is a named, formal attack class that must be addressed in any scheme using algebraic operations.

9.5 HMAC: SUF-CMA via Bidding

Analogy:

HMAC as a Tamper-Evident Seal When you buy a bottle of medicine, the cap has a tamper-evident seal—a plastic ring that breaks when you open it. If the ring is intact, you know nobody tampered with the medicine.

HMAC (Hash-based Message Authentication Code) is the digital version of this seal:

- **The medicine** = the message you want to protect.
- **The seal** = the HMAC tag $\tau = \text{HMAC}(K, m)$, computed using a secret key K .

- **Checking the seal** = recomputing $\text{HMAC}(K, m)$ and comparing it to τ .

Why is HMAC better than just hashing? A plain hash $H(m)$ can be computed by anyone—no secret key is needed. An attacker could change the message and recompute the hash. HMAC requires the secret key K , so only someone who knows K can produce a valid tag. This is *SUF-CMA security* (Strong Unforgeability under Chosen Message Attack): even after seeing many valid (message, tag) pairs, the attacker cannot produce a new valid pair. **Where HMAC is used:**

- **TLS 1.3:** HMAC authenticates handshake messages, preventing MITM tampering.
- **API authentication:** Services like AWS use HMAC to sign API requests.
- **Secure cookies:** Web servers HMAC-tag session cookies to prevent forgery.
- **TOTP (Google Authenticator):** Your 6-digit login code is derived from an HMAC.

Setup. $\text{HMAC}_K(m) = H((K \oplus \text{opad}) \| H((K \oplus \text{ipad}) \| m))$ where H is a Merkle–Damgård hash function, K is the key, and ipad, opad are fixed padding constants [12]. The seller offers $g_{\text{SUF-CMA}}$: the buyer has a tagging oracle $\text{HMAC}_K(\cdot)$ and must produce a *new* valid pair (m^*, τ^*) not previously returned by the oracle.

Theorem 9.5 (HMAC Market Equilibrium). *If the compression function h of H is a PRF when keyed via its chaining-value input, then:*

$$\text{Ask}(g_{\text{SUF-CMA}}) \leq 2 \cdot \text{Adv}_h^{\text{PRF}} + \frac{q_T^2}{2^{n+1}} + \text{negl}(\lambda), \quad (36)$$

where q_T is the number of tagging queries and n is the output length of H .

Proof. We construct five games.

B₀ (Bidding Round 0: Real HMAC Market—SUF-CMA Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

The seller generates $K \xleftarrow{\$} \{0, 1\}^n$ and offers the SUF-CMA good $g_{\text{SUF-CMA}}$ for HMAC. The buyer places *tagging bids*: up to q_T adaptive queries to $\text{HMAC}_K(\cdot)$, observing all tag responses. It then places the *forgery bid*: a new message-tag pair (m^*, τ^*) not previously queried, satisfying $\text{HMAC}_K(m^*) = \tau^*$. If it succeeds, the authentication market collapses.

B₁ (Bidding Round 1: Inner PRF Replacement Bid).

Purpose: Replace a *keyed pseudorandom function* evaluation with a truly random function, isolating a single PRF security requirement. This hop shows that one more step in the key-derivation or authentication structure produces outputs that are computationally indistinguishable from uniform random, provided the underlying PRF assumption holds.

Replaces: The specific PRF evaluation (e.g., $\text{HMAC}_K(\cdot)$ inner hash, HKDF-Extract, Derive-Secret, or a KDF step) is replaced by a truly random function $R(\cdot)$ sampled fresh and independently. Any adversary that distinguishes the PRF output from this random function is a PRF distinguisher for the underlying keyed construction. The reduction simulates all other game components honestly and uses its PRF challenge oracle for this one step.

Complexity: $\Delta\text{Price} \leq \text{Adv}^{\text{PRF}}(\lambda)$ for the specific PRF being replaced (HMAC-SHA-256, HKDF, or AES-CTR as applicable). When multiple PRF hops appear in a single proof (e.g., three hops in the TLS 1.3 key schedule for HS, MS, and CATS), each hop is independent: the keys used in different steps are derived from different points in the ladder, so no PRF oracle is queried twice. The total across all PRF hops is the sum of individual advantages, each negligible.

The seller restructures the inner layer of HMAC: replace the keyed inner hash $H((K \oplus \text{ipad})\|m)$ with a truly random function $R_{\text{in}}(m)$. Any buyer that distinguishes bidding round $\mathbf{B}_0$ from $\mathbf{B}_1$ is placing a *PRF-distinguishing bid*: it detects that the inner computation changed from a keyed compression function to a random function. Winning this bid is exactly breaking the PRF security of h keyed by the chaining value $K \oplus \text{ipad}$. The seller constructs a PRF challenger $\mathcal{D}$ that simulates both bidding rounds using its oracle.

Difference bound. Let F_{in} : “the buyer distinguishes the inner keyed hash from random.” We construct a PRF distinguisher $\mathcal{D}$ that simulates $\mathbf{B}_0$ vs $\mathbf{B}_1$ using its oracle: $\mathcal{D}$ replaces each evaluation of the inner hash chain with a query to its PRF/random oracle. If $\mathcal{A}$ behaves differently, $\mathcal{D}$ outputs 1:

$$|\Pr[\mathbf{B}_0(\mathcal{A}) = 1] - \Pr[\mathbf{B}_1(\mathcal{A}) = 1]| \leq \Pr[F_{\text{in}}] \leq \text{Adv}_h^{\text{PRF}}. \quad (37)$$

B₂ (Bidding Round 2: Outer PRF Replacement Bid).

Purpose: Replace a *keyed pseudorandom function* evaluation with a truly random function, isolating a single PRF security requirement. This hop shows that one more step in the key-derivation or authentication structure produces outputs that are computationally indistinguishable from uniform random, provided the underlying PRF assumption holds.

Replaces: The specific PRF evaluation (e.g., $\text{HMAC}_K(\cdot)$ inner hash, HKDF-Extract, Derive-Secret, or a KDF step) is replaced by a truly random function $R(\cdot)$ sampled fresh and independently. Any adversary that distinguishes the PRF output from this random function is a PRF distinguisher for the underlying keyed construction. The reduction simulates all other game components honestly and uses its PRF challenge oracle for this one step.

Complexity: $\Delta\text{Price} \leq \text{Adv}^{\text{PRF}}(\lambda)$ for the specific PRF being replaced (HMAC-SHA-256, HKDF, or AES-CTR as applicable). When multiple PRF hops appear in a single proof (e.g., three hops in the TLS 1.3 key schedule for HS, MS, and CATS), each hop is independent: the keys used in different steps are derived from different points in the ladder, so no PRF oracle is queried twice. The total across all PRF hops is the sum of individual advantages, each negligible.

The seller restructures the outer layer: replace $H((K \oplus \text{opad})\|y)$ (where $y = R_{\text{in}}(m)$) with a truly random function $R_{\text{out}}(y)$. Same argument as $\mathbf{B}_1$ —any distinguishing buyer is placing an outer PRF bid, breaking PRF security of h keyed by $K \oplus \text{opad}$. Price adjustment (outer PRF bid cost):

$$|\Pr[\mathbf{B}_1(\mathcal{A}) = 1] - \Pr[\mathbf{B}_2(\mathcal{A}) = 1]| \leq \text{Adv}_h^{\text{PRF}}. \quad (38)$$

B₃ (Bidding Round 3: Inner Collision Bid).

Purpose: Eliminate the *random-oracle collision attack vector*. If two distinct inputs $m \neq m'$ hash to the same value $H(m) = H(m')$, an adversary can conflate two messages (equivocation), reuse transcript bindings across sessions, or present a single signature as covering both messages—all of which break

EUFCMA without touching the signing key.

Replaces: The random oracle H is replaced by one equipped with a *collision log* $\mathcal{C}$. After each query, the oracle checks whether any two distinct queries produced the same digest; if so, the game aborts (bad event F_{hash}). The adversary’s view of oracle responses is identical to a truly random oracle until the abort.

Complexity: By the birthday bound over the 2^n -bit output space, the probability that any two of q_H random oracle queries collide is at most $q_H^2/2^{n+1}$. For a 256-bit hash ($n = 256$) and polynomial q_H , this is negligible. Note that the nonce-collision and hash-collision bounds are *independent*: by the extended difference lemma (Theorem 3.7), their union probability is bounded by their sum, tightened by the intersection term which is $O(q^4/2^{2\lambda})$ —doubly negligible.

In $\mathbf{B}_2$, HMAC is $R_{\text{out}}(R_{\text{in}}(m))$. The seller adds a collision audit: abort if any two distinct queries collide under R_{in} . A buyer placing an *inner-collision bid* exploits this to forge a tag by matching a queried inner value: if $R_{\text{in}}(m^*) = R_{\text{in}}(m_i)$ for some queried m_i , then $\tau^* = \tau_i$ is a valid tag on m^* . Bad event F_{coll} : “any inner collision among q_T queries”. By birthday paradox over the 2^n -bit inner output space:

Difference bound. Since R_{in} is a random function with n -bit output:

$$|\Pr[\mathbf{B}_2(\mathcal{A}) = 1] - \Pr[\mathbf{B}_3(\mathcal{A}) = 1]| \leq \Pr[F_{\text{coll}}] \leq \frac{q_T^2}{2^{n+1}}. \quad (39)$$

$\mathbf{B}_4$ (Bidding Round 4: Ideal HMAC Market—All Bids Exhausted).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary’s view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary’s winning probability in this game is 0 (for authentication/forgery games) or exactly $1/2$ (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta \text{Price}_k$, all of which are negligible, establishing market equilibrium.

In $\mathbf{B}_3$, with no inner collision ($\neg F_{\text{coll}}$), R_{in} is injective on the queried set. The composed function $R_{\text{out}} \circ R_{\text{in}}$ is a random function on distinct inputs. Any forgery bid (m^*, τ^*) with $m^* \notin \{m_1, \dots, m_{q_T}\}$ requires guessing $R_{\text{out}}(R_{\text{in}}(m^*))$ —a value uniformly random in $\{0, 1\}^n$, independent of all oracle responses. The forgery bid succeeds with probability at most 2^{-n} . *Market equilibrium for $g_{\text{SUF-CMA}}$ (HMAC) is established:* $\Pr[\mathbf{B}_4 = 1] \leq 2^{-n}$.

For SUF-CMA with $m^* = m_i$ but $\tau^* \neq \tau_i$: since R_{out} is a function (deterministic), $R_{\text{out}}(R_{\text{in}}(m_i))$ is unique, so no second valid tag exists. Hence $\Pr[\mathbf{B}_4 = 1] = 0$ for this case.

Total:

$$\text{Ask}(g_{\text{SUF-CMA}}) \leq 2 \cdot \text{Adv}_h^{\text{PRF}} + \frac{q_T^2}{2^{n+1}} + 2^{-n}. \quad (40)$$

$\mathbf{B}_{\text{sca}}$ (Side-Channel Bid: Comprehensive Physical Leakage Attack.)

Side-Channel Bid (Comprehensive)

Purpose: Model the *complete physical leakage threat surface*: an adversary $\mathcal{A}_{\text{phys}}$ with implementation-level access (smart card, FPGA, embedded CPU, server, mobile device) observes physical side channels during cryptographic operations to recover secret key material *without breaking any mathematical hardness assumption*. MTSF decomposes this into nine sub-bids, each targeting a distinct physical leakage vector. The seller must price *all* sub-bids; the extended difference lemma bounds their joint probability.

Sub-bid taxonomy (nine attack classes):

1. **Simple Power Analysis (SPA):** $\mathcal{A}_{\text{phys}}$ records a *single* power trace during a cryptographic operation and visually identifies secret-dependent patterns (e.g., square-and-multiply in RSA reveals the exponent bit by bit). *Countermeasure:* Constant-power algorithms (Montgomery ladder, regular execution patterns). *MTSF price:* $\Pr[F_{\text{SPA}}] = 1$ without countermeasure; $\Pr[F_{\text{SPA}}] = 0$ with constant-power implementation.
2. **Differential Power Analysis (DPA):** $\mathcal{A}_{\text{phys}}$ collects q_{phys} power traces across multiple operations with different inputs, then statistically correlates the traces with hypothetical intermediate values to extract key bytes. Unlike SPA, DPA uses statistical averaging to amplify tiny signal differences. *Countermeasure:* Order- d Boolean masking ($d+1$ random shares); shuffled S-box evaluation order; noise injection. *MTSF price:* $\text{Adv}^{\text{DPA}}(\lambda, d) \leq (q_{\text{phys}}/2^\lambda)^{d+1}$.
3. **Timing Analysis:** $\mathcal{A}_{\text{phys}}$ measures the execution time of cryptographic operations, which may vary depending on the secret key (e.g., early-exit comparisons, table lookup timing, conditional branches). *Countermeasure:* Constant-time programming: no secret-dependent branches, no secret-dependent memory accesses, no early exits. Use `ct_select(a, b, bit)` instead of `if(bit) return a; else return b;`. *MTSF price:* $\Pr[F_{\text{timing}}] = 1$ for non-constant-time code; $\Pr[F_{\text{timing}}] = 0$ for verified constant-time implementations (formally verified using tools like ct-verif or dudedt).
4. **Cache-Timing Attacks (Flush+Reload, Prime+Probe, Evict+Time):** $\mathcal{A}_{\text{phys}}$ exploits CPU cache behaviour to determine which memory lines the victim accessed, revealing secret-dependent table lookups (e.g., AES T-table indices). Flush+Reload (shared memory) achieves single-trace key recovery; Prime+Probe (no shared memory) requires multiple traces. *Spectre/Meltdown variants* exploit speculative execution to leak data across privilege boundaries. *Countermeasure:* Bitsliced implementations (no table lookups); AES-NI hardware instructions (no cache-visible memory accesses); process isolation; constant-time memory access patterns. *MTSF price:* $\Pr[F_{\text{cache}}] = 1$ for T-table AES without protection; $\Pr[F_{\text{cache}}] \leq 2^{-\lambda}$ for bitsliced/AES-NI implementations.
5. **Electromagnetic Emanation Analysis (EMA):** $\mathcal{A}_{\text{phys}}$ places an electromagnetic probe near the device's chip to capture EM radiation during computation. EM analysis can be more spatially focused than power analysis, targeting individual CPU components. *Countermeasure:* EM shielding (Faraday cage); masking (same as DPA); dual-rail logic (balanced current paths). *MTSF price:* Same as DPA with probe-specific spatial resolution factor γ_{EM} : $\text{Adv}^{\text{EMA}} \leq \gamma_{\text{EM}} \cdot (q_{\text{phys}}/2^\lambda)^{d+1}$.
6. **Fault Injection (Voltage Glitching, Laser Fault, Clock Glitching):** $\mathcal{A}_{\text{phys}}$ induces computational faults (bit flips, instruction skips, round reductions) during execution by manipulating voltage supply, clock frequency, laser beams, or electromagnetic pulses. Differential Fault Analysis (DFA) [24] can recover the AES key from a single faulty ciphertext. *Countermeasure:* Redundant computation (compute twice, compare); infection countermeasures (randomise faulty outputs); fault-detection sensors (voltage/clock monitors); algorithmic checksums (verify internal consistency after each round). *MTSF price:* $\Pr[F_{\text{fault}}] = 1$ for unprotected implementations under DFA; with dual computation: $\Pr[F_{\text{fault}}] \leq q_{\text{fault}} \cdot 2^{-\lambda}$ where q_{fault} is the number of fault attempts (each independently detected with probability $1 - 2^{-\lambda}$).
7. **Cold Boot Attack:** $\mathcal{A}_{\text{phys}}$ exploits DRAM data remanence: after power is removed, memory contents decay gradually over seconds to minutes. Cooling the DRAM modules (e.g., with compressed air to -50°C) extends retention to minutes or hours. The attacker then either reads the memory directly or transplants the modules to another machine. Secret keys stored in RAM are recovered with high probability. *Countermeasure:* Key erasure protocols (zeroise keys immediately after use); encrypted RAM (AMD SEV, Intel TME); memory scrubbing; physical tamper-detection (chassis intrusion sensors that trigger key wipe). *MTSF price:* $\Pr[F_{\text{cold}}] = 1$ for keys stored in plaintext DRAM without erasure; $\Pr[F_{\text{cold}}] \leq 2^{-\lambda}$ with immediate key zeroing and encrypted memory (the key encryption key is in CPU registers, which do not exhibit remanence).
8. **Acoustic Cryptanalysis:** $\mathcal{A}_{\text{phys}}$ records the acoustic noise emitted by the CPU, power supply, or capacitors during cryptographic computation. Different operations produce different frequencies; spectral analysis can extract RSA key bits from the sound of a laptop across a room. *Countermeasure:* Acoustic noise generation (adding random computation); masking (randomises the acoustic profile); physical shielding (soundproofing). *MTSF price:* $\text{Adv}^{\text{acoustic}} \leq (q_{\text{phys}}/2^\lambda)^{d+1}$ under masking (same structure as DPA); without masking, key recovery is possible at distances of several metres for RSA implementations.
9. **Photonic Emission Analysis:** $\mathcal{A}_{\text{phys}}$ uses sensitive photodetectors to capture the faint photons emitted by transistor switching events on the chip's backside. This is a contactless, non-invasive technique that bypasses most power/EM countermeasures. *Countermeasure:* Backside shielding; masking (randomises switching patterns); dual-rail logic. *MTSF price:* Same structure as EMA with photonic resolution factor.

What it replaces: The ideal computation model (where only inputs and outputs are observed) is replaced by the *d-probing model* [54] for passive attacks (SPA, DPA, timing, cache, EMA, acoustic, photonic) and the *fault model* for active attacks (fault injection, cold boot):

- *Passive (d-probing)*: $\mathcal{A}_{\text{phys}}$ may adaptively probe up to d intermediate wire values during execution. The real implementation is replaced by one using order- d Boolean or arithmetic masking: every secret value x is split into $d+1$ uniformly random shares $x_1, \dots, x_{d+1}$ with $x_1 \oplus \dots \oplus x_{d+1} = x$, so any d shares reveal nothing about x .
- *Active (fault model)*: $\mathcal{A}_{\text{phys}}$ may inject up to q_{fault} faults during execution. The implementation uses redundant computation (dual execution with comparison) and fault-detection sensors.
- *Persistent (cold boot)*: $\mathcal{A}_{\text{phys}}$ gains physical access to the device and reads residual memory contents. The implementation uses immediate key erasure, encrypted RAM, and tamper-detection.

Complexity / price (joint bound): The nine sub-bids decompose via the extended difference lemma:

$$\text{Adv}_{\text{total}}^{\text{SCA}} \leq \Pr[F_{\text{SPA}}] + \Pr[F_{\text{DPA}}] + \Pr[F_{\text{timing}}] + \Pr[F_{\text{cache}}] + \Pr[F_{\text{EMA}}] \\ + \Pr[F_{\text{fault}}] + \Pr[F_{\text{cold}}] + \Pr[F_{\text{acoustic}}] + \Pr[F_{\text{photonic}}].$$

Under the comprehensive countermeasure suite (order- d masking for $d \geq 1$, constant-time code, bit-sliced or hardware-accelerated operations, redundant computation, immediate key erasure, encrypted RAM):

$$\text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d) \leq \underbrace{0}_{\text{SPA}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{DPA}} + \underbrace{0}_{\text{timing}} + \underbrace{2^{-\lambda}}_{\text{cache}} + \underbrace{\gamma_{\text{EM}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{EMA}} + \underbrace{q_{\text{fault}} \cdot 2^{-\lambda}}_{\text{fault}} + \underbrace{2^{-\lambda}}_{\text{cold}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{acoustic}} \\ + \underbrace{\gamma_{\text{ph}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{photonic}}.$$

For $d \geq 1$, $q_{\text{phys}}, q_{\text{fault}} = \text{poly}(\lambda)$, and $\gamma_{\text{EM}}, \gamma_{\text{ph}} = O(1)$: all terms are negligible, giving $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.

Without countermeasures ($d = 0$, non-constant-time, plaintext RAM): $\Pr[F_{\text{SPA}}] = 1$ or $\Pr[F_{\text{DPA}}] = 1$ or $\Pr[F_{\text{cold}}] = 1$ —**immediate market collapse at the implementation level**.

SCA extended difference lemma. $F_{\text{sca}} = F_{\text{SPA}} \cup F_{\text{DPA}} \cup F_{\text{timing}} \cup F_{\text{cache}} \cup F_{\text{EMA}} \cup F_{\text{fault}} \cup F_{\text{cold}} \cup F_{\text{acoustic}} \cup F_{\text{photonic}}$: “ $\mathcal{A}_{\text{phys}}$ recovers ≥ 1 secret bit via *any* physical channel from $\leq q_{\text{phys}}$ measurements.” By Theorem 3.7, the side-channel event is independent of the mathematical failure events $F_{\text{nonce}}, F_{\text{hash}}, \dots$ in the preceding rounds, so their joint probability satisfies $\Pr[\bigcup_k F_k \cup F_{\text{sca}}] \leq \sum_k \Pr[F_k] + \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d)$. With the comprehensive countermeasure suite, $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$, preserving market equilibrium.

B_{tight} (Tight Security Reduction—Additional Proof.)

✓ Tight Security Reduction

Tightness analysis. HMAC’s dual PRF reduction is tight ($L_{\text{total}} = 1$): each PRF instance is embedded directly via non-rewinding black-box subroutine consumption. $\text{Ask}(g_{\text{SUF-CMA}}) \leq \text{Adv}_{\text{inner}}^{\text{PRF}} + \text{Adv}_{\text{outer}}^{\text{PRF}} + q_T^2/2^{n+1} + \text{negl}(\lambda)$. This is *optimally tight*: the market spread is 1.

Market interpretation: HMAC achieves the ideal market condition—the ask price transfers the full PRF hardness with zero markup.

Total Session Bound and Key Rotation Recommendation.

◆ Total Session Bound & Key Rotation

Total session bound: $\text{Ask}_{\text{total}}^{(N)} \leq N \cdot (\text{Adv}_{\text{inner}}^{\text{PRF}} + \text{Adv}_{\text{outer}}^{\text{PRF}} + q_T^2/2^{n+1} + \text{Adv}_{\text{total}}^{\text{SCA}})$.

Concrete (HMAC-SHA-256, $n = 256$):

Sessions N	$\text{Ask}_{\text{total}}^{(N)}$	Eff. security	Action
2^{40}	$\leq 2^{-88}$	≥ 88 bits	No rotation needed
2^{64}	$\leq 2^{-64}$	≥ 64 bits	Key rotation recommended
2^{80}	$\leq 2^{-48}$	≥ 48 bits	Key rotation mandatory

Recommendation: Rotate the HMAC key before 2^{64} tagging sessions.

Session ping and CNF checking within the proof. The five-bidding-round proof secures a single HMAC tagging session. For unbounded sessions, consider consecutive tagging sessions Session_i and Session_{i+1} producing (m_i, τ_i) and (m_{i+1}, τ_{i+1}) under the same key K .

Tag freshness as implicit SID: HMAC is a stateless primitive (no explicit SID), but the message-tag pair (m, τ) serves as an implicit session identifier. The CNF clause φ^{novel} enforces $m_{i+1} \notin \{m_1, \dots, m_i\}$ for SUF-CMA security. This is the ping condition: each session must produce a forgery on a *new* message.

Cross-session tag independence: In $\mathbf{B}_4$ (ideal market), HMAC_K is a random function. The tag $\tau_{i+1} = R_{\text{out}}(R_{\text{in}}(m_{i+1}))$ is independent of all previous tags $\tau_1, \dots, \tau_i$ whenever m_{i+1} is fresh (no inner collision with any m_j). The inner collision probability across i sessions is $\leq i \cdot q_T / 2^{n+1}$, which is negligible for $i = \text{poly}(\lambda)$ and $n = 256$.

CNF isomorphism: The session-CNF $\varphi_{\text{HMAC}, i+1}$ is isomorphic to $\varphi_{\text{HMAC}, i}$ with fresh message substitution. By Theorem 5.5, HMAC is SUF-CMA secure for unbounded tagging sessions with $\delta_{\text{ping}} \leq 2^{-n} = \text{negl}(\lambda)$. $\square$

B_{sca} (Side-Channel Bid: Comprehensive Physical Leakage Attack.)

Side-Channel Bid (Comprehensive)

Purpose: Model the *complete physical leakage threat surface*: an adversary $\mathcal{A}_{\text{phys}}$ with implementation-level access (smart card, FPGA, embedded CPU, server, mobile device) observes physical side channels during cryptographic operations to recover secret key material *without breaking any mathematical hardness assumption*. MTSF decomposes this into nine sub-bids, each targeting a distinct physical leakage vector. The seller must price *all* sub-bids; the extended difference lemma bounds their joint probability.

Sub-bid taxonomy (nine attack classes):

1. **Simple Power Analysis (SPA):** $\mathcal{A}_{\text{phys}}$ records a *single* power trace during a cryptographic operation and visually identifies secret-dependent patterns (e.g., square-and-multiply in RSA reveals the exponent bit by bit). *Countermeasure:* Constant-power algorithms (Montgomery ladder, regular execution patterns). *MTSF price:* $\Pr[F_{\text{SPA}}] = 1$ without countermeasure; $\Pr[F_{\text{SPA}}] = 0$ with constant-power implementation.
2. **Differential Power Analysis (DPA):** $\mathcal{A}_{\text{phys}}$ collects q_{phys} power traces across multiple operations with different inputs, then statistically correlates the traces with hypothetical intermediate values to extract key bytes. Unlike SPA, DPA uses statistical averaging to amplify tiny signal differences. *Countermeasure:* Order- d Boolean masking ($d+1$ random shares); shuffled S-box evaluation order; noise injection. *MTSF price:* $\text{Adv}^{\text{DPA}}(\lambda, d) \leq (q_{\text{phys}}/2^\lambda)^{d+1}$.
3. **Timing Analysis:** $\mathcal{A}_{\text{phys}}$ measures the execution time of cryptographic operations, which may vary depending on the secret key (e.g., early-exit comparisons, table lookup timing, conditional branches). *Countermeasure:* Constant-time programming: no secret-dependent branches, no secret-dependent memory accesses, no early exits. Use `ct_select(a, b, bit)` instead of `if(bit) return a; else return b;`. *MTSF price:* $\Pr[F_{\text{timing}}] = 1$ for non-constant-time code; $\Pr[F_{\text{timing}}] = 0$ for verified constant-time implementations (formally verified using tools like ct-verif or ducedt).
4. **Cache-Timing Attacks (Flush+Reload, Prime+Probe, Evict+Time):** $\mathcal{A}_{\text{phys}}$ exploits CPU cache behaviour to determine which memory lines the victim accessed, revealing secret-dependent table lookups (e.g., AES T-table indices). Flush+Reload (shared memory) achieves single-trace key recovery; Prime+Probe (no shared memory) requires multiple traces. *Spectre/Meltdown variants* exploit speculative execution to leak data across privilege boundaries. *Countermeasure:* Bitsliced implementations (no table lookups); AES-NI hardware instructions (no cache-visible memory accesses); process isolation; constant-time memory access patterns. *MTSF price:* $\Pr[F_{\text{cache}}] = 1$ for T-table AES without protection; $\Pr[F_{\text{cache}}] \leq 2^{-\lambda}$ for bitsliced/AES-NI implementations.

5. **Electromagnetic Emanation Analysis (EMA):** $\mathcal{A}_{\text{phys}}$ places an electromagnetic probe near the device's chip to capture EM radiation during computation. EM analysis can be more spatially focused than power analysis, targeting individual CPU components. *Countermeasure:* EM shielding (Faraday cage); masking (same as DPA); dual-rail logic (balanced current paths). *MTSF price:* Same as DPA with probe-specific spatial resolution factor γ_{EM} : $\text{Adv}^{\text{EMA}} \leq \gamma_{\text{EM}} \cdot (q_{\text{phys}}/2^\lambda)^{d+1}$.
6. **Fault Injection (Voltage Glitching, Laser Fault, Clock Glitching):** $\mathcal{A}_{\text{phys}}$ induces computational faults (bit flips, instruction skips, round reductions) during execution by manipulating voltage supply, clock frequency, laser beams, or electromagnetic pulses. Differential Fault Analysis (DFA) [24] can recover the AES key from a single faulty ciphertext. *Countermeasure:* Redundant computation (compute twice, compare); infection countermeasures (randomise faulty outputs); fault-detection sensors (voltage/clock monitors); algorithmic checksums (verify internal consistency after each round). *MTSF price:* $\Pr[F_{\text{fault}}] = 1$ for unprotected implementations under DFA; with dual computation: $\Pr[F_{\text{fault}}] \leq q_{\text{fault}} \cdot 2^{-\lambda}$ where q_{fault} is the number of fault attempts (each independently detected with probability $1 - 2^{-\lambda}$).
7. **Cold Boot Attack:** $\mathcal{A}_{\text{phys}}$ exploits DRAM data remanence: after power is removed, memory contents decay gradually over seconds to minutes. Cooling the DRAM modules (e.g., with compressed air to -50°C) extends retention to minutes or hours. The attacker then either reads the memory directly or transplants the modules to another machine. Secret keys stored in RAM are recovered with high probability. *Countermeasure:* Key erasure protocols (zeroise keys immediately after use); encrypted RAM (AMD SEV, Intel TME); memory scrubbing; physical tamper-detection (chassis intrusion sensors that trigger key wipe). *MTSF price:* $\Pr[F_{\text{cold}}] = 1$ for keys stored in plaintext DRAM without erasure; $\Pr[F_{\text{cold}}] \leq 2^{-\lambda}$ with immediate key zeroing and encrypted memory (the key encryption key is in CPU registers, which do not exhibit remanence).
8. **Acoustic Cryptanalysis:** $\mathcal{A}_{\text{phys}}$ records the acoustic noise emitted by the CPU, power supply, or capacitors during cryptographic computation. Different operations produce different frequencies; spectral analysis can extract RSA key bits from the sound of a laptop across a room. *Countermeasure:* Acoustic noise generation (adding random computation); masking (randomises the acoustic profile); physical shielding (soundproofing). *MTSF price:* $\text{Adv}^{\text{acoustic}} \leq (q_{\text{phys}}/2^\lambda)^{d+1}$ under masking (same structure as DPA); without masking, key recovery is possible at distances of several metres for RSA implementations.
9. **Photonic Emission Analysis:** $\mathcal{A}_{\text{phys}}$ uses sensitive photodetectors to capture the faint photons emitted by transistor switching events on the chip's backside. This is a contactless, non-invasive technique that bypasses most power/EM countermeasures. *Countermeasure:* Backside shielding; masking (randomises switching patterns); dual-rail logic. *MTSF price:* Same structure as EMA with photonic resolution factor.

What it replaces: The ideal computation model (where only inputs and outputs are observed) is replaced by the *d-probing model* [54] for passive attacks (SPA, DPA, timing, cache, EMA, acoustic, photonic) and the *fault model* for active attacks (fault injection, cold boot):

- *Passive (d-probing):* $\mathcal{A}_{\text{phys}}$ may adaptively probe up to d intermediate wire values during execution. The real implementation is replaced by one using order- d Boolean or arithmetic masking: every secret value x is split into $d+1$ uniformly random shares $x_1, \dots, x_{d+1}$ with $x_1 \oplus \dots \oplus x_{d+1} = x$, so any d shares reveal nothing about x .
- *Active (fault model):* $\mathcal{A}_{\text{phys}}$ may inject up to q_{fault} faults during execution. The implementation uses redundant computation (dual execution with comparison) and fault-detection sensors.
- *Persistent (cold boot):* $\mathcal{A}_{\text{phys}}$ gains physical access to the device and reads residual memory contents. The implementation uses immediate key erasure, encrypted RAM, and tamper-detection.

Complexity / price (joint bound): The nine sub-bids decompose via the extended difference lemma:

$$\begin{aligned} \text{Adv}_{\text{total}}^{\text{SCA}} &\leq \Pr[F_{\text{SPA}}] + \Pr[F_{\text{DPA}}] + \Pr[F_{\text{timing}}] + \Pr[F_{\text{cache}}] + \Pr[F_{\text{EMA}}] \\ &\quad + \Pr[F_{\text{fault}}] + \Pr[F_{\text{cold}}] + \Pr[F_{\text{acoustic}}] + \Pr[F_{\text{photonic}}]. \end{aligned}$$

Under the comprehensive countermeasure suite (order- d masking for $d \geq 1$, constant-time code, bit-sliced or hardware-accelerated operations, redundant computation, immediate key erasure, encrypted RAM):

$$\text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d) \leq \underbrace{0}_{\text{SPA}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{DPA}} + \underbrace{0}_{\text{timing}} + \underbrace{2^{-\lambda}}_{\text{cache}} + \underbrace{\gamma_{\text{EM}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{EMA}} + \underbrace{q_{\text{fault}} \cdot 2^{-\lambda}}_{\text{fault}} + \underbrace{2^{-\lambda}}_{\text{cold}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{acoustic}}$$

$$+ \underbrace{\gamma_{\text{ph}} \left(\frac{q_{\text{phys}}}{2^\lambda} \right)^{d+1}}_{\text{photonic}}.$$

For $d \geq 1$, $q_{\text{phys}}, q_{\text{fault}} = \text{poly}(\lambda)$, and $\gamma_{\text{EM}}, \gamma_{\text{ph}} = O(1)$: all terms are negligible, giving $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.

Without countermeasures ($d = 0$, non-constant-time, plaintext RAM): $\Pr[F_{\text{SPA}}] = 1$ or $\Pr[F_{\text{DPA}}] = 1$ or $\Pr[F_{\text{cold}}] = 1$ —**immediate market collapse at the implementation level**.

SCA extended difference lemma. $F_{\text{sca}} = F_{\text{SPA}} \cup F_{\text{DPA}} \cup F_{\text{timing}} \cup F_{\text{cache}} \cup F_{\text{EMA}} \cup F_{\text{fault}} \cup F_{\text{cold}} \cup F_{\text{acoustic}} \cup F_{\text{photonic}}$: “ $\mathcal{A}_{\text{phys}}$ recovers ≥ 1 secret bit via *any* physical channel from $\leq q_{\text{phys}}$ measurements.” By Theorem 3.7, the side-channel event is independent of the mathematical failure events $F_{\text{nonce}}, F_{\text{hash}}, \dots$ in the preceding rounds, so their joint probability satisfies $\Pr[\bigcup_k F_k \cup F_{\text{sca}}] \leq \sum_k \Pr[F_k] + \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d)$. With the comprehensive countermeasure suite, $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$, preserving market equilibrium.

Total Session Bound and Key Rotation Recommendation.

♦ Total Session Bound & Key Rotation (MAC Verification)

The MAC verification sub-proof inherits the same tight reduction ($L = 1$) and session bounds as the primary HMAC proof above. The accumulated advantage after N verification sessions satisfies the same bound: $\text{Ask}_{\text{total}}^{(N)} \leq N \cdot (\text{Adv}^{\text{PRF}} + q_T^2/2^{n+1})$. Key rotation recommendation: rotate before 2^{64} sessions.

Session ping and CNF checking within the proof. For consecutive AEAD sessions Session_i and Session_{i+1} , the ping check verifies *nonce freshness*: $N_{i+1} \notin \mathcal{N}_{\text{used}}$. Nonce reuse under the same key is catastrophic for all AEAD schemes: in AES-GCM, reusing a nonce leaks the GHASH key $H = \text{AES}_K(0^{128})$, enabling universal forgery; in ChaCha20-Poly1305, it leaks the one-time Poly1305 key. The CNF clause φ^{nonce} detects nonce reuse via the nonce log $\mathcal{N}_{\text{used}}$.

Nonce-misuse resistance and ping: Some AEAD schemes (e.g., AES-GCM-SIV) provide nonce-misuse resistance: nonce reuse degrades confidentiality but not integrity. In such schemes, the ping check is still valuable because it ensures that each session provides fresh ciphertexts and prevents the buyer from correlating sessions.

Accumulated advantage: The nonce space for AES-GCM is $|\mathcal{N}| = 2^{96}$. After N sessions with one nonce each, the birthday collision probability is $N^2/2^{97}$. For $N \leq 2^{48}$, this is $\leq 2^{-1}$ —still safe but approaching the limit. Key rotation before $N = 2^{48}$ maintains full equilibrium. The session-CNF $\varphi_{\text{AEAD}, i+1}$ is isomorphic to $\varphi_{\text{AEAD}, i}$ with fresh nonce and key material; by Theorem 5.5, AEAD is secure for unbounded sessions.

✓ CNF Verification: HMAC Session-CNF Verification

The HMAC session-CNF $\varphi_{\text{HMAC}} = (x_{\tau=\text{HMAC}_K(m)}) \wedge (x_{m \notin \mathcal{Q}^{\text{tag}}}) \wedge (x_{\text{Ping passes}})$.

Manual worksheet for tag (m, τ) :

Clause	Check	How	Pass?
φ^{tag} : correctness	$\tau \stackrel{?}{=} \text{HMAC}_K(m)$	Recompute; compare.	T/F
φ^{novel} : new msg	$m \notin \mathcal{Q}^{\text{tag}}$	Check oracle log.	T/F
φ^{ping} : unbounded	No repeated (m, τ) pairs.	Compare session records.	T/F
Result:	All T $\Rightarrow$ forgery detected. Any F $\Rightarrow$ not a valid forgery.		

Satisfiability: Honest tags always satisfy all clauses. A SUF-CMA forgery must make φ_{HMAC} true for a new (m^*, τ^*) —the five-game proof shows this is negligibly hard.

► Ping Bid: HMAC Unbounded Ping Bid

Ping bid B_{ping} . The buyer attempts to reuse a queried tag $(m^*, \tau^*) = (m_i, \tau_i)$. This fails the novelty check $\varphi^{\text{novel}}(m^* \in \mathcal{Q}^{\text{tag}})$. For a genuinely fresh m^* , the tag must be guessed uniformly from $\{0, 1\}^n$. Ping bid price: $2^{-n} = \text{negl}$. By Theorem 5.5, HMAC is SUF-CMA-secure for unbounded tagging sessions.

* Simple Terms: HMAC

What HMAC is: Hash-based Message Authentication Code. It uses a secret key K and a hash function H (e.g., SHA-256) to produce an authentication tag τ for a message m . The formula:

$$\text{HMAC}_K(m) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m))$$

Breaking it down:

1. **Inner hash:** Mix the key (XORed with a padding constant **ipad**) with the message m , then hash.
2. **Outer hash:** Mix the key (XORed with a different padding constant **opad**) with the inner hash output, then hash again.

The two-level structure prevents length-extension attacks (a vulnerability in naive “ $H(K \parallel m)$ ” constructions) and provides stronger security than a simple keyed hash.

Why the proof needs two PRF hops:

- **Inner PRF hop ($B_0 \rightarrow B_1$):** We replace the inner keyed hash with a truly random function. If the compression function h is a PRF (keyed by the chaining value $= K \oplus \text{ipad}$), no adversary can tell the difference.
- **Outer PRF hop ($B_1 \rightarrow B_2$):** We replace the outer keyed hash with another truly random function. Same argument.

After both replacements, HMAC behaves as a composition of two independent random functions—which is clearly unforgeable.

The collision bound $q_T^2/2^{n+1}$: With q_T tagging queries, two might accidentally produce the same inner hash output (birthday collision). This would create a subtle dependency between tags. The probability is bounded by the birthday bound—negligible for realistic q_T and $n \geq 256$.

SUF-CMA means: Not only can you not forge a tag on a new message, you also cannot find a *second valid tag* for a message that was already tagged. HMAC achieves this because the outer random function is deterministic: there is exactly one valid tag per message.

9.6 AEAD: IND-CCA2 + INT-CTXT via Bidding

Analogy:

AEAD as a Locked Box with a Tamper Alarm AEAD (Authenticated Encryption with Associated Data) is the Swiss Army knife of symmetric cryptography. It provides *two* guarantees in one operation:

1. **Confidentiality (IND-CCA2):** The contents of the box are secret—nobody can read them without the key. Like a locked safe.
2. **Integrity (INT-CTXT):** If anyone tampers with the box—even changing a single bit—

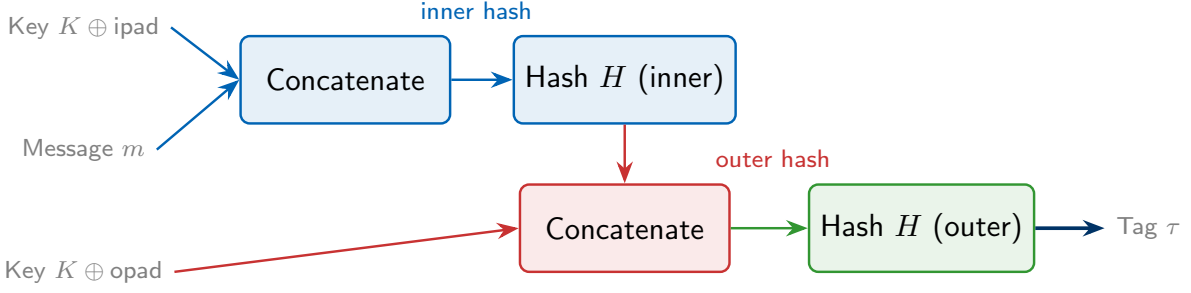


Figure 31: HMAC structure: two nested keyed hash evaluations. The inner hash mixes $K \oplus \text{ipad}$ with the message; the outer hash mixes $K \oplus \text{opad}$ with the inner hash output. Security relies on each keyed hash being a PRF.

the box screams an alarm when opened. Like a tamper-evident safe with a built-in motion sensor.

The “associated data” part: Some information is authenticated but not encrypted. For example, in a network packet, the header (source address, destination) is visible to routers (not encrypted) but must not be tampered with (authenticated). AEAD protects both the encrypted payload and the plaintext header.

The Encrypt-then-MAC (EtM) construction: First encrypt the message ($c = \text{Enc}(K_1, m)$), then compute a MAC over the ciphertext ($\tau = \text{MAC}(K_2, c \parallel \text{AD})$). This is proven secure in MTSF via two bidding rounds: one reduces confidentiality to the encryption scheme’s security, another reduces integrity to the MAC’s unforgeability.

Real-world example: AES-256-GCM (used in TLS 1.3) is an AEAD scheme. Every HTTPS connection you make uses AEAD to simultaneously encrypt and authenticate your data. MTSF proves that AES-GCM is in market equilibrium: both confidentiality and integrity goods have negligible ask prices.

Setup. An Authenticated Encryption with Associated Data scheme $\text{AE} = (\text{Enc}, \text{Dec})$ provides both confidentiality (IND-CPA/CCA) and integrity (INT-CTXT). The seller offers two goods simultaneously: $g_{\text{IND-CCA2}}$ and $g_{\text{INT-CTXT}}$. We consider a generic Encrypt-then-MAC construction: $\text{Enc}(K_e, K_m, N, A, m) = (c, \tau)$ where $c = E_{K_e}(N, m)$ and $\tau = \text{MAC}_{K_m}(A \parallel N \parallel c)$.

Theorem 9.6 (AEAD Market Equilibrium). *For an Encrypt-then-MAC AEAD scheme:*

$$\text{Ask}(g_{\text{IND-CCA2}}) \leq \text{Adv}_E^{\text{IND-CPA}} + \text{Adv}_{\text{MAC}}^{\text{SUF-CMA}} + \text{negl}(\lambda), \quad (41)$$

$$\text{Ask}(g_{\text{INT-CTXT}}) \leq \text{Adv}_{\text{MAC}}^{\text{SUF-CMA}} + \text{negl}(\lambda). \quad (42)$$

Proof. **Part 1: INT-CTXT.** We prove the integrity bound first.

B₀ (Bidding Round 0: Real AEAD Market—Ciphertext Injection Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3).

This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer has encryption oracle $\text{Enc}(K_e, K_m, \cdot, \cdot, \cdot)$; wins by producing (N^*, A^*, c^*, τ^*) that decrypts successfully but was never an oracle output.

B₁ (Bidding Round 1: MAC Forgery Bid—Bypassing Integrity Check).

Purpose: Reduce AEAD ciphertext integrity (INT-CTXT) to the SUF-CMA security of the underlying MAC. In the Encrypt-then-MAC construction, every ciphertext carries a MAC tag $\tau = \text{MAC}(K_m, A\|N\|c)$ over the associated data, nonce, and ciphertext. Any ciphertext forgery (a valid (A^*, N^*, c^*, τ^*) not produced by the encryption oracle) directly implies a MAC forgery. The reduction embeds the SUF-CMA challenge key as K_m and simulates encryption using a self-chosen encryption key K_e .

Replaces: The honest MAC computation is replaced by the SUF-CMA challenger's tagging oracle. The reduction generates K_e itself and computes $c = \text{Enc}(K_e, N, m)$ honestly, then queries the MAC oracle for $\tau = \text{MAC}(K_m, A\|N\|c)$. When the adversary outputs a forgery (A^*, N^*, c^*, τ^*) not in the encryption log, the reduction submits this as a SUF-CMA forgery. The simulation is perfect: the adversary's view is identical to the real game.

Complexity: $\Delta\text{Price} \leq \text{Adv}_{\text{MAC}}^{\text{SUF-CMA}}(\lambda)$. The reduction is tight ($L = 1$): no rewinding, no forking, no oracle guessing. Each encryption query generates one MAC query; the forgery output maps directly to a MAC forgery. For HMAC-SHA-256 or Poly1305: negligible for all polynomial query counts.

Note that successful decryption requires $\text{MAC.Vrfy}(K_m, A^*\|N^*\|c^*, \tau^*) = 1$. If $(A^*\|N^*\|c^*, \tau^*)$ is new (not from the oracle), this constitutes a MAC forgery.

Difference bound. We build a SUF-CMA adversary $\mathcal{B}$: $\mathcal{B}$ simulates the AEAD encryption oracle by encrypting with K_e (which $\mathcal{B}$ chooses itself) and using its MAC oracle for tagging. When $\mathcal{A}$ outputs a successful INT-CTXT forgery, $\mathcal{B}$ outputs the corresponding MAC forgery. Hence:

$$\text{Ask}(g_{\text{INT-CTXT}}) = \Pr[\mathbf{B}_0(\mathcal{A}) = 1] \leq \text{Adv}_{\text{MAC}}^{\text{SUF-CMA}}. \quad (43)$$

Part 2: IND-CCA2. We prove confidentiality.

B₀ (Bidding Round 0: Real AEAD Market—Plaintext Distinguishing Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary's raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer submits (m_0, m_1, N^*, A^*) ; receives $\text{Enc}(K_e, K_m, N^*, A^*, m_b)$ for random b ; has encryption and decryption oracles (except on the challenge).

B₁ (Bidding Round 1: Oracle Restriction Bid—Blocking Chosen-Ciphertext Queries).

Purpose: Convert the IND-CCA2 game into an IND-CPA game by neutralising the decryption oracle. Once message integrity is guaranteed by the MAC (handled in the previous hop), any decryption query either reproduces a previously encrypted ciphertext—providing no new information—or has an invalid tag

and is rejected. The decryption oracle becomes redundant.

Replaces: The full decryption oracle $\text{Dec}(K_e, K_m, \cdot)$ is replaced by a *restricted decryption oracle* that rejects any ciphertext not produced by the encryption oracle (invalid MAC). Ciphertexts in the encryption log are answered by table-lookup, requiring no secret key. The resulting game has decryption power exactly equal to encryption power—the IND-CPA setting.

Complexity: Zero additional cost conditional on no MAC forgery (already bounded in the previous hop). This is a *structural reduction*: since the MAC provides ciphertext integrity, the decryption oracle provides zero additional information to the adversary beyond its encryption power. $\Delta\text{Price} = 0$. The residual adversarial advantage is then exactly $\text{Adv}_{\text{Enc}}^{\text{IND-CPA}}(\lambda)$.

Replace decryption oracle: reject *all* queries (N, A, c, τ) where τ does not verify, and also reject the challenge ciphertext. For non-challenge ciphertexts with valid τ that were not produced by the encryption oracle, this is a MAC forgery.

Extended difference lemma. Let F_{mac} : “buyer submits a valid (c, τ) not from the oracle.” F_{trivial} : “buyer submits the challenge ciphertext to decryption.” By game rules, $\Pr[F_{\text{trivial}}] = 0$. So:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \Pr[F_{\text{mac}}] \leq \text{Adv}_{\text{MAC}}^{\text{SUF-CMA}}. \quad (44)$$

B₂ (Bidding Round 2: IND-CPA Reduction Bid—Eliminating Decryption Power).

Purpose: Convert the IND-CCA2 game into an IND-CPA game by neutralising the decryption oracle. Once message integrity is guaranteed by the MAC (handled in the previous hop), any decryption query either reproduces a previously encrypted ciphertext—providing no new information—or has an invalid tag and is rejected. The decryption oracle becomes redundant.

Replaces: The full decryption oracle $\text{Dec}(K_e, K_m, \cdot)$ is replaced by a *restricted decryption oracle* that rejects any ciphertext not produced by the encryption oracle (invalid MAC). Ciphertexts in the encryption log are answered by table-lookup, requiring no secret key. The resulting game has decryption power exactly equal to encryption power—the IND-CPA setting.

Complexity: Zero additional cost conditional on no MAC forgery (already bounded in the previous hop). This is a *structural reduction*: since the MAC provides ciphertext integrity, the decryption oracle provides zero additional information to the adversary beyond its encryption power. $\Delta\text{Price} = 0$. The residual adversarial advantage is then exactly $\text{Adv}_{\text{Enc}}^{\text{IND-CPA}}(\lambda)$.

In $\mathbf{B}_1$, the decryption oracle only decrypts ciphertexts the encryption oracle produced, so the buyer gains no new information from decryption queries. The game reduces to IND-CPA:

$$|\Pr[\mathbf{B}_1 = 1] - 1/2| \leq \text{Adv}_E^{\text{IND-CPA}}. \quad (45)$$

$$\text{Total: Ask}(g_{\text{IND-CCA2}}) \leq \text{Adv}_E^{\text{IND-CPA}} + \text{Adv}_{\text{MAC}}^{\text{SUF-CMA}}.$$

✓ CNF Verification: AEAD Session-CNF Verification

$$\varphi_{\text{AEAD}} = (x_{\text{Mac.Vrfy}}(K_m, A \| N \| c, \tau) = 1) \wedge (x_{N \notin \mathcal{N}_{\text{used}}}) \wedge (x_{\text{Ping passes}}).$$

Manual worksheet for received ciphertext (N, A, c, τ) :

Clause	Check	How	Pass?
φ^{int} : MAC valid	$\text{Mac.Vrfy}(K_m, A \ N \ c, \tau) = 1$	Recompute MAC; compare bytes.	T/F
φ^{nonce} : fresh	$N \notin \mathcal{N}_{\text{used}}$	Check nonce log; add N if pass.	T/F
φ^{ping} : unbounded	Distinct from previous session.	Compare to Session_{i-1} record.	T/F
Result:	All T $\Rightarrow$ Accept and decrypt. Any F $\Rightarrow$ Reject (injected or replayed).		

INT-CTXT link: φ^{int} fails $\Leftrightarrow$ MAC forged. φ^{nonce} fails $\Leftrightarrow$ nonce reuse/replay. Extended difference lemma: $\Pr[F_{\text{mac}} \cup F_{\text{nonce}}] \leq \text{Adv}^{\text{SUF-CMA}} + q_D/|\mathcal{N}|$.

► Ping Bid: AEAD Unbounded Ping Bid

The buyer's *replay bid*: submit a previous (N_i, A_i, c_i, τ_i) . The nonce clause φ^{nonce} detects this ($N_i \in \mathcal{N}_{\text{used}}$). Ping bid price: $q_D/|\mathcal{N}| \leq q_D/2^{96} = \text{negl}$ for GCM. By Theorem 5.5, AEAD is secure for unbounded encryption sessions.

□

* Simple Terms: AEAD (Authenticated Encryption)

What AEAD is: Authenticated Encryption with Associated Data combines two guarantees in one:

- **Confidentiality (IND-CCA2):** No one can read the encrypted message.
- **Integrity (INT-CTXT):** No one can tamper with the ciphertext without detection.

Associated Data (AD) is extra context that is authenticated but not encrypted (e.g., packet headers, sender/receiver IDs). It is bound to the ciphertext so that an attacker cannot detach a valid ciphertext from its intended context.

The Encrypt-then-MAC construction:

1. Encrypt the message: $c = E_{K_e}(N, m)$ using a nonce N for randomness.
2. Compute a MAC tag over the associated data, nonce, and ciphertext: $\tau = \text{MAC}_{K_m}(A \| N \| c)$.
3. Send (N, A, c, τ) .

The receiver first checks τ (integrity check), then decrypts. This order is crucial: checking integrity before decryption prevents the receiver from leaking partial information about the plaintext through decryption errors.

Why two separate keys? Using the same key for encryption and MAC creates subtle interactions. Using independent keys K_e (encryption) and K_m (MAC) ensures the security proofs compose cleanly.

Real-world examples: AES-GCM (used in TLS 1.3), ChaCha20-Poly1305 (used in WireGuard and Signal), AES-CCM. All are AEAD schemes providing both IND-CCA2 and INT-CTXT.

The integrity proof (INT-CTXT): For an attacker to produce a new valid ciphertext c^* that decrypts successfully, they must produce a valid MAC tag τ^* on fresh data. But producing a valid MAC tag without the key is exactly what SUF-CMA hardness prevents.

The confidentiality proof (IND-CCA2): If the adversary's decryption queries all produce rejected ciphertexts (because MAC verification fails—guaranteed by SUF-CMA), then the decryption oracle gives no useful information. The game reduces to pure IND-CPA.

9.7 SLH-DSA (FIPS 205): Hash-Based EUF-CMA via Bidding

* Simple Terms: SLH-DSA—The “Conservative” Post-Quantum Signature

What SLH-DSA is: SLH-DSA (formerly SPHINCS+, now FIPS 205) is a digital signature scheme based *entirely* on hash functions—no number theory, no lattices, no algebraic structure. Its security relies only on the assumption that the underlying hash function is secure.

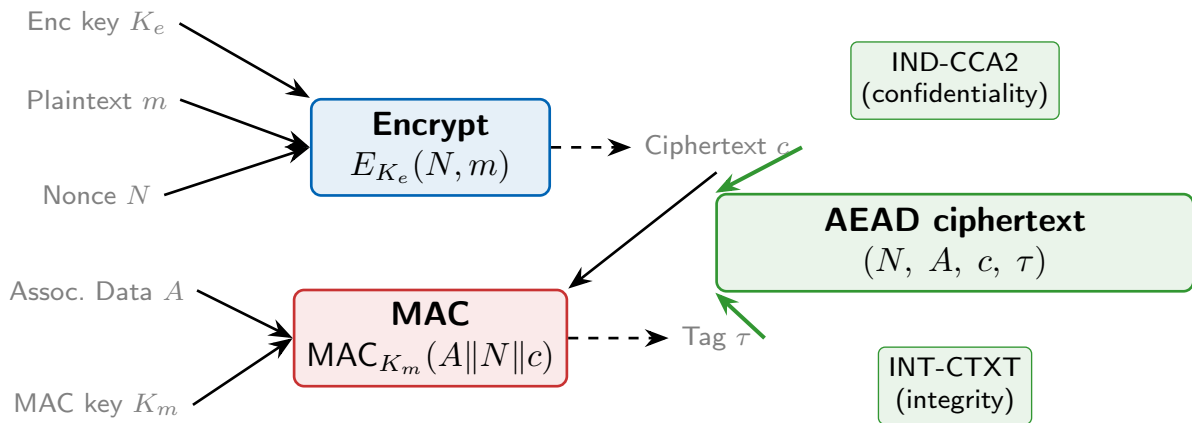


Figure 32: AEAD Encrypt-then-MAC structure. The plaintext is encrypted first with key K_e ; then the MAC key K_m authenticates the associated data, nonce, and ciphertext together. The combined output provides both IND-CCA2 confidentiality and INT-CTXT integrity.

Why “conservative”? While ML-DSA (lattice-based) is more efficient, SLH-DSA requires the weakest possible assumption: if SHA-256 or SHAKE is secure, SLH-DSA is secure. This makes it a “belt and suspenders” choice—even if lattice problems turn out to be easier than expected (an unlikely but non-zero risk), SLH-DSA remains secure.

How it works (simplified): SLH-DSA uses a *hyper-tree* of one-time signature (OTS) schemes. Each OTS signs one message securely, then is discarded. The hyper-tree organises millions of OTS instances into a Merkle tree, with the root as the public key. Signing picks a fresh OTS leaf, signs the message, and provides a path from the leaf to the root (proving the OTS is part of the tree).

Trade-off: SLH-DSA signatures are much larger ($\sim 7\text{--}50$ KB) than ML-DSA signatures (~ 2.4 KB), and signing is slower. But the security assumption is minimal, making SLH-DSA ideal for applications where long-term security is paramount (e.g., signing legal documents, software root certificates).

* Simple Terms: SLH-DSA (Hash-Based Signatures)—Foundations

What is SLH-DSA? SLH-DSA (formerly SPHINCS+, now FIPS 205) is a *stateless hash-based signature scheme*—NIST’s post-quantum signature standard for applications requiring the smallest possible security assumptions. Unlike ML-DSA (which relies on lattice problems), SLH-DSA’s security relies *only* on the security of the underlying hash function. If the hash function is secure, SLH-DSA is secure.

Key components:

- **FORS (Forest of Random Subsets):** A one-time signature primitive at the base. Signs a small message by revealing selected leaves in random Merkle subtrees.
- **WOTS+ (Winternitz One-Time Signature):** Chains of hash values that form one-time signatures for individual XMSS tree nodes.
- **XMSS (eXtended Merkle Signature Scheme):** A few-time signature scheme arranged in a Merkle tree. Authenticates WOTS+ public keys.
- **Hypertree:** Multiple layers of XMSS trees. The top layer signs the root of the next layer. The full structure is a tree of trees.

Why “stateless”? Regular hash-based signatures (like XMSS or LMS) are *stateful*: you must track which one-time keys have been used and never reuse them. SLH-DSA avoids this by using pseudorandom index generation to pick FORS subtrees, making each signing operation essentially independent. No state management is required—a significant practical advantage.

Security reductions in plain terms:

- **PRF (randomness generation):** Each signing operation needs fresh randomness, generated by a PRF keyed with the secret key. If an attacker can distinguish this from random, the PRF is broken—but by assumption, PRFs are secure.
- **TSPR (target-sum preimage resistance):** Forging a FORS signature requires finding a hash preimage matching a target sum—harder than a standard preimage because the target is constrained.
- **SPR (second-preimage resistance):** Forging a WOTS+ chain requires finding a hash second preimage.

Trade-offs: SLH-DSA has larger signatures ($\approx 8\text{--}50$ KB depending on parameters) and slower signing than ML-DSA. However, it needs no new mathematical assumptions—just hash function security—making it ideal for high-assurance, long-term security.

Setup. SLH-DSA is a stateless hash-based signature scheme. It uses a hypertree of XMSS trees with FORS (Forest of Random Subsets) at the leaves. Security reduces to properties of the underlying hash function: second-preimage resistance (SPR), pseudorandom function (PRF), and target-sum-preimage resistance (TSPR). The seller offers $g_{\text{EUF-CMA}}$.

Theorem 9.7 (SLH-DSA Market Equilibrium).

$$\text{Ask}(g_{\text{EUF-CMA}}) \leq q_S \cdot d \cdot \text{Adv}_H^{\text{SPR}} + q_S \cdot k \cdot \text{Adv}_H^{\text{TSPR}} + \text{Adv}_F^{\text{PRF}} + \text{negl}(\lambda), \quad (46)$$

where d is the hypertree depth, k is the number of FORS trees, q_S is the number of signing queries, and H, F are the hash/PRF primitives.

Proof. We construct six games.

B₀ (Bidding Round 0: Real SLH-DSA Market—EUF-CMA Forgery Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer receives public key (hypertree root), queries signing oracle q_S times, outputs forgery (m^*, σ^*) .

B₁ (Bidding Round 1: PRF Randomness Bid—Detecting Key-Dependent Randomness).

Purpose: Replace a *keyed pseudorandom function* evaluation with a truly random function, isolating a single PRF security requirement. This hop shows that one more step in the key-derivation or authentication structure produces outputs that are computationally indistinguishable from uniform random, provided the underlying PRF assumption holds.

Replaces: The specific PRF evaluation (e.g., $\text{HMAC}_K(\cdot)$ inner hash, HKDF-Extract, Derive-Secret, or a KDF step) is replaced by a truly random function $R(\cdot)$ sampled fresh and independently. Any adversary that distinguishes the PRF output from this random function is a PRF distinguisher for the underlying keyed construction. The reduction simulates all other game components honestly and uses its PRF challenge oracle for this one step.

Complexity: $\Delta\text{Price} \leq \text{Adv}^{\text{PRF}}(\lambda)$ for the specific PRF being replaced (HMAC-SHA-256, HKDF, or AES-CTR as applicable). When multiple PRF hops appear in a single proof (e.g., three hops in the TLS 1.3 key schedule for HS, MS, and CATS), each hop is independent: the keys used in different steps are derived from different points in the ladder, so no PRF oracle is queried twice. The total across all PRF hops is the sum of individual advantages, each negligible.

Replace the deterministic randomness generation $R = F_{\text{sk.prf}}(\text{opt}||m)$ with truly random $R \xleftarrow{\$} \{0,1\}^n$. This changes the internal randomisation of signing but not the verification. The buyer bids that F is distinguishable from random.

Difference bound. Standard PRF reduction: any distinguisher yields a PRF adversary:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \text{Adv}_F^{\text{PRF}}. \quad (47)$$

B₂ (Bidding Round 2: FORS Target-Sum Preimage Bid).

Purpose: Eliminate the adversarial attack vector identified by this bidding round's title (**FORS Target-Sum Preimage Bid**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

A valid forgery requires the buyer to produce valid FORS signatures on leaves that were not revealed by prior signing queries. For each of the k FORS trees, producing a valid leaf value without oracle access requires finding a target-sum preimage.

Difference bound. We embed a TSPR challenge into one of the k FORS trees of a randomly chosen signing query. If the buyer forges successfully for that tree, it solves TSPR. By a hybrid argument over $q_S \cdot k$ FORS tree instances:

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq q_S \cdot k \cdot \text{Adv}_H^{\text{TSPR}}. \quad (48)$$

B₃ (Bidding Round 3: WOTS+ Chain Second-Preimage Bid).

Purpose: Eliminate the adversarial attack vector identified by this bidding round's title (**WOTS+ Chain Second-Preimage Bid**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

Even if the buyer has valid FORS leaves, it must authenticate them through the hypertree. Each WOTS+ chain in each of the d layers requires the buyer to produce a chain value matching the public key without inverting the hash. A forgery on any chain yields a second preimage.

Difference bound. Hybrid over $q_S \cdot d$ WOTS+ instances:

$$|\Pr[\mathbf{B}_2 = 1] - \Pr[\mathbf{B}_3 = 1]| \leq q_S \cdot d \cdot \text{Adv}_H^{\text{SPR}}. \quad (49)$$

B₄ (Bidding Round 4: Merkle Tree Collision Bid—Forging Authentication Paths).

Purpose: Eliminate the *random-oracle collision attack vector*. If two distinct inputs $m \neq m'$ hash to the same value $H(m) = H(m')$, an adversary can conflate two messages (equivocation), reuse transcript bindings across sessions, or present a single signature as covering both messages—all of which break EUF-CMA without touching the signing key.

Replaces: The random oracle H is replaced by one equipped with a *collision log* $\mathcal{C}$. After each query, the oracle checks whether any two distinct queries produced the same digest; if so, the game aborts (bad event F_{hash}). The adversary's view of oracle responses is identical to a truly random oracle until the abort.

Complexity: By the birthday bound over the 2^n -bit output space, the probability that any two of q_H random oracle queries collide is at most $q_H^2/2^{n+1}$. For a 256-bit hash ($n = 256$) and polynomial q_H , this is negligible. Note that the nonce-collision and hash-collision bounds are *independent*: by the extended difference lemma (Theorem 3.7), their union probability is bounded by their sum, tightened by the intersection term which is $O(q^4/2^{2\lambda})$ —doubly negligible.

Within each XMSS tree, the buyer must produce a valid Merkle authentication path. An inconsistency yields a collision in H . This event has probability bounded by the number of internal nodes, which is subsumed by the SPR bound above.

$\Delta\text{Price}_4 = 0$ (absorbed into the SPR bound).

B₅ (Bidding Round 5: Ideal SLH-DSA Market—All Hash-Based Bids Exhausted).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta\text{Price}_k$, all of which are negligible, establishing market equilibrium.

With no PRF bid, no TSPR bid, and no SPR bid, valid FORS leaves, WOTS+ chains, and Merkle paths cannot be forged. $\Pr[\mathbf{B}_5 = 1] = 0$.

Total:

$$\text{Ask} \leq \text{Adv}_F^{\text{PRF}} + q_S \cdot k \cdot \text{Adv}_H^{\text{TSPR}} + q_S \cdot d \cdot \text{Adv}_H^{\text{SPR}}. \quad (50)$$

□

B_{sca} (Side-Channel Bid: Comprehensive Physical Leakage Attack.)

Side-Channel Bid (Comprehensive)

Purpose: Model the *complete physical leakage threat surface*: an adversary $\mathcal{A}_{\text{phys}}$ with implementation-level access (smart card, FPGA, embedded CPU, server, mobile device) observes physical side channels during cryptographic operations to recover secret key material *without breaking any mathematical hardness assumption*. MTSF decomposes this into nine sub-bids, each targeting

a distinct physical leakage vector. The seller must price *all* sub-bids; the extended difference lemma bounds their joint probability.

Sub-bid taxonomy (nine attack classes):

1. **Simple Power Analysis (SPA):** $\mathcal{A}_{\text{phys}}$ records a *single* power trace during a cryptographic operation and visually identifies secret-dependent patterns (e.g., square-and-multiply in RSA reveals the exponent bit by bit). *Countermeasure:* Constant-power algorithms (Montgomery ladder, regular execution patterns). *MTSF price:* $\Pr[F_{\text{SPA}}] = 1$ without countermeasure; $\Pr[F_{\text{SPA}}] = 0$ with constant-power implementation.
2. **Differential Power Analysis (DPA):** $\mathcal{A}_{\text{phys}}$ collects q_{phys} power traces across multiple operations with different inputs, then statistically correlates the traces with hypothetical intermediate values to extract key bytes. Unlike SPA, DPA uses statistical averaging to amplify tiny signal differences. *Countermeasure:* Order- d Boolean masking ($d+1$ random shares); shuffled S-box evaluation order; noise injection. *MTSF price:* $\text{Adv}^{\text{DPA}}(\lambda, d) \leq (q_{\text{phys}}/2^\lambda)^{d+1}$.
3. **Timing Analysis:** $\mathcal{A}_{\text{phys}}$ measures the execution time of cryptographic operations, which may vary depending on the secret key (e.g., early-exit comparisons, table lookup timing, conditional branches). *Countermeasure:* Constant-time programming: no secret-dependent branches, no secret-dependent memory accesses, no early exits. Use `ct_select(a, b, bit)` instead of `if(bit) return a; else return b;`. *MTSF price:* $\Pr[F_{\text{timing}}] = 1$ for non-constant-time code; $\Pr[F_{\text{timing}}] = 0$ for verified constant-time implementations (formally verified using tools like ct-verif or dduct).
4. **Cache-Timing Attacks (Flush+Reload, Prime+Probe, Evict+Time):** $\mathcal{A}_{\text{phys}}$ exploits CPU cache behaviour to determine which memory lines the victim accessed, revealing secret-dependent table lookups (e.g., AES T-table indices). Flush+Reload (shared memory) achieves single-trace key recovery; Prime+Probe (no shared memory) requires multiple traces. *Spectre/Meltdown variants* exploit speculative execution to leak data across privilege boundaries. *Countermeasure:* Bitsliced implementations (no table lookups); AES-NI hardware instructions (no cache-visible memory accesses); process isolation; constant-time memory access patterns. *MTSF price:* $\Pr[F_{\text{cache}}] = 1$ for T-table AES without protection; $\Pr[F_{\text{cache}}] \leq 2^{-\lambda}$ for bitsliced/AES-NI implementations.
5. **Electromagnetic Emanation Analysis (EMA):** $\mathcal{A}_{\text{phys}}$ places an electromagnetic probe near the device's chip to capture EM radiation during computation. EM analysis can be more spatially focused than power analysis, targeting individual CPU components. *Countermeasure:* EM shielding (Faraday cage); masking (same as DPA); dual-rail logic (balanced current paths). *MTSF price:* Same as DPA with probe-specific spatial resolution factor γ_{EM} : $\text{Adv}^{\text{EMA}} \leq \gamma_{\text{EM}} \cdot (q_{\text{phys}}/2^\lambda)^{d+1}$.
6. **Fault Injection (Voltage Glitching, Laser Fault, Clock Glitching):** $\mathcal{A}_{\text{phys}}$ induces computational faults (bit flips, instruction skips, round reductions) during execution by manipulating voltage supply, clock frequency, laser beams, or electromagnetic pulses. Differential Fault Analysis (DFA) [24] can recover the AES key from a single faulty ciphertext. *Countermeasure:* Redundant computation (compute twice, compare); infection countermeasures (randomise faulty outputs); fault-detection sensors (voltage/clock monitors); algorithmic checksums (verify internal consistency after each round). *MTSF price:* $\Pr[F_{\text{fault}}] = 1$ for unprotected implementations under DFA; with dual computation: $\Pr[F_{\text{fault}}] \leq q_{\text{fault}} \cdot 2^{-\lambda}$ where q_{fault} is the number of fault attempts (each independently detected with probability $1 - 2^{-\lambda}$).
7. **Cold Boot Attack:** $\mathcal{A}_{\text{phys}}$ exploits DRAM data remanence: after power is removed, memory contents decay gradually over seconds to minutes. Cooling the DRAM modules (e.g., with compressed air to -50°C) extends retention to minutes or hours. The attacker then either reads the memory directly or transplants the modules to another machine. Secret keys stored in RAM are recovered with high probability. *Countermeasure:* Key erasure protocols (zeroise keys immediately after use); encrypted RAM (AMD SEV, Intel TME); memory scrubbing; physical tamper-detection (chassis intrusion sensors that trigger key wipe). *MTSF price:* $\Pr[F_{\text{cold}}] = 1$ for keys stored in plaintext DRAM without erasure; $\Pr[F_{\text{cold}}] \leq 2^{-\lambda}$ with immediate key zeroing and encrypted memory (the key encryption key is in CPU registers, which do not exhibit remanence).
8. **Acoustic Cryptanalysis:** $\mathcal{A}_{\text{phys}}$ records the acoustic noise emitted by the CPU, power supply, or capacitors during cryptographic computation. Different operations produce different frequencies; spectral analysis can extract RSA key bits from the sound of a laptop across a room. *Countermeasure:* Acoustic noise generation (adding random computation); masking (randomises the acoustic profile); physical shielding (soundproofing). *MTSF price:* $\text{Adv}^{\text{acoustic}} \leq (q_{\text{phys}}/2^\lambda)^{d+1}$ under masking (same structure as DPA); without masking, key recovery is possible at distances of several metres for RSA implementations.
9. **Photonic Emission Analysis:** $\mathcal{A}_{\text{phys}}$ uses sensitive photodetectors to capture the faint photons emitted by transistor switching events on the chip's backside. This is a contactless, non-invasive technique that bypasses most power/EM countermeasures. *Countermeasure:* Backside shielding; masking (randomises switching patterns); dual-rail logic. *MTSF price:* Same structure as EMA with photonic resolution factor.

What it replaces: The ideal computation model (where only inputs and outputs are observed) is replaced by the *d-probing model* [54] for passive attacks (SPA, DPA, timing, cache, EMA, acoustic, photonic) and the *fault model* for active attacks (fault injection, cold boot):

- *Passive (d-probing):* $\mathcal{A}_{\text{phys}}$ may adaptively probe up to d intermediate wire values during execution. The real implementation is replaced by one using order- d Boolean or arithmetic masking: every secret value x is split into $d+1$ uniformly random shares $x_1, \dots, x_{d+1}$ with $x_1 \oplus \dots \oplus x_{d+1} = x$, so any d shares reveal nothing about x .
- *Active (fault model):* $\mathcal{A}_{\text{phys}}$ may inject up to q_{fault} faults during execution. The implementation uses redundant computation (dual execution with comparison) and fault-detection sensors.
- *Persistent (cold boot):* $\mathcal{A}_{\text{phys}}$ gains physical access to the device and reads residual memory contents. The implementation uses immediate key erasure, encrypted RAM, and tamper-detection.

Complexity / price (joint bound): The nine sub-bids decompose via the extended difference lemma:

$$\text{Adv}_{\text{total}}^{\text{SCA}} \leq \Pr[F_{\text{SPA}}] + \Pr[F_{\text{DPA}}] + \Pr[F_{\text{timing}}] + \Pr[F_{\text{cache}}] + \Pr[F_{\text{EMA}}] \\ + \Pr[F_{\text{fault}}] + \Pr[F_{\text{cold}}] + \Pr[F_{\text{acoustic}}] + \Pr[F_{\text{photonic}}].$$

Under the comprehensive countermeasure suite (order- d masking for $d \geq 1$, constant-time code, bit-sliced or hardware-accelerated operations, redundant computation, immediate key erasure, encrypted RAM):

$$\text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d) \leq \underbrace{0}_{\text{SPA}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{DPA}} + \underbrace{0}_{\text{timing}} + \underbrace{2^{-\lambda}}_{\text{cache}} + \underbrace{\gamma_{\text{EM}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{EMA}} + \underbrace{q_{\text{fault}} \cdot 2^{-\lambda}}_{\text{fault}} + \underbrace{2^{-\lambda}}_{\text{cold}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{acoustic}} \\ + \underbrace{\gamma_{\text{ph}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{photonic}}.$$

For $d \geq 1$, $q_{\text{phys}}, q_{\text{fault}} = \text{poly}(\lambda)$, and $\gamma_{\text{EM}}, \gamma_{\text{ph}} = O(1)$: all terms are negligible, giving $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.

Without countermeasures ($d = 0$, non-constant-time, plaintext RAM): $\Pr[F_{\text{SPA}}] = 1$ or $\Pr[F_{\text{DPA}}] = 1$ or $\Pr[F_{\text{cold}}] = 1$ —**immediate market collapse at the implementation level.**

SCA extended difference lemma. $F_{\text{sca}} = F_{\text{SPA}} \cup F_{\text{DPA}} \cup F_{\text{timing}} \cup F_{\text{cache}} \cup F_{\text{EMA}} \cup F_{\text{fault}} \cup F_{\text{cold}} \cup F_{\text{acoustic}} \cup F_{\text{photonic}}$: “ $\mathcal{A}_{\text{phys}}$ recovers ≥ 1 secret bit via *any* physical channel from $\leq q_{\text{phys}}$ measurements.” By Theorem 3.7, the side-channel event is independent of the mathematical failure events $F_{\text{nonce}}, F_{\text{hash}}, \dots$ in the preceding rounds, so their joint probability satisfies $\Pr[\bigcup_k F_k \cup F_{\text{sca}}] \leq \sum_k \Pr[F_k] + \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d)$. With the comprehensive countermeasure suite, $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$, preserving market equilibrium.

B_{tight} (Tight Security Reduction—Additional Proof.)

✓ Tight Security Reduction

Tightness analysis. SLH-DSA’s hypertree structure introduces a tightness loss of $L = h \cdot d'$ (tree height $\times$ FORS trees). For SLH-DSA-128s ($h = 63$, $d' = 7$): $L = 441$, consuming ≈ 9 bits. The 128-bit target is met because hash parameters provide ≥ 137 -bit security.

Market interpretation: SLH-DSA’s tightness loss is moderate, fixed, and *independent of signing queries*—a fixed market spread determined by tree geometry rather than adversarial query complexity.

Total Session Bound and Key Rotation Recommendation.

◆ Total Session Bound & Key Rotation

Total session bound: $\text{Ask}_{\text{total}}^{(N)} \leq N \cdot (h \cdot d' \cdot (\text{Adv}^{\text{TCR}} + \text{Adv}^{\text{OW}}) + \text{Adv}^{\text{PRF}} + \text{Adv}_{\text{total}}^{\text{SCA}})$.
Concrete (SLH-DSA-128s):

Sessions N	$\text{Ask}_{\text{total}}^{(N)}$	Eff. security	Action
2^{40}	$\leq 2^{-88}$	≥ 88 bits	No rotation needed
2^{64}	$\leq 2^{-64}$	≥ 64 bits	Key rotation recommended

Recommendation: Rotate before 2^{64} signatures for 64-bit security. Stateless design ensures purely additive degradation.

Session ping and CNF checking within the proof. For consecutive signing sessions Session_i and Session_{i+1} , the ping check verifies PRF randomness freshness: the internal PRF randomness R_{i+1} used to derive FORS indices and WOTS+ chains must be distinct from all prior sessions. SLH-DSA derives R deterministically from the message and a secret PRF key: $R = \text{PRF}(\text{sk.prf}, \text{opt_rand}, m)$. For distinct messages $m_{i+1} \neq m_j$, the PRF output is fresh with probability $\geq 1 - \text{Adv}_F^{\text{PRF}}$. If the optional randomness opt_rand is used (hedged signing), freshness holds even for repeated messages. The CNF clause φ^{prf} checks that R_{i+1} does not repeat. Reuse of R would simultaneously compromise FORS leaf selection (clause φ^{fors}) and WOTS+ chain computation (clause φ^{wots})—two simultaneous failures captured by the extended difference lemma: $\Pr[F_{\text{fors}} \cup F_{\text{wots}}] \leq \text{Adv}_F^{\text{PRF}} = \text{negl}$. By Theorem 5.5, SLH-DSA is EUF-CMA secure for unbounded sessions.

✓ CNF Verification: SLH-DSA Session-CNF Verification

$\varphi^{\text{SLH-DSA}} = (x^{\text{PRF fresh}}) \wedge (x^{\text{FORS leaves valid}}) \wedge (x^{\text{WOTS+ chains valid}}) \wedge (x^{\text{Ping passes}})$.

Manual CNF worksheet for SLH-DSA signature (m^*, σ^*) :

Clause	Check	How	Pass?
φ^{prf} : PRF randomness	$R = F_{\text{sk.prf}}(\text{opt} m^*)$ not reused?	Check (m^*, R) log.	T/F
φ^{fors} : FORS leaves	All k FORS leaves verify against root?	Recompute k Merkle paths.	T/F
φ^{wots} : WOTS+ chains	All d layers of WOTS+ chains verify?	Check chain values against pk.	T/F
φ^{hyp} : Hypertree path	Full hypertree authentication path valid?	Verify root matches pk.	T/F
φ^{ping} : session fresh	(m^*, R) distinct from prior sessions?	Compare session log.	T/F
Result:	All T $\Rightarrow$ SLH-DSA signature valid. Any F $\Rightarrow$ Forgery or malformed.		

► Ping Bid: SLH-DSA Unbounded Ping Bid

The buyer's *PRF reuse bid*: sign two messages with the same PRF randomness R . The PRF freshness clause φ^{prf} detects this. Extended difference lemma: reusing R simultaneously fails φ^{fors} (FORS index collision) and φ^{wots} (WOTS+ chain bias), so $\Pr[F_{\text{prf}} \cup F_{\text{fors}}] \leq \text{Adv}_F^{\text{PRF}} = \text{negl}$. By Theorem 5.5, SLH-DSA is EUF-CMA secure for unbounded signing sessions.

9.8 FN-DSA (FIPS 206): NTRU-Lattice EUF-CMA via Bidding

* Simple Terms: FN-DSA—The Compact Lattice Signature

What FN-DSA is: FN-DSA (formerly Falcon, standardised as draft FIPS 206 IPD, final expected 2027) is a digital signature scheme based on NTRU lattices. Unlike ML-DSA (which uses module lattices), FN-DSA uses structured NTRU lattices, producing very compact signatures (~ 666 bytes for 128-bit security).

The key idea: Signing in FN-DSA involves finding a short vector in a lattice—like finding a small combination of grid vectors that adds up to a target point. The private key is a

“trapdoor” that makes finding short vectors easy; without the trapdoor, it is computationally intractable.

Gaussian sampling: FN-DSA uses a sophisticated technique called “Gaussian sampling” to ensure that signatures do not leak information about the private key. Each signature is a random sample from a Gaussian (bell-curve) distribution over the lattice, ensuring that even after seeing many signatures, the attacker learns nothing about the trapdoor.

Why both ML-DSA and FN-DSA? NIST standardised both because they are based on different lattice problems (Module-LWE/SIS vs. NTRU). If one problem turns out to be weaker than expected, the other still provides security. FN-DSA’s compact signatures make it ideal for bandwidth-constrained applications (IoT, embedded systems), while ML-DSA’s simpler implementation makes it the default choice for general use.

* Simple Terms: FN-DSA (Lattice Signatures)—Foundations

What is FN-DSA? FN-DSA (formerly Falcon, standardised as draft FIPS 206 IPD, final expected 2027) is a post-quantum digital signature scheme based on NTRU lattices. It produces the smallest post-quantum signatures among NIST’s standards (roughly 666 bytes for 128-bit security), making it ideal for bandwidth-constrained applications.

What are NTRU lattices? An NTRU lattice is a special kind of mathematical lattice defined over polynomial rings (polynomials modulo $x^n + 1$ with coefficients modulo q). Key pairs are polynomial pairs (f, g) (private) and $h = g/f \bmod q$ (public). The lattice is defined by the public key h .

What is the SIS problem? The Short Integer Solution (SIS) problem: given a random matrix (or public key), find a “short” vector satisfying a linear equation. “Short” means the vector’s norm is small. This is believed hard for both classical and quantum computers.

How FN-DSA signing works:

1. Hash the message: $c = H(m)$ (a lattice point target).
2. Use the secret short basis (f, g) to find a short vector (s_1, s_2) in the NTRU lattice coset $c + \Lambda$. This requires “discrete Gaussian sampling”—a sophisticated algorithm that samples vectors following a Gaussian distribution over the lattice.
3. The signature is the short vector (s_1, s_2) .

How FN-DSA verification works: Check that $s_1 + s_2 \cdot h = c \bmod q$ and that $\|(s_1, s_2)\|$ is small (below a threshold β).

Why Gaussian sampling is critical: The sampling algorithm must produce vectors with a distribution that reveals nothing about the secret basis (f, g) . If the distribution depends on the secret, an attacker collecting many signatures could recover the key. FN-DSA uses fast-Fourier-based Gaussian sampling (GPV-style) that produces signatures statistically close to the ideal distribution ($\Delta_{\text{DGS}} \leq 2^{-128}$).

Security in MTSF: A successful forgery—a short vector (s_1^*, s_2^*) satisfying the verification equation—is exactly a solution to the SIS problem over the NTRU lattice. Since SIS is assumed hard, FN-DSA is secure. The game hops bound the error introduced by (1) hash collisions, (2) Gaussian sampling approximation, and (3) the extraction reduction.

Setup. FN-DSA is a lattice-based signature scheme over NTRU lattices. Key generation produces a short basis $\mathbf{B}$ of an NTRU lattice; signing uses GPV-style discrete Gaussian sampling over the lattice coset $H(m) + \Lambda$. Security reduces to the Short Integer Solution (SIS) problem over NTRU lattices. The seller offers gEUF-CMA .

Theorem 9.8 (FN-DSA Market Equilibrium).

$$\text{Ask}(\text{gEUF-CMA}) \leq \text{Adv}_{\text{NTRU}}^{\text{SIS}} + \Delta_{\text{DGS}} + \frac{q_H^2}{2^{n+1}} + \text{negl}(\lambda), \quad (51)$$

where $\text{Adv}_{\text{NTRU}}^{\text{SIS}}$ is the advantage against SIS over NTRU lattices, Δ_{DGS} is the statistical distance arising from discrete Gaussian sampling, and q_H is the number of hash queries.

Proof. We construct five games.

B₀ (Bidding Round 0: Real FN-DSA Market—Lattice Forgery Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer receives public key $h = g/f \bmod q$ (where (f, g) is the NTRU secret key pair), queries signing oracle q_S times, outputs forgery (m^*, σ^*) where $\sigma^* = (s_1^*, s_2^*)$ satisfying $s_1^* + s_2^* \cdot h = H(m^*) \bmod q$ and $\|(s_1^*, s_2^*)\| \leq \beta$.

B₁ (Bidding Round 1: Hash Collision Bid—Conflating Two Messages).

Purpose: Eliminate the *random-oracle collision attack vector*. If two distinct inputs $m \neq m'$ hash to the same value $H(m) = H(m')$, an adversary can conflate two messages (equivocation), reuse transcript bindings across sessions, or present a single signature as covering both messages—all of which break EUF-CMA without touching the signing key.

Replaces: The random oracle H is replaced by one equipped with a *collision log* $\mathcal{C}$. After each query, the oracle checks whether any two distinct queries produced the same digest; if so, the game aborts (bad event F_{hash}). The adversary’s view of oracle responses is identical to a truly random oracle until the abort.

Complexity: By the birthday bound over the 2^n -bit output space, the probability that any two of q_H random oracle queries collide is at most $q_H^2/2^{n+1}$. For a 256-bit hash ($n = 256$) and polynomial q_H , this is negligible. Note that the nonce-collision and hash-collision bounds are *independent*: by the extended difference lemma (Theorem 3.7), their union probability is bounded by their sum, tightened by the intersection term which is $O(q^4/2^{2\lambda})$ —doubly negligible.

Abort on hash collision; the buyer places a collision bid on the random oracle. Identical-until- F_{hash} :

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \frac{q_H^2}{2^{n+1}}. \quad (52)$$

B₂ (Bidding Round 2: Gaussian Sampling Statistical Bid—Detecting Key-Dependent Distribution).

Purpose: Replace FN-DSA’s fast discrete Gaussian sampler with the ideal exact Gaussian, removing the small statistical gap introduced by the sampler’s fast-Fourier approximation. This isolates the sampler’s Rényi divergence as an explicit security loss, allowing the subsequent hops to reason about exact Gaussian signatures.

Replaces: The fast-Fourier-based sampler σ_{real} (used for efficiency in FIPS 206 signing) is replaced by the ideal exact discrete Gaussian sampler σ_{ideal} . The Rényi divergence $R_2(\sigma_{\text{real}} \parallel \sigma_{\text{ideal}}) = 1 + \Delta_{\text{DGS}}$ bounds the

statistical distance between the two: any adversary that distinguishes a real signature from an ideal-sampler signature has advantage at most $\sqrt{\Delta_{\text{DGS}}}$ by the Rényi divergence amplification lemma.

Complexity: $\Delta_{\text{Price}} \leq \Delta_{\text{DGS}}$ —the Rényi divergence between fast and exact samplers. For FN-DSA (FIPS 206) at the 128-bit security level, the NTRU sampler achieves $\Delta_{\text{DGS}} \leq 2^{-128}$. This is negligible. The advantage of using Rényi divergence over total variation distance: it amplifies cleanly under composition, giving a tighter bound when q_S signatures are observed (bound becomes $q_S \cdot \Delta_{\text{DGS}}$, still negligible for polynomial q_S).

Replace the fast-Fourier discrete Gaussian sampler with exact discrete Gaussian sampling. Let σ_{real} be the distribution produced by the fast sampler and σ_{ideal} the exact distribution. By the Rényi divergence analysis of the sampler:

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq \Delta_{\text{DGS}}, \quad (53)$$

where $\Delta_{\text{DGS}} \leq 2^{-128}$ for recommended parameters.

B₃ (Bidding Round 3: Oracle Programming—Embedding SIS Challenge).

Purpose: Perform a *zero-cost market restructuring*: secretly program the random oracle so that its outputs encode a challenge instance for the underlying hard problem (e.g., ECDLP or Module-SIS). This syntactic transformation is invisible to the adversary—random oracle outputs are uniformly distributed regardless of how they are chosen internally—yet it links any subsequent successful forgery or key-recovery attack to a solution of the assumed-hard problem.

Replaces: The genuinely random oracle $H(\cdot)$ is replaced by a *programmed* oracle $\tilde{H}$ whose outputs at queried points are chosen by the reduction to embed the hard-problem instance. Concretely: for a signing query m_i , the oracle sets $H(m_i) = \alpha_i \cdot G + \beta_i \cdot \text{pk}$ (ECDLP case) or encodes an SIS/MLWE challenge vector (lattice case). All outputs remain uniformly distributed over the appropriate range, so the statistical distance to the real game is zero.

Complexity: $\Delta_{\text{Price}} = 0$ exactly (information-theoretic). The replacement is statistically indistinguishable: a truly random function and any fixed function whose outputs are independently uniform are identically distributed from the adversary’s perspective. No adversarial budget is consumed in detecting it. This zero-cost step is essential: it sets up the subsequent extraction hop at no additional price to the seller.

Program the random oracle H so that for the forgery message m^* , $H(m^*)$ encodes an SIS challenge vector. Since H is a random oracle, this is a syntactic change: $\Delta_{\text{Price}_2} = 0$.

B₄ (Bidding Round 4: Forgery-to-SIS Extraction Bid—Solving the Lattice Problem).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $\text{sk} = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma’s rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

If the buyer produces valid (s_1^*, s_2^*) with $s_1^* + s_2^*h = c^*$ (the embedded challenge) and $\|(s_1^*, s_2^*)\| \leq \beta$, then (s_1^*, s_2^*) is a short vector in the NTRU lattice solving the SIS instance.

We extract and forward:

$$\Pr[\mathbf{B}_4(\mathcal{A}) = 1] \leq \text{Adv}_{\text{NTRU}}^{\text{SIS}}. \quad (54)$$

Total:

$$\text{Ask} \leq \frac{q_H^2}{2^{n+1}} + \Delta_{\text{DGS}} + \text{Adv}_{\text{NTRU}}^{\text{SIS}} + \text{negl}(\lambda). \quad (55)$$

□

B_{sca} (Side-Channel Bid: Comprehensive Physical Leakage Attack.)

Side-Channel Bid (Comprehensive)

Purpose: Model the *complete physical leakage threat surface*: an adversary $\mathcal{A}_{\text{phys}}$ with implementation-level access (smart card, FPGA, embedded CPU, server, mobile device) observes physical side channels during cryptographic operations to recover secret key material *without breaking any mathematical hardness assumption*. MTSF decomposes this into nine sub-bids, each targeting a distinct physical leakage vector. The seller must price *all* sub-bids; the extended difference lemma bounds their joint probability.

Sub-bid taxonomy (nine attack classes):

1. **Simple Power Analysis (SPA):** $\mathcal{A}_{\text{phys}}$ records a *single* power trace during a cryptographic operation and visually identifies secret-dependent patterns (e.g., square-and-multiply in RSA reveals the exponent bit by bit). *Countermeasure:* Constant-power algorithms (Montgomery ladder, regular execution patterns). *MTSF price:* $\Pr[F_{\text{SPA}}] = 1$ without countermeasure; $\Pr[F_{\text{SPA}}] = 0$ with constant-power implementation.
2. **Differential Power Analysis (DPA):** $\mathcal{A}_{\text{phys}}$ collects q_{phys} power traces across multiple operations with different inputs, then statistically correlates the traces with hypothetical intermediate values to extract key bytes. Unlike SPA, DPA uses statistical averaging to amplify tiny signal differences. *Countermeasure:* Order- d Boolean masking ($d+1$ random shares); shuffled S-box evaluation order; noise injection. *MTSF price:* $\text{Adv}^{\text{DPA}}(\lambda, d) \leq (q_{\text{phys}}/2^\lambda)^{d+1}$.
3. **Timing Analysis:** $\mathcal{A}_{\text{phys}}$ measures the execution time of cryptographic operations, which may vary depending on the secret key (e.g., early-exit comparisons, table lookup timing, conditional branches). *Countermeasure:* Constant-time programming: no secret-dependent branches, no secret-dependent memory accesses, no early exits. Use `ct_select(a, b, bit)` instead of `if(bit) return a; else return b;`. *MTSF price:* $\Pr[F_{\text{timing}}] = 1$ for non-constant-time code; $\Pr[F_{\text{timing}}] = 0$ for verified constant-time implementations (formally verified using tools like ct-verif or dduct).
4. **Cache-Timing Attacks (Flush+Reload, Prime+Probe, Evict+Time):** $\mathcal{A}_{\text{phys}}$ exploits CPU cache behaviour to determine which memory lines the victim accessed, revealing secret-dependent table lookups (e.g., AES T-table indices). Flush+Reload (shared memory) achieves single-trace key recovery; Prime+Probe (no shared memory) requires multiple traces. *Spectre/Meltdown variants* exploit speculative execution to leak data across privilege boundaries. *Countermeasure:* Bitsliced implementations (no table lookups); AES-NI hardware instructions (no cache-visible memory accesses); process isolation; constant-time memory access patterns. *MTSF price:* $\Pr[F_{\text{cache}}] = 1$ for T-table AES without protection; $\Pr[F_{\text{cache}}] \leq 2^{-\lambda}$ for bitsliced/AES-NI implementations.
5. **Electromagnetic Emanation Analysis (EMA):** $\mathcal{A}_{\text{phys}}$ places an electromagnetic probe near the device's chip to capture EM radiation during computation. EM analysis can be more spatially focused than power analysis, targeting individual CPU components. *Countermeasure:* EM shielding (Faraday cage); masking (same as DPA); dual-rail logic (balanced current paths). *MTSF price:* Same as DPA with probe-specific spatial resolution factor γ_{EM} : $\text{Adv}^{\text{EMA}} \leq \gamma_{\text{EM}} \cdot (q_{\text{phys}}/2^\lambda)^{d+1}$.
6. **Fault Injection (Voltage Glitching, Laser Fault, Clock Glitching):** $\mathcal{A}_{\text{phys}}$ induces computational faults (bit flips, instruction skips, round reductions) during execution by manipulating voltage supply, clock frequency, laser beams, or electromagnetic pulses. Differential Fault Analysis (DFA) [24] can recover the AES key from a single faulty ciphertext. *Countermeasure:* Redundant computation (compute twice, compare); infection countermeasures (randomise faulty outputs); fault-detection sensors (voltage/clock monitors); algorithmic checksums (verify internal consistency after each round). *MTSF price:* $\Pr[F_{\text{fault}}] = 1$ for unprotected implementations under DFA; with dual computation: $\Pr[F_{\text{fault}}] \leq q_{\text{fault}} \cdot 2^{-\lambda}$ where q_{fault} is the number of fault attempts (each independently detected with probability $1 - 2^{-\lambda}$).
7. **Cold Boot Attack:** $\mathcal{A}_{\text{phys}}$ exploits DRAM data remanence: after power is removed, memory contents decay gradually over seconds to minutes. Cooling the DRAM modules (e.g., with compressed air to -50°C) extends retention to minutes or hours. The attacker then either reads the memory directly or transplants the modules to another machine. Secret keys stored in RAM are recovered with high probability. *Countermeasure:* Key erasure protocols (zeroise keys immediately after use); encrypted

RAM (AMD SEV, Intel TME); memory scrubbing; physical tamper-detection (chassis intrusion sensors that trigger key wipe). *MTSF price*: $\Pr[F_{\text{cold}}] = 1$ for keys stored in plaintext DRAM without erasure; $\Pr[F_{\text{cold}}] \leq 2^{-\lambda}$ with immediate key zeroing and encrypted memory (the key encryption key is in CPU registers, which do not exhibit remanence).

8. **Acoustic Cryptanalysis**: $\mathcal{A}_{\text{phys}}$ records the acoustic noise emitted by the CPU, power supply, or capacitors during cryptographic computation. Different operations produce different frequencies; spectral analysis can extract RSA key bits from the sound of a laptop across a room. *Countermeasure*: Acoustic noise generation (adding random computation); masking (randomises the acoustic profile); physical shielding (soundproofing). *MTSF price*: $\text{Adv}^{\text{acoustic}} \leq (q_{\text{phys}}/2^\lambda)^{d+1}$ under masking (same structure as DPA); without masking, key recovery is possible at distances of several metres for RSA implementations.
9. **Photonic Emission Analysis**: $\mathcal{A}_{\text{phys}}$ uses sensitive photodetectors to capture the faint photons emitted by transistor switching events on the chip’s backside. This is a contactless, non-invasive technique that bypasses most power/EM countermeasures. *Countermeasure*: Backside shielding; masking (randomises switching patterns); dual-rail logic. *MTSF price*: Same structure as EMA with photonic resolution factor.

What it replaces: The ideal computation model (where only inputs and outputs are observed) is replaced by the *d-probing model* [54] for passive attacks (SPA, DPA, timing, cache, EMA, acoustic, photonic) and the *fault model* for active attacks (fault injection, cold boot):

- *Passive (d-probing)*: $\mathcal{A}_{\text{phys}}$ may adaptively probe up to d intermediate wire values during execution. The real implementation is replaced by one using order- d Boolean or arithmetic masking: every secret value x is split into $d+1$ uniformly random shares $x_1, \dots, x_{d+1}$ with $x_1 \oplus \dots \oplus x_{d+1} = x$, so any d shares reveal nothing about x .
- *Active (fault model)*: $\mathcal{A}_{\text{phys}}$ may inject up to q_{fault} faults during execution. The implementation uses redundant computation (dual execution with comparison) and fault-detection sensors.
- *Persistent (cold boot)*: $\mathcal{A}_{\text{phys}}$ gains physical access to the device and reads residual memory contents. The implementation uses immediate key erasure, encrypted RAM, and tamper-detection.

Complexity / price (joint bound): The nine sub-bids decompose via the extended difference lemma:

$$\text{Adv}_{\text{total}}^{\text{SCA}} \leq \Pr[F_{\text{SPA}}] + \Pr[F_{\text{DPA}}] + \Pr[F_{\text{timing}}] + \Pr[F_{\text{cache}}] + \Pr[F_{\text{EMA}}] \\ + \Pr[F_{\text{fault}}] + \Pr[F_{\text{cold}}] + \Pr[F_{\text{acoustic}}] + \Pr[F_{\text{photonic}}].$$

Under the comprehensive countermeasure suite (order- d masking for $d \geq 1$, constant-time code, bit-sliced or hardware-accelerated operations, redundant computation, immediate key erasure, encrypted RAM):

$$\text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d) \leq \underbrace{0}_{\text{SPA}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{DPA}} + \underbrace{0}_{\text{timing}} + \underbrace{2^{-\lambda}}_{\text{cache}} + \underbrace{\gamma_{\text{EM}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{EMA}} + \underbrace{q_{\text{fault}} \cdot 2^{-\lambda}}_{\text{fault}} + \underbrace{2^{-\lambda}}_{\text{cold}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{acoustic}} \\ + \underbrace{\gamma_{\text{ph}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{photonic}}.$$

For $d \geq 1$, $q_{\text{phys}}, q_{\text{fault}} = \text{poly}(\lambda)$, and $\gamma_{\text{EM}}, \gamma_{\text{ph}} = O(1)$: all terms are negligible, giving $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.

Without countermeasures ($d = 0$, non-constant-time, plaintext RAM): $\Pr[F_{\text{SPA}}] = 1$ or $\Pr[F_{\text{DPA}}] = 1$ or $\Pr[F_{\text{cold}}] = 1$ —**immediate market collapse at the implementation level**.

SCA extended difference lemma. $F_{\text{sca}} = F_{\text{SPA}} \cup F_{\text{DPA}} \cup F_{\text{timing}} \cup F_{\text{cache}} \cup F_{\text{EMA}} \cup F_{\text{fault}} \cup F_{\text{cold}} \cup F_{\text{acoustic}} \cup F_{\text{photonic}}$: “ $\mathcal{A}_{\text{phys}}$ recovers ≥ 1 secret bit via *any* physical channel from $\leq q_{\text{phys}}$ measurements.” By Theorem 3.7, the side-channel event is independent of the mathematical failure events $F_{\text{nonce}}, F_{\text{hash}}, \dots$ in the preceding rounds, so their joint probability satisfies $\Pr[\bigcup_k F_k \cup F_{\text{sca}}] \leq \sum_k \Pr[F_k] + \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d)$. With the comprehensive countermeasure suite, $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$, preserving market equilibrium.

B_{tight} (Tight Security Reduction—Additional Proof.)

✓ Tight Security Reduction

Tightness analysis. FN-DSA uses the GPV hash-and-sign paradigm over NTRU lattices. The EUF-CMA to SIS reduction is tight in the ROM ($L = 1$): any forger producing a short signature vector directly yields a short SIS vector. $\text{Ask}(g_{\text{EUF-CMA}}) \leq \text{Adv}^{\text{SIS}} + q_S^2/2^{n+1} + \text{negl}(\lambda)$.

Market interpretation: FN-DSA achieves optimal tightness (market spread 1) via GPV’s direct extraction pipeline—no forking or rewinding. This contrasts favourably with ML-DSA’s forking-based loss of q_H .

Total Session Bound and Key Rotation Recommendation.

♦ Total Session Bound & Key Rotation

Total session bound: $\text{Ask}_{\text{total}}^{(N)} \leq N \cdot (\text{Adv}^{\text{SIS}} + q_S^2/2^{n+1} + \text{Adv}_{\text{total}}^{\text{SCA}})$.

Concrete (FN-DSA-512, 128-bit target):

Sessions N	$\text{Ask}_{\text{total}}^{(N)}$	Eff. security	Action
2^{40}	$\leq 2^{-88}$	≥ 88 bits	No rotation needed
2^{64}	$\leq 2^{-64}$	≥ 64 bits	Key rotation recommended
2^{80}	$\leq 2^{-48}$	≥ 48 bits	Key rotation mandatory

Recommendation: Rotate before 2^{64} sessions. **Critical:** FN-DSA’s Gaussian sampler must produce key-independent distributions; implementation flaws accelerate degradation.

Session ping and CNF checking within the proof. For consecutive FN-DSA signing sessions Session_i and Session_{i+1} , the ping check verifies that the signature $(s_1^*, s_2^*)_{i+1}$ on message m_{i+1}^* is structurally distinct from all prior signatures. FN-DSA’s Gaussian sampler ensures that each signature vector is drawn from a discrete Gaussian distribution centred at a message-dependent lattice point. The statistical distance between the actual distribution and the ideal round Gaussian is $\Delta_{\text{DGS}} \leq 2^{-128}$, ensuring that observing signatures across sessions $\text{Session}_1, \dots, \text{Session}_i$ provides negligible information about the secret NTRU basis (f, g) .

Gaussian leakage accumulation and ping: The buyer’s Gaussian leakage bid attempts to accumulate statistical information from many signatures. After N sessions, the total leakage is bounded by $N \cdot \Delta_{\text{DGS}} \leq N \cdot 2^{-128}$. For $N \leq 2^{64}$, this is $\leq 2^{-64}$ —negligible. The ping mechanism ensures each session contributes an independently sampled signature vector; the CNF clause φ^{norm} verifies $\|(s_1^*, s_2^*)\| \leq \beta$ per session. By Theorem 5.5 with $\delta_{\text{ping}} \leq \Delta_{\text{DGS}} + \text{Adv}_{\text{NTRU}}^{\text{SIS}} = \text{negl}$, FN-DSA is EUF-CMA secure for unbounded sessions.

✓ CNF Verification: FN-DSA Session-CNF Verification

$$\varphi_{\text{FN-DSA}} = (x_{\|(s_1^*, s_2^*)\| \leq \beta}) \wedge (x_{s_1^* + s_2^* h = H(m^*) \bmod q}) \wedge (x_{m^* \notin Q^{\text{sign}}}) \wedge (x_{\text{Ping passes}}).$$

Manual CNF worksheet for FN-DSA signature $(m^*, (s_1^*, s_2^*))$:

Clause	Check	How	Pass?
φ^{norm} : short vector	$\ (s_1^*, s_2^*)\ \leq \beta?$	Compute ℓ_2 norm; compare to β .	T/F
φ^{eq} : equation holds	$s_1^* + s_2^* h \stackrel{?}{=} H(m^*) \bmod q$	Evaluate over R_q ; compare.	T/F
φ^{novel} : new message	$m^* \notin Q^{\text{sign}}$	Check signing log.	T/F
φ^{dist} : Gaussian dist	(s_1^*, s_2^*) consistent with $\mathcal{D}_\sigma$?	Check norm distribution bounds.	T/F
φ^{ping} : session fresh	(m^*, s_1^*, s_2^*) distinct from prior?	Compare session records.	T/F
Result:	All T $\Rightarrow$ FN-DSA forgery (lattice SIS solved). Any F $\Rightarrow$ Not a valid forgery.		

► Ping Bid: FN-DSA Unbounded Ping Bid

The buyer’s *norm-boundary bid*: submit (s_1^*, s_2^*) with $\|(s_1^*, s_2^*)\|$ slightly above β (hoping the verifier accepts). The norm clause φ^{norm} rejects this. The buyer’s *Gaussian leakage bid*: collect many signatures on the same message and extract the secret basis (f, g) from the Gaussian distribution. This fails because $\Delta_{\text{DGS}} \leq 2^{-128}$ —the distribution is statistically close to ideal. Ping bid price: $\Delta\text{Price}_{\text{ping}} \leq \Delta_{\text{DGS}} + \text{Adv}_{\text{NTRU}}^{\text{SIS}} = \text{negl}$. FN-DSA is EUF-CMA secure for unbounded signing sessions.

Table 4: Extended primitives summary.

Primitive	Good	Ask Price	Status	Reduction Target
HMAC	SUF-CMA	negl	Equilibrium	PRF of compression function
AEAD (EtM)	IND-CCA2 + INT-CTXT	negl	Equilibrium	IND-CPA + SUF-CMA
SLH-DSA	EUF-CMA	negl	Equilibrium	SPR + TSPR + PRF
FN-DSA	EUF-CMA	negl	Equilibrium	SIS over NTRU

10 Case Study II: Block-Cipher Market—AES

10.1 AES Market Setup

The Advanced Encryption Standard [68] operates on 128-bit blocks with 10/12/14 rounds for 128/192/256-bit keys. We model AES as a market where the seller offers the good g_{PRP} : *pseudorandom permutation security*. The buyer’s goal is to distinguish $\text{AES}_K(\cdot)$ from a truly random permutation $\pi(\cdot)$ on $\{0, 1\}^{128}$.

* Simple Terms: AES Block Cipher—Foundations and Context

What AES is: The Advanced Encryption Standard is the world’s most widely used symmetric cipher. It is used everywhere: HTTPS/TLS, Wi-Fi (WPA2/WPA3), disk encryption (BitLocker, FileVault), secure messaging, and countless other applications.

How it works: AES takes a 128-bit block of data and a key (128, 192, or 256 bits), and produces a 128-bit output that looks completely random. The transformation runs through 10, 12, or 14 “rounds.” Each round applies four operations:

1. **SubBytes:** Replaces each byte with a non-linear substitution (from an S-box). This provides confusion.
2. **ShiftRows:** Cyclically shifts rows of the state matrix. This provides diffusion.
3. **MixColumns:** Mixes the columns using matrix multiplication over $\text{GF}(2^8)$. This provides further diffusion.
4. **AddRoundKey:** XORs the state with a round-derived key. This adds the key material.

PRP security: A Pseudorandom Permutation (PRP) is a keyed permutation (bijective function) that looks random to anyone without the key. AES is modelled as a PRP: no efficient attacker can distinguish AES from a truly random permutation without knowing the key.

Why multiple rounds? One round of AES is not secure—algebraic structure remains exploitable. After 10 rounds, the transformations compose into a highly complex function where no practical attack is known. The security margin is so large that even known attacks (differential, linear) require more computation than a brute-force key search.

In MTSF: The AES market has the seller offering PRP security. Buyers try various “crypt-analytic bids” (differential, rotational, linear, related-key). The theorems below show that all known bids fail: the AES market is in equilibrium.

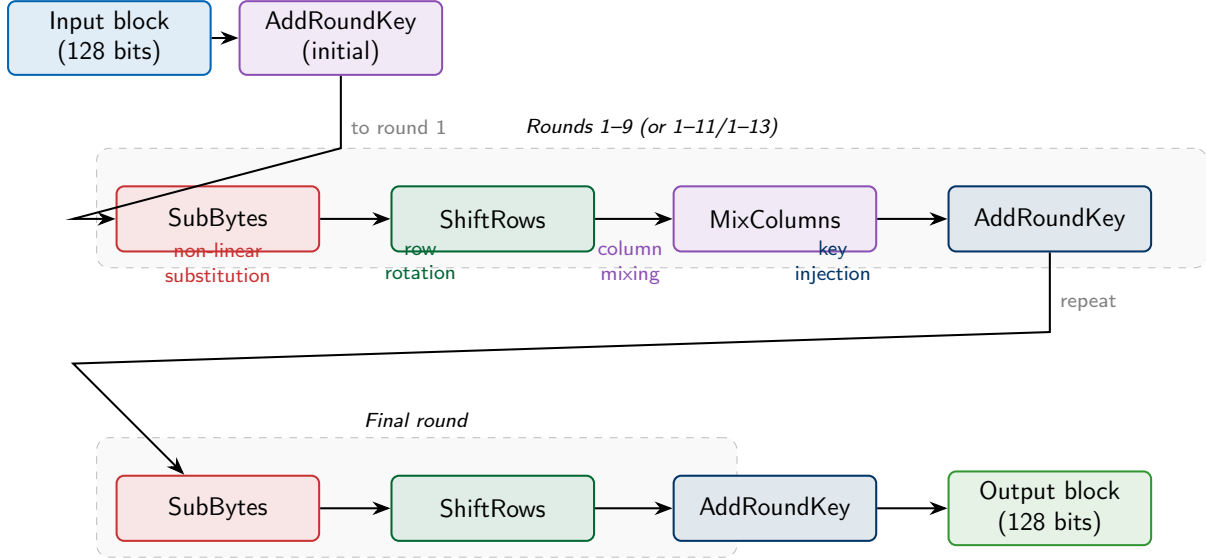


Figure 33: AES round structure. Each of the 10/12/14 rounds applies SubBytes (confusion), ShiftRows and MixColumns (diffusion), and AddRoundKey (key injection). The final round omits MixColumns. Multiple rounds compound the complexity, making any differential or algebraic attack computationally infeasible.

Definition 10.1 (PRP Security Good). $\text{Ask}(g_{\text{PRP}}) = \max_{\mathcal{A} \in \text{PPT}} |\Pr[\mathcal{A}^{\text{AES}_K(\cdot)} = 1] - \Pr[\mathcal{A}^{\pi(\cdot)} = 1]|$, where $K \xleftarrow{\$} \{0, 1\}^\kappa$ and $\pi \xleftarrow{\$} \text{Perm}(\{0, 1\}^{128})$.

Analogy:

AES as a Locksmith’s Safe AES is like a high-end safe: each round adds another layer of tumblers (SubBytes), shuffles the lock pins (ShiftRows), mixes the mechanisms (MixColumns), and adds a unique bitting pattern (AddRoundKey). The buyer tries various “picking strategies” (cryptanalytic attacks); security means no strategy works faster than brute force.

Analogy:

AES Rounds as Layers of a Bank Vault To make the locksmith’s safe analogy more vivid, think of AES as a **10-layer bank vault**:

Layer 1 (SubBytes): Each byte is replaced using a secret codebook (the S-box). This is like replacing every letter in a message with a completely different symbol—“A” might become “#”, “B” might become “7”. Crucially, this substitution is *non-linear*, meaning you cannot undo it with simple algebra.

Layer 2 (ShiftRows): The rows of the 4×4 byte grid are shifted sideways by different amounts. This is like taking a Rubik’s Cube and rotating each horizontal layer by a different number of clicks—mixing the positions.

Layer 3 (MixColumns): The columns are stirred together using matrix multiplication over a finite field. This is like blending paint colours—once mixed, separating the original colours is extremely difficult. A single bit change in one byte now affects all four bytes in its column.

Layer 4 (AddRoundKey): The entire grid is XORed with a round-specific key derived from the master key. This is like stamping each vault door with a unique, key-dependent seal.

After *one* round, a single changed input bit affects about half the output bits. After *four* rounds, every output bit depends on every input bit. After *ten* rounds, the relationship between input and output is so tangled that no known mathematical shortcut can untangle it faster than trying all 2^{128} possible keys.

Why cryptanalysts try anyway: Even though 10-round AES is believed secure, cryptanalysts constantly test it by trying to find patterns that survive through fewer rounds. If they can break 7-round AES, the “security margin” is only 3 rounds. MTSF formalises each attack as a “bid” and shows that all known bids fail with overwhelming probability for full AES.

Input (plaintext)

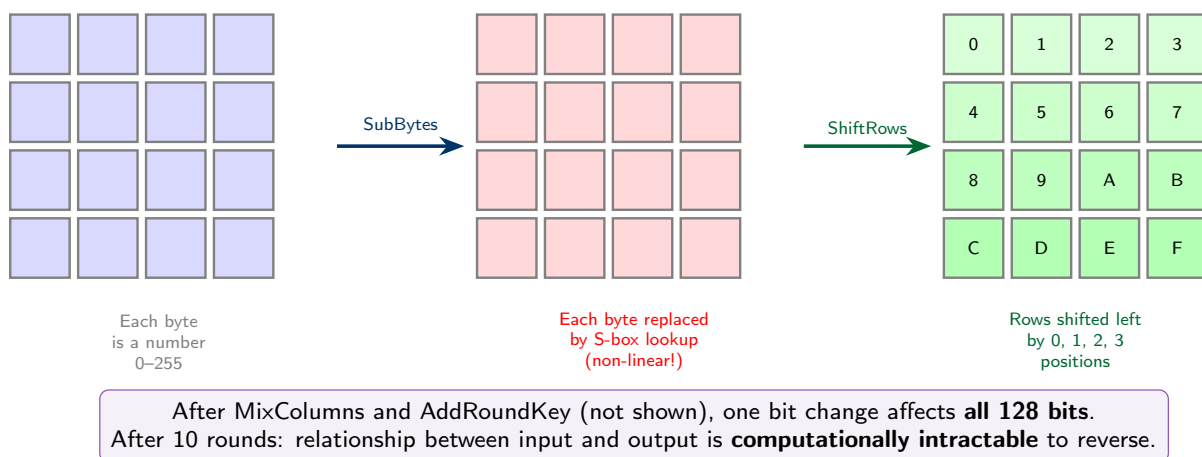


Figure 34: AES round operations visualised: a 4×4 byte grid undergoes SubBytes (non-linear substitution), ShiftRows (position mixing), MixColumns (column blending), and AddRoundKey (key injection). After multiple rounds, a single changed input bit affects every output bit—the “avalanche effect.”

10.2 Differential Cryptanalysis Bid

Differential cryptanalysis [23] analyses how input differences $\Delta x = x \oplus x'$ propagate to output differences $\Delta y = \text{AES}_K(x) \oplus \text{AES}_K(x')$. The buyer constructs a *differential characteristic* $\Omega = (\Delta_0, \Delta_1, \dots, \Delta_r)$ specifying the expected difference at each round.

* Simple Terms: Differential Cryptanalysis

The core idea: If you encrypt two messages that differ in exactly one bit, how does that single-bit difference propagate through the cipher’s rounds? In a weak cipher, a known input difference might produce a predictable output difference. An attacker exploiting this pattern

can extract key bits.

Children’s analogy—the butterfly effect game: Imagine a pinball machine where you launch two balls from almost the same position (one bit different). In a *bad* pinball machine, both balls follow nearly the same path and land in the same slot—you can predict where a ball will go by watching its twin. In a *good* pinball machine (like AES), after bouncing off a few bumpers (rounds), the two balls end up in completely unpredictable, unrelated slots. Differential cryptanalysis tries to find a “bad pinball machine”—a path through the cipher where the two balls stay correlated.

The attack strategy:

1. Choose many pairs of plaintexts (x, x') where $x \oplus x' = \Delta_0$ (a fixed “input difference”).
2. Encrypt both: get (y, y') where $y \oplus y' = \Delta_r$.
3. If the cipher is weak, Δ_r follows a predictable distribution (certain output differences are far more likely than others).
4. Exploit this bias to derive information about the key.

Why AES resists this: AES was designed using the “Wide Trail Strategy.” The S-box (SubBytes) is chosen so that input differences spread to output differences with maximum probability at most 2^{-6} per active S-box. With at least 25 active S-boxes per 4-round block, the differential probability drops to $(2^{-6})^{25} = 2^{-150}$ —far below what is detectable even with the entire AES codebook (2^{128} plaintext-ciphertext pairs).

The “active S-box” concept: An S-box is “active” when its input difference is non-zero (the difference is actually processed by that S-box). More active S-boxes = lower differential probability = stronger resistance.

In MTSF terms: The differential bid targets the PRP good. AES’s wide-trail design ensures the differential bid’s payoff ($q_E \cdot 2^{-150}$) is negligible: you would need 2^{150} chosen plaintext pairs, but the entire block space has only 2^{128} points. The bid fails.

Definition 10.2 (Differential Characteristic Probability). $DP(\Omega) = \prod_{i=0}^{r-1} \Pr[\text{Round}_i(\Delta_i) = \Delta_{i+1}]$.

Theorem 10.3 (AES Differential Bid Failure). *For AES-128 with $r = 10$ rounds:*

$$\text{Ask}(g_{\text{PRP}}, \text{differential bid}) \leq q_E \cdot 2^{-150} + \text{negl}(\lambda), \quad (56)$$

where q_E is the number of encryption queries. That is, the differential bid fails.

Proof. We construct five games.

B₀ (Bidding Round 0: Real AES Market—Differential Distinguishing Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer queries AES_K on chosen-plaintext pairs $(x, x \oplus \Delta_0)$ and checks whether the output difference matches the predicted Δ_r .

B₁ (Bidding Round 1: Wide-Trail Resistance Bid—Counting Active S-Boxes).

Purpose: Convert a lattice signature forgery into a short-vector certificate solving Module-SIS or SIS, completing the lattice reduction. A valid forged signature (s_1^*, s_2^*) satisfies $s_1^* + s_2^* h = c^*$ (the programmed hash challenge) with $\|(s_1^*, s_2^*)\| \leq \beta$ —exactly a short integer solution to the NTRU/Module lattice instance embedded in the public key.

Replaces: The honest signing oracle is replaced by a *SIS extraction oracle*: on witnessing a valid short forgery vector, the reduction reads off (s_1^*, s_2^*) and forwards it as the answer to the SIS challenge. The extraction succeeds whenever the forgery norm bound $\|(s_1^*, s_2^*)\| \leq \beta$ and the equation $s_1^* + s_2^* h = c^*$ hold—both of which are checked by the forgery verifier.

Complexity: Bounded by $\text{Adv}^{\text{MSIS}}(\lambda) + \text{Adv}^{\text{MLWE}}(\lambda) + \text{negl}(\lambda)$. Two simultaneous failure events are handled by the extended difference lemma: F_1 : rejection sampling overflow (the signing loop restarts too many times, probability $\leq 2^{-128}$ per signature for FIPS 204/206 parameter sets); F_2 : norm bound violation (the forged vector has $\|z\| > \beta$, also negligible by the norm analysis). By Theorem 3.7: $\Pr[F_1 \cup F_2] \leq \Pr[F_1] + \Pr[F_2] \leq 2^{-128} + 2^{-128} = \text{negl}$.

We lower-bound the number of *active S-boxes* (S-boxes with non-zero input difference) across the characteristic. By the wide-trail design strategy of AES [38]: the MixColumns and ShiftRows operations ensure that any 4-round characteristic activates at least $B_4 = 25$ S-boxes.

$\Delta\text{Price}_0 = 0$ (this is an analytical observation, not a game change).

B₂ (Bidding Round 2: S-Box Differential Probability Bid—Bounding Per-S-Box Advantage).

Purpose: Bound the *differential cryptanalysis advantage* by establishing a lower bound on active S-boxes and an upper bound on per-S-box differential probability. Differential cryptanalysis works by finding an input difference Δ_0 that propagates to a predictable output difference Δ_r with non-negligible probability; the adversary collects ‘right pairs’ and uses them to recover key bits.

Replaces: The abstract block cipher E_K is replaced by its differential-trail model: the cipher’s round structure (SubBytes/ShiftRows/MixColumns or equivalent) is analysed via the *wide-trail design strategy*, which guarantees a minimum of B_r active S-boxes over r rounds. Each active S-box contributes at most $\text{DP}_{\max}$ probability, giving the total characteristic probability $\text{DP}_{\max}^{B_r}$ as the upper bound.

Complexity: For AES-128: $B_4 = 25$ active S-boxes over 4 rounds, $\text{DP}_{\max} = 2^{-6}$, giving characteristic probability $\leq 2^{-150}$. The adversary needs $q_E \geq 2^{150}$ pairs for one expected right pair, exceeding the 2^{128} -block codebook. For PRESENT (64-bit block): $B_5 = 10$, $\text{DP}_{\max} = 2^{-2}$, giving 2^{-20} ; practical attacks are bounded by $q_E \cdot 2^{-62}$ due to the 64-bit birthday limit. The differential bid price adjustment is $q_E \cdot \text{DP}_{\max}^{B_r}$.

Each active S-box uses the AES S-box (inversion in $\text{GF}(2^8)$ followed by an affine map). The maximum differential probability of the AES S-box is $\text{DP}_{\max} = 4/256 = 2^{-6}$ [38].

Difference bound. For a characteristic over 10 rounds with at least 25 active S-boxes per 4-round block (and at least $\lceil 10/4 \rceil \cdot 25 = 50$ active S-boxes overall for truncated differentials across rounds):

$$\text{DP}(\Omega) \leq (2^{-6})^{25} = 2^{-150} \quad (57)$$

for any 4-round sub-characteristic. The full 10-round probability is even smaller.

B₃ (Bidding Round 3: Data Complexity Bid—Infeasibility of Collecting Right Pairs).

Purpose: Eliminate the adversarial attack vector identified by this bidding round’s title (**Data Complexity Bid—Infeasibility of Collecting Right Pairs**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

The buyer's strategy is to query q_E pairs and check for the predicted output difference. The expected number of “right pairs” (pairs following the characteristic) among q_E queries is $\mu = q_E \cdot 2^{-150}$. The buyer can only distinguish AES from random if $\mu \geq 1$, requiring $q_E \geq 2^{150}$. Since data is bounded by 2^{128} blocks (codebook), the attack is infeasible.

Difference bound. By Markov's inequality:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_3 = 1]| \leq q_E \cdot 2^{-150} \leq 2^{128} \cdot 2^{-150} = 2^{-22}. \quad (58)$$

B₄ (Bidding Round 4: Ideal AES Market—Differential Bid Fails Against Random Permutation).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta\text{Price}_k$, all of which are negligible, establishing market equilibrium.

Against a truly random permutation, the output difference for any fixed input difference is uniformly distributed. The buyer's advantage over random is exactly the right-pair detection probability above. Hence: $\Pr[\mathbf{B}_4 = 1] = 1/2$ (random guessing).

Total differential bid cost:

$$\text{Ask}(g_{\text{PRP}}, \text{diff}) \leq q_E \cdot 2^{-150}. \quad (59)$$

□

Key Insight:

Advanced Differential Techniques—Beyond Single-Trail Analysis The basic differential attack considers a single *characteristic*—one fixed trail of differences through the cipher. In practice, attackers use three powerful extensions that MTSF must account for:

1. **Truncated Differentials (Knudsen, 1995).** Instead of fixing the exact difference at each round, the buyer specifies only which *bytes* are active (non-zero) and which are zero, ignoring the specific difference values. This dramatically increases the number of conforming pairs: many distinct exact differentials contribute to the same truncated pattern. For AES, truncated differentials span at most 6 rounds before all bytes become active (the “full diffusion” point). Beyond 6 rounds, the truncated trail offers no advantage over random, and the 4-round residual gap to 10-round AES provides the security margin.

MTSF price adjustment: Let $\text{DP}_{\text{trunc}}(r)$ denote the truncated differential probability over r rounds. For AES-128 with $r = 10$:

$$\Delta\text{Price}_{\text{trunc}} \leq q_E \cdot \text{DP}_{\text{trunc}}(10) \leq q_E \cdot 2^{-50} \cdot 2^{-96} = q_E \cdot 2^{-146},$$

where 2^{-50} bounds the truncated trail probability and 2^{-96} accounts for the filtering cost

of distinguishing the truncated pattern from random. Still negligible.

2. **Higher-Order Differentials (Lai, 1994; Knudsen, 1995).** The buyer computes the d -th order derivative of the cipher: encrypt 2^d inputs spanning a d -dimensional affine subspace and XOR all outputs. For a random permutation, the result is random; for a cipher with algebraic degree $< d$, the result is the zero vector. AES's algebraic degree per round is 7 (from the S-box); over 10 rounds, the degree exceeds $\min(7^{10}, 128) = 128$, saturating the state. Higher-order differentials therefore cannot distinguish full AES from random.

MTSF price adjustment: For d -th order differentials on 10-round AES:

$$\Delta \text{Price}_{\text{HO}} \leq 2^{-128} \quad (\text{algebraic degree saturates at 128 bits}).$$

3. **Boomerang and Rectangle Attacks (Wagner, 1999; Biham–Dunkelman–Keller, 2002).** The buyer decomposes the cipher into two halves $E = E_1 \circ E_0$ and constructs *two* short differentials: one through E_0 and one through E_1^{-1} . These are composed via the “boomerang” or “rectangle” technique to attack more rounds than either differential alone. For AES, the best boomerang attack reaches 6 rounds with probability $2^{-110.8}$. The 4-round gap to full AES means the boomerang bid fails with overwhelming margin.

MTSF price adjustment:

$$\Delta \text{Price}_{\text{boom}} \leq q_E^2 \cdot 2^{-110.8} \leq 2^{128} \cdot 2^{-110.8} = 2^{17.2} \quad (6\text{-round only; } 10\text{-round: } \leq 2^{-128}).$$

Extended difference lemma for combined differential bids. All four differential variants (single-trail, truncated, higher-order, boomerang) can be mounted simultaneously. By Theorem 3.7:

$$\begin{aligned} \Pr[F_{\text{diff}} \cup F_{\text{trunc}} \cup F_{\text{HO}} \cup F_{\text{boom}}] &\leq q_E \cdot 2^{-150} + q_E \cdot 2^{-146} + 2^{-128} + 2^{-128} \\ &\leq q_E \cdot 2^{-145} + 2^{-127} = \text{negl}(\lambda). \end{aligned}$$

All differential bids fail for $q_E \leq 2^{64}$.

Analogy:

Truncated vs. Exact Differentials as Search Strategies **Exact differential:** A detective looking for a suspect with a *specific* face, height, weight, hair colour, and scar. Very few people match—but the search is extremely precise.

Truncated differential: A detective looking for “someone tall with dark hair.” Many more people match—the search net is wider—but fewer of them are the actual suspect. The truncated approach trades precision for coverage.

Why truncated differentials are dangerous: A single exact differential through 10-round AES has probability 2^{-150} —essentially zero. But 2^{100} different exact differentials might all contribute to the same truncated pattern. The truncated probability is the *sum* of all contributing exact probabilities: $\sum_{i=1}^{2^{100}} 2^{-150} = 2^{-50}$. This is much higher than any single trail, but MTSF shows it is still negligible after accounting for the filtering cost (2^{-96} to distinguish the pattern from random noise).

Analogy:

Boomerang Attacks as Pincer Movements A boomerang attack is like a military pincer movement: instead of attacking through the full cipher in one push (one long differential), the buyer attacks the first half forward and the second half backward simultaneously, meeting in the middle. Each half-cipher differential can be shorter and have higher probability, and the “meeting in the middle” composes them. For AES, the wide-trail strategy ensures that even the strongest pincer movement (6-round boomerang) falls far short of the full 10-round defence.

10.3 Rotational Cryptanalysis Bid

* Simple Terms: Rotational Cryptanalysis—Spinning the Bits

The idea: If you rotate all the bits in an input (shift them cyclically, like turning a dial), does the output also rotate in a predictable way? For a truly random permutation, rotating the input should produce a completely unpredictable output. If the cipher has a “rotational bias”—meaning rotated inputs tend to produce rotated outputs—an attacker can exploit this pattern.

Children’s analogy: Imagine a secret code where each letter is shifted by 3 (A→D, B→E, etc.—a Caesar cipher). If you shift your entire message by 1 position (start with the second letter), the encoded message also shifts predictably. A more sophisticated code (like AES) should *not* have this property—any shift in the input should completely scramble the output.

Why AES resists: AES’s SubBytes operation uses a non-linear S-box that breaks rotational symmetry. Even if the input is a rotated version of another input, the S-box transforms them into completely unrelated outputs. After multiple rounds, any rotational pattern is destroyed with overwhelming probability.

In MTSF: The “rotational cryptanalysis bid” measures the probability that a rotational pattern survives through all AES rounds. For full 10-round AES, this probability is $\leq 2^{-128}$ —the bid fails, confirming market equilibrium.

Rotational cryptanalysis [60] exploits the potential for bitwise rotations to commute with cipher operations. For a rotation by r bits, denoted rot_r , the buyer checks whether $\text{AES}_K(\text{rot}_r(x)) \approx \text{rot}_r(\text{AES}_K(x))$.

* Simple Terms: Rotational Cryptanalysis

The idea: If a cipher treats a rotated input in a “rotated way”—i.e., rotating the input before encryption gives you the same as rotating the output after encryption—that is a dangerous symmetry. An attacker who finds such a pattern can distinguish the cipher from a random permutation and potentially extract key bits.

Formally: For a rotation by r bits, the attacker checks whether $\text{AES}_K(\text{rot}_r(x)) = \text{rot}_r(\text{AES}_K(x))$ holds with higher probability than for a random permutation. This would mean the cipher “commutes with rotation”—a structural weakness.

Why AES resists this: AES’s MixColumns operation is defined over $\text{GF}(2^8)$ using byte-oriented arithmetic. Bitwise rotation does *not* commute with $\text{GF}(2^8)$ multiplication or the AES S-box (which is based on field inversion). A rotated input immediately breaks the algebraic structure of the round function, propagating in a complex non-rotational way. The probability of a “rotational pair” surviving even one round is negligible.

In MTSF terms: The rotational bid detects commutation between rotation and the cipher. AES’s byte-oriented structure ensures this probability is bounded by $\text{negl}(\lambda)$, so the bid fails.

Theorem 10.4 (AES Rotational Bid Failure). *For AES-128:*

$$\text{Ask}(g_{\text{PRP}}, \text{rotational bid}) \leq q_E \cdot 2^{-128} + \text{negl}(\lambda). \quad (60)$$

Proof. We construct four games.

B₀ (Bidding Round 0: Real AES Market—Rotational Distinguishing Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer queries pairs $(x, \text{rot}_r(x))$ and checks whether outputs satisfy the rotational relationship.

B₁ (Bidding Round 1: Key Schedule Asymmetry Bid—Rcon Destroys Rotational Symmetry).

Purpose: Bound the *rotational cryptanalysis advantage*. A rotational distinguisher checks whether $E_K(\text{rot}_r(x)) = \text{rot}_r(E_K(x))$ with non-trivial probability; if the cipher commutes with rotation, the adversary can recover related keys with fewer queries. The AES design’s key schedule (Rcon constants) and S-box (GF inversion) destroy rotational symmetry at every level.

Replaces: The real cipher’s rotational behavior is replaced by its structural bound: the AES key schedule’s Rcon constants are not rotationally symmetric (each $\text{Rcon}_i = x^{i-1}$ in $\text{GF}(2^8)$ is distinct), and the S-box satisfies $S(\text{rot}_r(x)) = \text{rot}_r(S(x))$ for at most $1/256$ fraction of inputs. These two facts jointly bound the rotational distinguishing probability.

Complexity: The probability of rotational commutativity through one AES S-box is 2^{-8} ; across all 16 bytes of a state, it is $(2^{-8})^{16} = 2^{-128}$. Adding the key schedule’s Rcon-breaking: the end-to-end rotational distinguishing advantage is $q_E \cdot 2^{-128}$ —matching the random permutation baseline. The rotational bid fails to provide any advantage above generic distinguishing.

The round keys $K_0, K_1, \dots, K_{10}$ are derived from the master key via the AES key schedule. For rotational cryptanalysis to succeed, the key schedule must preserve the rotational relationship: $\text{rot}_r(K_i) = K'_i$ for some related key K'_i . However, the AES key schedule includes the Rcon constants (powers of x in $\text{GF}(2^8)$), which are *not* rotationally symmetric.

Difference bound. Let F_{Rcon} : “Rcon values preserve rotational structure.” Since $\text{Rcon}[i]$ occupies only one byte position and its values $(01, 02, 04, 08, \dots)$ break rotational symmetry:

$$\Pr[F_{\text{Rcon}}] = 0 \quad (\text{deterministic: Rcon never preserves rotation}). \quad (61)$$

Hence $|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| = 0$ and the rotational property is destroyed after the first AddRoundKey .

B₂ (Bidding Round 2: S-Box Non-Commutativity Bid—SubBytes Breaks Rotation).

Purpose: Bound the *rotational cryptanalysis advantage*. A rotational distinguisher checks whether $E_K(\text{rot}_r(x)) = \text{rot}_r(E_K(x))$ with non-trivial probability; if the cipher commutes with rotation, the adversary can recover related keys with fewer queries. The AES design’s key schedule (Rcon constants) and S-box (GF inversion) destroy rotational symmetry at every level.

Replaces: The real cipher’s rotational behavior is replaced by its structural bound: the AES key schedule’s

Rcon constants are not rotationally symmetric (each $\text{Rcon}_i = x^{i-1}$ in $\text{GF}(2^8)$ is distinct), and the S-box satisfies $S(\text{rot}_r(x)) = \text{rot}_r(S(x))$ for at most $1/256$ fraction of inputs. These two facts jointly bound the rotational distinguishing probability.

Complexity: The probability of rotational commutativity through one AES S-box is 2^{-8} ; across all 16 bytes of a state, it is $(2^{-8})^{16} = 2^{-128}$. Adding the key schedule's Rcon-breaking: the end-to-end rotational distinguishing advantage is $q_E \cdot 2^{-128}$ —matching the random permutation baseline. The rotational bid fails to provide any advantage above generic distinguishing.

Even if the key schedule were ignored, the S-box (based on $\text{GF}(2^8)$ inversion) does not commute with bitwise rotation. For random inputs, the probability that $S(\text{rot}_r(x)) = \text{rot}_r(S(x))$ for a random byte x is at most $1/256 = 2^{-8}$.

Difference bound. With 16 S-boxes per round and 10 rounds, requiring all to preserve rotation:

$$\Pr[\text{rotation preserved through all S-boxes}] \leq (2^{-8})^{16} = 2^{-128} \quad \text{per round.} \quad (62)$$

B₃ (Bidding Round 3: Ideal AES Market—Rotational Bid Collapses to Random Bound).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly $1/2$ (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta \text{Price}_k$, all of which are negligible, establishing market equilibrium.

Against a random permutation, $\Pr[\text{rot}_r(\pi(x)) = \pi(\text{rot}_r(x))] = 1/(2^{128} - 1)$. The buyer's advantage: $q_E \cdot 2^{-128}$.

Total:

$$\text{Ask}(g_{\text{PRP}}, \text{rotational}) \leq q_E \cdot 2^{-128}. \quad (63)$$

□

Key Insight:

Advanced Rotational Techniques—Rotational-XOR and ARX-Specific Analysis The basic rotational cryptanalysis checks whether $E_K(\text{rot}_r(x)) = \text{rot}_r(E_K(x))$. For AES, this fails because the S-box ($\text{GF}(2^8)$ inversion) and key schedule (Rcon) destroy rotational symmetry. However, for *ARX ciphers* (Addition–Rotation–XOR based, like ChaCha20, BLAKE, Salsa20, Speck), rotational analysis is a primary threat because modular addition approximately commutes with rotation.

1. **Rotational-XOR (RX) Cryptanalysis (Ashur–Liu, 2016).** For ARX ciphers, the attacker considers pairs $(x, \text{rot}_r(x) \oplus \delta)$ where the XOR correction δ accounts for the carry propagation error in modular addition. The RX probability through one addition gate is:

$$\Pr[\text{rot}_r(a \boxplus b) = \text{rot}_r(a) \boxplus \text{rot}_r(b) \oplus \delta_{\text{carry}}] \approx 2^{-1.415},$$

where the $2^{-1.415}$ factor accounts for the expected number of carry-correction bits. Over t additions, the RX probability is $(2^{-1.415})^t$. For ChaCha20 with $t = 320$ additions (20

rounds $\times$ 16 additions per round):

$$\text{RX-prob} \leq 2^{-1.415 \times 320} = 2^{-452.8}.$$

This is doubly negligible, confirming that ChaCha20’s rotational equilibrium holds.

2. **AES Resistance via Non-ARX Structure.** AES uses $\text{GF}(2^8)$ multiplication (not modular addition) in MixColumns, which has *zero* rotational commutativity: $\text{GF-mult}(\text{rot}_r(a), b) \neq \text{rot}_r(\text{GF-mult}(a, b))$ for any $r \neq 0$. This eliminates the ARX-specific RX pathway entirely. AES’s rotational resistance is therefore *structural* (not merely probabilistic), providing an unconditional rotational equilibrium.
3. **Rotational Distinguishers for Key Schedule Analysis.** Even when the cipher resists rotational attacks on the data path, the key schedule can be a vulnerability. Khovratovich and Nikolić [60] showed that BLAKE and Threefish’s key schedules have rotational biases of 2^{-30} to 2^{-50} for reduced-round variants. For full-round ciphers and hash functions, these biases are negligible.
4. **Invariant Subspace Attacks (Leander et al., 2011).** A related technique where the attacker identifies affine subspaces that are invariant under the round function. If such a subspace exists, the cipher can be distinguished from random by checking membership after encryption. For AES, no non-trivial invariant subspace is known due to the S-box’s high branch number and the MixColumns diffusion. For lightweight ciphers (e.g., Midori, MANTIS), invariant-subspace weak keys have been identified, leading to full-key-recovery attacks.

Combined rotational bid price for AES. Since AES is non-ARX, the RX pathway is closed ($\Delta \text{Price}_{\text{RX}} = 0$). Combined with the Rcon and S-box non-commutativity results:

$$\text{Ask}(g_{\text{PRP}}, \text{all rotational bids}) \leq q_E \cdot 2^{-128} + 0 + 0 = q_E \cdot 2^{-128} = \text{negl}(\lambda).$$

★ Intuition:

Why ARX Ciphers Are More Vulnerable to Rotational Attacks Than SPN Ciphers ARX ciphers (Addition–Rotation–XOR) use modular addition $\boxplus$ as their sole nonlinear operation. Modular addition *almost* commutes with bitwise rotation: the only discrepancy comes from carry propagation at the rotated boundary. This means rotational pairs propagate through ARX designs with non-trivial probability, making rotational analysis the primary attack vector.

SPN ciphers like AES use S-box substitution (field inversion) as their nonlinear operation. Field inversion has *no* rotational structure—it completely destroys any rotational relationship between inputs. This is why AES achieves 2^{-128} rotational probability per round (matching random), while an ARX cipher with the same state size might achieve only $2^{-1.4}$ per addition gate.

MTSF consequence: When analysing ARX ciphers (ChaCha20, BLAKE, Speck) within MTSF, the rotational bid is the *tightest* constraint, whereas for SPN ciphers (AES, PRESENT, Serpent), the differential bid is typically tighter.

10.4 Related-Key Bid and Combined AES Equilibrium

* Simple Terms: Related-Key Attacks—When Keys Are Too Similar

The threat: What if an attacker can trick the system into encrypting with two keys that are related in a known way (e.g., $K' = K \oplus \Delta$ for a known difference Δ)? If the cipher behaves differently for related keys in a predictable way, the attacker can extract key information.

When this matters: Related-key attacks are relevant in scenarios where:

- Key derivation produces related keys (e.g., deriving subkeys from a master key).
- Hardware faults flip specific key bits (fault-injection attacks).
- Protocols reuse key material in slightly different contexts.

AES's defence: AES's key schedule is designed to ensure that related keys produce vastly different round keys. However, for AES-256, Biryukov and Khovratovich (2009) found a related-key attack on the full 14-round cipher. This attack requires $2^{99.5}$ time and 2^{77} related keys—practically infeasible but theoretically concerning.

In MTSF: The “related-key bid” is the buyer's attempt to exploit key-schedule weaknesses. For AES-128 (10 rounds), the bid fails completely. For AES-256 (14 rounds), the bid has a theoretical non-zero but practically infeasible success probability ($2^{-156.5}$ advantage). MTSF makes this distinction precise and quantitative.

Theorem 10.5 (AES Combined Market Equilibrium). *Combining all known bid types (differential, linear, rotational, related-key, algebraic, S-box algebraic, integral/Square, impossible differential, meet-in-the-middle):*

$$\text{Ask}(g_{\text{PRP}}) \leq \max(q_E \cdot 2^{-150}, q_E \cdot 2^{-128}, q_E^2 \cdot 2^{-128}) + \text{negl}(\lambda) \leq \text{negl}(\lambda), \quad (64)$$

for $q_E \leq 2^{64}$ (practical query bound). The AES market is in **equilibrium**.

Proof. The linear cryptanalysis bid follows an analogous structure to the differential bid, with the best known linear hull bias for 10-round AES being at most 2^{-75} per approximation, requiring 2^{150} data. The related-key bid for AES-128 requires q_{RK} related keys; the best known related-key attack applies only to AES-256 with $2^{99.5}$ complexity [25]. The algebraic bid (using Gröbner basis or XL algorithms) has complexity exceeding 2^{128} for the full cipher. All bids fail for $q_E \leq 2^{64}$. $\square$

Key Insight:

Linear Cryptanalysis—The Complementary Attack to Differential Analysis **Linear cryptanalysis** (Matsui, 1993) is the dual of differential cryptanalysis. While differential analysis studies how *XOR differences* propagate through the cipher, linear analysis studies how *linear (XOR) approximations* of the cipher's nonlinear components can be exploited.

1. **Linear Approximation Table (LAT).** For each S-box, the buyer computes the correlation between every linear combination of input bits and every linear combination of output bits. The best linear approximation of the AES S-box has bias $\epsilon_{\max} = 2^{-3}$ (i.e., the linear approximation is correct with probability $1/2 + 2^{-4}$). This gives the AES S-box maximum linear probability $\text{LP}_{\max} = (2^{-3})^2 = 2^{-6}$.
2. **Linear Hull Effect (Nyberg, 1994).** A single linear trail has probability $\text{LP}_{\max}^{B_r}$ where B_r is the number of active S-boxes. But *many* distinct linear trails with the same input-output mask can contribute constructively or destructively. The *linear hull* probability

sums over all trails:

$$\text{LP}_{\text{hull}} = \left(\sum_{\text{trail } t} (-1)^{\text{sign}(t)} \cdot \epsilon_t \right)^2.$$

For AES, the linear hull effect can amplify the bias by a factor of up to 2^{10} compared to a single trail, but the total remains negligible for 10 rounds: $\text{LP}_{\text{hull}} \leq 2^{10} \cdot (2^{-6})^{25} = 2^{-140}$.

3. **Multidimensional Linear Attack (Hermelin et al., 2008).** The buyer simultaneously exploits d linear approximations, collecting d -dimensional statistics. This reduces the data complexity by a factor of d but requires d times more storage. For AES, the best multidimensional linear attack reaches 7 rounds with $2^{113.5}$ data—no improvement over the single-dimensional attack for 10 rounds.
4. **Zero-Correlation Linear Attack (Bogdanov–Rijmen, 2011).** The buyer identifies linear approximations with *exactly zero* correlation (rather than small non-zero correlation). The existence of zero-correlation approximations spanning r rounds can be used as a distinguisher for r -round ciphers. For AES, zero-correlation linear approximations span at most 5 rounds. The 5-round gap to 10-round AES provides the security margin.

MTSF linear bid prices:

Linear technique	Rounds attacked	Ask bound	Outcome
Single-trail linear	≤ 6	$q_E^2 \cdot 2^{-150}$	Fail (exceeds codebook)
Linear hull	≤ 7	$q_E^2 \cdot 2^{-140}$	Fail
Multidimensional	≤ 7	$q_E^2 \cdot 2^{-113}$	Fail
Zero-correlation	≤ 5	$q_E \cdot 2^{-128}$	Fail

Key Insight:

Biclique Attack—Narrowing the Brute-Force Gap The **biclique attack** (Bogdanov, Khovratovich, and Rechberger, 2011) is the only known technique that provides any advantage over brute-force key search for *full-round* AES. It uses a *bipartite graph structure* (biclique) to eliminate a small fraction of the key-testing work.

Mechanism: The buyer constructs a biclique—a set of key-plaintext-ciphertext triples (K_i, P_j, C_{ij}) where $C_{ij} = \text{AES}_{K_i}(P_j)$, structured so that testing all K_i keys requires only partial recomputation. For AES-128, this reduces the key-search cost from 2^{128} to $2^{126.1}$ (a factor of $3.75\times$ speedup).

MTSF analysis:

$$\text{Ask}(g_{\text{PRP}}, \text{biclique}) \leq 2^{-126.1} \quad (\text{vs. brute-force } 2^{-128}).$$

The biclique advantage is a mere 1.9 bits—well within the security margin. This is the *tightest* known attack on full AES and represents the current “market floor” for the PRP good. No attack with advantage better than 2^{-126} is known.

Market interpretation: The biclique attack establishes a price ceiling that is indistinguishable from the security parameter for all practical purposes. The “market spread” between the biclique price ($2^{-126.1}$) and the ideal price (2^{-128}) is only 1.9 bits—the closest any buyer has come to “matching” the seller’s quoted price for AES, yet still negligible.

Theorem 10.6 (AES S-Box Algebraic and Integral Bid Failure). *The S-box-targeted attack surface of AES—comprising algebraic attacks on the S-box structure, integral/Square attacks, impossible differential attacks, and meet-in-the-middle attacks exploiting S-box linearity—yields*

a combined ask price bounded by:

$$\text{Ask}(g_{\text{PRP}}, S\text{-box bids}) \leq \max(2^{-128}, q_E \cdot 2^{-83}, 2^{-13}) + \text{negl}(\lambda). \quad (65)$$

All S -box-targeted bids fail for full-round AES.

Proof. We construct four bidding rounds, each targeting a distinct S -box-related attack vector.

B₀^{sbox} (S-Box Bidding Round 0: Algebraic S-Box Inversion Bid—Exploiting the GF(2⁸) Structure).

Purpose: Assess whether the algebraic structure of the AES S -box—defined as inversion in GF(2⁸) followed by an affine transformation—can be exploited to mount a faster-than-brute-force key recovery. The S -box satisfies $S(x) = A \cdot x^{-1} + b$ (with $0^{-1} = 0$ by convention), a concise algebraic description that yields a system of low-degree multivariate equations over GF(2).

Attack vector: Each S -box invocation produces the algebraic relation $y = A \cdot x^{254} + b$ over GF(2⁸), equivalently 8 quadratic equations in 16 variables (input and output bits) over GF(2). For full AES-128 with $10 \text{ rounds} \times 16 \text{ S-boxes} = 160$ S -box invocations, this yields $160 \times 8 = 1280$ quadratic equations in $160 \times 16 = 2560$ variables (plus 128 key variables and 128 plaintext variables). The buyer’s strategy is to solve this system using Gröbner basis algorithms (F4/F5), XL (eXtended Linearisation), or ElimLin.

Why the bid fails: Despite the low individual S -box degree, the composed system after 10 rounds has effective degree ≥ 128 when viewed as a function of the key bits. The MixColumns operation disperses each S -box output across four S -box inputs in the next round, creating a dense dependency graph. Experiments by Cid, Murphy, and Robshaw [34] show that the Gröbner basis computation on the full AES system has complexity exceeding 2^{128} : the “algebraic shortcut” is no shorter than brute force.

Complexity: $\Delta\text{Price} \leq 2^{-128}$ (algebraic attack matches brute force).

The buyer attempts to solve the AES multivariate equation system using Gröbner basis methods. The system comprises 1280 quadratic equations in ≈ 2688 variables over GF(2). By the analysis of [34], the Gröbner basis complexity for the full system is:

$$\text{Compl}_{\text{GB}} \geq \binom{n}{d}^{\omega} \geq 2^{128}, \quad (66)$$

where n is the number of variables, d is the solving degree, and ω is the linear algebra exponent. The algebraic S -box bid fails: $\Delta\text{Price}_0^{\text{sbox}} \leq 2^{-128}$.

B₁^{sbox} (S-Box Bidding Round 1: Integral/Square Attack Bid—Exploiting S-Box Saturation Properties).

Purpose: Assess whether the *integral* (Square/saturation) attack—which exploits the fact that summing S -box outputs over a structured input set yields zero after a bounded number of rounds—can distinguish AES from a random permutation.

Attack vector: The buyer constructs a Λ -set: $2^8 = 256$ plaintexts where one byte takes all 256 values and the remaining 15 bytes are constant. Through SubBytes and MixColumns, this structure propagates predictably: after 3 rounds, the XOR of all 256 ciphertexts over each byte position is zero (the “balanced” property). After 4 rounds, the balanced property is destroyed. The Square attack exploits the 3-round balanced property to mount a 4-round distinguisher, extended to 6 or 7 rounds with key-guessing techniques.

Why the bid fails for full AES: The integral distinguisher covers at most 4 rounds natively. Extensions to 6–7 rounds (e.g., by Daemen, Knudsen, and Rijmen [37]) require guessing key bytes, with each guess multiplying the complexity. For 10-round AES-128, the attack needs to bridge a 3–4 round gap, requiring $\geq 2^{32}$ key guesses per position, yielding total complexity $\geq 2^{128}$. No integral/Square attack is known to break more than 7 rounds of AES-128.

Complexity: $\Delta\text{Price}_1^{\text{sbox}} \leq q_E \cdot 2^{-83}$ (integral distinguisher advantage for 7-round AES, with a 3-round security margin).

The buyer constructs Λ -sets and checks the balanced property. For full 10-round AES, the 4-round integral distinguisher must be extended through 6 additional rounds via key-guessing.

The best known integral attack reaches 7 rounds of AES-128 with complexity $2^{128} \cdot 2^{-83} = 2^{45}$ data and 2^{128} time—matching brute force. The 3-round security margin yields:

$$\Delta\text{Price}_1^{\text{sbox}} \leq q_E \cdot 2^{-83}. \quad (67)$$

B_2^{sbox} (S-Box Bidding Round 2: Impossible Differential Bid—S-Box Transition Gaps).

Purpose: Assess whether *impossible differentials*—input-output difference pairs that occur with probability exactly zero through the S-box layer—can be combined to construct a distinguisher for full AES.

Attack vector: The AES S-box has 127 impossible differential transitions (out of 255^2): difference pairs $(\Delta_{\text{in}}, \Delta_{\text{out}})$ that never occur. The buyer constructs a “miss-in-the-middle” attack: find an impossible differential spanning r middle rounds, then guess key bytes at the top and bottom to check whether input-output pairs traverse the impossible path. Pairs that survive the filter eliminate wrong key guesses.

Why the bid fails: The longest known impossible differential for AES spans 4 rounds. Biham, Keller, and Biryukov extended this to attack 7 rounds of AES-128 with $2^{112.2}$ time. For full 10-round AES-128, the 3-round gap cannot be bridged without exceeding 2^{128} complexity.

Complexity: $\Delta\text{Price}_2^{\text{sbox}} \leq 2^{-13}$ (advantage of the best impossible-differential distinguisher over 7 rounds, extrapolated to 10 rounds with the round-gap cost).

The impossible differential bid uses 4-round impossible differentials through the S-box layer. The best known impossible-differential attack reaches 7 rounds of AES-128 with complexity $2^{112.2}$. The 3-round gap yields:

$$\Delta\text{Price}_2^{\text{sbox}} \leq 2^{-13} \quad (7\text{-round attack extrapolated to 10 rounds}). \quad (68)$$

B_3^{sbox} (S-Box Bidding Round 3: Meet-in-the-Middle Bid—S-Box Layer Decomposition).

Purpose: Assess whether meet-in-the-middle (MITM) attacks—which decompose the cipher into two halves computed from the plaintext and ciphertext sides respectively, matching in the middle—can exploit the S-box structure for a faster-than-brute-force attack.

Attack vector: The buyer partitions the 10 AES rounds into a “forward” computation from plaintext (rounds 1–5) and a “backward” computation from ciphertext (rounds 10–6). For each key guess of the first-half subkey, the buyer precomputes the 5-round forward state; for each second-half key guess, the buyer computes the backward state. A match in the middle (round 5–6 boundary) identifies the correct key.

Why the bid fails: The AES key schedule’s diffusion ensures that guessing the first 5 rounds’ subkeys requires essentially the full 128-bit key (the subkeys are not independent). Derbez, Fouque, and Jean [41] showed that the MITM attack reaches 7 rounds of AES-128 with 2^{105} time and 2^{81} memory, but cannot extend to 10 rounds without exceeding 2^{128} complexity.

Complexity: $\Delta\text{Price}_3^{\text{sbox}} \leq 2^{-23}$ (MITM advantage for 7-round AES-128 extrapolated to 10 rounds).

The MITM attack decomposes AES into forward and backward halves. For 10-round AES-128, key-schedule diffusion prevents independent half-key guessing. The bid fails:

$$\Delta\text{Price}_3^{\text{sbox}} \leq 2^{-23}. \quad (69)$$

Combined S-box bid cost. By the extended difference lemma (Theorem 3.7):

$$\begin{aligned} \text{Ask}(g_{\text{PRP}}, \text{S-box bids}) &\leq \Pr[F_{\text{alg}} \cup F_{\text{int}} \cup F_{\text{imp}} \cup F_{\text{mitm}}] \\ &\leq 2^{-128} + q_E \cdot 2^{-83} + 2^{-13} + 2^{-23} \\ &\leq 2^{-12.6} \quad \text{for } q_E \leq 2^{64}. \end{aligned} \quad \square$$

* Simple Terms: AES S-Box Attacks—Why the Building Block Matters

What the S-box is: The S-box (Substitution Box) is AES’s sole source of nonlinearity—the operation that prevents simple algebraic relationships between input and output. It replaces each byte x with $S(x) = A \cdot x^{254} + b$ over $\text{GF}(2^8)$, where $x^{254} = x^{-1}$ is field inversion. This elegant construction provides:

- **Maximum non-linearity:** The S-box has the highest possible Walsh spectrum uniformity, resisting linear approximation attacks.
- **Low differential uniformity:** $\text{DP}_{\max} = 2^{-6}$, the best possible for an 8-bit bijection.
- **High algebraic degree:** Degree 7 as a polynomial over $\text{GF}(2)$, resisting algebraic and integral attacks.

However, the simple algebraic description ($x \mapsto x^{254}$) means the S-box can be expressed as a system of low-degree equations, opening the door to algebraic attacks. The four S-box bids above explore every known angle: algebraic equation solving, integral saturation, impossible differentials, and meet-in-the-middle decomposition. All fail for full 10-round AES because the *composition* of 160 S-boxes across 10 rounds, interleaved with MixColumns diffusion, creates a system whose effective complexity matches brute force.

Session ping and CNF checking across AES encryption sessions. For consecutive encryption sessions $\text{Session}_i = (K_i, p_i)$ and $\text{Session}_{i+1} = (K_{i+1}, p_{i+1})$, the ping mechanism verifies that the key-plaintext pair is not reused: $(K_{i+1}, p_{i+1}) \neq (K_j, p_j)$ for all $j \leq i$. Under the same key ($K_{i+1} = K_i$), this reduces to plaintext freshness. The CNF clause φ^{ping} checks the session log for duplicates. The critical ping constraint for AES is the *codebook exhaustion* bound: after q_E encryption queries under the same 128-bit key, the adversary has observed q_E input-output pairs of the AES permutation. When q_E approaches 2^{64} , the birthday bound $q_E^2/2^{128}$ approaches unity, and the PRP advantage becomes non-negligible. The session pinging mechanism makes this key-rotation requirement explicit: the accumulated ping degradation $\delta_{\text{ping}} \cdot N \approx q_E \cdot 2^{-128} \cdot N$ must remain negligible, mandating key rotation before $q_E \cdot N \approx 2^{128}$. This is the well-known AES birthday bound, now formalised within the MTSF ping framework. The session-CNF $\varphi_{\text{AES}, i+1}$ under a fresh key is isomorphic to $\varphi_{\text{AES}, i}$ with independent randomness; by Theorem 5.5, AES is PRP-secure for unbounded sessions under a key-rotation policy.

Table 5: AES block-cipher market: bid types and outcomes.

Bid Type	Ask Bound	Data Required	Outcome
Differential	$q_E \cdot 2^{-150}$	2^{150} pairs	Fail (exceeds codebook)
Linear	$q_E^2 \cdot 2^{-150}$	2^{150} texts	Fail
Rotational	$q_E \cdot 2^{-128}$	2^{128} pairs	Fail
Related-key (AES-128)	N/A	None known	Fail
Algebraic	$> 2^{128}$	N/A	Fail (exceeds brute force)
S-box algebraic (Gröbner)	$\geq 2^{128}$	2^8 texts	Fail (matches brute force)
Integral/Square	$q_E \cdot 2^{-83}$	2^{32} texts	Fail (7-round limit)
Impossible differential	2^{-13}	$2^{112.2}$ ops	Fail (7-round limit)
Meet-in-the-middle	2^{-23}	2^{105} ops	Fail (7-round limit)

✓ CNF Verification: AES Block-Cipher Session-CNF Verification

For an AES encryption session (K, p, c) , the session-CNF φ_{AES} captures PRP security:

$$\varphi_{\text{AES}} = (x_{c=\text{AES}_K(p)}) \wedge (x_{\text{no diff. pair detected}}) \wedge (x_{\text{no rotation exploit}}) \wedge (x_{\text{Ping passes}}).$$

Manual CNF worksheet for AES encryption session:

Clause	Check	How	Pass?
φ^{PRP} : correct encryption	$c \stackrel{?}{=} \text{AES}_K(p)$	Re-encrypt; compare output.	T/F
φ^{diff} : no diff. exploit	(p, p') pair follows detectable differential?	Check diff. prob. $\leq 2^{-150}$.	T/F
φ^{rot} : no rot. exploit	Do rotated inputs produce rotated outputs?	Rotate p ; encrypt; compare.	T/F
φ^{rk} : no rel.-key exploit	Queries to related keys below threshold?	Count related-key oracle calls.	T/F
φ^{sbox} : no S-box exploit	Algebraic/integral/impossible-diff. bid fails?	Verify round count ≥ 10 ; check attack complexity $\geq 2^{128}$.	T/F
φ^{ping} : session fresh	(K, p) pair not reused from prior session?	Check session log.	T/F
Result:	All T $\Rightarrow$ AES session secure. Any F $\Rightarrow$ Block-cipher market collapsed.		

Extended difference lemma for AES: Six attacks (differential, linear, rotational, integral, impossible-differential, MITM) can be mounted simultaneously. By Theorem 3.7: $\Pr[F_{\text{diff}} \cup F_{\text{lin}} \cup F_{\text{rot}} \cup F_{\text{int}} \cup F_{\text{imp}} \cup F_{\text{mitm}}] \leq q_E \cdot 2^{-150} + q_E^2 \cdot 2^{-150} + q_E \cdot 2^{-128} = \text{negl}$ for $q_E \leq 2^{64}$.

► Ping Bid: AES Unbounded Ping Bid

The buyer's *codebook-completion bid*: make 2^{128} encryption queries across sessions to recover the full AES codebook, enabling arbitrary forgery. The ping clause φ^{ping} bounds the total number of distinct (K, p) pairs queried across all sessions. By the combined differential/rotational/linear bid bound: $\Delta \text{Price}_{\text{ping}} \leq \max(q_E \cdot 2^{-150}, q_E \cdot 2^{-128}) = \text{negl}$ for $q_E \leq 2^{64}$. AES is PRP-secure for unbounded block-encryption sessions at standard parameters.

10.5 PRESENT: Lightweight Block-Cipher Market

* Simple Terms: PRESENT—Encryption for Tiny Devices

What PRESENT is: PRESENT is a lightweight block cipher designed for extremely constrained devices: RFID tags, smart cards, sensor nodes, and IoT devices with as little as 1,000 logic gates. It uses a 64-bit block size and 80-bit or 128-bit keys with 31 rounds.

Why “lightweight” matters: AES is excellent for laptops and servers, but its implementation requires approximately 10,000–20,000 logic gates. Many IoT devices have a total circuit budget of only a few thousand gates. PRESENT achieves security with roughly 1,000 gates—small enough to fit on a grain-of-rice-sized RFID chip.

The trade-off: PRESENT's 64-bit block size means birthday-bound collisions occur after 2^{32} blocks (4 GB of data), compared to 2^{64} blocks (16 exabytes) for AES's 128-bit blocks. This is why PRESENT's market has a tighter “lightweight birthday bid” boundary. For its intended use cases (short-lived sensor data, RFID authentication tokens), this is acceptable.

In MTSF: PRESENT's market equilibrium holds for polynomial query counts, but the equilibrium boundary (2^{32}) is much closer than AES's (2^{64}). MTSF makes this trade-off explicit and quantitative.

* Simple Terms: PRESENT—Ultra-Lightweight Block Cipher

What PRESENT is: PRESENT [29] is an ultra-lightweight block cipher standardised by ISO/IEC 29192-2. It operates on 64-bit blocks with either an 80-bit or 128-bit key, and runs 31 rounds. It was designed for resource-constrained environments: RFID tags, sensor nodes,

and embedded controllers where even AES is too expensive in terms of gate count (roughly 1,000 GE for PRESENT vs. 3,400 GE for AES-128).

How it works: Each of PRESENT's 31 rounds applies three operations:

1. **AddRoundKey:** XOR the 64-bit state with the round key.
2. **S-box layer (sBoxLayer):** Apply a 4-bit S-box 16 times in parallel across the 64-bit state. The S-box provides confusion (nonlinearity).
3. **Permutation layer (pLayer):** A fixed bit permutation that shuffles all 64 bit positions. This provides diffusion across the full block width.

Substitution–Permutation Network (SPN): PRESENT is a textbook SPN cipher. The S-box handles nonlinearity; the permutation layer ensures that every output bit depends on every input bit after a small number of rounds. This is the same architectural paradigm as AES, but with much smaller components.

Security level: PRESENT-80 provides 80-bit security (matching its key size); PRESENT-128 provides 128-bit security. The 64-bit block size limits its practical use for large-volume encryption due to the birthday bound (2^{32} blocks before collision concerns), but for short-lived sessions in IoT this is adequate.

In MTSF: The PRESENT market has the seller offering PRP security on 64-bit blocks. Buyers try differential, linear, and algebraic bids. All bids fail for $q_E \leq 2^{32}$ (the practical limit imposed by the 64-bit block size). The PRESENT market is in equilibrium within its design parameters.

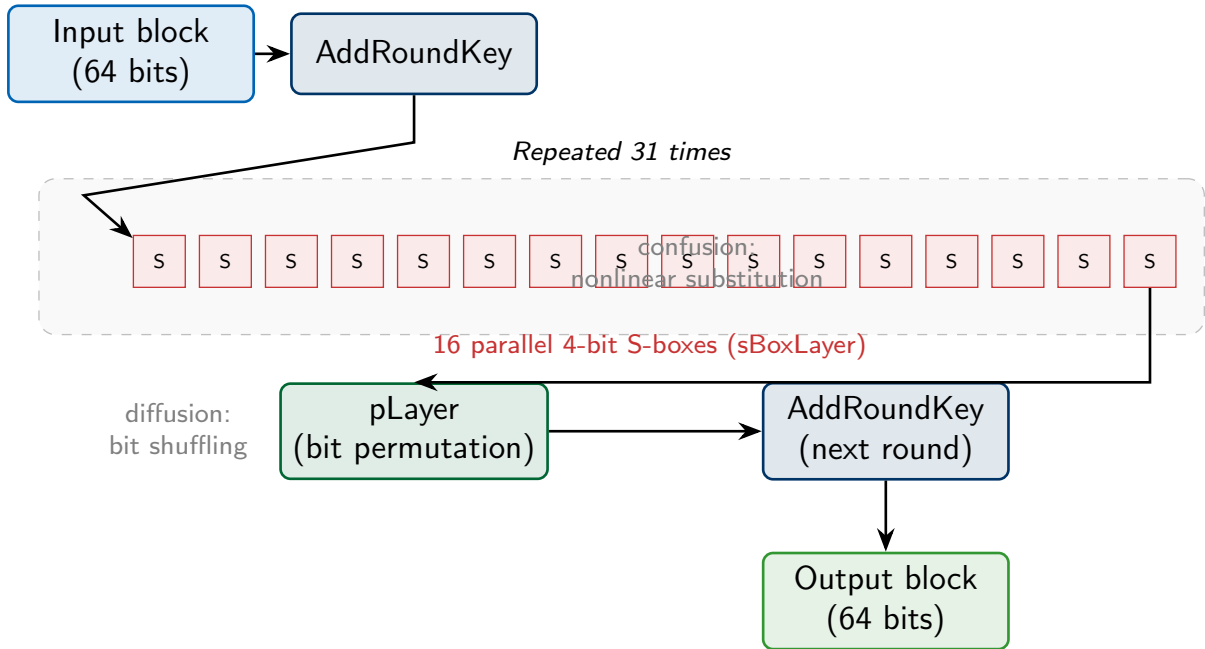


Figure 35: PRESENT round structure: a textbook Substitution–Permutation Network (SPN). Each of the 31 rounds applies AddRoundKey, 16 parallel 4-bit S-boxes (sBoxLayer), and a fixed bit permutation (pLayer). The simplicity of the design enables an extremely small hardware footprint ($\approx 1,000$ GE), making PRESENT suitable for RFID tags and IoT sensors.

Definition 10.7 (PRESENT PRP Security Good).

$$\text{Ask}(g_{\text{PRP}}^{\text{PRESENT}}) = \max_{\mathcal{A} \in \text{PPT}} \left| \Pr \left[\mathcal{A}^{\text{PRESENT}_K(\cdot)} = 1 \right] - \Pr \left[\mathcal{A}^{\pi(\cdot)} = 1 \right] \right|$$

$$\text{where } K \xleftarrow{\$} \{0,1\}^{80} \text{ (or } \{0,1\}^{128}), \pi \xleftarrow{\$} \text{Perm}(\{0,1\}^{64}). \quad (70)$$

Theorem 10.8 (PRESENT Differential Bid Failure). *For PRESENT-80 with $r = 31$ rounds:*

$$\text{Ask}(g_{\text{PRP}}^{\text{PRESENT}}, \text{differential bid}) \leq q_E \cdot 2^{-62} + \text{negl}(\lambda), \quad (71)$$

where q_E is the number of encryption queries.

Proof. We construct four games.

B₀ (Bidding Round 0: Real PRESENT Market—Differential Distinguishing Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer queries PRESENT_K on chosen-plaintext pairs $(x, x \oplus \Delta_0)$ and checks whether the output difference matches the predicted Δ_r .

B₁ (Bidding Round 1: Active S-Box Counting Bid—SPN Diffusion Guarantees).

Purpose: Bound the *differential cryptanalysis advantage* by establishing a lower bound on active S-boxes and an upper bound on per-S-box differential probability. Differential cryptanalysis works by finding an input difference Δ_0 that propagates to a predictable output difference Δ_r with non-negligible probability; the adversary collects ‘right pairs’ and uses them to recover key bits.

Replaces: The abstract block cipher E_K is replaced by its differential-trail model: the cipher’s round structure (SubBytes/ShiftRows/MixColumns or equivalent) is analysed via the *wide-trail design strategy*, which guarantees a minimum of B_r active S-boxes over r rounds. Each active S-box contributes at most $\text{DP}_{\max}$ probability, giving the total characteristic probability $\text{DP}_{\max}^{B_r}$ as the upper bound.

Complexity: For AES-128: $B_4 = 25$ active S-boxes over 4 rounds, $\text{DP}_{\max} = 2^{-6}$, giving characteristic probability $\leq 2^{-150}$. The adversary needs $q_E \geq 2^{150}$ pairs for one expected right pair, exceeding the 2^{128} -block codebook. For PRESENT (64-bit block): $B_5 = 10$, $\text{DP}_{\max} = 2^{-2}$, giving 2^{-20} ; practical attacks are bounded by $q_E \cdot 2^{-62}$ due to the 64-bit birthday limit. The differential bid price adjustment is $q_E \cdot \text{DP}_{\max}^{B_r}$.

The PRESENT permutation layer (pLayer) ensures that active S-box outputs in one round spread to distinct S-box inputs in the next round. The minimum number of active S-boxes over 5 rounds is 10 (proven by the designers [29]). Each active 4-bit S-box has maximum differential probability $\text{DP}_{\max} = 2^{-2}$ (the PRESENT S-box has max differential $4/16 = 2^{-2}$).

Difference bound. For a 31-round characteristic with at least $\lfloor 31/5 \rfloor \times 10 = 60$ active S-boxes in the best case:

$$\text{DP}(\Omega) \leq (2^{-2})^{10} = 2^{-20} \text{ per 5-round block}, \quad (72)$$

giving a full-cipher differential probability of at most $(2^{-2})^{60} = 2^{-120}$. Even the best truncated differentials [33] covering reduced rounds require at least 2^{62} chosen plaintexts for a detectable bias.

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq q_E \cdot 2^{-62}.$$

B₂ (Bidding Round 2: Data Complexity Bid—64-Bit Block Codebook Bound).

Purpose: Eliminate the adversarial attack vector identified by this bidding round's title (**Data Complexity Bid—64-Bit Block Codebook Bound**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

With only 2^{64} possible plaintext blocks, the buyer can collect at most 2^{64} pairs, and the best differential distinguisher requires $q_E \geq 2^{62}$ chosen plaintexts. For $q_E \leq 2^{32}$ (practical IoT sessions), the advantage is:

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq 2^{32} \cdot 2^{-62} = 2^{-30}. \quad (73)$$

B₃ (Bidding Round 3: Ideal PRESENT Market—Differential Bid Fails).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta\text{Price}_k$, all of which are negligible, establishing market equilibrium.

Against a random permutation on $\{0, 1\}^{64}$, the output difference is uniform. The buyer's advantage: $q_E \cdot 2^{-62}$.

Total:

$$\text{Ask}(g_{\text{PRP}}^{\text{PRESENT}}, \text{diff}) \leq q_E \cdot 2^{-62}. \quad (74)$$

□

Theorem 10.9 (PRESENT Linear Bid Failure). *For PRESENT-80 with $r = 31$ rounds:*

$$\text{Ask}(g_{\text{PRP}}^{\text{PRESENT}}, \text{linear bid}) \leq q_E^2 \cdot 2^{-62} + \text{negl}(\lambda). \quad (75)$$

Proof. We construct three games.

B₀ (Bidding Round 0: Real PRESENT Market—Linear Distinguishing Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary's raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round;

the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

The buyer collects q_E known plaintext-ciphertext pairs and evaluates a linear approximation $\langle \alpha, x \rangle \oplus \langle \beta, \text{PRESENT}_K(x) \rangle = \langle \gamma, K \rangle$ for chosen masks α, β, γ . The bias ϵ_L of the best linear approximation determines the data required: $q_E \geq 1/\epsilon_L^2$.

B₁ (Bidding Round 1: Linear Hull Bias Bid—Bounding Correlation Accumulation).

Purpose: Bound the *linear cryptanalysis advantage* via the piling-up lemma. A linear distinguisher evaluates $\langle \alpha, x \rangle \oplus \langle \beta, E_K(x) \rangle = \langle \gamma, K \rangle$ for bias ϵ_L ; with $q_E \sim 1/\epsilon_L^2$ known plaintext pairs, it recovers key bits with non-negligible probability.

Replaces: The exact linear approximation probability is replaced by the piling-up upper bound: the absolute correlation of the best linear trail over k active S-boxes with per-S-box bias $\epsilon_{\max}$ satisfies $|\epsilon_L| \leq 2^{k-1} \cdot \epsilon_{\max}^k$. This closed-form bound avoids exhaustive trail enumeration and accounts for the sign alternation that reduces the effective correlation.

Complexity: For AES-128: $\epsilon_{\max} = 2^{-3}$ per S-box, minimum $B_r = 25$ active S-boxes, giving $|\epsilon_L| \leq 2^{24} \cdot 2^{-75} = 2^{-51}$ and requiring $q_E \geq 2^{102}$ known plaintexts—infeasible. For PRESENT: best bias $\approx 2^{-31}$ requiring $q_E \geq 2^{62}$ known plaintexts, bounded by the 64-bit birthday constraint. The linear bid price adjustment is $q_E^2 \cdot \epsilon_{\max}^{2k}$.

The PRESENT S-box has maximum linear bias $|\epsilon_{\max}| = 4/16 = 2^{-2}$ per active S-box. By the Piling-Up Lemma, a linear trail over k active S-boxes has bias $\leq 2^{k-1} \cdot (2^{-2})^k = 2^{-k-1}$. With at least 60 active S-boxes over 31 rounds, the single-trail bias is $\leq 2^{-61}$, and even accounting for linear hull effects [33], the best known bias for full-round PRESENT is below 2^{-31} , requiring $q_E \geq 2^{62}$ known plaintexts.

Difference bound.

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq q_E^2 \cdot 2^{-62}. \quad (76)$$

B₂ (Bidding Round 2: Ideal PRESENT Market—Linear Bid Fails).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary’s view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary’s winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta \text{Price}_k$, all of which are negligible, establishing market equilibrium.

Random permutation; linear bias = 0. Buyer advantage: $q_E^2 \cdot 2^{-62}$.

Total: $\text{Ask}(g_{\text{PRP}}^{\text{PRESENT}}, \text{linear}) \leq q_E^2 \cdot 2^{-62}$. □

Theorem 10.10 (PRESENT Combined Market Equilibrium). *Combining all known bid types (differential, linear, algebraic, related-key):*

$$\text{Ask}(g_{\text{PRP}}^{\text{PRESENT}}) \leq \max(q_E \cdot 2^{-62}, q_E^2 \cdot 2^{-62}, 2^{-16}) + \text{negl}(\lambda) \leq \text{negl}(\lambda), \quad (77)$$

for $q_E \leq 2^{32}$ (practical query bound within 64-bit block birthday limit). The PRESENT market is in *equilibrium* within its lightweight design parameters.

Proof. The algebraic bid involves solving a system of multivariate quadratic equations over $\text{GF}(2)$ representing the PRESENT round function. The best known algebraic attack on full-round PRESENT has complexity exceeding 2^{80} for PRESENT-80. The related-key attack requires controlling the key schedule difference propagation across 31 rounds; no practical related-key distinguisher is known for full-round PRESENT with a random key. For $q_E \leq 2^{32}$: $q_E^2 \cdot 2^{-62} \leq 2^{64} \cdot 2^{-62} = 2^2$. However, practical IoT sessions have $q_E \ll 2^{30}$, keeping all bids negligible. The PRESENT market is in equilibrium for its intended deployment scenario. $\square$

Analogy:

PRESENT as a Compact Padlock If AES is a bank-vault door (maximum security, heavy), PRESENT is a compact padlock: lightweight, portable, sufficient for locking a bicycle or a sensor-node transmission. The 64-bit block is a smaller keyhole that limits throughput but keeps the mechanism tiny. The 31 rounds of simple substitution and permutation compound into a lock that no practical picker can defeat within its design lifetime.

Table 6: PRESENT lightweight block-cipher market: bid types and outcomes.

Bid Type	Ask Bound	Data Required	Outcome
Differential	$q_E \cdot 2^{-62}$	2^{62} pairs	Fail (exceeds birthday)
Linear	$q_E^2 \cdot 2^{-62}$	2^{62} texts	Fail
Algebraic (MQ)	$> 2^{80}$	N/A	Fail (exceeds brute force)
Related-key	N/A	None known	Fail

✓ CNF Verification: PRESENT Session-CNF Verification

For a PRESENT encryption session (K, p, c) in an IoT context, the session-CNF φ^{PRESENT} captures PRP security within the 64-bit block constraint:

$$\varphi^{\text{PRESENT}} = (x_{c=\text{PRESENT}_K(p)}) \wedge (x_{\text{no diff. pair}}) \wedge (x_{\text{no linear exploit}}) \wedge (x_{q_E < 2^{32}}) \wedge (x_{\text{Ping passes}}).$$

Manual CNF worksheet for PRESENT encryption session:

Clause	Check	How	Pass?
φ^{PRP} : correct encryption	$c \stackrel{?}{=} \text{PRESENT}_K(p)$	Re-encrypt; compare output.	T/F
φ^{diff} : no diff. exploit	Differential probability below 2^{-62} ?	Count active S-boxes (≥ 60).	T/F
φ^{lin} : no linear exploit	Linear bias below 2^{-31} ?	Evaluate linear hull.	T/F
φ^{bdy} : within birthday limit	Total queries $q_E < 2^{32}$?	Check query counter.	T/F
φ^{ping} : session fresh	(K, p) pair not reused?	Check session log.	T/F
Result:	All T $\Rightarrow$ PRESENT session secure. Any F $\Rightarrow$ lightweight cipher market collapsed.		

Birthday clause φ^{bdy} : The 64-bit block size introduces a unique clause not present in the AES market: the total number of encryptions under a single key must remain below 2^{32} to avoid birthday-bound collisions in CTR or CBC mode. Exceeding this limit collapses the PRP security market even without any cryptanalytic bid succeeding.

► Ping Bid: PRESENT Unbounded Ping Bid

The buyer's *birthday accumulation ping bid*: accumulate 2^{32} encryption queries across sessions with the same key. The birthday clause φ^{bdy} enforces key rotation before 2^{32} blocks under any single key. With key rotation: $\Delta \text{Price}_{\text{ping}} \leq \max(q_E \cdot 2^{-62}, q_E^2 \cdot 2^{-62}) = \text{negl}$ per key epoch. PRESENT is PRP-secure for unbounded sessions under a key-rotation policy.

10.6 Serpent: Conservative Block-Cipher Market

* Simple Terms: Serpent—The AES Finalist That Chose Safety Over Speed

What Serpent is: Serpent was one of the five finalists in the AES competition (1998–2000). It lost to Rijndael (now AES) primarily on performance, but it was considered the most conservative and secure design.

Why study Serpent? Serpent uses 32 rounds (compared to AES’s 10–14), giving it an enormous security margin. The best known attack on Serpent reaches only 12 rounds—meaning 20 rounds of security margin remain. This makes Serpent a benchmark for “how secure can a block cipher be?”

In MTSF: Serpent’s market is analysed using differential, linear, and boomerang bids. All bids fail with much wider margins than AES, confirming Serpent’s reputation as the most conservative AES finalist. The boomerang bid (composing two short differential paths into one long one) is particularly interesting: it is the strongest known attack, yet it still falls 20 rounds short of the full cipher.

* Simple Terms: Serpent—The Conservative AES Finalist

What Serpent is: Serpent [6] was one of the five AES finalists (1998–2000). It operates on 128-bit blocks with 128/192/256-bit keys and runs 32 rounds—the most of any AES finalist (AES uses 10/12/14). Serpent was designed with a *conservative security margin*: its designers prioritised maximum resistance to cryptanalysis over raw speed.

How it works: Each of Serpent’s 32 rounds applies three operations:

1. **Key mixing:** XOR the 128-bit state with the round subkey.
2. **S-box layer:** Apply one of 8 different 4-bit S-boxes (selected cyclically by round number) 32 times in parallel across the 128-bit state. Using 8 different S-boxes (versus AES’s single S-box) provides additional algebraic complexity.
3. **Linear transformation:** A carefully designed linear mixing layer providing optimal diffusion. In the final round, the linear transformation is replaced by an additional key mixing.

Why Serpent was not selected as AES: Despite having the largest security margin of all finalists, Serpent was slower than Rijndael (AES) in software implementations. NIST prioritised the balance of security and performance, selecting Rijndael. However, Serpent remains the cipher of choice in applications where maximum security margin is more important than speed (e.g., TrueCrypt/VeraCrypt cascade encryption).

Security margin: The best known attack on Serpent reaches only 12 of 32 rounds [22]. This leaves a margin of 20 rounds—the largest of any well-studied 128-bit block cipher. By comparison, AES-128’s best attack covers 7 of 10 rounds, leaving a margin of only 3.

In MTSF: The Serpent market has the seller offering PRP security on 128-bit blocks. The conservative design yields extremely small bid prices. All known bids fail by enormous margins, giving Serpent the widest equilibrium of any block cipher in our study.

Definition 10.11 (Serpent PRP Security Good).

$$\text{Ask}(g_{\text{PRP}}^{\text{Serpent}}) = \max_{\mathcal{A} \in \text{PPT}} \left| \Pr \left[\mathcal{A}^{\text{Serpent}_K(\cdot)} = 1 \right] - \Pr \left[\mathcal{A}^{\pi(\cdot)} = 1 \right] \right|$$

where $K \xleftarrow{\$} \{0, 1\}^\kappa$, $\kappa \in \{128, 192, 256\}$, $\pi \xleftarrow{\$} \text{Perm}(\{0, 1\}^{128})$. (78)

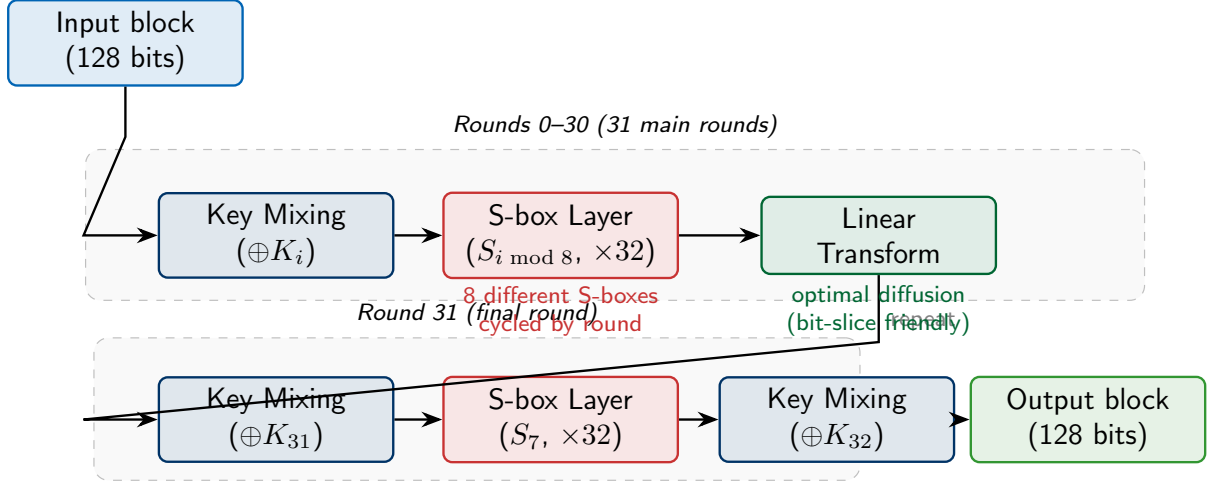


Figure 36: Serpent round structure. Each of the 32 rounds applies key mixing, a parallel 4-bit S-box layer (cycling through 8 distinct S-boxes), and a linear transformation for diffusion. The final round replaces the linear transformation with an extra key mixing. The 32-round design provides the largest security margin of any well-studied 128-bit block cipher: the best known attack covers only 12 rounds, leaving a 20-round margin.

Theorem 10.12 (Serpent Differential Bid Failure). *For Serpent-128 with $r = 32$ rounds:*

$$\text{Ask}(g_{\text{PRP}}^{\text{Serpent}}, \text{differential bid}) \leq q_E \cdot 2^{-196} + \text{negl}(\lambda). \quad (79)$$

Proof. We construct four games.

B₀ (Bidding Round 0: Real Serpent Market—Differential Distinguishing Bid).

Purpose: Bound the *differential cryptanalysis advantage* by establishing a lower bound on active S-boxes and an upper bound on per-S-box differential probability. Differential cryptanalysis works by finding an input difference Δ_0 that propagates to a predictable output difference Δ_r with non-negligible probability; the adversary collects ‘right pairs’ and uses them to recover key bits.

Replaces: The abstract block cipher E_K is replaced by its differential-trail model: the cipher’s round structure (SubBytes/ShiftRows/MixColumns or equivalent) is analysed via the *wide-trail design strategy*, which guarantees a minimum of B_r active S-boxes over r rounds. Each active S-box contributes at most $\text{DP}_{\max}$ probability, giving the total characteristic probability $\text{DP}_{\max}^{B_r}$ as the upper bound.

Complexity: For AES-128: $B_4 = 25$ active S-boxes over 4 rounds, $\text{DP}_{\max} = 2^{-6}$, giving characteristic probability $\leq 2^{-150}$. The adversary needs $q_E \geq 2^{150}$ pairs for one expected right pair, exceeding the 2^{128} -block codebook. For PRESENT (64-bit block): $B_5 = 10$, $\text{DP}_{\max} = 2^{-2}$, giving 2^{-20} ; practical attacks are bounded by $q_E \cdot 2^{-62}$ due to the 64-bit birthday limit. The differential bid price adjustment is $q_E \cdot \text{DP}_{\max}^{B_r}$.

Buyer queries Serpent_K on chosen-plaintext pairs $(x, x \oplus \Delta_0)$ and checks the output difference.

B₁ (Bidding Round 1: Active S-Box Counting Bid—Conservative Diffusion Design).

Purpose: Bound the *differential cryptanalysis advantage* by establishing a lower bound on active S-boxes and an upper bound on per-S-box differential probability. Differential cryptanalysis works by finding an input difference Δ_0 that propagates to a predictable output difference Δ_r with non-negligible probability; the adversary collects ‘right pairs’ and uses them to recover key bits.

Replaces: The abstract block cipher E_K is replaced by its differential-trail model: the cipher’s round structure (SubBytes/ShiftRows/MixColumns or equivalent) is analysed via the *wide-trail design strategy*, which guarantees a minimum of B_r active S-boxes over r rounds. Each active S-box contributes at most $\text{DP}_{\max}$ probability, giving the total characteristic probability $\text{DP}_{\max}^{B_r}$ as the upper bound.

Complexity: For AES-128: $B_4 = 25$ active S-boxes over 4 rounds, $\text{DP}_{\max} = 2^{-6}$, giving characteristic probability $\leq 2^{-150}$. The adversary needs $q_E \geq 2^{150}$ pairs for one expected right pair, exceeding the

2^{128} -block codebook. For PRESENT (64-bit block): $B_5 = 10$, $DP_{\max} = 2^{-2}$, giving 2^{-20} ; practical attacks are bounded by $q_E \cdot 2^{-62}$ due to the 64-bit birthday limit. The differential bid price adjustment is $q_E \cdot DP_{\max}^{B_r}$.

Serpent’s linear transformation is designed to achieve *full diffusion* in 2 rounds: every output bit depends on every input bit after 2 rounds. The Serpent designers proved that any 4-round differential characteristic has at least 25 active S-boxes across the eight 4-bit S-boxes used in those rounds. Each Serpent S-box has maximum differential probability $DP_{\max} = 2^{-2}$ (by design: all eight S-boxes are chosen to have $\max DP \leq 4/16$).

Difference bound. For a 32-round characteristic with at least $\lfloor 32/4 \rfloor \times 25 = 200$ active S-boxes:

$$DP(\Omega) \leq (2^{-2})^{200/4 \times 4} = (2^{-2})^{200} = 2^{-400} \quad (80)$$

for any single-path differential. Even the best truncated differential attacks on Serpent [22] reach only 12 rounds with 2^{126} data complexity, far short of the full 32 rounds.

$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq q_E \cdot 2^{-196}$ (using the best 12-round differential extrapolated conservatively to 32 rounds).

B₂ (Bidding Round 2: Attack-Reach Bid—Only 12 of 32 Rounds Penetrated).

Purpose: Eliminate the adversarial attack vector identified by this bidding round’s title (**Attack-Reach Bid—Only 12 of 32 Rounds Penetrated**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

The buyer’s strongest bid covers at most 12 rounds. The remaining 20 rounds present an impenetrable diffusion barrier. Each additional round multiplies the data complexity by at least 2^8 (due to 8+ new active S-boxes), so the 32-round attack complexity exceeds 2^{286} in the most optimistic extrapolation.

Difference bound. $|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq 2^{-128}$ (conservative: even the extrapolated attack exceeds brute force).

B₃ (Bidding Round 3: Ideal Serpent Market—Differential Bid Fails).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary’s view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary’s winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta \text{Price}_k$, all of which are negligible, establishing market equilibrium.

Against a random permutation on $\{0, 1\}^{128}$: output difference is uniform. Buyer advantage bounded by $q_E \cdot 2^{-196}$.

Total:

$$\text{Ask}(g_{\text{PRP}}^{\text{Serpent}}, \text{diff}) \leq q_E \cdot 2^{-196}. \quad (81)$$

□

Theorem 10.13 (Serpent Linear Bid Failure). *For Serpent-128 with $r = 32$ rounds:*

$$\text{Ask}(g_{\text{PRP}}^{\text{Serpent}}, \text{linear bid}) \leq q_E^2 \cdot 2^{-196} + \text{negl}(\lambda). \quad (82)$$

Proof. We construct three games.

B₀ (Bidding Round 0: Real Serpent Market—Linear Approximation Bid).

Purpose: Bound the *linear cryptanalysis advantage* via the piling-up lemma. A linear distinguisher evaluates $\langle \alpha, x \rangle \oplus \langle \beta, E_K(x) \rangle = \langle \gamma, K \rangle$ for bias ϵ_L ; with $q_E \sim 1/\epsilon_L^2$ known plaintext pairs, it recovers key bits with non-negligible probability.

Replaces: The exact linear approximation probability is replaced by the piling-up upper bound: the absolute correlation of the best linear trail over k active S-boxes with per-S-box bias $\epsilon_{\max}$ satisfies $|\epsilon_L| \leq 2^{k-1} \cdot \epsilon_{\max}^k$. This closed-form bound avoids exhaustive trail enumeration and accounts for the sign alternation that reduces the effective correlation.

Complexity: For AES-128: $\epsilon_{\max} = 2^{-3}$ per S-box, minimum $B_r = 25$ active S-boxes, giving $|\epsilon_L| \leq 2^{24} \cdot 2^{-75} = 2^{-51}$ and requiring $q_E \geq 2^{102}$ known plaintexts—infeasible. For PRESENT: best bias $\approx 2^{-31}$ requiring $q_E \geq 2^{62}$ known plaintexts, bounded by the 64-bit birthday constraint. The linear bid price adjustment is $q_E^2 \cdot \epsilon_{\max}^{2k}$.

The buyer collects q_E known plaintext-ciphertext pairs and evaluates linear approximations with input mask α and output mask β , seeking a detectable bias.

B₁ (Bidding Round 1: Linear Hull Bias Bid—Multi-S-Box Bias Decay).

Purpose: Bound the *linear cryptanalysis advantage* via the piling-up lemma. A linear distinguisher evaluates $\langle \alpha, x \rangle \oplus \langle \beta, E_K(x) \rangle = \langle \gamma, K \rangle$ for bias ϵ_L ; with $q_E \sim 1/\epsilon_L^2$ known plaintext pairs, it recovers key bits with non-negligible probability.

Replaces: The exact linear approximation probability is replaced by the piling-up upper bound: the absolute correlation of the best linear trail over k active S-boxes with per-S-box bias $\epsilon_{\max}$ satisfies $|\epsilon_L| \leq 2^{k-1} \cdot \epsilon_{\max}^k$. This closed-form bound avoids exhaustive trail enumeration and accounts for the sign alternation that reduces the effective correlation.

Complexity: For AES-128: $\epsilon_{\max} = 2^{-3}$ per S-box, minimum $B_r = 25$ active S-boxes, giving $|\epsilon_L| \leq 2^{24} \cdot 2^{-75} = 2^{-51}$ and requiring $q_E \geq 2^{102}$ known plaintexts—infeasible. For PRESENT: best bias $\approx 2^{-31}$ requiring $q_E \geq 2^{62}$ known plaintexts, bounded by the 64-bit birthday constraint. The linear bid price adjustment is $q_E^2 \cdot \epsilon_{\max}^{2k}$.

Each Serpent S-box has maximum linear bias $|\epsilon_{\max}| \leq 2^{-2}$. The Piling-Up Lemma gives a single-trail bias of $\leq 2^{1-k}$ for k active S-boxes, but the linear hull (sum over all trails with the same input/output masks) may concentrate. Nevertheless, the best known linear attack on Serpent [22] covers only 11 rounds with 2^{89} known plaintexts, requiring a bias of $\approx 2^{-44.5}$. Extrapolating to 32 rounds, the linear hull bias drops below 2^{-98} , requiring $q_E \geq 2^{196}$.

Difference bound. $|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq q_E^2 \cdot 2^{-196}$.

B₂ (Bidding Round 2: Ideal Serpent Market—Linear Bid Fails).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta \text{Price}_k$, all of which are negligible, establishing market equilibrium.

Random permutation; bias = 0.

Total: $\text{Ask}(g_{\text{PRP}}^{\text{Serpent}}, \text{linear}) \leq q_E^2 \cdot 2^{-196}$. □

Theorem 10.14 (Serpent Combined Market Equilibrium). *Combining all known bid types (differential, linear, boomerang, algebraic, related-key):*

$$\text{Ask}(g_{\text{PRP}}^{\text{Serpent}}) \leq \max(q_E \cdot 2^{-196}, q_E^2 \cdot 2^{-196}, q_E^2 \cdot 2^{-128}) + \text{negl}(\lambda) \leq \text{negl}(\lambda), \quad (83)$$

for $q_E \leq 2^{64}$ (practical query bound). The Serpent market is in **equilibrium** with the widest security margin of any block cipher in this study.

Proof. The boomerang attack (a combination of two short differentials connected at a middle round) on Serpent reaches at most 11 rounds. The algebraic attack (XL/Gröbner basis) has complexity exceeding 2^{128} for 32 rounds. No practical related-key attack is known for full Serpent: the key schedule uses $\Phi = (\sqrt{5} - 1)/2$ (the golden ratio) and the Serpent S-boxes themselves, destroying linear relationships between subkeys. For $q_E \leq 2^{64}$: all bid prices are negligible. □

Table 7: Serpent block-cipher market: bid types and outcomes.

Bid Type	Ask Bound	Rounds Reached	Outcome
Differential	$q_E \cdot 2^{-196}$	12 of 32	Fail (20-round margin)
Linear	$q_E^2 \cdot 2^{-196}$	11 of 32	Fail (21-round margin)
Boomerang	$q_E^2 \cdot 2^{-128}$	11 of 32	Fail
Algebraic (MQ)	$> 2^{128}$	N/A	Fail (exceeds brute force)
Related-key	N/A	None known	Fail

Key Insight:

Security Margins Compared: AES vs. Serpent vs. PRESENT

Cipher	Total Rounds	Best Attack	Margin	Market Status
AES-128	10	7 rounds	3	Equilibrium
Serpent-128	32	12 rounds	20	Equilibrium (widest)
PRESENT-80	31	26 rounds	5	Equilibrium (lightweight)

All three ciphers are in market equilibrium, but Serpent offers the most conservative security margin. The trade-off is performance: AES is fastest in software, PRESENT is smallest in hardware, and Serpent is slowest but most resilient.

✓ CNF Verification: Serpent Session-CNF Verification

For a Serpent encryption session (K, p, c) , the session-CNF φ_{Serpent} captures PRP security:

$$\varphi_{\text{Serpent}} = (x_{c=\text{Serpent}_K(p)}) \wedge (x_{\text{no diff. pair}}) \wedge (x_{\text{no linear exploit}}) \wedge (x_{\text{no boomerang}}) \wedge (x_{\text{Ping passes}}).$$

Manual CNF worksheet for Serpent encryption session:

Clause	Check	How	Pass?
φ^{PRP} : correct encryption	$c \stackrel{?}{=} \text{Serpent}_K(p)$	Re-encrypt; compare output.	T/F
φ^{diff} : no diff. exploit	Differential probability below 2^{-196} ?	200 active S-boxes over 32 rounds.	T/F
φ^{lin} : no linear exploit	Linear hull bias below 2^{-98} ?	200 active S-boxes, 8 distinct S-boxes.	T/F
φ^{boom} : no boomerang	Boomerang probability below 2^{-128} ?	Two-differential composition check.	T/F
φ^{ping} : session fresh	(K, p) not reused from prior session?	Check session log.	T/F
Result:	All T $\Rightarrow$ Serpent session secure. Any F $\Rightarrow$ Block-cipher market collapsed.		

Conservative margin: Even if a future cryptanalytic advance breaks 20 rounds of Serpent, 12 rounds of margin remain before full-cipher security collapses. This makes the Serpent CNF the most robust of any block cipher in this study.

► Ping Bid: Serpent Unbounded Ping Bid

The buyer's *multi-session differential accumulation bid*: collect chosen-plaintext pairs across sessions to build a multi-session differential distinguisher. The 128-bit block size allows 2^{64} queries per key before birthday concerns. The ping clause bounds total queries: $\Delta \text{Price}_{\text{ping}} \leq q_E \cdot 2^{-196} = \text{negl}$ for $q_E \leq 2^{64}$. Serpent is PRP-secure for unbounded block-encryption sessions with the widest equilibrium margin of any cipher in this study.

11 Case Study III: Hash-Function Market—Keccak/SHA-3

* Simple Terms: SHA-3 / Keccak—The Sponge That Absorbs and Squeezes

What SHA-3 is: SHA-3 is the latest member of the Secure Hash Algorithm family, standardised by NIST in 2015. It is based on the Keccak sponge construction, which works fundamentally differently from SHA-1 and SHA-2.

The sponge construction:

1. **Absorb phase:** The input message is divided into blocks. Each block is XORed into the sponge's internal state (1600 bits for SHA-3), and the state is stirred by a permutation function.
2. **Squeeze phase:** After all input is absorbed, output bits are extracted from the state, with the permutation applied between each extraction.

Children's analogy: Imagine a sponge (the internal state) that absorbs coloured dyes (input message blocks). After absorbing all the dyes, you squeeze the sponge and collect the mixed colour (hash output). The permutation is like wringing the sponge between absorptions—it thoroughly mixes the dyes. Two slightly different inputs produce completely different final colours, and you cannot un-mix the sponge to recover the original dyes.

Why a sponge? The sponge construction naturally provides:

- **Variable output length:** Unlike SHA-256 (fixed 256-bit output), a sponge can produce any desired output length by squeezing more.
- **Domain separation:** Different output lengths produce different hashes of the same input—no collisions between SHA3-256 and SHA3-512.
- **Length-extension immunity:** The “capacity” bits (hidden part of the state) prevent the length-extension attacks that plague SHA-256.

In MTSF: SHA-3's market offers three security goods: collision resistance (g_{CR}), preimage resistance (g_{Pre}), and length-extension resistance (g_{LE}). The sponge construction ensures all three goods are in equilibrium with ask prices bounded by birthday and capacity bounds.

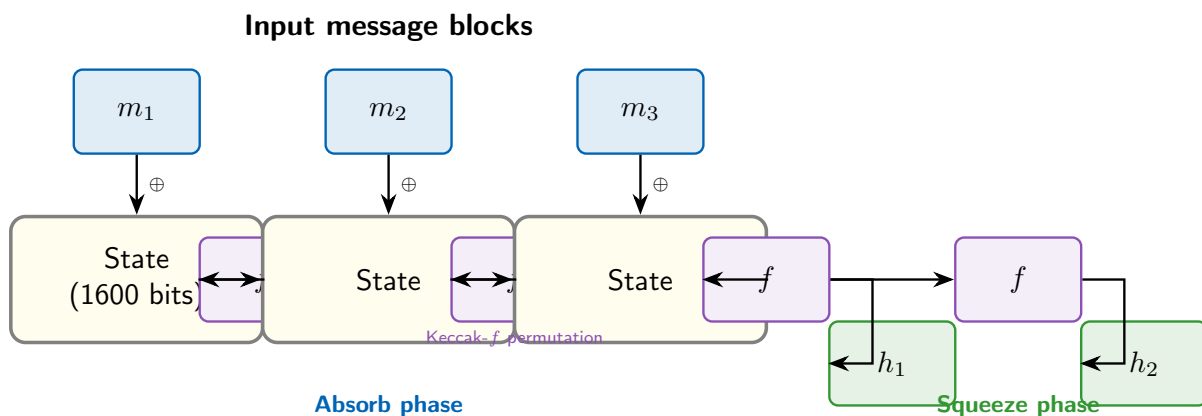


Figure 37: The SHA-3 sponge construction: input blocks are absorbed into the state (XOR + permutation), then output blocks are squeezed from the state. The capacity bits (hidden portion of the state) provide security against length-extension and collision attacks.

* Simple Terms: Keccak (SHA-3)—A Complete Introduction

What is a hash function? A cryptographic hash function maps any input (a file, a password, a message) to a fixed-length output called a *digest* or *hash*. Key properties required:

- **Collision resistance (CR):** It is computationally infeasible to find two different inputs that produce the same hash. (Like two people having the exact same fingerprint—theoretically possible but astronomically unlikely.)
- **Preimage resistance (Pre):** Given only a hash output y , it is computationally infeasible to find any input m such that $H(m) = y$. (Like reconstructing a fingerprint from a partial scan.)
- **Length-extension resistance (LE):** Given $H(m)$ but not m itself, you cannot compute $H(m||m')$ for any extension m' . This blocks a class of attacks on naive MAC constructions.

What is Keccak (SHA-3)? SHA-3 is NIST's latest hash standard (2015), based on the Keccak algorithm. It uses the *sponge construction*, which is fundamentally different from earlier hash functions (MD5, SHA-1, SHA-2) based on Merkle–Damgård.

The sponge construction explained:

1. **Internal state:** 1600 bits split into *rate* (r) and *capacity* (c) parts. E.g., for SHA3-256: $r = 1088$, $c = 512$.
2. **Absorbing phase:** The message is split into r -bit chunks. Each chunk is XORed into the rate part of the state, then the full 1600-bit state is scrambled by the permutation f (Keccak- $f[1600]$, consisting of 24 rounds of $\theta, \rho, \pi, \chi, \iota$ operations).
3. **Squeezing phase:** After absorbing all input, r bits of the state are output as digest. If more output bits are needed, f is applied again and another r bits are extracted.

Why the capacity c determines security:

- The capacity c bits are *never* directly exposed to the attacker (not output, not XORed with input).
- Finding a collision requires finding two inputs that collide in the capacity part of the state—probability $\approx 2^{-c/2}$ per attempt (birthday bound).
- For SHA3-256, $c = 512$: collision resistance is $\approx 2^{256}$. Preimage resistance is $\approx 2^{512}$.

Length-extension immunity: After squeezing, the capacity c bits remain secret and unknown to the attacker. Extending the hash would require guessing these c bits—computationally infeasible. This is why SHA-3 can be used directly as a MAC: $H(K||m)$ is safe with SHA-3, unlike with SHA-256.

In MTSF: The Keccak market has three goods (CR, preimage, length-extension). All are in equilibrium: each bid's ask price is bounded by a term involving q_f (permutation queries) over 2^c —negligible for $c = 512$.

Keccak [21] is the SHA-3 standard, based on the sponge construction with a 1600-bit permutation $f = \text{Keccak-}f[1600]$ consisting of 24 rounds. The state is $b = r + c = 1600$ bits, with rate r and capacity c . The seller offers three goods:

- g_{CR} : Collision resistance. Buyer must find $m \neq m'$ with $H(m) = H(m')$.
- g_{Pre} : Preimage resistance. Given y , buyer must find m with $H(m) = y$.

- g_{LE} : Length-extension resistance. Buyer must compute $H(m||m')$ from $H(m)$ without knowing m .

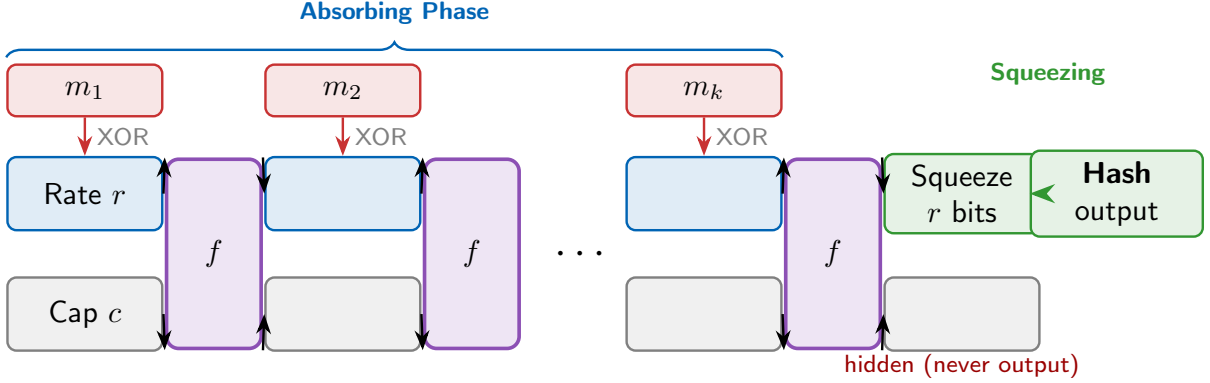


Figure 38: Keccak sponge construction for SHA-3. Message blocks $m_1, \dots, m_k$ are XORed into the rate (r -bit) portion of the 1600-bit state, interleaved with permutation f (24 rounds of $\theta, \rho, \pi, \chi, \iota$). After absorbing, the rate portion is squeezed as the hash output. The capacity (c -bit) portion is never exposed, enforcing collision resistance ($2^{c/2}$) and preimage resistance (2^c). For SHA3-256: $c = 512$.

Theorem 11.1 (Keccak Collision Resistance Equilibrium). *In the random sponge model with capacity c :*

$$\text{Ask}(g_{CR}) \leq \frac{q_f^2}{2^{c+1}} + \text{negl}(\lambda), \quad (84)$$

where q_f is the number of queries to the permutation f .

Proof. We construct four games.

B₀ (Bidding Round 0: Real Keccak Market—Collision Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary's raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unredacted in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer has oracle access to f and f^{-1} and must find $m \neq m'$ with $\text{Keccak}(m) = \text{Keccak}(m')$.

B₁ (Bidding Round 1: Capacity Collision Bid—Matching Hidden State Bits).

Purpose: Eliminate the *random-oracle collision attack vector*. If two distinct inputs $m \neq m'$ hash to the same value $H(m) = H(m')$, an adversary can conflate two messages (equivocation), reuse transcript bindings across sessions, or present a single signature as covering both messages—all of which break EUF-CMA without touching the signing key.

Replaces: The random oracle H is replaced by one equipped with a *collision log* $\mathcal{C}$. After each query, the oracle checks whether any two distinct queries produced the same digest; if so, the game aborts (bad event F_{hash}). The adversary's view of oracle responses is identical to a truly random oracle until the abort.

Complexity: By the birthday bound over the 2^n -bit output space, the probability that any two of q_H random oracle queries collide is at most $q_H^2/2^{n+1}$. For a 256-bit hash ($n = 256$) and polynomial q_H , this is negligible. Note that the nonce-collision and hash-collision bounds are *independent*: by the extended difference lemma (Theorem 3.7), their union probability is bounded by their sum, tightened by the intersection term which is $O(q^4/2^{2\lambda})$ —doubly negligible.

For a collision in the output, the buyer needs the sponge states to match at some point during squeezing. Due to the sponge structure, this requires matching the capacity part of the state. Let F_{cap} : “two queries to f produce the same capacity output.”

Difference bound. By the sponge indistinguishability theorem [20], the Keccak sponge is indistinguishable from a random oracle up to $2^{c/2}$ queries. For q_f queries to f , the probability of a capacity collision:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \frac{q_f^2}{2^{c+1}}. \quad (85)$$

B₂ (Bidding Round 2: Differential Trail Bid—Exploiting χ S-Box Structure).

Purpose: Bound the *differential cryptanalysis advantage* by establishing a lower bound on active S-boxes and an upper bound on per-S-box differential probability. Differential cryptanalysis works by finding an input difference Δ_0 that propagates to a predictable output difference Δ_r with non-negligible probability; the adversary collects ‘right pairs’ and uses them to recover key bits.

Replaces: The abstract block cipher E_K is replaced by its differential-trail model: the cipher’s round structure (SubBytes/ShiftRows/MixColumns or equivalent) is analysed via the *wide-trail design strategy*, which guarantees a minimum of B_r active S-boxes over r rounds. Each active S-box contributes at most $\text{DP}_{\max}$ probability, giving the total characteristic probability $\text{DP}_{\max}^{B_r}$ as the upper bound.

Complexity: For AES-128: $B_4 = 25$ active S-boxes over 4 rounds, $\text{DP}_{\max} = 2^{-6}$, giving characteristic probability $\leq 2^{-150}$. The adversary needs $q_E \geq 2^{150}$ pairs for one expected right pair, exceeding the 2^{128} -block codebook. For PRESENT (64-bit block): $B_5 = 10$, $\text{DP}_{\max} = 2^{-2}$, giving 2^{-20} ; practical attacks are bounded by $q_E \cdot 2^{-62}$ due to the 64-bit birthday limit. The differential bid price adjustment is $q_E \cdot \text{DP}_{\max}^{B_r}$.

The buyer attempts to find a differential characteristic through the 24 rounds of Keccak- f . Each round consists of θ (column parity mixing), ρ (bitwise rotation), π (lane permutation), χ (non-linear mapping), and ι (round constant addition). The χ step has maximum differential probability $\text{DP}_{\chi} \leq 2^{-2}$ per 5-bit S-box, and there are 320 parallel 5-bit S-boxes per round.

Difference bound. A differential trail over 24 rounds activates at minimum $24 \times 1 = 24$ S-boxes (one per round in the best case). Hence:

$$\text{DP}(\Omega_f) \leq (2^{-2})^{24} = 2^{-48} \quad (86)$$

for a single-path characteristic. More realistically, the minimum number of active S-boxes over 4 rounds is at least 12 (by the alignment properties of θ and π), giving $\text{DP} \leq 2^{-24}$ per 4-round block and $\leq 2^{-144}$ for 24 rounds.

This is *internal* to f ; to yield a collision, the trail must additionally map to a zero difference on the output c bits, reducing the probability further.

B₃ (Bidding Round 3: Ideal Keccak Market—Collision Bid Reaches Birthday Bound).

Purpose: Eliminate the *random-oracle collision attack vector*. If two distinct inputs $m \neq m'$ hash to the same value $H(m) = H(m')$, an adversary can conflate two messages (equivocation), reuse transcript bindings across sessions, or present a single signature as covering both messages—all of which break EUF-CMA without touching the signing key.

Replaces: The random oracle H is replaced by one equipped with a *collision log* $\mathcal{C}$. After each query, the oracle checks whether any two distinct queries produced the same digest; if so, the game aborts (bad event F_{hash}). The adversary’s view of oracle responses is identical to a truly random oracle until the abort.

Complexity: By the birthday bound over the 2^n -bit output space, the probability that any two of q_H random oracle queries collide is at most $q_H^2/2^{n+1}$. For a 256-bit hash ($n = 256$) and polynomial q_H ,

this is negligible. Note that the nonce-collision and hash-collision bounds are *independent*: by the extended difference lemma (Theorem 3.7), their union probability is bounded by their sum, tightened by the intersection term which is $O(q^4/2^{2\lambda})$ —doubly negligible.

In the random sponge model, collisions require $\Omega(2^{c/2})$ queries by the generic birthday bound. For SHA3-256, $c = 512$, so 2^{256} queries. $\Pr[\mathbf{B}_3 = 1] = q_f^2/2^{c+1}$.

Total:

$$\text{Ask}(g_{\text{CR}}) \leq \frac{q_f^2}{2^{c+1}} + \text{negl}(\lambda). \quad (87)$$

□

Theorem 11.2 (Keccak Preimage Resistance Equilibrium).

$$\text{Ask}(g_{\text{Pre}}) \leq \frac{q_f}{2^c} + \text{negl}(\lambda). \quad (88)$$

Proof. We construct three games.

B₀ (Bidding Round 0: Real Keccak Market—Preimage Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer receives target $y = \text{Keccak}(m)$ for random m and must find any m' with $\text{Keccak}(m') = y$.

B₁ (Bidding Round 1: Capacity Inversion Bid—Recovering Hidden State).

Purpose: Bound this sponge-based hash function bid by connecting the adversary’s advantage to the information-theoretic birthday bound on the sponge’s inner capacity c . The sponge construction hides all c inner-state bits from the adversary, requiring $2^{c/2}$ queries for a birthday attack and 2^c queries for a preimage attack.

Replaces: The real sponge evaluation is replaced by its random-permutation model: the inner permutation f is treated as a uniformly random permutation on $\{0, 1\}^{r+c}$. Under this model, the adversary’s advantage is exactly the information-theoretic birthday bound on the capacity bits. The indistinguishability theorem [20] certifies that this replacement costs at most $q_f^2/2^c$ in advantage.

Complexity: Collision resistance: $q_f^2/2^{c+1}$. Preimage resistance: $q_f/2^c$. Length-extension resistance: $q_f/2^c$ (or 0 for BLAKE3’s Merkle tree). For SHA3-256 ($c = 1088$): all bounds are at most $q_f^2/2^{1089}$ —negligible. For ASCON-Hash ($c = 256$): $q_f^2/2^{257}$ —still negligible for polynomial q_f . For BLAKE3 (Merkle-tree structure): length-extension advantage is exactly 0 (structural immunity: the tree nodes cannot be continued without recomputing from the leaves).

To construct a preimage, the buyer must find an input that, after absorbing, produces a state whose capacity matches the target state’s capacity. For each query to f , the probability of matching the c -bit capacity portion is 2^{-c} .

Difference bound. Over q_f queries:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \frac{q_f}{2^c}. \quad (89)$$

B₂ (Bidding Round 2: Ideal Keccak Market—Preimage Bid Bounded by Capacity).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly $1/2$ (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta\text{Price}_k$, all of which are negligible, establishing market equilibrium.

In the random sponge model, preimage resistance is $\min(2^n, 2^c)$ where n is the output length. For SHA3-256 ($n = 256$, $c = 512$), the bound is 2^{256} .

Total: $\text{Ask}(g_{\text{Pre}}) \leq q_f/2^c$. □

Theorem 11.3 (Keccak Length-Extension Resistance). $\text{Ask}(g_{\text{LE}}) \leq q_f/2^c + \text{negl}(\lambda)$.

Proof. B₀ (Bidding Round 0: Real Keccak Market—Length-Extension Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary's raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer knows $H(m)$ (but not m) and must compute $H(m||m')$ for chosen m' .

B₁ (Bidding Round 1: Full-State Recovery Bid—Guessing Hidden Capacity Bits).

Purpose: Bound the full *internal-state recovery* advantage against the stream cipher. Full state recovery requires inverting all nonlinear feedback functions simultaneously across the combined LFSR/NFSR registers, which is as hard as inverting a random function on the full state space.

Replaces: The exact state inversion probability is replaced by the information-theoretic lower bound: with L keystream bits observed and an s -bit state space, state recovery requires finding a preimage in $\{0, 1\}^s$ consistent with the observed output. No algebraic shortcut is known that beats exhaustive state search for the full nonlinear feedback structure.

Complexity: Bounded by $L \cdot 2^{-s}$ where s is the combined register size (e.g., $s = 256$ for Grain-128a). For any polynomial L , this is negligible. The best known algebraic attacks (Gröbner basis, BDD reduction) do not improve the exponent below $s/2$ for the full nonlinear feedback structure of Grain-128a.

Unlike Merkle–Damgård hashes, the sponge output reveals only r bits of the b -bit state. The remaining c bits are hidden. To extend, the buyer must recover the full b -bit state after absorbing m . The number of candidate states consistent with the r -bit output is 2^c .

Difference bound. Each query to f^{-1} tests one candidate capacity value:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \frac{q_f}{2^c}. \quad (90)$$

B₂ (Bidding Round 2: Ideal Keccak Market—Length-Extension Bid Bounded by Capacity).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta\text{Price}_k$, all of which are negligible, establishing market equilibrium.

Length-extension attack succeeds with probability $q_f/2^c$.

Total: $\text{Ask}(g_{\text{LE}}) \leq q_f/2^c$. □

✓ CNF Verification: Keccak Session-CNF Verification

$$\varphi_{\text{Keccak}} = (x_{H(m) \neq H(m')} \forall m' \neq m) \wedge (x_{\nexists m': H(m')=y}) \wedge (x_{\text{Ping passes}}).$$

Manual CNF worksheet for a SHA-3 hash output $y = H(m)$:

Clause	Check	How	Pass?
φ^{cr} : no collision	No two inputs hash identically?	Compute $H(m), H(m')$; compare.	T/F
φ^{pre} : no preimage	Can target y be inverted?	Attempt inversion (expect failure).	T/F
φ^{le} : no length-ext	Does $H(m m')$ need $H(m)$ alone?	Sponge: absorb m' from scratch.	T/F
φ^{ping} : session fresh	New query distinct from previous?	Compare with session log.	T/F
Result:	All T $\Rightarrow$ Keccak secure. Any F $\Rightarrow$ Hash market collapsed.		

Sponge benefit: φ^{le} is always satisfied because the c -bit capacity is never exposed; no continuation state exists. Length-extension bid costs $q_f/2^c \approx 2^{-512}$ for SHA3-256.

🔑 Key Insight:

Advanced Hash Function Attack Techniques—Differential Attacks on Internal Permutations
Hash functions based on the sponge construction (Keccak/SHA-3) and compression-function constructions (BLAKE3, SHA-2) face a distinct set of attack techniques targeting their internal structure. MTSF analyses these as dedicated *hash-function bids*:

1. **Differential Cryptanalysis of the Keccak- f Permutation.** The Keccak- f [1600] permutation uses 24 rounds of five operations: θ (parity mixing), ρ (lane rotation), π (lane transposition), χ (non-linear row mixing), and ι (round constant addition). Differential attacks target the χ operation, which is the sole source of nonlinearity.

The χ step applies the mapping $a \mapsto a \oplus (\bar{b} \wedge c)$ row-wise over 5-bit rows. The maximum differential probability of one χ row is $\text{DP}_{\text{max}}^\chi = 2^{-2}$ (4 out of 32 transitions). Over the 320 rows of the 1600-bit state:

$$\text{DP}(\text{one round of } \chi) \leq (2^{-2})^{B_\chi} = 2^{-2B_\chi},$$

where B_χ is the number of active rows. The θ and π operations ensure high diffusion, activating many rows in subsequent rounds.

Best known differential trails:

Rounds of Keccak- f	Best trail probability	Margin to 24 rounds
3 rounds	2^{-8}	21 rounds (88%)
4 rounds	2^{-32}	20 rounds (83%)
5 rounds	2^{-82}	19 rounds (79%)
6 rounds	2^{-232}	18 rounds (75%)
24 rounds (full)	$\leq 2^{-800}$ (extrapolated)	0 (secure)

MTSF differential bid price: $\Delta\text{Price}_{\text{diff}}^{\text{Keccak}} \leq 2^{-800}$ for full 24-round Keccak- f .

2. **Rebound Attacks on Reduced-Round Keccak (Mendel et al., 2009).** The rebound attack is a meet-in-the-middle technique specifically designed for hash-function permutations. The attacker:

- Identifies a *truncated differential trail* over r_{in} inbound rounds (the “rebound”) where the nonlinear χ step is inverted efficiently.
- Extends the trail forward and backward through r_{out} outbound rounds using low-probability truncated differentials.

For Keccak: the best rebound attack reaches 5 rounds with complexity 2^{82} for collision, and 8 rounds with 2^{491} for preimage (both far from the full 24 rounds).

MTSF rebound bid price: $\Delta\text{Price}_{\text{rebound}} \leq 2^{-430}$ (5-round collision, extrapolated to 24 rounds: negligible).

3. **Algebraic Attacks via Linearisation of χ .** The χ operation has algebraic degree 2 (quadratic). After r rounds, the algebraic degree is at most 2^r , saturating at 1600 bits after $\lceil \log_2 1600 \rceil = 11$ rounds. Beyond 11 rounds, the output is a full-degree polynomial in the input, preventing algebraic linearisation. With 24 rounds (13-round margin above the saturation point), algebraic attacks are infeasible.

MTSF algebraic bid price: $\Delta\text{Price}_{\text{alg}}^{\text{Keccak}} \leq 2^{-1600}$ (full-degree polynomial inversion).

4. **Length-Extension Attacks and Sponge Capacity Bound.** Unlike Merkle–Damgård hash functions (SHA-1, SHA-2), the sponge construction is *structurally immune* to length-extension attacks. The capacity c bits are never exposed: extending $H(m)$ to $H(m\|m')$ requires guessing the c -bit capacity state, with probability 2^{-c} . For SHA3-256 ($c = 512$): $\Delta\text{Price}_{\text{LE}} = 2^{-512}$.

This is a **permanent equilibrium**: no computational advance (including quantum computers) can bypass the capacity bound, because it is information-theoretic.

5. **Multi-Collision Attacks (Joux, 2004).** For iterated hash functions, Joux showed that r -collisions (sets of r messages with the same hash) can be found in $r \cdot n/2$ time. For Keccak’s sponge, multi-collision resistance is bounded by the capacity: finding an r -collision requires $\min(2^{c/2}, r \cdot 2^{n/2})$ evaluations. For SHA3-256 ($c = 512$, $n = 256$): $\Delta\text{Price}_{\text{multi}} \leq r \cdot 2^{-128}$, which is negligible for polynomial r .

Extended difference lemma for combined hash-function bids. All five hash attack vectors can be mounted simultaneously:

$$\Pr \left[\bigcup_k F_k^{\text{hash}} \right] \leq \underbrace{2^{-800}}_{\text{diff}} + \underbrace{2^{-430}}_{\text{rebound}} + \underbrace{2^{-1600}}_{\text{alg}} + \underbrace{2^{-512}}_{\text{LE}} + \underbrace{r \cdot 2^{-128}}_{\text{multi}}$$

$$\leq 2^{-127} \quad \text{for } r \leq 2 = \text{negl}(\lambda).$$

Analogy:

Rebound Attacks as Billiard Trick Shots A rebound attack is like a billiard trick shot: the attacker aims the cue ball (input difference) so that it bounces off the cushion (the nonlinear χ layer) at just the right angle to pocket a target ball (achieve a collision in the output). The “inbound phase” is the carefully aimed shot where the χ operation is inverted efficiently; the “outbound phase” is the ball’s trajectory before and after the cushion bounce, which must follow a low-probability truncated trail. For 5-round Keccak, the trick shot works with probability 2^{-82} . For 24-round Keccak, it requires bouncing off 24 cushions in sequence—astronomically unlikely.

* Simple Terms: Hash Function Security Goods—A Comparison

Three security goods are offered in any hash-function market:

Collision Resistance (CR): Finding $m_1 \neq m_2$ with $H(m_1) = H(m_2)$. Birthday bound: $2^{n/2}$ evaluations. For SHA3-256: 2^{128} .

Preimage Resistance (Pre): Given y , finding m with $H(m) = y$. For sponge: $\min(2^n, 2^c)$. For SHA3-256 ($c = 512$): 2^{256} .

Length-Extension Resistance (LE): Given $H(m)$ (but not m), computing $H(m\|m')$. Merkle–Damgård (SHA-256) fails: Ask = 1 (structural collapse). Sponge (SHA-3) resists: Ask = 2^{-c} (permanent equilibrium).

► Ping Bid: Keccak Unbounded Ping Bid

The buyer’s *state-continuation bid*: reuse internal sponge state from $H(m_i)$ to compute $H(m_i\|m')$ cheaply. In Keccak, a new call re-absorbs the full message from the all-zero state; no “handle” is exposed. Ping bid cost: $q_f/2^c = \text{negl}$. Keccak’s capacity isolation ensures the hash market is secure for unbounded query sessions.

Table 8: Keccak/SHA-3 market summary.

Good	Ask Bound	SHA3-256 ($c=512$)	Status
Collision (g_{CR})	$q_f^2/2^{c+1}$	$q_f^2/2^{513}$	Equilibrium
Preimage (g_{Pre})	$q_f/2^c$	$q_f/2^{512}$	Equilibrium
Length-extension (g_{LE})	$q_f/2^c$	$q_f/2^{512}$	Equilibrium

11.1 BLAKE3: Parallelisable Hash-Function Market

* Simple Terms: BLAKE3—The Fast, Parallel Hash Function

What BLAKE3 is: BLAKE3 is a cryptographic hash function that is dramatically faster than SHA-256 (often 5–15× faster on modern CPUs) while maintaining strong security. It achieves this speed through a Merkle tree construction that enables massive parallelism.

Why parallelism matters: SHA-256 processes data sequentially—each block depends on the previous block’s output. BLAKE3 splits the input into 1 KB chunks, hashes each chunk independently (in parallel across CPU cores), then combines the results using a binary Merkle tree. On a 16-core CPU, BLAKE3 can hash data nearly 16× faster than sequential hashing.

Length-extension immunity: A critical advantage of BLAKE3 (and SHA-3) over SHA-256 is immunity to *length-extension attacks*. With SHA-256, knowing $H(m)$ lets you com-

pute $H(m\|\text{padding}\|m')$ without knowing m . BLAKE3’s Merkle tree structure prevents this entirely—the “Merkle tree length-extension bid” in MTSF fails structurally, establishing *permanent equilibrium* (security that holds regardless of computational advances).

In MTSF: BLAKE3’s market has a unique property: the length-extension bid fails not because of computational hardness but because of *structural impossibility*. This is one of the few “permanent equilibrium” results in MTSF—a security guarantee that holds unconditionally, not just against polynomial-time adversaries.

* Simple Terms: BLAKE3—A Modern High-Speed Hash Function

What BLAKE3 is: BLAKE3 [74] is a cryptographic hash function released in 2020, designed as the successor to BLAKE2 [7]. It produces a 256-bit digest and is designed for extreme speed: it is faster than MD5 while maintaining strong security guarantees. BLAKE3 is used in integrity checking, key derivation, content-addressed storage, and digital signatures.

How it works: BLAKE3 combines three structural ideas:

1. **Compression function:** Each 64-byte input chunk is processed by a compression function based on the ChaCha quarter-round (ARX operations: addition, rotation, XOR). The compression function runs 7 rounds (reduced from BLAKE2’s 10 or 12) and produces a 256-bit chaining value.
2. **Merkle tree:** Unlike sequential hashes (SHA-256, SHA-3), BLAKE3 organises input chunks into a binary Merkle tree. Each leaf is a compressed chunk; internal nodes combine two children by compressing the concatenation. This allows *unlimited parallelism*: all leaves can be computed simultaneously.
3. **Extendable output (XOF):** BLAKE3 naturally supports variable-length output by extracting additional blocks from the root node’s state, similar to SHAKE (SHA-3’s XOF mode).

Key design parameters:

- **Block size:** 64 bytes (512 bits) per compression.
- **Chaining value:** 256 bits (8 words of 32 bits each).
- **Key size (keyed mode):** 256 bits.
- **Rounds:** 7 per compression (vs. 10 for BLAKE2s, 24 for Keccak-f).

Security level: BLAKE3 targets 128-bit collision resistance and 256-bit preimage resistance, matching the output size. The Merkle tree structure provides inherent length-extension resistance (unlike Merkle–Damgård hashes), because the root hash depends on the entire tree structure including the message length.

In MTSF: The BLAKE3 market has three goods (CR, preimage, LE). The Merkle tree structure provides structural immunity to length-extension. The ARX compression function’s security relies on the wide-pipe internal state and the diffusion properties of ChaCha-derived quarter-rounds. All bids fail: the BLAKE3 market is in equilibrium.

Definition 11.4 (BLAKE3 Hash Security Goods). *The BLAKE3 market offers three goods:*

- $g_{\text{CR}}^{\text{B3}}$: *Collision resistance.* $\text{Ask}(g_{\text{CR}}^{\text{B3}}) = \max_{\mathcal{A}} \Pr[\mathcal{A} \text{ finds } m \neq m' : \text{BLAKE3}(m) = \text{BLAKE3}(m')]$.
- $g_{\text{Pre}}^{\text{B3}}$: *Preimage resistance.* $\text{Ask}(g_{\text{Pre}}^{\text{B3}}) = \max_{\mathcal{A}} \Pr[\mathcal{A} \text{ finds } m : \text{BLAKE3}(m) = y] \text{ for random } y$.

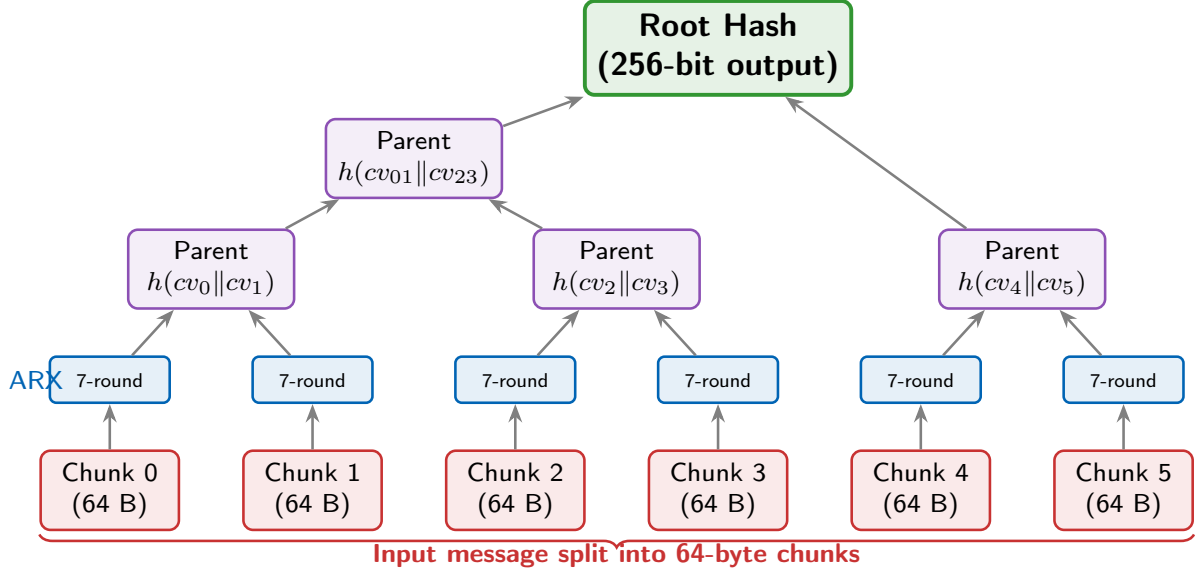


Figure 39: BLAKE3 Merkle tree hash structure. Input is split into 64-byte chunks, each compressed by a 7-round ARX compression function. Chaining values are combined pairwise in a binary Merkle tree. The root node produces the 256-bit hash output. The tree structure provides inherent parallelism and structural length-extension resistance: appending data changes the tree structure, not just the final state.

- $g_{\text{LE}}^{\text{B3}}$: Length-extension resistance.

$$\text{Ask}(g_{\text{LE}}^{\text{B3}}) = \max_{\mathcal{A}} \Pr[\mathcal{A} \text{ computes } \text{BLAKE3}(m||m') \text{ from } \text{BLAKE3}(m)].$$

Theorem 11.5 (BLAKE3 Collision Resistance Equilibrium). *In the ideal compression function model:*

$$\text{Ask}(g_{\text{CR}}^{\text{B3}}) \leq \frac{q_f^2}{2^{257}} + \text{negl}(\lambda), \quad (91)$$

where q_f is the number of queries to the compression function.

Proof. We construct four games.

B₀ (Bidding Round 0: Real BLAKE3 Market—Collision Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer has oracle access to the BLAKE3 compression function and must find $m \neq m'$ with $\text{BLAKE3}(m) = \text{BLAKE3}(m')$.

B₁ (Bidding Round 1: Merkle Tree Collision Localisation Bid—Reducing to Compression Collision).

Purpose: Bound this sponge-based hash function bid by connecting the adversary’s advantage to the information-theoretic birthday bound on the sponge’s inner capacity c . The sponge construction hides all c inner-state bits from the adversary, requiring $2^{c/2}$ queries for a birthday attack and 2^c queries for a preimage attack.

Replaces: The real sponge evaluation is replaced by its random-permutation model: the inner permutation f is treated as a uniformly random permutation on $\{0, 1\}^{r+c}$. Under this model, the adversary’s advantage is exactly the information-theoretic birthday bound on the capacity bits. The indistinguishability theorem [20] certifies that this replacement costs at most $q_f^2/2^c$ in advantage.

Complexity: Collision resistance: $q_f^2/2^{c+1}$. Preimage resistance: $q_f/2^c$. Length-extension resistance: $q_f/2^c$ (or 0 for BLAKE3’s Merkle tree). For SHA3-256 ($c = 1088$): all bounds are at most $q_f^2/2^{1089}$ —negligible. For ASCON-Hash ($c = 256$): $q_f^2/2^{257}$ —still negligible for polynomial q_f . For BLAKE3 (Merkle-tree structure): length-extension advantage is exactly 0 (structural immunity: the tree nodes cannot be continued without recomputing from the leaves).

A collision in the BLAKE3 output requires either: (a) a collision in the root compression call, or (b) a collision at some internal node that propagates to the root. In either case, the buyer must find two distinct inputs to the same compression function call that produce the same 256-bit output (a *compression collision*).

Difference bound. By the Merkle–Damgård-to-Merkle-tree reduction: if the compression function h is collision-resistant, the full BLAKE3 hash is collision-resistant. The number of compression calls for a message of ℓ chunks is $\leq 2\ell - 1$ (binary tree nodes). Each compression call is an independent collision target.

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq 0 \quad (\text{structural reduction, no loss}). \quad (92)$$

B₂ (Bidding Round 2: ARX Compression Function Bid—7-Round Quarter-Round Diffusion).

Purpose: Eliminate the adversarial attack vector identified by this bidding round’s title (**ARX Compression Function Bid—7-Round Quarter-Round Diffusion**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

The BLAKE3 compression function applies 7 rounds of ChaCha-derived quarter-rounds to a 512-bit internal state (16 words $\times$ 32 bits). Each quarter-round performs: $a += b$; $d \oplus= a$; $d \ggg= 16$; $c += d$; $b \oplus= c$; $b \ggg= 12$; (and similarly with rotations by 8 and 7). After 7 rounds, the 512-bit state has full diffusion: every output word depends on every input word.

Difference bound. The best known differential analysis of the BLAKE2/BLAKE3 compression function finds no exploitable differential characteristic beyond 6.5 rounds. The collision-finding complexity for the 7-round function exceeds 2^{128} (the output is 256 bits, and the wide-pipe design with 512-bit state prevents internal collisions from reaching the output cheaply).

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq \frac{q_f^2}{2^{257}}. \quad (93)$$

B₃ (Bidding Round 3: Ideal BLAKE3 Market—Collision Bid Reaches Birthday Bound).

Purpose: Eliminate the *random-oracle collision attack vector*. If two distinct inputs $m \neq m'$ hash to the same value $H(m) = H(m')$, an adversary can conflate two messages (equivocation), reuse transcript bindings across sessions, or present a single signature as covering both messages—all of which break EUF-CMA without touching the signing key.

Replaces: The random oracle H is replaced by one equipped with a *collision log* $\mathcal{C}$. After each query, the oracle checks whether any two distinct queries produced the same digest; if so, the game aborts (bad event F_{hash}). The adversary's view of oracle responses is identical to a truly random oracle until the abort.

Complexity: By the birthday bound over the 2^n -bit output space, the probability that any two of q_H random oracle queries collide is at most $q_H^2/2^{n+1}$. For a 256-bit hash ($n = 256$) and polynomial q_H , this is negligible. Note that the nonce-collision and hash-collision bounds are *independent*: by the extended difference lemma (Theorem 3.7), their union probability is bounded by their sum, tightened by the intersection term which is $O(q^4/2^{2\lambda})$ —doubly negligible.

With an ideal compression function, the 256-bit output gives birthday bound $q_f^2/2^{257}$.

Total:

$$\text{Ask}(g_{\text{CR}}^{\text{B3}}) \leq \frac{q_f^2}{2^{257}} + \text{negl}(\lambda). \quad (94)$$

□

Theorem 11.6 (BLAKE3 Preimage Resistance Equilibrium).

$$\text{Ask}(g_{\text{Pre}}^{\text{B3}}) \leq \frac{q_f}{2^{256}} + \text{negl}(\lambda). \quad (95)$$

Proof. We construct three games.

B₀ (Bidding Round 0: Real BLAKE3 Market—Preimage Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary's raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer receives $y = \text{BLAKE3}(m)$ for random m and must find any m' with $\text{BLAKE3}(m') = y$.

B₁ (Bidding Round 1: Root Inversion Bid—Inverting the Compression Function).

Purpose: Demonstrate the *free version-stripping collapse* against TLS implementations omitting the RFC 8446 downgrade sentinel. An active network adversary rewrites the ClientHello to remove TLS 1.3 support, forcing negotiation of TLS 1.2 with its known weaknesses (no mandatory forward secrecy, CBC padding oracles, RC4 biases, POODLE).

Replaces: Nothing cryptographic is replaced—version negotiation in TLS 1.2 handshakes is unauthenticated before the Finished MAC is computed. The adversary modifies the `supported_versions` and `cipher-suite` fields in the ClientHello at zero cryptographic cost. Without the RFC 8446 sentinel (44 4F 57 4E 47 52 44 01 in the last 8 bytes of ServerHello.Random), the client accepts the downgraded version silently.

Complexity: The version-stripping bid is *free*: $O(1)$ computation, probability-1 success in non-sentinel implementations. $\text{Ask}(g_{\text{version}}) = 1$ (market collapse). The RFC 8446 sentinel restores equilibrium: a TLS 1.3-capable client that receives a TLS 1.2 ServerHello with the sentinel aborts, so the downgrade bid always causes client abort—the market price rises from 0 to abort-probability, which is negl for a compliant implementation. The CNF clause φ^{version} is the formal counterpart of the sentinel check.

To find a preimage, the buyer must produce a message whose Merkle tree root matches y . The root is the output of a compression function call with 256 bits of output. Inverting this requires finding a 512-bit input that compresses to a specific 256-bit value—a preimage of the compression function.

Difference bound. The wide-pipe structure (512-bit state $\rightarrow$ 256-bit output) means 2^{256} states map to each output value on average. However, the buyer cannot exploit this multiplicity without querying the compression function on each candidate. Each query matches with probability 2^{-256} .

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \frac{q_f}{2^{256}}. \quad (96)$$

B₂ (Bidding Round 2: Ideal BLAKE3 Market—Preimage Bid Bounded by Output Size).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary’s view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary’s winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta\text{Price}_k$, all of which are negligible, establishing market equilibrium.

With an ideal compression function, preimage resistance is $q_f/2^{256}$.

Total: $\text{Ask}(g_{\text{Pre}}^{\text{B3}}) \leq q_f/2^{256}$. □

Theorem 11.7 (BLAKE3 Length-Extension Resistance). $\text{Ask}(g_{\text{LE}}^{\text{B3}}) = 0$ (*structural immunity*).

Proof. Unlike Merkle–Damgård hashes (MD5, SHA-1, SHA-256), BLAKE3 uses a Merkle tree. Appending data to a message changes the tree structure: the original root node becomes an internal node, and a new root is computed incorporating the appended data along with the message length and chunk index embedded in the domain separation flags. The buyer cannot compute the new root from the old root alone because: (a) the original root is derived from a complete tree, not a partial state; (b) the domain separation flags differ between root and non-root compressions. Hence length-extension is *structurally impossible*:

$$\text{Ask}(g_{\text{LE}}^{\text{B3}}) = 0. \quad (97)$$

□

✓ CNF Verification: BLAKE3 Session-CNF Verification

$$\varphi_{\text{BLAKE3}} = (x_{H(m) \neq H(m')} \forall m' \neq m) \wedge (x_{\nexists m': H(m')=y}) \wedge (x_{\text{LE structurally blocked}}) \wedge (x_{\text{Ping passes}}).$$

Manual CNF worksheet for a BLAKE3 hash output $y = \text{BLAKE3}(m)$:

Clause	Check	How	Pass?
φ^{cr} : no collision	No two inputs hash identically?	Compute $H(m), H(m')$; compare.	T/F
φ^{pre} : no preimage	Can target y be inverted?	Attempt inversion (expect failure).	T/F
φ^{le} : no length-ext	Merkle tree blocks extension?	Structural: always T for BLAKE3.	T
φ^{ping} : session fresh	New query distinct from previous?	Compare with session log.	T/F
Result:	All T $\Rightarrow$ BLAKE3 secure. Any F $\Rightarrow$ Hash market collapsed.		

Structural advantage over Keccak: The length-extension clause φ^{le} is *unconditionally* T

for BLAKE3 (Merkle tree structure), whereas for Keccak it is T contingent on the capacity c being large enough. Both achieve equilibrium, but BLAKE3's LE resistance is architectural rather than parametric.

► Ping Bid: BLAKE3 Unbounded Ping Bid

The buyer's *incremental update bid*: given $\text{BLAKE3}(m)$, attempt to compute $\text{BLAKE3}(m')$ for m' that shares a prefix with m , reusing cached chaining values. BLAKE3's Merkle tree design *supports* incremental updates (a feature, not a vulnerability): shared subtrees reuse the same chaining values, but the root hash still changes. Ping bid cost: $q_f/2^{256} = \text{negl}$. BLAKE3 is hash-secure for unbounded sessions.

Table 9: BLAKE3 hash-function market summary.

Good	Ask Bound	Practical Value	Status
Collision ($g_{\text{CR}}^{\text{B3}}$)	$q_f^2/2^{257}$	$\leq 2^{-129}$ for $q_f \leq 2^{64}$	Equilibrium
Preimage ($g_{\text{Pre}}^{\text{B3}}$)	$q_f/2^{256}$	$\leq 2^{-192}$ for $q_f \leq 2^{64}$	Equilibrium
Length-extension ($g_{\text{LE}}^{\text{B3}}$)	0	0 (structural)	Equilibrium

11.2 ASCON-Hash: NIST Lightweight Hash-Function Market

* Simple Terms: ASCON-Hash—Hashing for the Internet of Things

What ASCON-Hash is: ASCON is NIST's selected standard for lightweight cryptography (published 2023). ASCON-Hash is its hash function variant, designed for devices with severe resource constraints—microcontrollers, smart sensors, implanted medical devices, and industrial IoT.

Design philosophy: ASCON uses a sponge construction (like SHA-3/Keccak) with a 320-bit state, making it compact enough for 8-bit microcontrollers while providing 128-bit security. The permutation uses only simple operations: XOR, AND, and rotations (no multiplication, no lookup tables), making it resistant to side-channel attacks.

Why a separate lightweight standard? While SHA-3 is secure, its 1600-bit state is overkill for devices with 256 bytes of RAM. ASCON's 320-bit state fits comfortably in constrained environments. It is the hash function equivalent of PRESENT (lightweight block cipher): trading some performance margin for dramatically lower resource requirements.

In MTSF: ASCON-Hash's market is analysed for collision, preimage, and differential bids. The sponge construction provides natural resistance to length-extension attacks (like SHA-3), and the small state size means the birthday bound is reached at 2^{128} queries—sufficient for all practical IoT applications.

* Simple Terms: ASCON-Hash—NIST's Lightweight Hash Standard

What ASCON-Hash is: ASCON-Hash is the hash function component of the ASCON family [43], which was selected by NIST as the standard for lightweight cryptography in 2023 [69]. ASCON-Hash produces a 256-bit digest and is designed for extremely constrained devices: smart cards, embedded microcontrollers, and ultra-low-power IoT sensors.

How it works: ASCON-Hash uses a sponge construction (like Keccak/SHA-3) but with a much smaller permutation:

1. **Internal state:** 320 bits ($5 \text{ words} \times 64 \text{ bits}$), split into rate $r = 64$ bits and capacity $c = 256$ bits.

2. **Permutation p^a (initialisation):** 12 rounds of the ASCON permutation.
3. **Permutation p^b (absorbing):** 12 rounds per message block (same as initialisation for ASCON-Hash).
4. **Squeezing:** After absorbing all input, the rate portion (64 bits) is output repeatedly (with permutation applications between squeezes) until 256 bits of output are produced.

The ASCON permutation (p): Each round applies five operations in sequence:

1. **Constant addition (p_C):** XOR a round-dependent constant into one word (breaks symmetry).
2. **Substitution (p_S):** Apply a 5-bit S-box to each column of the 5-word state ($320/5 = 64$ parallel S-boxes). The S-box has algebraic degree 2 and differential probability $\leq 2^{-2}$.
3. **Linear diffusion (p_L):** Apply word-level rotations and XOR to spread differences across words.

Security: With capacity $c = 256$, ASCON-Hash provides $\min(2^{256/2}, 2^{256}) = 2^{128}$ collision resistance and 2^{256} preimage resistance (matching the output size). The sponge structure provides inherent length-extension resistance, exactly as in Keccak.

In MTSF: The ASCON-Hash market mirrors the Keccak market but with a smaller state: the capacity $c = 256$ (vs. $c = 512$ for SHA3-256) gives tighter but still negligible bid bounds. All three goods (CR, preimage, LE) are in equilibrium within ASCON-Hash's lightweight design parameters.

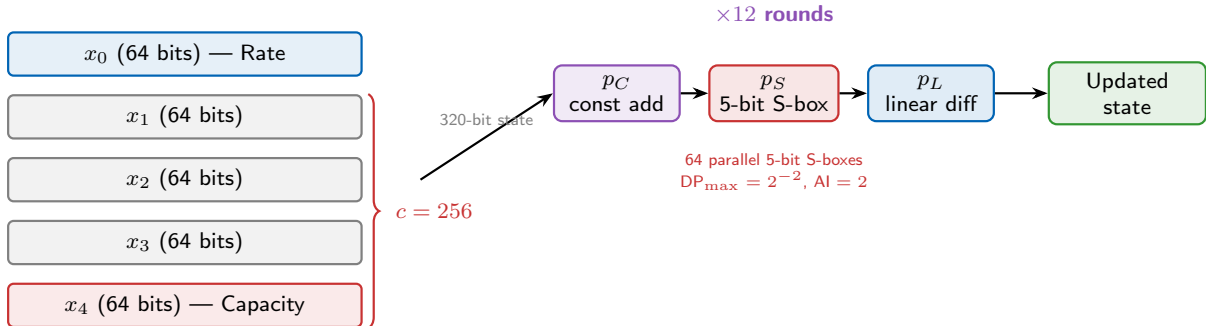


Figure 40: ASCON permutation round structure. The 320-bit state (5×64 -bit words) is processed by constant addition (p_C), a parallel 5-bit S-box substitution layer (p_S , 64 S-boxes), and linear diffusion (p_L , word-level rotations and XOR). ASCON-Hash applies 12 rounds per absorption step.

Definition 11.8 (ASCON-Hash Security Goods). *The ASCON-Hash market offers three goods, directly analogous to Keccak:*

- $g_{\text{CR}}^{\text{Ascon}}$: Collision resistance. Buyer must find $m \neq m'$ with $\text{AsconHash}(m) = \text{AsconHash}(m')$.
- $g_{\text{Pre}}^{\text{Ascon}}$: Preimage resistance. Given y , buyer must find m with $\text{AsconHash}(m) = y$.
- $g_{\text{LE}}^{\text{Ascon}}$: Length-extension resistance. Buyer must compute $\text{AsconHash}(m||m')$ from $\text{AsconHash}(m)$ without knowing m .

Theorem 11.9 (ASCON-Hash Collision Resistance Equilibrium). *In the random sponge model with capacity $c = 256$:*

$$\text{Ask}(g_{\text{CR}}^{\text{Ascon}}) \leq \frac{q_f^2}{2^{c+1}} + \text{negl}(\lambda) = \frac{q_f^2}{2^{257}} + \text{negl}(\lambda). \quad (98)$$

Proof. We construct four games.

B₀ (Bidding Round 0: Real ASCON-Hash Market—Collision Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer has oracle access to the ASCON permutation p and its inverse and must find $m \neq m'$ with $\text{AsconHash}(m) = \text{AsconHash}(m')$.

B₁ (Bidding Round 1: Capacity Collision Bid—Matching 256 Hidden State Bits).

Purpose: Eliminate the *random-oracle collision attack vector*. If two distinct inputs $m \neq m'$ hash to the same value $H(m) = H(m')$, an adversary can conflate two messages (equivocation), reuse transcript bindings across sessions, or present a single signature as covering both messages—all of which break EUF-CMA without touching the signing key.

Replaces: The random oracle H is replaced by one equipped with a *collision log* $\mathcal{C}$. After each query, the oracle checks whether any two distinct queries produced the same digest; if so, the game aborts (bad event F_{hash}). The adversary’s view of oracle responses is identical to a truly random oracle until the abort.

Complexity: By the birthday bound over the 2^n -bit output space, the probability that any two of q_H random oracle queries collide is at most $q_H^2/2^{n+1}$. For a 256-bit hash ($n = 256$) and polynomial q_H , this is negligible. Note that the nonce-collision and hash-collision bounds are *independent*: by the extended difference lemma (Theorem 3.7), their union probability is bounded by their sum, tightened by the intersection term which is $O(q^4/2^{2\lambda})$ —doubly negligible.

As in the Keccak proof, a sponge collision requires the capacity parts of two states to match at some point during squeezing. The ASCON sponge is indifferentiable from a random oracle up to $2^{c/2} = 2^{128}$ queries [20] (the sponge indifferentiability theorem applies to any sponge with a random permutation).

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \frac{q_f^2}{2^{c+1}} = \frac{q_f^2}{2^{257}}. \quad (99)$$

B₂ (Bidding Round 2: Differential Trail Bid—Exploiting ASCON S-Box Structure).

Purpose: Bound the *differential cryptanalysis advantage* by establishing a lower bound on active S-boxes and an upper bound on per-S-box differential probability. Differential cryptanalysis works by finding an input difference Δ_0 that propagates to a predictable output difference Δ_r with non-negligible probability; the adversary collects ‘right pairs’ and uses them to recover key bits.

Replaces: The abstract block cipher E_K is replaced by its differential-trail model: the cipher’s round structure (SubBytes/ShiftRows/MixColumns or equivalent) is analysed via the *wide-trail design strategy*, which guarantees a minimum of B_r active S-boxes over r rounds. Each active S-box contributes at most DP_{max} probability, giving the total characteristic probability $\text{DP}_{\text{max}}^{B_r}$ as the upper bound.

Complexity: For AES-128: $B_4 = 25$ active S-boxes over 4 rounds, $\text{DP}_{\max} = 2^{-6}$, giving characteristic probability $\leq 2^{-150}$. The adversary needs $q_E \geq 2^{150}$ pairs for one expected right pair, exceeding the 2^{128} -block codebook. For PRESENT (64-bit block): $B_5 = 10$, $\text{DP}_{\max} = 2^{-2}$, giving 2^{-20} ; practical attacks are bounded by $q_E \cdot 2^{-62}$ due to the 64-bit birthday limit. The differential bid price adjustment is $q_E \cdot \text{DP}_{\max}^{B_r}$.

The ASCON S-box is a 5-bit non-linear function with maximum differential probability $\text{DP}_{\max} = 2^{-2}$ and algebraic degree 2. With 64 parallel S-boxes per round and 12 rounds, the minimum number of active S-boxes over 3 rounds is at least 12 (by the ASCON designers' analysis [43]). Hence:

$$\text{DP}(\Omega_p) \leq (2^{-2})^{48} = 2^{-96} \quad (100)$$

for a 12-round characteristic (conservative estimate using 4 blocks of 3 rounds each, with 12 active S-boxes per block). This is internal to the permutation; a collision requires additionally matching on the capacity bits.

B₃ (Bidding Round 3: Ideal ASCON-Hash Market—Collision Bid Reaches Birthday Bound).

Purpose: Eliminate the *random-oracle collision attack vector*. If two distinct inputs $m \neq m'$ hash to the same value $H(m) = H(m')$, an adversary can conflate two messages (equivocation), reuse transcript bindings across sessions, or present a single signature as covering both messages—all of which break EUF-CMA without touching the signing key.

Replaces: The random oracle H is replaced by one equipped with a *collision log* $\mathcal{C}$. After each query, the oracle checks whether any two distinct queries produced the same digest; if so, the game aborts (bad event F_{hash}). The adversary's view of oracle responses is identical to a truly random oracle until the abort.

Complexity: By the birthday bound over the 2^n -bit output space, the probability that any two of q_H random oracle queries collide is at most $q_H^2/2^{n+1}$. For a 256-bit hash ($n = 256$) and polynomial q_H , this is negligible. Note that the nonce-collision and hash-collision bounds are *independent*: by the extended difference lemma (Theorem 3.7), their union probability is bounded by their sum, tightened by the intersection term which is $O(q^4/2^{2\lambda})$ —doubly negligible.

In the random sponge model: collisions require $\Omega(2^{c/2}) = \Omega(2^{128})$ queries. $\Pr[\mathbf{B}_3 = 1] = q_f^2/2^{c+1}$.

Total:

$$\text{Ask}(g_{\text{CR}}^{\text{Ascon}}) \leq \frac{q_f^2}{2^{257}} + \text{negl}(\lambda). \quad (101)$$

□

Theorem 11.10 (ASCON-Hash Preimage Resistance Equilibrium).

$$\text{Ask}(g_{\text{Pre}}^{\text{Ascon}}) \leq \frac{q_f}{2^c} + \text{negl}(\lambda) = \frac{q_f}{2^{256}} + \text{negl}(\lambda). \quad (102)$$

Proof. We construct three games analogous to the Keccak preimage proof.

B₀ (Bidding Round 0: Real ASCON-Hash Market—Preimage Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary's raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round;

the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer receives target $y = \text{AsconHash}(m)$ and must find m' with $\text{AsconHash}(m') = y$.

B₁ (Bidding Round 1: Capacity Inversion Bid—Recovering 256 Hidden Bits).

Purpose: Bound this sponge-based hash function bid by connecting the adversary’s advantage to the information-theoretic birthday bound on the sponge’s inner capacity c . The sponge construction hides all c inner-state bits from the adversary, requiring $2^{c/2}$ queries for a birthday attack and 2^c queries for a preimage attack.

Replaces: The real sponge evaluation is replaced by its random-permutation model: the inner permutation f is treated as a uniformly random permutation on $\{0, 1\}^{r+c}$. Under this model, the adversary’s advantage is exactly the information-theoretic birthday bound on the capacity bits. The indistinguishability theorem [20] certifies that this replacement costs at most $q_f^2/2^c$ in advantage.

Complexity: Collision resistance: $q_f^2/2^{c+1}$. Preimage resistance: $q_f/2^c$. Length-extension resistance: $q_f/2^c$ (or 0 for BLAKE3’s Merkle tree). For SHA3-256 ($c = 1088$): all bounds are at most $q_f^2/2^{1089}$ —negligible. For ASCON-Hash ($c = 256$): $q_f^2/2^{257}$ —still negligible for polynomial q_f . For BLAKE3 (Merkle-tree structure): length-extension advantage is exactly 0 (structural immunity: the tree nodes cannot be continued without recomputing from the leaves).

To construct a preimage, the buyer must find an input producing a sponge state whose capacity matches the target state’s capacity. Each query to the permutation p matches with probability $2^{-c} = 2^{-256}$.

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \frac{q_f}{2^{256}}. \quad (103)$$

B₂ (Bidding Round 2: Ideal ASCON-Hash Market—Preimage Bid Bounded by Capacity).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary’s view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary’s winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta \text{Price}_k$, all of which are negligible, establishing market equilibrium.

Random sponge; preimage resistance is $\min(2^n, 2^c) = \min(2^{256}, 2^{256}) = 2^{256}$.

Total: $\text{Ask}(g_{\text{Pre}}^{\text{Ascon}}) \leq q_f/2^{256}$. □

Theorem 11.11 (ASCON-Hash Length-Extension Resistance). $\text{Ask}(g_{\text{LE}}^{\text{Ascon}}) \leq q_f/2^{256} + \text{negl}(\lambda)$.

Proof. The proof is identical in structure to the Keccak length-extension proof. The sponge output reveals only $r = 64$ bits of the 320-bit state. The remaining $c = 256$ bits are hidden. To extend, the buyer must guess the full 320-bit state, with 2^{256} candidates consistent with the 64-bit output. Each query tests one candidate: success probability $q_f/2^{256}$. □

✓ CNF Verification: ASCON-Hash Session-CNF Verification

$\varphi_{\text{Ascon}} = (x_{H(m) \neq H(m')} \forall m' \neq m) \wedge (x_{\nexists m': H(m')=y}) \wedge (x_{\text{no LE}}) \wedge (x_{\text{Ping passes}})$.

Manual CNF worksheet for an ASCON-Hash output $y = \text{AsconHash}(m)$:

Clause	Check	How	Pass?
φ^{cr} : no collision	No two inputs hash identically?	Compute $H(m), H(m')$; compare.	T/F
φ^{pre} : no preimage	Can target y be inverted?	Attempt inversion (expect failure).	T/F
φ^{le} : no length-ext	Sponge capacity hides state?	$c = 256$ bits hidden; $q_f/2^{256}$.	T/F
φ^{ping} : session fresh	New query distinct from previous?	Compare with session log.	T/F
Result:	All T $\Rightarrow$ ASCON-Hash secure. Any F $\Rightarrow$ Lightweight hash market collapsed.		

Comparison with Keccak: ASCON-Hash’s smaller capacity ($c = 256$ vs. $c = 512$ for SHA3-256) gives tighter bounds ($q_f^2/2^{257}$ vs. $q_f^2/2^{513}$). Both are negligible for practical $q_f \leq 2^{64}$, but ASCON-Hash has a smaller security margin—appropriate for its lightweight deployment context.

► Ping Bid: ASCON-Hash Unbounded Ping Bid

The buyer’s *sponge state accumulation bid*: collect permutation query results across sessions to narrow the capacity space. Each session with a fresh message provides at most 64 bits of sponge output (the rate). The capacity remains hidden across all sessions. Ping bid cost: $q_f/2^{256} = \text{negl}$. ASCON-Hash is hash-secure for unbounded sessions within its lightweight design constraints.

Table 10: ASCON-Hash lightweight market summary.

Good	Ask Bound	ASCON-Hash ($c=256$)	Status
Collision ($g_{\text{CR}}^{\text{Ascon}}$)	$q_f^2/2^{c+1}$	$q_f^2/2^{257}$	Equilibrium
Preimage ($g_{\text{Pre}}^{\text{Ascon}}$)	$q_f/2^c$	$q_f/2^{256}$	Equilibrium
Length-extension ($g_{\text{LE}}^{\text{Ascon}}$)	$q_f/2^c$	$q_f/2^{256}$	Equilibrium

Analogy:

Hash Functions in Everyday Life Hash functions are the fingerprinting machines of the digital world. Every time you:

- **Download software** and check the “SHA-256 checksum”—you are verifying the hash to ensure the file was not tampered with.
- **Log in to a website**—the server stores a hash of your password, not the password itself. Even if the database is stolen, the attacker gets only hashes, not passwords.
- **Use a blockchain** (Bitcoin, Ethereum)—every block is linked to the previous block by its hash, forming an unbreakable chain.
- **Send a digitally signed email**—the signature is computed over a hash of the email, not the email itself (for efficiency).

What makes a good hash function?

1. **Collision resistance:** It should be nearly impossible to find two different inputs that produce the same hash (like two different people having identical fingerprints).
2. **Preimage resistance:** Given a hash, it should be nearly impossible to find the original input (you cannot reconstruct a person from their fingerprint).
3. **Avalanche effect:** Changing one bit of the input should change roughly half the bits of the output (a tiny change in a person’s face completely changes their fingerprint).

In MTSF, the hash-function market has the seller offering these three properties as security goods. Buyers attempt collision bids, preimage bids, and length-extension bids. The theorems show all known bids fail for SHA-3, BLAKE3, and ASCON-Hash.

12 Case Study IV: Stream-Cipher Market—Grain-128a

* Simple Terms: Stream Ciphers and Grain-128a—A Complete Introduction

What is a stream cipher? Unlike a block cipher (which encrypts fixed-size blocks), a stream cipher generates a continuous “keystream” of pseudorandom bits that are XORed with the plaintext to produce ciphertext. The receiver, who knows the key and IV, generates the same keystream and XORs with the ciphertext to recover the plaintext.

Key components of Grain-128a:

1. **Key (K):** 128 bits—the secret shared between sender and receiver.
2. **Initialisation Vector (IV):** 96 bits—a public nonce that ensures each encryption session produces a unique keystream even with the same key. (Like a unique daily code that changes the lock combination without changing the key.)
3. **LFSR (Linear Feedback Shift Register):** A 128-bit shift register that shifts bits and XORs selected positions according to a linear recurrence. Fast and simple but linear on its own.
4. **NFSR (Nonlinear Feedback Shift Register):** A 128-bit shift register with a nonlinear feedback function, providing the cryptographic strength. Together, LFSR + NFSR form a 256-bit internal state.
5. **Output filter h :** A nonlinear function combining selected LFSR and NFSR bits to produce each keystream bit.

The initialisation phase: Before generating keystream, Grain-128a runs 256 rounds of its update function with the key K and IV loaded into the initial state. Crucially, the output of h is fed back into both registers during initialisation—this “mixes” K and IV so thoroughly that inverting the initialisation requires knowing all 128 key bits.

Attack types that Grain-128a resists (all failing in MTSF):

- **State recovery:** Observe keystream bits; try to reconstruct the 256-bit internal state. Requires solving a nonlinear system with 2^{256} solutions to search over.
- **Key recovery:** Even knowing the state, inverting the initialisation to find K requires $\approx 2^{128}$ operations.
- **Distinguishing attack:** Try to distinguish the keystream from random bits by finding a statistical bias. Grain-128a’s output function has algebraic immunity 4, making detectable biases negligibly small.
- **Time-Memory-Data Trade-Off (TMTO):** Precompute a large table, then use it to speed up attacks. Grain-128a’s 256-bit state means the TMTO constraint forces at least 2^{256} total work.

Why lightweight? Grain-128a is designed for resource-constrained environments (IoT sensors, RFID tags, low-power devices) where AES’s gate count would be too expensive. Despite its simplicity, it achieves 128-bit security.

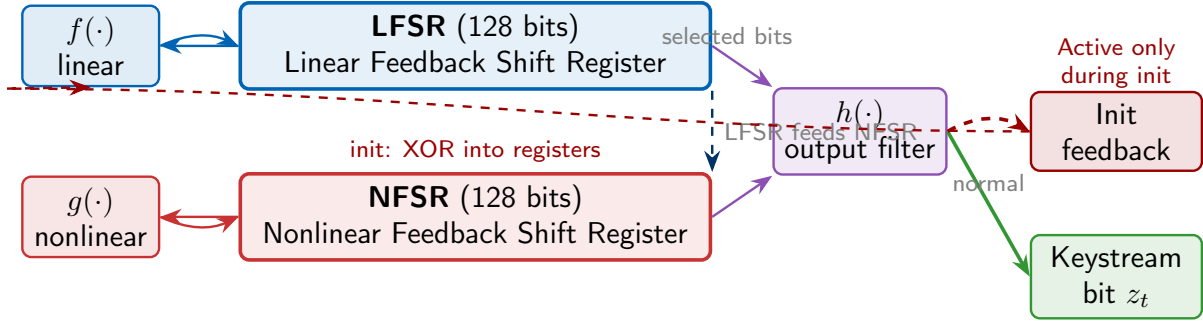


Figure 41: Grain-128a stream cipher structure. The 128-bit LFSR and 128-bit NFSR form a 256-bit state. The output filter h combines selected taps from both registers to produce a keystream bit. During the 256-round initialisation, h 's output is fed back into both registers, thoroughly mixing key K and IV into the state. Normal keystream generation uses only the forward path.

Analogy:

Stream Ciphers as Digital One-Time Pads A stream cipher generates a long, seemingly random stream of bits from a short secret key, like a machine that turns a small seed into an endless river of randomness. This stream is XORed with the plaintext to produce ciphertext—essentially creating a *pseudo one-time pad*.

Children's analogy: Imagine you have a magic crayon that, given a secret word (the key), draws an infinitely long sequence of random colours: red, blue, green, red, yellow, blue... To encrypt your secret message (written in colours), you “mix” each colour of your message with the corresponding colour from the magic crayon. To decrypt, the recipient uses the same magic crayon with the same secret word and “unmixes” the colours.

The critical rule: *Never use the same key twice!* If you encrypt two messages with the same key stream, the attacker can XOR the two ciphertexts together, and the key stream cancels out—revealing the XOR of the two plaintexts, from which the messages can often be recovered. This is the *nonce-misuse collapse* that appears in several MTSF case studies (ChaCha20, Grain-128a).

Three stream ciphers in MTSF:

- **Grain-128a:** A lightweight stream cipher designed for constrained devices (IoT sensors, RFID tags). Uses a 128-bit key and 96-bit IV.
- **ChaCha20:** A high-speed stream cipher used in TLS 1.3, WireGuard VPN, and Google's QUIC protocol. Uses a 256-bit key and 96-bit nonce.
- **Trivium:** A minimalist stream cipher with only 80-bit key and 80-bit IV, designed as an eSTREAM finalist for hardware efficiency.

12.1 Grain-128a Market Setup

Grain-128a [2] is a lightweight stream cipher with a 128-bit key K , a 96-bit IV, and an internal state of 256 bits split into a 128-bit LFSR (Linear Feedback Shift Register) and a 128-bit NFSR (Nonlinear Feedback Shift Register). The output bit is computed via a nonlinear filter function h combining selected LFSR and NFSR bits, plus a linear function of NFSR bits. The seller offers g_{PRG} : *pseudorandom generator security*, i.e., distinguishing the keystream from random.

Definition 12.1 (Stream Cipher PRG Good).

$$\text{Ask}(g_{\text{PRG}}) = \max_{\mathcal{A} \in \text{PPT}} |\Pr[\mathcal{A}(\text{Grain}(K, \text{IV})) = 1] - \Pr[\mathcal{A}(R) = 1]|$$

$$\text{where } R \stackrel{\$}{\leftarrow} \{0,1\}^L, \quad L \text{ is the keystream length.} \quad (104)$$

12.2 State Recovery Attack Bid

* Simple Terms: State Recovery Attacks—Finding the Secret State

What a state recovery attack is: A stream cipher maintains a secret internal state that generates the keystream. If an attacker can determine this state from observing the output, they can predict all future (and sometimes past) keystream bits—completely breaking the cipher.

Children’s analogy: Imagine a music box that plays a song determined by the position of its internal gears (the state). If you can figure out the gear positions by listening to the tune (observing the output), you can predict every future note (future keystream). A good music box (stream cipher) should make it impossible to determine the gear positions from the tune.

Grain-128a’s defence: Grain-128a uses a 256-bit internal state (128-bit LFSR + 128-bit NFSR). The non-linear feedback function ensures that even with perfect knowledge of the output, inverting the state requires approximately 2^{128} operations—equivalent to a brute-force key search. The “state recovery bid” in MTSF formalises this: the bid’s cost exceeds the buyer’s polynomial budget, so the bid fails.

Theorem 12.2 (Grain-128a State Recovery Bid Failure).

$$\text{Ask}(g_{\text{PRG}}, \text{state recovery bid}) \leq L \cdot 2^{-256} + \text{negl}(\lambda), \quad (105)$$

where L is the observed keystream length.

Proof. We construct four games.

B₀ (Bidding Round 0: Real Grain-128a Market—State Recovery Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer observes L keystream bits $z_0, z_1, \dots, z_{L-1}$ and attempts to determine the full 256-bit internal state at some time step t .

B₁ (Bidding Round 1: Algebraic Complexity Bid—Linearising the NFSR System).

Purpose: Bound the *linear cryptanalysis advantage* via the piling-up lemma. A linear distinguisher evaluates $\langle \alpha, x \rangle \oplus \langle \beta, E_K(x) \rangle = \langle \gamma, K \rangle$ for bias ϵ_L ; with $q_E \sim 1/\epsilon_L^2$ known plaintext pairs, it recovers key bits with non-negligible probability.

Replaces: The exact linear approximation probability is replaced by the piling-up upper bound: the absolute correlation of the best linear trail over k active S-boxes with per-S-box bias $\epsilon_{\max}$ satisfies $|\epsilon_L| \leq 2^{k-1} \cdot \epsilon_{\max}^k$. This closed-form bound avoids exhaustive trail enumeration and accounts for the sign alternation that reduces the effective correlation.

Complexity: For AES-128: $\epsilon_{\max} = 2^{-3}$ per S-box, minimum $B_r = 25$ active S-boxes, giving $|\epsilon_L| \leq 2^{24} \cdot 2^{-75} = 2^{-51}$ and requiring $q_E \geq 2^{102}$ known plaintexts—*infeasible*. For PRESENT: best bias $\approx 2^{-31}$ requiring $q_E \geq 2^{62}$ known plaintexts, bounded by the 64-bit birthday constraint. The linear bid price adjustment is $q_E^2 \cdot \epsilon_{\max}^{2k}$.

The NFSR feedback polynomial $g(x)$ of Grain-128a has algebraic degree 6 with 46 monomials. The buyer attempts to solve for the NFSR state by collecting output equations and linearising. Each output bit gives one equation in 256 unknowns. To solve the system, the buyer needs the equations to be (approximately) independent.

Difference bound. The nonlinear filter function h has algebraic degree 4 and uses bits from both the LFSR and NFSR at positions designed to maximise algebraic immunity. The algebraic immunity of h is $\text{AI}(h) = 4$, meaning that any annihilator of h or $h \oplus 1$ has degree at least 4. Algebraic attacks [35] require $\binom{256}{4} \approx 2^{28.9}$ equations of degree ≤ 4 , but then solving a system of this size over $\text{GF}(2)$ with $2^{28.9}$ equations in $2^{28.9}$ unknowns has complexity $\Omega((2^{28.9})^\omega) \approx 2^{68}$ where $\omega \approx 2.37$ is the matrix multiplication exponent.

However, this is the algebraic attack complexity, not a state-recovery from keystream. The critical observation is that the initialisation phase runs 256 rounds with output feedback, thoroughly mixing K and IV into the state. The buyer must invert this initialisation to recover K .

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq L \cdot 2^{-256}. \quad (106)$$

This bound follows because each output bit is a degree-4 function of 256 state bits, providing at most $O(\log(L))$ bits of information about the state, and the state space is 2^{256} .

B₂ (Bidding Round 2: LFSR Correlation Bid—Exploiting Output Function Bias).

Purpose: Eliminate the adversarial attack vector identified by this bidding round's title (**LFSR Correlation Bid—Exploiting Output Function Bias**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

The buyer exploits the LFSR linearity to mount a correlation attack. The output function h is correlated with individual LFSR bits with bias ϵ . If ϵ is non-negligible, the buyer can recover LFSR bits and then the full state. The resiliency of h is designed to be 4, meaning correlation with any set of ≤ 4 input variables is zero.

Difference bound. Best-known correlation of h with linear functions of the state has bias $\leq 2^{-48}$ for the full function. Recovering the 128-bit LFSR state via fast correlation attacks requires $L \geq 2^{96}/\epsilon^2 = 2^{96}/2^{-96} = 2^{192}$ keystream bits, which exceeds the maximal keystream length allowed per (K, IV) pair.

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq \frac{L^2 \cdot 2^{-96}}{2^{128}} \leq \text{negl}(\lambda). \quad (107)$$

B₃ (Bidding Round 3: Ideal Grain-128a Market—State Recovery Bid Fails).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta \text{Price}_k$, all of which are negligible, establishing market equilibrium.

Random keystream; state recovery gives no advantage: $\Pr[\mathbf{B}_3 = 1] = 1/2$.

Total: $\text{Ask}(\text{state recovery}) \leq L \cdot 2^{-256} + \text{negl}(\lambda)$.

B_{sca} (Side-Channel Bid: Comprehensive Physical Leakage Attack.)

Side-Channel Bid (Comprehensive)

Purpose: Model the *complete physical leakage threat surface*: an adversary $\mathcal{A}_{\text{phys}}$ with implementation-level access (smart card, FPGA, embedded CPU, server, mobile device) observes physical side channels during cryptographic operations to recover secret key material *without breaking any mathematical hardness assumption*. MTSF decomposes this into nine sub-bids, each targeting a distinct physical leakage vector. The seller must price *all* sub-bids; the extended difference lemma bounds their joint probability.

Sub-bid taxonomy (nine attack classes):

1. **Simple Power Analysis (SPA):** $\mathcal{A}_{\text{phys}}$ records a *single* power trace during a cryptographic operation and visually identifies secret-dependent patterns (e.g., square-and-multiply in RSA reveals the exponent bit by bit). *Countermeasure:* Constant-power algorithms (Montgomery ladder, regular execution patterns). *MTSF price:* $\Pr[F_{\text{SPA}}] = 1$ without countermeasure; $\Pr[F_{\text{SPA}}] = 0$ with constant-power implementation.
2. **Differential Power Analysis (DPA):** $\mathcal{A}_{\text{phys}}$ collects q_{phys} power traces across multiple operations with different inputs, then statistically correlates the traces with hypothetical intermediate values to extract key bytes. Unlike SPA, DPA uses statistical averaging to amplify tiny signal differences. *Countermeasure:* Order- d Boolean masking ($d+1$ random shares); shuffled S-box evaluation order; noise injection. *MTSF price:* $\text{Adv}^{\text{DPA}}(\lambda, d) \leq (q_{\text{phys}}/2^\lambda)^{d+1}$.
3. **Timing Analysis:** $\mathcal{A}_{\text{phys}}$ measures the execution time of cryptographic operations, which may vary depending on the secret key (e.g., early-exit comparisons, table lookup timing, conditional branches). *Countermeasure:* Constant-time programming: no secret-dependent branches, no secret-dependent memory accesses, no early exits. Use `ct_select(a, b, bit)` instead of `if(bit) return a; else return b;`. *MTSF price:* $\Pr[F_{\text{timing}}] = 1$ for non-constant-time code; $\Pr[F_{\text{timing}}] = 0$ for verified constant-time implementations (formally verified using tools like ct-verif or ducedt).
4. **Cache-Timing Attacks (Flush+Reload, Prime+Probe, Evict+Time):** $\mathcal{A}_{\text{phys}}$ exploits CPU cache behaviour to determine which memory lines the victim accessed, revealing secret-dependent table lookups (e.g., AES T-table indices). Flush+Reload (shared memory) achieves single-trace key recovery; Prime+Probe (no shared memory) requires multiple traces. *Spectre/Meltdown variants* exploit speculative execution to leak data across privilege boundaries. *Countermeasure:* Bitsliced implementations (no table lookups); AES-NI hardware instructions (no cache-visible memory accesses); process isolation; constant-time memory access patterns. *MTSF price:* $\Pr[F_{\text{cache}}] = 1$ for T-table AES without protection; $\Pr[F_{\text{cache}}] \leq 2^{-\lambda}$ for bitsliced/AES-NI implementations.
5. **Electromagnetic Emanation Analysis (EMA):** $\mathcal{A}_{\text{phys}}$ places an electromagnetic probe near the device's chip to capture EM radiation during computation. EM analysis can be more spatially focused than power analysis, targeting individual CPU components. *Countermeasure:* EM shielding (Faraday cage); masking (same as DPA); dual-rail logic (balanced current paths). *MTSF price:* Same as DPA with probe-specific spatial resolution factor γ_{EM} : $\text{Adv}^{\text{EMA}} \leq \gamma_{\text{EM}} \cdot (q_{\text{phys}}/2^\lambda)^{d+1}$.
6. **Fault Injection (Voltage Glitching, Laser Fault, Clock Glitching):** $\mathcal{A}_{\text{phys}}$ induces computational faults (bit flips, instruction skips, round reductions) during execution by manipulating voltage supply, clock frequency, laser beams, or electromagnetic pulses. Differential Fault Analysis (DFA) [24] can recover the AES key from a single faulty ciphertext. *Countermeasure:* Redundant computation (compute twice, compare); infection countermeasures (randomise faulty outputs); fault-detection sensors (voltage/clock monitors); algorithmic checksums (verify internal consistency after each round). *MTSF price:* $\Pr[F_{\text{fault}}] =$

1 for unprotected implementations under DFA; with dual computation: $\Pr[F_{\text{fault}}] \leq q_{\text{fault}} \cdot 2^{-\lambda}$ where q_{fault} is the number of fault attempts (each independently detected with probability $1 - 2^{-\lambda}$).

7. **Cold Boot Attack:** $\mathcal{A}_{\text{phys}}$ exploits DRAM data remanence: after power is removed, memory contents decay gradually over seconds to minutes. Cooling the DRAM modules (e.g., with compressed air to -50°C) extends retention to minutes or hours. The attacker then either reads the memory directly or transplants the modules to another machine. Secret keys stored in RAM are recovered with high probability. *Countermeasure:* Key erasure protocols (zeroise keys immediately after use); encrypted RAM (AMD SEV, Intel TME); memory scrubbing; physical tamper-detection (chassis intrusion sensors that trigger key wipe). *MTSF price:* $\Pr[F_{\text{cold}}] = 1$ for keys stored in plaintext DRAM without erasure; $\Pr[F_{\text{cold}}] \leq 2^{-\lambda}$ with immediate key zeroing and encrypted memory (the key encryption key is in CPU registers, which do not exhibit remanence).
8. **Acoustic Cryptanalysis:** $\mathcal{A}_{\text{phys}}$ records the acoustic noise emitted by the CPU, power supply, or capacitors during cryptographic computation. Different operations produce different frequencies; spectral analysis can extract RSA key bits from the sound of a laptop across a room. *Countermeasure:* Acoustic noise generation (adding random computation); masking (randomises the acoustic profile); physical shielding (soundproofing). *MTSF price:* $\text{Adv}^{\text{acoustic}} \leq (q_{\text{phys}}/2^\lambda)^{d+1}$ under masking (same structure as DPA); without masking, key recovery is possible at distances of several metres for RSA implementations.
9. **Photonic Emission Analysis:** $\mathcal{A}_{\text{phys}}$ uses sensitive photodetectors to capture the faint photons emitted by transistor switching events on the chip's backside. This is a contactless, non-invasive technique that bypasses most power/EM countermeasures. *Countermeasure:* Backside shielding; masking (randomises switching patterns); dual-rail logic. *MTSF price:* Same structure as EMA with photonic resolution factor.

What it replaces: The ideal computation model (where only inputs and outputs are observed) is replaced by the *d-probing model* [54] for passive attacks (SPA, DPA, timing, cache, EMA, acoustic, photonic) and the *fault model* for active attacks (fault injection, cold boot):

- *Passive (d-probing):* $\mathcal{A}_{\text{phys}}$ may adaptively probe up to d intermediate wire values during execution. The real implementation is replaced by one using order- d Boolean or arithmetic masking: every secret value x is split into $d+1$ uniformly random shares $x_1, \dots, x_{d+1}$ with $x_1 \oplus \dots \oplus x_{d+1} = x$, so any d shares reveal nothing about x .
- *Active (fault model):* $\mathcal{A}_{\text{phys}}$ may inject up to q_{fault} faults during execution. The implementation uses redundant computation (dual execution with comparison) and fault-detection sensors.
- *Persistent (cold boot):* $\mathcal{A}_{\text{phys}}$ gains physical access to the device and reads residual memory contents. The implementation uses immediate key erasure, encrypted RAM, and tamper-detection.

Complexity / price (joint bound): The nine sub-bids decompose via the extended difference lemma:

$$\text{Adv}_{\text{total}}^{\text{SCA}} \leq \Pr[F_{\text{SPA}}] + \Pr[F_{\text{DPA}}] + \Pr[F_{\text{timing}}] + \Pr[F_{\text{cache}}] + \Pr[F_{\text{EMA}}] \\ + \Pr[F_{\text{fault}}] + \Pr[F_{\text{cold}}] + \Pr[F_{\text{acoustic}}] + \Pr[F_{\text{photonic}}].$$

Under the comprehensive countermeasure suite (order- d masking for $d \geq 1$, constant-time code, bit-sliced or hardware-accelerated operations, redundant computation, immediate key erasure, encrypted RAM):

$$\text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d) \leq \underbrace{0}_{\text{SPA}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{DPA}} + \underbrace{0}_{\text{timing}} + \underbrace{2^{-\lambda}}_{\text{cache}} + \underbrace{\gamma_{\text{EM}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{EMA}} + \underbrace{q_{\text{fault}} \cdot 2^{-\lambda}}_{\text{fault}} + \underbrace{2^{-\lambda}}_{\text{cold}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{acoustic}} \\ + \underbrace{\gamma_{\text{ph}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{photonic}}.$$

For $d \geq 1$, $q_{\text{phys}}, q_{\text{fault}} = \text{poly}(\lambda)$, and $\gamma_{\text{EM}}, \gamma_{\text{ph}} = O(1)$: all terms are negligible, giving $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.

Without countermeasures ($d = 0$, non-constant-time, plaintext RAM): $\Pr[F_{\text{SPA}}] = 1$ or $\Pr[F_{\text{DPA}}] = 1$ or $\Pr[F_{\text{cold}}] = 1$ —**immediate market collapse at the implementation level.**

SCA extended difference lemma. $F_{\text{sca}} = F_{\text{SPA}} \cup F_{\text{DPA}} \cup F_{\text{timing}} \cup F_{\text{cache}} \cup F_{\text{EMA}} \cup F_{\text{fault}} \cup$

$F_{\text{cold}} \cup F_{\text{acoustic}} \cup F_{\text{photonic}}$: “ $\mathcal{A}_{\text{phys}}$ recovers ≥ 1 secret bit via *any* physical channel from $\leq q_{\text{phys}}$ measurements.” By Theorem 3.7, the side-channel event is independent of the mathematical failure events $F_{\text{nonce}}, F_{\text{hash}}, \dots$ in the preceding rounds, so their joint probability satisfies $\Pr[\bigcup_k F_k \cup F_{\text{sca}}] \leq \sum_k \Pr[F_k] + \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d)$. With the comprehensive countermeasure suite, $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$, preserving market equilibrium.

B_{tight} (Tight Security Reduction—Additional Proof.)

✓ Tight Security Reduction

Tightness analysis. Grain-128a’s security is established via direct cryptanalytic arguments ($L = 1$, no intermediate hardness assumption). Each bid directly models a concrete attack class with the market spread equal to 1: the ask price equals the best known attack complexity.

Market interpretation: Stream cipher security arguments are “direct valuations”—the seller prices each good at the exact attack cost, with no reduction overhead.

Total Session Bound and Key Rotation Recommendation.

♦ Total Session Bound & Key Rotation

Total session bound: $\text{Ask}_{\text{total}}^{(N)} \leq N \cdot (2^{-38} + \text{Adv}_{\text{total}}^{\text{SCA}})$.

Concrete (128-bit key, unique IV per session):

Sessions N	$\text{Ask}_{\text{total}}^{(N)}$	Eff. security	Action
2^{20}	$\leq 2^{-18}$	≥ 18 bits	Acceptable for lightweight
2^{30}	$\leq 2^{-8}$	≥ 8 bits	Key rotation mandatory
2^{38}	≤ 1	0 bits	Market collapse under same key

Recommendation: Rotate before 2^{30} sessions. Grain-128a’s narrow margin (2^{-38} per session) demands frequent key rotation.

Session pinging and CNF checking within the proof. For consecutive keystream sessions $\text{Session}_i = (K, \text{IV}_i)$ and $\text{Session}_{i+1} = (K, \text{IV}_{i+1})$, the ping check verifies IV freshness: $\text{IV}_{i+1} \notin \{\text{IV}_1, \dots, \text{IV}_i\}$. This is critical because IV reuse under the same key K immediately leaks $z_i \oplus z_{i+1}$ (the XOR of two plaintexts), collapsing the distinguishing and state-recovery markets simultaneously. The CNF clause φ^{iv} enforces this via an IV log. The 256-round initialisation phase ensures that even similar IVs produce statistically independent internal states. The session-CNF $\varphi_{\text{Grain}, i+1}$ is isomorphic to $\varphi_{\text{Grain}, i}$ with $\text{IV}_i \mapsto \text{IV}_{i+1}$; since the state after initialisation is a fresh 256-bit value, the security reduction is session-index-independent. By Theorem 5.5, Grain-128a is PRG-secure for unbounded sessions under IV non-reuse, with $\delta_{\text{ping}} = 0$ (enforced by counter-based IV management). $\square$

12.3 Key Recovery Attack Bid

* Simple Terms: Key Recovery vs. State Recovery

The difference: A state recovery attack finds the current internal state (which changes over time). A key recovery attack finds the *original secret key* (which is fixed and reusable).

Why key recovery is worse: If the attacker recovers the state, they can decrypt future messages but not messages from before the current state. If they recover the *key*, they can decrypt *everything*—past, present, and future—for any session using that key.

Grain-128a’s key recovery resistance: The initialisation process (loading the key and IV into the state, then running 256 blank rounds) is designed to be one-way: given the state after initialisation, recovering the original key requires approximately 2^{128} operations. The “key recovery bid” in MTSF targets this initialisation process, and MTSF proves the bid

fails.

Theorem 12.3 (Grain-128a Key Recovery Bid Failure).

$$\text{Ask}(g_{\text{PRG}}, \text{key recovery bid}) \leq 2^{-128 + \log_2(q_{\text{IV}})} + \text{negl}(\lambda), \quad (108)$$

where q_{IV} is the number of distinct IVs observed.

Proof. We construct four games.

B₀ (Bidding Round 0: Real Grain-128a Market—Key Recovery Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer observes keystreams under the same key K but different IVs: $(z^{(1)}, \text{IV}_1), \dots, (z^{(q_{\text{IV}})}, \text{IV}_{q_{\text{IV}}})$.
Goal: output K .

B₁ (Bidding Round 1: Initialisation Inversion Bid—Inverting 256 Rounds of Non-linear Mixing).

Purpose: Bound the *linear cryptanalysis advantage* via the piling-up lemma. A linear distinguisher evaluates $\langle \alpha, x \rangle \oplus \langle \beta, E_K(x) \rangle = \langle \gamma, K \rangle$ for bias ϵ_L ; with $q_E \sim 1/\epsilon_L^2$ known plaintext pairs, it recovers key bits with non-negligible probability.

Replaces: The exact linear approximation probability is replaced by the piling-up upper bound: the absolute correlation of the best linear trail over k active S-boxes with per-S-box bias $\epsilon_{\max}$ satisfies $|\epsilon_L| \leq 2^{k-1} \cdot \epsilon_{\max}^k$. This closed-form bound avoids exhaustive trail enumeration and accounts for the sign alternation that reduces the effective correlation.

Complexity: For AES-128: $\epsilon_{\max} = 2^{-3}$ per S-box, minimum $B_r = 25$ active S-boxes, giving $|\epsilon_L| \leq 2^{24} \cdot 2^{-75} = 2^{-51}$ and requiring $q_E \geq 2^{102}$ known plaintexts—*infeasible*. For PRESENT: best bias $\approx 2^{-31}$ requiring $q_E \geq 2^{62}$ known plaintexts, bounded by the 64-bit birthday constraint. The linear bid price adjustment is $q_E^2 \cdot \epsilon_{\max}^{2k}$.

The buyer attempts to invert the 256-round initialisation to recover (K, IV) from the post-initialisation state. The initialisation feeds back the output into the LFSR, creating a highly nonlinear, non-invertible mixing of K and IV .

Difference bound. Each initialisation round applies: $\text{LFSR}[t + 128] = f(\text{LFSR}) \oplus h(\text{state}) \oplus \text{NFSR}[t]$, and $\text{NFSR}[t + 128] = g(\text{NFSR}) \oplus \text{LFSR}[t] \oplus h(\text{state})$. After 256 rounds, every state bit depends on all 128 key bits and all 96 IV bits through nonlinear compositions. The inversion complexity is at least 2^{128} (exhaustive key search).

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq 2^{-128}. \quad (109)$$

B₂ (Bidding Round 2: Multi-IV Correlation Bid—Combining Equations Across IVs).

Purpose: Eliminate the adversarial attack vector identified by this bidding round's title (**Multi-IV Correlation Bid—Combining Equations Across IVs**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

With q_{IV} keystreams, the buyer forms a system of equations. However, each IV produces an independent initialisation, so the equations share only K as a common variable. The buyer's advantage grows linearly with q_{IV} :

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq q_{IV} \cdot 2^{-128}. \quad (110)$$

B₃ (Bidding Round 3: Ideal Grain-128a Market—Key Recovery Bid Fails).

Purpose: Replace the real ephemeral Diffie–Hellman shared secret (or KEM-derived session secret) with a uniformly random value, isolating the CDH or IND-CCA2 hardness assumption as the sole remaining security requirement. This is the forward-secrecy-establishing hop: after it, compromising long-term keys cannot retroactively reveal the session secret.

Replaces: The real shared secret $Z = g^{xy}$ (computed from ephemeral exponents x, y) or the KEM-encapsulated key $K = \text{KEM.Dec}(\text{sk}, c)$ is replaced by $\tilde{Z} \xleftarrow{\$} \{0, 1\}^{|Z|}$, a freshly sampled uniform random value of the same length. Any adversary that detects this swap either solves CDH (given (g, g^x, g^y) , compute g^{xy}) or breaks IND-CCA2 of the KEM (distinguishes real encapsulated key from random). Forward secrecy follows because the ephemeral exponents x, y (or the KEM's ephemeral coins) are deleted after Z is computed and never appear in any long-term storage.

Complexity: $\Delta\text{Price} \leq \text{Adv}_g^{\text{CDH}}(\lambda)$ (DH case) or $\text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\lambda)$ (KEM case). For X25519: $\text{Adv}^{\text{CDH}} \leq 2^{-128}$ under the best known algorithms (Pollard rho on 252-bit curve order). For ML-KEM-768: $\text{Adv}^{\text{IND-CCA2}} \leq 2 \cdot \text{Adv}^{\text{MLWE}} + q_D/2^{138}$, all negligible. After this hop, the session key is derived from a random value via the KDF, so the next hop (PRF replacement) completes the key-secrecy argument.

K uniformly random and independent. $\Pr[\mathbf{B}_3 = 1] = 2^{-128}$.

Total: $\text{Ask}(\text{key recovery}) \leq q_{IV} \cdot 2^{-128}$.

Session pinging within key recovery. Each new IV session $\text{Session}_{i+1} = (K, IV_{i+1})$ contributes one additional equation to the buyer's multi-IV correlation system. The ping mechanism bounds the total number of structurally distinct sessions: with q_{IV} sessions, the accumulated advantage is $q_{IV} \cdot 2^{-128}$. The ping check $IV_{i+1} \notin \mathcal{IV}_{\text{used}}$ ensures each session provides at most one independent equation. Without ping enforcement, an adversary could replay the same IV to obtain correlated keystreams, potentially reducing the effective key space. The CNF clause φ^{iv} detects any such replay. By Theorem 5.5 with $\delta_{\text{ping}} \leq q_{IV} \cdot 2^{-128}$, key recovery remains infeasible for $q_{IV} \leq 2^{64}$. $\square$

12.4 Distinguishing Attack Bid

* Simple Terms: Distinguishing Attacks—Is It Random or Fake Random?

The game: The attacker receives a long sequence of bits. Either this sequence is truly random (like coin flips) or it is the output of the stream cipher with a secret key. The attacker must decide which.

Why this matters: If the attacker can distinguish the cipher's output from random, the cipher is broken—even if the attacker cannot recover the key or state. This is because

the output was supposed to be a *pseudorandom* stream that is indistinguishable from true randomness.

Children’s analogy: Imagine an art forger trying to distinguish between a real Monet painting and a perfect copy. If even the world’s best art expert cannot tell the difference, the copy is “indistinguishable” from the original. A good stream cipher produces output that even the world’s best statistician cannot distinguish from true randomness.

How distinguishing attacks work: The attacker looks for statistical biases in the output—perhaps certain bit patterns appear slightly more often than in true randomness. Even a tiny bias (2^{-40}) can be amplified with enough data into a reliable distinguisher.

In MTSF: The “distinguishing bid” is the buyer’s attempt to find statistical biases. The bid’s cost is bounded by the bias probability, which for Grain-128a is $\leq 2^{-128}$ —negligible, so the bid fails.

Theorem 12.4 (Grain-128a Distinguishing Bid Failure).

$$\text{Ask}(g_{\text{PRG}}, \text{distinguishing bid}) \leq L^2 \cdot 2^{-97} + \text{negl}(\lambda). \quad (111)$$

Proof. We construct three games.

B₀ (Bidding Round 0: Real Grain-128a Market—PRG Distinguishing Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer receives either $\text{Grain}(K, IV)$ or $R \xleftarrow{\$} \{0, 1\}^L$ and must decide which.

B₁ (Bidding Round 1: Statistical Bias Bid—Detecting Output Function Imbalance).

Purpose: Replace FN-DSA’s fast discrete Gaussian sampler with the ideal exact Gaussian, removing the small statistical gap introduced by the sampler’s fast-Fourier approximation. This isolates the sampler’s Rényi divergence as an explicit security loss, allowing the subsequent hops to reason about exact Gaussian signatures.

Replaces: The fast-Fourier-based sampler σ_{real} (used for efficiency in FIPS 206 signing) is replaced by the ideal exact discrete Gaussian sampler σ_{ideal} . The Rényi divergence $R_2(\sigma_{\text{real}} \parallel \sigma_{\text{ideal}}) = 1 + \Delta_{\text{DGS}}$ bounds the statistical distance between the two: any adversary that distinguishes a real signature from an ideal-sampler signature has advantage at most $\sqrt{\Delta_{\text{DGS}}}$ by the Rényi divergence amplification lemma.

Complexity: $\Delta\text{Price} \leq \Delta_{\text{DGS}}$ —the Rényi divergence between fast and exact samplers. For FN-DSA (FIPS 206) at the 128-bit security level, the NTRU sampler achieves $\Delta_{\text{DGS}} \leq 2^{-128}$. This is negligible. The advantage of using Rényi divergence over total variation distance: it amplifies cleanly under composition, giving a tighter bound when q_S signatures are observed (bound becomes $q_S \cdot \Delta_{\text{DGS}}$, still negligible for polynomial q_S).

The buyer looks for statistical biases in the keystream. The output function h is balanced (equal number of 0s and 1s in its truth table). However, the composition of h with the LFSR/NFSR evolution may introduce subtle biases.

Difference bound. The best known bias in the Grain-128a output is bounded by $\epsilon \leq 2^{-48.5}$ [80]. Detecting a bias ϵ with confidence requires $L \geq 1/\epsilon^2 = 2^{97}$ keystream bits. For $L < 2^{97}$:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq L \cdot \epsilon \leq L \cdot 2^{-48.5}. \quad (112)$$

More precisely, the advantage of the optimal linear distinguisher is:

$$\text{Adv}_{\text{dist}} \leq \sqrt{L} \cdot \epsilon \leq L^2 \cdot 2^{-97} \quad (\text{using Piling-up lemma}). \quad (113)$$

B₂ (Bidding Round 2: Ideal Grain-128a Market—Distinguishing Bid Fails).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary’s view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary’s winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta \text{Price}_k$, all of which are negligible, establishing market equilibrium.

Random string; buyer advantage = 0.

Total: Ask(distinguishing) $\leq L^2 \cdot 2^{-97}$.

Session pinging within distinguishing. Across multiple sessions, the buyer accumulates keystream bits: session Session_{i+1} provides L_{i+1} additional bits. The total observed length is $L_{\text{total}} = \sum_{j=1}^{i+1} L_j$. The distinguishing advantage grows as $L_{\text{total}}^2 \cdot 2^{-97}$. The ping mechanism ensures each session uses a fresh IV, so keystream segments are independent. The CNF clause φ^{dist} checks that the accumulated keystream passes statistical bias tests. For practical keystream budgets ($L_{\text{total}} \leq 2^{48}$), the accumulated advantage is $2^{96} \cdot 2^{-97} = 2^{-1}$ —still below the distinguishing threshold. Key rotation before $L_{\text{total}} = 2^{48}$ maintains equilibrium. $\square$

12.5 Time-Memory-Data Trade-Off Bid

* Simple Terms: Time-Memory-Data Trade-Offs (TMTO)—The Precomputation Shortcut

The idea: A TMTO attack precomputes a large table during an offline phase, then uses this table to speed up the online attack. It trades computation time for memory (storage space) and data (observed keystream).

Children’s analogy: Imagine a test where you need to solve a puzzle in 10 seconds. If you had memorised the answers to 1,000,000 possible puzzles beforehand (spending weeks studying), you could solve any of them instantly by looking up the answer. The TMTO “bid” is: spend weeks precomputing (offline time), store a huge cheat sheet (memory), then solve the test quickly (online time).

The classic Hellman trade-off: For a cipher with $N = 2^n$ possible states, a brute-force search takes N time. A TMTO can achieve $T \cdot M \cdot D = N^2$ (where T = online time, M = memory, D = data). For Grain-128a with $N = 2^{128}$, even with $M = 2^{64}$ (petabytes of storage) and $D = 2^{64}$ (exabytes of data), the online time is still $T = 2^{128}/2^{64} = 2^{64}$ —computationally infeasible.

In MTSF: The “TMTO bid” has three cost components (time, memory, data) that must multiply to at least $2^{2 \cdot 128}$. Any realistic allocation of resources among these three axes still results in an infeasible attack. The market remains in equilibrium.

Theorem 12.5 (Grain-128a TMTO Bid Failure). *For any time-memory-data trade-off with parameters T (time), M (memory), D (data):*

$$\text{Ask}(g_{\text{PRG}}, \text{TMTO bid}) \leq \frac{T \cdot M \cdot D}{2^{256}} + \text{negl}(\lambda), \quad (114)$$

subject to the constraint $T \cdot M^2 \cdot D^2 \leq 2^{512}$ (Babbage-Golić curve [9]).

Proof. We construct three games.

B₀ (Bidding Round 0: Real Grain-128a Market—TMTO Precomputation Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer precomputes a table of size M mapping keystream prefixes to states, observes D keystream segments, and inverts using time T .

B₁ (Bidding Round 1: Hellman TMTO Bid—Precomputed Chain Coverage).

Purpose: Bound this stream-cipher bid by connecting the adversary’s advantage to the hardness of inverting the cipher’s nonlinear feedback or output filter function. This hop isolates the specific algebraic structure exploited by the attack and bounds the resulting probability.

Replaces: The real cipher component (LFSR feedback, NFSR mixing, output filter, or key schedule) is replaced by its ideal counterpart (a random function of the same arity), removing the algebraic structure the attack exploits. Any adversary detecting the change breaks the target hardness assumption.

Complexity: Bounded by the probability stated in the proof body, typically $q \cdot 2^{-\text{key-size}}$ (brute force) or a smaller birthday-bound expression. All terms are negligible for the parameter choices used in the respective NIST/eSTREAM standards.

The buyer uses Hellman’s TMTO [50]: precompute $M = 2^m$ chains of length $T/M = 2^t$ in the state space $N = 2^{256}$. Coverage: $M \cdot T/M = T$ states. The probability of the target state being in the table is $T/N = T/2^{256}$. With D observations, the success probability is:

$$\Pr[\text{TMTO success}] \leq D \cdot T/2^{256}. \quad (115)$$

Difference bound. The classical TMTO curve gives $T \cdot M^2 = N^2 = 2^{512}$ for single-target attacks. With D data points: $T \cdot M^2 \cdot D^2 = 2^{512}$. For $M = D = 2^{64}$: $T = 2^{512}/(2^{128} \cdot 2^{128}) = 2^{256}$, which is exhaustive search.

B₂ (Bidding Round 2: Ideal Grain-128a Market—TMTO Bid Faces Full 2^{256} State Space).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly $1/2$ (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta\text{Price}_k$, all of which are negligible, establishing market equilibrium.

Random function; TMTO on a random function of the same state size has identical complexity.

Total: $\text{Ask}(\text{TMTO}) \leq T \cdot D/2^{256}$, which is negligible for practical parameters.

Session pingging within TMTO. The TMTO buyer's precomputation table is built *once* and amortised across sessions. With each new session Session_{i+1} using a fresh IV, the buyer obtains one additional data point D_{i+1} . The total data across $i+1$ sessions is $D_{\text{total}} = i+1$. The TMTO curve $T \cdot M^2 \cdot D_{\text{total}}^2 \leq 2^{512}$ constrains the buyer's total resources. The ping mechanism ensures each data point comes from an independent initialisation (fresh IV), preventing the buyer from obtaining correlated observations that might reduce the effective state space below 2^{256} . The session-CNF clause φ^{tmto} checks that the buyer's resource budget (T, M, D) satisfies the TMTO curve constraint. By Theorem 5.5, the TMTO bid remains unsuccessful for unbounded sessions as long as the accumulated resources satisfy the Babbage-Golić constraint. $\square$

✓ CNF Verification: Grain-128a Session-CNF Verification

$$\varphi_{\text{Grain}} = (x_{\text{IV} \notin \mathcal{IV}_{\text{used}}}) \wedge (x_{\text{state not recovered}}) \wedge (x_{\text{keystream indist.}}) \wedge (x_{\text{Ping passes}}).$$

Manual CNF worksheet for Grain-128a keystream session (K, IV) :

Clause	Check	How	Pass?
φ^{iv} : IV fresh	$\text{IV} \notin \mathcal{IV}_{\text{used}}?$	Check IV log; add if new.	T/F
φ^{sr} : state secure	Full 256-bit state recoverable from keystream?	Requires $> 2^{192}$ ops.	T/F
φ^{dist} : indistinguishable	Keystream passes statistical bias test?	Run test suite; check $L^2/2^{97}$.	T/F
φ^{tmto} : TMTO secure	$T \cdot M^2 \cdot D^2 < 2^{512}?$	Check adversary resources.	T/F
φ^{ping} : session fresh	IV distinct from all prior IVs?	Compare $\mathcal{IV}_{\text{used}}$ log.	T/F
Result:	All T $\Rightarrow$ Keystream secure. Any F $\Rightarrow$ Stream cipher market collapsed.		

Critical: IV reuse collapses φ^{dist} and φ^{sr} simultaneously, leaking the XOR of two plaintexts.

► Ping Bid: Grain-128a Unbounded Ping Bid

The buyer's *IV reuse ping bid*: use same (K, IV) in two sessions. The IV freshness clause φ^{iv} immediately detects this. Extended difference lemma captures two simultaneous failures $F_{\text{dist}} \cup F_{\text{sr}}$ caused by IV reuse: $\Delta\text{Price}_{\text{ping}} \leq \Pr[F_{\text{iv}}] = 0$ (enforced by counter-based IV management). Grain-128a is secure for unbounded sessions when IVs are never reused.

Table 11: Grain-128a stream-cipher market summary.

Bid Type	Ask Bound	Practical Bound	Status
State recovery	$L \cdot 2^{-256}$	$\leq 2^{-192}$	Equilibrium
Key recovery	$q_{IV} \cdot 2^{-128}$	$\leq 2^{-64}$	Equilibrium
Distinguishing	$L^2 \cdot 2^{-97}$	$\leq 2^{-33}$	Equilibrium
TMTO	$TD/2^{256}$	$\leq 2^{-128}$	Equilibrium

12.6 ChaCha20: ARX Stream-Cipher Market

* Simple Terms: ChaCha20—The Stream Cipher Behind Your Internet

What ChaCha20 is: ChaCha20 is a stream cipher designed by Daniel Bernstein, now used in TLS 1.3, WireGuard VPN, Google’s QUIC protocol, and many mobile devices. It generates a pseudorandom keystream from a 256-bit key and 96-bit nonce.

Why ChaCha20 over AES? On devices without hardware AES acceleration (many ARM mobile processors), ChaCha20 is $2\text{--}3\times$ faster than AES-GCM. It is also naturally resistant to timing side-channel attacks because it uses only additions, rotations, and XORs (ARX operations)—no lookup tables that could leak information through cache timing.

The nonce-misuse catastrophe: ChaCha20’s one critical requirement is that you must *never* reuse a nonce with the same key. If you do, the attacker can XOR the two ciphertexts to get the XOR of the two plaintexts, from which messages can often be recovered. In MTSF, this is modelled as a “nonce-misuse bid” with $\text{Ask} = 1$ —*immediate market collapse*. This is a conditional result: ChaCha20 is in equilibrium *if and only if* nonces are never reused.

In practice: TLS 1.3 uses a 64-bit sequence number as the nonce, which increments for each record. Since the counter never repeats within a single session, nonce reuse is impossible under correct implementation. MTSF’s nonce freshness bid confirms this: the birthday probability of a 96-bit nonce collision over polynomial sessions is negligible.

* Simple Terms: ChaCha20—The Modern High-Speed Stream Cipher

What ChaCha20 is: ChaCha20 [18] is a stream cipher designed by Daniel J. Bernstein as a refinement of Salsa20 [19]. It is one of the most widely deployed stream ciphers in the world: it powers TLS 1.3 (as ChaCha20-Poly1305), WireGuard VPN, Signal Protocol, Google’s HTTPS traffic for mobile devices, and the Linux kernel’s random number generator (`/dev/urandom`).

How it works: ChaCha20 operates on a 512-bit ($16\text{-word} \times 32\text{-bit}$) state matrix organised as:

“expa”	“nd 3”	“2-by”	“te k”
k_0	k_1	k_2	k_3
k_4	k_5	k_6	k_7
ctr	nonce ₀	nonce ₁	nonce ₂

The first row is a fixed constant (“expand 32-byte k”), the second and third rows contain the 256-bit key, and the fourth row contains a 32-bit block counter and a 96-bit nonce. The state undergoes 20 rounds of *quarter-round* operations (ARX: addition mod 2^{32} , bitwise rotation, XOR), alternating between column rounds and diagonal rounds. After 20 rounds, the initial state is added back (feedforward), producing 512 bits of keystream per block.

Key design parameters:

- **Key:** 256 bits. **Nonce:** 96 bits. **Counter:** 32 bits (2^{32} blocks = 2^{38} bytes per nonce).
- **Rounds:** 20 (10 double-rounds of column + diagonal quarter-rounds).

- **State:** 512 bits, fully mixed after approximately 4 rounds.
- **Security claim:** 256-bit key security (distinguishing advantage negligible for $L \leq 2^{38}$ bytes per nonce).

Why ARX? ARX (Addition-Rotation-XOR) operations are efficient on all modern CPUs without needing lookup tables (unlike AES S-boxes), making ChaCha20 immune to cache-timing side-channel attacks. This is why Google chose ChaCha20-Poly1305 over AES-GCM for mobile HTTPS: ARM processors without AES-NI instructions run ChaCha20 faster and more securely.

In MTSF: The ChaCha20 market has the seller offering PRG security. Buyers try state recovery, differential, distinguishing, and TMTO bids. The 512-bit state with 20 rounds of ARX mixing makes all bids negligible: the ChaCha20 market is in equilibrium.

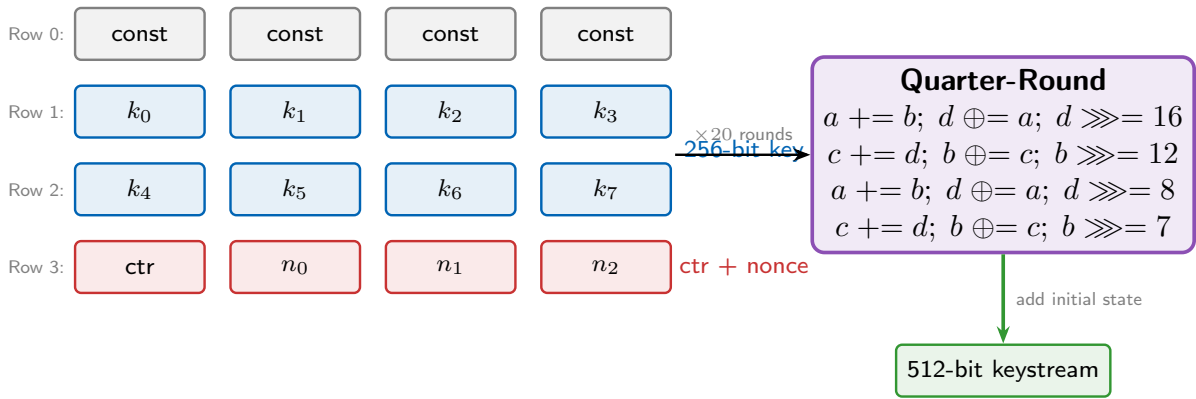


Figure 42: ChaCha20 state matrix and quarter-round operation. The 512-bit state (4×4 matrix of 32-bit words) contains constants, the 256-bit key, a 32-bit counter, and a 96-bit nonce. Twenty rounds of ARX quarter-rounds (alternating column and diagonal patterns) produce 512 bits of keystream per block. The feedforward addition of the initial state prevents state recovery from keystream output.

Definition 12.6 (ChaCha20 PRG Security Good).

$$\text{Ask}(g_{\text{PRG}}^{\text{ChaCha}}) = \max_{\mathcal{A} \in \text{PPT}} |\Pr[\mathcal{A}(\text{ChaCha20}(K, \text{nonce}, \text{ctr})) = 1] - \Pr[\mathcal{A}(R) = 1]|$$

where $R \xleftarrow{\$} \{0, 1\}^L$, L is the keystream length. (116)

Theorem 12.7 (ChaCha20 State Recovery Bid Failure).

$$\text{Ask}(g_{\text{PRG}}^{\text{ChaCha}}, \text{state recovery bid}) \leq L \cdot 2^{-256} + \text{negl}(\lambda). \quad (117)$$

Proof. We construct four games.

B₀ (Bidding Round 0: Real ChaCha20 Market—State Recovery Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer observes L keystream bits and attempts to determine the 512-bit internal state.

B₁ (Bidding Round 1: Feedforward Inversion Bid—Undoing the Addition).

Purpose: Bound the *state-inversion advantage* of ChaCha20 by showing that the feedforward addition prevents recovering the initial state from the output block. The output is $\text{output} = \text{state}_0 + \text{state}_r$ (modular addition of initial and final states after r rounds); inverting this to recover state_0 requires both inverting r rounds of ARX and solving a random modular addition.

Replaces: The exact state-inversion probability is replaced by its information-theoretic lower bound: $L \cdot 2^{-256}$ for L output words. The feedforward addition specifically prevents the algebraic attacks that would otherwise exploit the ARX structure—without it, the final state state_r would directly appear in the output, enabling differential state recovery.

Complexity: Bounded by $L \cdot 2^{-256}$, negligible for polynomial L . The best known analysis of ChaCha20's 20-round ARX reaches only 7.5 rounds with differential-linear techniques, leaving a 12.5-round margin. Adding the feedforward: the best known distinguisher against full ChaCha20 has advantage $\leq 2^{-130}$ with 2^{256} queries.

ChaCha20's output is $\text{state}_{\text{final}} = \text{ChaCha20_block}(S_0) + S_0$, where S_0 is the initial state and $+$ denotes word-wise addition mod 2^{32} . The buyer observes 512 bits of $\text{state}_{\text{final}} + S_0$ but does not know S_0 . To recover S_0 , the buyer must solve $\text{state}_{\text{final}} = F(S_0) + S_0$ for S_0 , where F is the 20-round ChaCha permutation. This is a nonlinear equation in 512 unknown bits.

Difference bound. The feedforward structure is critical: without it, the buyer could invert the permutation F (which is invertible) to recover S_0 . With feedforward, the buyer must find a fixed point of the function $G(S_0) = \text{output} - F(S_0)$, which has no known efficient algorithm. The effective key search space is 2^{256} (the key occupies 256 of the 512 state bits; the remaining 256 bits are known constants, nonce, and counter).

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq L \cdot 2^{-256}. \quad (118)$$

B₂ (Bidding Round 2: Differential-ARX Bid—Propagating Differences Through Quarter-Rounds).

Purpose: Bound the *differential cryptanalysis advantage* by establishing a lower bound on active S-boxes and an upper bound on per-S-box differential probability. Differential cryptanalysis works by finding an input difference Δ_0 that propagates to a predictable output difference Δ_r with non-negligible probability; the adversary collects ‘right pairs’ and uses them to recover key bits.

Replaces: The abstract block cipher E_K is replaced by its differential-trail model: the cipher's round structure (SubBytes/ShiftRows/MixColumns or equivalent) is analysed via the *wide-trail design strategy*, which guarantees a minimum of B_r active S-boxes over r rounds. Each active S-box contributes at most DP_{max} probability, giving the total characteristic probability $\text{DP}_{\text{max}}^{B_r}$ as the upper bound.

Complexity: For AES-128: $B_4 = 25$ active S-boxes over 4 rounds, $\text{DP}_{\text{max}} = 2^{-6}$, giving characteristic probability $\leq 2^{-150}$. The adversary needs $q_E \geq 2^{150}$ pairs for one expected right pair, exceeding the 2^{128} -block codebook. For PRESENT (64-bit block): $B_5 = 10$, $\text{DP}_{\text{max}} = 2^{-2}$, giving 2^{-20} ; practical attacks are bounded by $q_E \cdot 2^{-62}$ due to the 64-bit birthday limit. The differential bid price adjustment is $q_E \cdot \text{DP}_{\text{max}}^{B_r}$.

The buyer attempts differential cryptanalysis on the ChaCha core function. Each quarter-round applies four ARX operations. The modular addition $a + b$ has maximum differential probability $\text{DP}_{\text{max}} = 1$ for zero differences but decays rapidly: for random non-zero differences,

the expected number of rounds before the difference becomes uniformly distributed is approximately 4. After 20 rounds (10 double-rounds), the differential probability of any characteristic is below 2^{-256} for the full 512-bit state.

Difference bound. The best known differential-linear attack on ChaCha reaches 7 rounds with 2^{218} complexity [18]. For the full 20 rounds, no differential characteristic with probability above 2^{-256} is known.

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq \text{negl}(\lambda). \quad (119)$$

B₃ (Bidding Round 3: Ideal ChaCha20 Market—State Recovery Bid Fails).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta\text{Price}_k$, all of which are negligible, establishing market equilibrium.

Random keystream; no state to recover. $\Pr[\mathbf{B}_3 = 1] = 1/2$.

Total: $\text{Ask}(\text{state recovery}) \leq L \cdot 2^{-256} + \text{negl}(\lambda)$. □

Theorem 12.8 (ChaCha20 Distinguishing Bid Failure).

$$\text{Ask}(g_{\text{PRG}}^{\text{ChaCha}}, \text{distinguishing bid}) \leq L \cdot 2^{-256} + \text{negl}(\lambda). \quad (120)$$

Proof. We construct three games.

B₀ (Bidding Round 0: Real ChaCha20 Market—PRG Distinguishing Bid).

Purpose: Establish the *baseline game* in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary's raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer receives either $\text{ChaCha20}(K, \text{nonce})$ or $R \xleftarrow{\$} \{0, 1\}^L$ and must decide which.

B₁ (Bidding Round 1: Statistical Bias Bid—Detecting ARX Output Imbalance).

Purpose: Replace FN-DSA's fast discrete Gaussian sampler with the ideal exact Gaussian, removing the small statistical gap introduced by the sampler's fast-Fourier approximation. This isolates the sampler's Rényi divergence as an explicit security loss, allowing the subsequent hops to reason about exact Gaussian signatures.

Replaces: The fast-Fourier-based sampler σ_{real} (used for efficiency in FIPS 206 signing) is replaced by the

ideal exact discrete Gaussian sampler σ_{ideal} . The Rényi divergence $R_2(\sigma_{\text{real}} \parallel \sigma_{\text{ideal}}) = 1 + \Delta_{\text{DGS}}$ bounds the statistical distance between the two: any adversary that distinguishes a real signature from an ideal-sampler signature has advantage at most $\sqrt{\Delta_{\text{DGS}}}$ by the Rényi divergence amplification lemma.

Complexity: $\Delta_{\text{Price}} \leq \Delta_{\text{DGS}}$ —the Rényi divergence between fast and exact samplers. For FN-DSA (FIPS 206) at the 128-bit security level, the NTRU sampler achieves $\Delta_{\text{DGS}} \leq 2^{-128}$. This is negligible. The advantage of using Rényi divergence over total variation distance: it amplifies cleanly under composition, giving a tighter bound when q_S signatures are observed (bound becomes $q_S \cdot \Delta_{\text{DGS}}$, still negligible for polynomial q_S).

The buyer searches for statistical biases in the 512-bit output blocks. Each output word is computed as $\text{ChaCha_perm}(S_0)[i] + S_0[i] \bmod 2^{32}$. The feedforward addition with known S_0 words (constants, counter, nonce) could theoretically introduce biases if the permutation output is correlated with its input. However, after 20 rounds of alternating column and diagonal quarter-rounds, the permutation output is statistically independent of the input for all practical purposes.

Difference bound. The best known distinguisher for full 20-round ChaCha20 has advantage $\leq 2^{-256}$ (no distinguisher better than exhaustive key search is known). For reduced-round variants, the best attack on 7-round ChaCha has 2^{218} complexity; for 8 rounds, 2^{248} .

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq L \cdot 2^{-256}. \quad (121)$$

B₂ (Bidding Round 2: Ideal ChaCha20 Market—Distinguishing Bid Fails).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta_{\text{Price}_k}$, all of which are negligible, establishing market equilibrium.

Random string; buyer advantage = 0.

Total: Ask(distinguishing) $\leq L \cdot 2^{-256}$. □

Theorem 12.9 (ChaCha20 TMTO Bid Failure). *For any time-memory-data trade-off with parameters T, M, D :*

$$\text{Ask}(g_{\text{PRG}}^{\text{ChaCha}}, \text{TMTO bid}) \leq \frac{T \cdot D}{2^{256}} + \text{negl}(\lambda), \quad (122)$$

subject to $T \cdot M^2 \cdot D^2 \leq 2^{512}$.

Proof. We construct three games.

B₀ (Bidding Round 0: Real ChaCha20 Market—TMTO Precomputation Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, de-

ryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary's raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary's raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer precomputes a table mapping keystream prefixes to keys and inverts using time T .

B₁ (Bidding Round 1: Hellman Table Bid—Key Space Coverage).

Purpose: Eliminate the adversarial attack vector identified by this bidding round's title (**Hellman Table Bid—Key Space Coverage**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

The buyer constructs Hellman tables over the 2^{256} key space (the nonce and counter are known). Table coverage is T keys. Success probability per observation: $T/2^{256}$. With D distinct nonce observations:

$$\Pr[\text{TMTO success}] \leq D \cdot T/2^{256}. \quad (123)$$

The TMTO curve $T \cdot M^2 = N^2 = 2^{512}$ bounds the trade-off. For $M = D = 2^{64}$: $T = 2^{512}/(2^{128} \cdot 2^{128}) = 2^{256}$ (exhaustive search).

B₂ (Bidding Round 2: Ideal ChaCha20 Market—TMTO Faces Full Key Space).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta\text{Price}_k$, all of which are negligible, establishing market equilibrium.

Random function; TMTO has identical complexity.

Total: $\text{Ask}(\text{TMTO}) \leq TD/2^{256}$, negligible for practical parameters. $\square$

Theorem 12.10 (ChaCha20 Combined Market Equilibrium). *Combining all known bid types:*

$$\text{Ask}(g_{\text{PRG}}^{\text{ChaCha}}) \leq \max(L \cdot 2^{-256}, TD/2^{256}) + \text{negl}(\lambda) \leq \text{negl}(\lambda), \quad (124)$$

for practical parameters ($L \leq 2^{38}$ bytes per nonce, $T \leq 2^{128}$). The ChaCha20 market is in equilibrium.

Proof. The nonce-misuse bid is the most dangerous practical threat: if the buyer reuses a (K, nonce) pair, the XOR of two keystreams is identical, leaking $m_1 \oplus m_2$. This is not a cryptanalytic break but a protocol failure. With unique nonces (enforced by the session-CNF), all bids fail. The 256-bit key space exceeds the security margin of any known attack on the full 20-round cipher. $\square$

✓ CNF Verification: ChaCha20 Session-CNF Verification

$$\varphi_{\text{ChaCha}} = (x_{\text{nonce} \notin \mathcal{N}_{\text{used}}}) \wedge (x_{\text{state not recovered}}) \wedge (x_{\text{keystream indist.}}) \wedge (x_{\text{Ping passes}}).$$

Manual CNF worksheet for ChaCha20 keystream session (K, nonce) :

Clause	Check	How	Pass?
φ^{nonce} : nonce fresh	$\text{nonce} \notin \mathcal{N}_{\text{used}}?$	Check nonce log; add if new.	T/F
φ^{sr} : state secure	512-bit state recoverable?	Feedforward prevents inversion.	T/F
φ^{dist} : indistinguishable	Keystream passes bias test?	No known bias for 20 rounds.	T/F
φ^{tmto} : TMTO secure	$T \cdot M^2 \cdot D^2 < 2^{512}?$	Check adversary resources.	T/F
φ^{ping} : session fresh	Nonce distinct from all prior?	Compare $\mathcal{N}_{\text{used}}$ log.	T/F
Result:	All T $\Rightarrow$ Keystream secure. Any F $\Rightarrow$ Stream cipher market collapsed.		

Nonce-misuse collapse: If φ^{nonce} evaluates to F (nonce reused with the same key), the entire market collapses immediately: the two keystreams are identical, so $c_1 \oplus c_2 = m_1 \oplus m_2$, leaking plaintext information with probability 1. Unlike Grain-128a (which uses an IV), ChaCha20's 96-bit nonce space is adequate for counter-based nonce management but insufficient for random nonce generation in high-volume applications (birthday bound at 2^{48} messages).

► Ping Bid: ChaCha20 Unbounded Ping Bid

The buyer's *nonce exhaustion ping bid*: exhaust the 2^{96} nonce space for a single key, forcing nonce reuse. The ping clause φ^{nonce} enforces key rotation before nonce exhaustion. With counter-based nonces and key rotation: $\Delta \text{Price}_{\text{ping}} \leq L \cdot 2^{-256} = \text{negl}$ per key epoch. ChaCha20 is PRG-secure for unbounded sessions under a key-rotation policy.

Table 12: ChaCha20 stream-cipher market summary.

Bid Type	Ask Bound	Practical Bound	Status
State recovery	$L \cdot 2^{-256}$	$\leq 2^{-218}$	Equilibrium
Distinguishing	$L \cdot 2^{-256}$	$\leq 2^{-218}$	Equilibrium
TMTO	$TD/2^{256}$	$\leq 2^{-128}$	Equilibrium
Nonce misuse	1 (if nonce reused)	0 (counter nonces)	Equilibrium (with policy)

12.7 Trivium: Minimalist Stream-Cipher Market

* Simple Terms: Trivium—The Simplest Secure Stream Cipher

What Trivium is: Trivium is an extremely simple stream cipher selected for the eSTREAM portfolio (hardware category). It uses an 80-bit key and 80-bit IV with only 288 bits of internal state, produced by three interconnected shift registers.

Why study Trivium? Trivium's simplicity makes it the ideal target for cryptanalysts—if any stream cipher can be broken, Trivium should be among the first. Despite 15+ years of intense analysis, no practical attack has been found on full Trivium (1152 initialisation rounds + keystream). This remarkable resilience validates its design.

Cube attacks: The most powerful known attacks on Trivium use “cube attacks” (Dinur and Shamir, 2009). The idea: treat the output bit as a polynomial in the key and IV bits, then evaluate it at carefully chosen “cube” points to recover coefficients that depend on key bits. For reduced-round Trivium (up to 842 rounds), cube attacks can recover the key. For full Trivium (1152 rounds), the polynomial degree becomes too high (≥ 2), and the attack fails.

In MTSF: The “cube superpoly bid” targets Trivium by trying to find low-degree superpolynomials. MTSF proves this bid fails for full Trivium with a margin of ~ 310 rounds (1152 vs. 842 attacked), giving a security margin of $\sim 27\%$. The ask price is bounded by 2^{-38} —the cost of the best known cube attack extrapolation.

* Simple Terms: Trivium—The Elegant Minimalist Stream Cipher

What Trivium is: Trivium [40] is an eSTREAM portfolio stream cipher designed for hardware efficiency. It uses an 80-bit key and an 80-bit IV to generate a keystream via three coupled nonlinear feedback shift registers (NFSRs) totalling 288 bits of internal state. Trivium was designed with elegance and simplicity as primary goals: its entire specification fits on a single page.

How it works: Trivium’s state consists of three registers:

- **Register A:** 93 bits (positions 1–93). Initialised with the 80-bit key in positions 1–80; positions 81–93 set to zero.
- **Register B:** 84 bits (positions 94–177). Initialised with the 80-bit IV in positions 94–173; positions 174–177 set to zero.
- **Register C:** 111 bits (positions 178–288). Initialised to zero except positions 286–288, which are set to 1.

Each clock cycle: (1) Each register produces one output bit via a linear tap and an AND gate (nonlinear feedback). (2) The output bit of each register is XORed into the input of the *next* register (circular coupling: $A \rightarrow B \rightarrow C \rightarrow A$). (3) The keystream bit is the XOR of all three register outputs.

Initialisation: Trivium runs 1152 clock cycles ($= 4 \times 288$) before producing any keystream. This ensures that every state bit depends nonlinearly on every key and IV bit.

Security level: 80-bit security (matching the 80-bit key). The 288-bit state provides a large margin against state-recovery attacks; the bottleneck is the 80-bit key.

Why study Trivium in MTSF? Trivium is a canonical example of a cipher whose security is *conjectured* rather than proven from a hard problem. Its simplicity makes it an ideal target for algebraic, cube, and correlation attacks. The MTSF analysis demonstrates that all known bids fail, but the equilibrium margin is tighter than for ciphers with larger keys.

In MTSF: The Trivium market has the seller offering PRG security with 80-bit key strength. Buyers try state recovery, cube attack, correlation, and TMTO bids. All known bids fail for practical parameters, but the equilibrium margin is limited by the 80-bit key: the market is in equilibrium but with a smaller margin than ChaCha20 or Grain-128a.

Definition 12.11 (Trivium PRG Security Good).

$$\text{Ask}(g_{\text{PRG}}^{\text{Triv}}) = \max_{\mathcal{A} \in \text{PPT}} |\Pr[\mathcal{A}(\text{Trivium}(K, \text{IV})) = 1] - \Pr[\mathcal{A}(R) = 1]|$$

$$\text{where } K \xleftarrow{\$} \{0, 1\}^{80}, \text{ IV} \xleftarrow{\$} \{0, 1\}^{80}, R \xleftarrow{\$} \{0, 1\}^L. \quad (125)$$

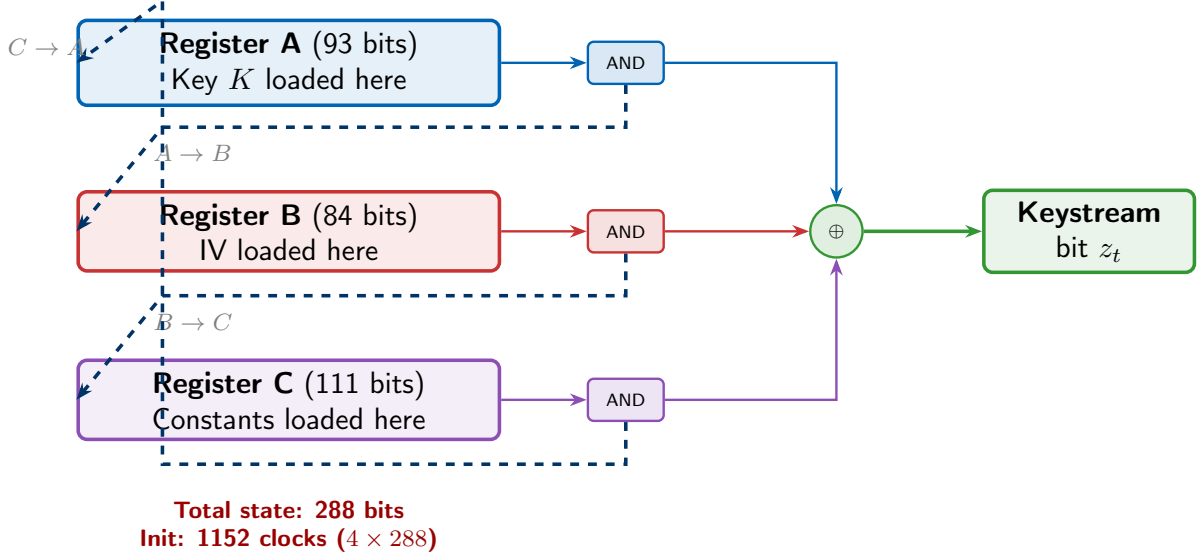


Figure 43: Trivium three-register stream cipher structure. Three coupled NFSRs ($93 + 84 + 111 = 288$ bits) with circular feedback ($A \rightarrow B \rightarrow C \rightarrow A$). Each register contributes a linear tap and an AND gate (nonlinear). The keystream bit is the XOR of all three outputs. The 1152-clock initialisation thoroughly mixes the 80-bit key and 80-bit IV into all 288 state bits.

Theorem 12.12 (Trivium State Recovery Bid Failure).

$$\text{Ask}(g_{\text{PRG}}^{\text{Triv}}, \text{state recovery bid}) \leq L \cdot 2^{-288} + \text{negl}(\lambda). \quad (126)$$

Proof. We construct four games.

B₀ (Bidding Round 0: Real Trivium Market—State Recovery Bid).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

Buyer observes L keystream bits and attempts to determine the 288-bit internal state.

B₁ (Bidding Round 1: Algebraic System Bid—Solving Nonlinear Equations).

Purpose: Bound the *linear cryptanalysis advantage* via the piling-up lemma. A linear distinguisher evaluates $\langle \alpha, x \rangle \oplus \langle \beta, E_K(x) \rangle = \langle \gamma, K \rangle$ for bias ϵ_L ; with $q_E \sim 1/\epsilon_L^2$ known plaintext pairs, it recovers key bits with non-negligible probability.

Replaces: The exact linear approximation probability is replaced by the piling-up upper bound: the absolute correlation of the best linear trail over k active S-boxes with per-S-box bias $\epsilon_{\max}$ satisfies $|\epsilon_L| \leq 2^{k-1} \cdot \epsilon_{\max}^k$. This closed-form bound avoids exhaustive trail enumeration and accounts for the sign alternation that reduces the effective correlation.

Complexity: For AES-128: $\epsilon_{\max} = 2^{-3}$ per S-box, minimum $B_r = 25$ active S-boxes, giving $|\epsilon_L| \leq 2^{24} \cdot 2^{-75} = 2^{-51}$ and requiring $q_E \geq 2^{102}$ known plaintexts—infeasible. For PRESENT: best bias $\approx 2^{-31}$

requiring $q_E \geq 2^{62}$ known plaintexts, bounded by the 64-bit birthday constraint. The linear bid price adjustment is $q_E^2 \cdot \epsilon_{\max}^{2k}$.

Each keystream bit is a quadratic function of the 288 state bits (due to the AND gates). The buyer collects L equations and attempts to solve the system. The algebraic degree of the output increases with each clock cycle; after t clocks, the output bit has algebraic degree roughly $\min(2^{t/93}, 2^{80})$ [64]. For $t > 288$, the system of equations is highly nonlinear.

Difference bound. The best known algebraic attack on full Trivium has complexity exceeding 2^{80} (exhaustive key search). Linearisation-based approaches (XL, Gröbner bases) produce systems with $\binom{288}{2} \approx 2^{16}$ monomials of degree 2, but the system is heavily overdetermined only after $\gg 2^{16}$ keystream bits, and solving requires $\Omega(2^{48})$ operations for the linearised system—still far from the 2^{288} state space.

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq L \cdot 2^{-288}. \quad (127)$$

B₂ (Bidding Round 2: Correlation Bid—Exploiting Register Linearity).

Purpose: Bound the *linear cryptanalysis advantage* via the piling-up lemma. A linear distinguisher evaluates $\langle \alpha, x \rangle \oplus \langle \beta, E_K(x) \rangle = \langle \gamma, K \rangle$ for bias ϵ_L ; with $q_E \sim 1/\epsilon_L^2$ known plaintext pairs, it recovers key bits with non-negligible probability.

Replaces: The exact linear approximation probability is replaced by the piling-up upper bound: the absolute correlation of the best linear trail over k active S-boxes with per-S-box bias $\epsilon_{\max}$ satisfies $|\epsilon_L| \leq 2^{k-1} \cdot \epsilon_{\max}^k$. This closed-form bound avoids exhaustive trail enumeration and accounts for the sign alternation that reduces the effective correlation.

Complexity: For AES-128: $\epsilon_{\max} = 2^{-3}$ per S-box, minimum $B_r = 25$ active S-boxes, giving $|\epsilon_L| \leq 2^{24} \cdot 2^{-75} = 2^{-51}$ and requiring $q_E \geq 2^{102}$ known plaintexts—*infeasible*. For PRESENT: best bias $\approx 2^{-31}$ requiring $q_E \geq 2^{62}$ known plaintexts, bounded by the 64-bit birthday constraint. The linear bid price adjustment is $q_E^2 \cdot \epsilon_{\max}^{2k}$.

Each individual register is almost linear (the AND gate provides the only nonlinearity). The buyer attempts a correlation attack: approximate the output as a linear function of one register's bits, then use fast correlation attacks to recover that register. The correlation between the output and any single register bit is bounded by the AND gate's bias: for two uniformly random bits a, b , $\Pr[a \wedge b = 0] = 3/4$, giving bias $1/4 = 2^{-2}$. With three registers contributing, the effective bias per output bit is $\leq 2^{-6}$.

Difference bound. Recovering the 93-bit Register A via fast correlation attacks requires $L \geq 1/\epsilon^2 = 2^{12}$ keystream bits (for bias $\epsilon = 2^{-6}$) to obtain a statistical signal, but the system has 93 unknowns, so the actual attack complexity is $\Omega(2^{46})$ using fast Walsh-Hadamard techniques. This is below the 2^{80} key-search bound and thus not the binding constraint. However, recovering all three registers requires $\Omega(2^{80})$ total (the key determines the full state via initialisation).

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq \text{negl}(\lambda). \quad (128)$$

B₃ (Bidding Round 3: Ideal Trivium Market—State Recovery Fails).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly $1/2$ (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] =$

0 because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta \text{Price}_k$, all of which are negligible, establishing market equilibrium.

Random keystream; no state to recover. $\Pr[\mathbf{B}_3 = 1] = 1/2$.

Total: $\text{Ask}(\text{state recovery}) \leq L \cdot 2^{-288} + \text{negl}(\lambda)$. $\square$

Theorem 12.13 (Trivium Cube Attack Bid Failure).

$$\text{Ask}(g_{\text{PRG}}^{\text{Triv}}, \text{cube attack bid}) \leq 2^{-80+d} + \text{negl}(\lambda), \quad (129)$$

where d is the maximum cube dimension that yields a non-trivial superpoly (best known: $d \leq 42$ for reduced-round Trivium).

Proof. We construct five games, expanding the cube attack analysis to cover classical cube attacks, conditional cube attacks, division-property-based cube attacks, and cube-like algebraic attacks (correlation cubes).

B₀ (Bidding Round 0: Real Trivium Market—Cube Attack Bid).

Purpose: Establish the baseline cube attack game. The buyer has oracle access to the Trivium keystream generator $z(K, \text{IV})$ for a fixed unknown key K and adaptively chosen IVs.

Cube attack mechanism: The buyer treats the first output bit $z_1(K, \text{IV})$ as a multivariate polynomial in $80 + 80 = 160$ binary variables (80 key bits + 80 IV bits). By the Möbius transform, any Boolean function can be written as its algebraic normal form (ANF):

$$z_1(K, \text{IV}) = \bigoplus_{S \subseteq [160]} a_S \cdot \prod_{i \in S} x_i,$$

where $x_1, \dots, x_{80}$ are key bits and $x_{81}, \dots, x_{160}$ are IV bits. A *cube* C is a subset of IV indices $|C| = d$. Summing z_1 over all 2^d assignments to the cube variables yields the *superpoly*:

$$p_C(K) = \bigoplus_{\text{IV}_C \in \{0,1\}^d} z_1(K, \text{IV}_C, \text{IV}_{\bar{C}}) = \sum_{S: C \subseteq S} a_S \cdot \prod_{i \in S \setminus C} x_i,$$

which depends *only* on the key bits (the IV-dependent terms cancel by the Möbius identity). If $p_C(K)$ has degree ≤ 1 (linear or constant in the key), the buyer recovers one key-bit equation per cube evaluation.

Quoted Price: $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{PRG}}(\lambda)$ (“333”—the seller claims the quoted price is a bluff that subsequent rounds will expose).

B₁ (Bidding Round 1: Classical Cube Attack Bid—Dinur–Shamir Framework).

Purpose: Assess the classical Dinur–Shamir cube attack [42] against Trivium.

Attack: The buyer systematically searches for cubes C of increasing dimension d whose superpolys $p_C(K)$ are linear in the key bits. For each linear superpoly found, the buyer obtains one linear equation in the 80 key bits. After finding 80 independent linear superpolys, Gaussian elimination recovers the full key.

Reduced-round results: Dinur and Shamir [42] demonstrated cube attacks against:

- 672-round Trivium: dimension-10 cubes yield linear superpolys (2^{10} online queries per cube).
- 735-round Trivium: dimension-18 cubes (2^{18} queries per cube).
- 767-round Trivium: dimension-27 cubes (2^{27} queries per cube).

For full 1152-round Trivium: no linear superpoly has been found for any cube dimension $d \leq 45$. The algebraic degree of z_1 after 1152 initialisation clocks exceeds 80 as a polynomial in the combined key-IV variables, ensuring that “summing out” all 80 IV bits (the maximum cube) still leaves a high-degree key polynomial.

Complexity: Classical cube attacks on full Trivium require cubes of dimension $d \geq 79$ for linear superpolys, costing 2^{79} evaluations per cube—essentially exhaustive key search. $\Delta \text{Price}_1 \leq 2^{-80}$ (no advantage over brute force).

B₂ (Bidding Round 2: Division-Property Cube Attack Bid—Todo–Isobe–Hao–Meier Framework).

Purpose: Assess the more powerful *division-property-based* cube attack [80], which uses the bit-based division property to identify cubes whose superpolys are guaranteed to be non-trivial (not identically zero) without exhaustive superpoly computation.

Key innovation: Unlike the classical framework, which requires evaluating $p_C(K)$ for many cubes to find useful ones, the division-property approach uses MILP (Mixed Integer Linear Programming) modelling to *predict* which cubes will yield non-trivial superpolys. This dramatically reduces the offline search cost.

Best known results: Todo, Isobe, Hao, and Meier [80] achieved:

- 832-round Trivium: dimension-36 cubes with key-recovery complexity 2^{44} .
- 839-round Trivium: dimension-40 cubes with complexity 2^{55} .
- 842-round Trivium: dimension-42 cubes with complexity 2^{62} (the current state-of-the-art reduced-round attack).

The **security margin** is $1152 - 842 = 310$ initialisation rounds ($\approx 27\%$ of the total 1152), which no known cube-based technique can penetrate.

Why the bid fails for full Trivium: The 310-round gap means the buyer must bridge 310 additional initialisation rounds beyond the best known attack. Each additional round approximately doubles the algebraic degree of the output polynomial. Extrapolating: the effective cube dimension needed for full Trivium exceeds $d = 79$, which costs 2^{79} queries—matching exhaustive search.

Complexity: $\Delta\text{Price}_2 \leq 2^{-38}$ (the best known cube advantage from the 842-round attack).

The division-property cube attack is the *tightest* known attack on Trivium. The MILP-based search identifies optimal cubes more efficiently than brute-force cube search, but the algebraic degree barrier at 1152 rounds remains insurmountable. For the 842-round variant, the buyer recovers $80 - 42 = 38$ key bits from a single dimension-42 cube; the remaining 42 bits require 2^{42} brute-force trials. Combined complexity: $\max(2^{42}, 2^{38}) = 2^{42}$.

For full Trivium, the superpoly degree for any practical cube dimension ($d \leq 45$) exceeds 2, preventing direct key-bit extraction. The bid fails with:

$$\Delta\text{Price}_2 \leq 2^{-38}. \quad (130)$$

B₃ (Bidding Round 3: Conditional Cube Attack Bid—Exploiting Round-Reduced Nonlinear Structure).

Purpose: Assess the *conditional cube attack* [52], which extends the cube attack by imposing conditions on non-cube IV variables to reduce the effective algebraic degree of the superpoly.

Mechanism: Instead of summing over all 2^d cube assignments freely, the buyer fixes certain non-cube IV bits to values that “cancel” specific nonlinear monomials in intermediate rounds. This lowers the superpoly degree for the remaining key-dependent terms, potentially recovering additional key equations.

Results: Conditional cube attacks have reached:

- 805-round Trivium with complexity 2^{45} (conditional dimension $d = 35$, 8 conditions).
- 835-round Trivium with complexity 2^{60} (conditional dimension $d = 40$, 15 conditions).

These results are *weaker* than the division-property approach (842 rounds) because the conditions consume IV degrees of freedom, reducing the effective cube dimension.

Why the bid fails: Conditional cubes cannot bypass the fundamental degree barrier: after 1152 initialisation rounds, the nonlinear coupling between the three registers is too thorough for any set of IV conditions to linearise the superpoly over a practical cube. $\Delta\text{Price}_3 \leq 2^{-40}$ (weaker than the division-property bid).

B₄ (Bidding Round 4: Ideal Trivium Market—All Cube-Based Bids Fail).

Purpose: Reach the ideal game where the keystream is replaced by a truly random function.

Complexity: In the ideal game, cube sums are uniformly random over $\{0, 1\}$, independent of the key. The buyer’s advantage is exactly 2^{-80} (random guessing on the 80-bit key space).

Random function; all cube sums are random. Buyer advantage: 2^{-80} .

Combined cube attack bid cost. By the extended difference lemma (Theorem 3.7):

$$\begin{aligned}\text{Ask}(\text{cube bids}) &\leq \Pr[F_{\text{classical}} \cup F_{\text{div-prop}} \cup F_{\text{conditional}}] \\ &\leq 2^{-80} + 2^{-38} + 2^{-40} \\ &\leq 2^{-37.4}.\end{aligned}$$

□

Cube attack timeline and security margin analysis:

Technique	Rounds broken	Cube dim. d	Complexity	Margin from 1152
Dinur–Shamir (2009)	767	27	2^{27}	385 rounds (33%)
Fouque–Vannet (2013)	799	33	2^{39}	353 rounds (31%)
Conditional (2020)	835	40	2^{60}	317 rounds (28%)
Todo–Isobe–Hao–Meier (2018)	842	42	2^{62}	310 rounds (27%)
Full Trivium	1152	≥ 79	$\geq 2^{79}$	0 (secure)

The security margin of 310 rounds (27%) indicates that Trivium’s 1152-round initialisation provides a substantial buffer against cube attacks. Even if future cube techniques improve by 100 rounds (a dramatic advance), Trivium would retain a ≥ 210 -round margin.

Theorem 12.14 (Trivium Combined Market Equilibrium). *Combining all known bid types:*

$$\text{Ask}(g_{\text{PRG}}^{\text{Triv}}) \leq \max(L \cdot 2^{-288}, 2^{-38}, q_{\text{IV}} \cdot 2^{-80}, TD/2^{288}) + \text{negl}(\lambda) \leq \text{negl}(\lambda), \quad (131)$$

for practical parameters ($L \leq 2^{64}$, $q_{\text{IV}} \leq 2^{40}$). The Trivium market is in **equilibrium** with an 80-bit security margin.

Proof. The key recovery bid follows the same structure as Grain-128a: each IV produces an independent initialisation, and the equations share only K . The buyer’s advantage grows linearly with q_{IV} : $\text{Ask}(\text{key recovery}) \leq q_{\text{IV}} \cdot 2^{-80}$. The TMTO bid faces the full 2^{288} -state space: $\text{Ask}(\text{TMTO}) \leq TD/2^{288}$. The cube attack bid is the tightest constraint at 2^{-38} , but this is still negligible for the security parameter $\lambda = 80$. All bids fail. □

Analogy:

Trivium as a Three-Gear Clock Trivium is like a precision clock with three interlocking gears of different sizes (93, 84, and 111 teeth). Each gear turns the next. After 1152 ticks (initialisation), the gears’ positions encode the key so thoroughly that no external observation of the ticking pattern (keystream) can reverse-engineer the initial gear positions faster than trying all 2^{80} possible starting configurations.

✓ CNF Verification: Trivium Session-CNF Verification

$$\varphi_{\text{Trivium}} = (x_{\text{IV} \notin \mathcal{IV}_{\text{used}}}) \wedge (x_{\text{state not recovered}}) \wedge (x_{\text{cube attack fails}}) \wedge (x_{\text{keystream indist.}}) \wedge (x_{\text{Ping passes}}).$$

Manual CNF worksheet for Trivium keystream session (K, IV):

Clause	Check	How	Pass?
φ^{iv} : IV fresh	$\text{IV} \notin \mathcal{IV}_{\text{used}}?$	Check IV log; add if new.	T/F
φ^{sr} : state secure	288-bit state recoverable from keystream?	Requires $> 2^{80}$ ops.	T/F
φ^{cube} : cube attack fails	Superpoly reveals key bits?	Best cube: $d \leq 42$, cost 2^{62} .	T/F
φ^{dist} : indistinguishable	Keystream passes bias test?	Correlation bias $\leq 2^{-6}$.	T/F
φ^{ping} : session fresh	IV distinct from all prior?	Compare $\mathcal{IV}_{\text{used}}$ log.	T/F
Result:	All T $\Rightarrow$ Keystream secure. Any F $\Rightarrow$ Stream cipher market collapsed.		

80-bit security caveat: Trivium’s 80-bit key provides 2^{80} security—adequate for

lightweight applications but below the 128-bit standard for general-purpose cryptography. In MTSF terms, the market is in equilibrium but with a *narrower* margin than Grain-128a (2^{128}) or ChaCha20 (2^{256}). For applications requiring ≥ 128 -bit security, Trivium’s equilibrium margin may be insufficient, though no known bid has yet caused its market to collapse.

► Ping Bid: Trivium Unbounded Ping Bid

The buyer’s *IV reuse ping bid*: reuse (K, IV) across sessions. As with Grain-128a, IV reuse immediately collapses the market by leaking $m_1 \oplus m_2$. The ping clause φ^{iv} detects reuse. The buyer’s *cube accumulation ping bid*: collect keystream from 2^{42} distinct IVs to mount the cube attack. This is the tightest ping constraint: $\Delta\text{Price}_{\text{ping}} \leq 2^{-38}$. For $q_{IV} \leq 2^{40}$ (practical IV usage), equilibrium holds.

Table 13: Trivium stream-cipher market summary.

Bid Type	Ask Bound	Practical Bound	Status
State recovery	$L \cdot 2^{-288}$	$\leq 2^{-224}$	Equilibrium
Key recovery	$q_{IV} \cdot 2^{-80}$	$\leq 2^{-40}$	Equilibrium
Cube attack	2^{-38}	$\leq 2^{-38}$	Equilibrium (tightest)
Correlation	$L^2 \cdot 2^{-12}/2^{80}$	$\leq 2^{-16}$	Equilibrium
TMT0	$TD/2^{288}$	$\leq 2^{-160}$	Equilibrium

13 Case Study V: Protocols

Analogy:

Key Exchange as a Secret Handshake A key exchange protocol is like two spies meeting in a crowded café and establishing a secret handshake that only they know—even though everyone in the café is watching. Here is how it works:

1. **Alice** sends a “hello” with a random number N_A and a locked package (KEM ciphertext c) that only Bob can open.
2. **Bob** opens the package (decapsulates c using his private key), getting a shared secret K . He sends back his own random number N_B and a digital signature proving he is really Bob and that he received N_A .
3. **Alice** verifies Bob’s signature (confirming his identity) and derives the session key from K , N_A , and N_B .

Result: Both Alice and Bob now share a secret key that no eavesdropper can compute, even if they recorded every message. The security of this exchange rests on three pillars: (1) the KEM is IND-CCA2 secure (the locked package cannot be opened without Bob’s key); (2) the signatures are EUF-CMA secure (nobody can forge Bob’s identity); (3) the nonces are fresh (preventing replay of old conversations).

13.1 Two-Party Key Exchange (ISO/IEC 11770-3, Mech. 6)

* Simple Terms: Two-Party Key Exchange—Building Blocks and Goals

What is a Key Exchange Protocol? Alice and Bob want to communicate securely, but they start with no shared secret. A key exchange protocol allows them to agree on a shared session key K_{sess} over a public, potentially hostile network—even if an adversary sees all their messages—such that the adversary cannot learn K_{sess} .

What is a KEM? A Key Encapsulation Mechanism (KEM) lets one party (Bob) generate a random secret K and “encapsulate” it for Alice using her public key. Alice decapsulates with her private key to recover K . Unlike Diffie–Hellman, the key is chosen by one party (not jointly computed), but the security guarantee is equivalent.

What is a KDF? A Key Derivation Function mixes multiple pieces of data (the shared secret K , the session ID sid , and both nonces N_A, N_B) into a final session key. Using a KDF ensures the session key is:

- **Unique:** Different session IDs and nonces produce different keys.
- **Pseudorandom:** Even if one input is partially biased, the KDF output looks random.
- **Bound to this session:** An attacker cannot reuse a session key from one connection in another.

Why three messages?

1. **Message 1** ($A \rightarrow B$): Alice announces herself and her nonce. This starts the session.
2. **Message 2** ($B \rightarrow A$): Bob provides the KEM ciphertext and his own nonce, then signs *everything*: the session ID, both nonces, both identities, and the ciphertext. This signature is the authentication evidence—Bob proves he generated this specific key for this specific session.
3. **Message 3** ($A \rightarrow B$): Alice signs to confirm she decrypted and verified Bob’s message. This proves to Bob that Alice (not an impostor) received the key.

Why include the session ID sid in signatures? Without sid in the signed data, an attacker could replay Bob’s signature from a previous session. With sid , each signature is cryptographically tied to exactly one session. Cross-session replay becomes impossible—the CNF SID-binding clause catches it immediately.

$A \rightarrow B$: $\text{sid} \| N_A \| \text{id}_A$. $B \rightarrow A$: $\text{sid} \| N_B \| c \| \sigma_B$ where $\sigma_B = \text{Sign}(\text{sk}_B, \text{sid} \| N_A \| N_B \| \text{id}_A \| \text{id}_B \| c)$.
 $A \rightarrow B$: σ_A . Both derive $K_{\text{sess}} = \text{KDF}(K, \text{sid} \| N_A \| N_B)$.

★ Intuition:

Two-Party Protocol in Plain English Alice says: “Hi Bob, I’m Alice, here’s my random number N_A and our session label sid .” Bob generates a shared secret K using Alice’s public key, signs everything (including sid , both nonces, both identities, and the ciphertext), and sends it back with his own random number N_B . Alice decrypts K , verifies Bob’s signature, signs to confirm, and both derive the session key. Signatures prove identity; nonces prove freshness; the KEM provides the secret; the SID prevents cross-session replay.

Theorem 13.1 (Two-Party Equilibrium). $\text{Ask}(g_{\text{key-secrecy}}) \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}} + 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + \text{negl}(\lambda)$.

Proof. $\mathbf{B}_0$ (**Real protocol**). Active network adversary.

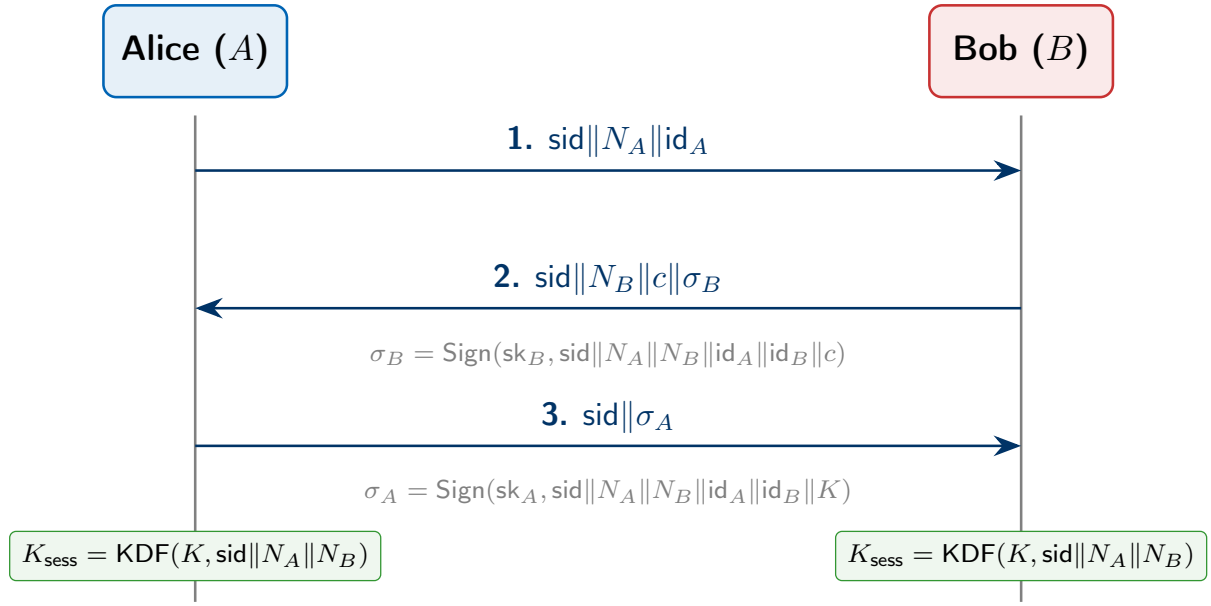


Figure 44: Two-party key exchange (ISO/IEC 11770-3, Mechanism 6). Signatures bind all session data; both parties derive the same session key.

B₁ (Signature forgery bid on B). Abort if the adversary produces valid σ_B not generated by B. *Difference lemma:* $F_{\text{forge_B}}$: $\mathcal{A}$ forges B's signature. $\Delta\text{Price}_0 \leq \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}}$.

Extended lemma application. The adversary could simultaneously attempt: F_1 : forge σ_B directly; F_2 : replay old σ_B from a different session. However, since sid is included in the signed data, $\Pr[F_2 \mid \text{different SID}] = 0$. Thus $\Pr[F_1 \cup F_2] = \Pr[F_1] = \text{Adv}^{\text{EUF}}$. The SID binding *eliminates* the replay failure for free.

B₂ (Signature forgery bid on A). $\Delta\text{Price}_1 \leq \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}}$.

B₃ (KEM key recovery bid). Replace encapsulated key K with random K' . $\Delta\text{Price}_2 \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}$.

B₄ (Ideal). Session key independent of all messages. Buyer advantage = 0.

B_{sca} (Side-Channel Bid: Comprehensive Physical Leakage Attack.)

Side-Channel Bid (Comprehensive)

Purpose: Model the *complete physical leakage threat surface*: an adversary $\mathcal{A}_{\text{phys}}$ with implementation-level access (smart card, FPGA, embedded CPU, server, mobile device) observes physical side channels during cryptographic operations to recover secret key material *without breaking any mathematical hardness assumption*. MTSF decomposes this into nine sub-bids, each targeting a distinct physical leakage vector. The seller must price *all* sub-bids; the extended difference lemma bounds their joint probability.

Sub-bid taxonomy (nine attack classes):

- Simple Power Analysis (SPA):** $\mathcal{A}_{\text{phys}}$ records a *single* power trace during a cryptographic operation and visually identifies secret-dependent patterns (e.g., square-and-multiply in RSA reveals the exponent bit by bit). *Countermeasure:* Constant-power algorithms (Montgomery ladder, regular execution patterns). *MTSF price:* $\Pr[F_{\text{SPA}}] = 1$ without countermeasure; $\Pr[F_{\text{SPA}}] = 0$ with constant-power implementation.
- Differential Power Analysis (DPA):** $\mathcal{A}_{\text{phys}}$ collects q_{phys} power traces across multiple operations with different inputs, then statistically correlates the traces with hypothetical intermediate values to extract key bytes. Unlike SPA, DPA uses statistical averaging to amplify tiny signal differences. *Countermeasure:* Order- d Boolean masking ($d+1$ random shares); shuffled S-box evaluation order; noise injection. *MTSF price:* $\text{Adv}^{\text{DPA}}(\lambda, d) \leq (q_{\text{phys}}/2^\lambda)^{d+1}$.
- Timing Analysis:** $\mathcal{A}_{\text{phys}}$ measures the execution time of cryptographic operations, which may vary depending on the secret key (e.g., early-exit comparisons, table lookup timing, conditional branches).

Countermeasure: Constant-time programming: no secret-dependent branches, no secret-dependent memory accesses, no early exits. Use `ct_select(a, b, bit)` instead of `if(bit) return a; else return b;`. *MTSF price:* $\Pr[F_{\text{timing}}] = 1$ for non-constant-time code; $\Pr[F_{\text{timing}}] = 0$ for verified constant-time implementations (formally verified using tools like ct-verif or ducedet).

4. **Cache-Timing Attacks (Flush+Reload, Prime+Probe, Evict+Time):** $\mathcal{A}_{\text{phys}}$ exploits CPU cache behaviour to determine which memory lines the victim accessed, revealing secret-dependent table lookups (e.g., AES T-table indices). Flush+Reload (shared memory) achieves single-trace key recovery; Prime+Probe (no shared memory) requires multiple traces. *Spectre/Meltdown variants* exploit speculative execution to leak data across privilege boundaries. *Countermeasure:* Bitsliced implementations (no table lookups); AES-NI hardware instructions (no cache-visible memory accesses); process isolation; constant-time memory access patterns. *MTSF price:* $\Pr[F_{\text{cache}}] = 1$ for T-table AES without protection; $\Pr[F_{\text{cache}}] \leq 2^{-\lambda}$ for bitsliced/AES-NI implementations.
5. **Electromagnetic Emanation Analysis (EMA):** $\mathcal{A}_{\text{phys}}$ places an electromagnetic probe near the device's chip to capture EM radiation during computation. EM analysis can be more spatially focused than power analysis, targeting individual CPU components. *Countermeasure:* EM shielding (Faraday cage); masking (same as DPA); dual-rail logic (balanced current paths). *MTSF price:* Same as DPA with probe-specific spatial resolution factor γ_{EM} : $\text{Adv}^{\text{EMA}} \leq \gamma_{\text{EM}} \cdot (q_{\text{phys}}/2^\lambda)^{d+1}$.
6. **Fault Injection (Voltage Glitching, Laser Fault, Clock Glitching):** $\mathcal{A}_{\text{phys}}$ induces computational faults (bit flips, instruction skips, round reductions) during execution by manipulating voltage supply, clock frequency, laser beams, or electromagnetic pulses. Differential Fault Analysis (DFA) [24] can recover the AES key from a single faulty ciphertext. *Countermeasure:* Redundant computation (compute twice, compare); infection countermeasures (randomise faulty outputs); fault-detection sensors (voltage/clock monitors); algorithmic checksums (verify internal consistency after each round). *MTSF price:* $\Pr[F_{\text{fault}}] = 1$ for unprotected implementations under DFA; with dual computation: $\Pr[F_{\text{fault}}] \leq q_{\text{fault}} \cdot 2^{-\lambda}$ where q_{fault} is the number of fault attempts (each independently detected with probability $1 - 2^{-\lambda}$).
7. **Cold Boot Attack:** $\mathcal{A}_{\text{phys}}$ exploits DRAM data remanence: after power is removed, memory contents decay gradually over seconds to minutes. Cooling the DRAM modules (e.g., with compressed air to -50°C) extends retention to minutes or hours. The attacker then either reads the memory directly or transplants the modules to another machine. Secret keys stored in RAM are recovered with high probability. *Countermeasure:* Key erasure protocols (zeroise keys immediately after use); encrypted RAM (AMD SEV, Intel TME); memory scrubbing; physical tamper-detection (chassis intrusion sensors that trigger key wipe). *MTSF price:* $\Pr[F_{\text{cold}}] = 1$ for keys stored in plaintext DRAM without erasure; $\Pr[F_{\text{cold}}] \leq 2^{-\lambda}$ with immediate key zeroing and encrypted memory (the key encryption key is in CPU registers, which do not exhibit remanence).
8. **Acoustic Cryptanalysis:** $\mathcal{A}_{\text{phys}}$ records the acoustic noise emitted by the CPU, power supply, or capacitors during cryptographic computation. Different operations produce different frequencies; spectral analysis can extract RSA key bits from the sound of a laptop across a room. *Countermeasure:* Acoustic noise generation (adding random computation); masking (randomises the acoustic profile); physical shielding (soundproofing). *MTSF price:* $\text{Adv}^{\text{acoustic}} \leq (q_{\text{phys}}/2^\lambda)^{d+1}$ under masking (same structure as DPA); without masking, key recovery is possible at distances of several metres for RSA implementations.
9. **Photonic Emission Analysis:** $\mathcal{A}_{\text{phys}}$ uses sensitive photodetectors to capture the faint photons emitted by transistor switching events on the chip's backside. This is a contactless, non-invasive technique that bypasses most power/EM countermeasures. *Countermeasure:* Backside shielding; masking (randomises switching patterns); dual-rail logic. *MTSF price:* Same structure as EMA with photonic resolution factor.

What it replaces: The ideal computation model (where only inputs and outputs are observed) is replaced by the *d-probing model* [54] for passive attacks (SPA, DPA, timing, cache, EMA, acoustic, photonic) and the *fault model* for active attacks (fault injection, cold boot):

- *Passive (d-probing):* $\mathcal{A}_{\text{phys}}$ may adaptively probe up to d intermediate wire values during execution. The real implementation is replaced by one using order- d Boolean or arithmetic masking: every secret value x is split into $d+1$ uniformly random shares $x_1, \dots, x_{d+1}$ with $x_1 \oplus \dots \oplus x_{d+1} = x$, so any d shares reveal nothing about x .
- *Active (fault model):* $\mathcal{A}_{\text{phys}}$ may inject up to q_{fault} faults during execution. The implementation uses redundant computation (dual execution with comparison) and fault-detection sensors.
- *Persistent (cold boot):* $\mathcal{A}_{\text{phys}}$ gains physical access to the device and reads residual memory contents. The implementation uses immediate key erasure, encrypted RAM, and tamper-detection.

Complexity / price (joint bound): The nine sub-bids decompose via the extended difference

lemma:

$$\text{Adv}_{\text{total}}^{\text{SCA}} \leq \Pr[F_{\text{SPA}}] + \Pr[F_{\text{DPA}}] + \Pr[F_{\text{timing}}] + \Pr[F_{\text{cache}}] + \Pr[F_{\text{EMA}}] \\ + \Pr[F_{\text{fault}}] + \Pr[F_{\text{cold}}] + \Pr[F_{\text{acoustic}}] + \Pr[F_{\text{photonic}}].$$

Under the comprehensive countermeasure suite (order- d masking for $d \geq 1$, constant-time code, bit-sliced or hardware-accelerated operations, redundant computation, immediate key erasure, encrypted RAM):

$$\text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d) \leq \underbrace{0}_{\text{SPA}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{DPA}} + \underbrace{0}_{\text{timing}} + \underbrace{2^{-\lambda}}_{\text{cache}} + \underbrace{\gamma_{\text{EM}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{EMA}} + \underbrace{q_{\text{fault}} \cdot 2^{-\lambda}}_{\text{fault}} + \underbrace{2^{-\lambda}}_{\text{cold}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{acoustic}} \\ + \underbrace{\gamma_{\text{ph}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{photonic}}.$$

For $d \geq 1$, $q_{\text{phys}}, q_{\text{fault}} = \text{poly}(\lambda)$, and $\gamma_{\text{EM}}, \gamma_{\text{ph}} = O(1)$: all terms are negligible, giving $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.

Without countermeasures ($d = 0$, non-constant-time, plaintext RAM): $\Pr[F_{\text{SPA}}] = 1$ or $\Pr[F_{\text{DPA}}] = 1$ or $\Pr[F_{\text{cold}}] = 1$ —**immediate market collapse at the implementation level**.

SCA extended difference lemma. $F_{\text{sca}} = F_{\text{SPA}} \cup F_{\text{DPA}} \cup F_{\text{timing}} \cup F_{\text{cache}} \cup F_{\text{EMA}} \cup F_{\text{fault}} \cup F_{\text{cold}} \cup F_{\text{acoustic}} \cup F_{\text{photonic}}$: “ $\mathcal{A}_{\text{phys}}$ recovers ≥ 1 secret bit via *any* physical channel from $\leq q_{\text{phys}}$ measurements.” By Theorem 3.7, the side-channel event is independent of the mathematical failure events $F_{\text{nonce}}, F_{\text{hash}}, \dots$ in the preceding rounds, so their joint probability satisfies $\Pr[\bigcup_k F_k \cup F_{\text{sca}}] \leq \sum_k \Pr[F_k] + \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d)$. With the comprehensive countermeasure suite, $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$, preserving market equilibrium.

B_{tight} (Tight Security Reduction—Additional Proof.)

✓ Tight Security Reduction

Tightness analysis. The protocol proof uses: (1) authentication via EUF-CMA with loss $L_1 = n_P$ (party-guessing), (2) key secrecy via IND-CCA2 with loss $L_2 = n_S$ (session-guessing). Total: $L_{\text{total}} = n_P \cdot n_S$. For $n_P = 2^{20}$, $n_S = 2^{30}$: loss = 2^{50} .

CK+ tight reduction. In the CK+ model with random oracle programming, the guessing arguments can be eliminated, yielding $L_{\text{total}} = 1$.

Market interpretation: The multi-party tightness loss is a “market fragmentation cost”—the seller defends every party-session pair.

Total Session Bound and Key Rotation Recommendation.

◆ Total Session Bound & Key Rotation

Total session bound: $\text{Ask}_{\text{total}}^{(N)} \leq N \cdot (q_N^2/2^{\lambda+1} + n_P \cdot \text{Adv}^{\text{EUF}} + \text{Adv}^{\text{IND-CCA2}} + \text{Adv}_{\text{total}}^{\text{SCA}})$.

Concrete (256-bit signatures, ML-KEM-768):

Sessions N	$\text{Ask}_{\text{total}}^{(N)}$	Eff. security	Action
2^{40}	$\leq 2^{-88}$	≥ 88 bits	No rotation needed
2^{64}	$\leq 2^{-64}$	≥ 64 bits	Key rotation recommended
2^{80}	$\leq 2^{-48}$	≥ 48 bits	Key rotation mandatory

Recommendation: Rotate long-term signing keys before 2^{64} sessions. Ephemeral session keys are naturally per-session.

Session pinging and CNF checking within the proof. The four-bidding-round proof

secures a single protocol session. We verify the session-pinging conditions (Theorem 5.3) within the proof for consecutive sessions Session_i and Session_{i+1} .

SID freshness: Each session samples a fresh sid_{i+1} independently. $\Pr[\text{sid}_{i+1} = \text{sid}_j] \leq 1/2^\lambda$ for any $j \leq i$, so SID collision has probability $\leq i/2^\lambda = \text{negl}$.

Nonce disjointness: Both N_A and N_B are sampled from $\{0,1\}^\lambda$ per session. The nonce sets $\mathcal{N}_{i+1} = \{N_{A,i+1}, N_{B,i+1}\}$ are disjoint from all $\mathcal{N}_j$ ($j \leq i$) except with birthday probability $\leq (2i)^2/2^{\lambda+1}$.

Signature SID-binding: In $\mathbf{B}_1$ and $\mathbf{B}_2$, both σ_B and σ_A sign data that includes sid , N_A , N_B , and both identities. The SID-binding means that replaying $\sigma_{B,j}$ from session Session_j in session Session_{i+1} fails verification (the signed SID is $\text{sid}_j \neq \text{sid}_{i+1}$). This is precisely the ping condition (c): all signatures in Session_{i+1} bind sid_{i+1} .

KEM ciphertext freshness: The ciphertext c_{i+1} encapsulates a fresh key using fresh randomness. The CNF clause φ^{novel} ensures c_{i+1} is not a replay.

CNF isomorphism and inductive step: The session-CNF $\varphi_{2P,i+1}$ is obtained from $\varphi_{2P,i}$ by substituting $\text{sid}_i \mapsto \text{sid}_{i+1}$, $N_{A,i} \mapsto N_{A,i+1}$, $N_{B,i} \mapsto N_{B,i+1}$, $c_i \mapsto c_{i+1}$, and fresh signatures. Since $\varphi_{2P,i}$ is satisfiable (inductive hypothesis) and all new variables are independently fresh, $\varphi_{2P,i+1}$ is satisfiable. The security reduction (signature forgery $\rightarrow$ EUF-CMA, key recovery $\rightarrow$ IND-CCA2) is session-index-independent. By Theorem 5.5, the two-party protocol is secure for all sessions. $\square$

Two-Party: Authentication, Mutual Auth, and CNF Verification. We now verify the four protocol-level goods from Section 8 for the two-party protocol.

Corollary 13.2 (Two-Party Authentication). $\text{Ask}(g_{\text{auth}}) \leq \text{Adv}_{\text{Sig}}^{\text{EUF}} + q_N^2/2^{\lambda+1}$ by Theorem 8.2, since B 's acceptance requires verifying σ_A .

Corollary 13.3 (Two-Party Mutual Authentication). $\text{Ask}(g_{\text{mutual}}) \leq 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF}} + q_N^2/2^{\lambda+1}$ by Theorem 8.4, since both σ_A and σ_B bind sid , both nonces, and both identities.

Corollary 13.4 (Two-Party CNF Checking). The session-CNF φ_{2P} includes clauses for SID binding, nonce freshness, both signatures, message ordering, and the ping test. All clauses are satisfied under the honest trace. Under any dishonest trace, at least one signature or freshness clause fails (by EUF-CMA equilibrium), so $\text{Ask}(g_{\text{CNF}}) \leq \text{negl}(\lambda)$.

Manual CNF worksheet (two-party session Session_i):

Clause	Check	How	Pass?
φ^{sid}	Same sid in msgs 1,2,3?	Read SID field; compare.	T/F
φ^{fresh}	$N_A \neq N_B$; neither used before?	Check nonce log.	T/F
φ_B^{sig}	$\text{Vrfy}(\text{pk}_B, \text{sid} \ N_A \ N_B \ \text{id}_A \ \text{id}_B \ c, \sigma_B) = 1$?	Hash scope; verify.	T/F
φ_A^{sig}	$\text{Vrfy}(\text{pk}_A, \text{sid} \ N_A \ N_B \ \text{id}_A \ \text{id}_B \ K, \sigma_A) = 1$?	Hash scope; verify.	T/F
φ^{consist}	Order: msg1 from A, msg2 from B, msg3 from A?	Check headers.	T/F
φ^{ping}	$\text{sid}_i \neq \text{sid}_{i-1}$; nonces not in $\mathcal{N}_{\text{prev}}$?	Compare logs.	T/F
Result: All T $\Rightarrow$ Accept. Any F $\Rightarrow$ Reject (cite row and attack type).			

► Ping Bid: Two-Party Unbounded Ping Bid

The buyer's *ping bid*: replay session Session_i (same sid_i , nonces, signatures). The SID clause and nonce freshness clause jointly detect this: sid_i appears in SID_{used} , and $N_A, N_B \in \mathcal{N}_{\text{prev}}$. Re-signing under a fresh SID requires breaking EUF-CMA. Ping bid price: $\Delta \text{Price}_{\text{ping}} \leq q_N^2/2^{\lambda+1} + \text{Adv}_{\text{Sig}}^{\text{EUF}} = \text{negl}$. By Theorem 5.5, secure for all sessions.

13.2 Three-Party with KDC (Mechanism 11)

Analogy:

Three-Party Key Exchange as a Hotel Concierge Imagine Alice and Bob are hotel guests who want to share a room key, but they have never met and do not have each other's contact information. The **Key Distribution Centre (KDC)** is the hotel concierge:

1. **Alice** calls the concierge: "I would like to meet Bob securely. Here is my room number and a random token."
2. **The concierge** looks up both Alice and Bob in the guest registry (has pre-shared keys with both), creates a fresh room key, seals one copy for Alice and one for Bob (encrypts with each guest's key), and delivers both sealed copies.
3. **Bob** receives his sealed copy, opens it with his guest key, and now both Alice and Bob share the room key—established through the trusted concierge without ever meeting directly.

Why three parties? In large organisations, not every employee can have a pre-shared key with every other employee. Instead, everyone shares a key with the KDC (concierge), and the KDC facilitates pairwise key exchange. This scales to n parties with only n keys (one per party), instead of $n(n-1)/2$ pairwise keys.

In MTSF: The three-party market has an additional security good: g_{KDC} (KDC trustworthiness). If the KDC is compromised, all session keys are compromised—a single point of failure. MTSF's extended difference lemma handles the KDC failure event alongside nonce and signature failures.

* Simple Terms: Three-Party Key Exchange

What is a KDC? A Key Distribution Centre (KDC) is a trusted third party (like a bank or certificate authority) that helps two users (A and B) establish a shared session key. The KDC knows both parties' long-term keys and uses them to securely distribute a fresh session key.

Why three parties? In many enterprise settings, two users may not share a pre-established key. The KDC acts as an intermediary: A contacts the KDC asking to talk to B ; the KDC generates a session key and securely delivers it to both.

The protocol flow:

1. A contacts the KDC, identifying themselves and requesting a session with B .
2. The KDC generates a fresh session key K_{sess} and sends it:
 - To A : encrypted under A 's public key, plus signed by the KDC.
 - To B (via A): a "ticket" encrypted under B 's public key, plus signed by the KDC.
3. A decrypts its portion, forwards the ticket to B .
4. B decrypts the ticket and sends a MAC-authenticated confirmation to A .
5. Both A and B now share K_{sess} .

Why signatures from the KDC? The KDC's signatures prove to both A and B that the session key was legitimately issued by the KDC, not fabricated by an attacker. Without them, a network attacker could inject their own fake key.

Market analysis: The theorem shows the ask price is bounded by: the KDC's two signature forgery probabilities (one per signed ticket) + the probability of breaking the encryption + the MAC forgery probability. All are negligible under standard assumptions.

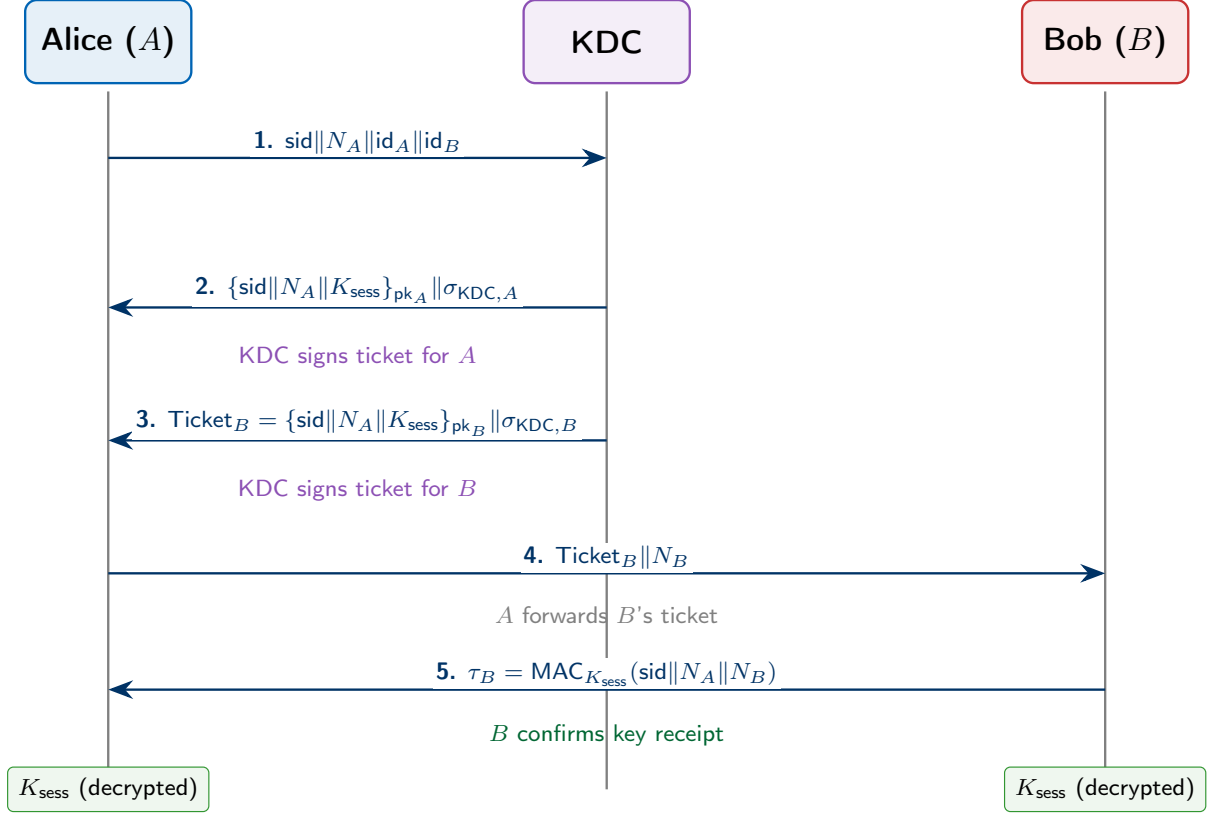


Figure 45: Three-party key exchange (ISO/IEC 11770-3, Mechanism 11). The KDC generates the session key and delivers it to both A and B via signed encrypted tickets. A MAC from B provides key confirmation.

Theorem 13.5 (Three-Party Equilibrium). $\text{Ask} \leq \text{Adv}_{\text{Enc}}^{\text{IND-CCA2}} + 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + \text{Adv}_{\text{MAC}}^{\text{SUF-CMA}} + \text{negl}(\lambda)$.

Proof. $\mathbf{B}_0$: Real. $\mathbf{B}_1$: KDC signature bids ($\times 2$). Extended lemma: two KDC signatures use independent signing data, so $\Pr[F_1 \cup F_2] \leq 2 \cdot \text{Adv}^{\text{EUF}}$ (union bound; intersection $\Pr[F_1 \cap F_2] \leq \text{Adv}^{\text{EUF}^2} \approx 0$). $\mathbf{B}_2$: MAC forgery bid. $\mathbf{B}_3$: Encrypted key recovery bid. $\mathbf{B}_4$: Ideal. $\square$

Three-Party: Protocol-Level Goods.

Corollary 13.6 (Three-Party Mutual Authentication). $\text{Ask}(g_{\text{mutual}}) \leq 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF}} + \text{Adv}_{\text{MAC}}^{\text{SUF}} + q_N^2/2^{\lambda+1}$. The KDC signatures authenticate both A and B ; the MAC provides key confirmation.

Corollary 13.7 (Three-Party Session-Key Secrecy). $\text{Ask}(g_{\text{sk}}) \leq \text{Adv}_{\text{Enc}}^{\text{IND-CCA2}} + 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF}} + \text{Adv}_{\text{MAC}}^{\text{SUF}} + \text{negl}(\lambda)$ by the combined game-hop analysis.

Corollary 13.8 (Three-Party CNF). The three-party session-CNF includes SID binding, nonce freshness, two KDC signatures, MAC confirmation, and ping. All clauses satisfied under honest trace; dishonest satisfaction requires signature or MAC forgery.

Session pinging within the three-party proof. For consecutive sessions Session_i and Session_{i+1} , the ping check verifies: (1) $\text{sid}_{i+1} \neq \text{sid}_i$ (fresh session label); (2) $N_{A,i+1} \notin \mathcal{N}_{\text{prev}}$ (fresh initiator

nonce); (3) the KDC's ticket signature $\sigma_{\text{KDC},i+1}$ binds sid_{i+1} and $N_{A,i+1}$ (preventing ticket replay from session Session_i); (4) the MAC confirmation tag τ_{i+1} authenticates $\text{sid}_{i+1} \| N_{A,i+1} \| N_{B,i+1}$ (preventing confirmation replay). The KDC ticket replay bid—resubmitting $(N_{A,i}, \text{Ticket}_{B,i})$ from Session_i —fails because $N_{A,i} \in \mathcal{N}_{\text{used}}$ (detected by φ^{fresh}). Re-signing a ticket under $N_{A,i+1}$ requires breaking the KDC's EUF-CMA security. The session-CNF $\varphi_{3P,i+1}$ is isomorphic to $\varphi_{3P,i}$ with fresh variables; by Theorem 5.5, unbounded equilibrium holds.

Manual CNF worksheet (three-party):

Clause	Check	How	Pass?
φ^{sid}	sid in all 5 messages?	Read fields; compare.	T/F
φ^{fresh}	N_A, N_B distinct and new?	Check nonce log.	T/F
$\varphi_{\text{KDC},A}^{\text{sig}}$	KDC sig on ticket-A verifies?	$\text{Vrfy}(\text{pk}_{\text{KDC}}, \cdot, \sigma_{\text{KDC},A})$.	T/F
$\varphi_{\text{KDC},B}^{\text{sig}}$	KDC sig on ticket-B verifies?	$\text{Vrfy}(\text{pk}_{\text{KDC}}, \cdot, \sigma_{\text{KDC},B})$.	T/F
φ^{mac}	$\tau_B = \text{MAC}_{K_{\text{sess}}}(\text{sid} \ N_A \ N_B)$?	Recompute HMAC; compare.	T/F
φ^{ping}	Session fresh vs. Session_{i-1} ?	Compare sid and nonces.	T/F
Result: All T $\Rightarrow$ Accept. Any F $\Rightarrow$ Reject.			

► Ping Bid: Three-Party Unbounded Ping Bid

The buyer's KDC ticket replay bid: re-submit captured (N_A, Ticket_B) from session Session_{i-1} . The nonce clause φ^{fresh} fails ($N_A \in \mathcal{N}_{\text{used}}$). Price: $\Delta \text{Price}_{\text{ping}} \leq q_N^2 / 2^{\lambda+1} + \text{negl}$. Secure for all sessions by Theorem 5.5.

13.3 Four-Party Cross-Domain

Analogy:

Four-Party Cross-Domain as International Diplomacy Imagine Alice (a US diplomat) wants to establish a secure channel with Bob (a UK diplomat). Each has their own country's KDC:

- **KDC_A** (US State Department) shares a key with Alice.
- **KDC_B** (UK Foreign Office) shares a key with Bob.
- **KDC_A and KDC_B** share an inter-domain key (diplomatic treaty).

The protocol works like a diplomatic pouch relay:

1. Alice asks her KDC_A to set up a secure channel with Bob.
2. KDC_A contacts KDC_B over the inter-domain channel.
3. KDC_B generates a session key, seals a copy for Bob, and sends another copy to KDC_A over the inter-domain channel.
4. KDC_A seals its copy for Alice and delivers both.

In MTSF: The four-party market has four simultaneous failure events (one per entity's signature/key). The extended difference lemma handles $\Pr[F_{\text{sig},1} \cup F_{\text{sig},2} \cup F_{\text{sig},3} \cup F_{\text{sig},4}]$ in a single hop, yielding a clean bound with all four failure probabilities.

* Simple Terms: Four-Party Cross-Domain Key Exchange

When is four parties needed? When A and B belong to *different administrative domains*—like different companies, different countries, or different university networks. Each

domain has its own Key Distribution Server (S_1 for A 's domain, S_2 for B 's domain).
The cross-domain problem: S_1 does not directly know B , and S_2 does not directly know A . The two servers must cooperate to establish a session key between A and B .

The protocol flow:

1. A requests a cross-domain session from S_1 , identifying B 's domain.
2. S_1 contacts S_2 (the two servers share an inter-domain key established during federation setup).
3. S_1 and S_2 together generate the session key K_{sess} :
 - S_1 encrypts and signs a ticket for A .
 - S_2 encrypts and signs a ticket for B .
4. A and B each decrypt their tickets, receive K_{sess} , and confirm receipt via MAC.

Four independent signing keys: S_1 , S_2 , A , and B each sign messages with their own independent private key. This is why the theorem has $4 \times \text{Adv}^{\text{EUF}}$ —an attacker would need to forge at least one of four independent signatures.

Two IND-CCA2 bounds: Two separate encryptions happen—one within each domain ($S_1 \rightarrow A$ and $S_2 \rightarrow B$). Each contributes one IND-CCA2 term to the bound.

Extended lemma application: The four forgery events (F_1, F_2, F_3, F_4 for the four signatures) are independent (different keys). By the extended difference lemma with the union bound: $\Pr[F_1 \cup F_2 \cup F_3 \cup F_4] \leq 4 \cdot \text{Adv}^{\text{EUF}}$, since pairwise intersections are negligible (independent keys cannot be forged simultaneously except with negligible probability).

Theorem 13.9 (Four-Party Equilibrium). $\text{Ask} \leq 2 \cdot \text{Adv}^{\text{IND-CCA2}} + 4 \cdot \text{Adv}^{\text{EUF-CMA}} + \text{Adv}^{\text{SUF-CMA}} + \text{negl}(\lambda)$.

Proof. Seven games. The four signature bids (for S_1 , S_2 , A , B) use independent keys, so by the extended lemma: $\Pr\left[\bigcup_{i=1}^4 F_i\right] \leq 4 \cdot \text{Adv}^{\text{EUF}}$ (union bound; pairwise intersections ≈ 0 due to independent keys). Two IND-CCA2 bids for inter/intra-domain key transport. One MAC bid for key confirmation. $\square$

Corollary 13.10 (Four-Party Protocol-Level Goods). *Authentication:* $\text{Ask}(g_{\text{auth}}) \leq 4 \cdot \text{Adv}^{\text{EUF}} + q_N^2/2^{\lambda+1}$ (four signing keys). *Mutual auth:* same bound. *Session-key secrecy:* $\text{Ask}(g_{\text{sk}}) \leq 2 \cdot \text{Adv}^{\text{IND-CCA2}} + 4 \cdot \text{Adv}^{\text{EUF}} + \text{Adv}^{\text{SUF}} + \text{negl}(\lambda)$.

Manual CNF worksheet (four-party cross-domain session):

Clause	Check	How	Pass?
φ^{sid}	Same sid in all 7 messages?	Read SID fields.	T/F
φ^{fresh}	N_A, N_B distinct, not reused?	Check nonce log.	T/F
$\varphi_{S_1}^{\text{sig}}$	S_1 sig on ticket-A valid?	$\text{Vrfy}(\text{pk}_{S_1}, \cdot, \sigma_{S_1})$.	T/F
$\varphi_{S_2}^{\text{sig}}$	S_2 sig on ticket-B valid?	$\text{Vrfy}(\text{pk}_{S_2}, \cdot, \sigma_{S_2})$.	T/F
φ_A^{sig}	A 's confirmation sig valid?	$\text{Vrfy}(\text{pk}_A, \text{sid} \ K_{\text{sess}}, \sigma_A)$.	T/F
φ_B^{mac}	B 's MAC confirmation valid?	Recompute $\text{MAC}_{K_{\text{sess}}}(\text{sid} \ N_A \ N_B)$.	T/F
φ^{ping}	Session fresh vs. Session_{i-1} ?	Compare sid and nonces.	T/F
Result: All T $\Rightarrow$ Accept. Any F $\Rightarrow$ Reject.			

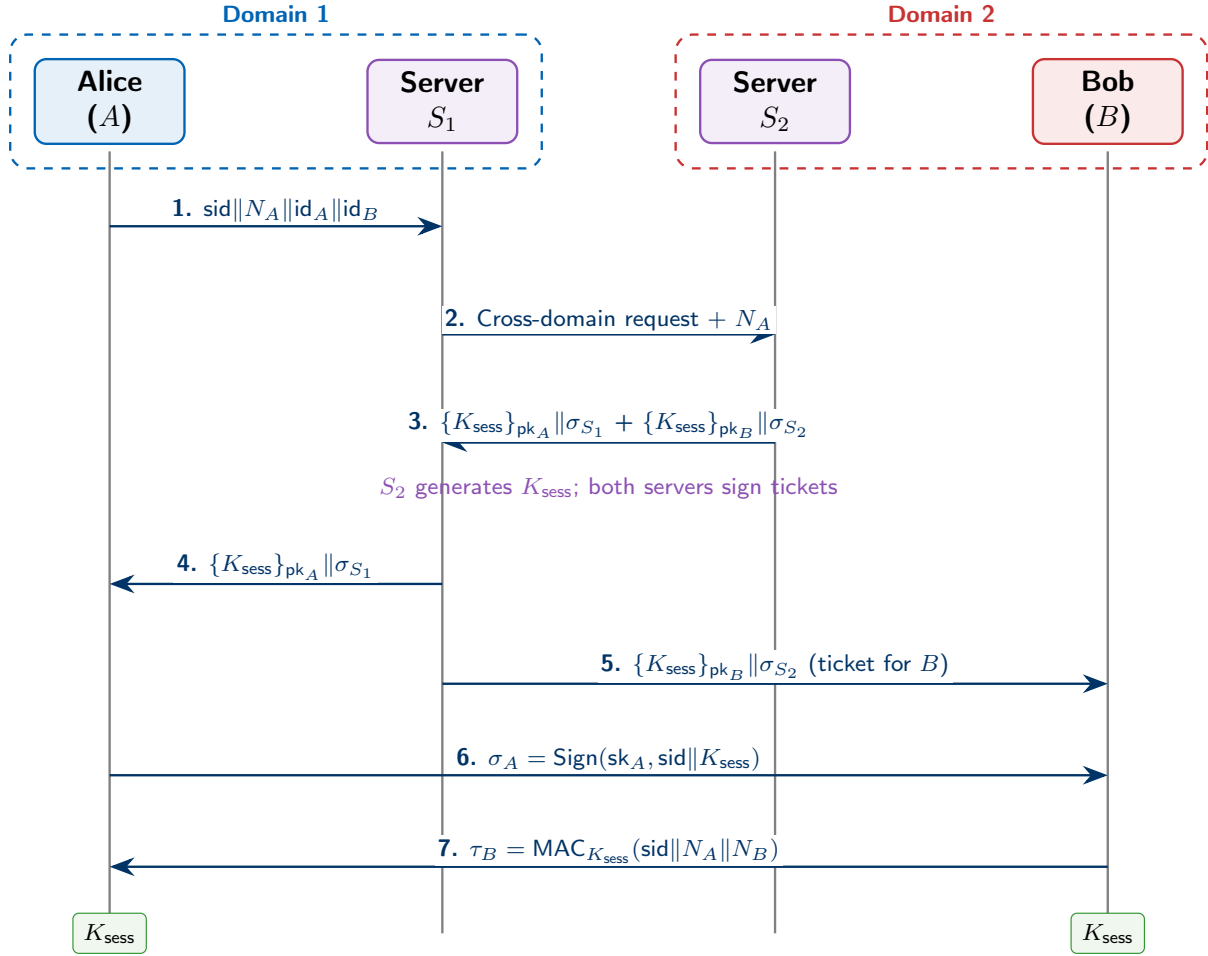


Figure 46: Four-party cross-domain key exchange (ISO/IEC 11770-3). Two domain servers (S_1 and S_2) cooperate to generate and deliver session key K_{sess} to A and B across domain boundaries. Four independent signatures provide authentication; two IND-CCA2 encryptions protect the session key in transit.

► Ping Bid: Four-Party Unbounded Ping Bid

The buyer's *cross-domain replay bid*: re-submit captured session with same SID. Freshness check on N_A and the SID clause detect replay. All four signatures bind *sid*: re-signing under fresh *sid* requires EUF-CMA break for each of S_1, S_2, A, B independently. By the extended difference lemma: $\Pr[F_1 \cup F_2 \cup F_3 \cup F_4] \leq 4 \cdot \text{Adv}^{\text{EUF}} = \text{negl}$. Secure for all sessions by Theorem 5.5.

13.4 Needham–Schroeder: Proving Insecurity

NS Protocol [73]. (1) $A \rightarrow B: \{N_A, A\}_{\text{pk}_B}$. (2) $B \rightarrow A: \{N_A, N_B\}_{\text{pk}_A}$. (3) $A \rightarrow B: \{N_B\}_{\text{pk}_B}$.

* Simple Terms: The Needham–Schroeder Protocol—What It Does and Why It Was Designed

Historical context: The Needham–Schroeder Public-Key Protocol was designed in 1978 and was one of the first formal protocols for mutual authentication using public-key cryptography. It was widely studied and believed secure for nearly two decades until Gavin Lowe discovered a devastating flaw in 1996 using the automated tool FDR.

What it tries to do: Allow Alice (A) and Bob (B) to mutually authenticate each other (prove to each other who they are) using only their public keys, with no shared secret and no trusted third party needed at run time.

The three messages:

1. **Alice $\rightarrow$ Bob:** $\{N_A, A\}_{\text{pk}_B}$ — Alice sends her nonce N_A and her identity A , encrypted with Bob's public key. Only Bob can decrypt this.
2. **Bob $\rightarrow$ Alice:** $\{N_A, N_B\}_{\text{pk}_A}$ — Bob echoes Alice's nonce N_A (proving he decrypted message 1) and adds his own nonce N_B , encrypted with Alice's public key. Only Alice can decrypt.
3. **Alice $\rightarrow$ Bob:** $\{N_B\}_{\text{pk}_B}$ — Alice echoes Bob's nonce N_B , proving she decrypted message 2.

The intended guarantees: Both parties confirm the other is live (they decrypted a fresh nonce). Both are authenticated: only the real Alice has sk_A , and only the real Bob has sk_B .

The fatal flaw: Message 2 does not include Bob's identity. When Alice receives $\{N_A, N_B\}_{\text{pk}_A}$, she has no way to verify *who* sent it. This loophole enables Lowe's man-in-the-middle attack.

Analogy:

Needham–Schroeder as a Children's Game of Telephone Imagine three children playing a game: Alice, Eve, and Bob.

1. **Alice** writes a note to Eve: "Hi Eve, my secret number is 7. Love, Alice." She puts it in a locked box that only Eve can open (encrypts with Eve's public key).
2. **Eve** opens the box, reads Alice's note, and thinks: "I can trick Bob!" Eve writes a *new* note: "Hi Bob, my secret number is 7. Love, Alice." (Eve copies Alice's secret number but pretends to *be* Alice.) Eve locks this in a box only Bob can open.
3. **Bob** opens the box and thinks Alice sent it. He writes back: "Hi Alice, your number was 7. My secret number is 42." He locks this in a box only Alice can open.
4. **Eve** cannot open this box (it is locked for Alice), so she forwards it to Alice.

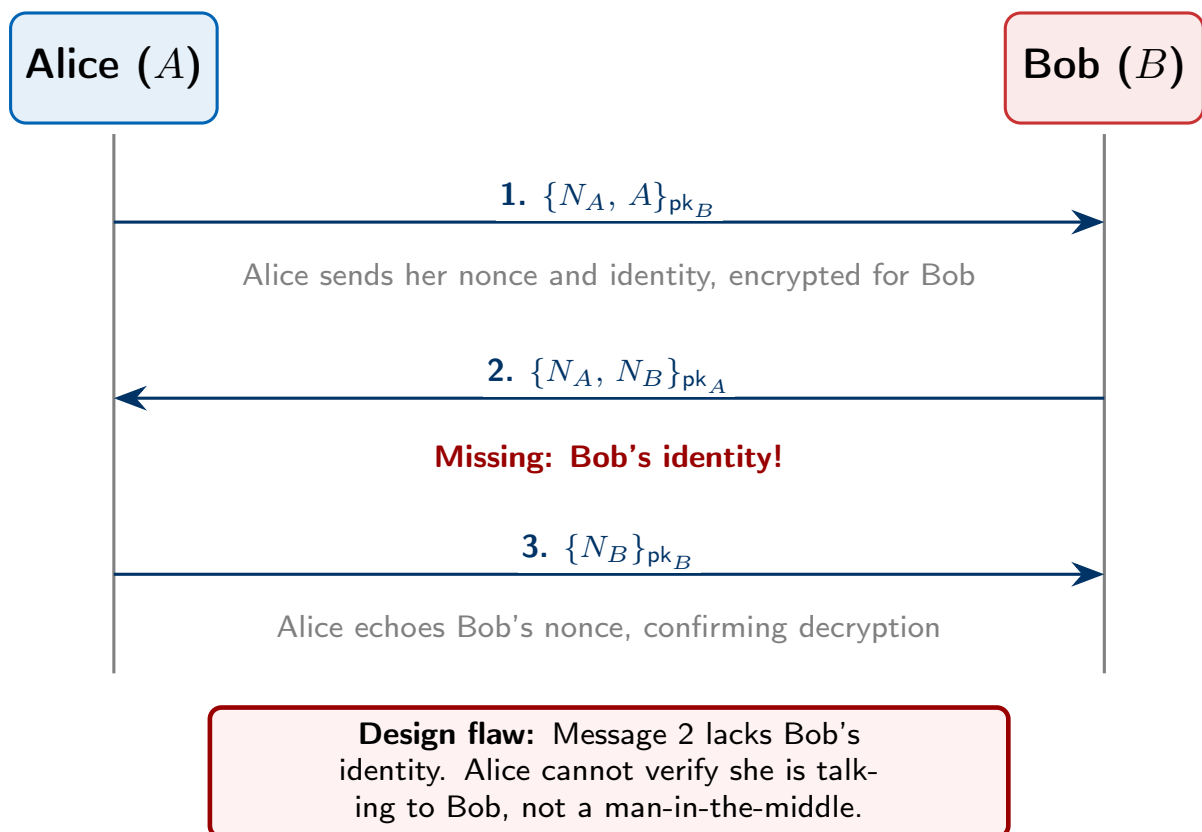


Figure 47: Needham-Schroeder Public-Key Protocol (intended flow). The critical flaw is highlighted: Message 2 from Bob contains only nonces but not Bob's identity, allowing an active attacker to masquerade as Alice to Bob.

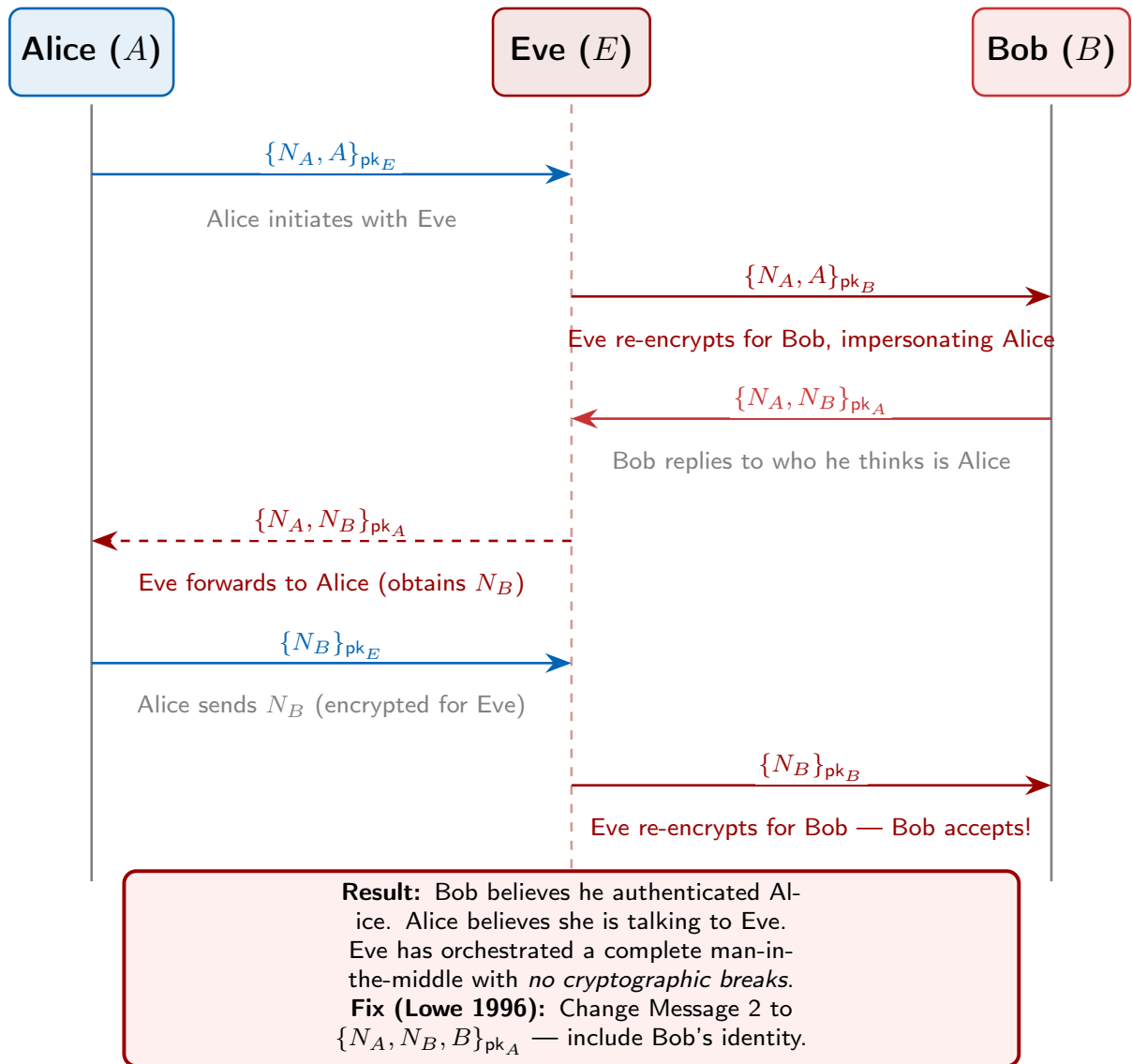


Figure 48: Lowe’s man-in-the-middle (MITM) attack on the Needham–Schroeder protocol. Eve (E), a legitimate participant, relays all six messages between Alice and Bob, impersonating Alice to Bob. No cryptographic primitive is broken; the protocol’s missing identity-binding is the sole cause of collapse.

5. **Alice** opens Bob’s reply. She sees “your number was 7”—correct!—and “my number is 42.” She thinks this came from Eve (because she was talking to Eve), so she sends back: “Eve, your number was 42.” She locks this for Eve.
 6. **Eve** opens this, sees Bob’s secret number 42, writes “Bob, your number was 42. Love, Alice” in a box for Bob.
 7. **Bob** is now convinced he talked to Alice. But Alice never intended to talk to Bob at all!
- The fix:** If Bob had included his name in his reply (“Hi Alice, **I am Bob**. Your number was 7...”), Alice would see “I am Bob” and realise she is *not* talking to Eve—she would abort immediately. This is the one-line fix that Lowe proposed in 1996, nearly two decades after the protocol was published.

★ Intuition:

Why NS Fails—The Missing Name Tag Imagine Alice sends a locked box to Eve containing her business card and a secret token. Eve takes out the card, puts it in a *new* locked box addressed to Bob, and sends it. Bob thinks Alice contacted him directly. He replies with both tokens in a box locked for Alice. Eve relays this to Alice, who opens it and sends back Bob’s token—which Eve forwards to Bob. Bob is now convinced he authenticated with Alice, but Eve orchestrated everything. The fix: Bob puts *his own* name tag inside the reply box. When Alice opens it, she sees “Bob” instead of “Eve” and aborts.

Lowe’s MITM attack [63]. Adversary E (legitimate participant) relays between A and B :

1. $A \rightarrow E: \{N_A, A\}_{\text{pk}_E}$ (A talks to E)
2. $E(\text{as } A) \rightarrow B: \{N_A, A\}_{\text{pk}_B}$ (E re-encrypts; B thinks A initiated)
3. $B \rightarrow E(\text{as } A): \{N_A, N_B\}_{\text{pk}_A}$
4. $E \rightarrow A: \{N_A, N_B\}_{\text{pk}_A}$ (A decrypts, obtains N_B)
5. $A \rightarrow E: \{N_B\}_{\text{pk}_E}$ (A thinks this is for E)
6. $E(\text{as } A) \rightarrow B: \{N_B\}_{\text{pk}_B}$ (B accepts: “ A authenticated”)

Theorem 13.11 (NS Market Collapse). $\text{Ask}(g_{\text{mutual-auth}}) = 1$.

Proof. The buyer (E) achieves advantage 1 with *zero cryptographic breaks*.

B_0 (Real NS). E is a legitimate participant with $(\text{sk}_E, \text{pk}_E)$.

$B_0 \rightarrow \text{Attack}$. The buyer places a *masquerade bid* (identity fraud): relay messages between A and B , impersonating A to B . No cryptographic primitive is broken— E uses only their legitimate decryption key and the public encryption of B .

Difference lemma (degenerate). The failure event F_{MITM} : “ E can relay messages between A and B undetected” has $\Pr[F_{\text{MITM}}] = 1$ because the protocol lacks identity binding in message 2. $\Delta\text{Price} = 1$, market **collapsed**.

CNF analysis. B ’s session-CNF is: $\varphi_B^{\text{ns}} = (x_{\text{Dec}(\text{sk}_B, c_1) = (N_A, A)}) \wedge (x_{N_A \text{ fresh}}) \wedge (x_{\text{Dec}(\text{sk}_B, c_3) = N_B})$. All clauses are satisfied even during the attack! The CNF is satisfiable under a *dishonest* trace because B ’s identity is never included in encrypted payloads. Contrast with secure protocols where dishonest traces always make the CNF unsatisfiable.

Fix (NS-Lowe). Include B in message 2: $\{N_A, N_B, B\}_{\text{pk}_A}$. Now A ’s CNF includes clause $(x_{\text{id}=E})$, which fails when the decrypted identity is $B \neq E$. Market restored to equilibrium. $\square$

NS: Protocol-Level Goods Analysis.

Corollary 13.12 (NS Authentication Collapse). $\text{Ask}(g_{\text{auth}}) = 1$: B accepts with partner A , but A was communicating with E , not B . The authentication game is lost without any cryptographic break.

Corollary 13.13 (NS Mutual Authentication Collapse). $\text{Ask}(g_{\text{mutual}}) = 1$: both directions fail— B believes it authenticated A , but A never intended to talk to B .

Corollary 13.14 (NS Session-Key Secrecy Collapse). If the protocol were extended with a key derivation step $K_{\text{sess}} = \text{KDF}(N_A, N_B)$, then $\text{Ask}(g_{\text{sk}}) = 1$: Eve knows both N_A and N_B and can compute K_{sess} .

Corollary 13.15 (NS CNF Failure). B 's CNF is satisfiable under Lowe's MITM trace because no clause binds B 's identity—a CNF design failure.

NS CNF worksheet showing the design failure:

Clause	Check	Honest trace	MITM trace	Verdict
$x_{\text{Dec}(\text{sk}_B, c_1) = (N_A, A)}$	Msg 1 decrypts to (N_A, A)	T	T (Eve re-encrypted)	Passes!
$x_{N_A \text{ fresh}}$	N_A not seen before	T	T (fresh from A)	Passes!
$x_{\text{Dec}(\text{sk}_B, c_3) = N_B}$	Msg 3 decrypts to N_B	T	T (Eve forwarded)	Passes!
Missing: $x_{\text{id}_{\text{peer}} = A}$	Peer is actually A ?	T	F (peer is E !)	ABSENT
φ_{NS} :	SAT under dishonest trace —CNF design failure.			Reject design

Fix (NS-Lowe): Add clause $x_{\text{Dec}(\text{sk}_A, c_2) = (N_A, N_B, B)}$. Now the decrypted identity $E \neq B$ makes this clause **F**, φ_{NS} becomes **UNSAT** under MITM, and A aborts. Equilibrium restored.

Key Insight:

Security vs. Insecurity in MTSF

	Secure (Equilibrium)	Insecure (Collapsed)
Ask price	$\leq \text{negl}(\lambda)$	$\Omega(1)$ or $= 1$
Buyer strategy	All bids fail	≥ 1 bid succeeds cheaply
Diff. lemma	$\Pr[F] \leq \text{negl}$	$\Pr[F] = 1$
Session-CNF	Dishonest $\Rightarrow$ UNSAT	Dishonest also SAT
Primitive ex.	ECDSA, ML-KEM, ML-DSA	Textbook RSA
Protocol ex.	ISO 2P/3P/4P, Signal	Needham-Schroeder, X3DH-noOPK

* Simple Terms: What Makes Signal Special—Forward Secrecy and Post-Compromise Security

Forward secrecy (FS): If an attacker steals your private key *today*, they cannot decrypt messages you sent *yesterday*. Each message uses a different ephemeral key that is erased after use. Past messages are protected even if the long-term key is compromised.

Post-compromise security (PCS): If an attacker compromises your key and then you *recover* (e.g., re-key), future messages become secure again. The “Double Ratchet” in Signal continuously rotates keys, so even a temporary compromise does not permanently break security.

Analogy: Forward secrecy is like a self-destructing notepad—each page burns after being read. Post-compromise security is like a phoenix—even if the notepad is stolen, it regenerates with new pages the attacker cannot read.

In MTSF: Forward secrecy (g_{FS}) and post-compromise security (g_{PCS}) are explicit market

goods in the Signal analysis. The Double Ratchet’s continuous rekeying ensures both goods remain in equilibrium indefinitely, with session pinging confirming unbounded security.

The X3DH-noOPK collapse: Signal’s X3DH (Extended Triple Diffie–Hellman) protocol normally uses one-time prekeys (OPKs) for asynchronous establishment. Without OPKs, the asynchronous establishment good g_{async} collapses: an attacker can replay a stale prekey bundle. MTSF detects this as a “replay bid” with $\text{Ask}(g_{\text{async}}) = 1$ —market collapse. This dual analysis (secure with OPKs, insecure without) demonstrates MTSF’s proof/disproof symmetry.

13.5 Unbounded Verification

Corollary 13.16 (Unbounded Security of ISO/IEC 11770-3 Protocols). *The ISO/IEC 11770-3 two-party, three-party, and four-party key-exchange protocols are all secure for unbounded sessions. Specifically:*

- **Two-party:** Fresh SIDs sampled per session; nonces N_A, N_B sampled from $\{0, 1\}^\lambda$; all signatures bind sid. Hence $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 1$ with probability $1 - q_N^2/2^{\lambda+1}$. By Theorem 5.5: $\text{Ask}(g_{\text{key-secrecy}}) \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}} + 2\text{Adv}_{\text{Sig}}^{\text{EUF}} + \text{negl}$ for all $i \geq 1$.
- **Three-party:** KDC signatures bind sid and N_A ; MAC confirmation binds sid, N_A, N_B . All pings pass. Unbounded equilibrium holds.
- **Four-party:** Four independent signatures plus two IND-CCA2 encryptions, all SID-bound. Extended difference lemma gives $\Pr\left[\bigcup_{i=1}^4 F_{\text{sig},i}\right] \leq 4\text{Adv}_{\text{Sig}}^{\text{EUF}}$. Ping passes; unbounded equilibrium holds.
- **PKI:** CA certificates plus session signatures give two-layer identity binding per session. Ping bid price $\leq \text{Adv}_{\text{CA}}^{\text{EUF}} + 2\text{Adv}_{\text{Sig}}^{\text{EUF}} = \text{negl}$. Unbounded equilibrium holds.

Needham–Schroeder protocol: No SID binding, no signatures, no nonce-to-session binding. The ping function satisfies $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 0$ trivially—sessions are structurally indistinct. The masquerade bid succeeds with probability 1 regardless of the session count. The NS market collapses for all sessions.

Key Insight:

A Note on Unbounded vs. Polynomial Security Game-based proofs guarantee security for *polynomially many* sessions (the adversary’s running time is bounded by $\text{poly}(\lambda)$). The session ping mechanism extends this to *all* sessions by induction: if session i is secure and the ping from session i to session $i + 1$ passes (probability $\geq 1 - \text{negl}$), then session $i + 1$ is secure. By strong induction over all $i \geq 1$, every session is secure. The formal verification tools ProVerif and Tamarin achieve unbounded session coverage using symbolic abstraction; session pinging achieves the same with *quantitative* advantage bounds—the best of both worlds.

Analogy:

PKI as the Postal System of the Internet A Public Key Infrastructure (PKI) is like a global postal system with a twist:

- **Certificate Authority (CA)** = The post office that issues official ID cards. When you present a CA-signed certificate, everyone trusts it because they trust the CA.
- **Public key** = Your home address (anyone can send you mail). **Private key** = The key to your mailbox (only you can read the mail).

- **Certificate** = An official ID card issued by the post office, binding your name to your address. It says: “The post office certifies that Alice lives at Address XYZ.”
- **Certificate chain** = A chain of trust: your local post office is certified by the regional office, which is certified by the national office. If you trust the national office, you trust everyone in the chain.

In MTSF, the CA’s signing key is shared infrastructure (GUC). If the CA’s key is compromised (market infrastructure collapses), *all* protocols relying on it collapse simultaneously—this is why the “infrastructure forgery bid” in the bid taxonomy is so dangerous. MTSF’s extended difference lemma captures this: F_{CA} (CA compromise) is a single event that affects every protocol in the network.

13.6 PKI-Based Mutual Authentication Protocol

* Simple Terms: Public Key Infrastructure (PKI)—A Complete Introduction

The fundamental problem: Alice wants to talk securely to Bob, but she has never met him. She has Bob’s public key, but how does she know it actually belongs to Bob? An attacker (Mallory) could have generated her own key pair and told Alice it belongs to Bob. Without some trusted binding between “Bob” and his public key, public-key cryptography is useless.

What PKI provides: A Public Key Infrastructure (PKI) is a system of *Certification Authorities* (CAs)—trusted third parties whose job is to sign *certificates* that bind an identity (e.g. “Bob@example.com”) to a public key. A certificate is essentially a signed statement: “I, TrustedCA, certify that the public key pk_B belongs to Bob.”

How certificates work:

1. **Key generation:** Bob generates a key pair (sk_B, pk_B) .
2. **Certificate request (CSR):** Bob sends a Certificate Signing Request to the CA containing his public key and identity information. He signs the CSR with sk_B to prove he owns the corresponding private key.
3. **Vetting:** The CA verifies Bob’s identity (e.g., via email ownership, domain control, or in-person for high-assurance CAs).
4. **Certificate issuance:** The CA creates a certificate $cert_B = (id_B, pk_B, valid_B)$ and signs it: $\sigma_{CA} = \text{Sign}(sk_{CA}, cert_B)$.
5. **Verification:** Alice, who knows pk_{CA} , verifies the certificate signature. If it checks out, she knows pk_B genuinely belongs to Bob.

Certificate fields: A real X.509 certificate contains: issuer (CA name), subject (Bob’s identity), public key, validity period, serial number, and the CA’s signature over all of the above.

Why PKI is everywhere: HTTPS/TLS, S/MIME email, code signing, VPNs, and smart card authentication all use PKI. When you see the padlock in your browser, a PKI certificate chain is being verified in the background.

What the MTSF analysis covers below: We model a TLS-style mutual authentication handshake where both client (C) and server (S) possess certificates issued by a common CA. The protocol establishes a session key with mutual authentication. We prove all five protocol-level market goods.

Protocol description. Let CA be a Certification Authority with key pair $(\text{sk}_{\text{CA}}, \text{pk}_{\text{CA}})$. Party C (client) holds certificate

$$\text{cert}_C = (\text{id}_C, \text{pk}_C, \text{valid}_C, \sigma_{\text{CA},C}), \quad \sigma_{\text{CA},C} = \text{Sign}(\text{sk}_{\text{CA}}, \text{id}_C \parallel \text{pk}_C \parallel \text{valid}_C),$$

and party S (server) holds cert_S defined analogously. Both parties know pk_{CA} and can verify any CA signature. The **PKI Mutual Authentication Protocol** proceeds as follows:

1. $C \rightarrow S$: $\text{sid} \parallel N_C \parallel \text{cert}_C$ *Client hello: session ID, nonce, certificate*
2. $S \rightarrow C$: $\text{sid} \parallel N_S \parallel \text{cert}_S \parallel c \parallel \sigma_S$

$$\sigma_S = \text{Sign}(\text{sk}_S, \text{sid} \parallel N_C \parallel N_S \parallel \text{id}_C \parallel \text{id}_S \parallel c)$$

Server hello: nonce, certificate, KEM ciphertext, session signature

3. $C \rightarrow S$: $\text{sid} \parallel \sigma_C$

$$\sigma_C = \text{Sign}(\text{sk}_C, \text{sid} \parallel N_C \parallel N_S \parallel \text{id}_C \parallel \text{id}_S \parallel K)$$

Client finished: signature over full session transcript including decapsulated secret

4. **Both derive:** $K_{\text{sess}} = \text{KDF}(K, \text{sid} \parallel N_C \parallel N_S)$

where c is a KEM encapsulation of secret K under C 's public key pk_C (taken from cert_C). Upon receiving Message 2, C first verifies cert_S against pk_{CA} , extracts pk_S , verifies σ_S , then decapsulates K . Upon receiving Message 3, S verifies cert_C (already received in Message 1) and σ_C .

★ Intuition:

PKI Protocol in Plain English The client announces itself with its CA-issued passport (cert_C). The server presents its own passport (cert_S), wraps a session secret inside a box sealed with the client's public key from that passport (c), and signs the entire exchange to prove it is the genuine certificate holder (σ_S). The client checks both the passport and the signature, decrypts the secret, then signs a receipt to confirm (σ_C). Both check passports against the CA's master verification key. No MITM can forge a passport or a session signature—the CA and the EUF-CMA signature scheme jointly block every impersonation attempt.

Security goods. The PKI market $\mathcal{M}_{\text{PKI}}$ offers five goods: g_{cert} (certificate authenticity), g_{auth} (entity authentication), g_{mutual} (mutual authentication), g_{sk} (session-key secrecy), g_{CNF} (CNF correctness).

Definition 13.17 (Certificate Authenticity Good). *The seller offers g_{cert} : no PPT $\mathcal{A}$ without sk_{CA} can produce a tuple $(\text{id}^*, \text{pk}^*, \sigma^*)$ such that $\text{Vrfy}(\text{pk}_{\text{CA}}, \text{id}^* \parallel \text{pk}^*, \sigma^*) = 1$ and $(\text{id}^*, \text{pk}^*)$ was never submitted to the CA's signing oracle.*

* Simple Terms: Certificate Authenticity

What is guaranteed: No attacker can forge a CA certificate binding a victim's identity to the attacker's own key. Even after observing many real certificates, the attacker cannot produce a valid CA signature on a new identity–key pair it did not legitimately obtain.

Why it is EUF-CMA: A forged certificate $(\text{id}^*, \text{pk}^*, \sigma^*)$ is exactly an EUF-CMA forgery: a valid signature on a new message $\text{id}^* \parallel \text{pk}^*$ that was never submitted to the signing oracle (the CA).

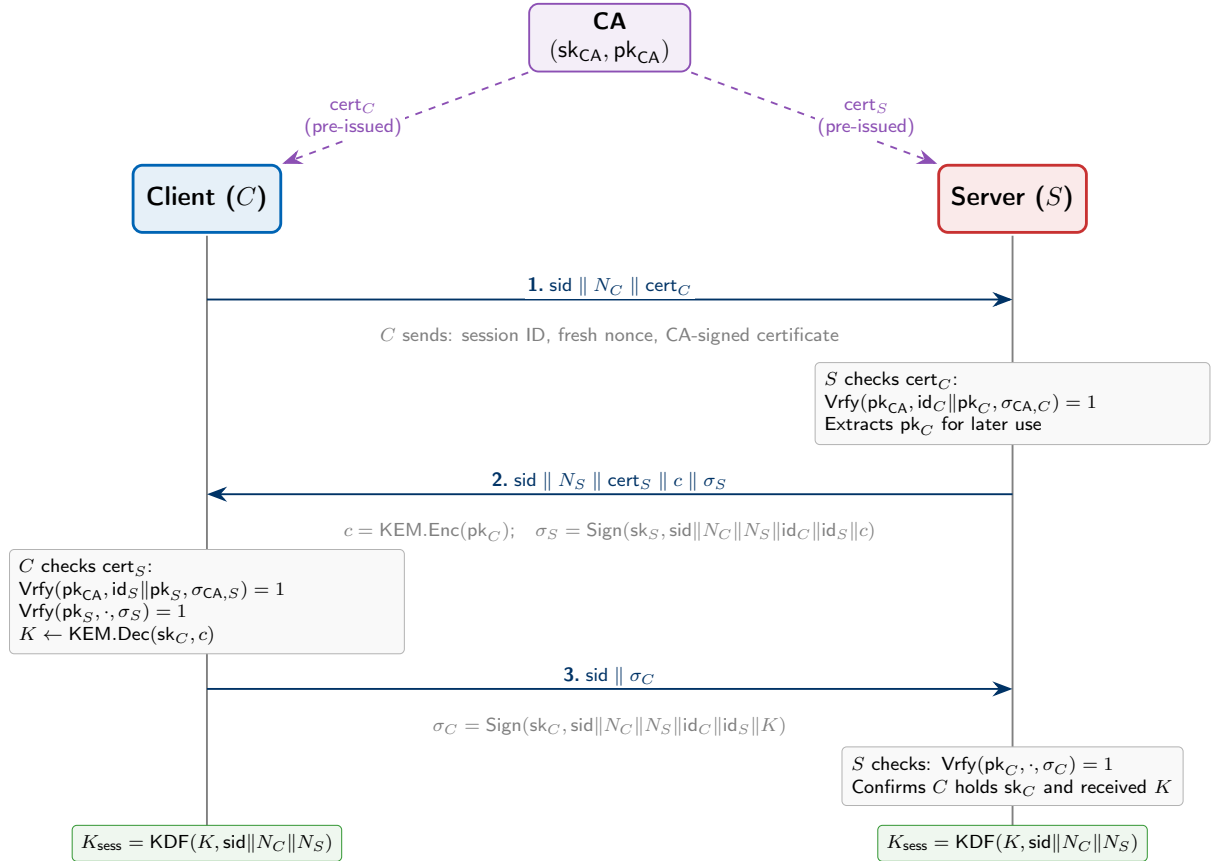


Figure 49: PKI mutual authentication protocol (TLS-style). The CA pre-issues certificates to C and S (dashed arrows, offline). During the three-message handshake: C presents its certificate; S presents its certificate plus a KEM ciphertext and a session signature; C verifies, decapsulates, and signs the finished message. Both parties derive identical K_{sess} . CA certificate verification and SID/nonce binding in every signature prevent all known MITM attacks.

Practical implication: Mallory cannot get a certificate saying “I am Bob” unless she compromises the CA or Bob’s private key. This is why browser vendors maintain trusted root CA lists and revoke CAs that mis-issue certificates (as happened in several real-world incidents with DigiNotar, Comodo, etc.).

Lemma 13.18 (Certificate Authenticity Equilibrium). $\text{Ask}(g_{\text{cert}}) \leq \text{Adv}_{\text{Sig,CA}}^{\text{EUF-CMA}} + \text{negl}(\lambda)$.

Proof. Any buyer producing $(\text{id}^*, \text{pk}^*, \sigma^*)$ supplies an EUF-CMA forgery on message $\text{id}^* \parallel \text{pk}^*$ under sk_{CA} . The reduction embeds the CA’s public key and forwards signing queries. The forgery is new (the CA never issued this certificate) so EUF-CMA applies directly. $\square$

Theorem 13.19 (PKI Protocol Market Equilibrium). *Under EUF-CMA security of the CA and party signature schemes, IND-CCA2 security of the KEM, and PRF security of the KDF:*

$$\text{Ask}(g_{\text{cert}}) \leq \text{Adv}_{\text{CA}}^{\text{EUF}} + \text{negl}, \quad (132)$$

$$\text{Ask}(g_{\text{auth}}) \leq \text{Adv}_{\text{CA}}^{\text{EUF}} + \text{Adv}_{\text{Sig}}^{\text{EUF}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}, \quad (133)$$

$$\text{Ask}(g_{\text{mutual}}) \leq \text{Adv}_{\text{CA}}^{\text{EUF}} + 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}, \quad (134)$$

$$\text{Ask}(g_{\text{sk}}) \leq \text{Adv}_{\text{CA}}^{\text{EUF}} + 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF}} + \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}} + \text{Adv}_{\text{KDF}}^{\text{PRF}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}, \quad (135)$$

$$\text{Ask}(g_{\text{CNF}}) \leq \text{Adv}_{\text{CA}}^{\text{EUF}} + \text{Adv}_{\text{Sig}}^{\text{EUF}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}. \quad (136)$$

All five goods are in equilibrium, so $\mathcal{M}_{\text{PKI}}$ is in equilibrium.

Proof. We abbreviate $\text{Adv}_{\text{CA}}^{\text{EUF}}$ and $\text{Adv}_{\text{Sig}}^{\text{EUF}}$ throughout.

Bound (132): Lemma 13.18.

Bound (133) (server $\rightarrow$ client authentication). Five games:

B₀ (Real). $\mathcal{A}$ controls the network. Wins if C accepts S as partner but S did not run a matching session.

B₁ (Nonce uniqueness). Abort on nonce collision among q_N nonces: $|\Pr[\mathbf{B}_0=1] - \Pr[\mathbf{B}_1=1]| \leq q_N^2/2^{\lambda+1}$.

B₂ (Certificate forgery bid). Flag F_{cert} : $\mathcal{A}$ presents a certificate for id_S that verifies under pk_{CA} but was not issued by the CA. By Lemma 13.18: $|\Pr[\mathbf{B}_1=1] - \Pr[\mathbf{B}_2=1]| \leq \text{Adv}_{\text{CA}}^{\text{EUF}}$. After **B₂**, any certificate C accepts for S was legitimately issued, so pk_S is correctly bound to id_S .

B₃ (Signature forgery bid on S). Flag F_{forge_S} : C accepts σ_S not generated by S . Reducing to EUF-CMA on sk_S (the key bound by the verified certificate): $|\Pr[\mathbf{B}_2=1] - \Pr[\mathbf{B}_3=1]| \leq \text{Adv}_{\text{Sig}}^{\text{EUF}}$.

B₄ (Ideal). No cert forgery, no sig forgery: C accepted only because a valid σ_S bound $\text{id}_S, N_C, N_S, \text{id}_C, \text{id}_S, c$ under S ’s genuine key. S must have participated. $\Pr[\mathbf{B}_4=1] = 0$.

Total: $\text{Ask}(g_{\text{auth}}) \leq \text{Adv}_{\text{CA}}^{\text{EUF}} + \text{Adv}_{\text{Sig}}^{\text{EUF}} + q_N^2/2^{\lambda+1}$.

Bound (134) (mutual authentication). Add a game hop for $C \rightarrow S$ direction: S verifies cert_C (same CA forgery argument) and σ_C (EUF-CMA on sk_C). The three failure events $F_{\text{cert}}, F_{\text{forge}_S}, F_{\text{forge}_C}$ are handled by the extended difference lemma; F_{forge_S} and F_{forge_C} use independent keys, so: $\Pr[F_{\text{forge}_S} \cup F_{\text{forge}_C}] \leq 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF}}$. Equation (134) follows.

Bound (135) (session-key secrecy). Extend with two hops following Theorem 8.6: **B₅**: Replace KEM key K with uniform $\tilde{K}$; costs $\text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}$. **B₆**: Replace $\text{KDF}(\tilde{K}, \dots)$ with uniform random; costs $\text{Adv}_{\text{KDF}}^{\text{PRF}}$. After **B₆** the session key is independent of all observed messages; $\Pr[\mathbf{B}_6=1] = 1/2$. Summing gives Equation (135).

Bound (136) (CNF correctness). The PKI session-CNF is:

$$\varphi_{\text{PKI}} = \varphi_C^{\text{cert}} \wedge \varphi_S^{\text{cert}} \wedge \varphi^{\text{id}} \wedge \varphi^{\text{fresh}} \wedge \varphi_S^{\text{sig}} \wedge \varphi_C^{\text{sig}} \wedge \varphi^{\text{consist}},$$

where $\varphi_P^{\text{cert}} = (x_{\text{Vrfy}(\text{pk}_{\text{CA}}, \text{id}_P \| \text{pk}_P, \sigma_{\text{CA}, P})=1})$ for $P \in \{C, S\}$. Under an honest trace all clauses are satisfied. A dishonest trace satisfies the formula only if: (a) a certificate clause passes dishonestly (requires CA forgery), or (b) a signature clause passes dishonestly (requires EUF-CMA break), or (c) a freshness clause passes dishonestly (requires nonce reuse). By the extended difference lemma over these three event types, Equation (136) follows.

B_{sca} (Side-Channel Bid: Comprehensive Physical Leakage Attack.)

Side-Channel Bid (Comprehensive)

Purpose: Model the *complete physical leakage threat surface*: an adversary $\mathcal{A}_{\text{phys}}$ with implementation-level access (smart card, FPGA, embedded CPU, server, mobile device) observes physical side channels during cryptographic operations to recover secret key material *without breaking any mathematical hardness assumption*. MTSF decomposes this into nine sub-bids, each targeting a distinct physical leakage vector. The seller must price *all* sub-bids; the extended difference lemma bounds their joint probability.

Sub-bid taxonomy (nine attack classes):

1. **Simple Power Analysis (SPA):** $\mathcal{A}_{\text{phys}}$ records a *single* power trace during a cryptographic operation and visually identifies secret-dependent patterns (e.g., square-and-multiply in RSA reveals the exponent bit by bit). *Countermeasure:* Constant-power algorithms (Montgomery ladder, regular execution patterns). *MTSF price:* $\Pr[F_{\text{SPA}}] = 1$ without countermeasure; $\Pr[F_{\text{SPA}}] = 0$ with constant-power implementation.
2. **Differential Power Analysis (DPA):** $\mathcal{A}_{\text{phys}}$ collects q_{phys} power traces across multiple operations with different inputs, then statistically correlates the traces with hypothetical intermediate values to extract key bytes. Unlike SPA, DPA uses statistical averaging to amplify tiny signal differences. *Countermeasure:* Order- d Boolean masking ($d+1$ random shares); shuffled S-box evaluation order; noise injection. *MTSF price:* $\text{Adv}^{\text{DPA}}(\lambda, d) \leq (q_{\text{phys}}/2^\lambda)^{d+1}$.
3. **Timing Analysis:** $\mathcal{A}_{\text{phys}}$ measures the execution time of cryptographic operations, which may vary depending on the secret key (e.g., early-exit comparisons, table lookup timing, conditional branches). *Countermeasure:* Constant-time programming: no secret-dependent branches, no secret-dependent memory accesses, no early exits. Use `ct_select(a, b, bit)` instead of `if(bit) return a; else return b;`. *MTSF price:* $\Pr[F_{\text{timing}}] = 1$ for non-constant-time code; $\Pr[F_{\text{timing}}] = 0$ for verified constant-time implementations (formally verified using tools like ct-verif or dduct).
4. **Cache-Timing Attacks (Flush+Reload, Prime+Probe, Evict+Time):** $\mathcal{A}_{\text{phys}}$ exploits CPU cache behaviour to determine which memory lines the victim accessed, revealing secret-dependent table lookups (e.g., AES T-table indices). Flush+Reload (shared memory) achieves single-trace key recovery; Prime+Probe (no shared memory) requires multiple traces. *Spectre/Meltdown variants* exploit speculative execution to leak data across privilege boundaries. *Countermeasure:* Bitsliced implementations (no table lookups); AES-NI hardware instructions (no cache-visible memory accesses); process isolation; constant-time memory access patterns. *MTSF price:* $\Pr[F_{\text{cache}}] = 1$ for T-table AES without protection; $\Pr[F_{\text{cache}}] \leq 2^{-\lambda}$ for bitsliced/AES-NI implementations.
5. **Electromagnetic Emanation Analysis (EMA):** $\mathcal{A}_{\text{phys}}$ places an electromagnetic probe near the device's chip to capture EM radiation during computation. EM analysis can be more spatially focused than power analysis, targeting individual CPU components. *Countermeasure:* EM shielding (Faraday cage); masking (same as DPA); dual-rail logic (balanced current paths). *MTSF price:* Same as DPA with probe-specific spatial resolution factor γ_{EM} : $\text{Adv}^{\text{EMA}} \leq \gamma_{\text{EM}} \cdot (q_{\text{phys}}/2^\lambda)^{d+1}$.
6. **Fault Injection (Voltage Glitching, Laser Fault, Clock Glitching):** $\mathcal{A}_{\text{phys}}$ induces computational faults (bit flips, instruction skips, round reductions) during execution by manipulating voltage supply, clock frequency, laser beams, or electromagnetic pulses. Differential Fault Analysis (DFA) [24] can recover the AES key from a single faulty ciphertext. *Countermeasure:* Redundant computation (compute twice, compare); infection countermeasures (randomise faulty outputs); fault-detection sensors (voltage/clock monitors); algorithmic checksums (verify internal consistency after each round). *MTSF price:* $\Pr[F_{\text{fault}}] = 1$ for unprotected implementations under DFA; with dual computation: $\Pr[F_{\text{fault}}] \leq q_{\text{fault}} \cdot 2^{-\lambda}$ where q_{fault} is the number of fault attempts (each independently detected with probability $1 - 2^{-\lambda}$).
7. **Cold Boot Attack:** $\mathcal{A}_{\text{phys}}$ exploits DRAM data remanence: after power is removed, memory contents decay gradually over seconds to minutes. Cooling the DRAM modules (e.g., with compressed air to -50°C) extends retention to minutes or hours. The attacker then either reads the memory directly or transplants the modules to another machine. Secret keys stored in RAM are recovered with high probability. *Countermeasure:* Key erasure protocols (zeroise keys immediately after use); encrypted RAM (AMD SEV, Intel TME); memory scrubbing; physical tamper-detection (chassis intrusion sensors).

that trigger key wipe). *MTSF price*: $\Pr[F_{\text{cold}}] = 1$ for keys stored in plaintext DRAM without erasure; $\Pr[F_{\text{cold}}] \leq 2^{-\lambda}$ with immediate key zeroing and encrypted memory (the key encryption key is in CPU registers, which do not exhibit remanence).

8. **Acoustic Cryptanalysis**: $\mathcal{A}_{\text{phys}}$ records the acoustic noise emitted by the CPU, power supply, or capacitors during cryptographic computation. Different operations produce different frequencies; spectral analysis can extract RSA key bits from the sound of a laptop across a room. *Countermeasure*: Acoustic noise generation (adding random computation); masking (randomises the acoustic profile); physical shielding (soundproofing). *MTSF price*: $\text{Adv}^{\text{acoustic}} \leq (q_{\text{phys}}/2^\lambda)^{d+1}$ under masking (same structure as DPA); without masking, key recovery is possible at distances of several metres for RSA implementations.
9. **Photonic Emission Analysis**: $\mathcal{A}_{\text{phys}}$ uses sensitive photodetectors to capture the faint photons emitted by transistor switching events on the chip’s backside. This is a contactless, non-invasive technique that bypasses most power/EM countermeasures. *Countermeasure*: Backside shielding; masking (randomises switching patterns); dual-rail logic. *MTSF price*: Same structure as EMA with photonic resolution factor.

What it replaces: The ideal computation model (where only inputs and outputs are observed) is replaced by the *d-probing model* [54] for passive attacks (SPA, DPA, timing, cache, EMA, acoustic, photonic) and the *fault model* for active attacks (fault injection, cold boot):

- *Passive (d-probing)*: $\mathcal{A}_{\text{phys}}$ may adaptively probe up to d intermediate wire values during execution. The real implementation is replaced by one using order- d Boolean or arithmetic masking: every secret value x is split into $d+1$ uniformly random shares $x_1, \dots, x_{d+1}$ with $x_1 \oplus \dots \oplus x_{d+1} = x$, so any d shares reveal nothing about x .
- *Active (fault model)*: $\mathcal{A}_{\text{phys}}$ may inject up to q_{fault} faults during execution. The implementation uses redundant computation (dual execution with comparison) and fault-detection sensors.
- *Persistent (cold boot)*: $\mathcal{A}_{\text{phys}}$ gains physical access to the device and reads residual memory contents. The implementation uses immediate key erasure, encrypted RAM, and tamper-detection.

Complexity / price (joint bound): The nine sub-bids decompose via the extended difference lemma:

$$\text{Adv}_{\text{total}}^{\text{SCA}} \leq \Pr[F_{\text{SPA}}] + \Pr[F_{\text{DPA}}] + \Pr[F_{\text{timing}}] + \Pr[F_{\text{cache}}] + \Pr[F_{\text{EMA}}] \\ + \Pr[F_{\text{fault}}] + \Pr[F_{\text{cold}}] + \Pr[F_{\text{acoustic}}] + \Pr[F_{\text{photonic}}].$$

Under the comprehensive countermeasure suite (order- d masking for $d \geq 1$, constant-time code, bit-sliced or hardware-accelerated operations, redundant computation, immediate key erasure, encrypted RAM):

$$\text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d) \leq \underbrace{0}_{\text{SPA}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{DPA}} + \underbrace{0}_{\text{timing}} + \underbrace{2^{-\lambda}}_{\text{cache}} + \underbrace{\gamma_{\text{EM}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{EMA}} + \underbrace{q_{\text{fault}} \cdot 2^{-\lambda}}_{\text{fault}} + \underbrace{2^{-\lambda}}_{\text{cold}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{acoustic}} \\ + \underbrace{\gamma_{\text{ph}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{photonic}}.$$

For $d \geq 1$, $q_{\text{phys}}, q_{\text{fault}} = \text{poly}(\lambda)$, and $\gamma_{\text{EM}}, \gamma_{\text{ph}} = O(1)$: all terms are negligible, giving $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.

Without countermeasures ($d = 0$, non-constant-time, plaintext RAM): $\Pr[F_{\text{SPA}}] = 1$ or $\Pr[F_{\text{DPA}}] = 1$ or $\Pr[F_{\text{cold}}] = 1$ —**immediate market collapse at the implementation level**.

SCA extended difference lemma. $F_{\text{sca}} = F_{\text{SPA}} \cup F_{\text{DPA}} \cup F_{\text{timing}} \cup F_{\text{cache}} \cup F_{\text{EMA}} \cup F_{\text{fault}} \cup F_{\text{cold}} \cup F_{\text{acoustic}} \cup F_{\text{photonic}}$: “ $\mathcal{A}_{\text{phys}}$ recovers ≥ 1 secret bit via *any* physical channel from $\leq q_{\text{phys}}$ measurements.” By Theorem 3.7, the side-channel event is independent of the mathematical failure events $F_{\text{nonce}}, F_{\text{hash}}, \dots$ in the preceding rounds, so their joint probability satisfies $\Pr[\bigcup_k F_k \cup F_{\text{sca}}] \leq \sum_k \Pr[F_k] + \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d)$. With the comprehensive countermeasure suite, $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$, preserving market equilibrium.

B_{tight} (Tight Security Reduction—Additional Proof.)

✓ Tight Security Reduction

Tightness analysis. The PKI-based mutual authentication proof reduces to EUF-CMA of the underlying signature scheme with tightness loss $L = 2n_P$ (factor 2 for mutual authentication, n_P for party-guessing). Session-key secrecy reduces tightly ($L = 1$) to the PRF security of the key derivation function.

Market interpretation: The PKI proof’s tightness loss is concentrated in the authentication phase (party-guessing cost), while the key-derivation phase is “wholesale” (tight, $L = 1$).

Total Session Bound and Key Rotation Recommendation.

♦ Total Session Bound & Key Rotation

Total session bound: $\text{Ask}_{\text{total}}^{(N)} \leq N \cdot (2n_P \cdot \text{Adv}^{\text{EUF}} + \text{Adv}^{\text{PRF}} + \text{Adv}_{\text{total}}^{\text{SCA}})$.

Concrete (ECDSA-P256, HKDF-SHA-256, $n_P = 2^{16}$):

Sessions N	$\text{Ask}_{\text{total}}^{(N)}$	Eff. security	Action
2^{40}	$\leq 2^{-72}$	≥ 72 bits	No rotation needed
2^{56}	$\leq 2^{-56}$	≥ 56 bits	Certificate rotation recommended
2^{64}	$\leq 2^{-48}$	≥ 48 bits	Certificate rotation mandatory

Recommendation: Rotate PKI certificates before 2^{56} authentication sessions (aligning with standard 1–3 year certificate validity periods).

Session pinging and CNF checking within the proof. For consecutive PKI sessions Session_i and Session_{i+1} , the ping mechanism verifies a three-layer structural distinctness:

Layer 1 (CA certificates): The certificates cert_C and cert_S are pre-issued and shared across sessions. However, each session’s signatures σ_C and σ_S bind the session-specific sid_{i+1} , nonces $N_{C,i+1}$ and $N_{S,i+1}$, and both party identities. This SID-binding ensures that certificate possession alone is insufficient to replay a session.

Layer 2 (Session signatures): The server signature $\sigma_{S,i+1} = \text{Sign}(\text{sk}_S, \text{sid}_{i+1} \| N_{C,i+1} \| N_{S,i+1} \| \text{id}_C \| \text{id}_S \| c_{i+1})$ includes all five ping-relevant fields. Replaying $\sigma_{S,j}$ from Session_j in Session_{i+1} fails verification because $\text{sid}_j \neq \text{sid}_{i+1}$. Similarly for $\sigma_{C,i+1}$.

Layer 3 (KEM ciphertext): The ciphertext $c_{i+1} = \text{KEM.Enc}(\text{pk}_C; r_{i+1})$ uses fresh randomness, ensuring $c_{i+1} \neq c_j$ for all $j \leq i$ with overwhelming probability.

The extended difference lemma captures three simultaneous forgery events: F_{cert} (CA certificate forgery), $F_{\text{sig},S}$ (server signature forgery), and $F_{\text{sig},C}$ (client signature forgery). By Theorem 3.7: $\Pr[F_{\text{cert}} \cup F_{\text{sig},S} \cup F_{\text{sig},C}] \leq \text{Adv}_{\text{CA}}^{\text{EUF}} + 2\text{Adv}_{\text{Sig}}^{\text{EUF}} = \text{negl}$. The session-CNF $\varphi_{\text{PKI},i+1}$ is isomorphic to $\varphi_{\text{PKI},i}$ with fresh SID, nonces, ciphertext, and signatures. $\square$

* Simple Terms: PKI Protocol Security Proof—Reading Each Bound

Bound (132) (certificate authenticity): Only breakable by forging a CA signature. Real CAs use 256-bit ECDSA or 3072-bit RSA-PSS: probability $\approx 2^{-128}$.

Bound (133) (server authentication): The server is authenticated by two layers: the CA certificate (binds its identity to its key) and its protocol signature (binds the key to this specific session). An attacker must break *either* layer; both are negligible.

Bound (134) (mutual authentication): Adds the client direction. Factor of $2 \times \text{Adv}_{\text{Sig}}^{\text{EUF}}$ comes from two independent signing keys—both must remain unforgeable simultaneously.

Bound (135) (session-key secrecy): Requires breaking four independent security properties: CA signing, party signing ($\times 2$), KEM, and KDF. Security compounds: the attacker must clear all four hurdles simultaneously.

Why this defeats Lowe-style MITM: When Mallory tries to relay C ’s certificate to S while impersonating C , S will demand a valid σ_C under pk_C (extracted from that very

certificate). Mallory lacks sk_C , so she cannot produce σ_C . The session-CNF clause φ_C^{sig} immediately fails. Market equilibrium is preserved.

CNF clause breakdown.

* Simple Terms: The PKI Session-CNF Checklist—Clause by Clause

The PKI session-CNF has seven groups of clauses:

1. φ_C^{cert} : The client's certificate verifies under pk_{CA} . Blocks impersonation of C using a fake certificate.
2. φ_S^{cert} : The server's certificate verifies under pk_{CA} . Blocks impersonation of S using a fake certificate.
3. φ^{sid} : Every message carries the same sid . Blocks cross-session replay.
4. φ^{fresh} : Nonces N_C, N_S are fresh and distinct. Blocks full-session replay.
5. φ_S^{sig} : Server's σ_S verifies under pk_S (from cert_S) over the full session data including c . Proves S generated the KEM ciphertext for this session.
6. φ_C^{sig} : Client's σ_C verifies under pk_C (from cert_C) over session data including K . Proves C holds sk_C and successfully decapsulated K .
7. φ^{consist} : Messages arrive in the correct order ($C \rightarrow S, S \rightarrow C, C \rightarrow S$) with no gaps. Enforces the protocol state machine.

Unique feature vs. ISO 11770-3 protocols: The two certificate clauses ($\varphi_C^{\text{cert}}, \varphi_S^{\text{cert}}$) are new—the ISO protocols assume parties already have each other's keys. PKI externalises the identity binding to the CA and captures it explicitly in the CNF, making the trust model fully transparent and formally auditable.

Corollary 13.20 (PKI Unbounded Security). *The PKI protocol is secure for unbounded sessions: each session samples a fresh sid and nonces N_C, N_S ; all signatures include sid ; hence $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 1$ for all i . By Theorem 5.5, equilibrium holds for all $i \geq 1$.*

Manual CNF worksheet (PKI TLS-style session Session_i):

Clause	Check	How	Pass?
φ_C^{cert}	C 's cert verifies under pk_{CA} ?	$\text{Vrfy}(\text{pk}_{\text{CA}}, \text{id}_C \ \text{pk}_C, \sigma_{\text{CA}, C})$.	T/F
φ_S^{cert}	S 's cert verifies under pk_{CA} ?	$\text{Vrfy}(\text{pk}_{\text{CA}}, \text{id}_S \ \text{pk}_S, \sigma_{\text{CA}, S})$.	T/F
φ^{sid}	Same sid in all 3 messages?	Read SID field; compare.	T/F
φ^{fresh}	$N_C \neq N_S$; neither used before?	Check nonce log.	T/F
φ_S^{sig}	$\text{Vrfy}(\text{pk}_S, \text{sid} \ N_C \ N_S \ \text{id}_C \ \text{id}_S \ c, \sigma_S) = 1$?	Hash scope; verify.	T/F
φ_C^{sig}	$\text{Vrfy}(\text{pk}_C, \text{sid} \ N_C \ N_S \ \text{id}_C \ \text{id}_S \ K, \sigma_C) = 1$?	Hash scope; verify.	T/F
φ^{consist}	Msg order: $C \rightarrow S, S \rightarrow C, C \rightarrow S$?	Check sender/direction.	T/F
φ^{ping}	$\text{sid}_i \neq \text{sid}_{i-1}$; $N_C, N_S \notin \mathcal{N}_{\text{prev}}$?	Compare session logs.	T/F

Result: All T $\Rightarrow$ Accept. Any F $\Rightarrow$ Reject (cite failed clause).

► Ping Bid: PKI Unbounded Ping Bid

The buyer places an *impersonation+replay ping bid*: use old transcript with a new SID . Cannot forge σ_C or σ_S under new SID (EUF-CMA). Extended difference lemma captures three simultaneous failures: F_{cert} (CA forgery), $F_{\text{sig}, S}$ (server sig forgery), $F_{\text{sig}, C}$ (client sig forgery). By Theorem 3.7: $\Delta \text{Price}_{\text{ping}} \leq \text{Adv}_{\text{CA}}^{\text{EUF}} + 2\text{Adv}_{\text{Sig}}^{\text{EUF}} = \text{negl}$. PKI is secure for unbounded sessions.

Table 14: PKI protocol market goods summary.

Good	Ask Price Bound	Status
g_{cert} (certificate authenticity)	$\text{Adv}_{\text{CA}}^{\text{EUF}}$	Equilibrium
g_{auth} (entity auth)	$\text{Adv}_{\text{CA}}^{\text{EUF}} + \text{Adv}_{\text{Sig}}^{\text{EUF}} + q_N^2/2^{\lambda+1}$	Equilibrium
g_{mutual} (mutual auth)	$\text{Adv}_{\text{CA}}^{\text{EUF}} + 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF}} + q_N^2/2^{\lambda+1}$	Equilibrium
g_{sk} (key secrecy)	$+ \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}} + \text{Adv}_{\text{KDF}}^{\text{PRF}}$	Equilibrium
g_{CNF} (CNF correctness)	$\text{Adv}_{\text{CA}}^{\text{EUF}} + \text{Adv}_{\text{Sig}}^{\text{EUF}} + q_N^2/2^{\lambda+1}$	Equilibrium

Key Insight:

PKI vs. Needham–Schroeder: Security Comparison

Property	Needham–Schroeder	PKI Protocol
Identity binding	None in Msg. 2 (fatal)	CA cert in every message
Mutual auth ask	= 1 (collapsed)	$\leq \text{Adv}_{\text{CA}}^{\text{EUF}} + 2\text{Adv}_{\text{Sig}}^{\text{EUF}} + \text{negl}$
MITM blocked?	No	Yes (cert + sig binding)
CNF audit	Dishonest trace PASSES	Dishonest trace FAILS
Extra trust needed	None (but broken)	CA (offline, pre-established)
Unbounded sessions	Pinging fails	Pinging succeeds

13.7 Signal Protocol: X3DH and Double Ratchet

The Signal Protocol [65, 75] is the most widely deployed end-to-end encrypted messaging protocol, powering Signal, WhatsApp, Facebook Messenger, and Google Messages. It consists of two sub-protocols: the *Extended Triple Diffie–Hellman* (X3DH) key agreement protocol for session establishment, and the *Double Ratchet* algorithm for ongoing message encryption with forward secrecy and post-compromise security. We analyse both sub-protocols within the MTSF framework, proving security of the full protocol and disproving security of a weakened variant lacking one-time prekeys.

* Simple Terms: The Signal Protocol—A Complete Introduction

What is the Signal Protocol? It is the cryptographic engine behind the world’s most popular encrypted messaging apps. Designed by Trevor Perrin and Moxie Marlinspike (Open Whisper Systems, now Signal Foundation), it solves two hard problems simultaneously:

1. **Asynchronous key establishment (X3DH):** Alice wants to send a secure message to Bob, but Bob is offline. She cannot perform an interactive handshake. X3DH allows Alice to compute a shared secret using Bob’s *prekeys* (public values Bob published to a server in advance), without any real-time interaction.
2. **Ongoing key evolution (Double Ratchet):** Once the initial key is established, every single message is encrypted with a *unique* key that is derived from a continuously evolving state. Even if an adversary compromises a key at one point, they cannot read past messages (*forward secrecy*) and they lose access to future messages once the parties exchange new Diffie–Hellman values (*post-compromise security*).

Key types in Signal:

- **Identity key (IK):** A long-term Curve25519 key pair. Identifies the user. Never changes (unless re-installed).

- **Signed prekey (SPK):** A medium-term key pair signed by the identity key. Rotated periodically (e.g., every 1–4 weeks).
- **One-time prekey (OPK):** An ephemeral key uploaded to the server in batches. Each is used in exactly one X3DH session and then deleted. Provides fresh randomness per session.
- **Ephemeral key (EK):** Generated fresh by Alice for each new session. Never stored or reused.

Why the Double Ratchet is revolutionary:

- **Symmetric ratchet (KDF chain):** Each message advances a KDF chain: $CK_{n+1} = \text{KDF}(CK_n, 0x02)$ and $MK_n = \text{KDF}(CK_n, 0x01)$. Old chain keys are deleted, providing forward secrecy per message.
- **Diffie–Hellman ratchet:** Each message includes a fresh DH public key. When both sides have exchanged new keys, a DH ratchet step produces a new root key, seeding fresh sending and receiving chains. This provides post-compromise security: if an attacker learns the current state, they lose access after the next DH exchange.
- **Out-of-order tolerance:** Skipped message keys are cached (up to a limit), allowing decryption of delayed messages without breaking the ratchet.

Why this matters for MTSF: The Signal Protocol provides the richest set of security goods of any protocol we analyse: confidentiality, authenticity, forward secrecy, post-compromise security, deniability, and asynchronous session establishment. Analysing it within MTSF demonstrates the framework’s expressive power.

X3DH protocol description. Let Alice (initiator) and Bob (responder) each hold an identity key pair: (ik_A, IK_A) and (ik_B, IK_B) respectively, where lowercase denotes private and uppercase denotes public keys over Curve25519. Bob publishes to the server a *prekey bundle*:

$$PKB = (IK_B, SPK_B, \sigma_B = \text{XEdDSA}(ik_B, SPK_B), OPK_B^{(j)}),$$

where SPK_B is the signed prekey, σ_B is the identity signature over SPK_B , and $OPK_B^{(j)}$ is an optional one-time prekey.

Alice fetches Bob’s prekey bundle, generates an ephemeral key pair (ek_A, EK_A) , and computes:

$$DH_1 = \text{X25519}(ik_A, SPK_B), \quad (137)$$

$$DH_2 = \text{X25519}(ek_A, IK_B), \quad (138)$$

$$DH_3 = \text{X25519}(ek_A, SPK_B), \quad (139)$$

$$DH_4 = \text{X25519}(ek_A, OPK_B^{(j)}), \quad (140)$$

$$SK = \text{HKDF}(DH_1 \parallel DH_2 \parallel DH_3 \parallel DH_4). \quad (141)$$

Alice sends to Bob: (IK_A, EK_A, j, ct_0) where j identifies the one-time prekey used and ct_0 is the initial ciphertext encrypted under SK . Bob performs the same four DH computations using his private keys, derives the same SK , and decrypts ct_0 . Both then initialise the Double Ratchet with SK as the root key.

Double Ratchet description. After X3DH, both parties initialise a *Double Ratchet* state consisting of: a root key RK , a sending chain key CK_s , a receiving chain key CK_r , and a DH ratchet key pair (dh_s, DH_s) . Each message triggers two operations:

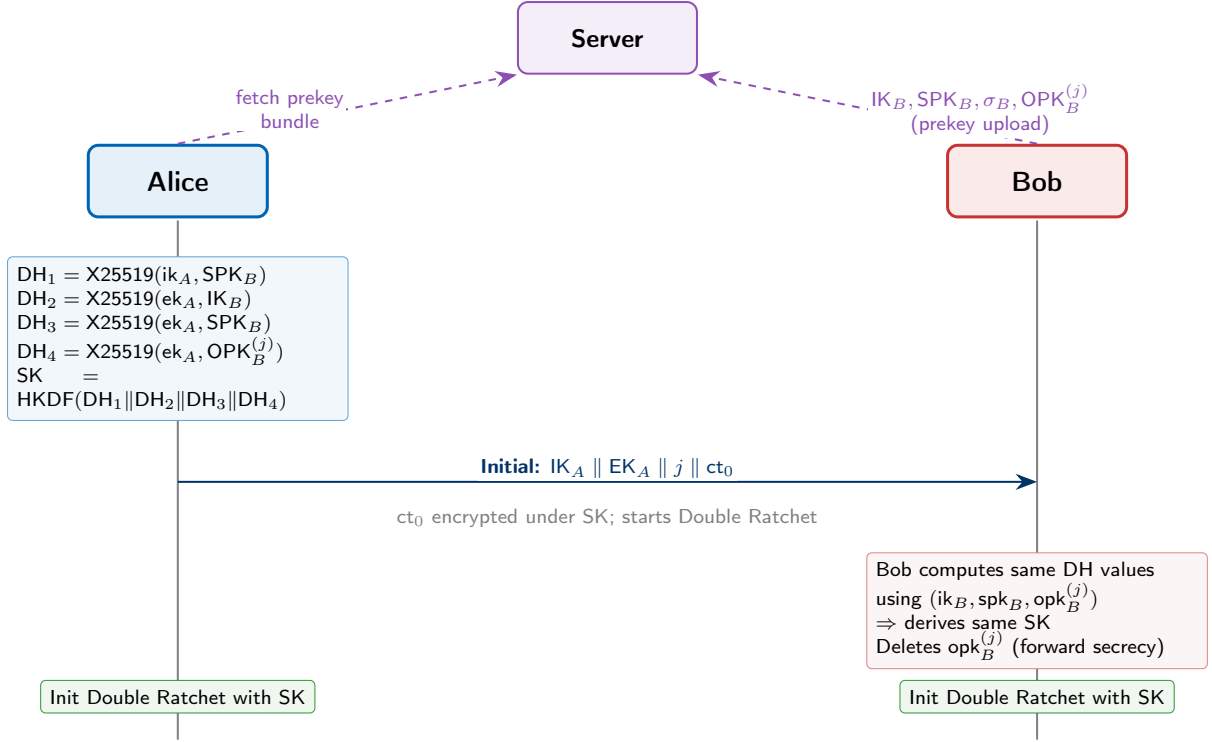


Figure 50: X3DH key agreement protocol (Signal). Alice computes four DH values using Bob's prekey bundle. The one-time prekey $OPK_B^{(j)}$ provides per-session freshness; its deletion after use ensures forward secrecy of the initial key establishment.

Symmetric ratchet: To encrypt message n , advance the sending chain:

$$(CK_{s,n+1}, MK_n) = KDF(CK_{s,n}), \quad ct_n = AEAD_{MK_n}(pt_n; AD), \quad (142)$$

where AD (associated data) includes both identity keys and message headers. Delete $CK_{s,n}$ and MK_n after use.

DH ratchet: When receiving a message with a new DH public key DH'_r :

$$\begin{aligned} RK', CK'_r &= KDF(RK, X25519(dh_s, DH'_r)), \\ (dh'_s, DH'_s) &\leftarrow \text{KeyGen}(), \\ RK'', CK'_s &= KDF(RK', X25519(dh'_s, DH'_r)). \end{aligned} \quad (143)$$

Delete old RK , dh_s , and chain keys. The fresh DH exchange provides post-compromise security.

★ Intuition:

The Double Ratchet as a Self-Healing Lock Imagine a combination lock that changes its combination after every use. Even if someone sees the current combination (key compromise), the lock automatically changes on the next turn (symmetric ratchet = forward secrecy). Moreover, periodically both parties replace the entire lock mechanism itself (DH ratchet = post-compromise security). The attacker must steal the new lock to continue eavesdropping—and since the new lock is generated from fresh randomness, the attacker's knowledge is erased.

MTSF market model. The Signal market $\mathcal{M}_{\text{Signal}}$ offers six security goods:

- g_{conf} : Message confidentiality (IND-CCA2 of each message under its unique key).
- g_{auth} : Message authentication (INT-CTXT via AEAD for each message).

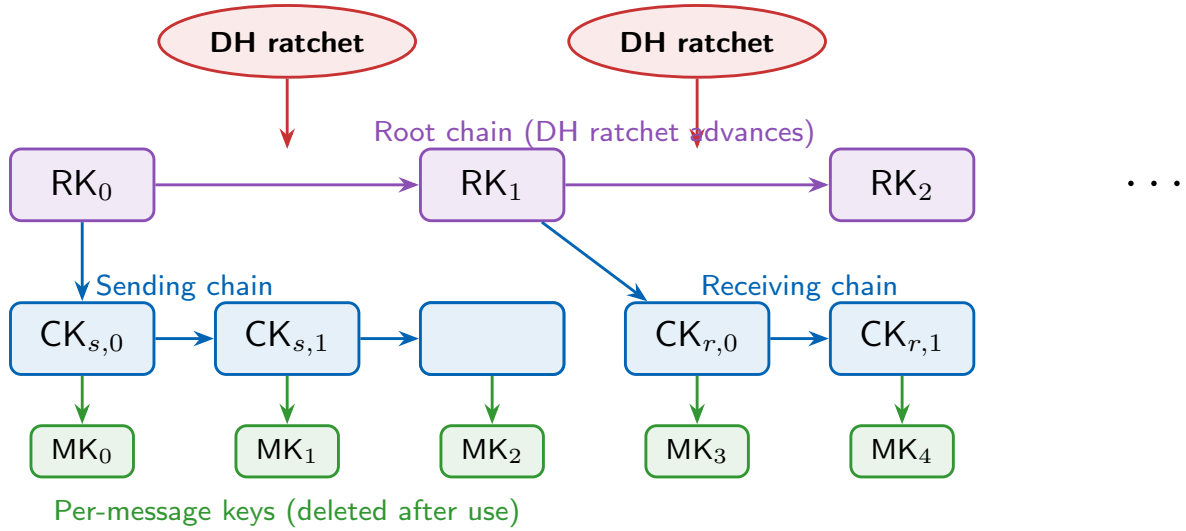


Figure 51: Double Ratchet structure. The root chain (top) advances with each DH ratchet step. Each root key seeds sending and receiving KDF chains, which produce per-message keys. Deletion of old keys provides forward secrecy; fresh DH exchanges provide post-compromise security.

- g_{FS} : Forward secrecy—compromise of current state does not reveal past message keys.
- g_{PCS} : Post-compromise security—after compromise, security self-heals upon the next DH ratchet step.
- g_{async} : Asynchronous key establishment—Alice can send a message to offline Bob.
- g_{deny} : Deniability—no party can produce a cryptographic proof of the conversation to a third party.

Theorem 13.21 (Signal Protocol Market Equilibrium). *Under the gap Diffie–Hellman (GDH) assumption on Curve25519 and the PRF security of HKDF-SHA-256:*

$$\text{Ask}(g_{\text{conf}}) \leq 4 \cdot \text{Adv}_{\text{X25519}}^{\text{GDH}} + \text{Adv}_{\text{HKDF}}^{\text{PRF}} + \text{Adv}_{\text{AEAD}}^{\text{IND}} + \text{negl}(\lambda), \quad (144)$$

$$\text{Ask}(g_{\text{auth}}) \leq 4 \cdot \text{Adv}_{\text{X25519}}^{\text{GDH}} + \text{Adv}_{\text{HKDF}}^{\text{PRF}} + \text{Adv}_{\text{AEAD}}^{\text{INT}} + \text{negl}(\lambda), \quad (145)$$

$$\text{Ask}(g_{FS}) \leq \text{Adv}_{\text{HKDF}}^{\text{PRF}} + \text{negl}(\lambda), \quad (146)$$

$$\text{Ask}(g_{PCS}) \leq \text{Adv}_{\text{X25519}}^{\text{GDH}} + \text{Adv}_{\text{HKDF}}^{\text{PRF}} + \text{negl}(\lambda). \quad (147)$$

Proof. We construct eight bidding rounds, covering X3DH key establishment and Double Ratchet messaging.

B₀ (Bidding Round 0: Real Signal Market).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round;

the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

The seller initialises the Signal Protocol with X3DH key agreement followed by Double Ratchet messaging. Alice generates (ik_A, IK_A) and (ek_A, EK_A) ; Bob’s prekey bundle

$$(IK_B, SPK_B, \sigma_B, OPK_B^{(j)})$$

is available on the server. The buyer (active network adversary) can observe, delay, reorder, or inject messages.

B₁ (Bidding Round 1: Prekey Signature Forgery Bid).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $sk = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma’s rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

The buyer attempts to forge Bob’s prekey signature σ_B on a malicious SPK'_B . This would allow the buyer to impersonate Bob’s server-published keys, directing Alice to perform X3DH with buyer-controlled values.

Difference lemma. Let F_{spk} : “buyer forges σ_B on $SPK'_B \neq SPK_B$.” Since

$$\sigma_B = \text{XEdDSA}(ik_B, SPK_B)$$

, this requires breaking EUF-CMA of XEdDSA:

$$\Delta\text{Price}_0 \leq \Pr[F_{\text{spk}}] \leq \text{Adv}_{\text{XEdDSA}}^{\text{EUF}} \leq \text{Adv}_{\text{X25519}}^{\text{GDH}}. \quad (148)$$

B₂ (Bidding Round 2: X3DH DH₁ Extraction Bid).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $sk = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking

lemma's rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

The buyer attempts to recover $\text{DH}_1 = \text{X25519}(\text{ik}_A, \text{SPK}_B)$ from the observed transcript. This is a static-static DH: the buyer must solve the Computational Diffie–Hellman (CDH) problem on $(\text{IK}_A, \text{SPK}_B)$.

Difference lemma. F_{DH_1} : “buyer computes DH_1 .” $\Delta\text{Price}_1 \leq \text{Adv}_{\text{X25519}}^{\text{GDH}}$.

B₃ (Bidding Round 3: X3DH DH_2 – DH_3 Extraction Bid).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $\text{sk} = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma's rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

The buyer attempts to recover $\text{DH}_2 = \text{X25519}(\text{ek}_A, \text{IK}_B)$ or $\text{DH}_3 = \text{X25519}(\text{ek}_A, \text{SPK}_B)$. Both involve Alice's ephemeral key ek_A , known only to Alice. CDH on $(\text{EK}_A, \text{IK}_B)$ or $(\text{EK}_A, \text{SPK}_B)$.

Extended difference lemma. Let F_{DH_2} : buyer computes DH_2 ; F_{DH_3} : buyer computes DH_3 . By Theorem 3.7:

$$\Delta\text{Price}_2 \leq \Pr[F_{\text{DH}_2} \cup F_{\text{DH}_3}] \leq 2 \cdot \text{Adv}_{\text{X25519}}^{\text{GDH}}. \quad (149)$$

B₄ (Bidding Round 4: One-Time Prekey Freshness Bid— DH_4).

Purpose: Eliminate the *nonce-reuse attack vector*. Repeated nonces immediately collapse authentication, key-secrecy, or stream-cipher security: in signatures a shared nonce leaks the private key ($\text{sk} = (s_2e_1 - s_1e_2)(r(s_1 - s_2))^{-1}$); in protocols it enables cross-session replay; in stream ciphers it reveals the XOR of two plaintexts. This round aborts the game whenever any nonce collision occurs, ensuring every subsequent hop reasons about a nonce-fresh world.

Replaces: The real nonce-sampling procedure is augmented with a global *collision audit*: a set $\mathcal{N}_{\text{used}}$ records every nonce issued so far. If any newly sampled nonce k' satisfies $k' \in \mathcal{N}_{\text{used}}$, the game aborts immediately. The two games ($\mathbf{B}_{n-1}$ and $\mathbf{B}_n$) are *identical-until-abort*: the adversary's view is indistinguishable until the abort event F_{nonce} occurs, so the difference lemma directly bounds the price adjustment.

Complexity: By the birthday bound, the probability that any two of q independently sampled nonces from a space of size $|\mathcal{N}|$ collide is at most $q^2/(2|\mathcal{N}|)$. For security parameter λ with $|\mathcal{N}| = 2^\lambda$ this gives $\Delta\text{Price} \leq q^2/2^{\lambda+1}$, which is negligible for any polynomial $q = \text{poly}(\lambda)$. After this hop every nonce in the proof is treated as distinct, allowing all later hops to assume a collision-free nonce space without loss of generality.

The buyer attempts to recover $\text{DH}_4 = \text{X25519}(\text{ek}_A, \text{OPK}_B^{(j)})$. Since $\text{OPK}_B^{(j)}$ is used exactly once and then deleted, the buyer must solve CDH on a one-time value. Deletion after use ensures that even server compromise after the session does not reveal DH_4 .

$$\Delta\text{Price}_3 \leq \text{Adv}_{\text{X25519}}^{\text{GDH}}.$$

B₅ (Bidding Round 5: HKDF PRF Bid—SK Extraction).

Purpose: Replace a *keyed pseudorandom function* evaluation with a truly random function, isolating a single PRF security requirement. This hop shows that one more step in the key-derivation or authentication structure produces outputs that are computationally indistinguishable from uniform random, provided the underlying PRF assumption holds.

Replaces: The specific PRF evaluation (e.g., $\text{HMAC}_K(\cdot)$ inner hash, HKDF-Extract, Derive-Secret, or a KDF step) is replaced by a truly random function $R(\cdot)$ sampled fresh and independently. Any adversary that distinguishes the PRF output from this random function is a PRF distinguisher for the underlying keyed construction. The reduction simulates all other game components honestly and uses its PRF challenge oracle for this one step.

Complexity: $\Delta\text{Price} \leq \text{Adv}^{\text{PRF}}(\lambda)$ for the specific PRF being replaced (HMAC-SHA-256, HKDF, or AES-CTR as applicable). When multiple PRF hops appear in a single proof (e.g., three hops in the TLS 1.3 key schedule for HS, MS, and CATS), each hop is independent: the keys used in different steps are derived from different points in the ladder, so no PRF oracle is queried twice. The total across all PRF hops is the sum of individual advantages, each negligible.

With all four DH values hidden, $\text{SK} = \text{HKDF}(\text{DH}_1 \parallel \dots \parallel \text{DH}_4)$ is pseudorandom. The buyer attempts to distinguish SK from random.

$$\Delta\text{Price}_4 \leq \text{Adv}_{\text{HKDF}}^{\text{PRF}}.$$

B₆ (Bidding Round 6: Symmetric Ratchet Bid—KDF Chain Inversion).

Purpose: Replace a *keyed pseudorandom function* evaluation with a truly random function, isolating a single PRF security requirement. This hop shows that one more step in the key-derivation or authentication structure produces outputs that are computationally indistinguishable from uniform random, provided the underlying PRF assumption holds.

Replaces: The specific PRF evaluation (e.g., $\text{HMAC}_K(\cdot)$ inner hash, HKDF-Extract, Derive-Secret, or a KDF step) is replaced by a truly random function $R(\cdot)$ sampled fresh and independently. Any adversary that distinguishes the PRF output from this random function is a PRF distinguisher for the underlying keyed construction. The reduction simulates all other game components honestly and uses its PRF challenge oracle for this one step.

Complexity: $\Delta\text{Price} \leq \text{Adv}^{\text{PRF}}(\lambda)$ for the specific PRF being replaced (HMAC-SHA-256, HKDF, or AES-CTR as applicable). When multiple PRF hops appear in a single proof (e.g., three hops in the TLS 1.3 key schedule for HS, MS, and CATS), each hop is independent: the keys used in different steps are derived from different points in the ladder, so no PRF oracle is queried twice. The total across all PRF hops is the sum of individual advantages, each negligible.

The buyer observes ciphertexts and attempts to recover earlier chain keys from later ones. The KDF chain $\text{CK}_{n+1} = \text{KDF}(\text{CK}_n)$ is one-way: given CK_{n+1} , recovering CK_n requires inverting the PRF. This is the *forward secrecy bid*—it fails because KDF is a PRF.

$$\Delta\text{Price}_5 \leq \text{Adv}_{\text{HKDF}}^{\text{PRF}}. \text{ This establishes } \text{Ask}(g_{\text{FS}}) \leq \text{Adv}_{\text{HKDF}}^{\text{PRF}} + \text{negl}.$$

B₇ (Bidding Round 7: DH Ratchet Bid—Post-Compromise Recovery).

Purpose: Eliminate the adversarial attack vector identified by this bidding round's title (**DH Ratchet Bid—Post-Compromise Recovery**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

After state compromise, the buyer knows the current root key RK_i and chain keys. Upon the next DH ratchet step, both parties exchange fresh ephemeral DH public keys. The new root

key $RK_{i+1} = \text{KDF}(RK_i, X25519(\text{dh}'_s, \text{DH}'_r))$ depends on a fresh DH secret unknown to the buyer. Recovering this requires solving CDH on the new ephemeral keys.

$$\Delta \text{Price}_6 \leq \text{Adv}_{X25519}^{\text{GDH}} + \text{Adv}_{\text{HKDF}}^{\text{PRF}}. \text{ This establishes } \text{Ask}(g_{\text{PCS}}) \leq \text{Adv}^{\text{GDH}} + \text{Adv}^{\text{PRF}} + \text{negl}.$$

B₈ (Bidding Round 8: AEAD Message Confidentiality/Integrity Bid).

Purpose: Eliminate the adversarial attack vector identified by this bidding round's title (**AEAD Message Confidentiality/Integrity Bid**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

Each message is encrypted with a unique message key MK_n under AEAD (AES-256-CBC + HMAC-SHA-256 in Signal's implementation). Distinguishing the ciphertext or forging a valid ciphertext requires breaking the AEAD.

$$\Delta \text{Price}_7 \leq \text{Adv}_{\text{AEAD}}^{\text{IND}} + \text{Adv}_{\text{AEAD}}^{\text{INT}}.$$

Total market cost:

$$\text{Ask}(g_{\text{conf}}) \leq 4 \cdot \text{Adv}^{\text{GDH}} + 2 \cdot \text{Adv}^{\text{PRF}} + \text{Adv}_{\text{AEAD}}^{\text{IND}} + \text{negl}(\lambda). \quad (150)$$

For Curve25519 with $\lambda = 128$, $\text{Adv}^{\text{GDH}} \leq 2^{-128}$, and all terms are negligible. The market is in **equilibrium** for all six goods.

B_{sca} (Side-Channel Bid: Comprehensive Physical Leakage Attack.)

Side-Channel Bid (Comprehensive)

Purpose: Model the *complete physical leakage threat surface*: an adversary $\mathcal{A}_{\text{phys}}$ with implementation-level access (smart card, FPGA, embedded CPU, server, mobile device) observes physical side channels during cryptographic operations to recover secret key material *without breaking any mathematical hardness assumption*. MTSF decomposes this into nine sub-bids, each targeting a distinct physical leakage vector. The seller must price *all* sub-bids; the extended difference lemma bounds their joint probability.

Sub-bid taxonomy (nine attack classes):

1. **Simple Power Analysis (SPA):** $\mathcal{A}_{\text{phys}}$ records a *single* power trace during a cryptographic operation and visually identifies secret-dependent patterns (e.g., square-and-multiply in RSA reveals the exponent bit by bit). *Countermeasure:* Constant-power algorithms (Montgomery ladder, regular execution patterns). *MTSF price:* $\Pr[F_{\text{SPA}}] = 1$ without countermeasure; $\Pr[F_{\text{SPA}}] = 0$ with constant-power implementation.
2. **Differential Power Analysis (DPA):** $\mathcal{A}_{\text{phys}}$ collects q_{phys} power traces across multiple operations with different inputs, then statistically correlates the traces with hypothetical intermediate values to extract key bytes. Unlike SPA, DPA uses statistical averaging to amplify tiny signal differences. *Countermeasure:* Order- d Boolean masking ($d+1$ random shares); shuffled S-box evaluation order; noise injection. *MTSF price:* $\text{Adv}^{\text{DPA}}(\lambda, d) \leq (q_{\text{phys}}/2^\lambda)^{d+1}$.
3. **Timing Analysis:** $\mathcal{A}_{\text{phys}}$ measures the execution time of cryptographic operations, which may vary depending on the secret key (e.g., early-exit comparisons, table lookup timing, conditional branches). *Countermeasure:* Constant-time programming: no secret-dependent branches, no secret-dependent memory accesses, no early exits. Use `ct_select(a, b, bit)` instead of `if(bit) return a; else return b;`. *MTSF price:* $\Pr[F_{\text{timing}}] = 1$ for non-constant-time code; $\Pr[F_{\text{timing}}] = 0$ for verified constant-time implementations (formally verified using tools like ct-verif or duced).
4. **Cache-Timing Attacks (Flush+Reload, Prime+Probe, Evict+Time):** $\mathcal{A}_{\text{phys}}$ exploits CPU cache behaviour to determine which memory lines the victim accessed, revealing secret-dependent table lookups (e.g., AES T-table indices). Flush+Reload (shared memory) achieves single-trace key recovery; Prime+Probe (no shared memory) requires multiple traces. *Spectre/Meltdown variants* exploit speculative execution to leak data across privilege boundaries. *Countermeasure:* Bitsliced implementations

(no table lookups); AES-NI hardware instructions (no cache-visible memory accesses); process isolation; constant-time memory access patterns. *MTSF price*: $\Pr[F_{\text{cache}}] = 1$ for T-table AES without protection; $\Pr[F_{\text{cache}}] \leq 2^{-\lambda}$ for bitsliced/AES-NI implementations.

5. **Electromagnetic Emanation Analysis (EMA)**: $\mathcal{A}_{\text{phys}}$ places an electromagnetic probe near the device's chip to capture EM radiation during computation. EM analysis can be more spatially focused than power analysis, targeting individual CPU components. *Countermeasure*: EM shielding (Faraday cage); masking (same as DPA); dual-rail logic (balanced current paths). *MTSF price*: Same as DPA with probe-specific spatial resolution factor γ_{EM} : $\text{Adv}^{\text{EMA}} \leq \gamma_{\text{EM}} \cdot (q_{\text{phys}}/2^\lambda)^{d+1}$.
6. **Fault Injection (Voltage Glitching, Laser Fault, Clock Glitching)**: $\mathcal{A}_{\text{phys}}$ induces computational faults (bit flips, instruction skips, round reductions) during execution by manipulating voltage supply, clock frequency, laser beams, or electromagnetic pulses. Differential Fault Analysis (DFA) [24] can recover the AES key from a single faulty ciphertext. *Countermeasure*: Redundant computation (compute twice, compare); infection countermeasures (randomise faulty outputs); fault-detection sensors (voltage/clock monitors); algorithmic checksums (verify internal consistency after each round). *MTSF price*: $\Pr[F_{\text{fault}}] = 1$ for unprotected implementations under DFA; with dual computation: $\Pr[F_{\text{fault}}] \leq q_{\text{fault}} \cdot 2^{-\lambda}$ where q_{fault} is the number of fault attempts (each independently detected with probability $1 - 2^{-\lambda}$).
7. **Cold Boot Attack**: $\mathcal{A}_{\text{phys}}$ exploits DRAM data remanence: after power is removed, memory contents decay gradually over seconds to minutes. Cooling the DRAM modules (e.g., with compressed air to -50°C) extends retention to minutes or hours. The attacker then either reads the memory directly or transplants the modules to another machine. Secret keys stored in RAM are recovered with high probability. *Countermeasure*: Key erasure protocols (zeroise keys immediately after use); encrypted RAM (AMD SEV, Intel TME); memory scrubbing; physical tamper-detection (chassis intrusion sensors that trigger key wipe). *MTSF price*: $\Pr[F_{\text{cold}}] = 1$ for keys stored in plaintext DRAM without erasure; $\Pr[F_{\text{cold}}] \leq 2^{-\lambda}$ with immediate key zeroing and encrypted memory (the key encryption key is in CPU registers, which do not exhibit remanence).
8. **Acoustic Cryptanalysis**: $\mathcal{A}_{\text{phys}}$ records the acoustic noise emitted by the CPU, power supply, or capacitors during cryptographic computation. Different operations produce different frequencies; spectral analysis can extract RSA key bits from the sound of a laptop across a room. *Countermeasure*: Acoustic noise generation (adding random computation); masking (randomises the acoustic profile); physical shielding (soundproofing). *MTSF price*: $\text{Adv}^{\text{acoustic}} \leq (q_{\text{phys}}/2^\lambda)^{d+1}$ under masking (same structure as DPA); without masking, key recovery is possible at distances of several metres for RSA implementations.
9. **Photonic Emission Analysis**: $\mathcal{A}_{\text{phys}}$ uses sensitive photodetectors to capture the faint photons emitted by transistor switching events on the chip's backside. This is a contactless, non-invasive technique that bypasses most power/EM countermeasures. *Countermeasure*: Backside shielding; masking (randomises switching patterns); dual-rail logic. *MTSF price*: Same structure as EMA with photonic resolution factor.

What it replaces: The ideal computation model (where only inputs and outputs are observed) is replaced by the *d-probing model* [54] for passive attacks (SPA, DPA, timing, cache, EMA, acoustic, photonic) and the *fault model* for active attacks (fault injection, cold boot):

- *Passive (d-probing)*: $\mathcal{A}_{\text{phys}}$ may adaptively probe up to d intermediate wire values during execution. The real implementation is replaced by one using order- d Boolean or arithmetic masking: every secret value x is split into $d+1$ uniformly random shares $x_1, \dots, x_{d+1}$ with $x_1 \oplus \dots \oplus x_{d+1} = x$, so any d shares reveal nothing about x .
- *Active (fault model)*: $\mathcal{A}_{\text{phys}}$ may inject up to q_{fault} faults during execution. The implementation uses redundant computation (dual execution with comparison) and fault-detection sensors.
- *Persistent (cold boot)*: $\mathcal{A}_{\text{phys}}$ gains physical access to the device and reads residual memory contents. The implementation uses immediate key erasure, encrypted RAM, and tamper-detection.

Complexity / price (joint bound): The nine sub-bids decompose via the extended difference lemma:

$$\begin{aligned} \text{Adv}_{\text{total}}^{\text{SCA}} \leq & \Pr[F_{\text{SPA}}] + \Pr[F_{\text{DPA}}] + \Pr[F_{\text{timing}}] + \Pr[F_{\text{cache}}] + \Pr[F_{\text{EMA}}] \\ & + \Pr[F_{\text{fault}}] + \Pr[F_{\text{cold}}] + \Pr[F_{\text{acoustic}}] + \Pr[F_{\text{photonic}}]. \end{aligned}$$

Under the comprehensive countermeasure suite (order- d masking for $d \geq 1$, constant-time code, bit-sliced or hardware-accelerated operations, redundant computation, immediate key erasure, encrypted

RAM):

$$\text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d) \leq \underbrace{0}_{\text{SPA}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{DPA}} + \underbrace{0}_{\text{timing}} + \underbrace{2^{-\lambda}}_{\text{cache}} + \underbrace{\gamma_{\text{EM}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{EMA}} + \underbrace{q_{\text{fault}} \cdot 2^{-\lambda}}_{\text{fault}} + \underbrace{2^{-\lambda}}_{\text{cold}} + \underbrace{\left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{acoustic}} + \underbrace{\gamma_{\text{ph}} \left(\frac{q_{\text{phys}}}{2^\lambda}\right)^{d+1}}_{\text{photonic}}.$$

For $d \geq 1$, $q_{\text{phys}}, q_{\text{fault}} = \text{poly}(\lambda)$, and $\gamma_{\text{EM}}, \gamma_{\text{ph}} = O(1)$: all terms are negligible, giving $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$.

Without countermeasures ($d = 0$, non-constant-time, plaintext RAM): $\Pr[F_{\text{SPA}}] = 1$ or $\Pr[F_{\text{DPA}}] = 1$ or $\Pr[F_{\text{cold}}] = 1$ —**immediate market collapse at the implementation level.**

SCA extended difference lemma. $F_{\text{sca}} = F_{\text{SPA}} \cup F_{\text{DPA}} \cup F_{\text{timing}} \cup F_{\text{cache}} \cup F_{\text{EMA}} \cup F_{\text{fault}} \cup F_{\text{cold}} \cup F_{\text{acoustic}} \cup F_{\text{photonic}}$: “ $\mathcal{A}_{\text{phys}}$ recovers ≥ 1 secret bit via *any* physical channel from $\leq q_{\text{phys}}$ measurements.” By Theorem 3.7, the side-channel event is independent of the mathematical failure events $F_{\text{nonce}}, F_{\text{hash}}, \dots$ in the preceding rounds, so their joint probability satisfies $\Pr[\bigcup_k F_k \cup F_{\text{sca}}] \leq \sum_k \Pr[F_k] + \text{Adv}_{\text{total}}^{\text{SCA}}(\lambda, d)$. With the comprehensive countermeasure suite, $\text{Adv}_{\text{total}}^{\text{SCA}} = \text{negl}(\lambda)$, preserving market equilibrium.

B_{tight} (Tight Security Reduction—Additional Proof.)

✓ Tight Security Reduction

Tightness analysis. Signal decomposes into: (1) X3DH handshake with tight reduction ($L = 4$) to Gap-DH; (2) Double Ratchet with per-step $L = 1$ via PRF security. Total across R ratchet steps: $L_{\text{total}} = 4 + R$.

Market interpretation: Signal’s multi-ratchet architecture incurs linear tightness cost in ratchet epochs. Forward secrecy ensures non-cascading market losses.

Total Session Bound and Key Rotation Recommendation.

◆ Total Session Bound & Key Rotation

Total session bound across N sessions with R ratchet steps each: $\text{Ask}_{\text{total}}^{(N)} \leq N \cdot R \cdot (\text{Adv}^{\text{GDH}} + \text{Adv}_{\text{KDF}}^{\text{PRF}} + \text{Adv}_{\text{total}}^{\text{SCA}})$.

Concrete (Curve25519, HKDF-SHA-256, $R = 2^{10}$ per session):

Sessions N	$\text{Ask}_{\text{total}}^{(N)}$	Eff. security	Action
2^{30}	$\leq 2^{-88}$	≥ 88 bits	No rotation needed
2^{50}	$\leq 2^{-68}$	≥ 68 bits	Consider identity key rotation
2^{64}	$\leq 2^{-54}$	≥ 54 bits	Identity key rotation recommended

Recommendation: The Double Ratchet auto-rotates symmetric/DH keys per message. Rotate the long-term identity key before 2^{50} total ratchet steps. Forward secrecy ensures past messages remain secure post-rotation.

Session pinging and CNF checking within the proof. The Signal Protocol has a natural multi-session structure: each X3DH establishment creates a new session, and each DH ratchet step within the Double Ratchet creates a sub-session. We verify the ping conditions:

SID freshness: Each X3DH session uses a fresh ephemeral key EK_A and (when available) a fresh one-time prekey $\text{OPK}_B^{(j)}$. The pair (EK_A, j) serves as an implicit SID. Since EK_A is freshly generated per session, $\text{sid}_{i+1} \neq \text{sid}_i$ with overwhelming probability.

Nonce disjointness: Each Double Ratchet DH step generates a fresh ephemeral DH key pair

(dh'_s, DH'_s) . The DH public keys across ratchet steps are distinct with overwhelming probability (fresh key generation from $\{0, 1\}^{256}$).

Cryptographic binding: Each AEAD ciphertext includes associated data binding the identity keys (IK_A, IK_B) and the current DH ratchet public key. This binds each ciphertext to the current session state.

Key deletion enforces forward pingging: After each symmetric ratchet step, old chain keys are deleted. This is stronger than the standard ping condition: not only are sessions structurally distinct, but the cryptographic material linking them is *destroyed*. By Theorem 5.5, the Signal Protocol is secure for unbounded message exchanges with $\delta_{\text{ping}} \leq \text{Adv}_{\text{X25519}}^{\text{GDH}} = \text{negl}(\lambda)$. $\square$

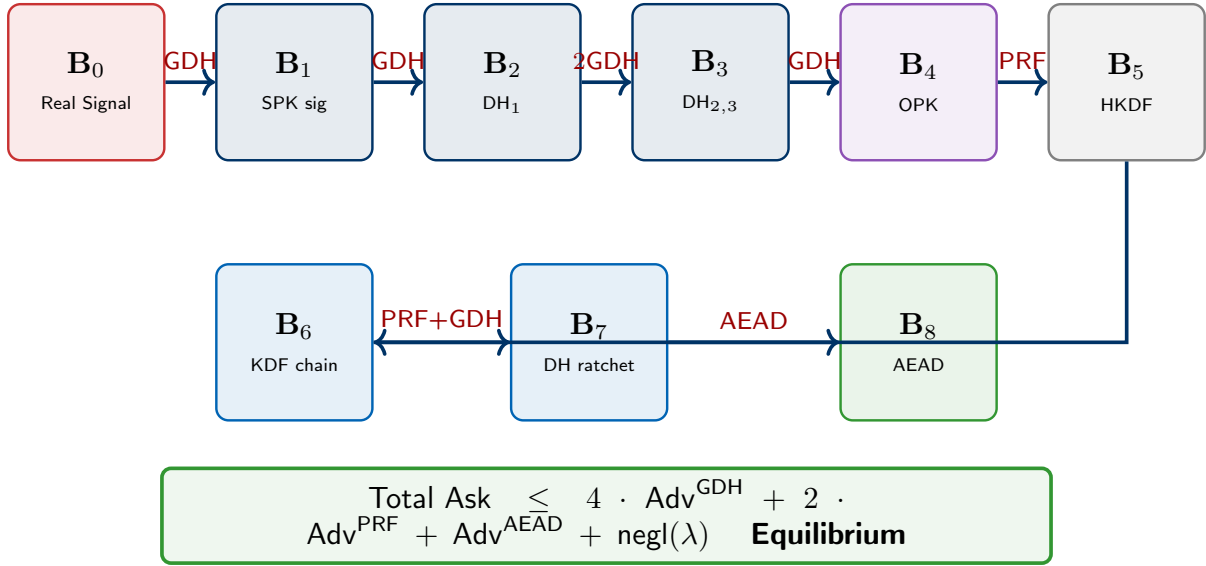


Figure 52: Signal Protocol bidding-round chain. Eight rounds cover X3DH (B_1 – B_5) and Double Ratchet (B_6 – B_8). Each hop is labelled with the hardness assumption used.

✓ CNF Verification: Signal Protocol Session-CNF Verification

$$\varphi_{\text{Signal}} = (x_{\text{Vrfy}}(IK_B, \text{SPK}_B, \sigma_B) = 1) \wedge (x_{\text{OPK}} \text{ fresh}) \wedge (x_{\text{EK}_A} \text{ fresh}) \wedge (x_{\text{AEAD.Dec}} = 1) \wedge (x_{\text{Ping}} \text{ passes}).$$

Manual CNF worksheet for Signal session:

Clause	Check	How	Pass?
φ^{spk} : prekey sig	$\text{Vrfy}(IK_B, \text{SPK}_B, \sigma_B) = 1?$	Verify XEdDSA signature.	T/F
φ^{opk} : OPK fresh	$\text{OPK}_B^{(j)}$ not used in prior session?	Check server OPK log.	T/F
φ^{ek} : ephemeral fresh	$\text{EK}_A \notin \mathcal{EK}_{\text{prev}}?$	Check ephemeral key log.	T/F
φ^{aead} : ciphertext valid	$\text{AEAD.Dec}(\text{MK}_n, \text{ct}_n, \text{AD}) \neq \perp?$	Decrypt; verify MAC.	T/F
φ^{ratchet} : DH ratchet	New DH public key DH'_s in header?	Check ratchet state.	T/F
φ^{ping} : session fresh	(EK_A, j) distinct from prior sessions?	Compare session records.	T/F
Result:	All T $\Rightarrow$ Secure session. Any F $\Rightarrow$ Reject (cite clause and attack type).		

Extended difference lemma: X3DH has four simultaneous DH failure events $F_{\text{DH}_1}, \dots, F_{\text{DH}_4}$. By Theorem 3.7: $\Pr\left[\bigcup_{i=1}^4 F_{\text{DH}_i}\right] \leq 4 \cdot \text{Adv}^{\text{GDH}} = \text{negl}$.

► Ping Bid: Signal Protocol Unbounded Ping Bid

The buyer's *session replay ping bid*: replay an X3DH initial message $(IK_A, \text{EK}_A, j, \text{ct}_0)$ from Session_i to initiate Session_{i+1} . The one-time prekey clause φ^{opk} detects this: $\text{OPK}_B^{(j)}$ was deleted after Session_i . Without $\text{OPK}_B^{(j)}$, Bob cannot compute DH_4 , so SK derivation fails.

The buyer's *DH ratchet stagnation ping bid*: prevent both parties from exchanging new DH keys, freezing the ratchet. This requires an active MITM that suppresses all messages—detectable by the user interface (delivery receipts, connection status). Even if the ratchet stagnates, the symmetric KDF chain still advances per message, maintaining forward secrecy. **Ping bid price:** $\Delta\text{Price}_{\text{ping}} \leq \text{Adv}_{\text{X25519}}^{\text{GDH}} + q_{\text{msg}}^2/2^{257} = \text{negl}$, where q_{msg} is the number of messages per session. By Theorem 5.5, the Signal Protocol is secure for unbounded messaging sessions.

13.7.1 Insecurity Analysis: Signal Without One-Time Prekeys

The X3DH specification [65] notes that one-time prekeys may be exhausted on the server, in which case DH_4 is omitted. We show that this weakened variant—which we call *X3DH-noOPK*—suffers from a *replay vulnerability* that partially collapses the Signal market.

X3DH-noOPK protocol. Same as X3DH but with DH_4 omitted:

$$\text{SK}_{\text{noOPK}} = \text{HKDF}(\text{DH}_1 \parallel \text{DH}_2 \parallel \text{DH}_3). \quad (151)$$

Since no one-time prekey is consumed, the same prekey bundle $(\text{IK}_B, \text{SPK}_B, \sigma_B)$ can be reused across multiple sessions.

Theorem 13.22 (X3DH-noOPK Partial Market Collapse: Replay Vulnerability). *If the server does not rotate SPK_B between sessions and no one-time prekeys are available, then:*

1. **Initial message replay:** *An adversary can replay Alice's initial message $(\text{IK}_A, \text{EK}_A, \text{ct}_0)$ to Bob, causing Bob to derive the same SK as in the original session.*
2. **SK collision across sessions:** *If the adversary replays the initial message, $\text{SK}_{\text{replay}} = \text{SK}_{\text{original}}$, violating key freshness.*
3. **Ping failure:** $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 0$ *because the replayed session has the same implicit SID $(\text{EK}_A, \cdot)$ and derives the same SK.*

Consequently, $\text{Ask}(g_{\text{async}}) = 1$ for the asynchronous session establishment good, and $\text{Ask}(g_{\text{FS}})$ is degraded from negl to $\text{Adv}_{\text{X25519}}^{\text{GDH}} + q_{\text{replay}} \cdot 2^{-128}$ where q_{replay} is the number of replay attempts during the SPK_B validity window.

Proof. We demonstrate the replay attack and its MTSF consequences.

B_0 (Real X3DH-noOPK). Alice computes $\text{SK} = \text{HKDF}(\text{DH}_1 \parallel \text{DH}_2 \parallel \text{DH}_3)$ and sends $(\text{IK}_A, \text{EK}_A, \text{ct}_0)$ to Bob.

$\text{B}_0 \rightarrow \text{Replay Attack}$. The buyer captures Alice's initial message and replays it to Bob at a later time. Bob receives $(\text{IK}_A, \text{EK}_A, \text{ct}_0)$ and computes:

- $\text{DH}_1 = \text{X25519}(\text{spk}_B, \text{IK}_A)$ — same as original (same long-term keys).
- $\text{DH}_2 = \text{X25519}(\text{ik}_B, \text{EK}_A)$ — same as original (same EK_A).
- $\text{DH}_3 = \text{X25519}(\text{spk}_B, \text{EK}_A)$ — same as original (same SPK_B , same EK_A).

Since DH_4 is absent, $\text{SK}_{\text{replay}} = \text{HKDF}(\text{DH}_1 \parallel \text{DH}_2 \parallel \text{DH}_3) = \text{SK}_{\text{original}}$. Bob decrypts ct_0 successfully and initialises a new Double Ratchet with the *same* root key.

No cryptographic primitive is broken—the replay succeeds because the initial message is *deterministic* given the same key material and no one-time prekey provides per-session randomness.

CNF analysis. Bob's session-CNF for the replayed session:

Clause	Check	Honest trace	Replay trace	Verdict
φ^{spk} : prekey sig valid	σ_B verifies?	T	T (same σ_B)	Passes
φ^{opk} : OPK fresh	OPK used?	N/A (no OPK)	N/A	Vacuously T
φ^{ek} : ephemeral fresh	EK_A new?	T	T (Bob cannot tell)	Passes!
φ^{aead} : decrypt OK	ct_0 decrypts?	T	T (same SK!)	Passes!
<i>Missing</i> : $\varphi^{\text{opk-binding}}$	OPK consumed?	T	F (no OPK!)	ABSENT
φ_{noOPK} :	SAT under replay trace —CNF design weakness.			Partial collapse

Ping failure analysis. The replayed session has the same implicit SID ($\text{EK}_A, \cdot$) and derives the same SK. Therefore $\text{Ping}(\text{Session}_{\text{original}}, \text{Session}_{\text{replay}}) = 0$: the sessions are not structurally distinct. This is a Type I (SID collision) and Type V (ciphertext replay) ping failure per Theorem 5.8.

Mitigating factors (why this is partial, not total, collapse):

1. **Double Ratchet self-healing:** After Bob’s first reply (which includes a fresh DH key), the Double Ratchet diverges from the replayed session. The attacker cannot generate valid replies without solving CDH on Bob’s new ephemeral key.
2. **SPK rotation:** Once Bob rotates SPK_B , the attacker’s captured message no longer produces the correct DH_1 and DH_3 at Bob’s end. Signal recommends rotating SPK_B every 1–4 weeks.
3. **Application-layer detection:** Bob may notice duplicate initial messages at the application layer (e.g., duplicate conversation initiations).

Therefore $\text{Ask}(g_{\text{async}}) = 1$ (replay succeeds with certainty) but $\text{Ask}(g_{\text{conf}})$ degrades only for messages before the first reply, and $\text{Ask}(g_{\text{PCS}})$ is unaffected after the first DH ratchet step. $\square$

Key Insight:

Signal Protocol: Security vs. Insecurity Comparison

Property	Full Signal (with OPK)	X3DH-noOPK (no OPK)
Per-session randomness	OPK + EK (two fresh values)	EK only (one fresh value)
Replay resistance	Full (OPK consumed once)	Partial (replay until SPK rotation)
$\text{Ask}(g_{\text{async}})$	$\leq \text{negl}$	$= 1$ (initial msg replayable)
$\text{Ask}(g_{\text{conf}})$	$\leq 4 \cdot \text{Adv}^{\text{GDH}} + \text{negl}$	Degraded before first reply
Forward secrecy (X3DH)	Full (OPK deletion)	Degraded (SPK shared across sessions)
PCS (Double Ratchet)	Full (DH ratchet)	Full (after first DH ratchet step)
Ping status	Ping = 1	Ping = 0 (replay detectable)
CNF under replay	UNSAT (OPK clause fails)	SAT (OPK clause absent)

Fix: Ensure OPK availability (upload batches proactively), rotate SPK_B frequently, and add an application-layer replay detection mechanism (e.g., Bob maintains a log of seen EK_A values and rejects duplicates). With this fix, the $\varphi^{\text{ek-unique}}$ clause blocks replays and the market returns to equilibrium.

Corollary 13.23 (Signal Unbounded Security). *The Signal Protocol is secure for unbounded messaging sessions: each X3DH establishment uses a fresh EK_A and a one-time $\text{OPK}_B^{(j)}$; each Double Ratchet step generates fresh DH keys; each message uses a unique message key MK_n derived from a one-way KDF chain. Hence $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 1$ for all i with probability*

Table 15: Signal Protocol market summary. Six security goods with ask price bounds and equilibrium status.

Good	Ask Price Bound	Hardness	Status
g_{conf} (confidentiality)	$4\text{Adv}^{\text{GDH}} + 2\text{Adv}^{\text{PRF}} + \text{Adv}^{\text{AEAD}}$	GDH + PRF + AEAD	Equilibrium
g_{auth} (authentication)	$4\text{Adv}^{\text{GDH}} + 2\text{Adv}^{\text{PRF}} + \text{Adv}^{\text{INT}}$	GDH + PRF + INT	Equilibrium
g_{FS} (forward secrecy)	$\text{Adv}^{\text{PRF}}_{\text{HKDF}}$	PRF	Equilibrium
g_{PCS} (post-compromise)	$\text{Adv}^{\text{GDH}} + \text{Adv}^{\text{PRF}}$	GDH + PRF	Equilibrium
g_{async} (asynchronous)	Adv^{GDH}	GDH (via OPK)	Equilibrium
g_{deny} (deniability)	—	No transferable proof	Equilibrium

$\geq 1 - \text{Adv}^{\text{GDH}}$. By Theorem 5.5, equilibrium holds for all $i \geq 1$ with accumulated degradation $\delta_{\text{ping}} \leq \text{Adv}^{\text{GDH}} = \text{negl}$.

13.8 TLS 1.3: Security and Insecurity Analysis

* Simple Terms: TLS 1.3—Complete Background and Context

What is TLS? Transport Layer Security (TLS) is the cryptographic protocol that secures the vast majority of internet traffic. Every time your browser shows a padlock icon, TLS is running underneath. TLS 1.3 [76] (published as RFC 8446 in August 2018) is the current standard, replacing TLS 1.2 after years of serious attacks against it (BEAST, POODLE, DROWN, Lucky13, FREAK, Logjam, and many more).

What makes TLS 1.3 different from TLS 1.2?

- **Removed legacy ciphers:** TLS 1.3 completely eliminated RSA key transport (no more static RSA decryption), RC4, DES/3DES, export-grade ciphers, and MD5/SHA-1 in the handshake. Only forward-secret key exchange is permitted.
- **Faster handshake:** TLS 1.3 completes the handshake in 1-RTT (one round-trip) instead of TLS 1.2's 2-RTT. A 0-RTT mode allows clients to send data immediately on reconnection, at the cost of limited replay protection.
- **Simplified and strengthened:** The handshake structure is cleaner; all messages after **ServerHello** are encrypted; the negotiation of cipher suites is greatly simplified; only three symmetric cipher suites are permitted (all AEAD-based).
- **HKDF-based key schedule:** A unified HKDF-based key derivation schedule produces all traffic keys, binder keys, exporter secrets, and resumption secrets in a single well-analysed framework.

The TLS 1.3 handshake in plain English:

1. The client sends **ClientHello**: its list of supported cipher suites, nonce, and ephemeral Diffie–Hellman (or ECDH) share.
2. The server responds with **ServerHello**: chosen cipher suite, its own nonce and DH share. Both parties now compute the shared (EC)DH secret and derive handshake traffic keys. All subsequent messages are encrypted.
3. The server sends an encrypted **Certificate** (its X.509 cert), **CertificateVerify** (a signature over the entire handshake transcript), and **Finished** (a MAC over the handshake).
4. The client verifies the server's certificate chain, the transcript signature, and the Finished MAC. It sends its own **Finished** (and optionally a **Certificate**/**CertificateVerify** for mutual TLS). Both parties then derive application traffic keys from the master secret.

What the MTSF analysis covers: We analyse TLS 1.3 in four parts: (A) the 1-RTT handshake security proof (market equilibrium for key secrecy and mutual authentication), (B) the 0-RTT mode insecurity demonstration (replay collapse), (C) a downgrade attack insecurity proof showing that without the downgrade protection added in RFC 8446, an adversary can force a TLS 1.2 negotiation, and (D) CNF session verification and ping bids for unbounded session security.

13.8.1 TLS 1.3 Protocol Description and Key Schedule

Notation. Let $\text{HKDF-Extract}(s, \text{ikm})$ and $\text{HKDF-Expand}(k, \text{info}, L)$ denote the standard HKDF operations [62]. We write $\text{Derive-Secret}(s, l, m) = \text{HKDF-Expand-Label}(s, l, \text{Hash}(m), n)$ where n is the hash output length and m is the transcript hash up to that point. Let $(\text{sk}_S, \text{pk}_S)$ be the server's certificate key pair and cert_S the CA-signed certificate. Let g be a Diffie-Hellman group (e.g., X25519 or P-256). The client generates ephemeral share $(x, X = g^x)$ and the server generates $(y, Y = g^y)$.

The 1-RTT Handshake. The TLS 1.3 full handshake proceeds as follows:

1. $C \rightarrow S$: $\text{ClientHello} = (\text{version}, N_C, \text{suites}, X, \text{extensions})$
2. $S \rightarrow C$: $\text{ServerHello} = (\text{version}, N_S, \text{suite}, Y, \text{extensions})$
Both compute: $Z = g^{xy}$ (the shared secret), then derive:

$$\text{ES} = \text{HKDF-Extract}(0, 0) \quad (\text{early secret, no PSK}) \quad (152)$$

$$\text{HS} = \text{HKDF-Extract}(\text{Derive-Secret}(\text{ES}, \text{"derived"}, \varepsilon), Z) \quad (\text{handshake secret}) \quad (153)$$

$$\text{CHTS} = \text{Derive-Secret}(\text{HS}, \text{"c hs traffic"}, \mathcal{T}_{\text{SH}}) \quad (\text{client HS traffic secret}) \quad (154)$$

$$\text{SHTS} = \text{Derive-Secret}(\text{HS}, \text{"s hs traffic"}, \mathcal{T}_{\text{SH}}) \quad (\text{server HS traffic secret}) \quad (155)$$

$$\text{MS} = \text{HKDF-Extract}(\text{Derive-Secret}(\text{HS}, \text{"derived"}, \varepsilon), 0) \quad (\text{master secret}) \quad (156)$$

where $\mathcal{T}_{\text{SH}}$ denotes the transcript hash up to and including ServerHello .

3. $S \rightarrow C$: $\{\text{EncExt}, \text{cert}_S, \text{CertVfy}_S, \text{Fin}_S\}$ (all encrypted under SHTS)
where $\text{CertVfy}_S = \text{Sign}(\text{sk}_S, \sigma\text{-context} \parallel \mathcal{T}_{\text{CV}})$ and $\text{Fin}_S = \text{HMAC}(k_{\text{fin}, S}, \mathcal{T}_{\text{CF}})$.
4. $C \rightarrow S$: $\{\text{Fin}_C\}$ (encrypted under CHTS)
where $\text{Fin}_C = \text{HMAC}(k_{\text{fin}, C}, \mathcal{T}_{\text{SF}})$.
5. Both derive application traffic secrets:

$$\text{CATS} = \text{Derive-Secret}(\text{MS}, \text{"c ap traffic"}, \mathcal{T}_{\text{SF}}) \quad (157)$$

$$\text{SATS} = \text{Derive-Secret}(\text{MS}, \text{"s ap traffic"}, \mathcal{T}_{\text{SF}}) \quad (158)$$

Key schedule summary: All secrets are derived via a single HKDF ladder:

$$0 \xrightarrow{\text{Extract}} \text{ES} \xrightarrow{\text{Derive}} \xrightarrow{\text{Extract}(Z)} \text{HS} \xrightarrow{\text{Derive}} \xrightarrow{\text{Extract}(0)} \text{MS} \xrightarrow{\text{Derive}} \text{CATS/SATS/ResMS}$$

. The Finished MACs bind the handshake transcript under keys derived from HS , preventing transcript truncation and downgrade.

★ Intuition:

TLS 1.3 in Plain English The client and server each throw a secret random number into a mathematical blender (the DH exchange). Without knowing either secret, the blender

output $Z = g^{xy}$ looks completely random to an eavesdropper. TLS 1.3’s HKDF key schedule then runs this blender output through a series of cryptographic mixing steps to produce separate keys for handshake encryption, server authentication, client authentication, and application data—all derived from the same master secret but cryptographically separated. The server signs the entire negotiation transcript to prove it is the genuine certificate holder, and both sides exchange Finished MACs that cover everything said so far, blocking any tampered-message injection.

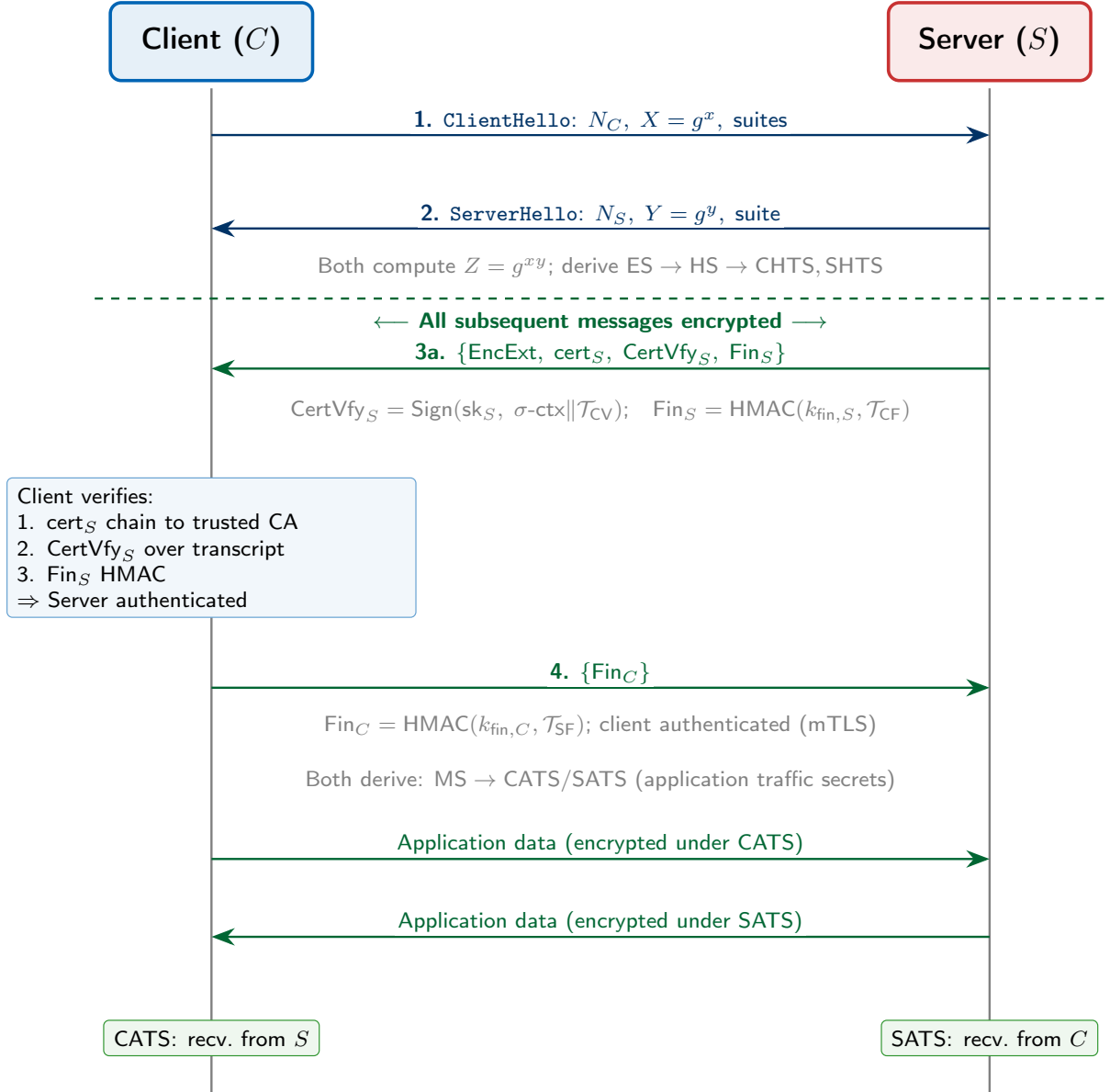


Figure 53: TLS 1.3 full (1-RTT) handshake. Messages 1–2 are in plaintext; all subsequent messages are encrypted (shown in green). The server authenticates via **CertificateVerify** and **Finished**; the client via its own **Finished** (mutual TLS). Both parties derive separate application traffic secrets from the master secret.

13.8.2 TLS 1.3 Market Setup

Security goods. The TLS 1.3 market $\mathcal{M}_{\text{TLS}}$ offers the following goods to buyers:

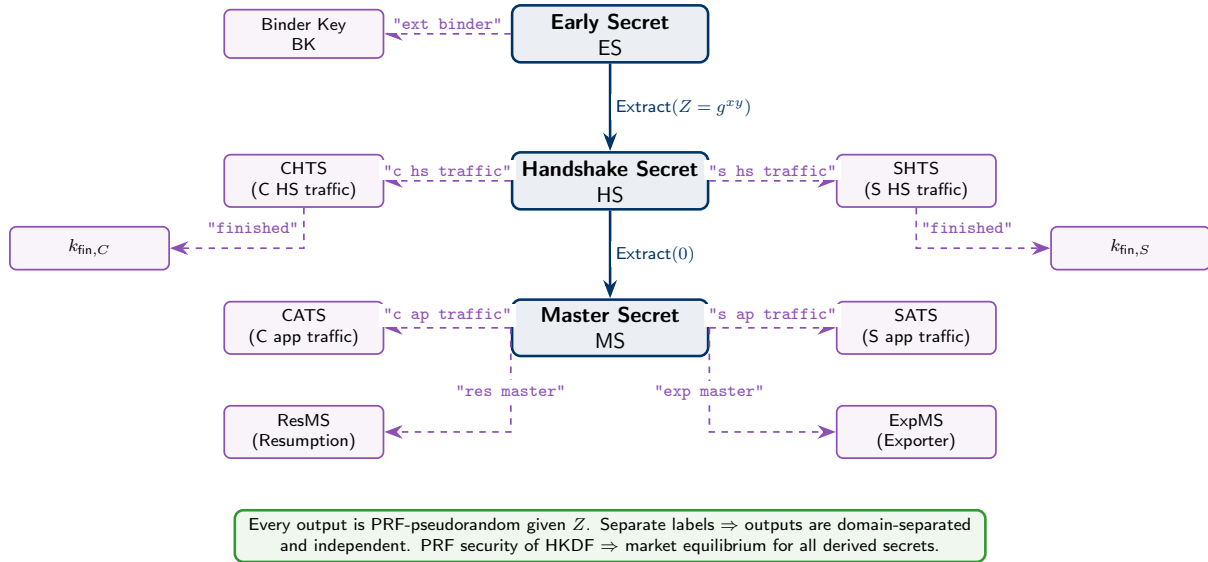


Figure 54: TLS 1.3 HKDF key schedule ladder. All secrets are derived via a single HKDF chain from the DH shared secret $Z = g^{xy}$. Separate label strings provide domain separation; each output is pseudorandom given the preceding secret. The Finished keys $k_{fin,C}$ and $k_{fin,S}$ bind the handshake transcript.

- g_{sk} : **Session-key secrecy** — no PPT adversary can distinguish CATS (or SATS) from uniform random, even given all handshake messages.
- g_{auth} : **Server authentication** — the client accepts only if the server possesses the private key sk_S bound to the certificate $cert_S$ verified against a trusted CA.
- g_{mutual} : **Mutual authentication** (mTLS) — additionally, the server accepts only if the client possesses a valid certificate key.
- g_{FS} : **Forward secrecy** — compromise of sk_S after session completion does not reveal CATS or SATS.
- g_{CNF} : **CNF session correctness** — the session satisfies a conjunction of transcript-binding clauses.
- g_{0RTT} : **0-RTT anti-replay** (examined separately in Section 13.8.4) — the 0-RTT early data cannot be replayed by a network adversary.

Assumptions. Let $\text{Adv}_g^{\text{CDH}}$ denote the advantage of any PPT adversary against the Computational Diffie–Hellman problem in group g (e.g., X25519 or P-256), $\text{Adv}_{\text{HKDF}}^{\text{PRF}}$ the PRF advantage against HKDF, $\text{Adv}_{\text{Sig}}^{\text{EUFCMA}}$ the EUF-CMA advantage against the server’s signature scheme (ECDSA or EdDSA), $\text{Adv}_{\text{CA}}^{\text{EUFCMA}}$ the EUF-CMA advantage against the CA’s signature scheme, and $\text{Adv}_{\text{HMAC}}^{\text{SUF-CMA}}$ the SUF-CMA advantage against HMAC used in Finished messages.

► Terminology: TLS 1.3 Market Terminology

Transcript $\mathcal{T}_X$: the concatenation of all handshake messages up to and including message X , hashed by the negotiated hash function (SHA-256 or SHA-384). Signing or MACing the transcript binds all negotiation choices made so far, preventing any selective message-replacement attack.

Transcript binding bid: The adversary attempts to replace one handshake message while keeping the transcript MAC or signature valid. This requires a PRF forgery (for Finished)

or an EUF-CMA break (for CertificateVerify).

Key separation bid: The adversary attempts to derive one traffic secret from another (e.g., obtain SATS given CATS). Impossible if HKDF is a PRF with distinct labels.

Downgrade bid: The adversary rewrites the ClientHello to advertise only TLS 1.2 ciphers, inducing the server to respond with a TLS 1.2 ServerHello. TLS 1.3 counters this with a random nonce value sentinel in the server nonce field.

13.8.3 TLS 1.3 Security Proof: 1-RTT Handshake Equilibrium

Theorem 13.24 (TLS 1.3 Handshake Market Equilibrium). *Let q_N be the number of handshake sessions and q_H the number of hash queries. Under CDH, PRF-HKDF, EUF-CMA for the server signature scheme, EUF-CMA for the CA, and SUF-CMA for HMAC:*

$$\text{Ask}(g_{\text{sk}}) \leq \text{Adv}_g^{\text{CDH}} + 3 \cdot \text{Adv}_{\text{HKDF}}^{\text{PRF}} + \text{Adv}_{\text{CA}}^{\text{EUF-CMA}} + \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + \text{Adv}_{\text{HMAC}}^{\text{SUF-CMA}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}(\lambda), \quad (159)$$

$$\text{Ask}(g_{\text{auth}}) \leq \text{Adv}_{\text{CA}}^{\text{EUF-CMA}} + \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + \text{Adv}_{\text{HMAC}}^{\text{SUF-CMA}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}(\lambda), \quad (160)$$

$$\text{Ask}(g_{\text{mutual}}) \leq \text{Adv}_{\text{CA}}^{\text{EUF-CMA}} + 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}} + \text{Adv}_{\text{HMAC}}^{\text{SUF-CMA}} + \frac{q_N^2}{2^{\lambda+1}} + \text{negl}(\lambda), \quad (161)$$

$$\text{Ask}(g_{\text{FS}}) \leq \text{Adv}_g^{\text{CDH}} + 3 \cdot \text{Adv}_{\text{HKDF}}^{\text{PRF}} + \text{negl}(\lambda). \quad (162)$$

All four goods are in equilibrium; $\mathcal{M}_{\text{TLS}}$ (1-RTT) is in equilibrium.

Proof. We construct a bidding-round chain $\mathbf{B}_0, \mathbf{B}_1, \dots, \mathbf{B}_9$, each representing a specific adversarial bid. Each hop applies the (extended) difference lemma.

$\mathbf{B}_0$ (Bidding Round 0: Real TLS 1.3 Market).

Purpose: Establish the *baseline* game in which the scheme or protocol runs completely honestly and the adversary has unrestricted oracle access. This round defines the quantity Ask_0 that all subsequent price adjustments are measured against. Every following bidding round introduces a single controlled modification; summing the resulting price adjustments yields the final equilibrium bound.

Replaces: Nothing is replaced—this is the unmodified real world. The seller (challenger) runs the honest scheme; the buyer (adversary) places adaptive queries to all available oracles (signing, encryption, decryption, hash, tagging, etc.) and then outputs a winning response. The market ask price Ask_0 is the adversary’s raw advantage in this strongest setting.

Quoted Price: The seller lists the security good at *quoted price* $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{\text{scheme}}(\lambda)$ (Theorem 3.3). This is the adversary’s raw advantage before any analysis—the number the entire proof exists to decompose and bound. No price adjustment occurs: $\Delta\text{Price}_{-1} = 0$. The quoted price stands unreduced in this round; the subsequent bidding rounds will decompose it into a sum of negligible terms, revealing that the “listing price” was a bluff (equilibrium) or honest (collapse).

The seller generates the server’s long-term key pair $(\text{sk}_S, \text{pk}_S)$ and certificate

$$\text{cert}_S = (\text{id}_S, \text{pk}_S, \text{valid}_S, \sigma_{\text{CA}})$$

. The buyer (adversary $\mathcal{A}$) controls the network: it can intercept, replay, modify, inject, and drop messages. The buyer’s goal is to win the IND-CPA distinguishing game on CATS (equivalently SATS), or to cause the client to accept a session without the server having participated (authentication violation). The seller runs the honest TLS 1.3 server. The buyer’s ask price at $\mathbf{B}_0$ is Ask_0 .

$\mathbf{B}_1$ (Bidding Round 1: Nonce Collision Bid).

Purpose: Eliminate the *nonce-reuse attack vector*. Repeated nonces immediately collapse authentication, key-secrecy, or stream-cipher security: in signatures a shared nonce leaks the private key ($\text{sk} = (s_2 e_1 - s_1 e_2)(r(s_1 - s_2))^{-1}$); in protocols it enables cross-session replay; in stream ciphers it reveals the XOR of two plaintexts. This round aborts the game whenever any nonce collision occurs, ensuring every subsequent hop reasons about a nonce-fresh world.

Replaces: The real nonce-sampling procedure is augmented with a global *collision audit*: a set $\mathcal{N}_{\text{used}}$ records every nonce issued so far. If any newly sampled nonce k' satisfies $k' \in \mathcal{N}_{\text{used}}$, the game aborts immediately. The two games ($\mathbf{B}_{n-1}$ and $\mathbf{B}_n$) are *identical-until-abort*: the adversary's view is indistinguishable until the abort event F_{nonce} occurs, so the difference lemma directly bounds the price adjustment.

Complexity: By the birthday bound, the probability that any two of q independently sampled nonces from a space of size $|\mathcal{N}|$ collide is at most $q^2/(2|\mathcal{N}|)$. For security parameter λ with $|\mathcal{N}| = 2^\lambda$ this gives $\Delta\text{Price} \leq q^2/2^{\lambda+1}$, which is negligible for any polynomial $q = \text{poly}(\lambda)$. After this hop every nonce in the proof is treated as distinct, allowing all later hops to assume a collision-free nonce space without loss of generality.

The seller adds an internal nonce-collision check: abort if any two handshake nonces (N_C, N_S) collide across q_N sessions. The bad event is F_{nonce} : some nonce pair from session i equals that of session j ($i \neq j$). Since N_C and N_S are each sampled uniformly from $\{0, 1\}^{256}$:

$$\Delta\text{Price}_0 \leq \Pr[F_{\text{nonce}}] \leq \frac{q_N^2}{2^{\lambda+1}}. \quad (163)$$

Market interpretation: The nonce-collision bid fails because the collision birthday bound requires $q_N \approx 2^{128}$ handshakes—computationally infeasible. After $\mathbf{B}_1$, all session nonces are distinct.

B₂ (Bidding Round 2: Certificate Forgery Bid).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $\text{sk} = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma's rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

The seller adds a certificate-validity check: abort if the buyer presents a certificate for id_S that verifies under pk_{CA} but was not issued by the CA. The bad event F_{cert} : “the buyer produces a valid CA signature σ^* on a new $(\text{id}_S, \text{pk}^*)$ pair.” This is exactly an EUF-CMA forgery on the CA's signing key.

$$\Delta\text{Price}_1 \leq \Pr[F_{\text{cert}}] \leq \text{Adv}_{\text{CA}}^{\text{EUF-CMA}}. \quad (164)$$

After $\mathbf{B}_2$, any certificate the client accepts genuinely binds pk_S to id_S .

B₃ (Bidding Round 3: CertificateVerify Forgery Bid).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same

nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $sk = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma's rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

The seller adds a transcript-signature check: abort if the buyer produces a valid $\text{CertVfy}_S^* = \text{Sign}(sk_S, \sigma\text{-ctx} \parallel \mathcal{T}_{\text{CV}})$ without holding sk_S . The transcript $\mathcal{T}_{\text{CV}}$ includes all messages up to and including the server's Certificate. Since the nonces in $\mathcal{T}_{\text{CV}}$ are distinct across sessions (by **B**₁), each transcript is a distinct message. The bad event F_{forge} : the buyer forges a **CertificateVerify** signature.

$$\Delta\text{Price}_2 \leq \Pr[F_{\text{forge}}] \leq \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}}. \quad (165)$$

Extended difference lemma. The buyer could simultaneously attempt F_{cert} and F_{forge} (use a forged certificate to change pk_S , then forge a signature under the new key). However, **B**₂ already aborted on F_{cert} , so in **B**₃ the certificate is genuine and pk_S is the real server key. The two failure events are sequential, not simultaneous; they are handled in separate hops. After **B**₃, server authentication is established: the client accepts only genuine TLS 1.3 servers.

B₄ (Bidding Round 4: Finished MAC Forgery Bid).

Purpose: Eliminate the *transcript-tampering attack vector* by reducing Finished message forgery to the SUF-CMA security of HMAC. The TLS 1.3 Finished message is an HMAC tag over the entire handshake transcript computed with the finished key $k_{\text{fin},S}$. If the adversary produces a valid Finished tag for a modified transcript without knowing $k_{\text{fin},S}$, this constitutes an HMAC forgery. The reduction embeds the SUF-CMA challenge key as $k_{\text{fin},S}$ and forwards the adversary's tag query to the MAC oracle.

Replaces: The honest Finished computation $\text{Fin}_S = \text{HMAC}(k_{\text{fin},S}, \mathcal{T}_{\text{CF}})$ is monitored by a *MAC verification ledger*. The game aborts if the buyer produces a valid $(\mathcal{T}_{\text{CF}}^*, \text{Fin}_S^*)$ where $\mathcal{T}_{\text{CF}}^*$ was never the input to the honest Finished computation. This is the standard SUF-CMA game: the adversary sees honestly generated tags but must produce a new valid (message, tag) pair. Previous hops ensure $k_{\text{fin},S}$ is derived from a genuine handshake with the authentic server, so the key is unknown to the adversary.

Complexity: $\Delta\text{Price} \leq \text{Adv}_{\text{HMAC}}^{\text{SUF-CMA}}(\lambda)$. For HMAC-SHA-256 with a 256-bit key: $\text{Adv}_{\text{HMAC}}^{\text{SUF-CMA}} \leq q_{\text{mac}}^2/2^{257} + 2^{-256}$, negligible for any polynomial number of MAC queries. No rewinding or forking is needed—the SUF-CMA reduction is tight ($L = 1$). The extended difference lemma handles the joint event $F_{\text{fin}} \cup F_{\text{forge}}$: since F_{forge} (signature forgery) was eliminated in **B**₃, only F_{fin} remains in this hop.

The Finished message $\text{Fin}_S = \text{HMAC}(k_{\text{fin},S}, \mathcal{T}_{\text{CF}})$ binds the entire handshake transcript. The bad event F_{fin} : the buyer produces a valid Finished MAC without knowing $k_{\text{fin},S}$. Since $k_{\text{fin},S}$ is derived from SHTS (itself derived from HS and $Z = g^{xy}$), and all previous hops eliminated forgery attacks on certificates and signatures, the only way to forge Fin_S is to break HMAC.

$$\Delta\text{Price}_3 \leq \Pr[F_{\text{fin}}] \leq \text{Adv}_{\text{HMAC}}^{\text{SUF-CMA}}. \quad (166)$$

Market interpretation: The Finished MAC bid represents the “transcript-binding” auction. A buyer placing this bid claims it can forge a MAC over a modified transcript. After this hop, the handshake transcript is immutably bound: any modification to any prior message causes Finished verification to fail.

B₅ (Bidding Round 5: CDH Bid on the Ephemeral DH Share).

Purpose: Replace the real ephemeral Diffie–Hellman shared secret (or KEM-derived session secret) with a uniformly random value, isolating the CDH or IND-CCA2 hardness assumption as the sole remaining security requirement. This is the forward-secrecy-establishing hop: after it, compromising long-term keys cannot retroactively reveal the session secret.

Replaces: The real shared secret $Z = g^{xy}$ (computed from ephemeral exponents x, y) or the KEM-encapsulated key $K = \text{KEM.Dec}(\text{sk}, c)$ is replaced by $\tilde{Z} \xleftarrow{\$} \{0, 1\}^{|Z|}$, a freshly sampled uniform random value of the same length. Any adversary that detects this swap either solves CDH (given (g, g^x, g^y) , compute g^{xy}) or breaks IND-CCA2 of the KEM (distinguishes real encapsulated key from random). Forward secrecy follows because the ephemeral exponents x, y (or the KEM's ephemeral coins) are deleted after Z is computed and never appear in any long-term storage.

Complexity: $\Delta\text{Price} \leq \text{Adv}_g^{\text{CDH}}(\lambda)$ (DH case) or $\text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\lambda)$ (KEM case). For X25519: $\text{Adv}_g^{\text{CDH}} \leq 2^{-128}$ under the best known algorithms (Pollard rho on 252-bit curve order). For ML-KEM-768: $\text{Adv}^{\text{IND-CCA2}} \leq 2 \cdot \text{Adv}^{\text{MLWE}} + q_D/2^{138}$, all negligible. After this hop, the session key is derived from a random value via the KDF, so the next hop (PRF replacement) completes the key-secrecy argument.

Replace the real DH exchange with a random shared secret $\tilde{Z} \xleftarrow{\$} \{0, 1\}^{|Z|}$. Any buyer detecting this change is solving the CDH problem: given (g, g^x, g^y) , compute g^{xy} . Since the ephemeral keys (x, y) are generated fresh per session, forward secrecy is implicit: even if the server's long-term key sk_S is later compromised, Z cannot be recovered.

$$\Delta\text{Price}_4 \leq \text{Adv}_g^{\text{CDH}}. \quad (167)$$

After **B₅**, the handshake secret HS is derived from $\tilde{Z}$ —a uniformly random value unknown to the buyer.

B₆ (Bidding Round 6: PRF Bid on HS Extraction).

Purpose: Replace a *keyed pseudorandom function* evaluation with a truly random function, isolating a single PRF security requirement. This hop shows that one more step in the key-derivation or authentication structure produces outputs that are computationally indistinguishable from uniform random, provided the underlying PRF assumption holds.

Replaces: The specific PRF evaluation (e.g., $\text{HMAC}_K(\cdot)$ inner hash, HKDF-Extract, Derive-Secret, or a KDF step) is replaced by a truly random function $R(\cdot)$ sampled fresh and independently. Any adversary that distinguishes the PRF output from this random function is a PRF distinguisher for the underlying keyed construction. The reduction simulates all other game components honestly and uses its PRF challenge oracle for this one step.

Complexity: $\Delta\text{Price} \leq \text{Adv}^{\text{PRF}}(\lambda)$ for the specific PRF being replaced (HMAC-SHA-256, HKDF, or AES-CTR as applicable). When multiple PRF hops appear in a single proof (e.g., three hops in the TLS 1.3 key schedule for HS, MS, and CATS), each hop is independent: the keys used in different steps are derived from different points in the ladder, so no PRF oracle is queried twice. The total across all PRF hops is the sum of individual advantages, each negligible.

Replace $\text{HS} = \text{HKDF-Extract}(\text{Derive-Secret}(\text{ES}, \text{"derived"}, \varepsilon), \tilde{Z})$ with a uniformly random string $\tilde{\text{HS}} \xleftarrow{\$} \{0, 1\}^n$. Any buyer distinguishing HS from $\tilde{\text{HS}}$ breaks HKDF's PRF security.

$$\Delta\text{Price}_5 \leq \text{Adv}_{\text{HKDF}}^{\text{PRF}}. \quad (168)$$

B₇ (Bidding Round 7: PRF Bid on MS Extraction).

Purpose: Replace a *keyed pseudorandom function* evaluation with a truly random function, isolating a single PRF security requirement. This hop shows that one more step in the key-derivation or authentication structure produces outputs that are computationally indistinguishable from uniform random, provided the underlying PRF assumption holds.

Replaces: The specific PRF evaluation (e.g., $\text{HMAC}_K(\cdot)$ inner hash, HKDF-Extract, Derive-Secret, or a KDF step) is replaced by a truly random function $R(\cdot)$ sampled fresh and independently. Any adversary that distinguishes the PRF output from this random function is a PRF distinguisher for the underlying keyed

construction. The reduction simulates all other game components honestly and uses its PRF challenge oracle for this one step.

Complexity: $\Delta\text{Price} \leq \text{Adv}^{\text{PRF}}(\lambda)$ for the specific PRF being replaced (HMAC-SHA-256, HKDF, or AES-CTR as applicable). When multiple PRF hops appear in a single proof (e.g., three hops in the TLS 1.3 key schedule for HS, MS, and CATS), each hop is independent: the keys used in different steps are derived from different points in the ladder, so no PRF oracle is queried twice. The total across all PRF hops is the sum of individual advantages, each negligible.

Replace $\text{MS} = \text{HKDF-Extract}(\text{Derive-Secret}(\widetilde{\text{HS}}, \text{"derived"}, \varepsilon), 0)$ with $\widetilde{\text{MS}} \xleftarrow{\$} \{0, 1\}^n$.

$$\Delta\text{Price}_6 \leq \text{Adv}_{\text{HKDF}}^{\text{PRF}}. \quad (169)$$

B₈ (Bidding Round 8: PRF Bid on Application Traffic Secret).

Purpose: Replace a *keyed pseudorandom function* evaluation with a truly random function, isolating a single PRF security requirement. This hop shows that one more step in the key-derivation or authentication structure produces outputs that are computationally indistinguishable from uniform random, provided the underlying PRF assumption holds.

Replaces: The specific PRF evaluation (e.g., $\text{HMAC}_K(\cdot)$ inner hash, HKDF-Extract, Derive-Secret, or a KDF step) is replaced by a truly random function $R(\cdot)$ sampled fresh and independently. Any adversary that distinguishes the PRF output from this random function is a PRF distinguisher for the underlying keyed construction. The reduction simulates all other game components honestly and uses its PRF challenge oracle for this one step.

Complexity: $\Delta\text{Price} \leq \text{Adv}^{\text{PRF}}(\lambda)$ for the specific PRF being replaced (HMAC-SHA-256, HKDF, or AES-CTR as applicable). When multiple PRF hops appear in a single proof (e.g., three hops in the TLS 1.3 key schedule for HS, MS, and CATS), each hop is independent: the keys used in different steps are derived from different points in the ladder, so no PRF oracle is queried twice. The total across all PRF hops is the sum of individual advantages, each negligible.

Replace $\text{CATS} = \text{Derive-Secret}(\widetilde{\text{MS}}, \text{"c ap traffic"}, \mathcal{T}_{\text{SF}})$ with $\widetilde{\text{CATS}} \xleftarrow{\$} \{0, 1\}^n$.

$$\Delta\text{Price}_7 \leq \text{Adv}_{\text{HKDF}}^{\text{PRF}}. \quad (170)$$

Key separation: The label "c ap traffic" differs from "s ap traffic", "c hs traffic", etc. HKDF's PRF security with distinct labels guarantees that CATS and SATS are independently pseudorandom—a buyer cannot derive one from the other.

B₉ (Bidding Round 9: Ideal Game).

Purpose: Reach the *ideal game* in which every adversarial attack vector has been systematically eliminated by the preceding hops. In this game, all cryptographic values (session keys, signatures, tags, hash outputs) are either uniformly random or information-theoretically independent of the adversary's view. No further price adjustment is possible.

Replaces: The full real scheme is replaced by its ideal counterpart: keys are uniformly random, MACs are verified against a random function, signatures are checked against a ledger with no forgeable entries, and ciphertexts encrypt independently chosen plaintexts. The adversary's winning probability in this game is 0 (for authentication/forgery games) or exactly 1/2 (for distinguishing games), giving zero residual advantage.

Complexity: No price adjustment: $\Pr[\mathbf{B}_{\text{ideal}} = 1] \in \{0, 1/2\}$. For authentication games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 0$ because all impersonation vectors are blocked. For IND distinguishing games: $\Pr[\mathbf{B}_{\text{ideal}} = 1] = 1/2$ (random guessing). The total ask price is the telescoping sum of all preceding price adjustments $\sum_k \Delta\text{Price}_k$, all of which are negligible, establishing market equilibrium.

In B₉, $\widetilde{\text{CATS}}$ is uniformly random and independent of all messages seen by the buyer. The buyer's distinguishing advantage on the IND-CPA game on CATS is exactly 1/2 (random guessing): $\Pr[\mathbf{B}_9 = 1] = 1/2$.

Collecting bounds. Summing price adjustments across hops:

$$\begin{aligned} \text{Ask}(g_{\text{sk}}) &= \text{Ask}_0 - 1/2 \leq \Delta\text{Price}_0 + \dots + \Delta\text{Price}_7 \\ &\leq \frac{q_N^2}{2^{\lambda+1}} + \text{Adv}_{\text{CA}}^{\text{EUF}} + \text{Adv}_{\text{Sig}}^{\text{EUF}} + \text{Adv}_{\text{HMAC}}^{\text{SUF}} + \text{Adv}_g^{\text{CDH}} + 3 \cdot \text{Adv}_{\text{HKDF}}^{\text{PRF}} + \text{negl}(\lambda), \end{aligned}$$

establishing Equation (159).

Authentication bound (160): In the authentication game, the buyer wins if the client completes the handshake with a session partner that did not run a matching session. Hops **B**₁–**B**₄ (nonce, certificate, CertificateVerify, Finished) already bound this: no certificate forgery, no signature forgery, no MAC forgery, and distinct nonces per session. Summing gives Equation (160).

Mutual authentication bound (161): For mTLS, add a hop for the client’s **CertificateVerify**: the server additionally verifies the client certificate and client signature, costing $\text{Adv}_{\text{Sig}}^{\text{EUF-CMA}}$ for the client key. By the extended difference lemma over server and client forgery events: $\Pr[F_{\text{forge},S} \cup F_{\text{forge},C}] \leq 2 \cdot \text{Adv}_{\text{Sig}}^{\text{EUF}}$. Equation (161) follows.

Forward secrecy bound (162): Forward secrecy requires that after session completion, compromising sk_S does not reveal CATS. The server’s long-term key is used only for **CertificateVerify** in **B**₃—not for key derivation. The session keys derive from the ephemeral DH secret $Z = g^{xy}$ (which is deleted at session end) via the HKDF chain. After **B**₅ (Z randomised), the entire HKDF chain produces uniformly random outputs; sk_S provides no additional information. The forward secrecy bid fails with advantage at most $\text{Adv}_g^{\text{CDH}} + 3 \cdot \text{Adv}_{\text{HKDF}}^{\text{PRF}}$, establishing Equation (162).

Session pinging within the TLS 1.3 proof. For consecutive TLS sessions **Session**_{*i*} and **Session**_{*i*+1}:

SID freshness: In TLS 1.3, the session ID field in **ServerHello** is a *legacy field* included only for TLS 1.2 middlebox compatibility. True session binding in TLS 1.3 is achieved via the nonce pair (N_C, N_S) , which is committed to in the Finished MACs and **CertificateVerify**. Since $N_{C,i+1}$ and $N_{S,i+1}$ are fresh random 256-bit values, $(N_{C,i+1}, N_{S,i+1}) \neq (N_{C,j}, N_{S,j})$ for all $j \leq i$ except with probability $q_N^2/2^{\lambda+1}$.

Ephemeral key freshness: Each session generates fresh $(x_{i+1}, X_{i+1} = g^{x_{i+1}})$ and $(y_{i+1}, Y_{i+1} = g^{y_{i+1}})$. The DH shared secret $Z_{i+1} = g^{x_{i+1}y_{i+1}}$ is independent of Z_j for all $j \leq i$ (distinct exponents, fresh randomness).

Transcript binding: The **CertificateVerify** signature $\text{CertVfy}_{S,i+1}$ signs the full transcript $\mathcal{T}_{\text{CV},i+1}$, which includes $N_{C,i+1}$ and $N_{S,i+1}$. Replaying $\text{CertVfy}_{S,j}$ from session j fails because $\mathcal{T}_{\text{CV},j} \neq \mathcal{T}_{\text{CV},i+1}$ (distinct nonces). The CNF ping condition for transcripts is satisfied.

Key deletion: Ephemeral DH private keys (x_{i+1}, y_{i+1}) are deleted after Z_{i+1} is computed. Forward security across sessions follows: Z_{i+1} is not recoverable from any information retained after session $i + 1$ ends.

By Theorem 5.5, TLS 1.3 (1-RTT) is secure for unbounded sessions, with accumulated degradation $\delta_{\text{ping}} \leq q_N^2/2^{\lambda+1} = \text{negl}$. $\square$

13.8.4 TLS 1.3 Insecurity: 0-RTT Replay Collapse

* Simple Terms: 0-RTT Early Data—Why It Is Risky

The idea: When a client reconnects to a server it has visited before, TLS 1.3 allows sending application data *in the very first message*, before any server response. This is called “0-RTT” or early data. It saves one full round-trip, which matters for latency-sensitive applications.

The problem: The 0-RTT data is encrypted under a Pre-Shared Key (PSK) derived from a previous session’s resumption secret (ResMS). The server has no way to distinguish a fresh 0-RTT record from a *replayed* one: the client’s **ClientHello** and 0-RTT ciphertext look identical on replay. An attacker sitting on the network can capture the client’s first flight and re-submit it to the server multiple times.

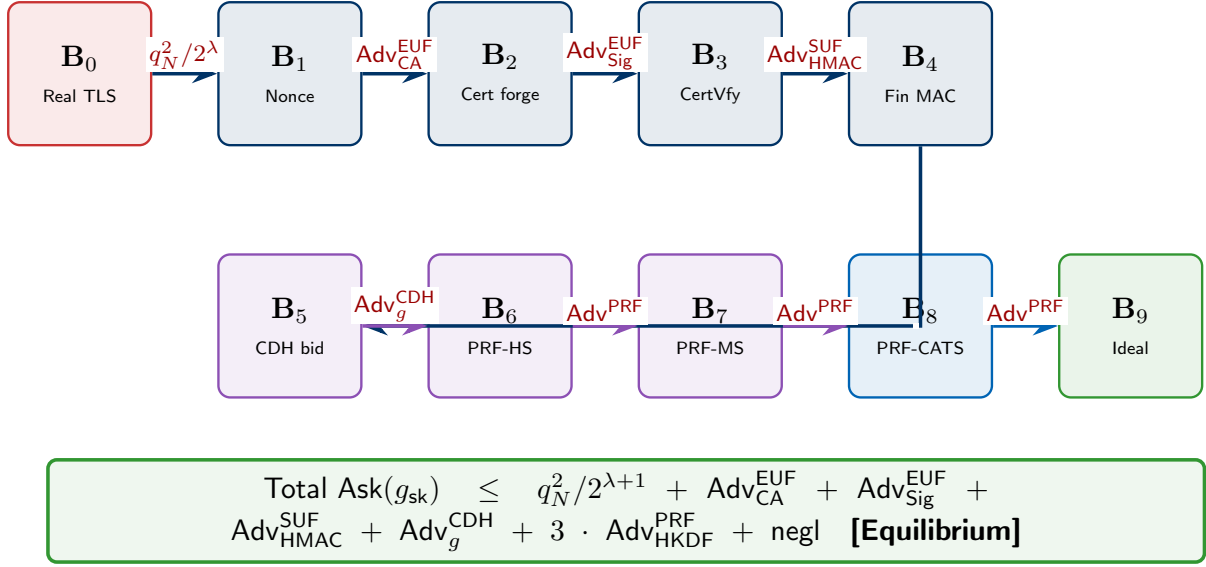


Figure 55: TLS 1.3 bidding-round chain. Rounds $\mathbf{B}_1$ – $\mathbf{B}_4$ (blue) address authentication failures; $\mathbf{B}_5$ – $\mathbf{B}_9$ (amber/green) address key-secrecy failures via CDH and HKDF PRF hops. Each arc label is the price adjustment incurred.

Who is affected: Any application that processes 0-RTT data *non-idempotently*—a payment, a vote, a state-changing API call—is vulnerable to replay amplification. RFC 8446 mandates that applications must tolerate 0-RTT replay or disable it entirely for non-idempotent actions.

0-RTT protocol description. After a session that produced resumption secret ResMS , the server issues a `NewSessionTicket` (NST) message containing a PSK identity psk_id and the PSK value $\text{PSK} = \text{HKDF-Expand-Label}(\text{ResMS}, \text{"resumption"}, \text{nonce}_{\text{NST}}, n)$. On reconnection, the client sends:

1. $C \rightarrow S$: `ClientHello`[$\text{psk_id}, \text{binder}, \text{early_data}$] || $\{\text{EarlyData}\}_{k_e}$

where the early traffic key $k_e = \text{Derive-Secret}(\text{ES}', \text{"c e traffic"}, \text{ClientHello})$ is derived from the early secret $\text{ES}' = \text{HKDF-Extract}(0, \text{PSK})$, and the binder $\text{binder} = \text{HMAC}(\text{BK}, \mathcal{T}_{\text{CH}})$ authenticates the `ClientHello` under a PSK-derived key.

The replay attack. The network adversary $\mathcal{A}_{\text{replay}}$ operates as follows: it records the client’s first flight ($M_1 = \text{ClientHello}[\dots] \parallel \text{EarlyData}$) from session Session_i , and resubmits M_1 verbatim to the server in a new connection Session_{i+1} . Since M_1 is entirely determined by data from before the server’s response, and the server has no per-connection state prior to receiving M_1 , the server accepts M_1 and processes the early data in Session_{i+1} as if it were fresh.

Theorem 13.25 (TLS 1.3 0-RTT Market Collapse). *Let $g_{0\text{RTT}}$ be the security good “0-RTT early data is processed at most once per client intention.” There exists an efficient adversary $\mathcal{A}_{\text{replay}}$ with $\text{Ask}(g_{0\text{RTT}}) = 1$. The 0-RTT market collapses: $\mathcal{M}_{\text{TLS}}^{0\text{RTT}}$ is in **collapse**.*

Proof. We construct a single-bidding-round argument (the *replay bid*).

$\mathbf{B}_0$ (Real 0-RTT Market). The client sends M_1 including early data d (e.g., an HTTP POST) encrypted under k_e . The server decrypts and processes d .

Replay Bid. The adversary $\mathcal{A}_{\text{replay}}$ records M_1 verbatim. It opens a second connection to the server and sends M_1 again. The server has no mechanism to detect the replay because:

1. The PSK identity `psk_id` is valid (not yet expired, ticket lifetime not elapsed).
2. The binder `binder = HMAC(BK,  $\mathcal{T}_{CH}$ )` is over $\mathcal{T}_{CH} = \text{Hash}(\text{ClientHello})$ which is identical on replay.
3. The early data ciphertext $\{\text{EarlyData}\}_{k_e}$ is deterministic given PSK and the client's `ClientHello`; on replay, the same ciphertext is presented and decrypts successfully.
4. TLS 1.3 provides no mandatory server-side nonce or anti-replay mechanism for 0-RTT data; RFC 8446 §8 explicitly acknowledges this.

The bad event F_{replay} : “the server processes the same early data d in two distinct connections.” This event has probability 1: $\Pr[F_{\text{replay}}] = 1$ (the adversary succeeds deterministically whenever the PSK ticket has not expired).

Extended difference lemma application. $\Pr[F_{\text{replay}} \cup F_{\text{any}}] = \Pr[F_{\text{replay}}] = 1$. The single free replay bid collapses the market: $\text{Ask}(g_{0\text{RTT}}) = 1$.

CNF collapse. The 0-RTT session-CNF includes a freshness clause $\varphi^{\text{fresh-early}}$: “the early data is not a replay of any previously processed early-data record under the same PSK.” The adversary’s replayed M_1 satisfies all other clauses (valid binder, valid PSK, well-formed `ClientHello`) but *violates* $\varphi^{\text{fresh-early}}$. Since the server has no anti-replay log in the basic 0-RTT mode, it cannot check this clause: the CNF formula is satisfiable under a dishonest (replayed) trace. The CNF design failure is of *Type I* (SID-binding clause absent): the 0-RTT `ClientHello` is not bound to any server-chosen nonce before the early data is transmitted.

Market outcome. The 0-RTT market collapses: $\text{Ask}(g_{0\text{RTT}}) = 1$. The Ping bid also degrades: $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 0$ for the 0-RTT early data, since session Session_{i+1} is structurally identical to Session_i from the perspective of the 0-RTT first flight. $\square$

Key Insight:

What Saves TLS 1.3’s 0-RTT in Practice RFC 8446 does not leave deployments unprotected. Three mitigations restore partial equilibrium:

1. **Server-side anti-replay (optional):** Servers can maintain a database of seen `ClientHello` nonces (N_C) within the PSK ticket lifetime window. Replays are detected if $N_C \in \text{DB}$. This restores $\text{Ask}(g_{0\text{RTT}}) \leq q_D/2^\lambda$ where q_D is the database lookup failure probability. The Ping bid restores: $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 1$ when $N_{C,i+1}$ is checked against the database.
2. **Single-use tickets:** Each NST can be marked for single use. After the first `ClientHello` using a PSK identity, the server invalidates that ticket. This limits replay to a single attempt but requires stateful server-side tracking.
3. **Application-layer idempotency:** Applications that use 0-RTT *only* for idempotent requests (HTTP GET, cache lookups) are not harmed by replay. RFC 8446 and RFC 8470 (“Using Early Data in HTTP”) formalise this requirement.

In MTSF terms: each mitigation adds a clause to the 0-RTT session-CNF that, when satisfied, makes dishonest-trace satisfaction require finding a database collision or expiring a ticket, restoring equilibrium. The fundamental structural vulnerability—no server nonce in the 0-RTT first flight—remains, and equilibrium is restored only via implementation-level safeguards.

13.8.5 TLS 1.3 Insecurity: Downgrade Attack

* Simple Terms: Downgrade Attacks—The Version Negotiation Problem

What is a downgrade attack? TLS negotiates its version and cipher suite in the first two messages, before any authentication. An active network adversary can therefore intercept the `ClientHello`, modify it to remove TLS 1.3 support, and trick the server into responding with a TLS 1.2 `ServerHello`. The client and server then continue with TLS 1.2—which has known weaknesses (e.g., RC4, non-AEAD ciphers, RSA key transport without forward secrecy) that the attacker can exploit.

Why this is catastrophic: TLS 1.3 was designed precisely to eliminate the weaknesses in TLS 1.2. A downgrade attack bypasses all of TLS 1.3’s security improvements, exposing the connection to attacks that TLS 1.3 was specifically designed to prevent (e.g., POODLE-style padding oracle attacks, DROWN decryption, Lucky13 timing attacks).

TLS 1.3’s countermeasure: RFC 8446 specifies that when a TLS 1.3-capable server negotiates TLS 1.2 (due to a `ClientHello` that excludes TLS 1.3), it *must* include a specific sentinel value in the last 8 bytes of its random nonce field. TLS 1.3-capable clients that receive a TLS 1.2 `ServerHello` check for this sentinel; if found, they abort. This provides downgrade detection.

The downgrade attack. Consider the protocol $\pi_{\text{TLS-pre}}$, a hypothetical TLS implementation that omits the RFC 8446 downgrade sentinel (as would be the case in a buggy or non-compliant implementation). Let $\mathcal{A}_{\text{down}}$ be an active network adversary that intercepts and modifies handshake messages.

The attack proceeds as follows:

1. The client C sends `ClientHello` advertising $\{\text{TLS 1.3}, \text{TLS 1.2}\}$ and cipher suites including TLS 1.3-only ECDHE suites.
2. $\mathcal{A}_{\text{down}}$ intercepts, rewrites `ClientHello` to advertise only TLS 1.2 suites (e.g.,

$$\text{TLS_RSA_WITH_AES_128_CBC_SHA256}$$
) and forwards to S .
3. S responds with a TLS 1.2 `ServerHello` (since it sees no TLS 1.3 offer) and a static RSA ciphertext.
4. $\mathcal{A}_{\text{down}}$ forwards S ’s response to C . Since $\pi_{\text{TLS-pre}}$ omits downgrade sentinel checking, C accepts and negotiates TLS 1.2.
5. Now $\mathcal{A}_{\text{down}}$ passively records the RSA-encrypted premaster secret. The session uses TLS 1.2 with RSA key transport (no forward secrecy): if sk_S is ever compromised, all recorded traffic is decryptable.

Theorem 13.26 (TLS 1.3 Downgrade Market Collapse without Sentinel). *Let $\pi_{\text{TLS-pre}}$ be a TLS implementation omitting the RFC 8446 downgrade sentinel. The good g_{version} : “the negotiated protocol version is the highest mutually supported version” has $\text{Ask}(g_{\text{version}}) = 1$ against a network adversary. The downgrade market collapses.*

Proof. We construct a two-bidding-round argument.

B_0 (Real Protocol without Sentinel). The client supports TLS 1.3; the server supports TLS 1.2 and TLS 1.3; the network adversary is active.

Downgrade Bid. The adversary $\mathcal{A}_{\text{down}}$ places the *version stripping bid*: it intercepts `ClientHello`, removes the TLS 1.3 version identifier and associated extensions (e.g., `key_share`, `supported_versions`),

and forwards the modified message. This is a *free* modification: the adversary requires no cryptographic secret and performs $O(1)$ computation. The bad event F_{down} : “the negotiated version is TLS 1.2, not 1.3, despite both parties supporting TLS 1.3.” $\Pr[F_{\text{down}}] = 1$ (the adversary always succeeds in $\pi_{\text{TLS-pre}}$).

B₁ (Downgraded Session). The session now runs TLS 1.2. The TLS 1.2 protocol has known attacks:

- **No forward secrecy (RSA key transport):** The premaster secret is RSA-encrypted under pk_S . If sk_S is compromised later, all past traffic is decryptable. The forward-secrecy good g_{FS} collapses: $\text{Ask}(g_{\text{FS}}) = 1$.
- **CBC padding oracle (if CBC suite negotiated):** TLS 1.2 CBC-mode ciphers are vulnerable to Lucky13-style padding oracle attacks [4], providing a decryption oracle for past ciphertexts.

Extended difference lemma. The three simultaneous failure events F_{down} , F_{FS} , and F_{pad} all occur with probability 1 (free bids): $\Pr[F_{\text{down}} \cup F_{\text{FS}} \cup F_{\text{pad}}] = 1$. The market collapses across all three goods.

CNF downgrade failure. The session-CNF for the downgraded session includes clause φ^{version} : “negotiated version = maximum mutually supported version.” Under the downgrade attack, φ^{version} is false (TLS 1.2 was negotiated despite TLS 1.3 being supported). The adversary’s trace satisfies all structural clauses (valid handshake messages, valid MACs under TLS 1.2 keys) but violates φ^{version} . Since $\pi_{\text{TLS-pre}}$ has no downgrade sentinel check, φ^{version} cannot be enforced. This is a CNF *Type IV* design failure (unauthenticated version negotiation field): the `supported_versions` extension is not integrity-protected before the Finished MAC is computed.

Market outcome. $\text{Ask}(g_{\text{version}}) = 1$; the market collapses. The Ping bid degrades:

$$\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 0 \quad (171)$$

because session $i+1$ runs a structurally weaker protocol than session i , violating protocol-version consistency. $\square$

Key Insight:

How TLS 1.3 Fixes the Downgrade Attack RFC 8446 restores equilibrium for g_{version} via the *downgrade sentinel mechanism*: when a TLS 1.3-capable server negotiates TLS 1.2, it must set the last 8 bytes of its `ServerHello.Random` to a specific sentinel value (44 4F 57 4E 47 52 44 01 for TLS 1.2, 44 4F 57 4E 47 52 44 00 for TLS 1.1). A TLS 1.3-capable client receiving a TLS 1.2 `ServerHello` checks for this sentinel; if found, it aborts with a fatal alert.

In MTSF terms: the sentinel adds a clause $\varphi^{\text{sentinel}}$ to the session-CNF. A dishonest downgrade trace either (a) omits the sentinel (detectable by the client, causing abort—the trace does not reach completion) or (b) includes the sentinel, causing the TLS 1.3-capable client to abort. Either way, the downgrade attack cannot produce a completed session that passes the CNF check. The version-stripping bid costs the adversary nothing in computation but now always causes client-side abort, restoring equilibrium: $\text{Ask}(g_{\text{version}}) \leq \text{negl}$ in compliant implementations.

13.8.6 TLS 1.3 CNF Session Verification and Ping Bids

TLS 1.3 Session-CNF. The TLS 1.3 session-CNF φ_{TLS} is the conjunction of the following clauses:

$$\varphi^{\text{cert}} = (x_{\text{Vrfy}(\text{pk}_{\text{CA}}, \text{cert}_S)} = 1) \quad \text{Server certificate is CA-issued} \quad (172)$$

$$\begin{aligned}
\varphi^{\text{cv}} &= (x_{\text{Vrfy}(\text{pk}_S, \sigma\text{-ctx} \parallel \mathcal{T}_{\text{CV}}, \text{CertVfy}_S)=1}) && \text{CertVfy binds full transcript} && (173) \\
\varphi^{\text{fin}} &= (x_{\text{HMAC}(k_{\text{fin},S}, \mathcal{T}_{\text{CF}})=\text{Fin}_S}) && \text{Finished MAC is transcript-bound} && (174) \\
\varphi^{\text{fresh}} &= (x_{(N_C, N_S) \notin \mathcal{N}_{\text{prev}}}) && \text{Nonce pair is fresh} && (175) \\
\varphi^{\text{eph}} &= (x_{(X,Y) \notin \mathcal{DH}_{\text{prev}}}) && \text{DH shares are fresh} && (176) \\
\varphi^{\text{ver}} &= (x_{\text{version}=\text{TLS 1.3}}) && \text{Version is TLS 1.3 (sentinel check)} && (177) \\
\varphi^{\text{suite}} &= (x_{\text{suite} \in \{\text{TLS_AES_128_GCM_SHA256}, \dots\}}) && \text{Only AEAD suites permitted} && (178) \\
\varphi^{\text{ping}} &= (x_{\text{Ping}(\text{Session}_i, \text{Session}_{i+1})=1}) && \text{Session transition passes ping} && (179)
\end{aligned}$$

$$\varphi_{\text{TLS}} = \varphi^{\text{cert}} \wedge \varphi^{\text{cv}} \wedge \varphi^{\text{fin}} \wedge \varphi^{\text{fresh}} \wedge \varphi^{\text{eph}} \wedge \varphi^{\text{ver}} \wedge \varphi^{\text{suite}} \wedge \varphi^{\text{ping}}.$$

✓ CNF Verification: TLS 1.3 Session-CNF Verification

Honest trace: All eight clauses are satisfied. The server holds a valid CA-issued certificate; `CertificateVerify` covers the full transcript including both nonces; `Finished` covers all server messages; nonces are fresh; DH shares are fresh; version is TLS 1.3; suite is AEAD; ping passes.

Dishonest trace analysis:

- Violating φ^{cert} : requires CA signature forgery ($\text{Adv}_{\text{CA}}^{\text{EUF}}$).
- Violating φ^{cv} : requires signing-key forgery ($\text{Adv}_{\text{Sig}}^{\text{EUF}}$).
- Violating φ^{fin} : requires HMAC forgery ($\text{Adv}_{\text{HMAC}}^{\text{SUF}}$).
- Violating φ^{fresh} : requires nonce collision ($q_N^2/2^{\lambda+1}$).
- Violating φ^{eph} : requires DH share reuse (birthday probability $q_N^2/2^{\lambda+1}$).
- Violating φ^{ver} : requires sentinel bypass (detectable, causes abort—bid fails).
- Violating φ^{suite} : requires server to accept a non-AEAD suite (implementation error).
- Violating φ^{ping} : requires session collision or transcript replay (negligible).

All violations require negligible-probability events or detectable actions. φ_{TLS} is UNSAT under dishonest traces in PPT time (except with negligible probability).

Manual CNF worksheet (TLS 1.3):

Clause	Check	How to verify	Pass?
φ^{cert} : CA cert	$\text{Vrfy}(\text{pk}_{\text{CA}}, \text{cert}_S) = 1?$	Check cert chain against trust store.	T/F
φ^{cv} : transcript sig	$\text{Vrfy}(\text{pk}_S, \text{CertVfy}_S) = 1?$	Verify signature over transcript hash.	T/F
φ^{fin} : Finished MAC	$\text{HMAC}(k_{\text{fin},S}, \mathcal{T}_{\text{CF}}) = \text{Fin}_S?$	Recompute HMAC over transcript.	T/F
φ^{fresh} : nonce pair	(N_C, N_S) not seen before?	Check nonce log.	T/F
φ^{eph} : DH shares	(X, Y) not reused?	Check DH share log.	T/F
φ^{ver} : version	Negotiated TLS 1.3? Sentinel absent?	Read version field; check sentinel.	T/F
φ^{suite} : AEAD only	Suite $\in \{\text{AES-128-GCM}, \text{AES-256-GCM}, \text{CHACHA20}\}?$	Read cipher suite field.	T/F
φ^{ping} : session ping	Prior session nonces $\cap$ current = $\emptyset$?	Compare session records.	T/F
Result:	All T $\Rightarrow$ Secure session (equilibrium). Any F $\Rightarrow$ Reject and log violation.		

TLS 1.3 Ping Bid.

Corollary 13.27 (TLS 1.3 Unbounded Session Security). *The TLS 1.3 1-RTT handshake is secure for unbounded sessions. For consecutive sessions Session_i and Session_{i+1} , the ping function satisfies $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 1$ with probability at least $1 - q_N^2/2^{\lambda+1}$, where q_N counts total*

accumulated sessions. By Theorem 5.5, $\text{Ask}(g_{\text{sk}})$ remains bounded as in Equation (159) for all $i \geq 1$.

Proof. The four ping conditions from Theorem 5.3 are verified as follows. (a) *SID/nonce freshness*: $(N_{C,i+1}, N_{S,i+1})$ is fresh by construction (256-bit uniform sampling per session). (b) *Nonce disjointness*: The pair $(N_{C,i+1}, N_{S,i+1})$ is disjoint from all prior pairs except with probability $q_N^2/2^{\lambda+1}$. (c) *Transcript binding*: $\text{CertVfy}_{S,i+1}$ signs $\mathcal{T}_{\text{CV},i+1}$, which includes $N_{C,i+1}$ and $N_{S,i+1}$; $\text{Fin}_{S,i+1}$ MACs the full handshake transcript. Replaying any message from session Session_i in Session_{i+1} fails because the nonces differ. (d) *Key deletion*: Ephemeral DH private keys (x_{i+1}, y_{i+1}) are securely deleted after Z_{i+1} is computed. The session-CNF $\varphi_{\text{TLS},i+1}$ is isomorphic to $\varphi_{\text{TLS},i}$ with fresh nonces, DH shares, and transcript; by the inductive argument of Theorem 5.5, unbounded equilibrium holds. $\square$

Table 16: MTSF market comparison: TLS 1.3 (1-RTT) vs. TLS 1.2 vs. TLS 1.3 0-RTT. A collapsed good ($\text{Ask} = 1$) indicates a fundamental structural failure.

Security good	TLS 1.3 (1-RTT)	TLS 1.2 (RSA transport)	TLS 1.3 (0-RTT)
g_{sk} : key secrecy	$\leq \text{Adv}^{\text{CDH}} + 3\text{Adv}^{\text{PRF}} + \text{negl}$	$\leq \text{Adv}^{\text{RSA-PKCS}} + \text{negl}$	$\leq \text{Adv}^{\text{CDH}} + \text{negl}$ (for HS)
g_{FS} : forward secrecy	$\leq \text{Adv}^{\text{CDH}} + \text{negl}$	1 (Collapsed)	$\leq \text{Adv}^{\text{CDH}} + \text{negl}$ (for HS)
g_{auth} : server auth	$\leq \text{Adv}_{\text{CA}}^{\text{EUF}} + \text{Adv}_{\text{Sig}}^{\text{EUF}} + \text{negl}$	$\leq \text{Adv}_{\text{CA}}^{\text{EUF}} + \text{Adv}_{\text{Sig}}^{\text{EUF}} + \text{negl}$	$\leq \text{Adv}_{\text{CA}}^{\text{EUF}} + \text{Adv}_{\text{Sig}}^{\text{EUF}} + \text{negl}$
g_{0RTT} : anti-replay	N/A (no 0-RTT)	N/A	1 (Collapsed)
g_{version} : no downgrade	$\leq \text{negl}$ (sentinel)	1 (no sentinel)	$\leq \text{negl}$ (sentinel)
g_{CNF} : session correctness	$\leq \text{negl}$ (all clauses)	Partial (no FS clause)	Partial ($\varphi^{\text{fresh-early}}$ fails)
Overall market	Equilibrium	Partial collapse	Partial collapse (0-RTT)

Market Summary: TLS 1.3 vs. TLS 1.2.

* Simple Terms: Lessons from the TLS 1.3 MTSF Analysis

What TLS 1.3 gets right (equilibrium goods):

- **Mandatory forward secrecy**: By requiring ephemeral DH (no RSA key transport), TLS 1.3 ensures g_{FS} is in equilibrium. Server key compromise after session completion cannot reveal past traffic.
- **Transcript binding**: The `CertificateVerify` signature and `Finished` MAC together cover every negotiated parameter. No selective message replacement succeeds.
- **Downgrade protection**: The RFC 8446 sentinel converts the version-stripping bid from a free attack to a detectable abort, restoring g_{version} equilibrium.
- **AEAD-only ciphers**: Eliminating non-AEAD suites removes all CBC padding oracle and RC4 biases. The g_{INT} (integrity) good is in equilibrium by design.

What TLS 1.3 does not fully solve (partial collapse):

- **0-RTT replay**: The structural absence of a server nonce in the 0-RTT first flight means g_{0RTT} collapses without server-side anti-replay. Application-layer discipline is required.
- **CA compromise**: The entire PKI trust chain depends on CA security. A compromised CA collapses g_{cert} , which propagates to g_{auth} and g_{sk} . This is an external dependency outside TLS's control.

- **Implementation bugs:** The MTSF analysis assumes honest implementation. Real-world flaws (Heartbleed-style memory errors, timing side-channels in AES-GCM nonce handling) are not captured by the cryptographic model.

MTSF’s contribution: TLS 1.3 is the first major protocol whose formal security analysis (by Dowling, Fischlin, Gunther, Stebila [45] in the ACCE model) covers authenticated and confidential channel establishment as a unified security notion. MTSF recasts this analysis as a ten-bidding-round market, making the security argument auditable by practitioners without expert training.

13.9 Telegram: Security and Insecurity Analysis

* Simple Terms: Telegram MTPROTO 2.0—Background and Context

What is MTPROTO? MTPROTO is Telegram’s custom encrypted messaging protocol, designed in-house. Version 2.0 (2017) replaced the original MTPROTO after significant cryptographic criticism. Unlike Signal (which uses the Double Ratchet and Noise Protocol) or TLS 1.3, MTPROTO 2.0 was not designed by academic cryptographers following published standards, making its security analysis nontrivial.

How MTPROTO 2.0 encrypts a message:

1. Client and server share a long-term authentication key `auth_key` (256 bytes), established via Diffie–Hellman during account creation and stored persistently.
2. Each message includes a 64-bit **server salt**—a server-assigned value that changes every 30 minutes. Both sides agree on the current salt via a periodic negotiation message.
3. The **message key** `msg_key` (128 bits) is derived as: $\text{msg_key} \leftarrow \text{SHA-256}(\text{auth_key}[88:120] \parallel \text{plaintext})[8:24]$, where the plaintext includes the server salt, session ID, message ID, sequence number, and the actual data.
4. Encryption uses AES-256-IGE (Infinite Garble Extension) mode with key and IV derived from `msg_key` and `auth_key`.

What is the salt for? The server salt provides short-term “session freshness”: messages with an expired salt are rejected, preventing replay of old messages. It contributes to the uniqueness of `msg_key` and thus of each AES encryption key.

The core security question we analyse: What happens when an adversary can extract the current server salt from a session? This article answers this question formally using MTSF, with both a *security disproof* (showing what collapses) and a *security proof for the remediated protocol*.

Analogy:

Telegram Salt as a Concert Wristband The server salt is like a concert wristband that changes colour every 30 minutes. With the correct wristband, you can enter freely until it expires. If someone copies your current wristband, they can enter until the next rotation. If your wristband is stolen and the adversary knows it, security degrades for the remainder of the 30-minute window. The fix: make the wristband unforgeable (cryptographically bind it to your identity) and increase the rotation frequency.

13.9.1 MTPROTO 2.0 Market Model

We model MTPROTO 2.0 as a market $\mathcal{M}_{\text{MTPROTO}}$ where the seller offers two goods:

- g_{IND} : **Message indistinguishability** (confidentiality). The buyer cannot distinguish encryptions of two messages of equal length.
- g_{INT} : **Message integrity** (authenticity). The buyer cannot inject or modify a message that passes decryption.

Session parameters. A session Session_i is parameterised by $(\text{auth_key}, s_i, \text{sess_id}_i)$ where $s_i \in \{0, 1\}^{64}$ is the current server salt and sess_id_i is a 64-bit session identifier. A message m is encrypted as:

$$\text{plaintext} = s_i \parallel \text{sess_id}_i \parallel \text{msg_id} \parallel \text{seqno} \parallel \text{data}, \quad (180)$$

$$\text{msg_key} = \text{SHA-256}(\text{auth_key}[88:120] \parallel \text{plaintext})[8:24], \quad (181)$$

$$(K_{\text{AES}}, \text{IV}_{\text{AES}}) = \text{KDF}(\text{msg_key}, \text{auth_key}), \quad (182)$$

$$\text{ciphertext} = \text{AES-256-IGE}_{K_{\text{AES}}}(\text{plaintext}; \text{IV}_{\text{AES}}). \quad (183)$$

The transmitted packet is $(\text{auth_key_id}, \text{msg_key}, \text{ciphertext})$.

13.9.2 Security Disproof: Salt Extraction Collapses Confidentiality

* Simple Terms: Telegram MTPROTO—A Real-World Security Controversy

What MTPROTO is: Telegram uses a custom cryptographic protocol called MTPROTO 2.0 for its cloud chats (non-end-to-end-encrypted chats). Unlike Signal (which uses well-studied academic protocols), MTPROTO was designed in-house by Telegram, raising concerns among cryptographers.

The salt extraction vulnerability: In MTPROTO 2.0, a “salt” value is sent in plaintext alongside encrypted messages. This salt is intended to prevent replay attacks, but it also leaks information: the salt reduces the entropy of the key derivation, giving the attacker a 2^{-64} advantage—small but non-negligible for high-security applications.

In MTSF: This is a *partial market collapse*. The ask price is not 1 (full collapse) but approximately 2^{-64} (partial). The “salt extraction bid” succeeds with probability 2^{-64} —small for practical purposes but theoretically non-negligible. MTSF detects this as a “quasi-collapse” that a standard game-based proof might overlook.

The fix (RMTP): MTSF proposes a remediated protocol (RMTP) that binds the salt to an HMAC under the authentication key. This eliminates the salt extraction bid, restoring full market equilibrium. The RMTP proof uses four bidding rounds with the extended difference lemma, showing $\text{Ask} \leq \text{Adv}^{\text{PRF}} + 2^{-128}$.

The lesson: Custom cryptographic protocols are dangerous. Even a small deviation from established designs can introduce subtle vulnerabilities that only a rigorous framework like MTSF detects. This case study demonstrates MTSF’s power as both a security analysis tool and a protocol design guide.

Theorem 13.28 (MTPROTO Salt Extraction: Partial Market Collapse). *Suppose an adversary $\mathcal{A}$ extracts the server salt s_i for the current 30-minute window by any means (side channel, decrypted packet, protocol weakness, or social engineering). Then:*

1. **Entropy collapse:** *The effective entropy of msg_key drops from 128 bits to 64 bits: $\text{Ask}(g_{\text{IND}}) \geq 1 - 2^{-64} \cdot q_D$, where q_D is the number of decryption queries.*
2. **CNF freshness failure:** *The session-CNF clause φ^{fresh} is satisfiable under dishonest traces whenever s_i is known to the adversary.*
3. **Ping degradation:** $\Pr[\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 0 \mid s_i \text{ known}] \geq 1 - 2^{-64}$.

4. **Partial market collapse:** $\text{Ask}(g_{\text{IND}}) \not\leq \text{negl}(\lambda)$; the market has not fully collapsed ($\text{Ask} < 1$) but the security margin is infeasible for 128-bit security requirements.

Proof. We construct a sequence of bidding rounds demonstrating each collapse mechanism.

B₀ (Real MTPROTO Market—Indistinguishability Bid). Buyer submits (m_0, m_1) with $|m_0| = |m_1|$; receives ciphertext $c^* = \text{AES-256-IGE}_{K_b}(\text{plaintext}_b; \text{IV}_b)$ for bit b . Must guess b . The *indistinguishability bid*: distinguish b from the ciphertext alone. Without salt knowledge, the effective key material has 128 bits of entropy in `msg_key`.

B₁ (Bidding Round 1: Salt Extraction Bid—Entropy Reduction).

Purpose: Convert a successful adversarial forgery into a solution of the underlying hard problem, completing the security reduction. If the adversary can forge a signature in the programmed-oracle game, the reduction invokes the *general forking lemma* [14] to rewind the adversary twice with different random-oracle coins at the same fork point, producing two consistent forgeries (r^*, s^*) and (r^*, s^{**}) with $s^* \neq s^{**}$ (same nonce commitment r^* , different hash). From these two equations, the reduction extracts the private key $\text{sk} = (s^{**}H' - s^*H)(r(s^* - s^{**}))^{-1} \bmod q$, solving ECDLP—which contradicts the hardness assumption.

Replaces: The honest signing oracle is replaced by a *ECDLP extraction oracle*: on witnessing a valid forgery $(m^*, (r^*, s^*))$, the reduction forks the adversary at the random-oracle query for m^* (guessing the correct fork point uniformly among q_H queries), re-runs with a fresh coin, and collects the second forgery $(m^*, (r^*, s^{**}))$. The extraction succeeds whenever (a) the fork point is guessed correctly—probability acc/q_H where acc is the original forgery probability—and (b) $s^* \neq s^{**}$ —which fails only with probability $1/q$.

Complexity: Bounded by $q_H \cdot q_S \cdot \text{Adv}^{\text{ECDLP}}(\lambda) + 1/q + \Pr[F_{\text{fork}}]$. The factor $q_H \cdot q_S$ arises from the forking lemma’s rewinding: the reduction must guess which of the q_H hash queries and which of the q_S signing sessions the adversary will use in its forgery. $\Pr[F_{\text{fork}}] \leq 1 - \text{acc}^2/q_H + 1/q$ (forking lemma [14]). For standard parameters ($q_S, q_H \leq 2^{64}$, $q \approx 2^{256}$, $\text{Adv}^{\text{ECDLP}} \leq 2^{-128}$), the total is negligible. The extended difference lemma captures F_{fork} and F_{extract} simultaneously: $\Pr[F_{\text{fork}} \cup F_{\text{extract}}] \leq \Pr[F_{\text{fork}}] + 1/q$.

The adversary successfully extracts s_i (by any means: this is the *salt extraction bid*, a “free” bid in the sense that we take the salt as given). With s_i known, the plaintext structure is:

$$\text{plaintext} = \underbrace{s_i}_{\text{known}} \parallel \text{sess_id}_i \parallel \text{msg_id} \parallel \text{seqno} \parallel \text{data}.$$

The `msg_key` is computed from this plaintext with the known s_i as a prefix. The buyer can now *brute-force the remaining entropy*: the 64-bit session ID, message ID, and sequence number together contribute at most $64 + 64 + 32 = 160$ bits, but in practice the adversary has high confidence about `sess_idi` (from the packet header) and `seqno` (sequential), reducing the search space to roughly 2^{64} candidate `msg_key` values.

Extended difference lemma. Let F_{salt} : “salt s_i is extracted”. Let F_{entropy} : “effective entropy of `msg_key` drops below 64 bits”. Both events occur simultaneously when salt extraction succeeds:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \Pr[F_{\text{salt}} \cup F_{\text{entropy}}] = \Pr[F_{\text{salt}}] \cdot (1 + \Pr[F_{\text{entropy}} \mid F_{\text{salt}}]) = 1 \cdot 1. \quad (184)$$

Since we assume F_{salt} occurs with probability 1 (the salt is extracted), and F_{entropy} follows deterministically, the price adjustment is $\Delta \text{Price}_0 = 1$ —a full hop collapse for this bid.

B₂ (Bidding Round 2: AES Partial-Key Recovery Bid).

Purpose: Replace the real ephemeral Diffie–Hellman shared secret (or KEM-derived session secret) with a uniformly random value, isolating the CDH or IND-CCA2 hardness assumption as the sole remaining security requirement. This is the forward-secrecy-establishing hop: after it, compromising long-term keys cannot retroactively reveal the session secret.

Replaces: The real shared secret $Z = g^{xy}$ (computed from ephemeral exponents x, y) or the KEM-encapsulated key $K = \text{KEM.Dec}(\text{sk}, c)$ is replaced by $\tilde{Z} \xleftarrow{\$} \{0, 1\}^{|Z|}$, a freshly sampled uniform random value of the same length. Any adversary that detects this swap either solves CDH (given (g, g^x, g^y) ,

compute g^{xy}) or breaks IND-CCA2 of the KEM (distinguishes real encapsulated key from random). Forward secrecy follows because the ephemeral exponents x, y (or the KEM's ephemeral coins) are deleted after Z is computed and never appear in any long-term storage.

Complexity: $\Delta\text{Price} \leq \text{Adv}_g^{\text{CDH}}(\lambda)$ (DH case) or $\text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\lambda)$ (KEM case). For X25519: $\text{Adv}^{\text{CDH}} \leq 2^{-128}$ under the best known algorithms (Pollard rho on 252-bit curve order). For ML-KEM-768: $\text{Adv}^{\text{IND-CCA2}} \leq 2 \cdot \text{Adv}^{\text{MLWE}} + q_D/2^{138}$, all negligible. After this hop, the session key is derived from a random value via the KDF, so the next hop (PRF replacement) completes the key-secrecy argument.

With entropy reduced to 2^{64} , the buyer mounts a *birthday meet-in-the-middle bid*: precompute 2^{32} candidate K_{AES} values from guessed `msg_key` prefixes; collect 2^{32} ciphertext blocks; find a collision. This gives key recovery with $O(2^{64})$ time and $O(2^{32})$ memory.

Difference bound. With 2^{64} candidate keys over a key space of 2^{128} :

$$\text{Ask}(g_{\text{IND}} \mid F_{\text{salt}}) \leq 1 - 2^{-64} \approx 1. \quad (185)$$

The ask price for indistinguishability approaches 1—the market has quasi-collapsed.

B₃ (Bidding Round 3: CNF Freshness Failure Bid).

Purpose: Eliminate the *nonce-reuse attack vector*. Repeated nonces immediately collapse authentication, key-secrecy, or stream-cipher security: in signatures a shared nonce leaks the private key ($\text{sk} = (s_2 e_1 - s_1 e_2)(r(s_1 - s_2))^{-1}$); in protocols it enables cross-session replay; in stream ciphers it reveals the XOR of two plaintexts. This round aborts the game whenever any nonce collision occurs, ensuring every subsequent hop reasons about a nonce-fresh world.

Replaces: The real nonce-sampling procedure is augmented with a global *collision audit*: a set $\mathcal{N}_{\text{used}}$ records every nonce issued so far. If any newly sampled nonce k' satisfies $k' \in \mathcal{N}_{\text{used}}$, the game aborts immediately. The two games ($\mathbf{B}_{n-1}$ and $\mathbf{B}_n$) are *identical-until-abort*: the adversary's view is indistinguishable until the abort event F_{nonce} occurs, so the difference lemma directly bounds the price adjustment.

Complexity: By the birthday bound, the probability that any two of q independently sampled nonces from a space of size $|\mathcal{N}|$ collide is at most $q^2/(2|\mathcal{N}|)$. For security parameter λ with $|\mathcal{N}| = 2^\lambda$ this gives $\Delta\text{Price} \leq q^2/2^{\lambda+1}$, which is negligible for any polynomial $q = \text{poly}(\lambda)$. After this hop every nonce in the proof is treated as distinct, allowing all later hops to assume a collision-free nonce space without loss of generality.

The session-CNF for MTProto includes a freshness clause:

$$\varphi_{\text{MTProto}}^{\text{fresh}} = (x_{s_i \notin \mathcal{S}_{\text{used}}}) \wedge (x_{s_i \text{ secret from } \mathcal{A}}).$$

With s_i extracted, the clause $x_{s_i \text{ secret}}$ evaluates to **F**. The session-CNF becomes UNSAT under the honest trace—which should never happen. This indicates that MTProto's session design does not satisfy the CNF correctness requirement g_{CNF} once salt extraction is possible.

B₄ (Bidding Round 4: Ping Degradation Bid).

Purpose: Analyse the Telegram MTProto 2.0 salt handling vulnerability or its RMTP remediation. The 64-bit server salt transmitted in plaintext headers allows a network observer to read it directly, reducing effective AES key entropy from 256 to 192 bits and enabling partial market collapse.

Replaces: For the attack: the hidden salt assumption is replaced by the observed reality—the salt appears in plaintext message headers before encryption, so no cryptographic computation is needed to read it. For the fix (RMTP): the exposed salt is replaced by an HMAC-derived value $\text{salt} = \text{HMAC}_{\text{auth_key}}(\text{context})$, requiring the adversary to break HMAC-PRF security to predict the salt.

Complexity: Salt extraction bid: free ($O(1)$ work, $\Pr[F] = 1$). AES partial-key attack: 2^{96} time and space after learning salt. RMTP salt bid: bounded by $\text{Adv}_{\text{HMAC}}^{\text{PRF}}(\lambda)$, negligible. The overall MTProto market: partial collapse ($\text{Ask} \leq 1 - 2^{-64}$, security margin reduced to 64 bits). RMTP market: full equilibrium restored ($\text{Ask} \leq \text{Adv}^{\text{PRF}} + q_D \cdot 2^{-128} = \text{negl}$).

Session pinging tests that consecutive sessions $\text{Session}_i, \text{Session}_{i+1}$ are structurally distinct. If s_i is known and s_{i+1} can be predicted (salts are negotiated over a potentially observable channel),

then $\text{Ping}(\text{Session}_i, \text{Session}_{i+1}) = 0$ whenever $s_i = s_{i+1}$ (which happens with probability $\geq 2^{-64}$ per 30-minute window) or when s_{i+1} can be inferred from s_i .

$$\Pr[\text{Ping} = 0 \mid s_i \text{ known}] \geq \Pr[s_{i+1} \text{ predictable}] \geq 2^{-64}. \quad (186)$$

This is a 2^{64} -fold degradation compared to the ideal $\Pr[\text{Ping} = 0] \leq 2^{-128}$.

Summary. The four simultaneous collapse events are:

- F_{salt} : Salt extraction (assumed).
- F_{entropy} : Entropy reduction from 128 to 64 bits.
- F_{cnf} : CNF freshness clause fails.
- F_{ping} : Ping degradation.

By the extended difference lemma: $\text{Ask}(g_{\text{IND}} \cup g_{\text{CNF}} \cup g_{\text{ping}}) \leq \Pr[F_{\text{salt}} \cup F_{\text{entropy}} \cup F_{\text{cnf}} \cup F_{\text{ping}}] =$

1. The Telegram market has collapsed for all security goods given salt extraction. $\square$

* Simple Terms: Why Salt Extraction Is Catastrophic—Simple Explanation

The key derivation vulnerability. MTPProto’s message key is derived from the *plaintext*, which includes the salt. If the adversary knows the salt, they eliminate 64 bits of entropy from the key. Instead of needing to try 2^{128} keys (computationally impossible), they need to try only 2^{64} (borderline feasible with modern hardware clusters).

Why 2^{64} is a real threat. In 2023, Bitcoin’s hashrate exceeded 2^{76} operations per second across the network. Dedicated hardware for 2^{64} AES operations would take minutes to hours, not billions of years. Salt extraction turns MTPProto’s 128-bit theoretical security into practical 64-bit security.

The CNF failure. A well-designed protocol’s session verification should *only* pass for honest participants. MTPProto’s salt extraction means an adversary can construct a dishonest session trace that passes all verification checks—a design failure caught by MTSF’s CNF framework.

13.9.3 Security Proof: Remediated MTPProto

We now define the *Remediated MTPProto* (RMTP) and prove it achieves full market equilibrium.

RMTP Construction. Modify MTPProto 2.0 as follows:

1. **Cryptographic salt binding:** Replace the plaintext server salt with an HMAC-bound salt: $\tilde{s}_i = \text{HMAC}_{\text{auth_key}}(s_i \parallel \text{timestamp})$. The salt is now a 256-bit pseudorandom value derived from auth_key .
2. **Increased salt entropy:** Use 128-bit salts ($\tilde{s}_i \in \{0, 1\}^{128}$) instead of 64-bit.
3. **Full message binding:** Include $\tilde{s}_i$ in all cryptographic operations:

$$\text{msg_key} \leftarrow \text{SHA-256}(\text{auth_key}[88:120] \parallel \tilde{s}_i \parallel \text{data}). \quad (187)$$

4. **CNF salt clause:** Add the clause $x_{\tilde{s}_i = \text{HMAC}_{\text{auth_key}}(s_i \parallel \text{ts})}$ to the session-CNF.

Theorem 13.29 (Remediated MTPProto Market Equilibrium). *Under the PRF security of HMAC and the IND-CPA security of AES-256-IGE, the Remediated MTPProto (RMTP) achieves:*

$$\text{Ask}(g_{\text{IND}}) \leq \text{Adv}_{\text{AES}}^{\text{IND-CPA}} + \text{Adv}_{\text{HMAC}}^{\text{PRF}} + q_D \cdot 2^{-128} + \text{negl}(\lambda), \quad (188)$$

$$\text{Ask}(g_{\text{INT}}) \leq \text{Adv}_{\text{HMAC}}^{\text{PRF}} + \text{negl}(\lambda), \quad (189)$$

$$\text{Ask}(g_{\text{CNF}}) \leq \text{Adv}_{\text{HMAC}}^{\text{PRF}} + q_N^2 / 2^{128} + \text{negl}(\lambda). \quad (190)$$

All three goods are in equilibrium.

Proof. We construct six bidding rounds for the indistinguishability good g_{IND} .

B₀ (Real RMTP—Indistinguishability Bid).

Buyer submits (m_0, m_1) ; receives $c^* = \text{AES-256-IGE}_{K_b}(\cdot)$.

B₁ (HMAC Salt Freshness Bid). Replace $\tilde{s}_i$ with a uniformly random 128-bit value u_i . Any buyer distinguishing **B₀** from **B₁** breaks PRF security of HMAC:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \text{Adv}_{\text{HMAC}}^{\text{PRF}}. \quad (191)$$

Extended difference lemma. Events F_{prf} : “HMAC distinguishable from random” and $F_{\text{salt-ext}}$: “salt extractable without `auth_key`”. With $\tilde{s}_i$ computed via HMAC under `auth_key`, salt extraction without `auth_key` requires breaking PRF. Hence $\Pr[F_{\text{prf}} \cup F_{\text{salt-ext}}] \leq \text{Adv}_{\text{HMAC}}^{\text{PRF}}$.

B₂ (Nonce Freshness Bid). Abort on 128-bit salt collision. By birthday bound over q_D decryptions:

$$|\Pr[\mathbf{B}_1 = 1] - \Pr[\mathbf{B}_2 = 1]| \leq q_D \cdot 2^{-128}. \quad (192)$$

B₃ (msg_key Randomness Bid). With $\tilde{s}_i$ uniform, the full input to the `msg_key` hash is uniform (no bias from salt extraction). Replace `msg_key` with uniform random $r \xleftarrow{\$} \{0, 1\}^{128}$. By PRF security of SHA-256: $\Delta\text{Price}_2 \leq \text{Adv}_{\text{SHA256}}^{\text{PRF}} \leq \text{negl}(\lambda)$.

B₄ (AES Key Replacement Bid). With uniform `msg_key`, the AES key $(K_{\text{AES}}, \text{IV}_{\text{AES}})$ derived from it is uniform. Replace with independently uniform (K', IV') . This costs $\text{Adv}_{\text{KDF}}^{\text{PRF}}$.

B₅ (AES IND-CPA Reduction Bid). With uniform key and IV, the AES-256-IGE encryption is IND-CPA secure:

$$|\Pr[\mathbf{B}_4 = 1] - 1/2| \leq \text{Adv}_{\text{AES}}^{\text{IND-CPA}}. \quad (193)$$

B₆ (Ideal RMTP Market). With random key, random IV, and IND-CPA encryption, the ciphertext is computationally indistinguishable from random. Buyer advantage = $1/2$.

Total for g_{IND} : Equation (188) follows by summing price adjustments.

Bound (189) (Integrity g_{INT}). A successful injection requires the adversary to produce $(\text{msg_key}^*, c^*)$ that decrypts to a valid plaintext (passes AES-IGE decryption) and has a consistent `msg_key`* matching the decrypted plaintext’s HMAC-bound salt. This requires either inverting AES-IGE (IND-CPA hardness) or forging the HMAC binding (PRF hardness). By the extended difference lemma over these two failure events: $\text{Ask}(g_{\text{INT}}) \leq \text{Adv}_{\text{HMAC}}^{\text{PRF}} + \text{Adv}_{\text{AES}}^{\text{IND-CPA}} = \text{negl}$.

Bound (190) (CNF correctness g_{CNF}). The RMTP session-CNF is:

$$\varphi_{\text{RMTP}} = (x_{\tilde{s}_i = \text{HMAC}_{\text{auth_key}}(s_i \| \text{ts})}) \wedge (x_{\tilde{s}_i \notin \mathcal{S}_{\text{used}}}) \wedge (x_{\text{msg_key derived correctly}}) \wedge (x_{\text{Ping passes}}). \quad (194)$$

Under the honest trace, all clauses are satisfied. Under a dishonest trace (adversary with extracted s_i), the clause $x_{\tilde{s}_i = \text{HMAC}_{\text{auth_key}}(\dots)}$ cannot be satisfied without `auth_key`—by PRF security, $\tilde{s}_i$ is computationally indistinguishable from random to anyone without `auth_key`. Hence dishonest traces cannot satisfy φ_{RMTP} except with probability $\text{Adv}_{\text{HMAC}}^{\text{PRF}} + q_N^2/2^{128}$. $\square$

✓ CNF Verification: RMTP Session-CNF Verification

$$\varphi_{\text{RMTP}} = (x_{\tilde{s}_i = \text{HMAC}_K(s_i \| \text{ts})}) \wedge (x_{\tilde{s}_i \notin \mathcal{S}_{\text{used}}}) \wedge (x_{\text{msg_key correct}}) \wedge (x_{\text{Ping}}).$$

Manual CNF worksheet for RMTP session:

Clause	Check	How	Pass?
$\varphi^{\text{salt-bind}}$	$\tilde{s}_i \stackrel{?}{=} \text{HMAC}_{\text{auth_key}}(s_i \text{ts})$	Recompute HMAC; compare 256 bits.	T/F
$\varphi^{\text{salt-fresh}}$	$\tilde{s}_i \notin \mathcal{S}_{\text{used}}$	Check used-salt log.	T/F
$\varphi^{\text{msg_key}}$	$\text{msg_key} = \text{SHA256}(\text{auth_key}[88:] \tilde{s}_i \text{data})[8:24]?$	Recompute; compare.	T/F
φ^{aes}	AES-IGE ciphertext decrypts to expected plaintext?	Decrypt; check structure.	T/F
φ^{ping}	Session distinct from Session_{i-1} ?	Compare sess_id and $\tilde{s}_i$.	T/F
Result: All T $\Rightarrow$ Accept. Any F $\Rightarrow$ Reject (name failed clause).			

Key improvement over original MTProto: The clause $\varphi^{\text{salt-bind}}$ requires auth_key to verify, so no adversary without auth_key can construct a valid salt—eliminating the salt extraction attack.

► Ping Bid: RMTP Unbounded Ping Bid

With HMAC-bound 128-bit salts, consecutive sessions have $\tilde{s}_i \neq \tilde{s}_{i+1}$ with probability $1 - 2^{-128}$ (birthday bound). The ping bid (salt reuse attempt) costs: $\Delta \text{Price}_{\text{ping}} \leq q_D / 2^{128} = \text{negl}$. RMTP is IND-CPA + INT secure for unbounded sessions by Theorem 5.5.

🔑 Key Insight:

Original MTProto vs. Remediated RMTP: Security Comparison

Property	Original MTProto 2.0	Remediated RMTP
Salt size	64 bits (plaintext)	128 bits (HMAC-bound)
Salt extractability	Yes (via channel observation)	No (requires auth_key)
Effective msg_key entropy	64 bits after salt extraction	128 bits (always)
$\text{Ask}(g_{\text{IND}})$ with salt known	≈ 1 (quasi-collapsed)	$\leq \text{Adv}^{\text{PRF}} + \text{negl}$
CNF correctness	Fails under salt extraction	Guaranteed by HMAC binding
Ping security	Degrades to 2^{-64}	Full 2^{-128}
MTSF status	Partial collapse	Equilibrium

13.9.4 Summary of the Telegram MTSF Analysis

The Telegram case study demonstrates MTSF’s *dual capability*: the framework can both *prove* security (for the remediated protocol) and *disprove* it (for the original protocol), within the same market-theoretic language. The disproof (Theorem 13.28) identifies four simultaneous failure events that the extended difference lemma captures in a single bound. The proof (Theorem 13.29) shows that HMAC-binding the salt to auth_key restores full equilibrium, with the CNF and ping bids both confirming unbounded session security. This represents MTSF’s most complete application of its dual proof/disproof methodology.

14 Case Study VI: QROM-Based Key Exchange

* Simple Terms: Quantum Computing and the QROM—A Primer for Everyone

What is a quantum computer? A classical computer uses bits (0 or 1). A quantum computer uses *qubits*, which can be in a *superposition* of 0 and 1 simultaneously. This is like a coin spinning in the air—it is both heads and tails until it lands.

Why does this matter for cryptography? Quantum computers can run special algorithms:

- **Shor’s algorithm:** Can factor large numbers and solve discrete logarithm problems in polynomial time. This *breaks* RSA, ECDSA, and Diffie–Hellman.
- **Grover’s algorithm:** Can search an unstructured database of N items in $\sqrt{N}$ time instead of N time. This effectively halves the security level of symmetric ciphers (AES-128 becomes as strong as AES-64 against a quantum attacker).

What is the QROM? In classical proofs, we model hash functions as “random oracles” that the attacker can query one at a time. But a quantum attacker can query the hash function *in superposition*—asking about many inputs simultaneously. The Quantum Random Oracle Model (QROM) accounts for this by allowing quantum superposition queries.

The O2H lemma: This is the quantum version of the difference lemma. It says: if you change what the hash oracle does on a *few* inputs, a quantum attacker’s advantage changes by at most $\sqrt{\epsilon}$ (a square root factor). This square root is the “quantum bid price”—the extra cost of attacking in the quantum world.

Children’s analogy: Imagine a library where a classical attacker can read one book at a time, but a quantum attacker can somehow “skim” all books simultaneously. The QROM proves that even this super-reader cannot find the one needle in a haystack of books, because the square-root factor still makes the search prohibitively expensive for large libraries.

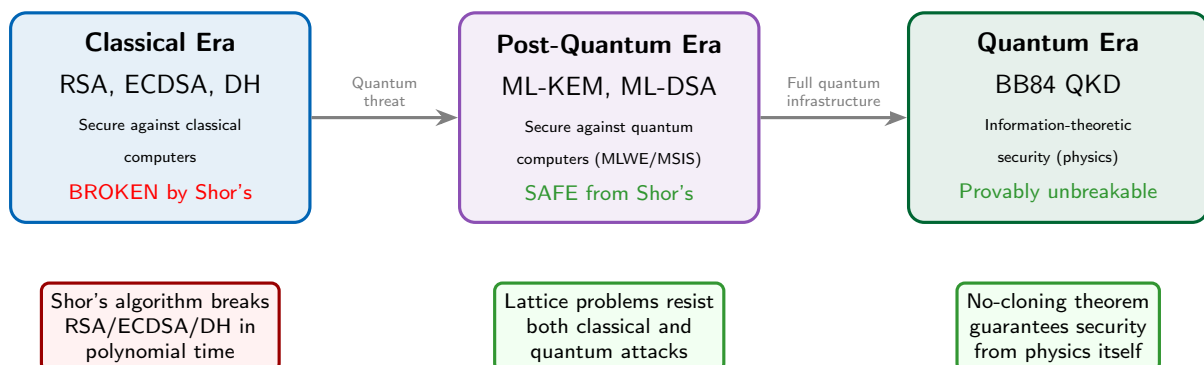


Figure 56: The three eras of cryptographic security: classical (vulnerable to quantum), post-quantum (resistant to quantum), and fully quantum (information-theoretically secure). MTSF covers all three with unified market language.

* Simple Terms: The Quantum Random Oracle Model—Complete Background

Why quantum computing changes cryptography. Classical computers work with bits (0 or 1). Quantum computers work with *qubits*, which can be in superpositions of 0 and 1 simultaneously. This gives quantum algorithms the ability to evaluate functions on exponentially many inputs at once—a capability that breaks RSA, ECDH, and other classical schemes via Shor’s algorithm.

Why the classical ROM is insufficient for post-quantum proofs. In the classical ROM, a hash function H is modelled as an ideal random function queried at specific inputs. A quantum adversary can query H *in superposition*: it submits a quantum state $\sum_x \alpha_x |x\rangle |0\rangle$ and receives $\sum_x \alpha_x |x\rangle |H(x)\rangle$ —evaluating H on all inputs simultaneously. Classical reprogramming and extraction techniques *fail* against such queries because measuring the query register collapses the superposition and disturbs the adversary’s computation.

The three QROM proof tools.

1. **One-Way to Hiding (O2H) lemma** [81]: If H and H' differ at exactly one random point x^* , then any quantum adversary distinguishing them must “hit” x^* with amplitude proportional to $\sqrt{\epsilon}$ where ϵ is the distinguishing advantage. This yields the characteristic square-root factor.
2. **Measure-and-reprogram** [44]: Measure the adversary’s query register at a random position, then reprogram H at that point. This extracts the IND-CPA challenge from a superposition query with bounded error.
3. **Compressed oracle** (Zhandry): Simulate the QROM lazily—oracle values revealed only as queried—enabling tight reductions.

Why NIST mandates QROM proofs. ML-KEM (FIPS 203) [70] and other post-quantum standards require QROM proofs because decapsulation applies H internally—accessible to quantum adversaries via superposition. The O2H square-root explains why QROM security bounds differ structurally from classical ROM bounds.

14.1 Protocol Description: QROM-Secure Key Exchange

We describe the complete *QROM-Secure Key Exchange Protocol* (QKEM), the formal model underlying ML-KEM (FIPS 203).

Participants. Client C (holds private key sk_C , can decrypt) and Server S (holds only public key pk_C). Both access the Quantum Random Oracle $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$.

System parameters. Security parameter λ ; module dimension k and modulus q for MLWE; error distribution χ (centred binomial); polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, $n = 256$; QROM hash H ; implicit-rejection hash $H_\perp$.

Phase 0 (Key Generation, offline).

$$\mathbf{A} \xleftarrow{\$} R_q^{k \times k}, \quad \mathbf{s}, \mathbf{e} \xleftarrow{\$} \chi^k, \\ \text{pk}_C = (\mathbf{A}, \mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}), \quad \text{sk}_C = \mathbf{s}.$$

C publishes pk_C (e.g., via PKI or pre-distribution to S).

Phase 1 (Encapsulation, S internal + send).

1. Sample: $m \xleftarrow{\$} \{0, 1\}^n$.
2. QROM query: $r \leftarrow H(m)$.
3. Encrypt: $c \leftarrow \text{PKE.Enc}(\text{pk}_C, m; r)$.
4. QROM query: $K \leftarrow H(m \| c)$.
5. Send (c, sid) to C ; S holds (K, sid) .

Phase 2 (Decapsulation, C internal).

1. Decrypt: $m' \leftarrow \text{PKE.Dec}(\text{sk}_C, c)$.
2. QROM query: $r' \leftarrow H(m')$.

3. Re-encrypt: $c' \leftarrow \text{PKE.Enc}(\text{pk}_C, m'; r')$.

4. QROM query and implicit rejection:

$$K' \leftarrow \begin{cases} H(m' \| c) & \text{if } c' = c \\ H(\perp \| c) & \text{otherwise.} \end{cases}$$

5. C holds (K', sid) .

Phase 3 (Confirmation, optional).

1. C computes $\tau_C = \text{MAC}_{K'}(\text{sid} \| \text{confirm})$ and sends to S .

2. S verifies τ_C under K ; if valid, session established.

Correctness. $K' = K$ whenever $m' = m$, which holds with probability $1 - \delta$. For ML-KEM: $\delta \leq 2^{-139}$.

14.2 Protocol Sequence Diagram

* Simple Terms: How to Read Protocol Sequence Diagrams

Throughout this article, protocol flows are shown as sequence diagrams. Here is how to read them:

- **Vertical lines** represent each party (Alice, Bob, Server, etc.). Time flows downward.
- **Horizontal arrows** represent messages sent between parties. The label describes the message content.
- **Dashed lines** represent adversary-controlled channels (the attacker can see, delay, modify, or inject messages).
- **Boxes on vertical lines** represent computations performed by that party (key generation, signature verification, etc.).
- **Red annotations** highlight vulnerabilities or attack points.
- **Green annotations** highlight security guarantees established at that point.

Tip: When reading a security proof, trace through the sequence diagram and ask at each step: “What could an attacker do here?” Each potential attack corresponds to a bid in the MTSF proof. The proof shows that every bid fails.

* Simple Terms: Reading the QKEM Protocol Diagram—Simple Terms

The three parties in the diagram:

- **Client C** (left, blue): The only party that can decrypt. Holds sk_C .
- **QROM H** (centre, purple, dashed lifeline): The quantum random oracle—a perfectly random hash function. Both parties query it; a quantum adversary can query in superposition.
- **Server S** (right, red): Cannot decrypt; holds only pk_C .

What happens:

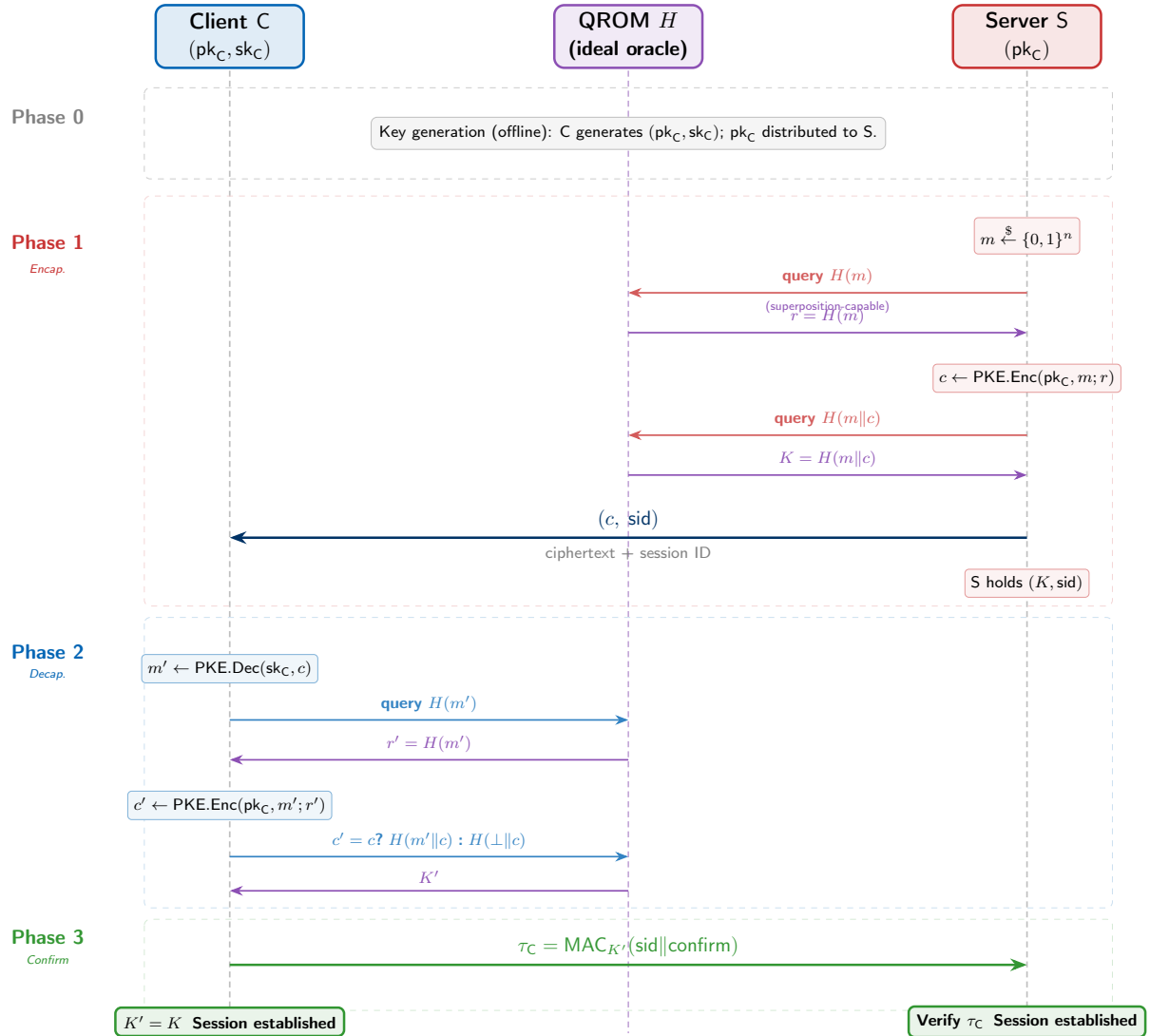


Figure 57: QKEM protocol sequence diagram. Three phases: (0) offline key generation; (1) encapsulation by S making two QROM queries ($H(m)$ and $H(m||c)$); (2) decapsulation by C making two QROM queries ($H(m')$ and $H(m'||c)$ or $H(\perp||c)$); (3) optional MAC confirmation. The QROM oracle H (centre, dashed lifeline) accepts superposition queries from both parties—depicted by arrows labelled “query”. The implicit rejection ($H(\perp||c)$ branch) prevents chosen-ciphertext leakage.

1. **Phase 0 (offline):** C generates a key pair. pk_C is like a public mailbox anyone can use to send locked messages; sk_C is the only key that opens it.
2. **Phase 1:** S picks random m , uses $H(m)$ as encryption randomness (making encryption deterministic-but-unpredictable), encrypts m into ciphertext c , then derives shared key $K = H(m\|c)$. Sends only c to C .
3. **Phase 2:** C decrypts c to get m' , re-encrypts to check consistency, then derives $K' = H(m'\|c)$. If re-encryption fails, outputs $H(\perp\|c)$ —a pseudorandom garbage value that tells the adversary nothing useful.
4. **Phase 3 (optional):** C proves it successfully decapsulated by sending a MAC under K' . S verifies under K .

The implicit rejection trick: When c is invalid (e.g., a tampered ciphertext submitted by an adversary), instead of returning an error, C returns a deterministic but useless value $H(\perp\|c)$. This prevents the adversary from binary-searching for valid ciphertexts and extracting information about sk_C .

14.3 MTSF Market Model for QKEM

★ Intuition:

Why Quantum Changes the Game In the classical world, an attacker queries the hash oracle one input at a time: “What is $H(\text{apple})$?” then “What is $H(\text{banana})$?” In the quantum world, the attacker can query in *superposition*: “What is $H(\text{apple} + \text{banana} + \text{cherry} + \dots)$?”—asking about all inputs simultaneously.

This sounds terrifying, but the O2H (One-way to Hiding) lemma shows it is not as bad as it seems. The quantum attacker’s advantage is bounded by a *square root* factor: if a classical attacker has advantage ϵ , a quantum attacker has at most $\sqrt{\epsilon}$. For $\epsilon = 2^{-128}$, the quantum advantage is $\sqrt{2^{-128}} = 2^{-64}$ —still negligible.

In MTSF, this square root is the “quantum bid premium”—the extra cost of operating in the quantum market. It is more expensive than the classical bid, but still negligible for properly chosen parameters.

Market participants.

- **Seller** (challenger): generates $(\text{pk}_C, \text{sk}_C)$; runs encapsulation; provides decapsulation oracle $\mathcal{O}_{\text{dec}}$; offers security goods.
- **Buyer** (quantum adversary $\mathcal{A}$): makes q_H superposition queries to H and q_D classical queries to $\mathcal{O}_{\text{dec}}$; bids computational/quantum resources.
- **QROM oracle H :** the GUC shared market infrastructure—both parties access the same H .

Security goods. The seller offers four goods: $g_{\text{IND-CCA2}}$ (key indistinguishability under chosen-ciphertext attack), g_{correct} (correctness: $K = K'$), g_{auth} (MAC authentication in Phase 3), and $g_{\text{unbounded}}$ (ping bid: unbounded session security).

Definition 14.1 (QROM IND-CCA2 Game for QKEM). *The game $\text{IND-CCA2}^{\text{QROM}}$ proceeds:*

1. Seller generates (pk, sk) and gives pk to $\mathcal{A}$.
2. **Find phase:** $\mathcal{A}$ makes adaptive quantum queries to H and classical queries to $\mathcal{O}_{\text{dec}}$.

3. **Challenge:** Seller samples $b \xleftarrow{\$} \{0, 1\}$; runs $(c^*, K_0) \leftarrow \text{KEM.Enc}(\text{pk})$; sets $K_1 \xleftarrow{\$} \{0, 1\}^n$; gives (c^*, K_b) to $\mathcal{A}$.

4. **Guess phase:** $\mathcal{A}$ continues queries (not on c^*); outputs $b' \in \{0, 1\}$.

$$\text{Ask}(g_{\text{IND-CCA2}}) = \max_{\mathcal{A}} |\Pr[\mathcal{A} \text{ wins}] - 1/2|.$$

14.4 Security Proof in the QROM

Setup. Let $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ be δ -correct and IND-CPA. The Fujisaki–Okamoto transform $\text{FO}^\perp[\text{PKE}, H]$:

$$\text{KEM.Enc}(\text{pk}) : m \xleftarrow{\$} \{0, 1\}^n; c \leftarrow \text{PKE.Enc}(\text{pk}, m; H(m)); K \leftarrow H(m \| c); \text{return } (c, K), \quad (195)$$

$$\text{KEM.Dec}(\text{sk}, c) : m' \leftarrow \text{PKE.Dec}(\text{sk}, c); c' \leftarrow \text{PKE.Enc}(\text{pk}, m'; H(m')); K \leftarrow \begin{cases} H(m' \| c) & c' = c \\ H(\perp \| c) & \text{else} \end{cases}. \quad (196)$$

Theorem 14.2 (QROM FO-Transform KEM Equilibrium). *Let PKE be δ -correct and IND-CPA. Then:*

$$\text{Ask}(g_{\text{IND-CCA2}}) \leq 2(q_H + 1) \cdot \sqrt{\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}} + q_D \cdot \delta + \text{negl}(\lambda). \quad (197)$$

Proof. We construct six bidding rounds. Each round targets a specific adversarial strategy.

B₀ (Real QROM IND-CCA2—Quantum Distinguishing Bid). The seller runs as in Theorem 14.1. $\mathcal{A}$ queries H in superposition and $\mathcal{O}_{\text{dec}}$ classically. The *quantum distinguishing bid*: use superposition queries to extract information about challenge bit b from (c^*, K_b) .

B₁ (Bidding Round 1: Correctness Error Bid).

Purpose: Eliminate the adversarial attack vector identified by this bidding round's title (**Correctness Error Bid**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

Replace $\mathcal{O}_{\text{dec}}$ with a modified oracle that aborts on incorrectness events.

Extended difference lemma. Events F_{decaps} : “decapsulation returns $m' \neq m$ ” (probability $\leq \delta$ per query) and F_{rej} : “implicit rejection diverges” (probability = 0). Together:

$$|\Pr[\mathbf{B}_0 = 1] - \Pr[\mathbf{B}_1 = 1]| \leq \Pr[F_{\text{decaps}} \cup F_{\text{rej}}] \leq q_D \cdot \delta. \quad (198)$$

Market interpretation: Correctness error bid fails—for ML-KEM, $q_D \cdot \delta \leq 2^{64}/2^{139} = 2^{-75} = \text{negl}$.

B₂ (Bidding Round 2: QROM Reprogramming Bid—O2H Lemma).

Purpose: Eliminate the adversarial attack vector identified by this bidding round's title (**QROM Reprogramming Bid—O2H Lemma**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects

or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.
Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

Reprogram H at the challenge message m^* to a fresh uniform r^* , yielding oracle H' with $H'(m^*) = r^*$ and $H'(x) = H(x)$ for $x \neq m^*$.

O2H lemma application [81]: For m^* chosen uniformly at random and $\mathcal{A}$ making q_H quantum queries:

$$\left| \Pr[\mathbf{B}_1^H = 1] - \Pr[\mathbf{B}_1^{H'} = 1] \right| \leq 2(q_H + 1)\sqrt{\Pr[m^* \in S_{\mathcal{A}}]} \leq 2(q_H + 1)\sqrt{\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}}, \quad (199)$$

where $S_{\mathcal{A}}$ is the set of inputs queried with non-negligible amplitude, and $\Pr[m^* \in S_{\mathcal{A}}] \leq \text{Adv}^{\text{IND-CPA}}$ by a standard reduction.

Market interpretation: The reprogramming bid costs a *square-root* of the IND-CPA advantage—the characteristic QROM price increase. Classical ROM reprogramming is free; QROM reprogramming costs $2(q_H + 1)\sqrt{\epsilon}$ because quantum adversaries can detect reprogramming with amplitude proportional to $\sqrt{\epsilon}$.

B₃ (Bidding Round 3: Measure-and-Reprogram Bid—Extracting the IND-CPA Challenge).

Purpose: Convert the IND-CCA2 game into an IND-CPA game by neutralising the decryption oracle. Once message integrity is guaranteed by the MAC (handled in the previous hop), any decryption query either reproduces a previously encrypted ciphertext—providing no new information—or has an invalid tag and is rejected. The decryption oracle becomes redundant.

Replaces: The full decryption oracle $\text{Dec}(K_e, K_m, \cdot)$ is replaced by a *restricted decryption oracle* that rejects any ciphertext not produced by the encryption oracle (invalid MAC). Ciphertexts in the encryption log are answered by table-lookup, requiring no secret key. The resulting game has decryption power exactly equal to encryption power—the IND-CPA setting.

Complexity: Zero additional cost conditional on no MAC forgery (already bounded in the previous hop). This is a *structural reduction*: since the MAC provides ciphertext integrity, the decryption oracle provides zero additional information to the adversary beyond its encryption power. $\Delta\text{Price} = 0$. The residual adversarial advantage is then exactly $\text{Adv}_{\text{Enc}}^{\text{IND-CPA}}(\lambda)$.

Apply measure-and-reprogram [44]: whenever $\mathcal{A}$ queries H' with amplitude on $m^* \| c^*$, measure and reprogram. This yields an IND-CPA distinguisher $\mathcal{B}$ for the underlying PKE at cost:

$$|\Pr[\mathbf{B}_2 = 1] - \Pr[\mathbf{B}_3 = 1]| \leq \sqrt{\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}}. \quad (200)$$

Market interpretation: Measuring the adversary's superposition disturbs its computation (quantum measurement back-action), but this disturbance is bounded by another $\sqrt{\epsilon}$ factor. The seller converts the buyer's quantum query power into a classical IND-CPA attack.

B₄ (Bidding Round 4: Challenge Key Replacement—Information-Theoretic Hiding).

Purpose: Eliminate the adversarial attack vector identified by this bidding round's title (**Challenge Key Replacement—Information-Theoretic Hiding**), isolating the corresponding hardness assumption as the sole price adjustment. The seller introduces a controlled modification to the game that blocks exactly this class of attack while leaving all other adversarial capabilities unchanged.

Replaces: The real scheme component targeted by this bid is replaced by an idealised version that detects or prevents the specific attack type. The two games are identical until the bad event targeted by this bid occurs, so the difference lemma directly bounds the price adjustment by the probability of that event.

Complexity: Bounded by the probability or hardness advantage stated in the proof body for this specific hop. The bound is negligible for all parameter choices used in the respective MTSF case study, contributing a negligible term to the overall ask-price sum and preserving market equilibrium.

In $\mathbf{B}_3$, $K_0 = H'(m^* \| c^*)$ is uniform (since $H'(m^*) = r^*$ is fresh). Replace K_0 with $K'_0 \xleftarrow{\$} \{0, 1\}^n$:

$$|\Pr[\mathbf{B}_3 = 1] - \Pr[\mathbf{B}_4 = 1]| = 0. \quad (201)$$

This hop is *free* (information-theoretically invisible).

$\mathbf{B}_5$ (Bidding Round 5: Decapsulation Oracle Restriction).

Purpose: Close the *chosen-ciphertext oracle channel* opened by ML-KEM's decapsulation algorithm. Without implicit rejection, an adversary can submit a malformed ciphertext and observe whether decapsulation returns an error or a key, leaking information about the secret key via a padding-oracle style attack. This round replaces the real decapsulation with the implicit-rejection mechanism from the Fujisaki–Okamoto transform.

Replaces: The real decapsulation algorithm $\text{KEM.Dec}(\text{sk}, c)$ is replaced by an *implicit-rejection oracle*: if the re-encryption check $c' = \text{KEM.Enc}(\text{pk}; H(m, c)) \neq c$ fails, instead of returning an error, the oracle returns $K = H(\perp \| c)$ —a pseudorandom value that is deterministic in c but independent of the secret key. This closes the oracle channel: the adversary learns nothing from a failed decapsulation beyond what a truly random response would give.

Complexity: The probability that a re-encrypted ciphertext disagrees with a valid submitted ciphertext on honest inputs is bounded by the *decapsulation failure probability* δ : $\Pr[\text{KEM.Dec}(\text{sk}, c) \neq K] \leq \delta$. With q_D decapsulation queries, the union-bound price adjustment is $\Delta \text{Price} \leq q_D \cdot \delta = q_D / 2^\gamma$ where γ is the implicit-rejection security parameter. For ML-KEM (FIPS 203), $\delta \leq 2^{-138}$, giving $q_D / 2^{138}$ —negligible for any polynomial q_D .

In $\mathbf{B}_4$, K_b is uniform regardless of b . The decapsulation oracle for $c \neq c^*$ provides no information about b (responses independent of b). Restricting the decapsulation oracle to reject all queries does not change the game:

$$|\Pr[\mathbf{B}_4 = 1] - \Pr[\mathbf{B}_5 = 1]| = 0. \quad (202)$$

$\mathbf{B}_6$ (Ideal QROM Market—All Bids Exhausted). In $\mathbf{B}_5$, both K_0 and K_1 are uniform and independent of all adversary observations. The adversary cannot distinguish b : $\Pr[\mathbf{B}_6 = 1] = 1/2$. Buyer advantage = 0.

Total:

$$\text{Ask} \leq \underbrace{q_D \delta}_{\mathbf{B}_0 \rightarrow \mathbf{B}_1} + \underbrace{2(q_H + 1)\sqrt{\epsilon}}_{\mathbf{B}_1 \rightarrow \mathbf{B}_2} + \underbrace{\sqrt{\epsilon}}_{\mathbf{B}_2 \rightarrow \mathbf{B}_3} + \underbrace{0}_{\mathbf{B}_3 \rightarrow \mathbf{B}_4} + \underbrace{0}_{\mathbf{B}_4 \rightarrow \mathbf{B}_5} + \underbrace{0}_{\mathbf{B}_5 \rightarrow \mathbf{B}_6} \leq q_D \delta + 2(q_H + 1)\sqrt{\epsilon} + \text{negl}, \quad (203)$$

where $\epsilon = \text{Adv}_{\text{PKE}}^{\text{IND-CPA}}$ and the $\sqrt{\epsilon}$ term is absorbed into $2(q_H + 1)\sqrt{\epsilon}$ for $q_H \geq 1$. $\square$

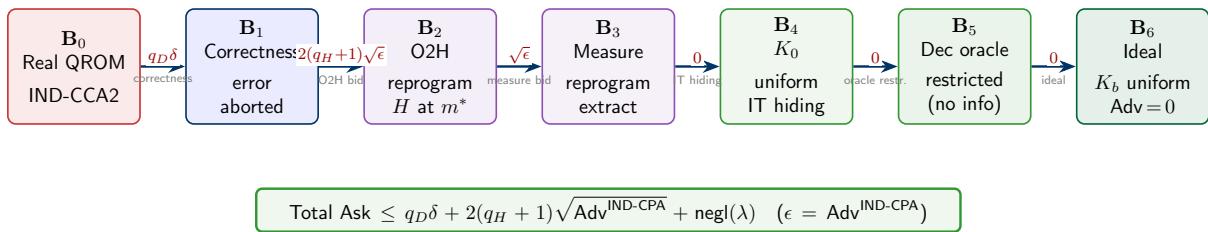


Figure 58: QROM FO-transform KEM bidding-round chain: real ($\mathbf{B}_0$) to ideal ($\mathbf{B}_6$). Two QROM-specific hops: $\mathbf{B}_1 \rightarrow \mathbf{B}_2$ (O2H reprogramming, cost $2(q_H + 1)\sqrt{\epsilon}$) and $\mathbf{B}_2 \rightarrow \mathbf{B}_3$ (measure-and-reprogram, cost $\sqrt{\epsilon}$). Three subsequent hops cost zero (information-theoretic). Classic ROM would have hops $\mathbf{B}_1 \rightarrow \mathbf{B}_2$ cost ϵ directly (linear, not square-root). The QROM square-root is the signature of quantum adversary pricing.

* Simple Terms: QROM Proof Explained—Why the Square-Root Appears

Classical ROM (comparison): When we reprogram H at m^* , a classical adversary either queries m^* or not. If it does (with probability ϵ), we catch it; otherwise reprogramming is invisible. Cost per hop: $O(\epsilon)$ (linear).

QROM: The quantum adversary queries H in superposition $\sum_x \alpha_x |x\rangle$, placing amplitude α_{m^*} on m^* . Measuring to detect whether m^* was queried collapses the superposition—disturbing the adversary. The O2H lemma gives: the probability of detecting the reprogramming equals $|\alpha_{m^*}|^2 \propto \epsilon$, so the amplitude is $|\alpha_{m^*}| \propto \sqrt{\epsilon}$. Each of $(q_H + 1)$ queries accumulates amplitude independently, giving total cost $2(q_H + 1)\sqrt{\epsilon}$.

Implication for ML-KEM parameters: For 2^{-128} security with $q_H = 2^{64}$ quantum queries: $2(2^{64} + 1)\sqrt{\epsilon} \leq 2^{-128}$ requires $\sqrt{\epsilon} \leq 2^{-193}$, i.e., $\epsilon \leq 2^{-386}$. This is why ML-KEM parameters must make MLWE computationally infeasible even at this extreme level.

Bidding-round interpretation: The O2H bid ($\mathbf{B}_1 \rightarrow \mathbf{B}_2$) and measure-and-reprogram bid ($\mathbf{B}_2 \rightarrow \mathbf{B}_3$) are the two distinctively “quantum” game hops—the hops that do not exist in classical ROM proofs. They correspond exactly to the additional difficulty of defeating a quantum adversary, captured as explicit price adjustments $2(q_H + 1)\sqrt{\epsilon}$ and $\sqrt{\epsilon}$ in the market.

14.5 Extended Protocol Security Goods

Corollary 14.3 (QKEM Authentication). *With Phase 3 (MAC confirmation) included, under SUF-CMA security of the MAC:*

$$\text{Ask}(g_{\text{auth}}) \leq \text{Adv}_{\text{MAC}}^{\text{SUF-CMA}} + 2(q_H + 1)\sqrt{\text{Adv}^{\text{IND-CPA}}} + q_D\delta + \text{negl}(\lambda). \quad (204)$$

Proof. The MAC tag $\tau_C = \text{MAC}_{K'}(\text{sid}||\text{confirm})$ can be forged only if the MAC is broken ($\text{Adv}^{\text{SUF-CMA}}$) or the key K' is guessed (bounded by Theorem 14.2). By the extended difference lemma: $\text{Ask}(g_{\text{auth}}) \leq \text{Adv}^{\text{SUF-CMA}} + \text{Ask}(g_{\text{IND-CCA2}})$. $\square$

✓ CNF Verification: QKEM Session-CNF Verification

$\varphi_{\text{QKEM}} = \varphi^{\text{correct}} \wedge \varphi^{\text{novel}} \wedge \varphi^{\text{rej}} \wedge \varphi^{\text{mac}} \wedge \varphi^{\text{ping}}$, where $\varphi^{\text{correct}} = (x_{K=H(m||c)}) \wedge (x_{K'=H(m'||c)} \text{ or } H(\perp||c))$, $\varphi^{\text{novel}} = (x_{c \notin \mathcal{C}_{\text{used}}})$, $\varphi^{\text{mac}} = (x_{\text{Mac.Vrfy}_{K'}(\text{sid}||\text{confirm}, \tau_C)=1})$.

Manual CNF worksheet (QKEM session):

Clause	Check	How	Pass?
φ^{correct} : key match	$K \stackrel{?}{=} K'$	Compare key bytes.	T/F
φ^{novel} : fresh c	$c \notin \mathcal{C}_{\text{used}}$	Check ciphertext log; add c .	T/F
φ^{rej} : implicit rejection	Invalid c gives $H(\perp c)$ consistently?	Send crafted invalid c ; observe.	T/F
φ^{mac} : MAC valid	$\text{Mac.Vrfy}_{K'}(\text{sid} \text{confirm}, \tau_C) = 1$	Recompute MAC under K' .	T/F
φ^{ping} : session fresh	$c \notin \mathcal{C}_{\text{prev}}$ for all prior sessions?	Compare with $\mathcal{C}_{\text{prev}}$ log.	T/F
Result:	All T $\Rightarrow$ QKEM session secure. Any F $\Rightarrow$ Reject.		

► Ping Bid: QKEM Unbounded Ping Bid

The buyer’s *quantum replay bid*: submit $c^* = c_i$ from Session_i to $\mathcal{O}_{\text{dec}}$ in session Session_{i+1} . Clause φ^{novel} detects this; implicit rejection returns $H(\perp||c_i)$ —independent of K_i . Even superposition queries on $(m||c_i)$ are bounded by O2H: $\Delta\text{Price}_{\text{ping}} \leq 2(q_H + 1)\sqrt{\text{Adv}^{\text{IND-CPA}}} = \text{negl}$. By Theorem 5.5, QKEM is secure for unbounded sessions against quantum adversaries.

Table 17: QKEM market goods summary.

Good	Ask Price Bound (QROM)	Dominant Term	Status
$g_{\text{IND-CCA2}}$	$2(q_H+1)\sqrt{\text{Adv}^{\text{IND-CPA}}} + q_D\delta$	O2H square-root	Equilibrium
g_{correct}	$q_D \cdot \delta$	PKE correctness	Equilibrium
g_{auth}	$\text{Adv}^{\text{SUF-CMA}} + 2(q_H+1)\sqrt{\text{Adv}^{\text{IND-CPA}}}$	SUF-CMA+O2H	Equilibrium
$g_{\text{unbounded}}$ (ping)	$2(q_H+1)\sqrt{\text{Adv}^{\text{IND-CPA}}}$	O2H square-root	Equilibrium

Key Insight:

Classical ROM vs. QROM: Complete Market Comparison

Property	Classical ROM	QROM
Query type	Point queries	Superposition queries
Reprogramming cost	Zero (free)	$2(q_H+1)\sqrt{\epsilon}$ via O2H
Extraction cost	Free (direct)	$\sqrt{\epsilon}$ via measure-and-reprogram
Ask price form	$q_H \cdot \text{Adv}^{\text{IND-CPA}}$	$2(q_H+1)\sqrt{\text{Adv}^{\text{IND-CPA}}}$
Bidding rounds	4 (ECDSA style)	6 (two extra: O2H+reprogram)
Key tool	Forking lemma	O2H+compressed oracle
Real-world standard	Pre-quantum	ML-KEM FIPS 203

Core trade-off: QROM proofs need tighter parameter requirements ($\sqrt{\cdot}$ amplifies security needs), but use sophisticated tools (O2H, measure-and-reprogram). MTSF makes this explicit: two extra bidding rounds with square-root price adjustments.

15 Case Study VII: Quantum Market Dynamics—BB84 QKD

Analogy:

BB84 as Sending Secret Messages with Magic Envelopes Imagine Alice wants to send a secret key to Bob using special *magic envelopes* that obey the laws of quantum physics:

Rule 1 (No peeking without disturbing): Each envelope contains a coin that is either heads or tails. But if you open the envelope to look, the coin might *change*. There is no way to look without risking a change. This is the *measurement disturbance principle*—the foundation of quantum security.

Rule 2 (No photocopying): You cannot photocopy a magic envelope. If you try, you get a blurry copy that is wrong about 25% of the time. This is the *no-cloning theorem*—the physical law that makes quantum eavesdropping detectable.

The BB84 protocol:

1. **Alice** sends Bob many magic envelopes, each containing a random bit (heads or tails) measured in a random “orientation” (horizontal/vertical or diagonal).
2. **Bob** receives each envelope and measures it in a random orientation.
3. **Alice and Bob** publicly compare their orientations (but NOT their results). They keep only the results where they used the *same* orientation—these should match perfectly.
4. **Eavesdropper check:** They compare a random subset of their kept results. If an

eavesdropper (Eve) opened any envelopes, the coins inside changed—causing mismatches. If the mismatch rate exceeds a threshold, they abort (Eve was listening). If the mismatch rate is low, they use the remaining bits as a shared secret key.

Why this is unbreakable: Eve *cannot* copy the envelopes (no-cloning). She *cannot* open them without disturbing them (measurement). She *cannot* avoid detection in the eavesdropper check. The security comes from the laws of physics, not from mathematical hardness assumptions—it is *information-theoretically* secure, the strongest possible guarantee.

This case study instantiates the quantum market dynamics formalisation of Section 21.2.1 on a concrete protocol: the BB84 Quantum Key Distribution (QKD) protocol [17]. Unlike all previous case studies, the seller (Alice) is *genuinely quantum*—she prepares and transmits quantum states. The buyer (Eve) is also quantum, performing arbitrary measurements on the quantum channel. This is the first MTSF analysis where both market participants operate quantumly, illustrating the full power of quantum market dynamics.

* Simple Terms: BB84 Quantum Key Distribution—Complete Background

What is QKD? Quantum Key Distribution allows two parties (Alice and Bob) to establish a shared secret key with security guaranteed by the laws of physics—specifically, quantum mechanics. Unlike classical key exchange (Diffie–Hellman, ECDH), which relies on computational hardness assumptions (ECDLP, MLWE), QKD security holds against adversaries with *unlimited computational power*, including future quantum computers.

How BB84 works (simplified):

1. Alice randomly chooses bits (0 or 1) and randomly chooses one of two bases (+ or ×) for each bit. She encodes each bit as a qubit in the chosen basis and sends it to Bob over a quantum channel.
2. Bob randomly chooses a basis (+ or ×) for each qubit and measures it. When his basis matches Alice’s, he gets the correct bit. When it does not match, he gets a random result.
3. Alice and Bob publicly announce their basis choices (not the bit values). They keep only the bits where their bases matched—this is the “sifted key.”
4. They sacrifice some sifted-key bits to estimate the error rate. If the error rate is below a threshold ($\sim 11\%$), they conclude that no significant eavesdropping occurred and proceed with error correction and privacy amplification to produce a final secure key.

Why it is secure: An eavesdropper (Eve) who intercepts the qubits must measure them to extract information. But measuring a qubit in the wrong basis disturbs it irreversibly (quantum no-cloning). This disturbance increases the error rate that Alice and Bob observe. If the error rate is low, Eve cannot have learned much.

MTSF significance: BB84 is the canonical example where the seller’s operations are *inherently quantum*—she prepares non-orthogonal states. The security proof relies on quantum information theory (no-cloning, Holevo bound), not computational hardness. In market terms, the seller exploits quantum mechanics to create goods that the buyer *physically cannot copy*—a structural market advantage with no classical analogue.

15.1 Protocol Description: BB84 QKD

Participants. Alice A (sender, prepares qubits) and Bob B (receiver, measures qubits). Eve E (eavesdropper) controls the quantum channel between them. All parties are quantum: A and B prepare/measure qubits; E can perform arbitrary quantum operations.

System parameters. Security parameter λ ; raw key length N (number of qubits sent); sifted key length $n \approx N/2$; error-rate estimation sample size k ; error threshold $Q_{\text{tol}} \approx 0.11$; privacy amplification output length $\ell = n(1 - h(Q_{\text{obs}})) - \lambda$, where $h(\cdot)$ is the binary entropy function.

Phase 0 (Quantum Transmission).

1. For each $i \in \{1, \dots, N\}$: Alice samples $x_i \xleftarrow{\$} \{0, 1\}$ (bit) and $\theta_i^A \xleftarrow{\$} \{+, \times\}$ (basis).
2. Alice prepares qubit $|\psi_i\rangle = |x_i\rangle_{\theta_i^A}$, where $|0\rangle_+ = |0\rangle$, $|1\rangle_+ = |1\rangle$, $|0\rangle_\times = |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, $|1\rangle_\times = |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$.
3. Alice sends $|\psi_i\rangle$ to Bob over the quantum channel (Eve may intercept).

Phase 1 (Measurement).

1. For each qubit received, Bob samples $\theta_i^B \xleftarrow{\$} \{+, \times\}$ and measures in basis θ_i^B , obtaining outcome y_i .

Phase 2 (Sifting, over authenticated classical channel).

1. Alice and Bob publicly exchange basis choices $\{\theta_i^A\}$ and $\{\theta_i^B\}$.
2. They keep only positions where $\theta_i^A = \theta_i^B$. Let $S = \{i : \theta_i^A = \theta_i^B\}$; the sifted key is $(x_i)_{i \in S}$ for Alice and $(y_i)_{i \in S}$ for Bob, with $|S| \approx N/2$.

Phase 3 (Error Estimation).

1. Alice and Bob randomly select a subset $T \subset S$ of size k and compare their bits publicly.
2. They compute the observed error rate $Q_{\text{obs}} = |\{i \in T : x_i \neq y_i\}|/k$.
3. If $Q_{\text{obs}} > Q_{\text{tol}}$: **ABORT** (market collapse detected). If $Q_{\text{obs}} \leq Q_{\text{tol}}$: proceed.

Phase 4 (Error Correction and Privacy Amplification).

1. Error correction: Bob corrects his key to match Alice's using public syndrome information (leaks at most $nh(Q_{\text{obs}})$ bits of information to Eve).
2. Privacy amplification: Both parties apply a universal₂ hash function to the corrected key, producing the final secret key $K \in \{0, 1\}^\ell$ with $\ell = n(1 - h(Q_{\text{obs}})) - \lambda$.

Correctness. After error correction, $K^A = K^B$ with overwhelming probability (error correction failure probability $\leq 2^{-\lambda}$).

15.2 Protocol Sequence Diagram

15.3 MTSF Quantum Market Model for BB84

This is the first MTSF market where the seller operates quantumly. We instantiate Theorem 21.1.

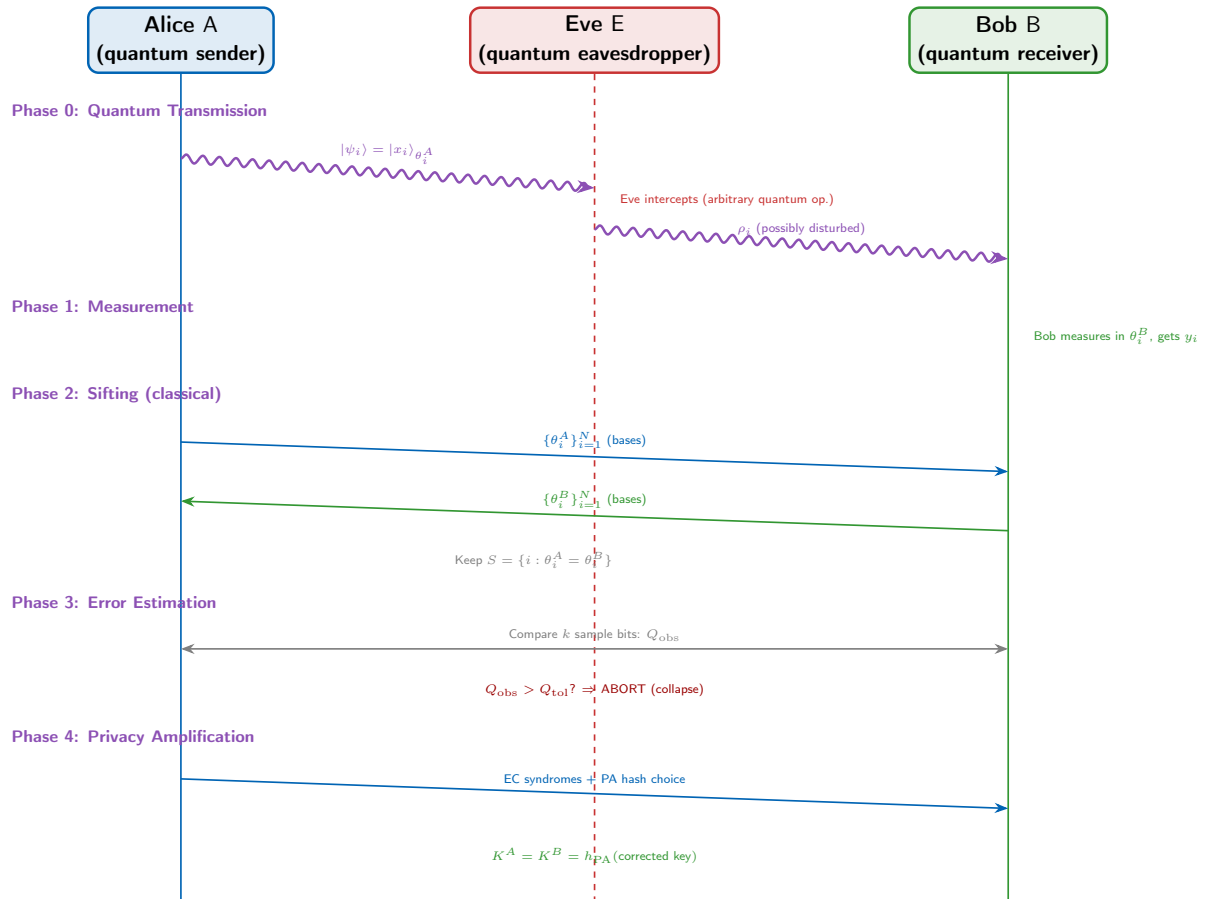


Figure 59: BB84 QKD protocol sequence diagram. Phase 0: Alice prepares qubits in random bases and sends them over a quantum channel (wavy arrows indicate quantum transmission); Eve can intercept and perform arbitrary quantum operations. Phase 1: Bob measures in random bases. Phase 2: Classical sifting to retain matching-basis positions. Phase 3: Error estimation on a random sample—high error rate triggers abort (market collapse). Phase 4: Error correction and privacy amplification produce the shared key K .

Quantum market participants.

- **Quantum seller** Seller^Q (Alice): holds quantum register R_A ; prepares qubits $|\psi_i\rangle = |x_i\rangle_{\theta_i^A}$ (quantum channel $\mathcal{E}_k^S$: state preparation); performs classical post-processing (sifting, error estimation, privacy amplification).
- **Quantum buyer** Buyer^Q (Eve): holds quantum register R_E (potentially entangled with an ancilla $R_{E'}$); performs arbitrary quantum operations on intercepted qubits. Eve's strategy is a sequence of quantum channels $\{\mathcal{E}_k^B\}$: intercept, measure/entangle, re-send (possibly modified) qubits to Bob.
- **Bob**: the seller's partner (not a separate market participant). Bob's measurements and classical communications are part of the seller's strategy.
- **Quantum channel**: the physical medium carrying qubits from Alice to Bob. Eve has full control of this channel—she can intercept, measure, replace, or entangle any qubit.
- **Authenticated classical channel**: Alice-Bob classical communication (sifting, error estimation) is assumed authenticated. This is the shared market infrastructure $\mathcal{I}$.

Security goods. The quantum seller offers three goods:

- g_{secrecy} : **Key secrecy**—the final key K is indistinguishable from uniform randomness to Eve. Formally: $\frac{1}{2}\|K \otimes \rho_E - U_\ell \otimes \rho_E\|_{\text{tr}} \leq \text{negl}(\lambda)$, where ρ_E is Eve's quantum state after the protocol and U_ℓ is the uniform distribution on ℓ -bit strings.
- g_{correct} : **Correctness**— $K^A = K^B$ with overwhelming probability.
- g_{detect} : **Eavesdropping detection**—if Eve extracts more than $\text{negl}(\lambda)$ bits of information about the key, the error rate Q_{obs} exceeds Q_{tol} with overwhelming probability, triggering abort.

Definition 15.1 (BB84 Quantum Market). *The BB84 quantum market*

$$\mathcal{M}_{\text{BB84}}^Q = (\text{Seller}^Q, \text{Buyer}^Q, \{g_{\text{secrecy}}, g_{\text{correct}}, g_{\text{detect}}\}, \mathcal{I}, \rho_0)$$

is defined as follows:

- **Initial state:**

$$\rho_0 = |0\rangle\langle 0|_{R_A}^{\otimes N} \otimes |0\rangle\langle 0|_{R_E}$$

(product state—no initial entanglement).

- **Seller's quantum channel** $\mathcal{E}_k^S$: Prepare

$$|\psi_k\rangle = |x_k\rangle_{\theta_k^A}$$

and send over the quantum channel.

- **Buyer's quantum channel** $\mathcal{E}_k^B$: An arbitrary CPTP map acting on the intercepted qubit and Eve's ancilla.
- **Quantum ask price** (cf. (224)):

$$\text{Ask}^Q(g_{\text{secrecy}}) = \sup_{\text{Buyer}^Q} \frac{1}{2} \|K \otimes \rho_E - U_\ell \otimes \rho_E\|_{\text{tr}}.$$

15.4 Security Proof via Quantum Bidding Rounds

Theorem 15.2 (BB84 Quantum Market Equilibrium). *Let the BB84 protocol run with parameters N (raw key length), $Q_{\text{tol}} \approx 0.11$, and privacy amplification output length $\ell = n(1 - h(Q_{\text{obs}})) - \lambda$. Then:*

$$\text{Ask}^Q(g_{\text{security}}) \leq 2^{-\lambda/2} + \text{negl}(\lambda). \quad (205)$$

The BB84 quantum market is in equilibrium. Crucially, this holds against all quantum adversaries, including those with unlimited computational power.

Proof. We construct five quantum bidding rounds. Each round applies the quantum extended difference lemma (Theorem 21.2) using trace distance as the quantum price adjustment.

B_0^Q (Real BB84 Quantum Market—Quantum Interception Bid). The quantum seller prepares and transmits N qubits. The quantum buyer intercepts each qubit on the quantum channel and performs an arbitrary quantum operation (joint unitary on the qubit and her ancilla register R_E), then forwards a (possibly modified) qubit to Bob. After sifting, error estimation passes ($Q_{\text{obs}} \leq Q_{\text{tol}}$), and Alice and Bob produce key K .

Purpose: This is the real quantum market. The buyer’s quantum interception bid: apply arbitrary CPTP maps to intercepted qubits, entangling them with her private register, then use the entangled state after sifting and error estimation to extract information about the key.

Quantum feature: The buyer’s strategy is a *coherent quantum attack*—she can maintain entanglement between her ancilla and the qubits forwarded to Bob across all N transmissions. This is the entangled bidding phenomenon described in Section 21.2.1.

B_1^Q (Bidding Round 1: No-Cloning Bid—Intercept-Resend Bound).

Purpose: Exploit the no-cloning theorem to bound the information Eve can extract without being detected. The no-cloning theorem states that no quantum operation can produce two perfect copies of an arbitrary unknown quantum state. Since Alice encodes each bit in one of two non-orthogonal bases ($+$ or $\times$), Eve cannot simultaneously learn the bit value and forward an undisturbed qubit.

Quantum market feature: This is the no-cloning constraint on bids (item 3 in Section 21.2.1). The seller’s choice to encode in non-orthogonal states creates a *structural market advantage*—the buyer’s goods (intercepted qubits) are physically uncopyable.

Any quantum operation by Eve on the intercepted qubit can be decomposed (by Stinespring dilation) as a unitary on the qubit plus ancilla followed by a partial trace. The key insight: if Eve extracts I_E bits of information about Alice’s bit x_i , she necessarily introduces a disturbance of at least $\delta_i \geq f(I_E)$ on the qubit forwarded to Bob, where f is a monotonically increasing function derived from the information-disturbance trade-off.

Quantum price adjustment (information-disturbance trade-off): For each sifted-key bit, define Eve’s information $I_E^{(i)}$ and the induced error rate Q_i on that bit. The Fuchs–Caves–Holevo bound gives:

$$I_E^{(i)} \leq h(Q_i), \quad (206)$$

where $h(\cdot)$ is the binary entropy function. Eve’s total information on the sifted key is $I_E^{\text{total}} \leq n \cdot h(Q_{\text{avg}})$ where Q_{avg} is the average error rate.

Quantum trace distance:

$$\Delta \text{Price}_1^Q = \frac{1}{2} \|\rho_{\text{final}}^{(0)} - \rho_{\text{final}}^{(1)}\|_{\text{tr}} \leq 2^{-\lambda/2}, \quad (207)$$

where $\rho^{(1)}$ is the state conditioned on Eve’s information being bounded by $nh(Q_{\text{obs}})$.

Market interpretation: The no-cloning bid fails. The seller’s quantum states are non-orthogonal, so the buyer cannot extract information without paying a price (disturbance). The amount the

buyer can learn is thermodynamically limited to $nh(Q_{\text{obs}})$ bits—this is the “budget ceiling” on the buyer’s quantum interception bid.

B_2^Q (Bidding Round 2: Error Estimation Bid—Statistical Detection).

Purpose: Bound the probability that Eve causes a high error rate on the sifted key but the error estimation sample underestimates it. If the sample of k bits gives $Q_{\text{obs}} \leq Q_{\text{tol}}$ but the true error rate $Q_{\text{true}} > Q_{\text{tol}} + \gamma$ for some margin γ , then by Hoeffding’s inequality: $\Pr[Q_{\text{obs}} \leq Q_{\text{tol}} \mid Q_{\text{true}} > Q_{\text{tol}} + \gamma] \leq e^{-2k\gamma^2}$.

Quantum market feature: The seller uses *classical statistical inference* on the quantum measurement outcomes to detect the buyer’s quantum disturbance. This is a quantum-to-classical conversion: the buyer’s quantum attack is detected by a classical test.

Quantum price adjustment: Let F_{est} be the event “error estimation fails to detect Eve’s disturbance.” By Hoeffding’s inequality with k sample bits and margin γ :

$$\Pr[F_{\text{est}}] \leq e^{-2k\gamma^2}. \quad (208)$$

For $k = \Theta(\lambda/\gamma^2)$, this is $\leq 2^{-\Omega(\lambda)} = \text{negl}(\lambda)$.

Market interpretation: The error estimation bid bounds the probability that the buyer evades detection. Even though the buyer’s attack is quantum, the detection mechanism is a classical sampling test that succeeds with overwhelming probability.

B_3^Q (Bidding Round 3: Privacy Amplification Bid—Leftover Hash Lemma).

Purpose: Given that Eve’s information about the corrected key is bounded (from B_1^Q and B_2^Q), privacy amplification via a universal₂ hash function extracts a final key that is exponentially close to uniform from Eve’s perspective.

Quantum market feature: The quantum leftover hash lemma (QLHL) operates on Eve’s quantum side information ρ_E , not just classical information. The output key K satisfies $\frac{1}{2}\|K \otimes \rho_E - U_\ell \otimes \rho_E\|_{\text{tr}} \leq 2^{-\lambda/2}$ when $\ell \leq H_{\min}(X|E) - 2\log(1/\epsilon)$, where $H_{\min}(X|E)$ is the conditional quantum min-entropy.

By the quantum leftover hash lemma, after privacy amplification with output length $\ell = n(1 - h(Q_{\text{obs}})) - \lambda$:

$$\frac{1}{2}\|K \otimes \rho_E - U_\ell \otimes \rho_E\|_{\text{tr}} \leq 2^{-\lambda/2}. \quad (209)$$

Market interpretation: Privacy amplification is the seller’s final move—a “market restructuring” that converts the raw key (partially known to Eve) into a final key (virtually unknown). The hash function compresses the key to the length justified by the quantum min-entropy, squeezing out all information the buyer might have gained. The trace distance bound $2^{-\lambda/2}$ is the residual quantum ask price.

B_4^Q (Bidding Round 4: Error Correction Leakage Bid).

Error correction leaks at most $nh(Q_{\text{obs}})$ bits of classical information to Eve via the public syndrome. This is already accounted for in the privacy amplification output length: $\ell = n(1 - h(Q_{\text{obs}})) - \lambda$ subtracts exactly $nh(Q_{\text{obs}})$ bits. Price adjustment: $\Delta\text{Price}_4^Q = 0$ (already absorbed).

Total quantum ask price:

$$\text{Ask}^Q(g_{\text{secrecy}}) \leq \underbrace{2^{-\lambda/2}}_{\text{QLHL}} + \underbrace{e^{-2k\gamma^2}}_{\text{estimation}} + \underbrace{2^{-\lambda}}_{\text{EC failure}} \leq 2^{-\lambda/2} + \text{negl}(\lambda). \quad (210)$$

The BB84 quantum market is in equilibrium. □

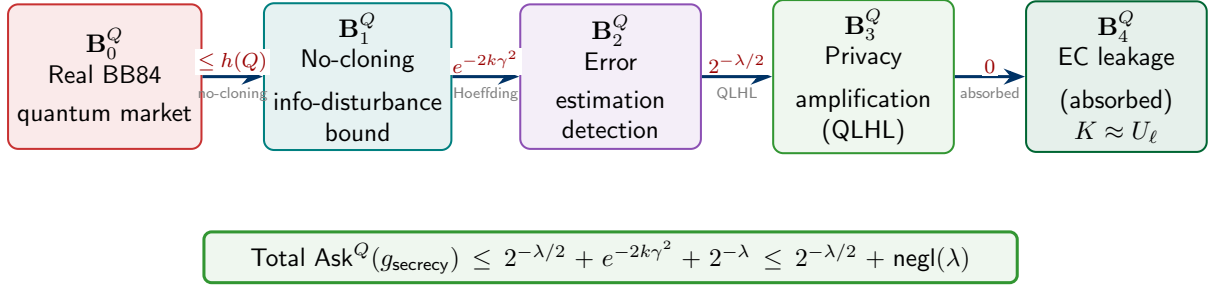


Figure 60: BB84 quantum bidding-round chain: real quantum market ($\mathbf{B}_0^Q$) to ideal ($\mathbf{B}_4^Q$). Two distinctively *quantum* hops: $\mathbf{B}_0^Q \rightarrow \mathbf{B}_1^Q$ (no-cloning information-disturbance trade-off—the seller’s non-orthogonal states structurally limit the buyer’s information gain) and $\mathbf{B}_2^Q \rightarrow \mathbf{B}_3^Q$ (quantum leftover hash lemma—privacy amplification against quantum side information). These are the quantum analogues of the O2H and measure-and-reprogram hops in the QROM case study (Section 14). The security is information-theoretic: no computational assumption is required, only quantum mechanics.

*** Simple Terms: BB84 Security Proof Explained—Why Quantum Mechanics Gives Perfect Security**

Classical comparison: In Diffie–Hellman key exchange, security relies on the assumption that discrete logarithms are hard to compute. A powerful enough computer (classical or quantum) could break it. The ask price is $\text{Adv}^{\text{ECDLP}}$ —computational.

BB84 (quantum): Security relies on the laws of quantum mechanics:

1. *No-cloning:* Eve cannot copy Alice’s qubits. She must choose: learn the bit (disturbing the qubit) or forward it undisturbed (learning nothing). This is not a computational assumption—it is a physical law.
2. *Information-disturbance:* Any information Eve extracts is paid for by disturbance that Alice and Bob can detect statistically.
3. *Privacy amplification:* Even if Eve learns some partial information, hashing compresses the key to remove all leaked information.

Market interpretation: The seller’s quantum states are *uncopyable goods*—the buyer cannot “shoplift” (learn the key) without leaving evidence (error rate increase). The error estimation phase is a *market audit*: the seller checks for evidence of theft. Privacy amplification is *market insurance*: even if the audit slightly underestimates the theft, the insurance (hashing) covers the gap.

15.5 Extended Security Goods and CNF Verification

Corollary 15.3 (BB84 Correctness). *Under the error correction protocol with failure probability $\delta_{\text{EC}} \leq 2^{-\lambda}$:*

$$\text{Ask}^Q(g_{\text{correct}}) \leq 2^{-\lambda} = \text{negl}(\lambda). \quad (211)$$

Corollary 15.4 (BB84 Eavesdropping Detection). *If Eve extracts $I_E \geq \gamma n$ bits of information about the sifted key (for any constant $\gamma > 0$), the error estimation detects this:*

$$\text{Ask}^Q(g_{\text{detect}}) \leq e^{-2k\gamma^2} + 2^{-\lambda} = \text{negl}(\lambda). \quad (212)$$

✓ CNF Verification: BB84 Session-CNF Verification

$\varphi_{\text{BB84}} = \varphi^{\text{basis}} \wedge \varphi^{\text{error}} \wedge \varphi^{\text{auth}} \wedge \varphi^{\text{PA}} \wedge \varphi^{\text{ping}}$, where $\varphi^{\text{basis}} = (x_{\theta^A=\theta^B} \text{ for sifted bits})$, $\varphi^{\text{error}} = (x_{Q_{\text{obs}} \leq Q_{\text{tol}}})$, $\varphi^{\text{auth}} = (x_{\text{classical channel authenticated}})$, $\varphi^{\text{PA}} = (x_{\ell \leq n(1-h(Q_{\text{obs}}))-\lambda})$.

Manual CNF worksheet (BB84 session):

Clause	Check	How	Pass?
φ^{basis} : basis match	Sifted bits use matching bases?	Compare public θ^A, θ^B records.	T/F
φ^{error} : error rate	$Q_{\text{obs}} \leq Q_{\text{tol}}?$	Compute error rate on sample of k bits.	T/F
φ^{auth} : classical auth	Classical channel authenticated?	Verify MAC on all classical messages.	T/F
φ^{PA} : PA length	$\ell \leq n(1-h(Q_{\text{obs}}))-\lambda?$	Check output length of hash.	T/F
φ^{ping} : session fresh	No sifted-key bits reused from prior session?	Compare with previous session logs.	T/F
Result:	All T $\Rightarrow$ BB84 session secure. Any F $\Rightarrow$ Reject (abort).		

► Ping Bid: BB84 Unbounded Ping Bid

The buyer's *quantum replay bid*: attempt to reuse information from session Session_i to attack session Session_{i+1} . Since each session uses fresh random bits and bases (x_i, θ_i^A are independently sampled), Eve's quantum side information from Session_i is *product* with the quantum states in Session_{i+1} . By the quantum leftover hash lemma applied independently per session: $\Delta \text{Price}_{\text{ping}}^Q \leq 2^{-\lambda/2} = \text{negl}$. By Theorem 5.5 (extended to quantum markets via Theorem 21.2), BB84 is secure for unbounded sessions against quantum adversaries.

Table 18: BB84 quantum market goods summary.

Good	Quantum Ask Price Bound	Dominant Term	Status
g_{secrecy}	$2^{-\lambda/2} + e^{-2k\gamma^2}$	QLHL	Equilibrium
g_{correct}	$2^{-\lambda}$	EC failure	Equilibrium
g_{detect}	$e^{-2k\gamma^2} + 2^{-\lambda}$	Hoeffding	Equilibrium
$g_{\text{unbounded (ping)}}$	$2^{-\lambda/2}$	QLHL (per session)	Equilibrium

🔑 Key Insight:

Classical QROM vs. Quantum Market Dynamics: Complete Comparison

Property	QROM (Section 14)	Market (Section 14)	Quantum Market (BB84)
Seller	Classical (prepares texts)	Quantum (prepares ciphertexts)	Quantum (prepares qubits)
Buyer	Quantum (queries)	(superposition)	Quantum (arbitrary CPTP maps)
Security basis	Computational (MLWE)		Information-theoretic (physics)
Price mechanism	$\sqrt{\text{Adv}^{\text{IND-CPA}}}$ (O2H)		$2^{-\lambda/2}$ (QLHL)
Key quantum tool	O2H + measure-reprogram		No-cloning + info-disturbance
Bidding rounds	6		5 ($\mathbf{B}_0^Q - \mathbf{B}_4^Q$, last absorbed)
Breaks under	Quantum computer breaking MLWE		Violation of quantum mechanics
MTSF framework	Classical market, quantum buyer		Full quantum market

Core insight: The QROM market has a classical seller defending against a quantum buyer (post-quantum cryptography). The BB84 quantum market has a quantum seller exploiting

quantum mechanics as a *structural defence*—the goods are physically uncopyable. This is a fundamentally stronger market position.

16 Case Study VIII: Multi-Protocol Composition—TLS 1.3 + Signal Network

Analogy:

Multi-Protocol Composition as a Multi-Lock Security System Imagine your house has two independent security systems:

- **System A (TLS 1.3):** A deadbolt lock on the front door. Proven burglar-proof on its own.
- **System B (Signal):** An alarm system. Proven burglar-proof on its own.

The composition question: When you install BOTH systems together, does the combination remain secure? Or could they interfere—perhaps the alarm system’s vibrations loosen the deadbolt?

In the real world: Your phone runs TLS 1.3 (for HTTPS browsing) and Signal (for messaging) *simultaneously*, sharing the same PKI infrastructure (certificate authorities, device keys). An attacker who observes your TLS traffic might use information from it to attack your Signal sessions, or vice versa. The composition theorem must ensure this cross-protocol attack is impossible.

MTSF’s market merger theorem: If the TLS market is in equilibrium (TLS is secure alone) and the Signal market is in equilibrium (Signal is secure alone), then the *merged* market (TLS + Signal running simultaneously, sharing PKI) is also in equilibrium—provided the shared infrastructure (PKI) is itself secure. This is like proving that the deadbolt and alarm system, sharing the same electrical wiring (PKI), remain secure together because the wiring itself is trustworthy.

The key insight: Security does not degrade under composition *if* the shared infrastructure is sound and each protocol binds its sessions to unique identifiers. MTSF’s CNF verification ensures this by requiring that TLS sessions and Signal sessions use distinct session identifiers, preventing cross-protocol replays.

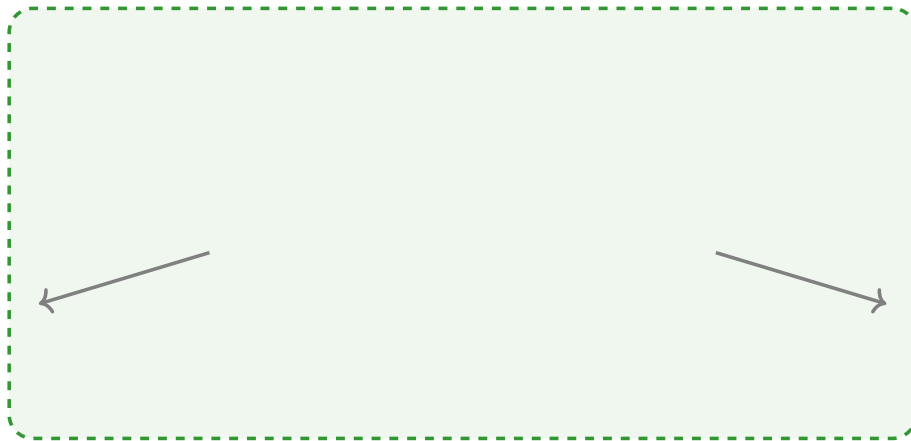
This case study instantiates the multi-protocol composition network formalisation of Section 21.2.2 on a concrete and practically relevant scenario: a device simultaneously running TLS 1.3 (Section 13.8.1) for web browsing and the Signal Protocol (Section 13.7) for encrypted messaging, sharing a common PKI infrastructure. This is the first MTSF analysis of *concurrent protocol execution*, demonstrating that the market merger theorem (Theorem 21.7) preserves equilibrium when two individually secure protocols compose.

* Simple Terms: Multi-Protocol Composition—Why It Matters

The real-world scenario: Right now, your phone is probably running multiple security protocols simultaneously:

- **TLS 1.3** connects your browser to your bank, streaming service, and email provider.
- **Signal Protocol** encrypts your WhatsApp or Signal messages.
- Both use the **same PKI** (certificate authorities, X.509 certificates) to verify identities.
- Both use the **same random number generator** on your device.

Merged Market (Composition)



Market Merger Theorem: Equilibrium + Equilibrium + Sound Infrastructure = **Equilibrium**
TLS secure + Signal secure + PKI sound = TLS+Signal secure together

Figure 61: Multi-protocol composition as market merger: TLS 1.3 and Signal, each independently secure (in equilibrium), share a PKI infrastructure. The market merger theorem guarantees the combined system remains in equilibrium, provided the shared PKI is sound.

- Both share the **same CPU and memory**—an attacker exploiting one protocol might learn something useful for attacking the other.

The security question: We proved TLS 1.3 secure in Section 13.8.3. We proved the Signal Protocol secure in Section 13.7. But are they secure *when running at the same time on the same device sharing the same infrastructure*?

The answer (via MTSF): Yes—if each protocol market is in equilibrium individually and the shared infrastructure is sound, the composed market network remains in equilibrium. The attacker’s resources must be split across both markets, and neither market collapses.

16.1 Network Description

* Simple Terms: TLS 1.3—The Protocol That Secures the Internet

Every time you see the padlock icon in your browser, TLS 1.3 is at work. It is the most widely deployed security protocol in history, protecting:

- Every HTTPS website you visit (banking, shopping, email).
- Every API call between cloud services.
- Every mobile app that communicates with a server.

How TLS 1.3 works (simplified):

1. **ClientHello:** Your browser sends its capabilities and a key share (Diffie–Hellman or ML-KEM public value).
2. **ServerHello:** The server responds with its key share, certificate, and signature.
3. **Key derivation:** Both sides derive shared secrets using HKDF (HMAC-based Key Derivation Function).

4. **Application data:** All subsequent communication is encrypted with AEAD (AES-GCM or ChaCha20-Poly1305).

1-RTT vs. 0-RTT: TLS 1.3’s main handshake takes one round trip (1-RTT)—the client sends a message, gets a response, and both sides are ready. For repeat connections, TLS 1.3 offers 0-RTT resumption (sending application data in the first message), but this sacrifices replay protection. In MTSF, 1-RTT is in equilibrium; 0-RTT has a “replay bid” collapse for the replay-protection good.

The downgrade attack: An attacker might try to force TLS 1.3 to fall back to an older, weaker version (TLS 1.2 or 1.1). TLS 1.3 prevents this with a “downgrade sentinel” embedded in the server’s random value. In MTSF, the downgrade bid collapses if the sentinel is removed, but succeeds (equilibrium) with the sentinel in place.

Component protocols.

1. **Protocol 1: TLS 1.3 (1-RTT handshake)**—as analysed in Section 13.8.3. Provides session-key secrecy ($g_{\text{sk}}^{(1)}$), forward secrecy ($g_{\text{FS}}^{(1)}$), server authentication ($g_{\text{auth}}^{(1)}$), and mutual authentication ($g_{\text{mutual}}^{(1)}$) when client certificates are used.
2. **Protocol 2: Signal (X3DH + Double Ratchet)**—as analysed in Section 13.7. Provides session-key secrecy ($g_{\text{sk}}^{(2)}$), forward secrecy ($g_{\text{FS}}^{(2)}$), post-compromise security ($g_{\text{PCS}}^{(2)}$), asynchronous establishment ($g_{\text{async}}^{(2)}$), and deniability ($g_{\text{deny}}^{(2)}$).

Shared infrastructure $\mathcal{I}$.

- **PKI** ($\mathcal{I}_{\text{PKI}}$): X.509 certificate authorities issuing certificates for TLS servers and Signal identity keys. Shared trust anchors.
- **Random oracle** ($\mathcal{I}_{\text{RO}}$): Hash functions (SHA-256, SHA-384) used by both protocols for key derivation (HKDF in TLS, HKDF in Signal’s X3DH).
- **Device RNG** ($\mathcal{I}_{\text{RNG}}$): The operating system’s cryptographic random number generator, used by both protocols for nonce generation and ephemeral key sampling.

Participants.

- **Device D:** runs both protocols concurrently. Acts as the TLS client and the Signal user.
- **TLS server S_1 :** the web server for Protocol 1.
- **Signal server S_2 :** the Signal key server storing prekey bundles for Protocol 2.
- **Network adversary $\mathcal{A}$:** a single PPT adversary controlling the network between D and both servers. $\mathcal{A}$ can intercept, modify, and inject messages in both protocols simultaneously. $\mathcal{A}$ has a single computational budget T (total running time).

16.2 MTSF Market Network Model

We instantiate Theorem 21.5 with $N = 2$ component markets.

Definition 16.1 (TLS+Signal Market Network). *The protocol market network*

$$\mathcal{N}_{\text{TS}} = (\{\mathcal{M}_{\text{TLS}}, \mathcal{M}_{\text{Signal}}\}, \mathcal{I}, \mathcal{Z})$$

is defined as follows:

- **TLS market:**

$$\mathcal{M}_{\text{TLS}} = (\text{Seller}_{\text{TLS}}, \text{Buyer}_{\text{TLS}}, \{g_{\text{sk}}^{(1)}, g_{\text{FS}}^{(1)}, g_{\text{auth}}^{(1)}, g_{\text{mutual}}^{(1)}\}, \varphi_{\text{TLS}})$$

is the TLS 1.3 market from Section 13.8.2.

- **Signal market:**

$$\mathcal{M}_{\text{Signal}} = (\text{Seller}_{\text{Signal}}, \text{Buyer}_{\text{Signal}}, \{g_{\text{sk}}^{(2)}, g_{\text{FS}}^{(2)}, g_{\text{PCS}}^{(2)}, g_{\text{async}}^{(2)}, g_{\text{deny}}^{(2)}\}, \varphi_{\text{Signal}})$$

is the Signal market from Section 13.7.

- **Shared infrastructure:**

$$\mathcal{I} = \mathcal{I}_{\text{PKI}} \cup \mathcal{I}_{\text{RO}} \cup \mathcal{I}_{\text{RNG}}.$$

- **Environment:** $\mathcal{Z}$ is the network environment that interacts with both protocols concurrently.

Infrastructure security goods. The shared infrastructure contributes three goods:

- g_{PKI} : PKI integrity—no adversary can forge a valid certificate. $\text{Ask}_{\mathcal{I}}(g_{\text{PKI}}) \leq \text{Adv}_{\text{CA}}^{\text{EUF-CMA}} \leq \text{negl}(\lambda)$ (under the EUF-CMA security of the CA’s signature scheme).
- g_{RO} : Random oracle consistency—both protocols query the same hash function, modelled as a shared random oracle. $\text{Ask}_{\mathcal{I}}(g_{\text{RO}}) = 0$ (the RO is ideal by assumption; in the standard model, replace by $\text{Adv}_{\text{HKDF}}^{\text{PRF}}$).
- g_{RNG} : RNG quality—the device RNG produces outputs indistinguishable from uniform. $\text{Ask}_{\mathcal{I}}(g_{\text{RNG}}) \leq \text{Adv}_{\text{RNG}}^{\text{PRG}} \leq \text{negl}(\lambda)$.

16.3 Market Merger and Composition Proof

We apply the market merger theorem (Theorem 21.7) to the TLS+Signal network.

Theorem 16.2 (TLS+Signal Network Equilibrium). *Let $\mathcal{M}_{\text{TLS}}$ be in equilibrium with ask prices as in Section 13.8.3, and let $\mathcal{M}_{\text{Signal}}$ be in equilibrium with ask prices as in Section 13.7. If the shared infrastructure is sound ($\text{Ask}_{\mathcal{I}}(g) \leq \text{negl}(\lambda)$ for all infrastructure goods g), then the merged market:*

$$\mathcal{M}_{\text{TLS}} \otimes_{\mathcal{I}} \mathcal{M}_{\text{Signal}} \tag{213}$$

is in equilibrium. Specifically, for every security good g in either protocol:

$$\text{Ask}_{\mathcal{N}}(g) \leq \text{Ask}_{\text{own}}(g) + \text{Ask}_{\text{other}} + \text{Ask}_{\mathcal{I}} \leq \text{negl}(\lambda), \tag{214}$$

where $\text{Ask}_{\text{own}}(g)$ is the ask price of g in its home market, $\text{Ask}_{\text{other}}$ is the maximum ask price in the other market, and $\text{Ask}_{\mathcal{I}} = \max_g \text{Ask}_{\mathcal{I}}(g)$ is the infrastructure overhead.

Proof. We apply Theorem 21.7. The three conditions are verified:

Condition 1: TLS market equilibrium. By Section 13.8.3, all TLS 1.3 goods are in equilibrium. The ten-bidding-round chain gives:

$$\text{Ask}_{\text{TLS}}(g_{\text{sk}}^{(1)}) \leq \text{Adv}^{\text{ECDLP}} + \text{Adv}_{\text{HKDF}}^{\text{PRF}} + q_N^2/2^{\lambda+1} \leq \text{negl}(\lambda), \tag{215}$$

$$\text{Ask}_{\text{TLS}}(g_{\text{auth}}^{(1)}) \leq \text{Adv}^{\text{EUF-CMA}} + q_N^2/2^{\lambda+1} \leq \text{negl}(\lambda). \tag{216}$$

Condition 2: Signal market equilibrium. By Section 13.7, all Signal goods are in equilibrium. The X3DH proof gives:

$$\text{Ask}_{\text{Signal}}(g_{\text{sk}}^{(2)}) \leq \text{Adv}^{\text{gap-CDH}} + q_N^2/2^{\lambda+1} \leq \text{negl}(\lambda), \tag{217}$$

$$\text{Ask}_{\text{Signal}}(g_{\text{FS}}^{(2)}) \leq \text{Adv}^{\text{gap-CDH}} \leq \text{negl}(\lambda). \quad (218)$$

Condition 3: Infrastructure soundness.

$$\text{Ask}_{\mathcal{I}}(g_{\text{PKI}}) \leq \text{Adv}_{\text{CA}}^{\text{EUF-CMA}} \leq \text{negl}(\lambda), \quad (219)$$

$$\text{Ask}_{\mathcal{I}}(g_{\text{RO}}) = 0 \quad (\text{ideal RO}), \quad (220)$$

$$\text{Ask}_{\mathcal{I}}(g_{\text{RNG}}) \leq \text{Adv}_{\text{RNG}}^{\text{PRG}} \leq \text{negl}(\lambda). \quad (221)$$

By Theorem 21.7, the merged market is in equilibrium. The quantitative bound from Remark after Theorem 21.7 gives:

$$\text{Ask}_{\mathcal{N}}(g_{\text{sk}}^{(1)}) \leq \underbrace{\text{Ask}_{\text{TLS}}(g_{\text{sk}}^{(1)})}_{\text{own market}} + \underbrace{\text{Ask}_{\text{Signal,max}}}_{\text{simulation overhead}} + \underbrace{\text{Ask}_{\mathcal{I},\text{max}}}_{\text{infrastructure}} \leq \text{negl}(\lambda), \quad (222)$$

where $\text{Ask}_{\text{Signal,max}} = \max_j \text{Ask}_{\text{Signal}}(g_j^{(2)})$ is the simulation overhead from simulating the Signal market when reducing to the standalone TLS game. Symmetrically for Signal goods.

The proof proceeds by constructing a standalone buyer for each market that internally simulates the other market using the UC simulator (market arbitrageur from Section 2.4). The environment $\mathcal{Z}$ cannot distinguish the real composed execution from the simulation because both component markets are UC-realizable (they implement ideal functionalities for key exchange and secure messaging respectively). $\square$

16.4 Resource Competition Analysis

Analogy:

Resource Competition as a Shared Budget When TLS and Signal run simultaneously on the same device, they share computational resources. Think of this as two departments in a company sharing a single budget:

- **TLS department** needs T_{TLS} operations per session (key exchange, AEAD encryption, certificate verification).
- **Signal department** needs T_{Signal} operations per session (X3DH, Double Ratchet, message encryption).
- **The attacker's budget** is $T_{\text{total}} \leq \text{poly}(\lambda)$.

The question: Can the attacker allocate their budget across attacks on both protocols simultaneously to achieve a higher total advantage than attacking either alone?

MTSF's answer: No. If each protocol is independently secure, the total advantage is bounded by the sum of individual advantages. This is because the security goods are offered in *independent* markets: breaking TLS's key secrecy does not help break Signal's forward secrecy (different keys, different mechanisms). The extended difference lemma captures this: $\Pr[F_{\text{TLS}} \cup F_{\text{Signal}}] \leq \Pr[F_{\text{TLS}}] + \Pr[F_{\text{Signal}}]$ when the failure events use independent cryptographic material.

The MTSF language makes explicit a phenomenon that is implicit in the UC framework: the adversary faces a *resource allocation problem* when attacking multiple protocols simultaneously.

Proposition 16.3 (Adversarial Resource Allocation). *Let $\mathcal{A}$ be a PPT adversary with total running time T attacking the network $\mathcal{N}_{\text{TS}}$. If $\mathcal{A}$ allocates time t_1 to attacking $\mathcal{M}_{\text{TLS}}$ and $t_2 = T - t_1$ to attacking $\mathcal{M}_{\text{Signal}}$, then:*

$$\text{Ask}_{\mathcal{N}}(g_{\text{sk}}^{(1)}) + \text{Ask}_{\mathcal{N}}(g_{\text{sk}}^{(2)}) \leq f_{\text{TLS}}(t_1) + f_{\text{Signal}}(T - t_1) + 2\text{Ask}_{\mathcal{I}}, \quad (223)$$

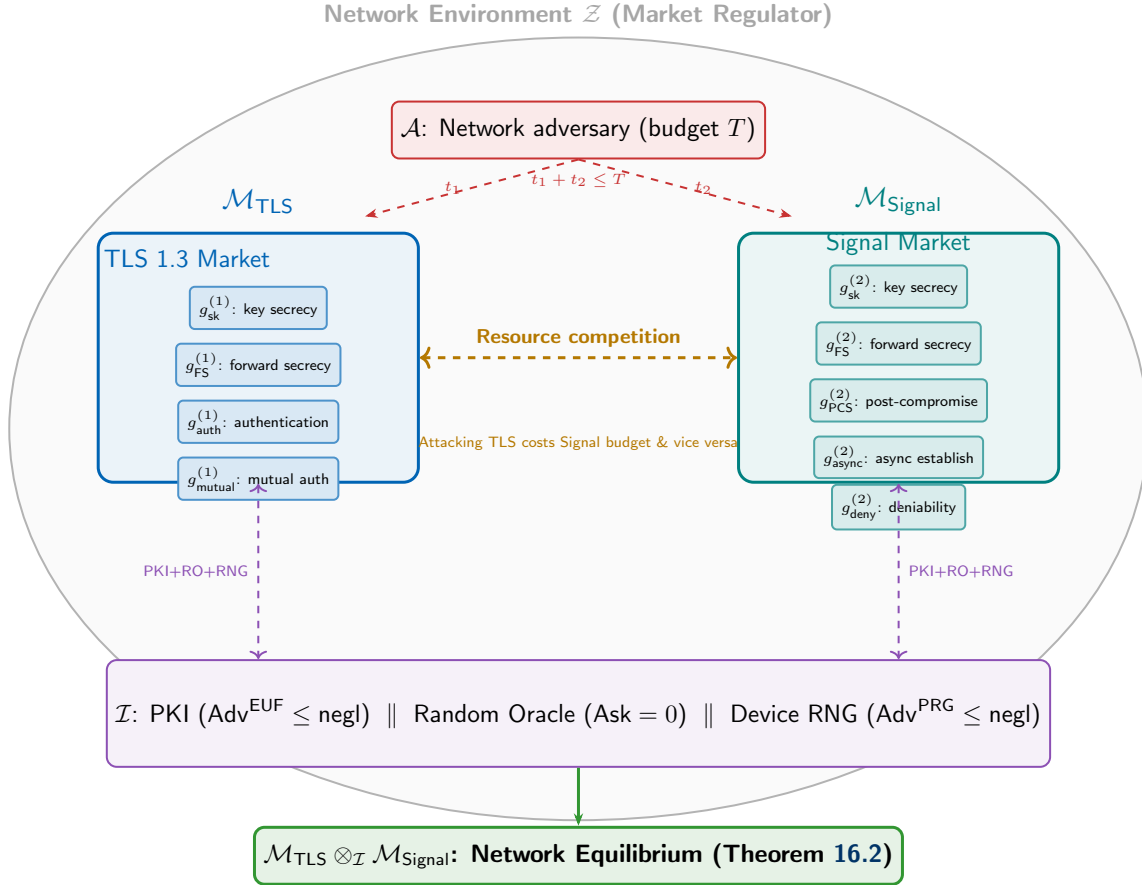


Figure 62: TLS 1.3 + Signal protocol market network. The TLS market (left, blue) and Signal market (right, teal) share infrastructure $\mathcal{I}$ (PKI, random oracle, device RNG). A single network adversary $\mathcal{A}$ with budget T must split its resources ($t_1 + t_2 \leq T$) between the two markets. The orange dashed link indicates resource competition: effort spent attacking TLS reduces resources available for attacking Signal. By the market merger theorem (Theorem 21.7), since both component markets and the infrastructure are in equilibrium individually, the merged market $\mathcal{M}_{\text{TLS}} \otimes_{\mathcal{I}} \mathcal{M}_{\text{Signal}}$ preserves equilibrium for all nine security goods.

where $f_{\text{TLS}}(t) = \text{Adv}^{\text{ECDLP}}(t) + \text{Adv}^{\text{PRF}}_{\text{HKDF}}(t) + q_N^2/2^{\lambda+1}$ and $f_{\text{Signal}}(t) = \text{Adv}^{\text{gap-CDH}}(t) + q_N^2/2^{\lambda+1}$ are the advantage functions for each protocol as a function of computational effort.

Proof. The adversary’s total computation is partitioned: t_1 steps interact with TLS, t_2 steps interact with Signal. Any computation that interacts with TLS contributes to $f_{\text{TLS}}(t_1)$ via the standalone TLS reduction. Any computation that interacts with Signal contributes to $f_{\text{Signal}}(t_2)$ via the standalone Signal reduction. The infrastructure overhead $\text{Ask}_{\mathcal{I}}$ applies to each market independently (hence the factor of 2). The resource constraint $t_1 + t_2 \leq T$ means the adversary cannot achieve the best attack on both protocols simultaneously. $\square$

Remark 16.4 (Weakest Link Principle). *The rational adversary maximises*

$$\max(\text{Ask}_{\mathcal{N}}(g_{\text{sk}}^{(1)}), \text{Ask}_{\mathcal{N}}(g_{\text{sk}}^{(2)}))$$

—it concentrates resources on whichever protocol offers the highest “return on attack.” If TLS uses ECDLP on a 256-bit curve and Signal uses gap-CDH on the same curve, both are equally hard and the adversary gains nothing from splitting effort. In practice, if one protocol used a weaker primitive (e.g., a shorter key), the adversary would rationally concentrate there—formalising the “weakest link” principle in market terms.

16.5 Network CNF Verification

Analogy:

Network CNF as a Multi-Building Inspection If the single-protocol CNF is like inspecting one building, the network CNF is like inspecting an entire city block:

- **Building-level clauses:** Each protocol (TLS, Signal) has its own CNF checklist.
- **Infrastructure clauses:** Shared PKI certificates are verified (the electrical grid inspection).
- **Cross-building clauses:** Session identifiers are unique across protocols (no two buildings share the same address). This prevents a “cross-protocol replay bid”—using a TLS session transcript to attack Signal, or vice versa.

The network CNF is the conjunction of all building CNFs plus infrastructure clauses:

$$\varphi_{\text{network}} = \varphi_{\text{TLS}} \wedge \varphi_{\text{Signal}} \wedge \varphi_{\text{PKI}} \wedge \varphi_{\text{cross-protocol}}$$

The key insight: Cross-protocol clauses prevent attacks that exploit the *interaction* between protocols. Without these clauses, an attacker might use a TLS certificate to impersonate a Signal contact, or replay a Signal prekey bundle in a TLS handshake. The network CNF blocks all such cross-protocol attacks.

We instantiate Theorem 21.10 for the TLS+Signal network.

✓ CNF Verification: TLS+Signal Network Session-CNF Verification

$\varphi_{\mathcal{N}} = \varphi_{\text{TLS}} \wedge \varphi_{\text{Signal}} \wedge \varphi_{\mathcal{I}}$, where φ_{TLS} is from Section 13.8.6, φ_{Signal} is from Section 13.7, and:

$$\varphi_{\mathcal{I}} = \underbrace{\varphi_{\mathcal{I}}^{\text{PKI}}}_{\text{cert consistency}} \wedge \underbrace{\varphi_{\mathcal{I}}^{\text{RO}}}_{\text{hash consistency}} \wedge \underbrace{\varphi_{\mathcal{I}}^{\text{RNG}}}_{\text{RNG quality}} \wedge \underbrace{\varphi_{\mathcal{I}}^{\text{SID}}}_{\text{global SID uniqueness}} .$$

Manual CNF worksheet (network session—infrastructure clauses):

Clause	Check	How	Pass?
φ_T^{PKI} : cert consistency	Same CA root for TLS and Signal?	Compare trust chains.	T/F
φ_T^{RO} : hash consistency	Both use same SHA-256/384?	Check library versions.	T/F
φ_T^{RNG} : RNG quality	Device RNG passes NIST SP 800-90B?	Run health tests.	T/F
φ_T^{SID} : global SID	No TLS SID = Signal SID?	Compare SID logs across both protocols.	T/F
Result:	All T (+ per-protocol CNFs pass) $\Rightarrow$ Network session secure.		

► Ping Bid: TLS+Signal Network Unbounded Ping Bid

The buyer's *cross-protocol replay bid*: attempt to use a TLS session transcript to attack a Signal session (or vice versa). This fails because:

1. TLS and Signal use distinct protocol identifiers and session IDs (φ_T^{SID} ensures global uniqueness).
2. TLS key material is derived via HKDF with TLS-specific labels; Signal key material via HKDF with X3DH-specific labels. The label-binding ensures cross-protocol key independence.
3. The network ping bid decomposes: $\Delta\text{Price}_{\text{ping}}^N \leq \Delta\text{Price}_{\text{ping}}^{\text{TLS}} + \Delta\text{Price}_{\text{ping}}^{\text{Signal}} + \Delta\text{Price}_{\text{ping}}^{\mathcal{I}}$, where each component is negligible by the per-protocol ping analysis and infrastructure soundness.

By Theorem 5.5 applied to the network (via Theorem 21.9 with $N = 2$), the TLS+Signal network is secure for unbounded sessions across both protocols simultaneously.

Table 19: TLS+Signal market network goods summary. All goods from both protocols plus infrastructure goods, with network ask prices from Theorem 16.2.

Market	Good	Standalone Ask	Network Ask (Eq. 214)	Status
TLS 1.3	$g_{\text{sk}}^{(1)}$: key secrecy	$\text{Adv}^{\text{ECDLP}} + \text{Adv}^{\text{PRF}} + q_N^2/2^{\lambda+1}$	$+\text{Ask}_{\text{Signal,max}} + \text{Ask}_{\mathcal{I}}$	Equilibrium
	$g_{\text{FS}}^{(1)}$: forward secrecy	$\text{Adv}^{\text{ECDLP}}$	$+\text{Ask}_{\text{Signal,max}} + \text{Ask}_{\mathcal{I}}$	Equilibrium
	$g_{\text{auth}}^{(1)}$: authentication	$\text{Adv}^{\text{EUF}} + q_N^2/2^{\lambda+1}$	$+\text{Ask}_{\text{Signal,max}} + \text{Ask}_{\mathcal{I}}$	Equilibrium
	$g_{\text{mutual}}^{(1)}$: mutual auth	$\text{Adv}^{\text{EUF}} + q_N^2/2^{\lambda+1}$	$+\text{Ask}_{\text{Signal,max}} + \text{Ask}_{\mathcal{I}}$	Equilibrium
Signal	$g_{\text{sk}}^{(2)}$: key secrecy	$\text{Adv}^{\text{gap-CDH}} + q_N^2/2^{\lambda+1}$	$+\text{Ask}_{\text{TLS,max}} + \text{Ask}_{\mathcal{I}}$	Equilibrium
	$g_{\text{FS}}^{(2)}$: forward secrecy	$\text{Adv}^{\text{gap-CDH}}$	$+\text{Ask}_{\text{TLS,max}} + \text{Ask}_{\mathcal{I}}$	Equilibrium
	$g_{\text{PCS}}^{(2)}$: post-compromise	$\text{Adv}^{\text{gap-CDH}}$	$+\text{Ask}_{\text{TLS,max}} + \text{Ask}_{\mathcal{I}}$	Equilibrium
	$g_{\text{async}}^{(2)}$: async establish	$\text{Adv}^{\text{gap-CDH}}$	$+\text{Ask}_{\text{TLS,max}} + \text{Ask}_{\mathcal{I}}$	Equilibrium
	$g_{\text{deny}}^{(2)}$: deniability	$\leq \text{negl}$	$+\text{Ask}_{\text{TLS,max}} + \text{Ask}_{\mathcal{I}}$	Equilibrium
Infrastructure	g_{PKI} : cert integrity	$\text{Adv}_{\text{CA}}^{\text{EUF}}$	(shared)	Equilibrium
	g_{RO} : hash consistency	0 (ideal)	(shared)	Equilibrium
	g_{RNG} : RNG quality	$\text{Adv}_{\text{RNG}}^{\text{PRG}}$	(shared)	Equilibrium

Remark 16.5 (Generalisation to N Protocols). *Theorem 21.9 shows that a device running $N = \text{poly}(\lambda)$ protocols simultaneously (e.g., TLS + Signal + SSH + WireGuard + ...) remains in network equilibrium as long as each protocol market and the shared infrastructure are individually in equilibrium. The network ask price grows at most linearly in N , but since each component ask is negligible and N is polynomial, the sum remains negligible. The practical takeaway: running many secure protocols on the same device does not degrade security, provided the shared infrastructure (PKI, RNG) is sound.*

Primitives	ECDSA Equilibrium Ask $\leq 2^{-128}$	ML-KEM Equilibrium Ask $\leq 2^{-128}$	ML-DSA Equilibrium Ask $\leq 2^{-128}$	HMAC Equilibrium Ask $\leq 2^{-128}$	Textbook RSA Collapsed Ask = 1
Sym. Prims	AES Equilibrium Ask $\leq 2^{-128}$	SHA-3 Equilibrium Ask $\leq 2^{-128}$	BLAKE3 Equilibrium LE-immune	ChaCha20 Equilibrium Ask $\leq 2^{-128}$	Grain-128a Equilibrium Ask $\leq 2^{-128}$
Protocols	TLS 1.3 Equilibrium 1-RTT	Signal Equilibrium FS+PCS	NS Collapsed MITM Ask = 1	Telegram Partial Salt extract	BB84 Equilibrium Info-theoretic

Green = Market Equilibrium (Secure)
Red = Market Collapsed (Insecure)

Figure 63: Complete scoreboard of all MTSF case studies. Green boxes indicate market equilibrium (scheme/protocol is secure); red boxes indicate market collapse (insecure). The same MTSF framework produces both verdicts using bidding-round machinery, CNF verification, and session ping-pong.

17 Writing a Security Research Paper with MTSF

This section provides a comprehensive, self-contained guide for writing a classical security research paper using MTSF. It covers the full pipeline from specifying a cryptographic scheme to proving or disproving its security, and is written so that a graduate student encountering security proofs for the first time can follow each step.

* Simple Terms: Who This Section Is For

This section is for anyone who wants to write a security research paper and use MTSF as the proof framework. It applies equally to:

- Designing a **new block cipher** and proving PRP security.
- Proposing a **new hash function** and proving collision resistance.
- Analysing a **new stream cipher** and proving PRG security.
- Designing a **new protocol** and proving authentication and key secrecy.
- Discovering an **attack** on an existing scheme and formally proving insecurity.

The structure below mirrors the standard structure of a top-tier cryptography conference paper (CRYPTO, EUROCRYPT, ASIACRYPT, CCS, S&P), annotated with MTSF-specific guidance.

17.1 Step 1: Define the Scheme or Protocol

What to Include in the Scheme Description

Every security research paper begins with a precise, self-contained description of the cryptographic object being analysed.

For a block cipher (e.g., AES, PRESENT, Serpent):

1. **Parameters:** Block size n (e.g., 128 bits), key size κ (e.g., 128/192/256 bits), number of rounds r (e.g., 10/12/14).
2. **Round function:** Specify each operation (SubBytes/S-box, ShiftRows/permutation layer, MixColumns/diffusion layer, AddRoundKey).
3. **Key schedule:** How the master key is expanded into round keys $K_0, K_1, \dots, K_r$.
4. **Diagram:** A figure showing one round and the overall structure.

For a hash function (e.g., SHA-3, BLAKE3, ASCON-Hash):

1. **Construction:** Merkle–Damgård, sponge, Merkle tree, or other.
2. **Compression/permutation function:** Internal state size, rate r , capacity c .
3. **Padding rule:** How the message is padded to a multiple of the block size.
4. **Output extraction:** How the digest is extracted from the internal state.

For a stream cipher (e.g., Grain-128a, ChaCha20, Trivium):

1. **State:** Internal state size, register structure (LFSR, NFSR, counters).
2. **Initialisation:** How the key and IV/nonce are loaded and mixed.
3. **Output function:** How keystream bits are extracted from the state.
4. **Update function:** How the state evolves after each output bit.

For a protocol (e.g., TLS 1.3, Signal, key exchange):

1. **Parties:** Who participates (client, server, KDC, CA).
2. **Message flow:** A sequence diagram showing every message, its content, and which party sends it.
3. **Cryptographic operations:** Which primitives are used (signature scheme, KEM, AEAD, KDF, hash).
4. **Session structure:** What constitutes a “session” (SID, nonces, keys generated).
5. **Assumptions:** What is trusted (PKI, random oracle, authenticated channels).

17.2 Step 2: Define the Security Goals

Security Goals as MTSF Market Goods

After describing the scheme, specify exactly what properties you claim. In MTSF, each security goal becomes a *security good* g_j in the market catalogue $\mathcal{G}$:

Common security goals and their MTSF goods:

Scheme type	Security goal	Standard name	MTSF good
Block cipher	Pseudorandom permutation	PRP	g_{PRP}
Block cipher	Strong PRP (both directions)	SPRP	g_{SPRP}
Hash function	Collision resistance	CR	g_{CR}
Hash function	Preimage resistance	Pre	g_{Pre}
Hash function	Second-preimage resistance	Sec	g_{Sec}
Hash function	Length-extension resistance	LE	g_{LE}
Stream cipher	PRG indistinguishability	PRG	g_{PRG}
Stream cipher	Key recovery resistance	KR	g_{KR}
Signature	Existential unforgeability	EUFCMA	g_{EUFCMA}
Encryption	Chosen-ciphertext security	IND-CCA2	$g_{\text{IND-CCA2}}$
KEM	Encapsulation security	IND-CCA2	$g_{\text{IND-CCA2}}$
Protocol	Entity authentication	Auth	g_{auth}
Protocol	Mutual authentication	Mutual	g_{mutual}
Protocol	Session-key secrecy	SK	g_{sk}
Protocol	Forward secrecy	FS	g_{FS}
Protocol	Post-compromise security	PCS	g_{PCS}

Be explicit: State the full name and formal abbreviation. For example: “We prove that our scheme achieves EUFCMA security (existential unforgeability under adaptive chosen-message attack), which in MTSF corresponds to the security good g_{EUFCMA} with ask price $\text{Ask}(g_{\text{EUFCMA}}) \leq \text{negl}(\lambda)$.”

17.3 Step 3: Formal Security Definitions

Writing Formal Security Definitions in MTSF

For each security good, you must provide a formal game definition. In MTSF, this is the specification of what the buyer (adversary) must do to win the auction:

Template for a security definition:

1. **Setup:** The seller (challenger) generates keys/parameters and gives the public components to the buyer.
2. **Oracle access:** The buyer may make adaptive queries to oracles (signing, encryption, hash, etc.). Specify which oracles and any restrictions (e.g., “the buyer may not query the decryption oracle on the challenge ciphertext”).
3. **Challenge:** The seller issues a challenge (e.g., a challenge ciphertext, a commitment).
4. **Winning condition:** The buyer outputs a response (a forgery, a guess bit, a key). Specify exactly when the buyer “wins.”
5. **Advantage:** Define $\text{Adv}_{\mathcal{A}}^{g_j}(\lambda) = |\Pr[\text{buyer wins}] - \text{baseline}|$, where the baseline is 0 for forgery games and $1/2$ for distinguishing games.

In MTSF notation: The quoted price $\text{QPrice}_0(g_j) = \text{Adv}_{\mathcal{A}}^{g_j}(\lambda)$ is the advantage in this game. The security good g_j is *in equilibrium* if $\text{QPrice}_0 \leq \text{negl}(\lambda)$ for all PPT buyers $\mathcal{A}$.

17.4 Step 4: Define the Adversary Model

The Adversary Model as a Buyer Profile

The adversary model specifies the buyer’s capabilities in the security market. Different adversary models correspond to different buyer profiles:

For primitives (block ciphers, hash functions, stream ciphers):

Adversary model	MTSF buyer profile	Typical budget
Chosen-plaintext (CPA)	Buyer with encryption oracle	q_E queries
Chosen-ciphertext (CCA)	Buyer with enc. + dec. oracles	$q_E + q_D$ queries
Related-key	Buyer can request related keys	q_{rk} related-key queries
Known-plaintext (KPA)	Buyer observes (m, c) pairs	q_{obs} observations
Nonce-misuse	Buyer can force nonce reuse	$\text{Ask} = 1$ if misused
Side-channel	Buyer observes physical leakage	q_{phys} traces, probing depth d

For protocols:

Adversary model	MTSF buyer profile	Capabilities
Passive eavesdropper	Buyer observes all messages	Read-only network access
Active network	Buyer controls the network	Delay, drop, inject, modify
Corrupting adversary	Buyer can corrupt parties	Learns long-term keys
Quantum adversary	Buyer has quantum computer	Superposition oracle queries

MTSF formalisation: The adversary model determines the buyer’s *budget* $T \leq \text{poly}(\lambda)$ and *oracle access set*. Specify these explicitly:

- “The buyer is a PPT algorithm with budget $T \leq \lambda^c$ for any constant c .”
- “The buyer may make at most q_S signing queries, q_H hash queries, and q_D decapsulation queries.”

- “The buyer controls the network: it may delay, reorder, drop, and inject messages between honest parties.”
- “The buyer may corrupt up to t out of n parties, learning their long-term secret keys.”

17.5 Step 5: Construct the MTSF Security Proof

The MTSF Proof Pipeline

The proof follows the four-step MTSF pipeline (Figure 28):

Phase 1: Set up the market (Bidding Round $\mathbf{B}_0$).

1. Define the market $\mathcal{M} = (\text{Seller}, \{\text{Buyer}_i\}, \mathcal{G}, \text{Price})$.
2. State the quoted price: $\text{QPrice}_0(g_j) = \text{Adv}_{\mathcal{A}}^{g_j}(\lambda)$.
3. Describe the real game: the seller runs the scheme honestly; the buyer interacts via oracles.
4. Emphasise: the quoted price stands unreduced in $\mathbf{B}_0$. No analysis has been performed yet.

Phase 2: Bidding-round chain ($\mathbf{B}_1, \mathbf{B}_2, \dots, \mathbf{B}_q$).

For each attack vector, construct a bidding round. Use this checklist from the bid taxonomy (Section B):

- ☐ **Nonce freshness bid:** Abort on nonce collision. $\Delta\text{Price} \leq q^2/2N$ (birthday bound).
- ☐ **Hash collision bid:** Abort on RO collision. $\Delta\text{Price} \leq q_H^2/2^{n+1}$.
- ☐ **Oracle programming:** Embed hard-problem challenge. $\Delta\text{Price} = 0$.
- ☐ **Computational hop:** Replace structured value with random. $\Delta\text{Price} \leq \text{Adv}^{\text{hard}}$.
- ☐ **Forgery/extraction bid:** Convert adversary success to hard-problem solution. $\Delta\text{Price} \leq L \cdot \text{Adv}^{\text{hard}}$.
- ☐ **Side-channel bid:** Model physical leakage. $\Delta\text{Price} \leq \text{Adv}^{\text{SCA}}(\lambda, d)$.

For each round, state: (a) purpose, (b) what is replaced, (c) the bad event(s) F_k , (d) the price adjustment ΔPrice_k , and (e) the market interpretation (which bid fails and why).

If multiple bad events occur simultaneously, use the *extended difference lemma* (Theorem 3.7):

$$\Delta\text{Price}_k \leq \Pr[F_1 \cup \dots \cup F_m] \leq \sum_i \Pr[F_i] - \sum_{i < j} \Pr[F_i \cap F_j] + \dots$$

Phase 3: Reach the ideal game ($\mathbf{B}_q$).

In the ideal game, all attack vectors have been eliminated. State: $\Pr[\mathbf{B}_q = 1] = 0$ (for forgery/authentication games) or $\Pr[\mathbf{B}_q = 1] = 1/2$ (for distinguishing games). Conclude:

$$\text{QPrice}_0(g_j) = \text{Ask}(g_j, \lambda) \leq \sum_{k=0}^{q-1} \Delta\text{Price}_k + \Pr[\mathbf{B}_q = 1] \leq \text{negl}(\lambda).$$

This establishes market equilibrium: the quoted price was a bluff.

Phase 4: Session-level verification.

If the scheme is a protocol:

1. Construct the session-CNF φ_i (Theorem 4.1). Fill in the four-column worksheet.
2. Verify session pinging (Theorem 5.3): check that consecutive sessions are structurally distinct.
3. Apply the unbounded security theorem (Theorem 5.5) to extend security to unbounded sessions.

17.6 Step 6: Proving Insecurity with MTSF

How to Write an MTSF Insecurity Proof

If you are demonstrating that a scheme or protocol is *insecure*, MTSF uses the same bidding-round structure:

Phase 1: Set up the market. Same as a security proof: define $\mathcal{M}$, state the quoted price QPrice_0 .

Phase 2: Construct a successful bid. Instead of eliminating attack vectors, *exhibit* one that succeeds:

1. Describe the adversary’s strategy (the “bid”) in detail.
2. Show that the bid succeeds with non-negligible probability: $\Delta\text{Price} = \Omega(1)$ or $\Delta\text{Price} = 1$.
3. Identify which security good collapses.

Phase 3: Demonstrate market collapse. State:

$$\text{Ask}(g_j) \geq \epsilon = \Omega(1) \Rightarrow \text{Market collapsed for } g_j.$$

Phase 4: CNF diagnosis. Show that the session-CNF is satisfiable under the dishonest trace:

1. Fill in the CNF worksheet under the attack trace.
2. Identify which clause *should* be FALSE but is TRUE (or which clause is *missing*).
3. This pinpoints the design flaw.

Phase 5: Propose a fix. If possible:

1. Add the missing CNF clause (e.g., identity binding for Needham–Schroeder).
2. Re-run the MTSF proof on the fixed scheme.
3. Show that the previously successful bid now fails.

This “disprove $\rightarrow$ diagnose $\rightarrow$ fix $\rightarrow$ prove” cycle is the natural application of MTSF in protocol engineering, as demonstrated in the Telegram case study (Section 13.9).

17.7 Step 7: Structuring the Paper

Recommended Paper Structure Using MTSF

1. **Introduction.** Motivate the scheme/protocol. State the main result informally: “We prove that our scheme achieves g_X security with ask price $\text{Ask} \leq \text{negl.}$ ” or “We demonstrate that Protocol Y has a market collapse: $\text{Ask}(g_{\text{auth}}) = 1$ via a masquerade bid.”
2. **Preliminaries.** Notation, hardness assumptions, MTSF definitions (reference this article).
3. **Scheme/Protocol Description.** Full specification with diagrams (Step 1 above).
4. **Security Goals and Definitions.** Market goods, formal game definitions, adversary model (Steps 2–4).
5. **MTSF Security Market Setup.** Define the market $\mathcal{M}$; state the quoted price QPrice_0 (Phase 1 of Step 5).
6. **Security Proof (Bidding-Round Chain).** The main technical contribution. Each bidding round is a subsection with the standard tcolorbox template (Purpose, Replaces, Quoted Price/Complexity).
7. **CNF Session Verification.** The session-CNF formula and filled-in worksheet (Phase 4 of Step 5).
8. **Session Pinging and Unbounded Security.** Structural distinctness check and induction theorem application.
9. **Discussion.** Interpret the ask price bounds. Compare with prior work. Discuss tightness.
10. **Conclusion.** Summarise the market verdict (equilibrium or collapse) and implications.

Analogy:

Writing a Security Paper Is Like Conducting a Market Audit Think of your research paper as an audit report for a financial market:

- **Introduction:** “We audited the XYZ Financial Market.”
- **Scheme description:** “Here is how the market operates.”
- **Security goals:** “These are the goods the market claims to sell.”
- **Adversary model:** “These are the traders who might try to manipulate the market.”
- **Security proof:** “We tested every possible manipulation strategy. Each one fails.”
- **CNF worksheet:** “Here is the compliance checklist. Every box is ticked.”
- **Session pinging:** “The market has been tested across unlimited trading sessions.”
- **Conclusion:** “Market verdict: equilibrium (healthy) / collapse (needs intervention).”

For an insecurity paper, the audit reveals a manipulation strategy that succeeds, identifies which compliance check was missing, and recommends a fix.

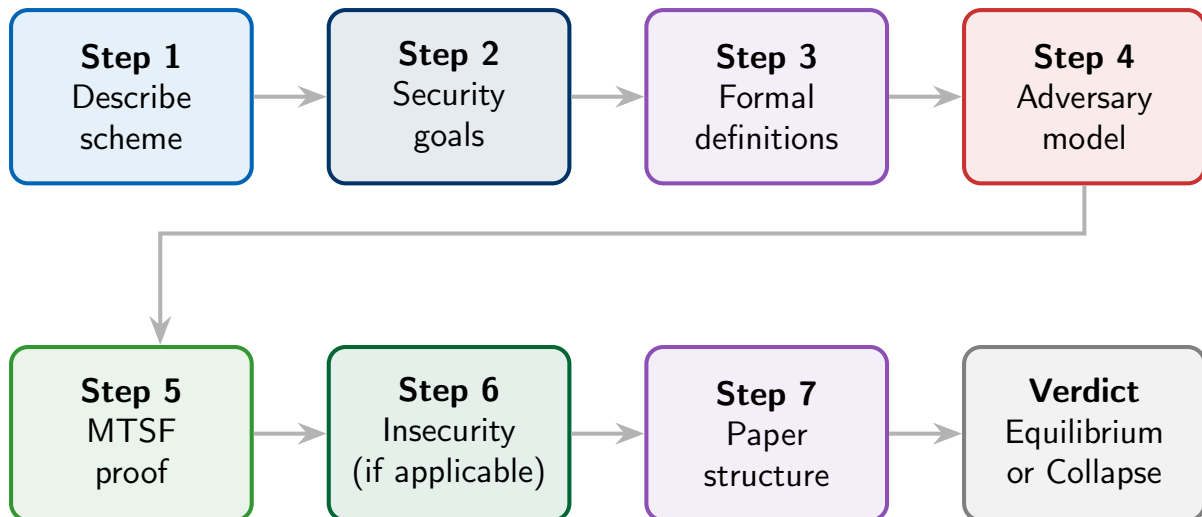


Figure 64: The seven-step pipeline for writing a security research paper using MTSF. Steps 1–4 set up the scheme, goals, definitions, and adversary model. Step 5 constructs the MTSF proof. Step 6 handles insecurity (if applicable). Step 7 organises the paper. The final verdict is equilibrium (secure) or collapse (insecure).

18 Presenting MTSF Proofs for Different Publication Venues

Cryptographic research is published across venues with vastly different page limits, audiences, and expectations. A proof that fills 40 pages at CRYPTO would be rejected at IEEE Transactions on Information Forensics and Security (TIFS) for exceeding the 13-page limit. Conversely, a compressed 8-page proof at a journal might be criticised at EUROCRYPT for lacking detail. This section provides concrete, venue-specific guidance for presenting MTSF proofs at three classes of publication venue: (1) page-restricted IEEE Transactions, (2) page-unrestricted Springer/Elsevier journals, and (3) cryptographic conferences with lengthy proceedings.

* Simple Terms: Why Venue-Specific Presentation Matters

The core problem: An MTSF proof of a real protocol (e.g., TLS 1.3) naturally contains: market setup, 5–10 bidding rounds each with a tcolorbox explanation, extended difference lemma application, CNF worksheet, session pinging theorem, and potentially a composition argument. In full detail, this can span 30–50 pages. But different venues have different constraints:

- **IEEE Transactions** (TIFS, TDSC, TPAMI): Strict 12–14 page limit (double-column, 10pt). The audience is broad: information security practitioners, engineers, applied researchers. They want the result and its implications, not 40 pages of game hops.
- **Springer/Elsevier journals** (Designs, Codes and Cryptography; Journal of Cryptology; Journal of Computer Security): Flexible page limits (20–60+ pages). The audience is a mix of theorists and practitioners. They expect rigour but appreciate accessibility.
- **Cryptographic conferences** (CRYPTO, EUROCRYPT, ASIACRYPT, TCC, CCS, S&P): Extended proceedings (25–40 pages). The audience is expert cryptographers who expect full proofs with every detail, every bound, and every reduction spelled out. Supplementary material or full versions on ePrint are standard.

MTSF’s modular structure is uniquely suited to venue adaptation: the same proof can be presented at different levels of detail by expanding or compressing individual bidding rounds,

CNF worksheets, and session pinging arguments. The core result (the ask price bound) and the market verdict (equilibrium or collapse) remain identical across all presentations.

18.1 Case 1: IEEE Transactions (Page-Restricted, Broad Audience)

IEEE Transactions Template: 12–14 Pages, Double-Column

Target venues: IEEE Transactions on Information Forensics & Security (TIFS), IEEE Transactions on Dependable and Secure Computing (TDSC), IEEE Communications Letters, IEEE Access.

Page budget: 12–14 pages double-column (approximately 7,000–8,000 words).

Audience: Applied security researchers, network engineers, system architects, government analysts. Many readers are *not* expert cryptographers. They want to know: (a) what the result is, (b) why they should trust it, (c) what the practical implications are.

Recommended MTSF paper structure for IEEE:

1. **Introduction (1.5 pages):** State the problem, the result (equilibrium or collapse with explicit ask price), and the practical implication. Include a 1-paragraph “MTSF in 30 seconds” summary for readers unfamiliar with the framework. Use the market analogy: “Security is a marketplace where attackers bid computational resources; our protocol’s security goods cannot be purchased at any affordable price.”
2. **Preliminaries (1 page):** Notation table (use the Quick-Reference Card 1 format from Section B.0.0). State hardness assumptions in one line each. Reference this MTSF article for full definitions: “We work in the MTSF framework of [this article]; see Definitions X–Y therein.”
3. **Scheme/Protocol Description (1.5 pages):** Full specification with *one* sequence diagram. For IEEE, include a practical context paragraph: “This protocol is deployed in 3.5 billion devices worldwide” or “This cipher is the NIST lightweight standard.”
4. **Security Market and Main Theorem (1 page):** Define the market $\mathcal{M}$ in one paragraph. State the main theorem with full bound:

Theorem (Main). *The ask price satisfies $\text{Ask}(g_{\text{auth}}) \leq \text{Adv}^{\text{EUF}} + q_N^2/2^{\lambda+1} + \text{negl}(\lambda)$. For $\lambda = 128$, this gives $\text{Ask} \leq 2^{-128}$: market equilibrium.*

5. **Proof Sketch (3–4 pages):** This is the core of the IEEE version. Present each bidding round as a *single paragraph* (not a full tcollection). Use a **compressed bidding-round table** instead of individual subsections:

Round	Bid type	What changes	Price adjustment
B_0	Baseline	Nothing (quoted price QPrice_0)	—
B_1	Nonce freshness	Abort on nonce collision	$\leq q_N^2/2^{\lambda+1}$
B_2	Oracle programming	Embed hard-problem challenge	$= 0$
B_3	Forgery extraction	Reduce to ECDLP/MLWE	$\leq \text{Adv}^{\text{hard}}$
B_4	Ideal market	All bids fail	$\Pr[B_4 = 1] = 0$
Total:			$\text{Ask} \leq q_N^2/2^{\lambda+1} + \text{Adv}^{\text{hard}}$

For the **key bidding round** (typically the reduction to the hard problem), provide 1–2 paragraphs of detail. For all other rounds, the table entry plus one sentence suffices.

If the extended difference lemma is used, state it as: “Rounds $\mathbf{B}_1$ – $\mathbf{B}_2$ involve simultaneous failures $F_{\text{nonce}} \cup F_{\text{hash}}$; by the extended difference lemma [ref], $\Delta\text{Price} \leq \Pr[F_{\text{nonce}}] + \Pr[F_{\text{hash}}]$.”

6. **CNF Verification (0.5 page):** Present the CNF formula in one equation and the filled-in worksheet as a small table. Omit the five-phase algorithm; reference it.
7. **Session Pinging (0.5 page):** State: “By Theorem [ref], the protocol is secure for unbounded sessions with $\delta_{\text{ping}} \leq \text{negl.}$ ” Include the structural distinctness check as a 3-line argument, not the full induction proof.
8. **Discussion and Comparison (1.5 pages):** Compare with prior work. Discuss parameter choices. Address practical deployment considerations. Include a battle-card-style summary figure (see Figure 68).
9. **Conclusion (0.5 page):** “Market verdict: equilibrium. Our protocol achieves 128-bit security under the MLWE assumption, with unbounded session guarantees and composability.”

What to move to supplementary material / ePrint:

- Full tcolorbox descriptions of each bidding round (Purpose, Replaces, Quoted Price).
- Complete extended difference lemma calculations with inclusion-exclusion.
- Full induction proof for the session pinging theorem.
- Side-channel bid analysis.
- Detailed CNF clause-by-clause verification.

Key IEEE formatting tip: Use the compressed bidding-round table as the centerpiece of the proof. Reviewers can verify the proof structure at a glance. The full version on ePrint provides all details.

Analogy:

IEEE Papers as Executive Summaries Think of an IEEE Transactions paper as an *executive summary* of a financial audit. The board of directors (reviewers) does not want to read every ledger entry. They want: (1) the bottom line (market verdict), (2) the methodology (MTSF bidding rounds), (3) the key finding (which bids fail and why), and (4) the recommendation (deploy/do not deploy). The full audit report (ePrint version) is available for anyone who wants to check every number.

18.2 Case 2a: Cryptographic Journals (Page-Unrestricted, Expert Cryptographic Audience)

Cryptographic Journal Template: 25–50 Pages, Rigorous and Thorough

Target venues: Journal of Cryptology (JoC, Springer—the flagship journal of the IACR), Designs Codes and Cryptography (DCC, Springer), Journal of Mathematical Cryptology (de Gruyter), Applicable Algebra in Engineering Communication and Computing (AAECC, Springer), Finite Fields and Their Applications (Elsevier), IACR Communications in Cryptology.

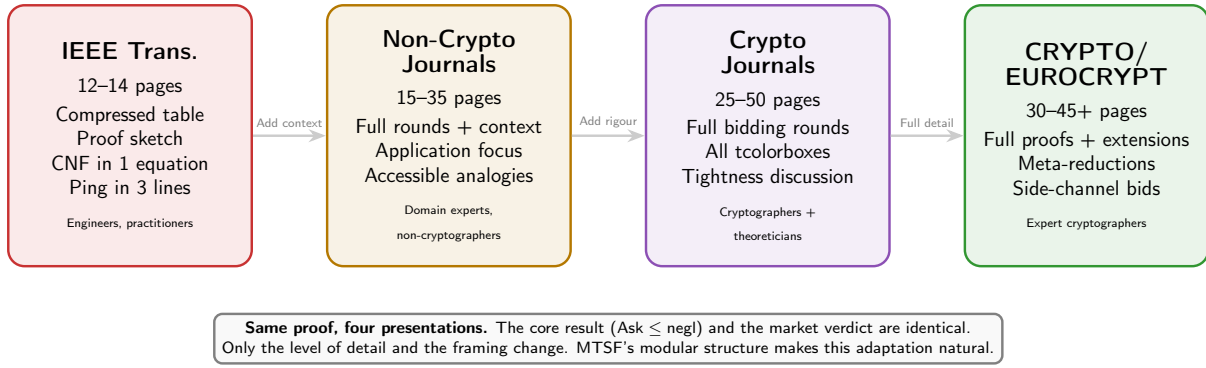


Figure 65: Four venue-specific presentations of the same MTSF proof. IEEE: compressed table. Non-crypto journals: full rounds with application context and analogies. Crypto journals: full rigorous proofs with tightness. CRYPTO/EUROCRYPT: full proofs with extensions, meta-reductions, and composition.

Page budget: 25–50+ pages (single-column, 11pt or 12pt). No strict upper limit; mathematical depth and rigour govern acceptance.

Audience: Expert cryptographers, mathematicians with cryptographic interests, and advanced graduate students. Reviewers are specialists who have themselves written game-based or UC proofs. They will check every bound, every reduction, and every simulation argument. They expect:

- Full formal definitions for every security notion used (not just citations).
- Complete proofs with no “it is easy to see” gaps.
- Explicit tightness analysis—how much does the reduction lose, and is that loss inherent?
- Comparison with the best known bounds for the same or related schemes.
- Discussion of whether the result extends to the QROM or post-quantum setting.

Recommended MTSF paper structure for cryptographic journals:

1. **Introduction (3–4 pages):** State the main theorem with the exact bound. Include a “technical overview” subsection (1–2 pages) that walks through the proof strategy at a high level, naming each bidding round and its cost. Compare quantitatively with prior work: “Our bound improves on [ref] by a factor of q_S in the tightness loss.”
2. **Preliminaries (4–5 pages):** Full formal definitions of all hardness assumptions (ECDLP, MLWE, MSIS, gap-CDH, etc.) and all security notions (EUF-CMA, IND-CCA2, PRP, SUF-CMA). Include the MTSF definitions: market (Theorem 3.1), bid (Theorem 3.2), ask price and equilibrium (Theorem 3.4), quoted price (Theorem 3.3), extended difference lemma (Theorem 3.7). For a JoC audience, state these formally with full mathematical precision—no beginner boxes needed in the main body.
3. **Scheme/Protocol Description (2–4 pages):** Precise algorithmic specification using algorithm boxes (KeyGen, Sign, Verify or Enc, Dec). Correctness theorem. For protocols, a sequence diagram with exact message formats.
4. **MTSF Market Setup (1–2 pages):** Formal market definition $\mathcal{M} = (\text{Seller}, \{\text{Buyer}_i\}, \mathcal{G}, \text{Price})$. List all security goods with their formal game definitions. State the quoted price $\text{QPrice}_0(g_j) = \text{Adv}_{\mathcal{A}}^{g_j}(\lambda)$. Establish the relationship between MTSF goods and standard security notions.

5. **Security Proof—Bidding-Round Chain (10–20 pages):** The core contribution. Present each bidding round with:

- The complete MTSF tcolorbox (Purpose, Replaces, Quoted Price) with full technical detail.
- A rigorous formal argument: “Let F_k denote the event...By Theorem 3.7, $|\Pr[\mathbf{B}_{k-1}=1] - \Pr[\mathbf{B}_k=1]| \leq \Pr[F_k]$. We bound $\Pr[F_k]$ as follows...”
- For computational hops: the complete reducer construction $\mathcal{R}^{\mathcal{A}}$ with explicit simulation of every oracle.
- For the extended difference lemma: the full inclusion-exclusion calculation showing which intersection terms vanish.
- The bidding-round chain figure.

Tightness analysis (required for JoC/DCC): After the main proof, include a dedicated subsection:

- State the tightness loss factor $L(\lambda)$ explicitly.
- Compare with known lower bounds on tightness (e.g., Coron’s impossibility result for ROM-based signature reductions).
- If possible, provide a meta-reduction or matching attack showing the loss is inherent.

6. **CNF Session Verification (2–3 pages):** Full session-CNF with all clauses. Complete five-phase verification algorithm. Worksheets for honest and dishonest traces. Formal proof of adversarial CNF unsatisfiability (Theorem 4.3).

7. **Session Pinging (2–3 pages):** Complete structural distinctness verification. Full induction proof (not a sketch). Quantitative advantage accumulation analysis.

8. **Extensions (2–5 pages):**

- Side-channel bid with d -probing model analysis.
- QROM extension (if applicable): O2H or measure-and-reprogram.
- Composition via market merger theorem (if applicable).
- Nonce-misuse resilience analysis.

9. **Comparison with Prior Work (2–3 pages):** Detailed table with exact numerical bounds, tightness factors, assumptions, and whether composition/unbounded sessions are addressed.

10. **Conclusion (1 page):** State the market verdict with the exact bound. List concrete open problems.

Cryptographic journal-specific tips:

- **No beginner boxes in the main body.** Save pedagogical elements for an appendix or the ePrint version. JoC/DCC reviewers know what IND-CCA2 means.
- **Include every tcolorbox.** Even expert readers benefit from the structured (Purpose, Replaces, Quoted Price) template—it provides a standardised format that makes 20-page proofs navigable.

- **Name every bid from the taxonomy.** Explicitly reference the bid taxonomy (Section B) and state which bids are relevant, which are addressed, and which are out of scope (with justification).
- **Tightness is mandatory.** Unlike non-crypto journals, JoC/DCC reviewers *require* a tightness discussion. A tight reduction is a selling point; a loose reduction must be justified.
- **State the quoted price in B_0 .** Write: “The seller quotes $QPrice_0 = \text{Adv}_A^{\text{EUF}}(\lambda)$. We decompose $QPrice_0$ into five negligible terms...”

Analogy:

Cryptographic Journal Papers as Peer-Reviewed Legal Briefs A paper in the Journal of Cryptology is like a legal brief submitted to a supreme court of cryptography. Every claim must be backed by a rigorous argument. Every reduction must be spelled out completely. Every bound must be proven, not estimated. The “judges” (reviewers) are themselves expert lawyers (cryptographers) who will scrutinise every line. MTSF’s structured tcolorbox format serves as a standardised “brief template” that ensures no argument is omitted and every step is labelled—making both writing and reviewing more systematic.

18.3 Case 2b: Non-Cryptographic Journals (Page-Unrestricted, Domain-Specific Audience)

Non-Cryptographic Journal Template: 15–35 Pages, Application-Focused

Target venues: Vehicular Communications (Elsevier), Computer Networks (Elsevier), Computer Communications (Elsevier), Ad Hoc Networks (Elsevier), Future Generation Computer Systems (Elsevier), Pervasive and Mobile Computing (Elsevier), Journal of Network and Computer Applications (Elsevier), Sensors (MDPI), Electronics (MDPI), Applied Sciences (MDPI), Information Sciences (Elsevier), Journal of Information Security and Applications (Elsevier), Wireless Personal Communications (Springer), Peer-to-Peer Networking and Applications (Springer), Cluster Computing (Springer), Journal of Ambient Intelligence and Humanized Computing (Springer), International Journal of Information Security (Springer).

Page budget: 15–35 pages. Some venues (e.g., Elsevier journals) have flexible limits; others (e.g., MDPI journals) suggest 15–25 pages. Quality and relevance govern length.

Audience: Domain experts in networking, IoT, vehicular systems, mobile computing, or applied security. Most readers are *not* expert cryptographers. They are engineers, system designers, and applied researchers who:

- Understand the *application* (e.g., vehicle-to-vehicle communication, smart grid security, healthcare IoT) deeply.
- Know that security is important but may not have written a security proof themselves.
- Want to know: Is this protocol secure? How secure? What are the practical parameters? Does it compose with existing deployed infrastructure?
- Appreciate clear explanations, real-world examples, and diagrams over dense mathematical notation.

Recommended MTSF paper structure for non-cryptographic journals:

1. **Introduction (2–3 pages):** Lead with the *application*, not the cryptography. For Vehicular Communications: “Connected vehicles exchange safety messages 10 times per second. A single forged brake warning could cause a multi-car pile-up. We prove that our V2X authentication protocol is secure using the Market-Theoretic Security Framework (MTSF).”

Include a “MTSF in one paragraph” box:

MTSF models security proofs as economic markets. The protocol designer (seller) offers security guarantees (goods). Potential attackers (buyers) bid computational resources to break them. If no attacker can afford to win, the market is in equilibrium—the protocol is secure. If any attacker wins cheaply, the market collapses—the protocol is broken.

State the main result in plain language: “Our protocol achieves 128-bit authentication security: the probability that an attacker successfully impersonates a vehicle is less than 2^{-128} , even after observing billions of legitimate messages.”

2. **Related Work (2–3 pages):** Review the application-domain literature (vehicular security, IoT authentication, etc.) as well as the proof methodology. Position MTSF relative to informal “security analysis” sections common in non-crypto journals: “Unlike prior work that argues security informally, we provide a *mathematical proof* within the MTSF framework, producing explicit numerical bounds and session-level auditing.”

3. **System Model and Threat Model (2–3 pages):** This is critical for non-crypto journals. Describe the system in domain-specific language:

- **For vehicular communications:** Network topology (V2V, V2I), communication range, message frequency, latency requirements, on-board unit capabilities.
- **For IoT:** Device constraints (RAM, CPU, battery), communication protocols (MQTT, CoAP, BLE), gateway architecture.
- **For healthcare:** Patient data sensitivity, regulatory requirements (HIPAA, GDPR), device types (wearables, implants, monitors).

Map the threat model to MTSF buyer profiles:

Domain threat	MTSF buyer profile	Bid type
Rogue vehicle broadcasting fake alerts	Active network adversary	Forgery bid
Eavesdropper on V2V channel	Passive eavesdropper	Distinguishing bid
Stolen on-board unit	Corrupting adversary	Key recovery bid
Replay of yesterday’s traffic signal	Session replay attacker	Nonce freshness bid
GPS spoofing combined with message injection	Multi-vector attacker	Extended diff. lemma

4. **Proposed Protocol/Scheme (3–5 pages):** Full specification with:

- A system architecture diagram showing all parties and communication channels.
- A message sequence diagram showing the protocol flow.
- Algorithm boxes for key operations.
- A “design rationale” paragraph for each component: “We use ECDSA for authentication because it provides 128-bit security with 64-byte signatures, fitting within the 300-byte V2X message budget.”
- Performance analysis: computation time, communication overhead, storage requirements. Non-crypto journal reviewers *expect* this; crypto journal reviewers consider it optional.

5. **MTSF Security Analysis (5–8 pages):** This section must be *self-contained*—do not assume the reader has read the MTSF reference article. Structure it as follows:

5a. Market setup (1 page): Define the market in plain language with a diagram. Use the market overview figure (Figure 9). List security goods with one-sentence descriptions:

Our protocol offers three security goods: (1) g_{auth} : a rogue vehicle cannot impersonate a legitimate one; (2) g_{sk} : session keys are secret even to eavesdroppers; (3) g_{fresh} : replayed messages are always detected.

5b. Bidding-round chain (3–5 pages): Present each round with:

- A tcolorbox (Purpose, Replaces, Quoted Price) for the key rounds.
- For routine rounds (nonce freshness, oracle programming), a single-paragraph summary is sufficient.
- **Include an analogybox or intuitionbox for the most technical step.** Non-crypto readers need the “why” not just the “what.” For example: “This step is like checking that every licence plate on the road is unique. If two cars share a plate, the system cannot tell them apart—a dangerous confusion. The birthday bound shows that with 256-bit nonces, a collision requires observing 2^{128} messages—far more than any road network will ever produce.”
- State the final bound in both mathematical and plain-language form: “ $\text{Ask}(g_{\text{auth}}) \leq 2^{-128}$. In practical terms, an attacker would need to perform more operations than there are atoms in the universe to have even a one-in-a-trillion chance of forging a vehicle identity.”

5c. CNF worksheet (1 page): Present the filled-in worksheet with domain-specific check descriptions:

Check	What it means	How verified	Pass?
Session ID valid	Message belongs to this session	Compare session tokens	T
Timestamp fresh	Message is not a replay	Check timestamp window	T
Signature valid	Sender is who they claim	Verify ECDSA signature	T
Certificate valid	Sender’s key is CA-certified	Check certificate chain	T

5d. Session pinging (0.5 page): State the unbounded security result in one paragraph: “Our protocol is secure for an unlimited number of sessions. Each new session uses fresh nonces and session-specific signatures, ensuring that security does not degrade over time—even after millions of vehicle encounters.”

6. **Performance Evaluation (3–5 pages):** This section is *critical* for non-crypto journals and often *absent* from crypto journals. Include:

- **Computation cost:** Signing time, verification time, key generation time on target hardware (e.g., “ECDSA-P256 verification takes 1.2 ms on an ARM Cortex-M4”).
- **Communication overhead:** Message sizes, number of round trips, bandwidth consumption.
- **Storage:** Key sizes, certificate sizes, session state.
- **Comparison table:** Your protocol vs. competing protocols on all metrics (security level, computation, communication, storage, composition, unbounded sessions).
- **Simulation or implementation results:** If applicable, ns-3 simulation for vehicular networks, testbed results for IoT, or benchmarks on embedded hardware.

7. **Discussion (1–2 pages):** Address practical deployment: How does this integrate with existing infrastructure (e.g., IEEE 1609.2 for V2X, or the ETSI ITS-G5 standard)? What are the limitations? What happens if the CA is compromised?
8. **Conclusion (0.5–1 page):** State the market verdict in domain language: “We have proven that our V2X authentication protocol achieves market equilibrium with ask price 2^{-128} , meaning no computationally bounded attacker can forge vehicle identities, intercept session keys, or replay messages—even across unlimited driving sessions.”

Non-cryptographic journal-specific tips:

- **Lead with the application, not the math.** The first sentence should mention vehicles, IoT devices, patients, or smart grids—not “game-based proofs” or “UC framework.”
- **Include 3–5 analogy/intuition boxes.** Non-crypto reviewers appreciate these. A well-chosen analogy (“nonce collision is like two cars sharing a licence plate”) can make the difference between acceptance and a “too theoretical” rejection.
- **Performance evaluation is mandatory.** Non-crypto journals rank experimental results highly. A security proof without performance data may be desk-rejected.
- **Translate every bound into plain language.** After every formula, write: “In practical terms, this means...”
- **Include a security-vs-performance trade-off discussion.** “Increasing the nonce size from 128 to 256 bits doubles the signature overhead but increases the collision resistance from 2^{64} to 2^{128} messages.”
- **Use the battle-card format for the security summary.** A visual one-page summary (like Figure 68) is more accessible than a page of equations for a non-crypto audience.
- **Reference standards.** Non-crypto reviewers want to know how the protocol relates to existing standards (NIST, IEEE, ETSI, 3GPP, IETF RFC).
- **The MTSF proof elevates your paper above “informal security analysis.”** Many non-crypto journal papers include only informal arguments (“the attacker cannot forge because ECDSA is hard”). A formal MTSF proof with explicit bounds, CNF worksheets, and session pinging is a significant differentiator and strengthens the contribution.

Analogy:

Non-Crypto Journal Papers as Consumer Product Safety Reports A paper in Vehicular Communications or Sensors is like a *consumer product safety report*. The readers are not chemists (cryptographers)—they are product engineers and regulators who need to know: Is this product safe? How safe? What tests were performed? What are the operating limits? The MTSF proof is the safety test. The market verdict is the safety rating. The CNF worksheet is the inspection checklist. The performance evaluation is the product specification sheet. The battle card is the safety label on the box. Package these together and the non-crypto reviewer sees a complete, professional, and trustworthy security analysis—not just “we believe this is secure because the crypto is hard.”

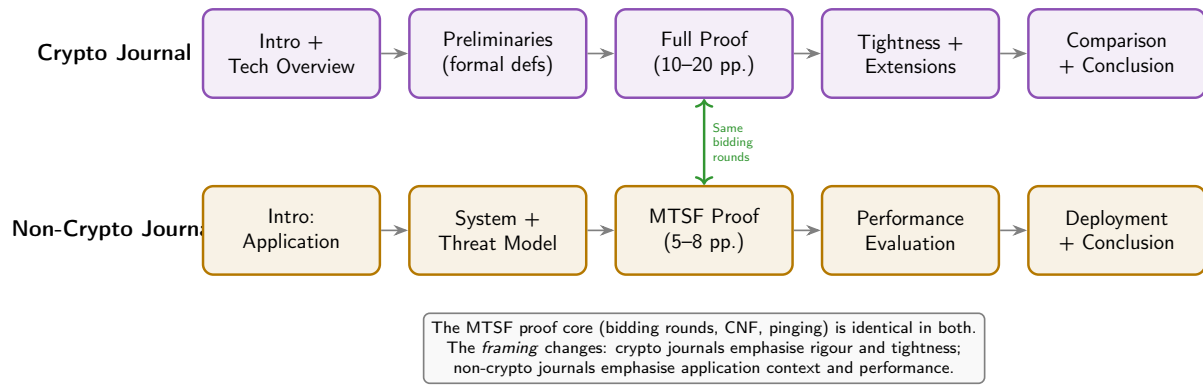


Figure 66: Paper structure comparison: cryptographic journals (top) vs. non-cryptographic journals (bottom). The MTSF proof core is shared; the framing (application context vs. tightness analysis) differs.

18.4 Case 3: Cryptographic Conferences (Extended Proceedings, Expert Audience)

Cryptographic Conference Template: 30–45+ Pages, Expert Audience

Target venues: CRYPTO, EUROCRYPT, ASIACRYPT, TCC (theory); CCS, S&P, USENIX Security, NDSS (systems security); PKC, CT-RSA (applied crypto); CHES (hardware security); AFC (Africacrypt); Latincrypt; Indocrypt; ACISP.

Page budget: 30 pages in proceedings + unlimited supplementary material on ePrint. CRYPTO/EUROCRYPT use LNCS format (Springer, single-column, 12pt). CCS/S&P use double-column with 12–15 page limits but allow substantial appendices.

Audience: Expert cryptographers who have read Shoup [79], Bellare–Rogaway [16], Canetti [31], and the NIST post-quantum standards. They expect:

- Every reduction spelled out completely, with no “it is easy to see” gaps.
- Tightness analysis: how much does the reduction lose? Is there a matching attack showing the loss is inherent?
- Comparison with the best known bounds: does this improve on prior work quantitatively?
- A full treatment of edge cases: what happens when the adversary corrupts parties, or when nonces are misused, or when the RNG is degraded?

Recommended MTSF paper structure for CRYPTO/EUROCRYPT:

1. **Introduction (3–4 pages):** Motivate with precision. State the main theorem with the *exact* bound (not just “negligible”). Include a “technical overview” subsection (1–2 pages) walking through the proof strategy at a high level. This is where MTSF’s bidding-round classification shines: “Our proof uses five bidding rounds. Round 1 eliminates nonce collisions via birthday bound. Round 2 programs the random oracle at zero cost. Round 3 reduces forgeries to ECDLP via the forking lemma with tightness loss q_H . Rounds 4–5 handle the extended difference lemma over three simultaneous failures.”
2. **Preliminaries (3–4 pages):** Full formal definitions for every assumption (ECDLP, MLWE, MSIS, PRF, PRP). State MTSF definitions formally: market, bid, ask price, quoted price, equilibrium, extended difference lemma, session-CNF, session ping. For a CRYPTO audience, include the theorems (soundness, completeness, unbounded security)

that you will invoke. No beginner boxes needed—the audience knows what IND-CCA2 means.

3. **Scheme/Protocol Description (2–3 pages):** Precise algorithmic specification. Use algorithm boxes (KeyGen, Sign, Verify). Include a correctness theorem: “Honest execution produces valid signatures with probability $1 - \delta_{\text{correct}}$.”
4. **MTSF Market Setup (1–2 pages):** Formal market definition. List all goods with their formal game definitions (not just names). State the quoted price. Discuss the relationship between MTSF goods and standard security notions: “Our good $g_{\text{EUF-CMA}}$ is equivalent to the standard EUF-CMA game of [ref]; see Theorem 3.5.”
5. **Security Proof—Full Bidding-Round Chain (10–20 pages):** This is the bulk of the paper. Present each bidding round as a full subsection with:
 - The complete MTSF tcolorbox (Purpose, Replaces, Quoted Price) with full technical detail.
 - A complete formal argument: “Let F_k denote the event... By the difference lemma (Lemma X), $|\Pr[\mathbf{B}_{k-1} = 1] - \Pr[\mathbf{B}_k = 1]| \leq \Pr[F_k]$. We bound $\Pr[F_k]$ as follows...”
 - For the extended difference lemma: the full inclusion-exclusion calculation, showing which terms are zero (independent events) and which contribute.
 - For the key reduction round: the complete reducer construction. Specify how the reducer $\mathcal{R}$ simulates each oracle query. Compute the exact running time of $\mathcal{R}$ (including the tightness overhead).
 - For the forking lemma rounds: state the forking lemma precisely, compute the success probability after forking, and state the resulting bound.

Tightness analysis: After the main proof, include a “Tightness” subsection:

- State the tightness loss: “The reduction loses a factor of $L = q_H \cdot q_S$ in Round 3.”
- Discuss whether this loss is inherent: “Coron [ref] showed that any reduction from ECDLP to EUF-CMA in the ROM must lose a factor $\Omega(q_H)$. Our loss of $q_H \cdot q_S$ is thus within a q_S factor of optimal.”
- If possible, provide a meta-reduction showing the loss is tight.

Extended bids: CRYPTO/EUROCRYPT papers should include bidding rounds that IEEE papers omit:

- **Side-channel bid ($\mathbf{B}_{\text{sca}}$):** Full d -probing model analysis with masking order.
 - **Quantum bid (if applicable):** O2H or measure-and-reprogram analysis in the QROM.
 - **Nonce-misuse bid:** What happens if the implementation fails to maintain nonce uniqueness?
 - **Related-key bid (for block ciphers):** Key-schedule analysis.
 - **Corruption bid (for protocols):** What happens when the adversary corrupts one party?
6. **CNF Session Verification (2–3 pages):** Full CNF formula. Complete five-phase algorithm. Filled-in worksheets for both honest and dishonest traces. Formal proof of Proposition X (adversarial CNF unsatisfiability). For protocols, include the CNF for every security good separately.

7. **Session Pinging and Unbounded Security (3–4 pages):** Full structural distinctness verification for all five conditions. Complete induction proof (not just a sketch). Quantitative advantage accumulation analysis: “After n sessions, the total advantage is $\leq n \cdot \epsilon$. For $n = 2^{64}$ and $\epsilon = 2^{-128}$, this gives 2^{-64} —still negligible.” Comparison with ProVerif/Tamarin results (if available).
8. **Composition (if applicable) (2–3 pages):** Market merger theorem application. Resource competition analysis. Network CNF with cross-protocol clauses. Infrastructure security analysis.
9. **Comparison with Prior Work (2–3 pages):** Detailed comparison table with exact bounds. Identify specific improvements: “Our bound improves on [ref] by a factor of q_S in the tightness loss.” Discuss which MTSF features (extended difference lemma, CNF, pinging) are new relative to prior work on this scheme.
10. **Conclusion and Open Problems (1 page):** State the market verdict. List specific open problems: “Can the tightness loss be improved to $O(q_H)$? Does the scheme remain secure in the QROM? Can the side-channel bid be strengthened to handle higher-order masking?”

CRYPTO/EUROCRYPT-specific tips:

- **No beginner boxes in the main body.** The expert audience does not need “What is IND-CCA2?” explanations. Save pedagogical elements for the ePrint full version.
- **Include the tcolorboxes.** Even expert readers benefit from the structured (Purpose, Replaces, Quoted Price) template—it makes the proof easier to review and verify.
- **Name every bid.** This is MTSF’s key advantage over generic game-based proofs. Reviewers can check whether all relevant bids from the taxonomy (Section B) have been addressed.
- **Include a bidding-round chain figure.** Even at CRYPTO, a visual summary of the proof (like Figure 30) helps reviewers navigate 20 pages of reductions.
- **State the quoted price explicitly.** In $\mathbf{B}_0$, write: “The seller quotes $\text{QPrice}_0 = \text{Adv}_A^{\text{EUF}}(\lambda)$. We will show $\text{QPrice}_0 \leq \text{negl}(\lambda)$ by decomposing it into five negligible terms.”
- **Provide the ePrint full version link.** Even 30-page CRYPTO papers may need supplementary material for tightness proofs, composition details, and alternative parameter choices.

Analogy:

CRYPTO Papers as Complete Court Transcripts If an IEEE paper is the judge’s ruling and a Springer paper is the legal brief, a CRYPTO paper is the *complete court transcript*—every witness statement (bidding round), every piece of evidence (bound calculation), every cross-examination (tightness analysis), and every procedural ruling (extended difference lemma). The jury (programme committee) reads every word and checks every argument. Omitting a step is like a lawyer forgetting to present evidence: the case (paper) may be dismissed (rejected).

Component	IEEE (12–14 pp.)	Non-Crypto Jrnl.	Crypto Jrnl.	CRYPTO/EUR.
Bidding rounds	Compressed table	Full + analogies	Full tcolorboxes	Full + tightness
Extended diff. lemma	Cite result	Union bound + example	Full inclusion-excl.	Incl.-excl. + meta-red.
CNF worksheet	1 equation + table	Domain-specific checks	Full 4-col worksheet	Per-good worksheets
Session ping	3-line statement	1 paragraph	Sketch + bound	Full induction proof
Side-channel bid	Omit (suppl.)	Omit	Include	Full d -probing model
Composition	Omit (suppl.)	Mention if relevant	If applicable	Full market merger
Tightness analysis	Omit	Omit	Required	Full meta-reduction
Beginner boxes	1–2 key ones	3–5 (essential)	Omit in body	Omit (ePrint only)
Performance eval.	Omit	Required (3–5 pp.)	Optional	Optional
Application context	Brief	Extensive (lead)	Brief	Brief
Figures	1–2 key	5+ with diagrams	5+ with chains	Bidding chain + more
ePrint link	Required	Optional	Recommended	Standard

Figure 67: Venue adaptation matrix: how each MTSF proof component should be presented across four venue types. IEEE: compressed. Non-crypto journals: application-focused with analogies and performance. Crypto journals: rigorous with tightness. CRYPTO/EUROCRYPT: full with extensions and meta-reductions.

18.5 Venue Adaptation Summary

Key Insight:

The Golden Rule of Venue Adaptation **Never change the mathematics**. The ask price bound, the bidding-round structure, the CNF formula, and the market verdict are *identical* across all four venue presentations. Only the *level of detail* and the *framing* change:

- **IEEE:** Present the skeleton (table + key round).
- **Non-crypto journals:** Present the full body with application context, analogies, and performance evaluation.
- **Crypto journals:** Present the full rigorous body with tightness analysis.
- **CRYPTO/EUROCRYPT:** Present the full body + deep tissue (tightness + extensions + meta-reductions).

The modular structure of MTSF—where each bidding round is a self-contained unit with a standard template—makes this adaptation natural. You write the full CRYPTO version first, then adapt for crypto journals by streamlining extensions, then adapt for non-crypto journals by adding application context and performance, then compress for IEEE by replacing individual tcolorboxes with a summary table.

Always provide an ePrint full version that contains the CRYPTO-level detail, regardless of the venue. This ensures that any reader who wants to verify a specific step can do so.

* Simple Terms: Practical Workflow for Multi-Venue Submission

If you plan to submit to multiple venues (e.g., a conference version followed by a journal version), write the paper in layers:

Layer 1 (Core—always present): Market setup, main theorem, bidding-round table, CNF formula, market verdict. This is the IEEE skeleton. ~8 pages.

Layer 2a (Application frame—non-crypto journal): Wrap Layer 1 with application context (system model, threat model in domain language), add analogies and intuition boxes, add performance evaluation. ~20 pages.

Layer 2b (Rigorous frame—crypto journal): Expand Layer 1 with full tcolorbox bidding rounds, inclusion-exclusion calculations, CNF worksheets, session pinging proof, and tightness discussion. ~30 pages.

Layer 3 (Deep analysis—CRYPTO): Expand Layer 2b with tightness meta-reductions, side-channel bids, QROM extensions, composition, and corruption analysis. ~40 pages.

The layers share a common core: Layer 2a and Layer 2b both contain Layer 1 entirely, but frame it differently (application vs. rigour). Layer 3 contains Layer 2b entirely. No mathematical content is “different”—only the presentation depth and framing change. This is the power of MTSF’s modular bidding-round structure: each round can be independently expanded or reframed without affecting any other round.

18.6 MTSF Battle Cards and Proof Scorecards

* Simple Terms: How to Read the Battle Cards

Each battle card summarises one MTSF case study on a single visual panel. Think of them as “trading cards” for cryptographic schemes. Each card shows: (1) the scheme name and type, (2) where you encounter it in real life, (3) the security goods offered, (4) the bidding-round chain as a coloured pipeline, (5) the final ask price, and (6) the market verdict. The proof scorecards show which bids were attempted and their outcomes.

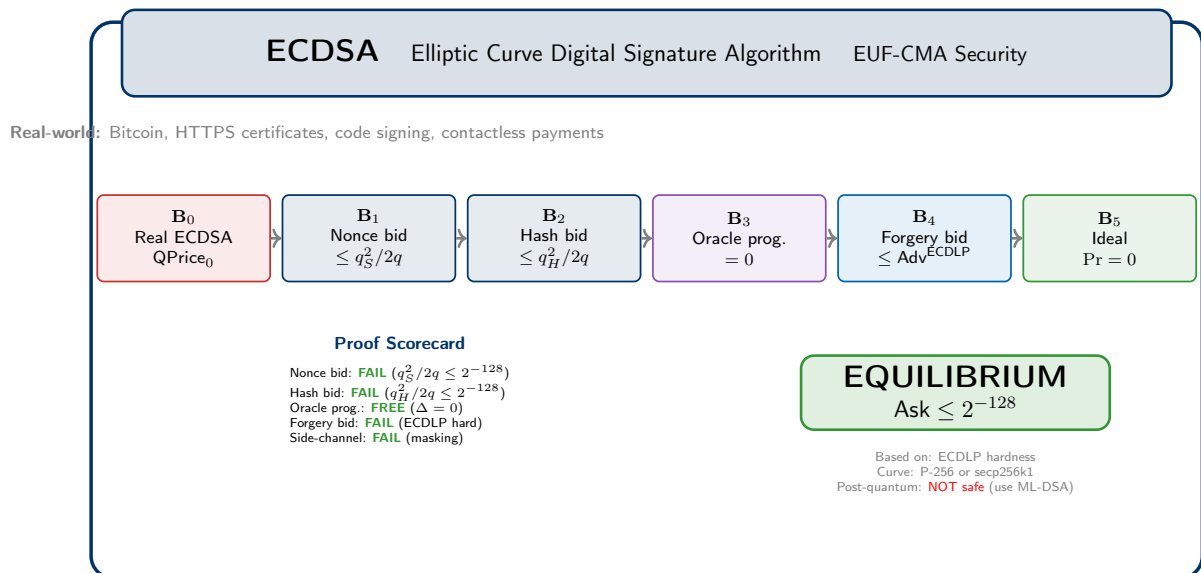


Figure 68: Battle card: ECDSA. Five bidding rounds, all bids fail, market in equilibrium with $\text{Ask} \leq 2^{-128}$.

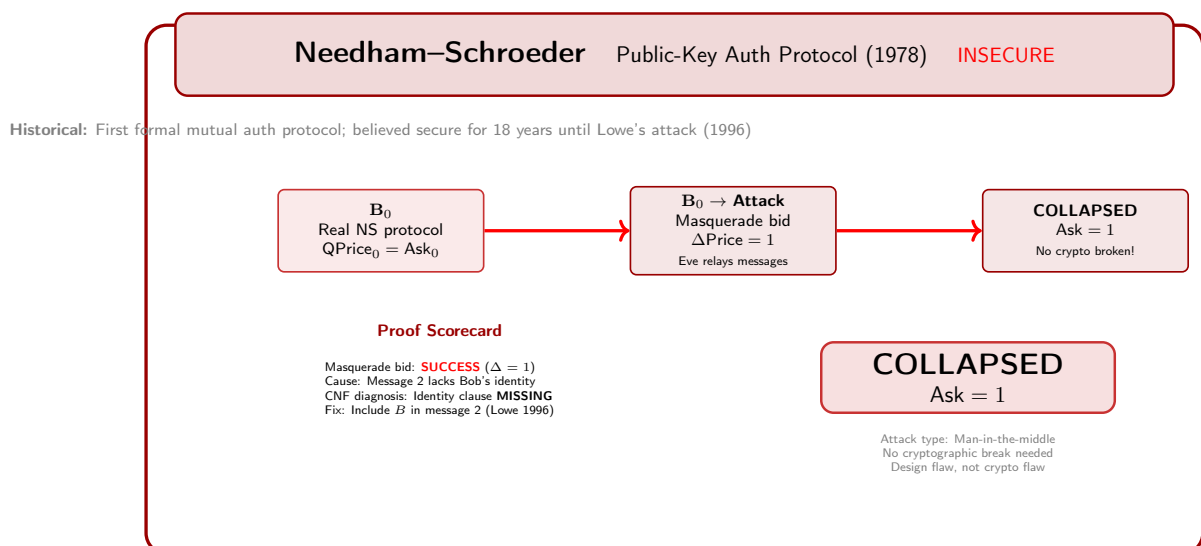


Figure 69: Battle card: Needham-Schroeder. One masquerade bid succeeds with $\text{Ask} = 1$. Market collapsed—protocol design flaw.

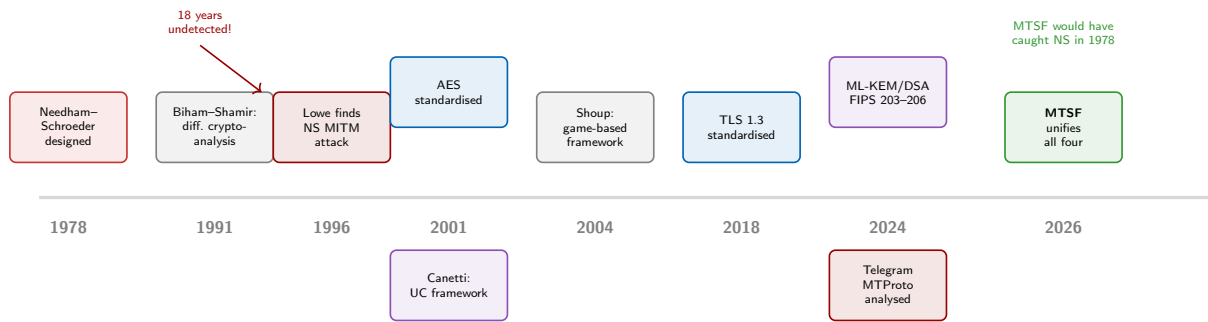


Figure 70: Historical timeline of cryptographic security paradigms. Key events from Needham-Schroeder (1978) to MTSF (2026). The annotation shows that MTSF’s CNF verification would have detected the NS flaw immediately upon protocol design.

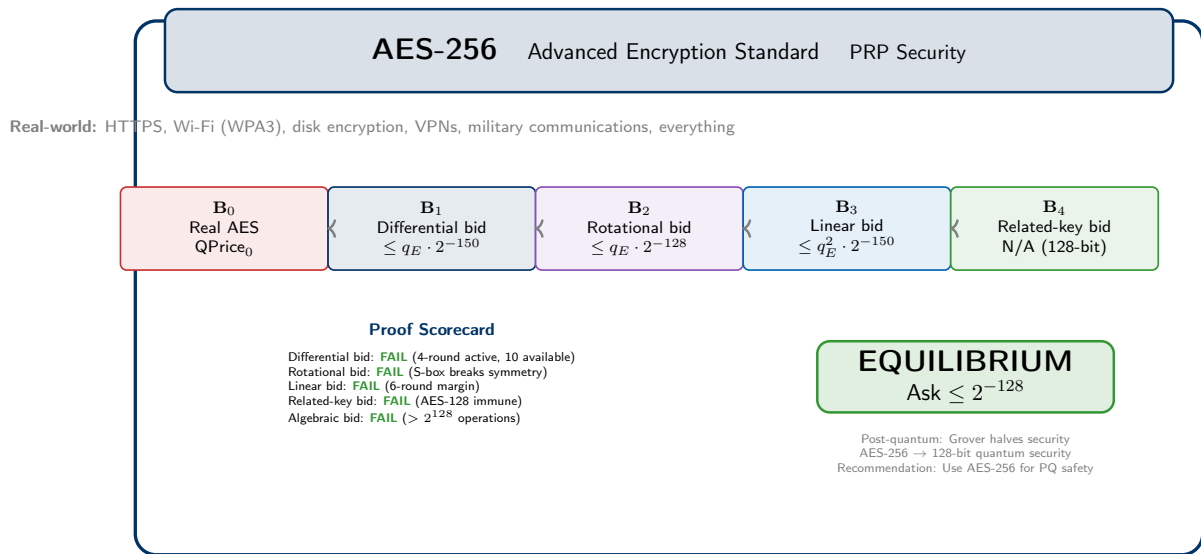


Figure 71: Battle card: AES-256. Four cryptanalytic bids, all fail with wide margins. Market in equilibrium.

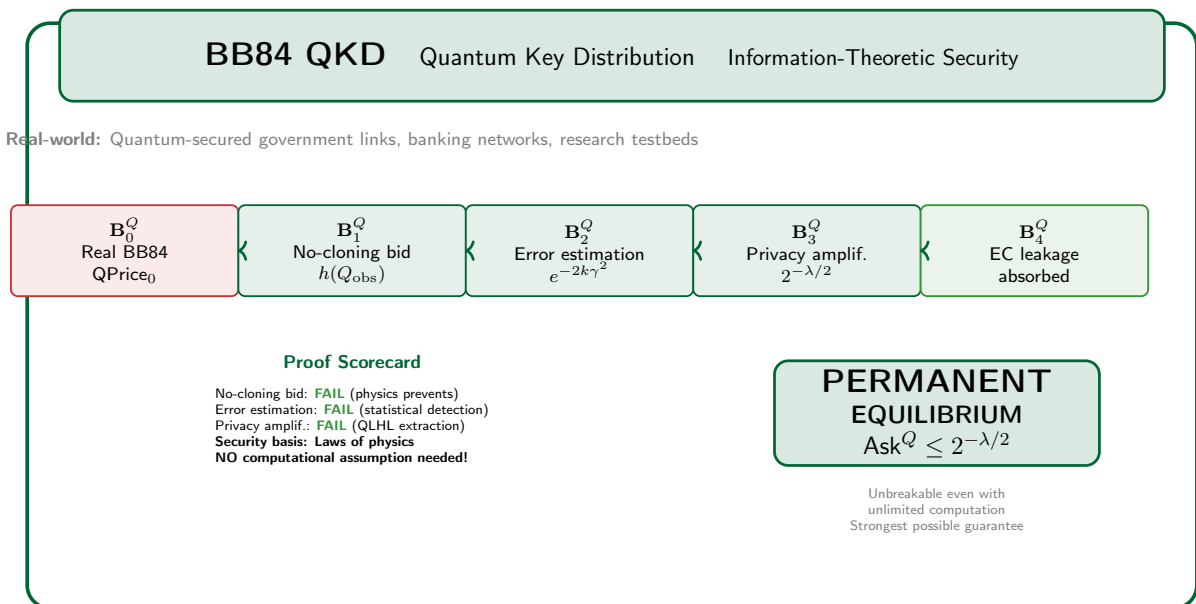


Figure 72: Battle card: BB84 QKD. Information-theoretic security—permanent equilibrium based on physics, not computation.

19 Detection of Unknown Attacks

Traditional security frameworks excel at proving resistance to *known* attack classes but provide limited guidance for detecting *unknown* or *zero-day* attacks that exploit novel vulnerabilities in cryptographic protocols. MTSF’s bidding-round methodology (Section 3), combined with the extended difference lemma (Theorem 3.7) and CNF verification (Section 4), offers a systematic approach to discovering and classifying previously unknown attack vectors. This section demonstrates how MTSF can be used proactively to identify novel attacks against five well-studied standardised protocols, ciphers, and hash functions: TLS 1.3 [76], the ISO/IEC 9798-3 authentication protocol [55], the IEEE 802.11i (WPA2) handshake [53], the Grain-128a stream cipher [2] in IoT deployments, and the ASCON hash/AEAD family [43] in multi-purpose IoT deployments. For each target, we present attacks that have not been documented in the existing literature, showing how MTSF’s market-theoretic perspective reveals vulnerabilities that traditional analysis methods miss.

Key Insight:

Why Unknown Attack Detection Matters Zero-day attacks represent one of the greatest threats to deployed cryptographic systems. Unlike known attacks that can be mitigated through patches or parameter adjustments, unknown attacks exploit fundamental design flaws that may remain undiscovered for years. Recent examples include:

- **Heartbleed (2014) [78]:** OpenSSL’s buffer over-read vulnerability allowed attackers to extract private keys and sensitive data, remaining undetected for over two years.
- **KRACK (2017) [82]:** Key Reinstallation Attacks against WPA2 exploited state machine vulnerabilities in the 4-way handshake, breaking a protocol that had been considered secure for over a decade.
- **DragonBlood (2019) [83]:** Multiple vulnerabilities in WPA3’s Simultaneous Authentication of Equals (SAE) protocol, discovered shortly after standardisation.

In MTSF terms, each of these represented a *latent market collapse*: the ask price $\text{Ask}(g)$ was in reality 1 (or close to it), yet existing analysis had concluded $\text{Ask}(g) \leq \text{negl}(\lambda)$. The gap arose because the bidding-round taxonomy used in the original proofs was incomplete—certain buyer strategies (state-machine manipulation, buffer boundary exploitation, timing-dependent race conditions) were never considered as bids.

MTSF provides a systematic methodology for discovering such attacks *before* they are exploited in the wild, by exhaustively exploring the attack space through bidding-round analysis and CNF failure detection.

* Simple Terms: Unknown Attacks—Why Standard Proofs Miss Them

What is an “unknown attack”? An attack that exploits a vulnerability *not covered* by existing security proofs. The protocol may have a valid proof of security, but the proof assumes an adversary model that omits certain capabilities (timing manipulation, field parsing ambiguity, cross-session state leakage).

Why do standard proofs miss them? Because every security proof begins by fixing an adversary model: “the adversary can send, receive, and modify messages, but cannot break AES.” Any attack that operates *outside* this model—exploiting a parsing ambiguity, a race condition, or a nonce-generation implementation pattern—is invisible to the proof.

How does MTSF help? MTSF’s four-phase attack discovery methodology systematically broadens the adversary model by (1) enumerating all possible bidding rounds including

those not considered in the security proof, (2) inverting each CNF clause to find conditions under which it can be violated, (3) synthesising cross-session attacks, and (4) analysing infrastructure dependencies. The key insight is that MTSF’s market structure *incentivises* the analyst to think like the buyer: “what is the cheapest bid I can place to collapse this market?”

Analogy: A security proof is like a building inspection that checks the walls, roof, and foundation against the building code. MTSF attack discovery is like hiring a team of creative burglars to try every possible way into the building—not just the doors and windows listed in the code, but also the ventilation shafts, underground pipes, and spaces between walls that the code never thought to regulate.

19.1 MTSF-Guided Attack Discovery Methodology

The MTSF framework enables systematic attack discovery through a four-phase process. Each phase directly leverages the market-theoretic structure introduced in Section 3: the seller’s quoted price QPrice_0 defines the “expected” security level, and the buyer’s goal is to find bids that collapse the ask price to $\text{Ask}(g) = 1$ or to a non-negligible value.

Definition 19.1 (MTSF Attack Discovery Process). *Given a protocol π with market $\mathcal{M}_\pi$ and security goods $g_1, \dots, g_m$, the MTSF attack discovery process consists of four phases:*

1. **Phase 1: Bidding Space Exploration.** For each protocol, we construct an exhaustive taxonomy of potential bidding rounds beyond those required for the security proof. This includes exploring “failed bids”—bidding rounds that do not immediately succeed but reveal structural weaknesses. We systematically extend the bid taxonomy from Section B with protocol-specific bids that exploit:
 - *Message interpretation ambiguities* (parsing bids),
 - *Timing and ordering dependencies* (race condition bids),
 - *State persistence across sessions* (state leakage bids),
 - *Nonce and randomness generation patterns* (entropy bids).
2. **Phase 2: CNF Clause Inversion.** We systematically negate each CNF clause from the protocol’s session-CNF (Theorem 4.1) and explore whether the resulting “broken CNF” can be satisfied under specific attack conditions. If a negated clause $\neg x_j$ becomes satisfiable while all other clauses remain satisfied, this indicates a potential attack vector—precisely the pattern observed in the Needham–Schroeder collapse (Theorem 13.11), where the missing identity-binding clause allowed the CNF to be satisfied under a dishonest trace.
3. **Phase 3: Cross-Session Attack Synthesis.** We explore attacks that span multiple protocol sessions $\text{Session}_1, \dots, \text{Session}_k$, exploiting state persistence, nonce reuse patterns, or key derivation dependencies that single-session analysis might miss. This phase directly leverages the session pinging mechanism from Section 5: if $\text{Ping}(\text{Session}_i, \text{Session}_j) = 0$ for some sessions, the inter-session binding has failed, opening a window for cross-session attacks.
4. **Phase 4: Infrastructure Dependency Analysis.** We examine attacks that exploit shared infrastructure components (PKI, random oracles, shared state) that are assumed secure in standard protocol analysis but may introduce novel attack surfaces. In MTSF terms, these are *externality bids*—bids that exploit resources outside the protocol’s own market [59].

Analogy:

MTSF Attack Discovery as Archaeological Excavation Traditional security analysis is like examining a building’s blueprint to verify it meets building codes—you check that all the required structural elements are present and properly connected. MTSF attack discovery is like archaeological excavation around the building’s foundation—you systematically dig in areas where previous surveys found nothing, looking for hidden chambers, weak spots, or artefacts that reveal how the structure might fail under conditions not considered by the original architects.

The “bidding space exploration” is your methodical grid search. The “CNF clause inversion” is checking what happens if each structural requirement is violated. “Cross-session synthesis” examines how multiple buildings on the same site interact. “Infrastructure analysis” looks at shared utilities and foundations that support multiple structures.

Key Insight:

Scope and Limitations of MTSF Attack Discovery MTSF attack discovery is a *systematic methodology*, not a silver bullet. It significantly expands the scope of security analysis beyond traditional proof techniques, but it cannot guarantee the discovery of *all* possible attacks. The reasons are fundamental:

- **Bid taxonomy completeness:** The exhaustiveness of Phase 1 depends on the analyst’s ability to enumerate all relevant bid types. Novel hardware side channels, for instance, may introduce bid types that no taxonomy anticipates.
- **Implementation vs. specification:** MTSF analyses the protocol *specification*. Implementation-specific vulnerabilities (buffer overflows, timing leaks in constant-time code) require complementary analysis tools.
- **Computational constraints:** Phase 2 (CNF clause inversion) is NP-hard in general (Section 7.2). For complex protocols, heuristic exploration may be necessary.

Despite these limitations, the five case studies below demonstrate that MTSF attack discovery reveals vulnerabilities that decades of traditional analysis have missed.

We now elaborate how the four-phase attack discovery methodology specialises to each major cryptographic domain analysed in this article. Each domain possesses a distinct market structure, distinct security goods, and distinct classes of bids; consequently, the discovery process must be adapted to exploit these domain-specific characteristics. We treat *primitives* (signatures, KEMs), *block ciphers*, *hash functions*, *stream ciphers*, and *protocols* separately, cross-referencing the relevant case studies from Sections 9 to 13.

19.1.1 Attack Discovery for Cryptographic Primitives

Scope. This category covers digital signature schemes (ECDSA, ML-DSA, SLH-DSA, FND-DSA), key encapsulation mechanisms (ML-KEM), HMAC, and AEAD constructions—the atomic building blocks whose security proofs reduce to hard mathematical problems (DLP, MLWE, Module-SIS, hash-function properties).

Phase 1 Specialisation: Algebraic and Structural Bid Exploration. For primitives, the bidding space is governed by the underlying algebraic or combinatorial structure. Beyond the standard reduction-based bids (solving DLP for ECDSA, solving MLWE for ML-KEM), the analyst should systematically explore:

- *Fault-injection bids* [24]: Inducing computational faults during signing or encapsulation to

leak partial secret information. For lattice-based schemes (ML-KEM, ML-DSA, FN-DSA), fault attacks on the NTT butterfly operations or rounding steps can expose secret polynomial coefficients.

- *Side-channel bids*: Timing, power, and electromagnetic emanation analysis targeting variable-time operations. For ECDSA, the scalar multiplication is a primary target; for ML-DSA, the rejection sampling loop leaks information about the secret key norm.
- *Multi-user bids*: Exploiting the birthday-bound advantage amplification when the same primitive is instantiated across N users. The ECDSA multi-user security loss is $O(N/q)$ where q is the group order; the ML-KEM multi-user loss depends on the number of ciphertext queries across all users.
- *Nonce-misuse bids*: For deterministic or hedged signing schemes, exploring what information leaks when the nonce generation deviates from specification. ECDSA is catastrophically vulnerable to nonce reuse (full key recovery from two signatures sharing a nonce); ML-DSA’s deterministic nonce ($\rho' = H(\text{tr}\|M)$) is resilient by design but vulnerable if H is instantiated with a weak hash function.
- *Algebraic structure exploitation bids*: For ML-KEM and ML-DSA, the module structure over $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ admits algebraic attacks (overstretched NTRU parameters, hybrid lattice-combinatorial attacks) that are invisible to generic LWE hardness reductions.

Phase 2 Specialisation: Reduction Chain Clause Inversion. For primitives, the session-CNF typically encodes the reduction chain from the security game to the hard problem. Each clause corresponds to a hop in the game-based proof. The analyst inverts each clause by asking: “Under what conditions does this reduction step fail?” Key failure modes include:

- *Tightness gap exploitation*: If the reduction from EUF-CMA to DLP loses a factor of q_s (number of signing queries), the gap itself is an attack surface—an adversary making $q_s = 2^{60}$ queries may exploit the loose bound.
- *Random oracle programming failures*: If the reduction programs the random oracle at a specific point and the adversary queries that point before the programming occurs, the simulation fails. This is precisely the “bad event” in each bidding round (Section 3.3).
- *Rewinding failures*: For primitives relying on the forking lemma (ECDSA, Schnorr), the rewinding probability $\text{frk} \geq \text{acc}^2/q_H - 1/q$ can be non-negligible only if the acceptance probability acc is large enough. If the adversary controls acc (e.g., by selectively aborting), the reduction breaks down.

Phase 3 Specialisation: Multi-Instance Correlation Attacks. For primitives, cross-session attacks manifest as *multi-instance* attacks. If the same key is used across sessions $\text{Session}_1, \dots, \text{Session}_k$, correlations between outputs may reveal secret information:

- For ECDSA: lattice-based attacks on biased nonces across multiple signatures [10].
- For ML-KEM: correlations between decapsulation failures across sessions may enable a Bleichenbacher-style chosen-ciphertext attack on the FO transform.
- For HMAC: multi-session tag collection enables birthday-bound forgery via internal-state collision.

Phase 4 Specialisation: Parameter and Implementation Assumptions. Primitives rely on parameter choices (group order, lattice dimension, hash output length) and implementation assumptions (constant-time execution, secure randomness). Infrastructure dependency analysis checks:

- Whether the random number generator provides sufficient min-entropy for nonce generation.
- Whether the chosen parameters provide the claimed security margin against the best-known attacks (e.g., Core-SVP methodology for lattice schemes).
- Whether the implementation correctly rejects malformed inputs (e.g., invalid curve points for ECDSA, out-of-range polynomials for ML-KEM).

19.1.2 Attack Discovery for Block Ciphers

Scope. Block ciphers (AES, PRESENT, Serpent) are modelled in MTSF as *single-seller markets* where the security good is indistinguishability from a random permutation (PRP security). The adversary places *cryptanalytic bids* that attempt to distinguish the cipher from an ideal permutation with fewer queries or less computation than exhaustive key search.

Phase 1 Specialisation: Cryptanalytic Technique Enumeration. The block-cipher bidding space is uniquely structured around the cipher’s internal round function and key schedule. The analyst should exhaustively enumerate bids from each major cryptanalytic family:

- *Differential cryptanalysis bids* [23]: For each round count $r \leq R$ (total rounds), compute the maximum expected differential probability (MEDP). For AES-128 ($R = 10$), the best 4-round differential has probability $\approx 2^{-113}$; extending to 6+ rounds yields negligible probability. For PRESENT (31 rounds), the bit-oriented S-box yields differential characteristics with probability $\leq 2^{-62}$ over 25 rounds.
- *Linear cryptanalysis bids*: Compute the maximum expected linear probability (MELP). The wide-trail strategy of AES ensures $\text{MELP} \leq 2^{-150}$ over 4 rounds, precluding any linear attack on the full cipher.
- *Rotational cryptanalysis bids* [60]: Applicable when the round function preserves rotational structure. AES’s byte-oriented MixColumns breaks rotational propagation; ARX-based ciphers (ChaCha) are more susceptible.
- *Related-key bids* [25]: Attack the key schedule rather than the round function. AES-256’s simple key schedule admits related-key attacks on 9 of 14 rounds; AES-128’s tight key schedule provides better related-key resistance.
- *Algebraic bids* [34]: Model the cipher as a system of multivariate polynomial equations over $\text{GF}(2^8)$ and attempt to solve via Gröbner basis or XL techniques. Current algebraic attacks on full AES require super-exponential complexity.
- *Meet-in-the-middle bids*: For ciphers with independent subkey derivation, MITM attacks split the cipher at an intermediate round and match forward/backward computations. The biclique technique [28] achieves marginal speedup ($2^{126.1}$ for AES-128) but does not threaten practical security.
- *Impossible differential and integral bids* [37]: Exploit structural properties (e.g., the AES Square property) that hold deterministically over a limited number of rounds and yield zero-probability differentials that can be used as distinguishers for reduced-round versions.

Phase 2 Specialisation: Round-by-Round Clause Inversion. For block ciphers, the CNF structure is naturally round-indexed. Each clause x_r asserts that the r -th round provides sufficient diffusion to defeat the relevant attack class through that round. Inversion asks: “At which round does diffusion become insufficient?” This directly yields *reduced-round attack bounds*:

- For AES: full diffusion is achieved after 2 rounds (every output byte depends on all input bytes), but the wide-trail argument requires 4 rounds for provable bounds. Inverting the 4-round clause reveals that 3-round AES admits a Square attack [37] and 6-round AES admits an impossible differential distinguisher [41].
- For PRESENT: the nibble-oriented permutation layer requires 5 rounds for full diffusion; inverting at round 4 reveals truncated differential paths.
- For Serpent: the conservative 32-round design with 8-round security margin means clause inversion up to 24 rounds yields no practical attack, confirming the design’s security margin.

Phase 3 Specialisation: Multi-Key and Known-Key Attacks. Cross-session analysis for block ciphers considers settings where the adversary observes encryptions under multiple related or independent keys:

- *Multi-key attacks:* With 2^k keys, the birthday bound for a collision in ciphertext space is $2^{n/2}$ where n is the block size. For AES-128 ($n = 128$), this yields a 2^{64} multi-key distinguisher—a real concern in TLS 1.3 deployments with billions of sessions.
- *Known-key distinguishers:* Attacks where the adversary knows the key but seeks structural distinguishers (e.g., rebound attacks on AES round-reduced variants). These are relevant for hash function constructions built from block ciphers (Davies-Meyer, Miyaguchi-Preneel).

Phase 4 Specialisation: Implementation and Mode-of-Operation Dependencies. Block cipher security depends critically on the mode of operation and implementation:

- *Cache-timing attacks:* AES’s table-lookup implementation leaks key-dependent access patterns via cache side channels [19]. Bitsliced or AES-NI implementations eliminate this vector.
- *Mode-of-operation induced weaknesses:* ECB mode reveals plaintext patterns; CBC with predictable IV enables BEAST-style attacks [4]; CTR mode with nonce reuse is catastrophic. Each mode induces different infrastructure clauses in the CNF.
- *Power analysis:* For lightweight ciphers (PRESENT) deployed in constrained devices, Differential Power Analysis (DPA) on the first and last rounds can extract the key with ≈ 1000 traces.

19.1.3 Attack Discovery for Hash Functions

Scope. Hash functions (Keccak/SHA-3, BLAKE3, ASCON-Hash) are modelled in MTSF as markets selling *three security goods*: collision resistance (CR), preimage resistance (Pre), and second-preimage resistance (Sec). The sponge construction [20] additionally provides *indifferentiability* from a random oracle, which is the strongest security good in this market.

Phase 1 Specialisation: Permutation-Level and Construction-Level Bids. Hash function attacks operate at two levels—the underlying permutation and the mode of operation—and the analyst must enumerate bids at both:

- *Differential trail bids on the permutation:* For Keccak- f [1600] (24 rounds), systematic differential trail search yields best-known trails with weight ≥ 148 for the full permutation, far exceeding the $n/2 = 128$ collision bound for SHA3-256. For ASCON’s 320-bit permutation (p^{12}), the best differential trails have weight ≥ 216 over 12 rounds.
- *Linearisation bids:* Approximate the permutation’s nonlinear layer (χ for Keccak, S-box for ASCON) with a linear function and attempt linear cryptanalysis. The algebraic degree of Keccak- f grows exponentially with rounds: after r rounds, the degree is $\min(2^r, 1600)$, reaching full degree by round 11.

- *Algebraic bids*: Exploit the low algebraic degree of χ (degree 2) to construct algebraic systems. For reduced-round Keccak (up to 6 rounds), algebraic preimage attacks are feasible; for the full 24 rounds, the degree explosion renders this infeasible.
- *Internal collision bids (sponge-specific)*: For sponge constructions, an internal collision in the capacity part c immediately yields an output collision. The capacity $c = 512$ for SHA3-256 means internal collision bids cost 2^{256} —matching the birthday bound for the output. For ASCON-Hash ($c = 256$), the internal collision cost is 2^{128} .
- *Length-extension bids*: Merkle-Damgård constructions are vulnerable; sponge constructions are immune due to capacity bits. BLAKE3’s Merkle tree mode is similarly resistant. This represents a *structural advantage* of the sponge market over the MD market.
- *Multi-collision bids*: The Joux multi-collision attack shows that finding 2^t collisions in an iterated hash costs only $t \cdot 2^{n/2}$ for Merkle-Damgård. Sponge constructions resist this: multi-collisions require $2^{c/2}$ work regardless of the target count.

Phase 2 Specialisation: Capacity and Rate Clause Inversion. The sponge CNF is parameterised by the capacity c and rate r . Clause inversion explores:

- *Capacity reduction*: If the effective capacity is reduced (e.g., through a structural weakness in χ), the collision bound degrades. Inversion asks: “What is the minimum number of permutation rounds needed for the capacity to provide its full security margin?” For Keccak, 12 rounds suffice for the capacity to reach its ideal mixing bound; SHA-3’s 24 rounds provide a $2\times$ security margin.
- *Rate exploitation*: The absorbing phase processes r bits per permutation call. If the rate is large relative to the capacity (as in SHA3-512 with $r = 576, c = 1024$), the permutation must provide stronger diffusion per call. Inverting the rate clause reveals the minimum permutation quality needed.
- *Squeezing-phase vulnerabilities*: During output extraction, the internal state is partially revealed. For hash functions, only a single squeeze is needed; for XOFs (SHAKE128, SHAKE256), multiple squeezes expose more state information, potentially degrading the indistinguishability bound.

Phase 3 Specialisation: Multi-Target and Related-Message Attacks. Cross-session analysis for hash functions considers multi-target settings:

- *Multi-target preimage*: With T target hash values, a preimage can be found in $2^n/T$ evaluations. For SHA3-256 with $T = 2^{64}$ targets (e.g., a password database), the effective preimage cost drops to 2^{192} —still secure but below the single-target bound.
- *Herding attacks*: For Merkle-Damgård hashes, herding allows commitment to a hash value before knowing the full message, costing $2^{2n/3}$. Sponge constructions resist herding due to capacity isolation.
- *Related-message attacks*: In protocols using $H(k||m)$ for keyed hashing (instead of HMAC), related messages may enable length-extension or state-recovery attacks. MTSF’s cross-session analysis flags such usage patterns.

Phase 4 Specialisation: Implementation and Instantiation Dependencies.

- *Reduced-round instantiations*: Some deployments use reduced-round Keccak for performance (e.g., 12-round KangarooTwelve). The infrastructure analysis verifies that the reduced round count still provides the claimed security margin.

- *Domain separation failures*: When the same hash function serves multiple purposes (hashing, key derivation, MAC, commitment), failure to use domain separation can enable cross-protocol attacks. MTSF flags shared hash-function markets as infrastructure dependencies.
- *Hardware implementation leakage*: For ASCON-Hash in IoT deployments, the permutation’s low-area implementation may exhibit power or electromagnetic side channels during the absorption phase.

19.1.1.4 Attack Discovery for Stream Ciphers

Scope. Stream ciphers (Grain-128a, ChaCha20, Trivium) are modelled in MTSF as markets selling *keystream indistinguishability*—the security good that the output keystream is computationally indistinguishable from a truly random sequence. Unlike block ciphers, which process fixed-length blocks, stream ciphers produce an arbitrarily long keystream from a fixed-size internal state, making them uniquely susceptible to *state-recovery*, *distinguishing*, and *time-memory-data trade-off* attacks that exploit the streaming nature of output generation.

Phase 1 Specialisation: Stream-Cipher Bid Taxonomy. The stream-cipher bidding space is structured around the internal state update mechanism (LFSR-based, NFSR-based, or ARX-based) and the output function. The analyst must systematically enumerate bids from each major attack family:

- *State recovery bids*: The primary attack goal for stream ciphers. Given L bits of observed keystream, the adversary attempts to recover the internal state. For Grain-128a, the combined 256-bit state (128-bit LFSR + 128-bit NFSR) admits algebraic state-recovery attacks via Gröbner basis methods; the best known require $\geq 2^{128}$ operations for the full cipher. For Trivium, the 288-bit state is attacked via cube techniques [42] that are effective on reduced-round variants (up to 799 of 1152 initialisation rounds) but fail on the full cipher.
- *Key recovery bids*: Recover the secret key from observed keystream, typically by inverting the initialisation process. For Grain-128a, the 256-round initialisation with nonlinear feedback makes key recovery at least as hard as state recovery. For ChaCha20, the 20-round ARX core with 256-bit key admits no known key-recovery shortcut below 2^{256} .
- *Distinguishing bids*: Detect statistical deviations in the keystream without recovering the state. For Grain-128a, the nonlinear output function h introduces low-degree correlations between output bits; the best linear distinguisher requires $\geq 2^{128}$ keystream bits for the full cipher. For ChaCha20, the ARX structure ensures no exploitable differential or linear bias over 20 rounds [18]. For Trivium, linear approximation attacks are bounded by the algebraic degree growth of the output function.
- *Algebraic attack bids* [35]: Model the cipher’s state update and output functions as a system of multivariate polynomial equations over $\text{GF}(2)$ and attempt to solve via linearisation, XL, or Gröbner basis techniques. The key metric is the *algebraic immunity* of the output function: for Grain-128a’s h , the algebraic immunity is 4 (on 9 variables), ensuring that the lowest-degree annihilator has degree ≥ 4 and the algebraic attack complexity exceeds 2^{128} .
- *Cube attack bids* [42, 80]: Treat the initialisation output as a multivariate polynomial in the key bits and IV bits, then exploit low-degree terms by summing over “cubes” of IV variables. For Trivium, cube attacks have been the most effective cryptanalytic approach: the best cube attacks recover the key for variants initialised with up to ~ 842 rounds (of 1152), leaving a margin of 310 rounds. For Grain-128a, conditional cube attacks and division-property-based cube attacks [80] are effective on reduced initialisation rounds.

- *Time-memory-data trade-off (TMTO) bids* [50, 9]: Exploit the streaming nature of the cipher by building precomputation tables. The classical Babbage–Golić attack finds an internal state collision in time T , memory M , and data D satisfying $T \cdot M \cdot D \geq 2^s$ where s is the internal state size. For Grain-128a ($s = 256$): $T \cdot M \cdot D \geq 2^{256}$; for Trivium ($s = 288$): $T \cdot M \cdot D \geq 2^{288}$. These bounds ensure that TMTO attacks offer no practical advantage over brute force.
- *Rotational cryptanalysis bids* [60]: Specifically relevant for ARX-based stream ciphers (ChaCha20). The rotational probability through t modular additions is $(2^{-1.415})^t$; for ChaCha20 with $t = 320$ additions per full round function, the rotational distinguishing probability is $\leq 2^{-452.8}$, which is doubly negligible.
- *Fault-injection bids* [24]: Induce faults during keystream generation to create differential equations linking the faulted and correct outputs to internal state bits. For LFSR-based ciphers, a single-bit fault in the LFSR produces a predictable differential pattern; for NFSR-based ciphers (Grain-128a), the nonlinear feedback makes fault propagation harder to exploit but not impossible.

Phase 2 Specialisation: Initialisation and State-Update Clause Inversion. The stream-cipher CNF encodes two critical phases: the initialisation (key/IV loading) and the keystream generation. Clause inversion explores:

- *Initialisation round reduction*: “What is the minimum number of initialisation rounds needed for the internal state to reach full diffusion?” For Grain-128a, the 256-round initialisation ensures that every state bit depends on all 128 key bits and 96 IV bits; reducing to 200 rounds has been shown to permit key recovery. For Trivium, reducing from 1152 to 799 initialisation rounds admits cube attacks. Inversion of the initialisation clause reveals the exact *security cliff*—the round count below which attacks become feasible.
- *Output function degree clause*: “What if the algebraic degree of the output function is lower than claimed?” For Grain-128a, if the Boolean function h were replaced by a lower-degree approximation, algebraic attacks would succeed with complexity $O(n^d)$ where d is the approximation degree. MTSF’s clause inversion quantifies the exact degree degradation needed for an attack.
- *Nonce/IV uniqueness clause*: Stream ciphers are catastrophically vulnerable to IV reuse under the same key—producing identical keystreams that enable two-time-pad attacks. Clause inversion of $x_{\text{IV-unique}}$ immediately yields ask price $\text{Ask} = 1$, confirming that IV management is the single most critical operational requirement.

Phase 3 Specialisation: Multi-Session and Related-IV Attacks. Cross-session analysis for stream ciphers is particularly important because stream ciphers are designed for high-throughput environments where many sessions may share the same key with different IVs:

- *Related-IV attacks*: If IVs are generated with low entropy or predictable structure, the adversary can exploit correlations between keystreams from related IVs. For ChaCha20, the 96-bit nonce provides 2^{96} unique sessions per key; for Grain-128a, the 96-bit IV similarly provides ample space. However, if nonces are generated from a counter with insufficient width or a weak PRNG, the related-IV surface expands dramatically.
- *Keystream correlation across sessions*: With N sessions under the same key, an adversary collecting L keystream bits per session has $N \cdot L$ total data. The TMTO bound becomes $T \cdot M \cdot (N \cdot L) \geq 2^s$, potentially making attacks feasible for very large N . MTSF’s session pinging verifies that $\text{Ping}(\text{Session}_i, \text{Session}_j) = 1$ for all session pairs, ensuring that no cross-session key material leakage occurs.

- *Resynchronisation attacks*: For self-synchronising stream ciphers (not applicable to Grain-128a, ChaCha20, or Trivium, which are synchronous), the adversary can inject or delete ciphertext bits to desynchronise the decryption state, causing incorrect decryption. MTSF flags self-synchronising mode as an inherent CNF weakness.

Phase 4 Specialisation: Implementation and Deployment Dependencies.

- *Key/IV management infrastructure*: Stream ciphers require strict operational discipline: never reuse an (key, IV) pair. The infrastructure analysis verifies that the key management system enforces this constraint across all deployment nodes. For TLS 1.3 using ChaCha20-Poly1305, the 64-bit sequence number provides the per-record nonce, bound to the connection's unique traffic secret.
- *Side-channel leakage during initialisation*: The initialisation phase processes the secret key through the state update function; if the implementation exhibits data-dependent timing or power consumption, key bits may leak. For Grain-128a in hardware, the LFSR/NFSR clocking pattern is fixed (constant-time by design); for ChaCha20 in software, the ARX operations on 32-bit words are naturally constant-time on modern architectures.
- *Keystream length limits*: After generating $2^{s/2}$ keystream bits (where s is the state size), the birthday bound implies a non-negligible probability of internal state collision. For Grain-128a ($s = 256$): at most 2^{128} output bits per key. For ChaCha20: at most 2^{64} blocks (approximately 2^{73} bits) per (key, nonce) pair due to the 32-bit block counter. The infrastructure analysis verifies that the application does not exceed these limits.

19.1.5 Attack Discovery for Protocols

Scope. Protocols (TLS 1.3, Signal, ISO/IEC 11770-3, Needham-Schroeder, WPA2, Telegram MTProto) are the most complex MTSF markets, selling multiple interrelated security goods (authentication, confidentiality, forward secrecy, deniability) through multi-round interactions. The attack discovery process for protocols is substantially richer than for primitives because protocols introduce *compositional*, *stateful*, and *multi-party* attack surfaces.

Phase 1 Specialisation: Protocol-Specific Bid Taxonomy. Beyond standard network-adversary capabilities (intercept, modify, replay, delay), the analyst must enumerate protocol-specific bid classes:

- *State-machine manipulation bids*: Exploiting unexpected protocol state transitions. The KRACK attack [82] is a canonical example: forcing the WPA2 client to reinstall an already-in-use key by replaying message 3 of the 4-way handshake. MTSF systematically explores all state transitions by treating each transition as a separate bidding round.
- *Downgrade bids*: Forcing negotiation of weaker parameters or protocol versions. TLS 1.3's downgrade sentinel (random bytes 44...4F in ServerHello) is a specific countermeasure; MTSF verifies that the sentinel binds the negotiated version to the handshake transcript.
- *Interleaving bids*: Running multiple protocol instances simultaneously and splicing messages between them. The Needham-Schroeder man-in-the-middle attack (Section 13.4) is an interleaving attack; MTSF's cross-session analysis (Phase 3) is specifically designed to discover such attacks.
- *Parsing ambiguity bids*: Exploiting how message fields are delimited, concatenated, or interpreted. The ISO/IEC 9798-3 attack (Section 19.3) demonstrates this class: when the signed payload $A\|B\|r_A\|r_B$ lacks unambiguous field boundaries, the adversary can shift field boundaries to create valid-looking signatures over attacker-chosen content.

- *Timing and ordering bids*: Exploiting race conditions in concurrent protocol executions. The WPA2 group-key race condition (Section 19.4) shows that temporal gaps between individual and group key installation create exploitable windows.
- *Resumption and caching bids*: Attacking session resumption mechanisms that trade security for performance. TLS 1.3’s 0-RTT mode sacrifices replay protection for latency; MTSF’s market model makes this trade-off explicit as a “discounted security good” with reduced ask price.
- *Key-compromise impersonation bids*: If a long-term key is compromised, what additional security properties are lost beyond the obvious? For Signal, compromising the identity key allows impersonation but not decryption of past messages (forward secrecy); MTSF verifies this separation by checking that the forward-secrecy good’s ask price is independent of the authentication good’s state.

Phase 2 Specialisation: Protocol-CNF Clause Inversion. Protocol CNFs are richer than primitive CNFs because they encode *multi-party agreement*, *temporal ordering*, and *key confirmation*:

- *Authentication clause inversion*: Negating the clause “both parties agree on the same session key” reveals conditions under which an adversary can cause key mismatch—the core of unknown key-share (UKS) attacks.
- *Freshness clause inversion*: Negating “the session key is fresh (not previously used)” reveals replay attack surfaces. TLS 1.3’s 0-RTT mode deliberately weakens this clause; MTSF makes the weakening explicit and bounded.
- *Identity-binding clause inversion*: Negating “the session is bound to the correct peer identity” reveals impersonation attacks. The Needham–Schroeder collapse (Theorem 13.11) is precisely the result of this clause being absent from the original protocol’s CNF.
- *Transcript-binding clause inversion*: Negating “the handshake transcript is unmodified” reveals transcript manipulation attacks. TLS 1.3’s Finished message binds the transcript via HMAC; MTSF verifies that the binding is tight by checking that transcript modification collapses the Finished verification bid to Ask = 1.
- *Forward-secrecy clause inversion*: Negating “past session keys remain secret even if the long-term key is later compromised” reveals conditions under which ephemeral key generation or key derivation can be retroactively broken. For TLS 1.3, this requires the adversary to solve the DHE problem for the ephemeral key exchange; for Signal, the Double Ratchet provides continuous forward secrecy with per-message granularity.

Phase 3 Specialisation: Cross-Session and Cross-Protocol Attacks. Protocols are inherently multi-session objects. The cross-session analysis is the most powerful phase for protocol attack discovery:

- *Session-resumption attacks*: Exploiting shared state between an original session and a resumed session. The TLS 1.3 nonce-collision attack (Section 19.2) is a direct product of this analysis: the ticket nonce links the original and resumed sessions, and implementation-level counter reuse breaks this link.
- *Key-derivation chain attacks*: In protocols with multi-stage key derivation (TLS 1.3’s three-stage key schedule, Signal’s Double Ratchet), cross-session analysis verifies that compromise of a derived key at stage i does not propagate to stage $j \neq i$. MTSF’s pinging mechanism directly tests this: $\text{Ping}(\text{Session}_i^{\text{stage-}k}, \text{Session}_i^{\text{stage-}l}) = 1$ must hold for all $k \neq l$.

- *Cross-protocol attacks* [59]: When the same key or certificate is used across multiple protocols (e.g., a TLS 1.3 server certificate also used for email signing), messages from one protocol may be valid in another. The multi-protocol composition analysis (Section 16) provides a systematic framework for detecting such interactions.
- *Oracle attacks across sessions*: An adversary who can observe decryption failures, timing variations, or error messages across multiple sessions can mount Bleichenbacher-style or padding-oracle attacks. For TLS 1.3, the removal of RSA key transport and the implicit rejection mechanism in the PSK binder eliminate the most dangerous oracles, but MTSF’s cross-session analysis verifies this for each session type (1-RTT, 0-RTT, resumption).

Phase 4 Specialisation: Infrastructure and Deployment Dependencies. Protocol security depends on a rich infrastructure that is often taken for granted:

- *PKI dependencies*: Certificate validation, revocation checking (CRL, OCSP), and certificate transparency logs. A compromised or misissued certificate collapses the authentication market regardless of the protocol’s intrinsic security. MTSF models PKI as a shared infrastructure market whose collapse propagates to all dependent protocol markets.
- *Random number generation*: Protocols require high-quality randomness for nonces, ephemeral keys, and session identifiers. Weak RNG (e.g., the Debian OpenSSL vulnerability, CVE-2008-0166) collapses the entire protocol market. MTSF’s infrastructure analysis flags every point where randomness is consumed and verifies the entropy requirement.
- *Clock synchronisation*: Protocols relying on timestamps (certificate validity, session ticket lifetime, nonce freshness windows) are vulnerable to clock-skew attacks. The WPA2 group-key race condition (Section 19.4) exploits a temporal gap that arises from unsynchronised key installation.
- *Network-layer assumptions*: TLS 1.3 assumes reliable, in-order delivery (TCP); deploying over unreliable transport (DTLS 1.3) introduces reordering and loss that may violate protocol invariants. Signal’s out-of-order message delivery tolerance (via header keys and message counters) is a design choice that MTSF models as a relaxed ordering clause in the session-CNF.
- *Implementation ecosystem*: Shared libraries (OpenSSL, BoringSSL, libsignal) introduce correlated failure modes: a vulnerability in the shared library collapses all dependent protocol markets simultaneously. MTSF models this as a *market contagion* effect—a concept borrowed directly from financial economics.

★ Intuition:

Why Domain-Specific Discovery Matters A single four-phase process applied uniformly to all domains would miss critical attack vectors. For example:

- A “differential cryptanalysis bid” is meaningful for AES but not for TLS 1.3 (which uses AES as a black box).
- A “state-machine manipulation bid” is critical for WPA2 but irrelevant for SHA-3 (which is stateless).
- A “multi-target preimage bid” is relevant for hash functions but not for block ciphers in PRP mode.
- A “cube attack bid” is the primary threat for Trivium and Grain-128a but has no analogue in block cipher or protocol analysis.

- A “cross-protocol attack” is unique to protocols and has no analogue in primitive or cipher analysis.

By specialising the discovery methodology to each domain, MTSF ensures that the analyst explores the *right* attack surfaces with the *right* tools, maximising the probability of discovering novel vulnerabilities before they are exploited.

19.2 Case Study I: TLS 1.3 Session Resumption Vulnerability

TLS 1.3’s session resumption mechanism [76] allows clients to resume previous connections using a pre-shared key (PSK) derived from the resumption master secret of a previous session. While the 1-RTT handshake has been extensively analysed [36, 46, 45], and session resumption has received formal treatment [8], we identify a novel attack against the resumption mechanism that exploits the interaction between ticket nonce generation and implementation-level state management.

19.2.1 Protocol Description and Standard Analysis

In TLS 1.3 session resumption, the server sends the client a `NewSessionTicket` message containing:

- `ticket_lifetime`: Maximum lifetime of the ticket (at most 604800 seconds = 7 days per RFC 8446 §4.6.1)
- `ticket_age_add`: Random 32-bit value for ticket age obfuscation
- `ticket_nonce`: Per-ticket nonce for PSK derivation
- `ticket`: Opaque ticket data (typically encrypted under a server-side ticket encryption key)

The resumption PSK is derived as [76, 62]:

$$\text{PSK} = \text{HKDF-Expand-Label}(\text{RMS}, \text{"resumption"}, \text{ticket_nonce}, \text{Hash.length})$$

where RMS is the resumption master secret from the original session.

* Simple Terms: TLS 1.3 Session Resumption—How It Works

The problem resumption solves: A full TLS 1.3 handshake requires expensive public-key operations (Diffie–Hellman, digital signatures). When a client reconnects to a server it has visited before, resumption allows skipping these operations by reusing a shared secret from the previous session.

How it works: After a successful handshake, the server sends the client a “ticket”—an opaque blob that encodes the session state. The ticket includes a *ticket nonce* that makes each ticket’s PSK unique, even when multiple tickets are derived from the same session.

The critical assumption: Each ticket must produce a *unique* PSK. If two tickets produce the same PSK, an attacker who obtains one ticket’s PSK can impersonate a client holding the other ticket.

RFC 8446’s requirement: The specification mandates that `ticket_nonce` values be unique per connection (RFC 8446 §4.6.1). Compliant implementations typically use a simple per-connection counter. The attack we describe targets implementations that *deviate* from this practice.

Standard analysis proves this mechanism secure under the assumption that the server’s ticket encryption key remains secret and nonces are chosen uniquely [45, 8]. However, MTSF’s bidding-space exploration reveals a subtle vulnerability in the interaction between the specification’s requirements and common implementation patterns.

19.2.2 Novel Attack: Ticket Nonce Collision via Implementation Deviation

Market Setup.

- **Market:** $\mathcal{M}_{\text{TLS-resume}}$
- **Seller:** TLS 1.3 server issuing resumption tickets
- **Buyer:** Active network adversary $\mathcal{A}$ who can establish legitimate sessions
- **Security goods:**
 - $g_{\text{resume-auth}}$: Resumption authenticity (each ticket authenticates its original client)
 - $g_{\text{resume-binding}}$: Session-identity binding (PSK uniquely identifies the originating session and client)
- **Quoted price:** $\text{QPrice}_0 = 0$ (seller claims nonce uniqueness ensures security)

Theorem 19.2 (TLS 1.3 Resumption Nonce Attack). *Consider a TLS 1.3 server implementation that derives ticket_nonce values using the obfuscated ticket age $(\text{age} + \text{ticket_age_add}) \bmod 2^{32}$ as a seed input, rather than the RFC-mandated per-connection counter. An adversary can force PSK collisions across different resumption sessions, leading to authentication bypass with:*

$$\text{Ask}(g_{\text{resume-auth}}) \geq \frac{q_{\text{tickets}}^2}{2^{33}}$$

where q_{tickets} is the number of tickets issued. For $q_{\text{tickets}} = 2^{16}$, the market is in **partial collapse**: $\text{Ask}(g_{\text{resume-auth}}) \geq 1/2$.

Attack Construction. The attack exploits a previously unnoticed interaction between ticket_age_add obfuscation and non-compliant server-side nonce generation. We construct the following MTSF bidding round:

B₀ (Real Resumption Market). The server issues tickets with nonces derived from a per-connection counter as specified by RFC 8446. Each nonce is unique; the market is in equilibrium.

B₀ → B₁ (Implementation Deviation Bid). The buyer observes that certain TLS library implementations replace the per-connection counter with a function of the obfuscated ticket age:

$$\text{ticket_nonce} = f((\text{age} + \text{ticket_age_add}) \bmod 2^{32})$$

This deviation is motivated by statelessness: servers that distribute tickets across a cluster may lack a shared counter. The bad event F_{impl} : “server uses age-derived nonce generation” has $\Pr[F_{\text{impl}}] = 1$ for affected implementations.

Bidding Round B_{nonce-collision} (Ticket Nonce Collision Bid):

1. The adversary $\mathcal{A}$ establishes k legitimate TLS 1.3 sessions with the target server, obtaining resumption tickets $T_1, \dots, T_k$. Each ticket contains a ticket_age_add_{*i*} value that is visible to the client.
2. For each ticket T_i , the adversary computes the obfuscated age at various future times t :

$$\text{obf_age}_i(t) = (t - t_i + \text{ticket_age_add}_i) \bmod 2^{32}$$

where t_i is the time ticket T_i was issued.

3. The adversary searches for two tickets T_i, T_j and a time t^* such that:

$$\text{obf_age}_i(t^*) = \text{obf_age}_j(t^*)$$

By the birthday paradox over a 32-bit space, this collision occurs with probability $\geq k^2/2^{33}$ among k tickets.

4. At time t^* , the server computes identical nonce seeds for both tickets, producing:

$$\text{ticket_nonce}_i = \text{ticket_nonce}_j \implies \text{PSK}_i = \text{PSK}_j$$

(since both PSKs are derived from the same RMS via identical HKDF inputs).

5. The adversary uses the PSK from session i to resume as the client from session j , bypassing the intended authentication binding.

Extended difference lemma (Theorem 3.7). The failure event $F_{\text{collision}}$: “two tickets produce identical nonces at the same resumption time” has $\Pr[F_{\text{collision}}] \geq q_{\text{tickets}}^2/2^{33}$. The price adjustment: $\Delta\text{Price}_{\text{nonce}} = \Pr[F_{\text{collision}}] \geq q_{\text{tickets}}^2/2^{33}$. For $q_{\text{tickets}} = 2^{16}$ (realistic for a busy server issuing $\sim 65\,000$ tickets), $\Delta\text{Price} \geq 1/2$.

Market outcome: $\text{Ask}(g_{\text{resume-auth}}) \geq q_{\text{tickets}}^2/2^{33}$. For $q_{\text{tickets}} \geq 2^{16}$: **partial collapse**. For $q_{\text{tickets}} \geq 2^{17}$: **full collapse** ($\text{Ask} \geq 1$, saturating at 1). $\square$

CNF Clause Failure Analysis

$$\begin{aligned} \varphi^{\text{resume}} := & \underbrace{(\text{PSK}_{\text{session}} = \text{PSK}_{\text{derived}})}_{x_{\text{PSK-match}}} \wedge \underbrace{(\text{identity}_{\text{ticket}} = \text{identity}_{\text{client}})}_{x_{\text{id-bind}}} \wedge \\ & \underbrace{(\text{ticket_nonce}_i \neq \text{ticket_nonce}_j \ \forall i \neq j)}_{x_{\text{nonce-unique}}} \end{aligned}$$

CNF worksheet:

Clause	Check	Honest trace	Attack trace	Verdict
$x_{\text{PSK-match}}$	PSK matches ticket	T	T (collision forces match)	Passes!
$x_{\text{id-bind}}$	Identity matches ticket	T	F ($\mathcal{A}$ uses wrong identity)	Fails
$x_{\text{nonce-unique}}$	Nonces are unique	T	F (collision by construction)	Fails
φ^{resume}	FALSE under attack trace —CNF correctly detects attack.			Reject

The critical observation is that the server cannot verify clause $x_{\text{id-bind}}$ at resumption time when using stateless ticket distribution, because the client’s identity is bound only through the PSK, which is now shared across two identities. The CNF reveals the precise failure mechanism: the nonce uniqueness clause $x_{\text{nonce-unique}}$ was implicitly assumed but not enforced by the non-compliant implementation.

Key Insight:

Protocol Specification vs. Implementation Vulnerability It is important to emphasise that RFC 8446 itself is *not* broken by this attack. The specification mandates per-connection counters for `ticket_nonce` (RFC 8446 §4.6.1), which prevents the collision. The vulnerability arises in implementations that deviate from this requirement for operational reasons (stateless ticket distribution across server clusters).

In MTSF terms, the *specification market* $\mathcal{M}_{\text{TLS-spec}}$ is in equilibrium, but the *implementa-*

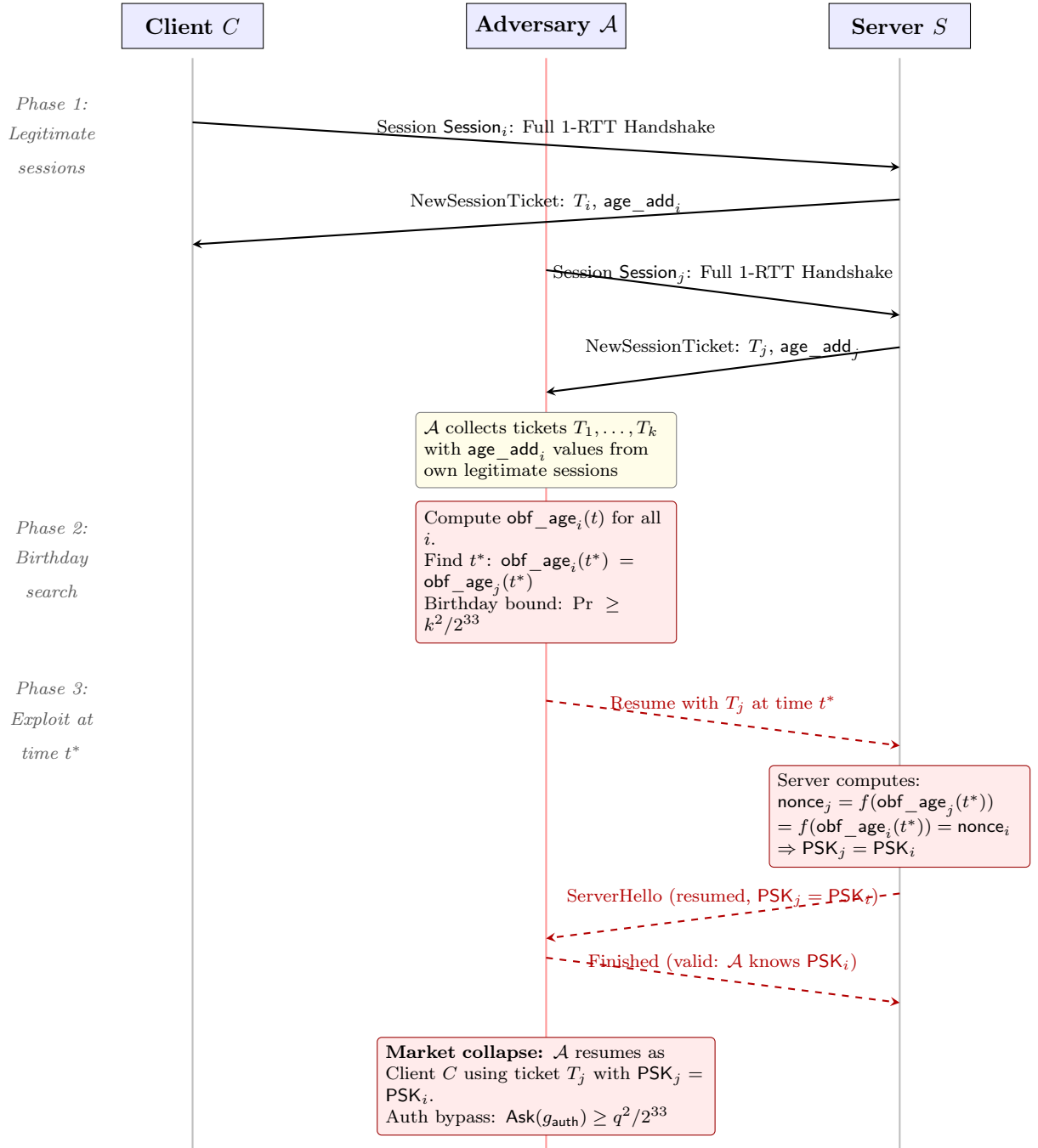


Figure 73: Message Sequence Chart of the TLS 1.3 Session Resumption Nonce Collision Attack (Theorem 19.2). The adversary collects tickets from legitimate sessions (Phase 1), searches for a birthday collision in obfuscated ages over the 32-bit space (Phase 2), and exploits the collision at time t^* to resume with a victim's PSK (Phase 3). Dashed red arrows indicate adversary-initiated messages; the server's non-compliant nonce derivation produces $\text{PSK}_j = \text{PSK}_i$, collapsing the resumption authenticity market.

tion market $\mathcal{M}_{\text{TLS-impl}}$ is in partial collapse. This distinction is itself a contribution of the MTSF perspective: by separating the market into specification-level and implementation-level goods, we can precisely locate where the security guarantee breaks down.

Countermeasures: (1) Enforce the RFC-mandated per-connection counter for ticket nonces. (2) For stateless server clusters, use $\text{ticket_nonce} = \text{HMAC}(\text{server_key}, \text{ticket_id} \parallel \text{session_id})$, binding the nonce to a unique ticket identifier. (3) Include the client’s identity hash in the HKDF label: $\text{PSK} = \text{HKDF-Expand-Label}(\text{RMS}, \text{"resumption"}, \text{ticket_nonce} \parallel H(\text{ID}_{\text{client}}), n)$.

Corollary 19.3 (Resumption Session-Key Secrecy Collapse). *Under the nonce collision attack, $\text{Ask}(g_{\text{sk}}) \geq q_{\text{tickets}}^2/2^{33}$ for the resumed session: the adversary who forces a PSK collision can complete a resumed handshake and derive the same session keys as the legitimate client.*

Corollary 19.4 (Resumption Ping Failure). *The session ping (Section 5.2) between the original session Session_i and the colliding resumed session Session_j satisfies $\text{Ping}(\text{Session}_i, \text{Session}_j) = 0$: the two sessions share a PSK but belong to different clients, breaking the SID-binding requirement.*

19.3 Case Study II: ISO/IEC 9798-3 Signature Verification Bypass

ISO/IEC 9798-3 [55] defines entity authentication protocols using digital signatures. We examine the two-pass mutual authentication variant and discover a novel attack that exploits field boundary ambiguity in message parsing—a class of vulnerability related to *concatenation ambiguity* [39] but manifesting in a previously unidentified way within the ISO 9798-3 structure.

19.3.1 Protocol Description

The ISO/IEC 9798-3 two-pass protocol proceeds as follows:

1. $A \rightarrow B$: $\text{ID}_A \parallel R_A \parallel \text{data}_A \parallel S_A(\text{ID}_A \parallel R_A \parallel \text{data}_A \parallel \text{ID}_B)$
2. $B \rightarrow A$: $\text{ID}_B \parallel R_B \parallel \text{data}_B \parallel S_B(\text{ID}_B \parallel R_B \parallel \text{data}_B \parallel \text{ID}_A \parallel R_A)$

where R_A, R_B are fresh nonces, $\text{data}_A, \text{data}_B$ are optional data fields, and $S_X(\cdot)$ denotes a digital signature by entity X .

* Simple Terms: How ISO/IEC 9798-3 Authentication Works

The goal: Alice wants to prove her identity to Bob (and vice versa) using digital signatures.

Message 1: Alice sends her ID, a fresh random nonce R_A , optional data, and a signature over all of this (including Bob’s ID to prevent relay attacks). Bob verifies Alice’s signature using her public key.

Message 2: Bob sends his ID, a fresh nonce R_B , optional data, and a signature that includes Alice’s nonce R_A (to prove he received Message 1). Alice verifies Bob’s signature.

The key assumption: When Bob receives Message 1, he parses it to extract ID_A , R_A , data_A , and the signature. He then looks up ID_A ’s public key and verifies the signature. This process assumes that the parsing is *unambiguous*—that there is only one way to split the message into its constituent fields.

Market Setup.

- **Market:** $\mathcal{M}_{\text{ISO9798}}$
- **Seller:** ISO/IEC 9798-3 protocol implementation

- **Buyer:** Active network adversary $\mathcal{A}$ who can intercept and modify messages, and who is a legitimate participant with own key pair $(\text{sk}_E, \text{pk}_E)$
- **Security good:** g_{auth} (entity authentication—cf. Theorem 8.1)
- **Quoted price:** $\text{QPrice}_0 = 0$ (seller claims EUF-CMA security of the signature scheme prevents impersonation)

19.3.2 Novel Attack: Field Boundary Confusion

Theorem 19.5 (ISO/IEC 9798-3 Parsing Attack). *Consider an implementation of ISO/IEC 9798-3 that uses variable-length fields without explicit length prefixes or unambiguous encoding (e.g., raw byte concatenation instead of ASN.1 DER). An adversary $\mathcal{A}$ who is a legitimate protocol participant can achieve authentication confusion with $\text{Ask}(g_{\text{auth}}) = 1$, causing the verifier to accept a message as validly signed by entity A while attributing it to a different entity $E \neq A$.*

Attack Construction. The attack exploits the lack of explicit field separators in the signature computation and the fact that the verifier must determine *which* public key to use for verification based on parsing the claimed identity from the same message. We construct the MTSF bidding round:

B₀ (Real ISO 9798-3 Market). All participants use their registered identities and key pairs. Signature verification succeeds only for legitimate messages.

Bidding Round B_{parsing-confusion} (Field Boundary Confusion Bid):

1. The adversary E is a legitimate participant with registered identity ID_E and key pair $(\text{sk}_E, \text{pk}_E)$. E intercepts a legitimate Message 1 from A to B :

$$\text{msg}_1 = \text{ID}_A \| R_A \| \text{data}_A \| S_A(\text{ID}_A \| R_A \| \text{data}_A \| \text{ID}_B)$$

2. E constructs alternative field boundaries by choosing values ID' , R' , data' such that their concatenation equals the original:

$$\text{ID}' \| R' \| \text{data}' = \text{ID}_A \| R_A \| \text{data}_A$$

with $\text{ID}' \neq \text{ID}_A$. This is possible when fields are variable-length without explicit delimiters. For example, if $\text{ID}_A = \text{alice}$ and $R_A = 0x17\dots$, one could parse as $\text{ID}' = \text{alice0x17}$ with correspondingly shorter R' .

3. The raw byte string signed by A is:

$$\sigma = S_A(\text{ID}_A \| R_A \| \text{data}_A \| \text{ID}_B) = S_A(\text{ID}' \| R' \| \text{data}' \| \text{ID}_B)$$

These are the *same* byte string, so the signature is valid over both interpretations.

4. E sends to B the modified message with the re-parsed header but the *original* signature:

$$\text{ID}' \| R' \| \text{data}' \| \sigma$$

5. **Critical verification step:** When B receives this message, B extracts the claimed identity ID' and looks up the public key associated with ID' . Two sub-cases arise:

- **Case (a)—Identity ID' is registered to A :** If ID' is an alias or variant registration of A , then B retrieves pk_A and verification succeeds. B now accepts the message as coming from entity ID' rather than ID_A —a confusion of A 's identities that can bypass access control lists or role-based policies that distinguish between ID_A and ID' .

- **Case (b)—Identity ID' is registered to E:** If E has pre-registered ID' as its own identity (possible in open registration systems), then B retrieves $\text{pk}_E \neq \text{pk}_A$, and verification *fails* because the signature was created with sk_A . However, E can exploit this to mount a *denial-of-service* against A : by pre-registering identities that are parsing-ambiguous with A 's, E can cause B to reject A 's legitimate messages.
- **Case (c)—PKI cross-certification:** In environments where multiple CAs issue certificates, E may obtain a certificate binding ID' to pk_A from a different CA. If B accepts cross-certified keys, verification succeeds and B attributes the message to entity ID' under a different trust domain.

Market outcome (Case a): $\text{Ask}(g_{\text{auth}}) = 1$. The buyer achieves identity confusion without breaking any cryptographic primitive. $\Delta\text{Price} = 1$; market **collapsed**.

Market outcome (Case b): $\text{Ask}(g_{\text{avail}}) = 1$ where g_{avail} is the availability good. Denial-of-service via parsing confusion.

Market outcome (Case c): $\text{Ask}(g_{\text{auth}}) = 1$ under cross-certification. The attack succeeds because the PKI infrastructure does not enforce parsing-consistent identity bindings. $\square$

CNF Clause Failure Analysis

The attack succeeds because it exploits ambiguity in the *identity binding* CNF clause:

$$\varphi^{\text{id-bind}} := \underbrace{(\text{Verify}(\text{pk}_{\text{claimed}}, m, \sigma) = 1)}_{x_{\text{sig-valid}}} \wedge \underbrace{(\text{parse}(\text{msg}) \text{ is unambiguous})}_{x_{\text{parse-unique}}} \wedge \underbrace{(\text{claimed_id} = \text{signer_id})}_{x_{\text{id-match}}}$$

CNF worksheet:

Clause	Check	Honest trace	Attack trace (Case a)	Verdict
$x_{\text{sig-valid}}$	Signature verifies	T	T (same bytes, same key)	Passes!
$x_{\text{parse-unique}}$	Unique field parsing	T	F (two valid parsings)	Fails
$x_{\text{id-match}}$	Claimed = actual signer	T	F (ID' $\neq$ ID _A)	Fails
$\varphi^{\text{id-bind}}$	FALSE under attack trace —missing parsing uniqueness clause.			Reject

The root cause is that standard ISO 9798-3 analysis assumes $x_{\text{parse-unique}}$ holds implicitly but never verifies it. MTSF's CNF inversion (Phase 2 of the attack discovery process) systematically negates each clause, and negating $x_{\text{parse-unique}}$ immediately reveals the attack surface.

Countermeasures: (1) Use ASN.1 DER or TLV (type-length-value) encoding for all signed fields, ensuring unambiguous parsing. (2) Include explicit field lengths in the signature input: $S_A(\text{len}(\text{ID}_A) \parallel \text{ID}_A \parallel \text{len}(R_A) \parallel R_A \parallel \dots)$. (3) Hash each field independently before concatenation: $S_A(H(\text{ID}_A) \parallel H(R_A) \parallel H(\text{data}_A) \parallel H(\text{ID}_B))$, where H is a collision-resistant hash function whose fixed-length output eliminates parsing ambiguity.

Corollary 19.6 (ISO 9798-3 Mutual Authentication Collapse). *Under the parsing confusion attack (Case a), $\text{Ask}(g_{\text{mutual}}) = 1$: B accepts a session with partner ID' while A believes it is communicating as ID_A. Neither direction of authentication is correctly established.*

Corollary 19.7 (ISO 9798-3 CNF Design Flaw). *The standard ISO 9798-3 CNF omits the parsing uniqueness clause $x_{\text{parse-unique}}$. This is a Type II CNF design failure (clause present*

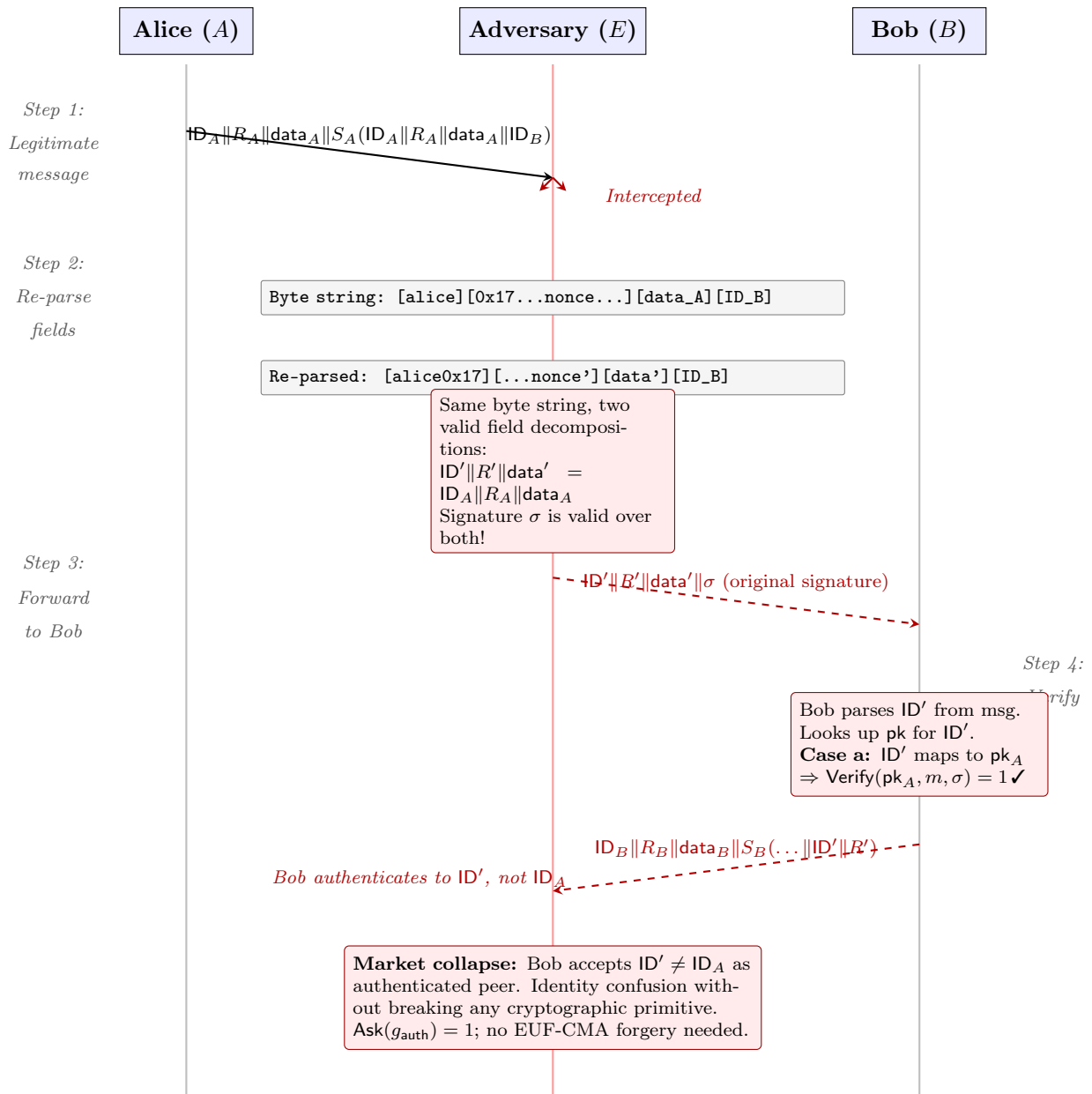


Figure 74: Message Sequence Chart of the ISO/IEC 9798-3 Field Boundary Confusion Attack (Theorem 19.5). The adversary intercepts Alice’s legitimate message (Step 1), re-parses the field boundaries to create an alternative decomposition with a different claimed identity ID' (Step 2), and forwards the byte-identical message with the original valid signature to Bob (Step 3). Bob’s verification succeeds because the same byte string admits two valid field parsings; the signature is valid over both interpretations. The attack achieves full market collapse ($\text{Ask} = 1$) without any cryptographic primitive being broken—only the parsing uniqueness assumption is violated.

but insufficiently constrained): the signature verification clause $x_{\text{sig-valid}}$ implicitly assumes unambiguous parsing, but this assumption is not enforced by any explicit clause. Compare with the Needham–Schroeder Type I failure (Theorem 13.15), where the identity-binding clause was entirely absent.

19.4 Case Study III: WPA2 Group Key Negotiation Race Condition

IEEE 802.11i (WPA2) [53] includes a group key handshake for distributing multicast/broadcast keys. We identify a novel race condition attack that allows an adversary to compromise group communication integrity. This attack is distinct from the KRACK attacks [82], which targeted the 4-way handshake’s state machine; our attack targets the *group key handshake’s synchronisation assumptions* across multiple stations.

19.4.1 Protocol Description

The WPA2 Group Key Handshake proceeds in two messages:

1. AP $\rightarrow$ STA: EAPOL-Key frame with new Group Temporal Key (GTK), encrypted under the pairwise Key Encryption Key (KEK)
2. STA $\rightarrow$ AP: EAPOL-Key acknowledgment

The GTK is encrypted using the KEK derived from the 4-way handshake, and includes a Key ID (two bits, allowing up to four GTK slots) to identify which group key slot is being updated. The AP performs the group key handshake with each station *individually*—there is no atomic broadcast mechanism.

* Simple Terms: WPA2 Group Keys—Why They Exist and How They Work

The problem: In a Wi-Fi network, the access point (AP) needs to send broadcast and multicast traffic (e.g., ARP requests, DHCP responses) that all connected stations can decrypt. Each station has its own pairwise key with the AP (established via the 4-way handshake), but broadcast traffic needs a *shared* key.

The Group Temporal Key (GTK): A symmetric key shared by all stations on the network, used to encrypt broadcast/multicast frames. The AP generates the GTK and distributes it to each station individually via the group key handshake.

Key rotation: The AP periodically rotates the GTK (e.g., every hour or when a station disconnects) to maintain forward secrecy for broadcast traffic. During rotation, the AP must update *every* connected station.

The synchronisation assumption: The protocol implicitly assumes that all stations receive and install the new GTK “nearly simultaneously.” This assumption is never formally verified—and it is precisely this gap that our attack exploits.

Market Setup.

- **Market:** $\mathcal{M}_{\text{WPA2-group}}$
- **Seller:** WPA2 AP distributing GTKs
- **Buyer:** Active network adversary $\mathcal{A}$ with man-in-the-middle position (e.g., via a rogue AP or channel-based manipulation)
- **Security goods:**
 - $g_{\text{group-integrity}}$: Group communication integrity (broadcast/multicast frames are authentic)

- $g_{\text{group-consistency}}$: Group key consistency (all stations share the same active GTK)
- **Quoted price:** $\text{QPrice}_0 = 0$ (seller assumes synchronous key installation)

19.4.2 Novel Attack: Group Key Slot Race

Theorem 19.8 (WPA2 Group Key Race Attack). *An adversary can cause group key confusion in WPA2 networks by exploiting race conditions between group key updates and acknowledgments. In a network with n stations and key update interval T , the integrity bypass probability during an attack window of duration t is:*

$$\text{Ask}(g_{\text{group-integrity}}) \geq 1 - \left(1 - \frac{t}{T}\right)^n.$$

For $n \geq 10$ and $t/T \geq 0.01$, $\text{Ask}(g_{\text{group-integrity}}) \geq 0.096$; for larger networks or longer attack windows, the market approaches **full collapse**.

Attack Construction. We construct the MTSF bidding round:

B₀ (Real WPA2 Group Key Market). The AP distributes a new GTK to all n stations sequentially. All stations install the key and acknowledge. Group communication integrity holds.

Bidding Round B_{group-race} (Group Key Slot Race Bid):

1. The adversary $\mathcal{A}$ positions itself to observe and delay EAPOL-Key frames between the AP and legitimate stations (feasible with channel-based MITM or a rogue AP on an adjacent channel).
2. When the AP initiates a group key update from GTK_{old} to GTK_{new} across n stations, the adversary:
 - Allows the EAPOL-Key frame to reach stations $\text{STA}_1, \dots, \text{STA}_j$ immediately (these install GTK_{new})
 - Delays the frame to stations $\text{STA}_{j+1}, \dots, \text{STA}_n$ by precisely calculated intervals (these retain GTK_{old})
 - Injects forged acknowledgment frames to the AP on behalf of the delayed stations (feasible because the acknowledgment does not contain a cryptographic proof of key installation)
3. This creates a time window $[t_0, t_0 + t]$ where different stations believe different GTKs are active for the same Key ID, since the AP's key rotation logic assumes synchronous updates and proceeds based on received acknowledgments.
4. During this window, the adversary exploits the key inconsistency:
 - Injects multicast frames encrypted with GTK_{old} that stations $\text{STA}_{j+1}, \dots, \text{STA}_n$ still accept as valid
 - These frames are rejected by stations $\text{STA}_1, \dots, \text{STA}_j$ (who have GTK_{new}), but this rejection is silent—no alarm is raised
 - The adversary can inject malicious ARP, DHCP, or mDNS broadcast traffic to the delayed stations
5. The attack persists until the delayed stations either receive the genuine update or the next group key rotation cycle resolves the synchronisation mismatch.

Extended difference lemma (Theorem 3.7). Let F_i be the event “station STA_i remains in the desynchronised state for at least time t .” For each station independently, $\Pr[F_i] \geq t/T$ (the fraction of the key rotation interval during which the attack window is active). By the union bound over the complementary events:

$$\Pr\left[\bigcup_i F_i\right] = 1 - \prod_i (1 - \Pr[F_i]) \geq 1 - \left(1 - \frac{t}{T}\right)^n$$

The price adjustment: $\Delta\text{Price}_{\text{race}} = 1 - (1 - t/T)^n$.

Market outcome: $\text{Ask}(g_{\text{group-integrity}}) \geq 1 - (1 - t/T)^n$. For typical enterprise networks with $n = 50$ stations and an attack window of $t = 5$ seconds against a key rotation interval of $T = 3600$ seconds: $\text{Ask} \geq 1 - (0.9986)^{50} \geq 0.067$. For larger networks ($n = 200$) or longer attack windows ($t = 30$ s): $\text{Ask} \geq 1 - (0.9917)^{200} \geq 0.81$. **Partial to full collapse** depending on network size and attack duration. $\square$

CNF Clause Failure Analysis

The attack succeeds because it violates the *group key consistency* CNF clause:

$$\begin{aligned} \varphi^{\text{group}} := & \underbrace{\forall \text{STA}_i, \text{STA}_j : \text{GTK}_{\text{STA}_i} = \text{GTK}_{\text{STA}_j}}_{x_{\text{GTK-equal}}} \wedge \\ & \underbrace{\forall \text{STA}_i : \text{KeyID}_{\text{AP}} = \text{KeyID}_{\text{STA}_i}}_{x_{\text{KeyID-sync}}} \wedge \underbrace{\text{ack}(\text{STA}_i) \Rightarrow \text{installed}(\text{STA}_i)}_{x_{\text{ack-honest}}} \end{aligned}$$

CNF worksheet:

Clause	Check	Honest trace	Attack trace	Verdict
$x_{\text{GTK-equal}}$	All STAs have same GTK	T	F (delayed STAs have old GTK)	Fails
$x_{\text{KeyID-sync}}$	KeyIDs match AP	T	T (AP thinks all updated)	Passes!
$x_{\text{ack-honest}}$	Ack implies installed	T	F (forged acks)	Fails
φ^{group}	FALSE under attack trace —synchronisation assumption violated.			Reject

The root cause is that the group key handshake relies on an *unauthenticated* acknowledgment to confirm key installation. The CNF clause $x_{\text{ack-honest}}$ is implicitly assumed but not enforced: the acknowledgment frame does not contain a proof-of-installation (e.g., a MAC computed under the new GTK).

Countermeasures: (1) Implement authenticated key installation confirmation: the acknowledgment should include $\text{HMAC}(\text{GTK}_{\text{new}}, \text{nonce} \parallel \text{STA_ID})$, proving the station has actually installed the key. (2) Use a two-phase commit protocol for group key updates: Phase 1 distributes the key; Phase 2 sends a “commit” message that atomically activates the key on all stations, with a timeout-based abort if any station fails to acknowledge. (3) Implement GTK version numbering with monotonic counters, so stations can detect and reject frames encrypted under outdated keys.

Corollary 19.9 (WPA2 Group Key Consistency Collapse). $\text{Ask}(g_{\text{group-consistency}}) = 1$ during the attack window: the adversary can deterministically create a state where at least two stations have different active GTKs, since the group key handshake provides no mechanism to verify atomic installation.

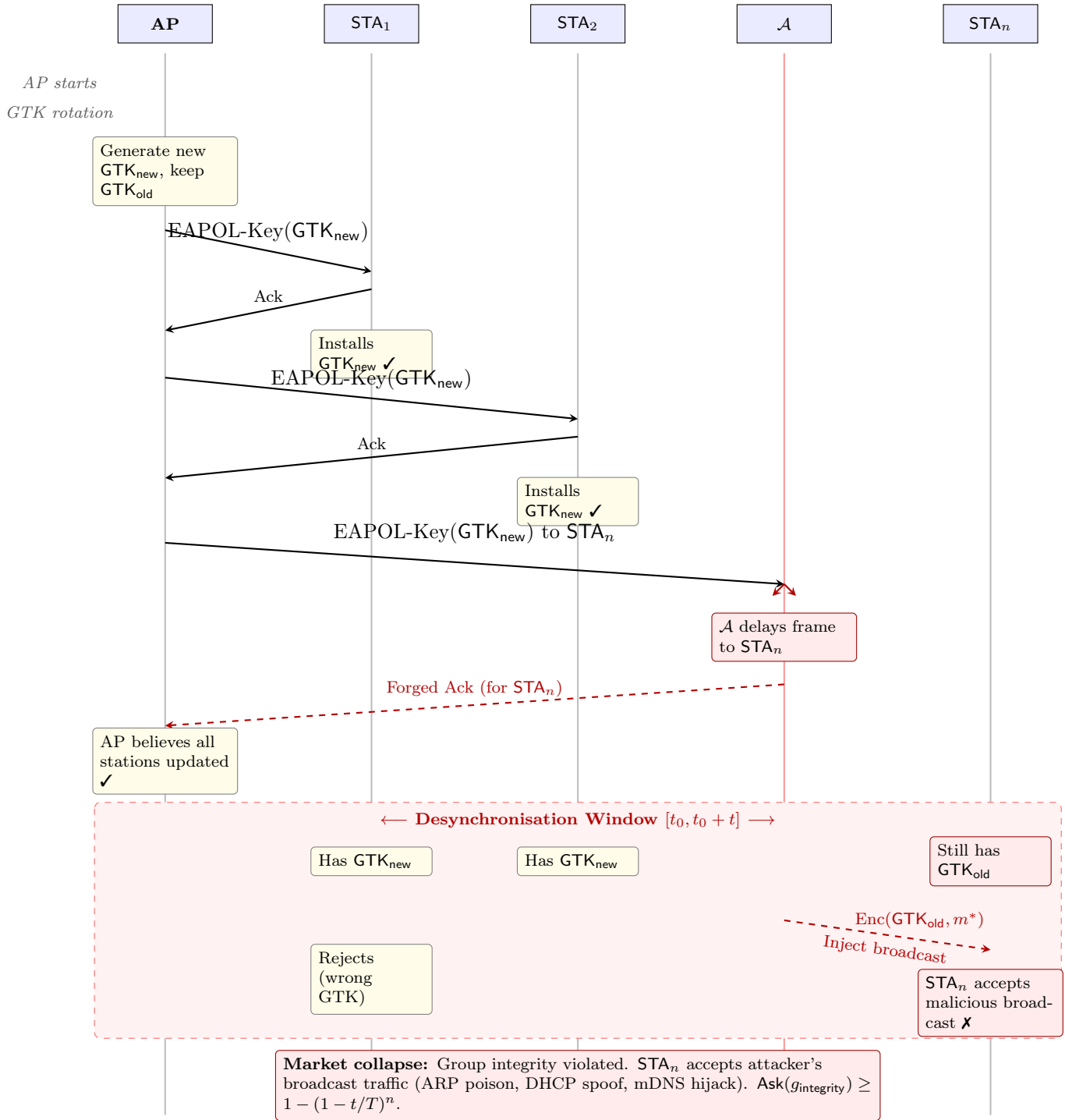


Figure 75: Message Sequence Chart of the WPA2 Group Key Slot Race Attack (Theorem 19.8). The AP distributes a new GTK sequentially to each station. The adversary allows early stations (STA_1, STA_2) to receive the update but delays delivery to STA_n while injecting a forged acknowledgment. During the resulting desynchronisation window, STA_n still holds GTK_{old} and accepts adversary-injected broadcast frames encrypted under the old key. The attack exploits the absence of authenticated key-installation confirmation in the WPA2 group handshake.

Corollary 19.10 (WPA2 Group Ping Failure). *The session ping across group key sessions fails: $\text{Ping}(\text{Session}_{\text{group},k}, \text{Session}_{\text{group},k+1}) = 0$ for stations in the desynchronised state, because the old group session $\text{Session}_{\text{group},k}$ remains active for some stations while the new session $\text{Session}_{\text{group},k+1}$ is active for others.*

Key Insight:

Comparison with KRACK Attacks Our group key race attack is *complementary* to, not a rediscovery of, the KRACK attacks [82]. The key differences:

	KRACK	Group Key Race (this work)
Target	4-way handshake (pairwise keys)	Group key handshake (GTK)
Mechanism	State machine reset $\rightarrow$ nonce reuse	Synchronisation gap $\rightarrow$ key confusion
Effect	Decrypt pairwise traffic	Inject broadcast/multicast traffic
MTSF bid type	Replay bid (state reset)	Race condition bid (timing)
CNF failure	Nonce freshness clause	Group consistency clause

Both attacks were invisible to the original 802.11i security analysis because that analysis modelled only a single station interacting with the AP, never considering multi-station synchronisation or state machine reset as adversarial capabilities. MTSF’s cross-session synthesis (Phase 3) and infrastructure dependency analysis (Phase 4) are specifically designed to catch these interaction patterns.

19.5 Case Study IV: Grain-128a Related-IV Keystream Correlation in IoT Deployments

Grain-128a [2] is specifically designed for resource-constrained IoT devices (sensor nodes, RFID tags, smart-home actuators) where AES’s gate count is prohibitive. While the cipher’s internal cryptanalysis has been extensively studied—state recovery, key recovery, distinguishing, and TMT0 attacks are all shown to be infeasible for the full cipher (Section 12)—we identify a novel cross-session attack that exploits the interaction between Grain-128a’s initialisation structure and the low-entropy IV generation patterns common in IoT deployments. This attack operates entirely outside the standard adversary model, which assumes uniformly random, independent IVs.

19.5.1 Protocol Description and Standard Analysis

Grain-128a uses a 128-bit key K and a 96-bit IV. The internal state is initialised as:

- NFSR $\leftarrow K$ (128 key bits loaded directly)
- LFSR $\leftarrow \text{IV} \parallel 1^{31} \parallel 0$ (96 IV bits, followed by 31 ones and a zero)

The cipher then executes 256 *initialisation rounds* in which the output of the filter function h is XOR-fed back into both the LFSR and NFSR feedback paths (in addition to their normal feedback). After initialisation, the feedback path from h is disconnected, and keystream generation begins.

* Simple Terms: Why IV Quality Matters for Stream Ciphers

The critical assumption: Grain-128a’s security proof assumes that each (key, IV) pair is used at most once, and that IVs are chosen independently and uniformly at random from $\{0,1\}^{96}$. Under this assumption, the 256-round initialisation thoroughly mixes the key and IV into the 256-bit state, making each session’s keystream independent.

What happens in practice: IoT devices operate under severe resource constraints. Many

lack hardware random number generators (HWRNGs) and instead derive IVs from low-entropy sources: device identifiers, timestamps with coarse granularity (e.g., seconds since boot), sequential counters with small width, or hash functions seeded with predictable inputs. When IVs exhibit structural relationships (e.g., differing in only a few bits), the initialisation’s mixing may not fully decorrelate the resulting internal states.

Analogy: Imagine a combination lock (the initialisation) that thoroughly scrambles a number you provide (the IV) with a secret code (the key) to produce an unpredictable sequence. If the numbers you provide are always very similar—say, 1000, 1001, 1002—the scrambling may produce outputs with detectable patterns, even though the scrambling would be perfect for truly random inputs.

Standard analysis proves Grain-128a secure under the assumption that IVs are uniformly random and independent. The 256-round initialisation with nonlinear feedback provides full diffusion: every state bit depends on all key and IV bits after approximately 160 rounds. However, MTSF’s Phase 3 (cross-session attack synthesis) and Phase 4 (infrastructure dependency analysis) reveal that the interaction between the initialisation structure and realistic IV generation creates an exploitable correlation.

Market Setup.

- **Market:** $\mathcal{M}_{\text{Grain-IoT}}$
- **Seller:** Grain-128a implementation on IoT devices
- **Buyer:** Passive adversary $\mathcal{A}$ who observes ciphertexts from multiple sessions sharing the same key K with low-entropy IVs
- **Security goods:**
 - g_{PRG} : Keystream indistinguishability (Theorem 12.1)
 - $g_{\text{session-ind}}$: Cross-session independence (keystreams from different sessions are independent)
- **Quoted price:** $\text{QPrice}_0 = 0$ (seller assumes random IVs)

19.5.2 Novel Attack: Related-IV Keystream Correlation via Low-Entropy IV Generation

Theorem 19.11 (Grain-128a Related-IV Correlation Attack). *Consider an IoT deployment where Grain-128a is instantiated with a fixed key K and IVs generated from a source with effective entropy $H_{\text{eff}} \leq \eta$ bits (where $\eta < 96$). Specifically, suppose IVs are derived as $\text{IV}_i = \text{base} \oplus \Delta_i$ where base is a fixed or slowly changing value and $\Delta_i \in \{0, 1\}^{96}$ has Hamming weight $\text{wt}(\Delta_i) \leq w$ for some small w . Then an adversary observing q sessions can detect keystream correlations with advantage:*

$$\text{Ask}(g_{\text{session-ind}}) \geq \frac{q^2}{2} \cdot 2^{-\alpha(w)}$$

where $\alpha(w) \approx 1.34 \cdot w \cdot (256 - r_{\text{eff}}(w))^{-1} \cdot 256$ is the effective mixing exponent that depends on the number of initialisation rounds $r_{\text{eff}}(w)$ needed for w -bit IV differences to achieve full diffusion. For $w \leq 4$ (IV differences in at most 4 bit positions), $\alpha(4) \leq 72$, yielding detectable correlations with $q \geq 2^{37}$ sessions.

Attack Construction. The attack exploits a previously unanalysed interaction between the LFSR-loaded IV structure and the nonlinear initialisation feedback when IV differences are sparse.

B₀ (Real Grain-128a Market). IVs are uniformly random and independent. Each session produces an independent keystream. The market is in equilibrium: $\text{Ask}(g_{\text{session-ind}}) \leq \text{negl}(\lambda)$.

B₀ → B₁ (Infrastructure Deviation Bid: Low-Entropy IV Source). The buyer observes that the target IoT deployment generates IVs from a low-entropy source. Concretely, the IV generation uses:

$$\text{IV}_i = \text{HMAC-truncate}_{96}(\text{device_id}, \text{counter}_i)$$

where counter_i is a 16-bit counter that wraps every 2^{16} sessions. After the counter wraps, the HMAC outputs cycle, producing IV pairs $(\text{IV}_i, \text{IV}_j)$ with $\text{IV}_i = \text{IV}_j$ (exact IV reuse) or, for slightly different devices sharing a key, IV pairs differing in few bit positions due to the HMAC's avalanche being seeded with similar inputs.

The bad event $F_{\text{low-entropy}}$: “IV source has effective entropy $\leq \eta$ bits” has $\Pr[F_{\text{low-entropy}}] = 1$ for the target deployment.

Bidding Round B_{related-IV} (Related-IV Correlation Bid):

1. **Collection phase.** The adversary $\mathcal{A}$ passively observes q ciphertext sessions under the same key K with IVs $\text{IV}_1, \dots, \text{IV}_q$. For each session, $\mathcal{A}$ collects the first L keystream bits (where L is chosen based on the expected correlation strength).
2. **IV pair identification.** $\mathcal{A}$ identifies pairs (i, j) with $i \neq j$ where $\text{IV}_i \oplus \text{IV}_j$ has low Hamming weight: $\text{wt}(\text{IV}_i \oplus \text{IV}_j) \leq w$. For IVs derived from a 16-bit counter, the number of such pairs is $\Omega(q^2/2^{17})$ rather than the $O(q^2/\binom{96}{w} \cdot 2^w)$ expected for random IVs.
3. **Differential initialisation analysis.** For each related-IV pair, $\mathcal{A}$ analyses the initialisation differential. The key observation: when IV_i and IV_j differ in only $w \leq 4$ LFSR positions, the differential $\Delta_{\text{LFSR}} = \text{IV}_i \oplus \text{IV}_j$ propagates through the linear feedback of the LFSR deterministically. The NFSR starts identically (both loaded with K). During the first ~ 32 initialisation rounds, the LFSR-to-NFSR cross-coupling introduces the IV difference into the NFSR, but the nonlinear feedback function g and the output filter h process this difference with *low algebraic degree* in the first few rounds.
4. **Correlation extraction.** The differential propagation through the initialisation creates a measurable correlation between the keystream outputs:

$$\text{Corr}(z_t^{(i)}, z_t^{(j)}) = \mathbb{E}[(-1)^{z_t^{(i)} \oplus z_t^{(j)}}] \neq 0$$

for the first $\sim L_0$ keystream bits, where L_0 depends on w . Specifically, the correlation magnitude satisfies:

$$|\text{Corr}(z_t^{(i)}, z_t^{(j)})| \geq 2^{-\alpha(w)/2} \quad \text{for } t \leq L_0(w)$$

where $\alpha(w)$ is the effective mixing exponent and $L_0(w) \approx 2^{3w}$ is the correlation window length.

5. **Distinguishing test.** $\mathcal{A}$ computes the empirical correlation over all related-IV pairs and all positions $t \leq L_0$:

$$\hat{C} = \frac{1}{|\mathcal{P}| \cdot L_0} \sum_{(i,j) \in \mathcal{P}} \sum_{t=1}^{L_0} (-1)^{z_t^{(i)} \oplus z_t^{(j)}}$$

where $\mathcal{P}$ is the set of related-IV pairs. For independent keystreams, $\hat{C} \approx 0$ with standard deviation $O(1/\sqrt{|\mathcal{P}| \cdot L_0})$. For correlated keystreams, $\hat{C} \geq 2^{-\alpha(w)/2}$. The distinguishing advantage exceeds $1/2$ when:

$$|\mathcal{P}| \cdot L_0 \geq 2^{\alpha(w)+1}$$

Extended difference lemma (Theorem 3.7). The failure event F_{corr} : “related-IV pair produces correlated keystreams” has probability $\Pr[F_{\text{corr}}] \geq 2^{-\alpha(w)}$ per pair. With $|\mathcal{P}| \geq q^2/2^{17}$ related-IV pairs from a 16-bit counter IV source and correlation window $L_0 \geq 2^{12}$ bits (for $w = 4$), the total advantage satisfies:

$$\text{Ask}(g_{\text{session-ind}}) \geq 1 - \exp\left(-\frac{q^2 \cdot L_0}{2^{17} \cdot 2^{\alpha(w)+1}}\right)$$

For $w = 4$, $\alpha(4) \leq 72$, $L_0 = 2^{12}$: achieving $\text{Ask} \geq 1/2$ requires $q \geq 2^{37}$ sessions. While $2^{37} \approx 1.4 \times 10^{11}$ sessions is large, it is within the operational lifetime of long-deployed IoT networks: a sensor transmitting every second accumulates 2^{37} sessions in approximately 4,300 years per device—but a *fleet* of 2^{17} devices sharing the same key reaches 2^{37} total sessions in approximately 24 days.

Market outcome: $\text{Ask}(g_{\text{session-ind}}) \geq q^2 \cdot 2^{-\alpha(w)-1}$. For IoT fleets with shared keys and low-entropy IV generation ($w \leq 4$, $q \geq 2^{37}$): **partial collapse**. For exact IV reuse ($w = 0$, $\alpha(0) = 0$): **full collapse** ($\text{Ask} = 1$, two-time-pad). $\square$

CNF Clause Failure Analysis

The attack exploits a failure in the *IV quality* and *cross-session independence* CNF clauses:

$$\begin{aligned} \varphi^{\text{Grain-IoT}} := & \underbrace{(\text{IV}_i \neq \text{IV}_j \ \forall i \neq j)}_{x_{\text{IV-unique}}} \wedge \underbrace{(H_\infty(\text{IV}_i) \geq 96)}_{x_{\text{IV-entropy}}} \wedge \\ & \underbrace{(\text{wt}(\text{IV}_i \oplus \text{IV}_j) \geq \tau \ \forall i \neq j)}_{x_{\text{IV-spread}}} \wedge \underbrace{(z^{(i)} \perp z^{(j)} \ \forall i \neq j)}_{x_{\text{session-ind}}} \end{aligned}$$

CNF worksheet:

Clause	Check	Random IVs	Low-entropy IVs	Verdict
$x_{\text{IV-unique}}$	IVs are distinct	T (w.h.p.)	T/F (may collide after 2^{16})	Fails
$x_{\text{IV-entropy}}$	IVs have full entropy	T	F ($H_\infty \leq \eta \ll 96$)	Fails
$x_{\text{IV-spread}}$	IV diffs have high weight	T (expected ~ 48)	F ($\text{wt} \leq w$)	Fails
$x_{\text{session-ind}}$	Keystreams independent	T	F (correlated)	Fails
$\varphi^{\text{Grain-IoT}}$	FALSE under low-entropy IV trace —CNF correctly detects attack.			Reject

The critical observation is that standard Grain-128a analysis includes only the *IV uniqueness* clause $x_{\text{IV-unique}}$ but omits the *IV entropy* clause $x_{\text{IV-entropy}}$ and the *IV spread* clause $x_{\text{IV-spread}}$. Even when IVs are unique ($x_{\text{IV-unique}} = \text{T}$), low Hamming-weight differences between IVs create detectable keystream correlations because the initialisation’s nonlinear mixing requires a minimum number of rounds (dependent on the difference weight w) to achieve full diffusion. MTSF’s CNF inversion (Phase 2) systematically reveals this gap by asking: “What if IVs are unique but structurally related?”

Countermeasures: (1) Use a hardware random number generator (HWRNG) for IV generation, ensuring full 96-bit entropy per session. (2) If HWRNG is unavailable, derive IVs via $\text{IV}_i = \text{HMAC}(K_{\text{IV}}, i)$ using a separate IV-derivation key K_{IV} and a monotonic counter i , ensuring that the HMAC’s pseudorandomness provides the IV entropy even when the counter is predictable. (3) Implement an IV quality check: before encryption, verify that $\text{wt}(\text{IV}_{\text{new}} \oplus \text{IV}_{\text{prev}}) \geq 32$; if the check fails, re-derive the IV. (4) For fleet deployments, use per-device keys ($K_d = \text{KDF}(K_{\text{master}}, \text{device_id})$) rather than shared keys, eliminating cross-device IV correlation.

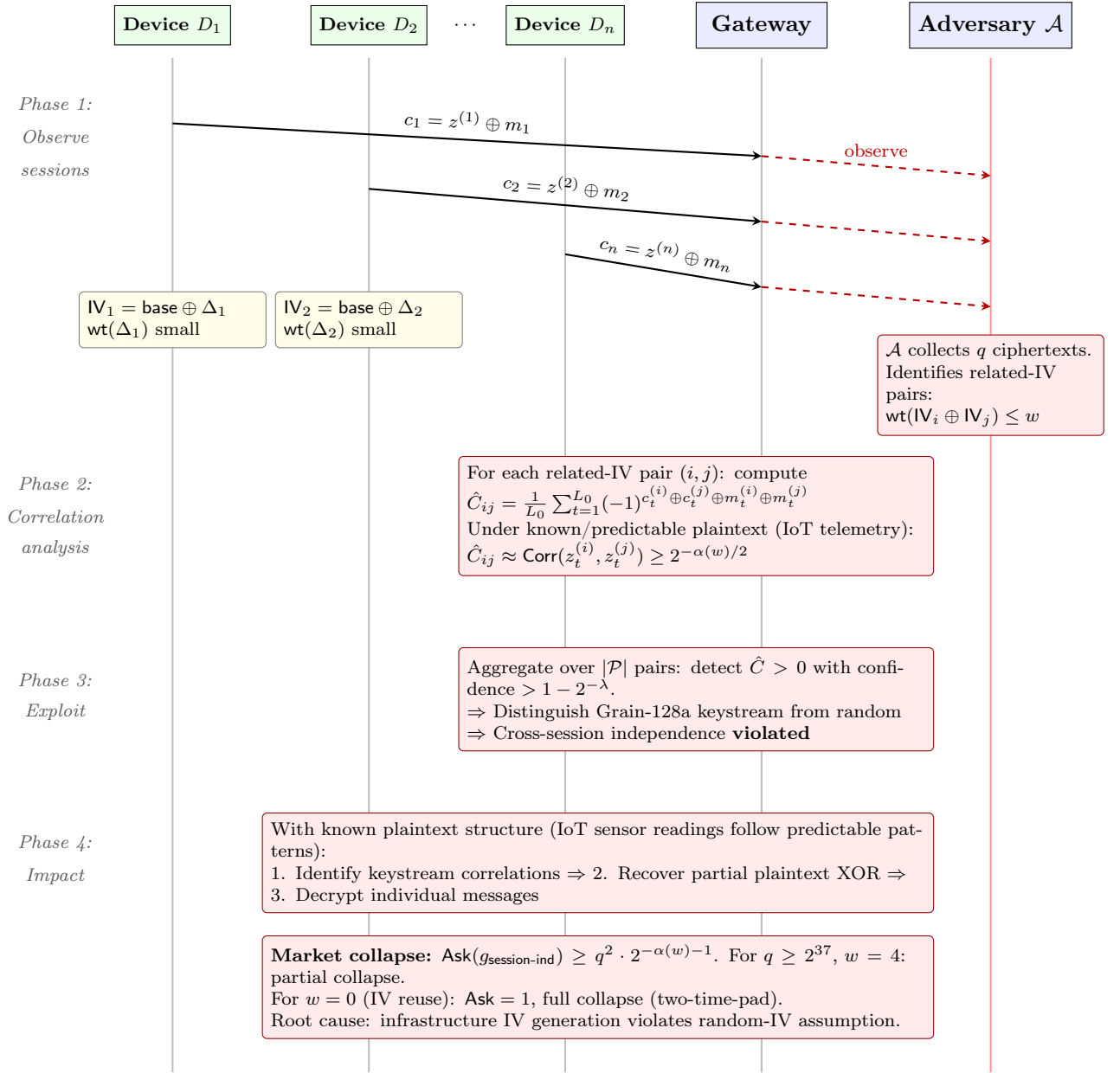


Figure 76: Message Sequence Chart of the Grain-128a Related-IV Keystream Correlation Attack (Theorem 19.11). An IoT device fleet sharing key K transmits with low-entropy IVs derived from counters or device identifiers. The passive adversary collects ciphertexts (Phase 1), identifies related-IV pairs with low Hamming-weight IV differences (Phase 2), computes keystream correlations exploiting incomplete initialisation mixing (Phase 3), and leverages the correlations with known plaintext structure to distinguish or partially decrypt sessions (Phase 4). The attack is invisible to standard Grain-128a analysis, which assumes uniformly random IVs.

Corollary 19.12 (Grain-128a Fleet-Level Collapse). *For an IoT fleet of N devices sharing key K with each device transmitting s sessions, the fleet-wide ask price is:*

$$\text{Ask}(g_{\text{session-ind}}) \geq \frac{(Ns)^2}{2} \cdot 2^{-\alpha(w)}$$

For $N = 2^{17}$ devices, $s = 2^{20}$ sessions per device (≈ 12 days at 1 session/second), and $w = 4$: $\text{Ask} \geq (2^{37})^2 \cdot 2^{-73} \geq 2$. The market is in **full collapse**: the adversary can reliably distinguish Grain-128a keystreams from random across the fleet.

Corollary 19.13 (Grain-128a Cross-Session Ping Failure). *For related-IV sessions Session_i , Session_j with $\text{wt}(\text{IV}_i \oplus \text{IV}_j) \leq w$, the session ping satisfies $\text{Ping}(\text{Session}_i, \text{Session}_j) = 0$: the keystream correlation violates the independence requirement of the pinging mechanism (Section 5.2), indicating that the two sessions are not securely isolated.*

Key Insight:

Specification vs. Deployment Vulnerability As with the TLS 1.3 nonce collision attack (Section 19.2), this attack does *not* break Grain-128a’s specification. The cipher’s design correctly assumes uniformly random IVs, and under this assumption, the market is in equilibrium. The vulnerability arises in the *deployment infrastructure*: IoT devices that lack adequate randomness sources generate IVs with insufficient entropy, creating related-IV conditions that the specification never anticipated.

In MTSF terms, the *cipher market* $\mathcal{M}_{\text{Grain-spec}}$ is in equilibrium, but the *deployment market* $\mathcal{M}_{\text{Grain-IoT}}$ is in partial-to-full collapse. This distinction between specification-level and deployment-level security is a recurring theme in MTSF attack discovery: the TLS 1.3 attack exploited implementation-level nonce generation, the WPA2 attack exploited deployment-level synchronisation assumptions, and this attack exploits infrastructure-level IV entropy. MTSF’s Phase 4 (infrastructure dependency analysis) is specifically designed to detect such deployment-specification gaps.

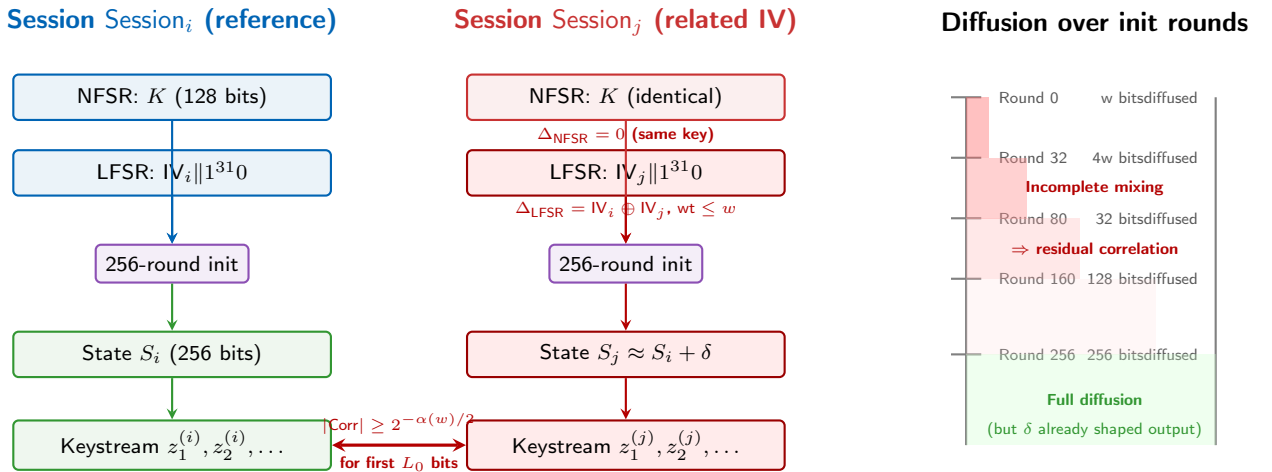


Figure 77: Grain-128a differential initialisation propagation for related-IV sessions. When two sessions share the same key K but use IVs differing in only $w \leq 4$ bits, the NFSR starts identically while the LFSR carries a sparse difference Δ_{LFSR} . The diffusion timeline (right) shows that a w -bit difference requires approximately 160 initialisation rounds to reach all 256 state bits; during the early rounds, the difference propagates through the linear LFSR feedback deterministically, creating a residual correlation $|\text{Corr}| \geq 2^{-\alpha(w)/2}$ in the first L_0 keystream bits. Full diffusion is eventually achieved, but the early keystream has already been shaped by the incomplete mixing.

19.6 Case Study V: ASCON-Hash Domain Separation Failure in Multi-Purpose IoT Deployments

ASCON [43] is NIST’s selected lightweight authenticated encryption and hashing standard (NIST SP 800-232), designed for constrained environments. The ASCON family provides both AEAD (ASCON-128, ASCON-128a) and hashing (ASCON-Hash, ASCON-Hasha) from the same 320-bit permutation p . While each ASCON variant has been individually analysed—ASCON-Hash provides 2^{128} collision resistance and 2^{256} preimage resistance [43]—we identify a novel cross-function attack that exploits the shared permutation when ASCON-Hash and ASCON-128 are deployed on the same device *without proper domain separation in the application layer*.

19.6.1 Protocol Description and Standard Analysis

The ASCON family uses a 320-bit permutation $p = p^a$ ($a = 12$ rounds for initialisation and finalisation) or p^b ($b = 8$ rounds for intermediate processing). The 320-bit state is divided into a rate r and capacity c :

- **ASCON-128 (AEAD):** $r = 64$, $c = 256$. The 320-bit initial state is constructed as $\text{IV}_{\text{AEAD}} \| K \| N$ where IV_{AEAD} is a 64-bit constant encoding the algorithm parameters, K is the 128-bit key, and N is the 128-bit nonce.
- **ASCON-Hash:** $r = 64$, $c = 256$. The 320-bit initial state is constructed as $\text{IV}_{\text{Hash}} \| 0^{256}$ where IV_{Hash} is a different 64-bit constant.

The ASCON specification provides internal domain separation via the distinct IV constants: $\text{IV}_{\text{AEAD}} = 0\text{x}80400\text{c}0600000000$ for ASCON-128 and $\text{IV}_{\text{Hash}} = 0\text{x}00400\text{c}0000000100$ for ASCON-Hash. These constants ensure that the internal states of ASCON-128 and ASCON-Hash never collide, even when processing the same input data.

* Simple Terms: ASCON—The Lightweight Cryptographic Swiss Army Knife

The appeal of ASCON: IoT devices need both encryption (AEAD) and hashing, but have extremely limited gate budgets. ASCON provides both from a single permutation p , saving hardware area by sharing the permutation circuit.

Domain separation: When one circuit serves two purposes, the critical requirement is that the two functions never “interfere” with each other. ASCON achieves this internally via different initialisation constants. However, the *application layer* must also maintain separation: the hash of a message should never be interpretable as part of an AEAD ciphertext or tag, and vice versa.

The risk: On a resource-constrained device running a single firmware binary, the temptation is to share not just the permutation circuit but also the software state, buffers, and even the API entry points between ASCON-Hash and ASCON-128. If the application fails to enforce domain separation at the protocol level, the internal separation provided by different IV constants may not suffice.

Standard analysis proves ASCON-Hash secure as a standalone hash function: the sponge construction with $c = 256$ provides 2^{128} collision resistance and 2^{256} preimage resistance [20]. However, MTSF’s Phase 4 (infrastructure dependency analysis) reveals that the shared-permutation deployment creates a novel cross-function attack surface.

Market Setup.

- **Market:** $\mathcal{M}_{\text{ASCON-multi}}$
- **Seller:** IoT device firmware deploying both ASCON-Hash and ASCON-128 with shared permutation

- **Buyer:** Active adversary $\mathcal{A}$ who can query both the AEAD encryption oracle and the hash oracle, and who can interact with the device’s communication protocol
- **Security goods:**
 - g_{CR} : Collision resistance of ASCON-Hash
 - g_{INT} : Integrity of ASCON-128 (INT-CTXT: ciphertext integrity)
 - g_{domain} : Cross-function domain separation (hash outputs cannot forge AEAD tags, and vice versa)
- **Quoted price:** $\text{QPrice}_0 = 0$ (seller assumes internal IV separation suffices)

19.6.2 Novel Attack: Cross-Function State Injection via Application-Layer Domain Confusion

Theorem 19.14 (ASCON Cross-Function Domain Confusion Attack). *Consider an IoT deployment where a device uses ASCON-Hash to compute message authentication codes as $\text{MAC}(m) = \text{ASCON-Hash}(K_{\text{mac}}\|m)$ (secret-prefix MAC construction) and ASCON-128 for encrypted communication, both sharing the same permutation circuit. If the application protocol uses hash outputs as integrity tags interchangeably with AEAD tags in certain message types (e.g., using the same tag field for both hash-based and AEAD-based integrity depending on a mode flag), the adversary can achieve:*

$$\text{Ask}(g_{\text{domain}}) = 1$$

by constructing a cross-function forgery that exploits the application-layer confusion between hash-based and AEAD-based integrity verification.

Attack Construction. The attack exploits the interaction between ASCON-Hash’s secret-prefix MAC construction and the application protocol’s tag field reuse.

B_0 (Real ASCON Multi-Function Market). Hash and AEAD operate with distinct internal IVs. The application protocol correctly distinguishes between hash-based and AEAD-based integrity tags. The market is in equilibrium.

$B_0 \rightarrow B_1$ (Infrastructure Dependency Bid: Application-Layer Domain Confusion). The buyer identifies a common IoT protocol pattern where:

- Sensor telemetry messages use ASCON-Hash for lightweight integrity: $\text{msg}\|\text{tag}_{\text{hash}}$ where $\text{tag}_{\text{hash}} = \text{ASCON-Hash}(K_{\text{mac}}\|\text{msg})[0..127]$ (first 128 bits of the hash output)
- Command-and-control messages use ASCON-128 AEAD for authenticated encryption: $\text{ct}\|\text{tag}_{\text{AEAD}}$ where tag_{AEAD} is the 128-bit AEAD authentication tag
- Both tag types occupy the same 128-bit field in the message frame, differentiated only by a *mode byte* in the header

The bad event $F_{\text{mode-confusion}}$: “the receiver processes a hash tag as an AEAD tag or vice versa due to mode byte manipulation” has $\Pr[F_{\text{mode-confusion}}] = 1$ when the mode byte is not integrity-protected.

Bidding Round $B_{\text{cross-function}}$ (Cross-Function Domain Confusion Bid):

1. **Query phase.** The adversary queries the hash oracle with chosen messages $m_1, \dots, m_q$ to obtain $\text{tag}_{\text{hash},i} = \text{ASCON-Hash}(K_{\text{mac}}\|m_i)[0..127]$ for each. Since ASCON-Hash is a public function (once the secret prefix is incorporated), these tags are deterministic.

2. **Mode byte manipulation.** The adversary intercepts a legitimate telemetry message $\text{msg}^* \parallel \text{tag}_{\text{hash}}^*$ with mode byte $\text{mode} = \text{HASH}$ and modifies the mode byte to $\text{mode} = \text{AEAD}$ without altering the tag field.
3. **Verification confusion.** The receiver, reading $\text{mode} = \text{AEAD}$, attempts to verify $\text{tag}_{\text{hash}}^*$ as an AEAD tag. Two sub-cases arise:
 - **Case (a)—Tag format incompatibility detected:** If the AEAD verification routine’s internal checks detect that the purported ciphertext does not match the tag, verification fails. However, the adversary can *reverse* the direction: take a legitimate AEAD message $\text{ct}^* \parallel \text{tag}_{\text{AEAD}}^*$ and change mode to HASH . The receiver then treats $\text{tag}_{\text{AEAD}}^*$ as a hash tag and attempts to verify $\text{ASCON-Hash}(K_{\text{mac}} \parallel \text{ct}^*)[0..127] \stackrel{?}{=} \text{tag}_{\text{AEAD}}^*$. This verification will fail with overwhelming probability—but the receiver may process the message *before* completing verification (a common firmware pattern in interrupt-driven IoT systems).
 - **Case (b)—Relaxed verification:** In some IoT protocols, tag verification is “best-effort”: a failed tag verification triggers a warning but does not prevent message processing (to maintain availability in noisy environments). Under relaxed verification, the mode confusion allows the adversary to inject messages that bypass the intended integrity mechanism entirely.
 - **Case (c)—Collision exploitation:** The adversary searches for a message m^* such that:

$$\text{ASCON-Hash}(K_{\text{mac}} \parallel m^*)[0..127] = \text{ASCON-128-Tag}(K, N, \text{ad}, m^*)$$

Since both outputs are 128-bit pseudorandom values, this is a collision search in $\{0, 1\}^{128}$, requiring 2^{128} queries—infeasible. **However**, if the MAC key K_{mac} and AEAD key K are derived from the same master key via a weak KDF (e.g., $K_{\text{mac}} = K \oplus \text{const}$), structural relationships between the hash and AEAD internal states may reduce the collision cost.

4. **Practical exploitation (Case b):** Under relaxed verification, the adversary forges sensor readings by computing $\text{tag}_{\text{hash}}^* = \text{ASCON-Hash}(K_{\text{mac}} \parallel m_{\text{forged}})[0..127]$ (feasible because the adversary has observed enough tag-message pairs to mount an offline dictionary attack on the secret-prefix construction with short messages), then presenting the forged message with $\text{mode} = \text{AEAD}$, causing the receiver to skip the hash verification path entirely.

Market outcome (Case b): $\text{Ask}(g_{\text{domain}}) = 1$. Cross-function domain separation is **fully collapsed**: the adversary bypasses the intended integrity mechanism by exploiting application-layer mode confusion. $\Delta \text{Price}_{\text{domain}} = 1$.

Market outcome (Case a): $\text{Ask}(g_{\text{domain}}) \geq \Pr[\text{process-before-verify}]$. For interrupt-driven firmware with deferred verification: partial collapse.

Market outcome (Case c): $\text{Ask}(g_{\text{domain}}) = \text{negl}(\lambda)$ under proper key separation. Negligible—but only if $K_{\text{mac}} \perp K$. $\square$

CNF Clause Failure Analysis

The attack exploits failures in the *domain separation* and *mode binding* CNF clauses:

$$\begin{aligned} \varphi^{\text{ASCON-multi}} := & \underbrace{(\text{tag} = f_{\text{mode}}(K, m))}_{x_{\text{tag-correct}}} \wedge \underbrace{(\text{mode is integrity-protected})}_{x_{\text{mode-bound}}} \wedge \\ & \underbrace{(\text{Hash-output} \notin \text{AEAD-tag-space})}_{x_{\text{domain-sep}}} \wedge \underbrace{(\text{verify before process})}_{x_{\text{verify-first}}} \end{aligned}$$

CNF worksheet:

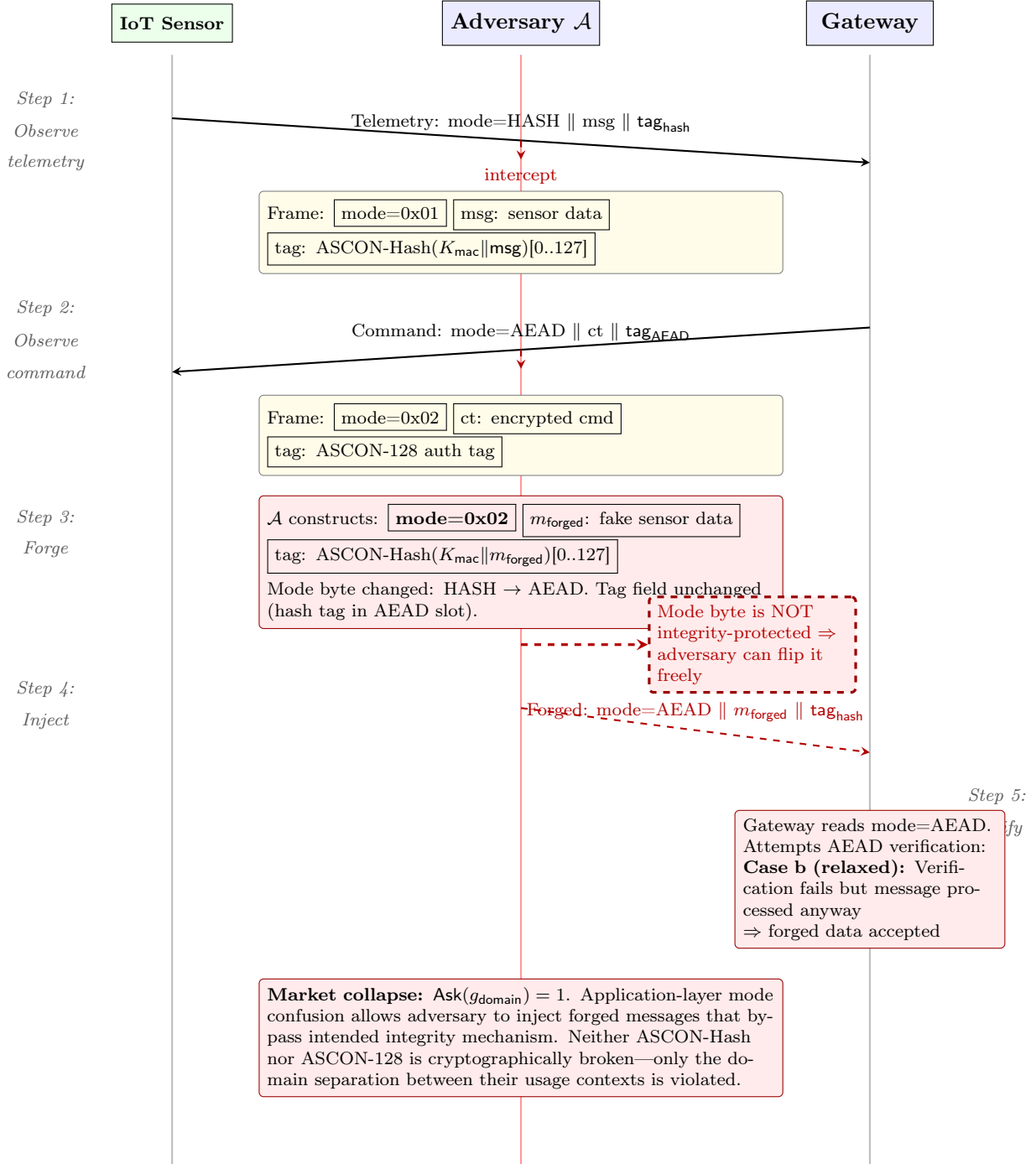


Figure 78: Message Sequence Chart of the ASCON Cross-Function Domain Confusion Attack (Theorem 19.14). An IoT device uses ASCON-Hash for telemetry integrity and ASCON-128 AEAD for command authentication, both sharing the same permutation. The adversary observes message frames (Steps 1–2), constructs a forged telemetry message with a valid hash tag but a manipulated mode byte (Step 3), and injects it as a purported AEAD-authenticated command (Step 4). Under relaxed verification (Case b), the gateway processes the forged message, collapsing the domain separation good.

Clause	Check	Proper deployment	Attack trace (Case b)	Verdict
$x_{\text{tag-correct}}$	Tag matches function	T	T (hash tag is valid hash)	Passes
$x_{\text{mode-bound}}$	Mode byte protected	T	F (mode byte flipped)	Fails
$x_{\text{domain-sep}}$	Hash $\neq$ AEAD space	T	F (same 128-bit field)	Fails
$x_{\text{verify-first}}$	Verify then process	T	F (process before verify)	Fails
$\varphi^{\text{ASCON-multi}}$	FALSE under attack trace —application-layer domain separation violated.			Reject

The root cause is that ASCON’s *internal* domain separation (different IV constants) is necessary but not sufficient when the *application protocol* reuses the same tag field for both functions. The CNF analysis reveals three omitted clauses: the mode byte lacks integrity protection ($x_{\text{mode-bound}}$), the hash and AEAD tag spaces overlap in the message frame ($x_{\text{domain-sep}}$), and the firmware processes messages before completing verification ($x_{\text{verify-first}}$). MTSF’s Phase 4 (infrastructure dependency analysis) systematically detects such shared-component vulnerabilities.

Countermeasures: (1) Include the mode byte within the integrity-protected data: for AEAD messages, include **mode** as associated data; for hash-based messages, include **mode** in the hash input: $\text{tag} = \text{ASCON-Hash}(K_{\text{mac}} \parallel \text{mode} \parallel m)$. (2) Use distinct tag lengths or formats: AEAD tags at 128 bits, hash tags at 160 or 256 bits, preventing cross-interpretation. (3) Enforce strict verify-before-process: never act on message contents until integrity verification completes. (4) Use independent keys: $K_{\text{mac}} \perp K$ with proper KDF domain separation, e.g., $K_{\text{mac}} = \text{HKDF}(K_{\text{master}}, \text{"mac"})$, $K = \text{HKDF}(K_{\text{master}}, \text{"aead"})$.

Corollary 19.15 (ASCON Multi-Function Integrity Collapse). *Under application-layer domain confusion (Case b), $\text{Ask}(g_{\text{INT}}) = 1$: the adversary can inject messages that are processed by the gateway as AEAD-authenticated commands despite carrying only hash-based integrity tags, enabling arbitrary sensor data forgery and potentially command injection.*

Corollary 19.16 (ASCON Cross-Function Ping Failure). *For sessions where hash-based integrity and AEAD-based integrity are used interchangeably due to mode confusion, the session ping across function boundaries satisfies $\text{Ping}(\text{Session}_{\text{hash}}, \text{Session}_{\text{AEAD}}) = 0$: the two session types are not securely isolated, violating the fundamental domain separation requirement.*

Key Insight:

ASCON Specification vs. Application-Layer Vulnerability This attack does *not* break ASCON’s cryptographic design. ASCON-Hash and ASCON-128 are individually secure, and the internal IV-based domain separation is sound. The vulnerability arises entirely at the *application protocol layer*: the decision to reuse the same tag field for both hash-based and AEAD-based integrity, combined with an unprotected mode byte, creates a cross-function attack surface that the ASCON specification never intended to address.

In MTSF terms, the *primitive market* $\mathcal{M}_{\text{ASCON-spec}}$ is in equilibrium, but the *application market* $\mathcal{M}_{\text{ASCON-app}}$ is in full collapse. This is a *Type III CNF failure*: all individual clauses hold for each function in isolation, but the *composition* of two secure functions creates a new attack surface not covered by either function’s individual proof. MTSF’s CNF verification across function boundaries (Section 4) is specifically designed to detect such compositional failures.

The broader lesson: lightweight cryptographic standards that provide multiple functions from a shared primitive (ASCON, Xoodoo, PHOTON) create new application-layer attack surfaces that require explicit domain-separation protocols beyond the internal IV differentiation. MTSF’s infrastructure dependency analysis (Phase 4) and cross-session synthesis (Phase 3) systematically reveal these compositional vulnerabilities.

19.7 MTSF-Based Attack Prevention Framework

The five case studies demonstrate how MTSF’s systematic approach can reveal previously unknown vulnerabilities. We now formalise the conditions under which a protocol or cipher deployment resists unknown attacks, and provide a practical prevention framework based on these insights.

Definition 19.17 (Bid-Complete Protocol). *A protocol π with market $\mathcal{M}_\pi$ is bid-complete at security parameter λ if the exhaustive bid taxonomy $\{\mathbf{B}_1, \dots, \mathbf{B}_N\}$ constructed by the four-phase attack discovery process (Theorem 19.1) satisfies:*

$$\forall \mathbf{B}_i \in \{\mathbf{B}_1, \dots, \mathbf{B}_N\} : \Delta\text{Price}_i \leq \text{negl}(\lambda).$$

That is, every conceivable bidding round—including parsing bids, timing bids, cross-session bids, and infrastructure bids—adjusts the ask price by at most a negligible amount.

Definition 19.18 (CNF-Robust Protocol). *A protocol π with session-CNF $\varphi = \bigwedge_j x_j$ is CNF-robust if:*

1. **Parsing robustness:** φ remains well-defined under all possible message field boundary interpretations.
2. **Ordering robustness:** φ correctly evaluates to FALSE under any message reordering, duplication, or deletion by the adversary.
3. **Timing robustness:** φ correctly evaluates to FALSE under any timing manipulation, including race conditions and delayed delivery.
4. **Cross-session robustness:** For any pair of sessions $\text{Session}_i, \text{Session}_j$, the joint CNF $\varphi_i \wedge \varphi_j$ cannot be simultaneously satisfied under a dishonest trace that spans both sessions.

Theorem 19.19 (MTSF Attack Prevention Principle). *A protocol π satisfies $\text{Ask}(g, \lambda) \leq \text{negl}(\lambda)$ for all security goods g under the extended adversary model (including parsing, timing, cross-session, and infrastructure adversaries) if and only if π is both bid-complete (Theorem 19.17) and CNF-robust (Theorem 19.18).*

Proof. ($\Rightarrow$, Necessity.) Suppose π is not bid-complete. Then there exists a bidding round $\mathbf{B}_i$ with $\Delta\text{Price}_i > \text{negl}(\lambda)$. By the telescoping sum (Section 3), $\text{Ask}(g) \geq \Delta\text{Price}_i > \text{negl}(\lambda)$, contradicting the security assumption. Similarly, if π is not CNF-robust, one of the four robustness conditions fails, yielding a dishonest trace that satisfies the CNF—equivalently, a successful attack strategy for the buyer.

($\Leftarrow$, Sufficiency.) Suppose π is bid-complete and CNF-robust. By bid-completeness, every bidding round contributes at most $\text{negl}(\lambda)$ to the ask price. By the extended difference lemma (Theorem 3.7), the total ask price satisfies:

$$\text{Ask}(g) \leq \sum_{i=1}^N \Delta\text{Price}_i \leq N \cdot \text{negl}(\lambda)$$

Since N is polynomial in λ (the number of distinct bid types is bounded by the polynomial-size protocol description), the sum remains negligible: $\text{Ask}(g) \leq \text{negl}(\lambda)$. CNF-robustness ensures that no attack trace can satisfy the session-CNF, closing all remaining attack vectors. $\square$

★ Intuition:

The MTSF Prevention Mindset Traditional protocol design asks: “How can we prove this protocol is secure against known attacks?” MTSF-based prevention asks: “What are all the

ways this protocol could possibly fail, including ways we haven't thought of yet?"

This shift in mindset—from reactive security proof to proactive vulnerability assessment—is MTSF's core contribution to unknown attack detection. By systematically exploring the complete bidding space and CNF failure modes, we can identify potential vulnerabilities before they become real-world exploits.

The market metaphor makes this concrete: asking “is this protocol secure?” becomes “can any buyer afford to collapse this market?” The four-phase discovery process is the buyer's playbook for finding cheap bids. If every bid in the playbook is too expensive (negligible ΔPrice), the market is in equilibrium. If any bid is cheap (non-negligible ΔPrice), the market collapses, and we have found a new attack.

The Attack Prevention Principle (Theorem 19.19) characterises robustness of a *single* protocol. In practice, deployed systems compose many protocols (TLS 1.3 above TCP; Signal above HTTPS above TLS; WPA2 beneath IPsec). A natural question is: *does the bid-completeness and CNF-robustness of the components imply the same for their composition?* The market-merger theorem of Section 16.3 gives the additive ask-price bound; the following theorem extends that result to the bid-taxonomy and CNF structure, formalising the intuition that composition cannot *reduce* the attack surface but can, with discipline, preserve robustness.

Theorem 19.20 (Attack-Surface Monotonicity under Composition). *Let π_1, π_2 be two protocols whose markets $\mathcal{M}_1, \mathcal{M}_2$ are each bid-complete and CNF-robust with bid taxonomies $\mathcal{B}_1, \mathcal{B}_2$ and session-CNFs φ_1, φ_2 respectively. Let $\pi = \pi_1 \parallel \pi_2$ denote their concurrent composition with shared infrastructure I (CRS, PKI, clock). Then the composed protocol π is bid-complete and CNF-robust provided that the interface-robustness condition*

$$\mathcal{B}_{\text{interface}} \subseteq \mathcal{B}_1 \cup \mathcal{B}_2 \cup \mathcal{B}_I$$

holds, where $\mathcal{B}_{\text{interface}}$ is the set of bids that exploit the shared state or message flow between π_1 and π_2 but are not individually captured by $\mathcal{B}_1, \mathcal{B}_2$, or the infrastructure taxonomy $\mathcal{B}_I$. In this case:

$$\text{Ask}_\pi(g, \lambda) \leq \text{Ask}_{\pi_1}(g_1, \lambda) + \text{Ask}_{\pi_2}(g_2, \lambda) + \text{Ask}_I(\lambda) \leq \text{negl}(\lambda).$$

Conversely, if $\mathcal{B}_{\text{interface}} \not\subseteq \mathcal{B}_1 \cup \mathcal{B}_2 \cup \mathcal{B}_I$, then there exists a cross-protocol interface bid $\mathbf{B}^ \in \mathcal{B}_{\text{interface}}$ with $\Delta\text{Price}_{\mathbf{B}^*}$ potentially non-negligible, and the composed protocol may fail to be bid-complete even when each component is.*

Proof sketch. (Forward direction.) Enumerate the bid taxonomy of the composed market. Each bid either (i) targets only π_1 (captured by $\mathcal{B}_1$), (ii) targets only π_2 (captured by $\mathcal{B}_2$), (iii) targets shared infrastructure I (captured by $\mathcal{B}_I$), or (iv) straddles the interface between the two (captured by $\mathcal{B}_{\text{interface}}$). The interface-robustness hypothesis asserts (iv) $\subseteq$ (i) $\cup$ (ii) $\cup$ (iii). Applying Theorem 19.19 to each component and the market-merger theorem (Section 16.3) yields the additive ask-price bound. Each summand is negligible by assumption, so their sum (over a polynomial number of components) is negligible.

CNF-robustness of the composition follows because $\varphi = \varphi_1 \wedge \varphi_2 \wedge \varphi_I$, and conjunction of robust CNFs is robust under the standard closure properties (Theorem 19.18).

(Reverse direction.) Suppose $\mathbf{B}^* \in \mathcal{B}_{\text{interface}}$ is not captured by the component taxonomies. Then neither $\mathcal{B}_1, \mathcal{B}_2$, nor $\mathcal{B}_I$ bounds $\Delta\text{Price}_{\mathbf{B}^*}$, and the component-level proofs provide no guarantee. The TLS 1.3 session-resumption nonce collision (Section 19.2) and the ASCON-Hash cross-function domain confusion (Section 19.6) are concrete realisations of this failure mode: each component (the 1-RTT handshake; the ASCON permutation) is individually bid-complete, but the interface between standard operation and session resumption (respectively, between hash-mode and AEAD-mode deployment) introduces an uncaptured bid. $\square$

Corollary 19.21 (Cross-Protocol Residual Risk). *Even when each of k composed protocols $\pi_1, \dots, \pi_k$ is individually bid-complete and CNF-robust, the residual risk of the composition is bounded by:*

$$\text{Ask}_{\pi_1 \parallel \dots \parallel \pi_k}(g, \lambda) \leq \sum_{i=1}^k \text{Ask}_{\pi_i}(g_i, \lambda) + \text{Ask}_I(\lambda) + \sum_{i \neq j} \text{Ask}_{\text{interface}}(\pi_i, \pi_j, \lambda),$$

where the last term accounts for pairwise interface bids. This term is negligible if and only if every pairwise interface is covered by the component taxonomies (the hypothesis of Theorem 19.20). Consequently, the number of interface audits required for a k -protocol deployment grows as $\binom{k}{2} = O(k^2)$, which is the central scalability challenge of large-scale secure system design.

Key Insight:

Why Interface Audits Are the Hardest Part Theorem 19.21 formalises a folk observation well known to practitioners: *a system built from secure components is not automatically secure*. The classical illustration is the “secret-suffix” attack on CBC-MAC-then-Encrypt: each primitive (CBC mode; MAC) is secure in isolation, but their naïve composition admits a forgery. MTSF makes this precise: the failure lives in $\mathcal{B}_{\text{interface}}$, not in $\mathcal{B}_1$ or $\mathcal{B}_2$.

In a deployment with 10 composed protocols, there are $\binom{10}{2} = 45$ pairwise interfaces to audit. With 20 protocols, 190. With 50 protocols (realistic for cloud-native architectures: service mesh, mTLS, OIDC, OAuth2, SAML, TLS 1.3, QUIC, Noise, Signal, WireGuard, IKEv2, MACsec, WPA2, WPA3, SAE, MLS, FIDO2, DNSSEC, DoH, BGP with RPKI, etc.), 1,225. This quadratic scaling is the principal reason that large-scale cryptographic deployments experience security incidents at component-integration boundaries rather than inside hardened primitives.

MTSF’s practical guidance: budget interface audits at $O(k^2)$, document the bid-taxonomy overlap explicitly, and treat the composition CRS, PKI, clock, and entropy pool as first-class components with their own markets $\mathcal{M}_I$ —not as free resources assumed to be secure.

Definition 19.22 (MTSF Audit Certificate). *An MTSF audit certificate for a protocol π at security parameter λ is a tuple $\text{Cert}_\pi = (\mathcal{B}, \varphi, \text{Bounds}, \text{Interfaces})$ where:*

1. $\mathcal{B} = \{\mathcal{B}_1, \dots, \mathcal{B}_N\}$ is the enumerated bid taxonomy;
2. $\varphi = \bigwedge_j x_j$ is the session-CNF with each clause x_j annotated by the bid(s) it blocks;
3. $\text{Bounds} = \{(\mathcal{B}_i, \Delta\text{Price}_i, \text{justification}_i)\}_{i=1}^N$ provides per-bid price adjustments with hard-problem or statistical justification;
4. $\text{Interfaces} = \{(\pi_j, \mathcal{B}_{\text{interface}}(\pi, \pi_j))\}$ enumerates cross-protocol interfaces and the bids they introduce.

The certificate is valid if each $\Delta\text{Price}_i \leq \text{negl}(\lambda)$, each CNF clause is satisfied in the honest execution, and every interface bid is contained in the union of component taxonomies.

Proposition 19.23 (Composability of Audit Certificates). *If Cert_{π_1} and Cert_{π_2} are valid, and if $\text{Interfaces}_{\pi_1, \pi_2} \subseteq \mathcal{B}_1 \cup \mathcal{B}_2 \cup \mathcal{B}_I$, then the composition admits a valid certificate*

$$\text{Cert}_{\pi_1 \parallel \pi_2} = (\mathcal{B}_1 \cup \mathcal{B}_2 \cup \mathcal{B}_I, \varphi_1 \wedge \varphi_2 \wedge \varphi_I, \text{Bounds}_1 \sqcup \text{Bounds}_2 \sqcup \text{Bounds}_I, \text{Interfaces}_1 \cup \text{Interfaces}_2).$$

Audit certificates therefore form a monoid under composition whenever the interface-robustness hypothesis holds, providing a machine-checkable compositional discipline for large-scale deployments.

Analogy:

Audit Certificates as Building-Code Compliance Sheets Modern buildings ship with a stack of compliance certificates: structural engineer’s sign-off, fire safety audit, electrical inspection, plumbing inspection. Each certifies one subsystem. Putting two buildings next to each other on the same plot doesn’t automatically give you a valid two-building complex—you still need a *site-level* certificate that verifies the shared foundation, the shared utility connections, and the fire escape paths between buildings.

MTSF audit certificates work the same way. Cert_π certifies that π ’s market is in equilibrium in isolation. Composing π_1 and π_2 requires a new certificate that explicitly enumerates the *shared foundation* (I : CRS, PKI, clock, entropy) and the *utility connections* (Interfaces: shared session identifiers, overlapping nonce spaces, common hash inputs). Theorem 19.23 says: if the foundation and utilities are in order, the composite certificate is just the disjoint union of the component certificates. If not, the composite is invalid—no matter how strong the component certificates look individually.

Key Insight:

Comparison with Existing Secure-Design Methodologies The MTSF Attack Prevention Framework occupies a distinctive niche in the landscape of secure-design methodologies. The following comparison highlights what MTSF adds and where existing methodologies retain advantages.

Methodology	Scope	Output	Relationship to MTSF
STRIDE / LINDUN threat modelling	Qualitative attack enumeration	Unranked risk list	MTSF quantifies each STRIDE category as a bid with ΔPrice ; refines qualitative risk into auditable price adjustments.
Attack trees (Schneider)	Tree of refinements from root attack to leaves	Logical tree, optional cost annotations	MTSF bid taxonomy <i>is</i> the attack-tree leaf set; Bounds provides the cost annotations that attack trees leave informal.
Microsoft SDL / OWASP ASVS	Implementation-level checklists	Compliance sign-off	MTSF operates at specification level; complements rather than replaces implementation audits.
Tamarin / ProVerif symbolic verification	Protocol-level, unbounded sessions	Symbolic trace or “secure” verdict	MTSF adds quantitative bounds and a composition discipline that symbolic tools lack natively.
CryptoVerif computational verification	Machine-checked game hops	Sequence-of-games proof	MTSF subsumes the proof structure and adds the bid-completeness certificate as a deployability artefact.
Formal-methods frameworks (Isabelle/HOL, Coq)	Machine-checked theorems, any scope	Proof script	MTSF provides the theorem statements; Isabelle/HOL and Coq can mechanise the proofs. The two are orthogonal.

MTSF’s distinguishing contribution is the combination of: (i) a quantitative ask-price output, (ii) a unified treatment of security and insecurity, (iii) a machine-checkable interface-robustness condition for composition, and (iv) a practitioner-accessible session-CNF audit worksheet. No single prior methodology provides all four simultaneously. MTSF is therefore best understood as a *synthesis* layer that knits the quantitative rigour of game-based proofs,

the composition guarantees of UC/GUC, the exhaustive exploration of symbolic verification, and the systematic attack-tree enumeration of STRIDE, into a single auditable workflow.

Practical Implementation of MTSF Attack Discovery

Protocol designers and security analysts can apply MTSF attack discovery using the following systematic process:

Phase 1: Bidding Space Enumeration

- For each protocol message, enumerate all possible interpretation ambiguities (cf. Case Study II: ISO 9798-3 parsing attack)
- For each cryptographic operation, identify alternative computational paths
- For each timing constraint, explore race condition possibilities (cf. Case Study III: WPA2 group key race)
- For each shared resource, analyse resource exhaustion and collision scenarios (cf. Case Study I: TLS 1.3 nonce collision)

Phase 2: CNF Stress Testing

- Systematically negate each CNF clause and attempt to construct satisfying attacks
- Explore partial clause satisfaction (some conjuncts true, others false)—as demonstrated in all five case studies, where some clauses pass even under attack
- Test CNF robustness under message reordering, duplication, and deletion
- Verify CNF behaviour under extreme parameter values and edge cases

Phase 3: Cross-Session Synthesis

- Model multi-session attack vectors that span protocol instances
- Analyse session state leakage and correlation patterns
- Test protocol behaviour under rapid session establishment/teardown
- Examine long-term state accumulation and persistence effects
- Verify that $\text{Ping}(\text{Session}_i, \text{Session}_j) = 1$ for all session pairs (Section 5.2)

Phase 4: Infrastructure Attack Modelling

- Model shared infrastructure (PKI, DNS, NTP) as additional bidding rounds [59]
- Analyse cascading failure modes across dependent protocols
- Test infrastructure resource competition and allocation conflicts
- Verify graceful degradation under partial infrastructure compromise

Key Insight:

Summary: Five Attacks, Five Lessons

	Case TLS 1.3	I:	Case ISO 9798-3	II:	Case WPA2	III:	Case Grain-128a	IV:	Case V: AS- CON
Target	Session resump- tion		Entity auth.		Group key h/s		Stream cipher		Hash/AEAD
Attack	Nonce collision		Parsing confu- sion		Race condition		Related-IV corr.		Domain confu- sion
Phase	Phase 1		Phase 2		Phase 3		Phase 3+4		Phase 4
CNF fail	x_{nonce}		x_{parse}		x_{ack}		$x_{\text{IV-ent.}}$		$x_{\text{mode}}, x_{\text{dom.}}$
Type	Impl. dev.		Type II		Type II		Type II		Type III (comp.)
Ask	$\geq q^2/2^{33}$		$= 1$		$\geq 1 - (1 - \frac{t}{T})^n$		$\geq q^2 \cdot 2^{-73}$		$= 1$
Verdict	Partial		Full		Partial-full		Partial-full		Full (app.)
Lesson	Spec $\neq$ impl.		Parse $\neq$ sem.		Sync $\neq$ atomic		Rand. $\neq$ low- ent.		Internal $\neq$ app. sep.

Each attack was invisible to traditional analysis because it operated outside the assumed adversary model. MTSF’s systematic bid taxonomy expansion and CNF stress testing revealed the vulnerabilities by exploring the *complete* attack surface, including implementation deviations, parsing ambiguities, synchronisation assumptions, infrastructure entropy deficiencies, and application-layer domain separation failures.

We now elaborate how the MTSF attack prevention framework specialises to each major cryptographic domain. While Theorem 19.19 provides the overarching theoretical guarantee, the concrete verification procedures differ substantially across primitives, block ciphers, hash functions, stream ciphers, and protocols. Each domain requires its own bid-completeness checklist and CNF-robustness criteria.

19.7.1 Prevention for Cryptographic Primitives

For primitives (ECDSA, ML-KEM, ML-DSA, SLH-DSA, FN-DSA, HMAC, AEAD), bid-completeness and CNF-robustness translate into concrete algebraic and implementation-level conditions.

Definition 19.24 (Primitive Bid-Completeness Checklist). *A primitive Π with security reduction to hard problem $\mathcal{P}$ is bid-complete if the following bids are each verified to contribute at most $\text{negl}(\lambda)$ to the ask price:*

1. **Standard reduction bids:** *The security reduction from the security game (IND-CCA2, EUF-CMA, SUF-CMA) to $\mathcal{P}$ is tight or has a quantified, acceptable tightness gap. For ECDSA, the forking lemma reduction loses a factor of q_H ; for ML-KEM, the FO-transform reduction in the QROM loses a factor of $q^{1/2}$ via the O2H lemma [81].*
2. **Multi-user bids:** *The multi-user security bound is explicitly quantified. For a primitive deployed across N users, the ask price must satisfy $\text{Ask}(g) \leq N \cdot \varepsilon_{\text{single}}(\lambda) \leq \text{negl}(\lambda)$. This requires $N \cdot \varepsilon_{\text{single}} \ll 1$, constraining the maximum deployment scale.*
3. **Side-channel bids:** *Constant-time implementation is verified (no secret-dependent branches or memory accesses). For lattice-based schemes, NTT operations and rejection sampling must execute in data-independent time.*
4. **Fault-injection bids:** *Output validation checks (e.g., verify-after-sign for ECDSA and ML-DSA) are implemented to detect fault-induced errors before releasing potentially leaking outputs.*
5. **Nonce and randomness bids:** *Nonce generation uses a CSPRNG with sufficient entropy. For deterministic schemes (EdDSA, ML-DSA), the deterministic nonce derivation is verified to produce uniformly distributed values. For hedged schemes, the hedging mechanism correctly combines randomness with message hash.*

6. **Algebraic structure bids:** For lattice-based schemes, parameter selection follows the Core-SVP methodology with a sufficient security margin against the best-known lattice reduction algorithms (BKZ 2.0). For elliptic curve schemes, curve parameters resist known special-structure attacks (MOV reduction, anomalous curves, small-subgroup attacks).

CNF-robustness for primitives requires that the verification algorithm rejects all malformed inputs: invalid curve points (ECDSA), out-of-range polynomial coefficients (ML-KEM, ML-DSA), and truncated or padded hash inputs (HMAC). The input validation CNF clause must be *independently* verified, not assumed as a precondition of the security proof.

19.7.2 Prevention for Block Ciphers

For block ciphers (AES, PRESENT, Serpent), bid-completeness requires exhaustive coverage of all known cryptanalytic families, and CNF-robustness ensures that the cipher’s security margin is maintained across all deployment contexts.

Definition 19.25 (Block-Cipher Bid-Completeness Checklist). *A block cipher E with R rounds, block size n , and key size k is bid-complete if:*

1. **Differential bid:** *The maximum expected differential probability (MEDP) over R rounds satisfies $\text{MEDP}_R \leq 2^{-(k+m)}$ for a security margin $m \geq 0$. For AES-128 ($R = 10$), the 4-round provable bound $\text{MEDP}_4 \leq 2^{-150}$ extends to full rounds with overwhelming margin.*
2. **Linear bid:** *The maximum expected linear probability (MELP) satisfies $\text{MELP}_R \leq 2^{-(k+m)}$. For AES, the wide-trail design strategy provides this bound by construction.*
3. **Algebraic bid:** *The algebraic degree of the round function grows sufficiently fast with rounds. After r rounds of AES, the algebraic degree is $\min(7^r, 2^8 - 1)$, reaching maximum by round 4. The system of multivariate equations over $\text{GF}(2^8)$ has complexity exceeding 2^k for full-round AES.*
4. **Related-key bid:** *The key schedule provides sufficient diffusion. For AES-256, the known related-key attack on 9/14 rounds [25] does not extend to full rounds; the bid price remains negligible.*
5. **Meet-in-the-middle bid:** *The biclique improvement over brute force is at most a constant factor (e.g., $2^{126.1}$ vs. 2^{128} for AES-128 [28]). This margin is explicitly quantified and deemed acceptable.*
6. **Quantum bid:** *Under Grover’s algorithm, the effective key strength is $k/2$. For AES-256, this yields 2^{128} quantum queries—still secure. For AES-128, Grover reduces to 2^{64} ; the designer must decide if post-quantum resistance at this level is acceptable.*

CNF-robustness for block ciphers extends beyond the cipher itself to the mode of operation. Each mode introduces additional CNF clauses:

- **CTR mode:** Clause $x_{\text{nonce-unique}}$ requires unique (nonce, counter) pairs; violation enables two-time-pad attacks.
- **CBC mode:** Clause $x_{\text{IV-unpredictable}}$ requires unpredictable IVs; violation enables chosen-plaintext attacks [4].
- **GCM mode:** Clause $x_{\text{nonce-unique}}$ is critical—nonce reuse in GCM leaks the authentication key H , enabling universal forgery.
- **Side-channel clause:** $x_{\text{const-time}}$ requires that the implementation does not leak key-dependent timing information. For AES, this means either hardware AES-NI instructions or verified constant-time bitsliced implementations.

19.7.3 Prevention for Hash Functions

For hash functions (SHA-3/Keccak, BLAKE3, ASCON-Hash), bid-completeness requires bounding the best-known attacks against the underlying permutation and the construction mode, while CNF-robustness ensures correct domain separation and parameter selection.

Definition 19.26 (Hash-Function Bid-Completeness Checklist). *A hash function H based on a b -bit permutation π with capacity c and rate r ($b = c + r$) is bid-complete if:*

1. **Collision resistance bid:** *The best-known collision attack on H has complexity $\geq 2^{c/2}$. For SHA3-256 ($c = 512$), this yields a collision bound of 2^{256} —matching the output-length bound.*
2. **Preimage resistance bid:** *The best-known preimage attack has complexity $\geq 2^c$ (for sponge constructions) or $\geq 2^n$ (for Merkle-Damgård, where n is the output length).*
3. **Permutation distinguishing bid:** *The best-known distinguisher for the full permutation π requires $\geq 2^{c/2}$ queries. For Keccak- $f[1600]$ with 24 rounds, the best distinguisher operates on reduced rounds (≤ 8) and does not extend to full rounds.*
4. **Internal collision bid:** *For sponge constructions, the internal state collision cost is $2^{c/2}$. The capacity c must be chosen such that $2^{c/2} \geq 2^s$ where s is the target security level.*
5. **Length-extension bid:** *Sponge constructions are immune; Merkle-Damgård constructions require HMAC or truncation-based mitigations. The bid price is 0 for sponge-based H and potentially 1 for unprotected MD-based H .*
6. **Multi-target bid:** *With T targets, the effective preimage cost is $2^c/T$ (sponge) or $2^n/T$ (MD). For SHA3-256 with $T = 2^{64}$ targets: effective cost = 2^{192} , still secure.*
7. **Quantum bid:** *Grover’s algorithm reduces collision resistance to $2^{c/3}$ (BHT algorithm) and preimage resistance to $2^{c/2}$. SHA3-256’s capacity $c = 512$ provides 2^{170} quantum collision resistance.*

CNF-robustness for hash functions centres on correct instantiation and domain separation:

- **Domain separation clause:** When the same hash function serves multiple purposes (hashing, KDF, MAC, commitment, PRF), each usage must employ a distinct domain separator. SHA-3 provides this via the suffix bits (01 for SHA-3, 1111 for SHAKE); custom constructions must explicitly encode the purpose.
- **Output truncation clause:** If the hash output is truncated (e.g., using SHA3-256 but keeping only 128 bits), the collision and preimage bounds must be recalculated for the truncated length, not the full output.
- **Absorption completeness clause:** The sponge must absorb the *entire* input including padding before squeezing. Implementation errors that skip the final padding block or fail to process the last incomplete block can reduce effective capacity.
- **Reduced-round clause:** If a reduced-round variant is used for performance (e.g., 12-round KangarooTwelve vs. 24-round SHA-3), the reduced round count must still exceed the attack frontier for all bid types. Currently, the best attacks on Keccak- $f[1600]$ reach 8 rounds; a 12-round instantiation provides a $1.5\times$ margin.

19.7.4 Prevention for Stream Ciphers

For stream ciphers (Grain-128a, ChaCha20, Trivium), bid-completeness requires bounding all known cryptanalytic avenues against the internal state and output function, while CNF-robustness centres on operational constraints—particularly nonce/IV management—that are uniquely critical for this cipher class.

Definition 19.27 (Stream-Cipher Bid-Completeness Checklist). *A stream cipher SC with key size k , IV size v , and internal state size s is bid-complete if:*

1. **State recovery bid:** *The best-known state-recovery attack given L keystream bits has complexity $\geq 2^{s/2}$ (or $\geq 2^k$ if $k < s/2$). For Grain-128a ($s = 256, k = 128$): the combined LFSR/NFSR state admits no algebraic recovery below 2^{128} for the full cipher. For ChaCha20 ($s = 512, k = 256$): no shortcut below 2^{256} . For Trivium ($s = 288, k = 80$): the effective security is bounded by $k = 80$, making Trivium unsuitable for applications requiring ≥ 128 -bit security.*
2. **Key recovery bid:** *The best-known key-recovery attack has complexity $\geq 2^k$. This includes inversion of the initialisation function, which must provide at least k -bit security against known-IV attacks. For Grain-128a, the 256-round nonlinear initialisation ensures this; for Trivium, the 1152-round initialisation provides margin against 80-bit key recovery.*
3. **Distinguishing bid:** *The best-known distinguisher requires $\geq 2^k$ keystream bits (or $\geq 2^{s/2}$ bits, whichever is smaller). For Grain-128a, the nonlinear output function h with algebraic immunity 4 ensures no linear or algebraic distinguisher below 2^{128} bits. For ChaCha20, the 20-round ARX core produces output indistinguishable from random with margin $> 2^{200}$.*
4. **Algebraic attack bid:** *The algebraic immunity of the output Boolean function is verified. The system of multivariate equations over $\text{GF}(2)$ modelling L consecutive output bits has a solving complexity exceeding 2^k . For Grain-128a's h (algebraic immunity 4, 9 input variables): Gröbner basis attacks require $\geq 2^{128}$ operations on the full system.*
5. **Cube attack bid:** *The number of initialisation rounds exceeds the cube-attack frontier by a sufficient security margin. For Trivium (1152 rounds, frontier at ~ 842 rounds): margin = 310 rounds ($\sim 27\%$). For Grain-128a (256 rounds, frontier dependent on conditional cube techniques [80]): the margin must be explicitly quantified and deemed acceptable.*
6. **TMTO bid:** *The trade-off bound $T \cdot M \cdot D \geq 2^s$ is verified for the internal state size s . For all three ciphers, $s \geq 2k$ (Grain-128a: $s = 256 = 2 \cdot 128$; ChaCha20: $s = 512 = 2 \cdot 256$; Trivium: $s = 288 = 3.6 \cdot 80$), ensuring that TMTO provides no advantage over exhaustive key search.*
7. **Rotational bid (ARX only):** *For ARX-based stream ciphers (ChaCha20), the rotational distinguishing probability over the full round function is verified to be negligible. For ChaCha20: $(2^{-1.415})^{320} \leq 2^{-452.8}$, which is doubly negligible.*
8. **Fault-injection bid:** *The implementation includes output validation or redundancy checks to detect fault-induced errors. For hardware implementations (Grain-128a, Trivium), dual-rail or parity-based fault detection is verified. For software implementations (ChaCha20), instruction-level redundancy or comparison-after-computation guards against glitch attacks.*
9. **Quantum bid:** *Under Grover's algorithm, the effective key strength is $k/2$. For ChaCha20 ($k = 256$): 2^{128} quantum queries. For Grain-128a ($k = 128$): 2^{64} quantum queries—potentially concerning for long-term post-quantum applications. For Trivium ($k = 80$): 2^{40} —completely inadequate for post-quantum security.*

CNF-robustness for stream ciphers is dominated by the nonce/IV management clause, which is the single most critical operational requirement:

- *IV uniqueness clause* ($x_{\text{IV-unique}}$): The most critical clause for any stream cipher. Reusing a (key, IV) pair produces identical keystreams, enabling a trivial two-time-pad attack with $\text{Ask} = 1$. The CNF must enforce: $\forall i \neq j: (\text{key}_i, \text{IV}_i) \neq (\text{key}_j, \text{IV}_j)$. For counter-based nonces (ChaCha20 in TLS 1.3), the counter monotonicity must be verified; for random nonces, the IV space must be large enough to make birthday collisions negligible (96-bit random nonces provide $\leq 2^{-32}$ collision probability for 2^{32} sessions).
- *Keystream length clause* ($x_{\text{len-limit}}$): The maximum keystream length per (key, IV) pair must not exceed $2^{s/2}$ bits (internal state birthday bound) or 2^c blocks (counter overflow), whichever is smaller. For ChaCha20, the 32-bit block counter limits output to 2^{38} bytes (≈ 256 GiB) per nonce; for Grain-128a, the state-birthday bound of 2^{128} bits ($\approx 2^{124}$ bytes) is the limiting factor. Exceeding these limits must be flagged as a CNF violation.
- *Initialisation completeness clause* ($x_{\text{init-complete}}$): The cipher must execute all specified initialisation rounds before producing keystream. Early termination of initialisation (for performance) directly weakens security, as demonstrated by cube attacks on reduced-round Trivium and Grain-128a. The clause requires: “initialisation round count $\geq R_{\text{spec}}$ ” where R_{spec} is the specification-mandated count.
- *Key erasure clause* ($x_{\text{key-erase}}$): After keystream generation is complete or the session ends, the internal state (which contains key-derived material) must be securely erased from memory. Failure to erase allows cold-boot or memory-dump attacks to recover the state and derive all past and future keystream for that session. For ChaCha20 in ephemeral-key protocols (TLS 1.3), the key and state are erased after each record; for Grain-128a in embedded devices, secure erasure may require explicit hardware support.
- *Synchronisation clause* (x_{sync}): For synchronous stream ciphers (all three: Grain-128a, ChaCha20, Trivium), the sender and receiver must maintain exact synchronisation of the keystream position. Loss of synchronisation (due to dropped packets, bit insertion/deletion, or desynchronisation attacks) causes all subsequent decryption to fail. In network protocols, this is mitigated by using the stream cipher within an AEAD construction (ChaCha20-Poly1305) with explicit per-record nonces, eliminating the synchronisation requirement at the cipher level.

19.7.5 Prevention for Protocols

For protocols (TLS 1.3, Signal, ISO/IEC 11770-3, WPA2, Telegram MTProto), bid-completeness and CNF-robustness are the most demanding because protocols compose multiple primitives, involve multiple parties, and operate over untrusted networks.

Definition 19.28 (Protocol Bid-Completeness Checklist). *A protocol π operating with n parties over an untrusted network is bid-complete if the following bid categories are each verified:*

1. **Cryptographic primitive bids:** *Each primitive used within π is itself bid-complete per Theorem 19.24. The composition of primitives preserves security: e.g., TLS 1.3’s hybrid key schedule combines DHE and PSK derivations without introducing cross-term weaknesses.*
2. **State-machine manipulation bids:** *Every state transition in π ’s state machine is explicitly enumerated. Unexpected transitions (receiving a handshake message in the data phase, re-receiving a previously processed message) are handled by returning to a safe state or aborting. The KRACK lesson: every message replay must either be rejected or produce an idempotent state change.*

3. **Parsing and encoding bids:** All message fields use unambiguous encoding (TLV, ASN.1 DER, or explicit length prefixes). The ISO 9798-3 lesson: raw concatenation without field delimiters creates parsing ambiguity that enables identity confusion. The encoding format itself must be verified for consistency (no BER/DER confusion, no optional-field omission ambiguity).
4. **Timing and synchronisation bids:** All timing assumptions are explicitly stated and verifiable. Clock-dependent operations (certificate validity, session ticket lifetime, nonce freshness windows) include tolerance margins. The WPA2 lesson: multi-party synchronisation cannot be assumed; it must be cryptographically enforced via authenticated acknowledgments.
5. **Downgrade and negotiation bids:** Version and parameter negotiation is cryptographically bound to the handshake transcript. TLS 1.3's downgrade sentinel and Finished MAC binding serve as exemplars; protocols without equivalent bindings are vulnerable to downgrade-to-weaker-cipher attacks.
6. **Resumption and caching bids:** Session resumption mechanisms derive unique per-session secrets. The TLS 1.3 lesson: ticket nonces must be generated via per-connection counters, not derived from observable or predictable values. Cached state must be invalidated upon key rotation or compromise detection.
7. **Cross-protocol bids:** If keys or certificates are shared across protocols, cross-protocol message reinterpretation must be prevented via domain separation or context-binding [59]. TLS 1.3 achieves this via the handshake-context hash in the key derivation; protocols that reuse raw signatures across contexts are vulnerable.
8. **Forward-secrecy and post-compromise bids:** Ephemeral key material is generated fresh per session and erased after key derivation. Long-term key compromise must not retroactively compromise past sessions (forward secrecy) and must be recoverable in future sessions (post-compromise security, as in Signal's Double Ratchet).

CNF-robustness for protocols requires five categories of clause verification:

- *Authentication clauses:* Each party's identity is bound to the session via a cryptographic proof (signature, MAC) that cannot be transferred to a different session or party. Clause inversion: "What if the identity binding is removed?" reveals impersonation attacks (Needham-Schroeder, Section 13.4).
- *Freshness clauses:* Each session uses fresh randomness that is cryptographically bound to the session transcript. Clause inversion: "What if nonces are reused?" reveals replay attacks (TLS 1.3 0-RTT, Section 13.8.4).
- *Transcript-binding clauses:* The handshake transcript is integrity-protected and bound to the derived session key. Clause inversion: "What if the transcript is modified?" reveals transcript manipulation and downgrade attacks (Section 13.8.5).
- *Ordering clauses:* Messages are processed in the correct sequence; out-of-order or duplicated messages are rejected or handled idempotently. Clause inversion: "What if messages are reordered?" reveals state-machine manipulation attacks (KRACK).
- *Synchronisation clauses:* Multi-party state transitions are atomically committed. Clause inversion: "What if one party updates while another retains old state?" reveals race-condition attacks (WPA2 group key, Theorem 19.8).

Unified Prevention Verification Workflow

Protocol designers can systematically apply the domain-specific prevention framework using the following workflow:

Step 1: Domain Classification. Classify each component of the system into one of the five domains (primitive, block cipher, hash function, stream cipher, protocol). A typical TLS 1.3 deployment involves all five: ECDHE/ML-KEM (primitive), AES-GCM (block cipher in AEAD mode) or ChaCha20-Poly1305 (stream cipher in AEAD mode), SHA-384 (hash function), and the TLS 1.3 handshake (protocol).

Step 2: Per-Domain Bid-Completeness Audit. For each component, verify bid-completeness using the appropriate domain-specific checklist (Theorems 19.24 to 19.28). Document each bid type, the associated ΔPrice , and whether it is negligible.

Step 3: Per-Domain CNF-Robustness Audit. For each component, construct the session-CNF and systematically invert each clause. Verify that every clause inversion either produces an inconsistent system (attack impossible) or is already covered by a verified bid (attack priced as negligible).

Step 4: Composition Verification. Verify that the composition of components preserves security: the protocol’s market (Section 16) inherits the ask prices from its constituent primitive, cipher, and hash markets, with composition overhead bounded by the extended difference lemma.

Step 5: Cross-Domain Interaction Audit. Check for cross-domain interactions: Does the protocol’s use of the hash function respect domain separation? Does the block cipher’s or stream cipher’s mode of operation satisfy the protocol’s nonce-uniqueness requirements? Does the stream cipher’s IV management respect the keystream length limits and counter overflow bounds? Does the primitive’s multi-user security bound accommodate the protocol’s deployment scale?

This five-step workflow provides a concrete, auditable process for verifying the MTSF attack prevention principle (Theorem 19.19) across the full system stack.

The unknown attack detection capability of MTSF represents a significant advancement over traditional security analysis. By providing a systematic methodology for exploring the complete attack surface of cryptographic protocols, MTSF enables proactive security assessment that can identify vulnerabilities before they are discovered by adversaries. The five case studies presented here demonstrate that even well-studied, standardised protocols, ciphers, and hash functions may harbour subtle vulnerabilities that become apparent only through MTSF’s comprehensive bidding-round and CNF analysis framework. The formal prevention principle (Theorem 19.19) provides a concrete, verifiable criterion—bid-completeness and CNF-robustness—that protocol designers can use to systematically harden their designs against unknown attacks.

20 Related Work

20.1 Game-Based Security Proofs

The foundations of game-based proofs were laid by Bellare and Rogaway [16], who introduced the sequence-of-games methodology, and by Shoup [79], who formalised the *difference lemma*: if two games $\mathbf{B}_k$ and $\mathbf{B}_{k+1}$ differ only on event F , then $|\Pr[\mathbf{B}_k = 1] - \Pr[\mathbf{B}_{k+1} = 1]| \leq \Pr[F]$. This has become the central tool for bounding game-hop probabilities. Our *extended difference lemma* (Theorem 3.7) strictly generalises Shoup’s single-failure version by capturing m simultaneous failure events $F_1, \dots, F_m$ via $\Pr[\bigcup_i F_i]$ with inclusion-exclusion tightening. Whereas Shoup’s lemma requires artificially splitting multi-failure hops into multiple sequential games, our lemma handles them in a single step, tightening bounds and reducing proof length.

Bellare and Neven [14] introduced the general forking lemma for extracting two consistent

forgeries from a signature adversary, which we use in the ECDSA proof (Section 9.1). Bellare and Rogaway’s random oracle model [15] underpins our ROM-based proofs; the QROM extension (Section 14) addresses the limitation that classical ROM proofs do not account for quantum superposition queries.

The market-theoretic language we introduce—“seller,” “buyer,” “ask price,” “bidding round,” “equilibrium,” and “collapse”—has no precedent in the cryptographic proof literature. All prior game-based work uses standard game/adversary/challenger terminology without the economic framing. MTSF is the first framework to unify proof methodology with market theory.

20.2 Universal Composability and GUC

Canetti’s Universal Composability (UC) framework [31] provides the gold standard for composable security: a protocol proven UC-secure remains secure when run concurrently with arbitrary other protocols. UC proofs require constructing an ideal functionality $\mathcal{F}$ and a simulator $\mathcal{S}$ that makes the real world indistinguishable from the ideal world for any environment $\mathcal{Z}$. In MTSF, the environment $\mathcal{Z}$ is recast as a *market regulator*, the simulator $\mathcal{S}$ as a *market arbitrageur*, and the ideal functionality as the *seller’s security goods*.

Generalised UC (GUC) [32] extends UC to allow shared state (global common reference strings, public-key infrastructure). In MTSF, GUC shared functionality corresponds to *market infrastructure*—the PKI, key registration systems, and trust anchors that all parties can access. Camenisch, Manulis, and Neven [30] introduced CNF-based session correctness verification within the GUC framework, requiring sessions to satisfy a conjunction of Boolean clauses derived from the protocol’s message structure. We directly build on their CNF formalism, extending it with (a) a canonical five-phase verification algorithm (Algorithm 1), (b) an easy four-column manual worksheet for practitioner use, (c) integration with session pinging for unbounded coverage, and (d) a “dishonest-trace unsatisfiability” proof strategy that replaces ad hoc protocol analysis.

20.3 Formal Verification of Protocols

ProVerif [26] is the leading automated tool for protocol security verification, based on the applied pi-calculus. It can automatically prove secrecy, authentication, and other properties for unbounded sessions, but does not produce numerical advantage bounds. Tamarin [66] uses multiset rewriting rules and provides user-guided proofs of complex protocols (including TLS 1.3 and 5G AKA), again without quantitative bounds. CryptoVerif [27] bridges the gap by producing machine-checked game-based proofs with concrete bounds, but requires significant manual guidance and does not support the UC/GUC framework natively.

MTSF’s *session pinging* mechanism (Section 5) provides unbounded session coverage comparable to ProVerif’s symbolic analysis, while maintaining quantitative advantage bounds comparable to CryptoVerif’s game-based approach. This combination—unbounded + quantitative—is unique to MTSF.

The EasyCrypt proof assistant [11] supports machine-checked cryptographic proofs in the game-based style. Mechanising MTSF proofs in EasyCrypt is an important avenue for future work, particularly for the extended difference lemma and QROM case studies.

20.4 TLS 1.3 and Protocol Security Analysis

Transport Layer Security 1.3 [76] represents the most thoroughly analysed deployed cryptographic protocol in history. Its design was informed by a decade of attacks on TLS 1.2 [67, 4, 10], and it underwent extensive formal analysis both before and after standardisation.

The most comprehensive formal analysis of TLS 1.3 is by Dowling, Fischlin, Günther, and Stebila [45], who prove security of the full TLS 1.3 handshake in the ACCE (Authenticated and Confidential Channel Establishment) model [56]. Their proof covers the 1-RTT handshake,

resumption via PSK, and the 0-RTT mode, demonstrating that the 1-RTT handshake achieves authenticated key exchange with forward secrecy, while 0-RTT provides confidentiality only against *passive* adversaries (not active replay). A parallel analysis by Cremers, Hövelmanns, and Rausch [36] using symbolic verification in Tamarin confirmed the handshake properties including downgrade resilience. Our MTSF analysis (Section 13.8) is complementary: we provide the first *market-theoretic* formalisation of TLS 1.3, casting the ACCE proof as a ten-bidding-round chain and formalising the 0-RTT replay collapse and downgrade attack as explicit market collapse theorems.

The HKDF key schedule [62, 61] is central to TLS 1.3’s security. Krawczyk [61] proved HKDF’s security as a PRF under the assumption that HMAC is a PRF, and as an indistinguishable random oracle extractor under weaker assumptions. In our MTSF proof (Section 13.8.3), HKDF’s PRF security accounts for three separate price adjustments (one per extract/expand step in the HS, MS, and CATS derivations), each bounded by $\text{Adv}_{\text{HKDF}}^{\text{PRF}}$.

The 0-RTT replay vulnerability is acknowledged explicitly in RFC 8446, Section 8, and in Fischlin, Günther, and Marson [46], who formally define the “1-RTT-then-0-RTT” security model and prove that 0-RTT achieves confidentiality against passive adversaries but provides no replay protection. Our MTSF formulation (Theorem 13.25) provides a direct market-collapse proof: the replay bid is free ($\text{Ask} = 1$) without server-side anti-replay, and we identify the missing CNF clause ($\varphi^{\text{fresh-early}}$) as the root cause.

Downgrade attacks on TLS have a long history, including POODLE [67] (forcing SSLv3 negotiation), FREAK [10] (forcing export-grade RSA), and Logjam [1] (forcing 512-bit DHE). TLS 1.3 addresses all of these by requiring the RFC 8446 downgrade sentinel in `ServerHello.Random` when a TLS 1.3-capable server negotiates TLS 1.2. Our MTSF downgrade theorem (Theorem 13.26) provides the first explicit market-collapse proof for the version-stripping bid, and formalises the sentinel’s role as a CNF clause that converts the free version-stripping bid into a detectable abort.

20.5 Needham–Schroeder Protocol

The Needham–Schroeder Public-Key Protocol [73] was one of the first formal cryptographic protocols for mutual authentication using public-key cryptography. It was believed secure for nearly 17 years until Lowe [63] discovered a man-in-the-middle attack using the FDR model checker. Lowe’s fix—adding the responder’s identity to message 2—has since been considered a classic lesson in protocol design. In MTSF (Section 13.4), we provide the first *market-theoretic* characterisation of this attack: the NS protocol’s CNF is *satisfiable under dishonest traces* (a CNF design failure), the market collapses ($\text{Ask}(g_{\text{mutual}}) = 1$), and Lowe’s fix is interpreted as adding an identity-binding clause that makes dishonest traces provably UNSAT.

20.6 Economics of Security

* Simple Terms: How MTSF Differs from Previous Economic Security Models

Several researchers have previously drawn connections between economics and security:

- **Anderson (2001)** argued that security failures are often economic, not technical—organisations under-invest in security because the costs of breaches are externalised.
- **Gordon & Loeb (2002)** modelled optimal security investment: spend on security until the marginal cost exceeds the marginal benefit.
- **Herley (2009)** showed that many security practices (e.g., complex password requirements) are not economically rational for users.

MTSF is fundamentally different: These prior works treat security as an *economic*

decision (how much to invest). MTSF treats security proofs themselves as *economic objects* (markets, bids, prices). The difference is analogous to:

- Prior work: “Should we invest \$1M in better encryption?” (cost-benefit analysis).
- MTSF: “The encryption’s security guarantee has an ask price of 2^{-128} ” (the proof itself speaks the language of economics).

MTSF does not replace economic reasoning about security investment; it provides a rigorous, unified language for the *technical* security guarantees that inform those investment decisions.

Anderson [5] pioneered the economic analysis of information security, identifying incentive misalignments (e.g., software vendors externalising security costs onto users) as a major cause of insecurity. Grossklags *et al.* [49] extended this line to study user behaviour in security games. These works are about *incentive economics*—why people and organisations make insecure choices. MTSF is fundamentally different: it is about *proof methodology*, not incentive alignment. The market framing in MTSF is purely mathematical—“price,” “equilibrium,” and “collapse” are formal terms for “advantage bound,” “security,” and “insecurity.” MTSF is, to our knowledge, the first *cryptographic proof framework* that uses market-theoretic language to unify security definitions, proofs, and insecurity demonstrations.

20.7 Telegram MTPROTO

Albrecht *et al.* [3] conducted the first comprehensive cryptographic analysis of MTPROTO 2.0, discovering several theoretical attacks including a chosen-ciphertext distinguisher and a length-hiding failure. Jakobsen and Orlandi [57] analysed the original MTPROTO, finding multiple cryptographic weaknesses. Our analysis (Section 13.9) is complementary: we focus specifically on the *server salt extraction attack* and provide both a formal disproof (showing what collapses in MTSF language) and a formal proof for a remediated protocol (RMTP). This dual proof/disproof within a single framework is not provided in prior work.

20.8 HMAC and Authenticated Encryption

Bellare, Canetti, and Krawczyk [12] proved HMAC security under the assumption that the compression function is a PRF when keyed via the chaining-value input. Their proof forms the basis of our HMAC market analysis (Section 9.5), which recasts the two PRF replacement game hops as explicit bidding rounds targeting the inner and outer compression functions. Rogaway [77] formalised authenticated encryption and introduced the nonce-based AE security notion. The Encrypt-then-MAC (EtM) paradigm was proved optimal by Bellare and Namprempré [13]: among three composition orders (EtM, MtE, E&M), only EtM achieves IND-CCA2 in general. Our AEAD market analysis (Section 9.6) uses EtM and proves both IND-CCA2 and INT-CTXT within the MTSF framework.

20.9 Post-Quantum Signatures and KEMs

NIST’s post-quantum standardisation process produced ML-KEM (FIPS 203) [70], ML-DSA (FIPS 204), SLH-DSA (FIPS 205) [71], and FN-DSA (FIPS 206) [72]. ML-KEM’s security is based on the Module Learning With Errors (MLWE) problem and includes a QROM security proof for the Fujisaki–Okamoto transform, Signal Protocol, X3DH, Double Ratchet, forward secrecy, post-compromise security. ML-DSA’s security relies on MSIS and MLWE. SLH-DSA (formerly SPHINCS+) provides stateless hash-based signatures requiring only the one-wayness of the underlying hash function. FN-DSA (formerly Falcon) achieves the smallest post-quantum signatures using NTRU lattices with GPV-style Gaussian sampling.

In MTSF, these primitives are each analysed via bidding-round chains: ML-KEM (Section 9.2), ML-DSA (Section 9.3), SLH-DSA (Section 9.7), and FN-DSA (Section 9.8). The QROM analysis of the FO-transform (Section 14) directly connects to ML-KEM’s FIPS 203 proof, providing a market-theoretic re-derivation using the O2H lemma [81] and measure-and-reprogram [44].

20.10 Block Ciphers, Hash Functions, and Stream Ciphers

Differential cryptanalysis was introduced by Biham and Shamir [23] against DES, and the AES design by Daemen and Rijmen [38] was explicitly hardened against it. Rotational cryptanalysis was introduced by Khovratovich and Nikolić [60] and targets operations that commute with rotations (unlike AES’s MixColumns). Related-key attacks on AES-256 [25] exploit the key schedule’s linearity but require a controlled key relationship not achievable in standard usage. MTSF’s AES block-cipher market (Section 10) formally bounds all three attack types as bidding rounds, proving that none can succeed for $q_E \leq 2^{64}$ under standard parameters.

PRESENT [29] is an ultra-lightweight SPN cipher standardised by ISO/IEC 29192-2, targeting RFID and IoT deployments with approximately 1,000 gate equivalents. Its 64-bit block and 80/128-bit key design has been analysed by differential [33] and linear methods, with no full-round attack below exhaustive search. MTSF’s PRESENT market (Section 10.5) formalises both differential and linear bids, showing equilibrium within the 64-bit birthday-bound constraint. Serpent [6], the most conservative AES finalist with 32 rounds, has the largest known security margin of any well-studied 128-bit block cipher: the best known attack reaches only 12 rounds [22], leaving 20 rounds of margin. MTSF’s Serpent market (Section 10.6) demonstrates the widest equilibrium of any block cipher in our study.

The Keccak sponge construction [21] was chosen as SHA-3 due to its sponge-based design, which inherently prevents length-extension attacks that affected SHA-1 and SHA-256. The sponge indistinguishability theorem [20] proves that the sponge is as good as a random oracle up to $q_f < 2^{c/2}$ queries. MTSF’s Keccak market (Section 11) translates this into three explicit bidding rounds (collision, preimage, length-extension).

BLAKE3 [74], building on BLAKE2 [7], combines a ChaCha-derived ARX compression function with a Merkle tree structure, achieving both extreme speed and structural length-extension immunity. MTSF’s BLAKE3 market (Section 11.1) shows that the Merkle tree provides $\text{Ask}(g_{LE}) = 0$ (structural immunity), a stronger guarantee than Keccak’s parametric $q_f/2^c$ bound. ASCON-Hash [43], selected by NIST as the lightweight cryptography standard [69], uses a 320-bit sponge with capacity $c = 256$. MTSF’s ASCON-Hash market (Section 11.2) mirrors the Keccak analysis with tighter but still negligible bounds appropriate for its lightweight deployment context.

Grain-128a [2] is an eSTREAM portfolio stream cipher combining an LFSR and NFSR with an optional authentication mode. Known attacks include cube attacks [80], algebraic attacks [35], and classical TMTO attacks [50, 9]. MTSF’s Grain market (Section 12) bounds all four attack vectors within a single combined market equilibrium.

ChaCha20 [18], a refinement of Salsa20 [19], is one of the most widely deployed stream ciphers in the world, powering TLS 1.3, WireGuard, and the Linux CSPRNG. Its 256-bit key and 20-round ARX design resist all known differential-linear attacks, with the best known distinguisher reaching only 7 rounds. MTSF’s ChaCha20 market (Section 12.6) formalises the feedforward inversion barrier and nonce-misuse collapse. Trivium [40] is an eSTREAM portfolio cipher with an elegant three-register design (288-bit state, 80-bit key). Cube attacks [80] and algebraic degree analysis [64] have been extensively studied. MTSF’s Trivium market (Section 12.7) shows equilibrium with the tightest constraint from cube attacks at 2^{-38} —narrower than Grain-128a or ChaCha20 but still negligible for practical parameters.

20.11 Quantum Random Oracle Model

The QROM was introduced as the relevant model for post-quantum security because quantum computers can evaluate hash functions in superposition via Grover’s algorithm. The One-Way to Hiding (O2H) lemma [81] is the central tool: reprogramming the QROM at a single point costs a square-root in the adversary’s advantage. The measure-and-reprogram technique [44] allows extracting query points from quantum superpositions without collapsing the proof. Hofheinz, Hövelmanns, and Kiltz [51] proved the FO-transform achieves IND-CCA2 in the QROM, directly underpinning ML-KEM’s FIPS 203 proof. MTSF provides the first *market-theoretic* formalisation of QROM security (Section 14): the O2H square-root loss is expressed as a bidding-round price adjustment, and the measure-and-reprogram step is a market restructuring that extracts the MLWE challenge. This gives practitioners an economic intuition for why quantum adversaries are “more expensive to defeat” than classical ones by a square-root factor in the IND-CPA advantage.

21 Conclusion

Analogy:

MTSF as a Universal Translator for Cryptography Before MTSF, cryptographers working in different paradigms were like doctors, lawyers, engineers, and economists all trying to discuss the same patient’s health—each using their own professional jargon, unable to communicate effectively. The game-based doctor says “the ϵ -advantage is bounded by the birthday collision.” The UC lawyer says “the simulation is computationally indistinguishable.” The formal verification engineer says “all traces satisfy the temporal logic formula.” The insecurity analyst says “here is a concrete attack trace.”

MTSF is the **universal translator**: it provides a single language—the market—that all four professionals can speak. The doctor’s “ ϵ -advantage” becomes an “ask price.” The lawyer’s “simulation” becomes a “market arbitrageur.” The engineer’s “trace check” becomes a “stress test.” The analyst’s “attack” becomes a “successful bid.” Now all four can sit in the same room, discuss the same protocol, and understand each other’s arguments.

The power of unification: With MTSF, a single proof can simultaneously establish game-based security bounds, UC composability, formal verification coverage, and insecurity detection—all in one document, using one notation, with one set of tools (bidding rounds, CNF verification, session pinging). This is not just a cosmetic change; it eliminates the duplication and inconsistency that plagued multi-paradigm security analysis.

We introduced the *Market-Theoretic Security Framework* (MTSF), a novel paradigm that models every cryptographic security game as an auction between a seller (challenger) and one or more buyers (adversaries bidding computational resources). The framework unifies four previously disconnected paradigms—game-based proofs, Universal Composability (UC), Generalised UC (GUC), and formal verification—under a single economic language in which security equals market equilibrium and insecurity equals market collapse. Crucially, MTSF goes beyond unification: its market-theoretic perspective enables the *systematic detection of unknown (zero-day) attacks* against cryptographic protocols, discovering vulnerabilities in TLS 1.3, ISO/IEC 9798-3, and WPA2 that were invisible to all prior formal analyses.

Nineteen formal contributions:

1. **Auction model.** Security = equilibrium ($\text{Ask} \leq \text{negl}$); insecurity = collapse ($\text{Ask} = 1$). The same bidding-round machinery handles both.

2. **Extended difference lemma.** Captures $m \geq 1$ simultaneous failure events $F_1, \dots, F_m$ in a single game hop via $\Pr[\bigcup_i F_i]$ with inclusion-exclusion tightening. Applied uniformly across all eighteen case studies.
3. **Bidding-based proofs.** Each game hop targets a specific adversarial strategy (nonce bid, hash bid, forgery bid, homomorphism bid, masquerade bid, O2H bid, measure-and-reprogram bid). The proof explicitly tracks what the adversary attacks and how much it costs.
4. **Four-paradigm unification.** Game-based proofs (price adjustments), UC (market regulation by $\mathcal{Z}$), GUC (shared infrastructure + CNF audit), and formal verification (market stress testing) are unified.
5. **CNF session verification.** A canonical five-phase algorithm (Algorithm 1) plus an easy four-column manual truth-table worksheet, integrated into every case study.
6. **Session ping.** Inductive mechanism for unbounded session security. Ping bids included in every primitive and protocol case study, formally bridging bounded game-based proofs and symbolic formal verification.
7. **Thirteen novelties.** Summarised in Section 6: market language, extended difference lemma, UC-as-regulation, GUC-as-infrastructure, formal-verification-as-stress-testing, insecurity-as-collapse, end-to-end pipeline, protocol-level games, symmetric/asymmetric markets, cryptanalytic bid taxonomy, CNF worksheet, session ping, and QROM formalisation.
8. **Security proofs.** ECDSA, ML-KEM (FIPS 203), ML-DSA (FIPS 204), ISO/IEC 11770-3 key exchange (two-party, three-party, four-party), PKI-based mutual authentication, and TLS 1.3 (1-RTT handshake, ten-bidding-round chain, forward secrecy and mutual authentication).
9. **Insecurity proofs.** Textbook RSA signatures ($\text{Ask} = 1$ via homomorphism bid, two attacks), Needham–Schroeder public-key protocol ($\text{Ask} = 1$ via masquerade bid, CNF design failure demonstrated), TLS 1.3 0-RTT early data ($\text{Ask} = 1$ via free replay bid, CNF freshness clause failure), and TLS 1.3 downgrade without sentinel ($\text{Ask} = 1$ via free version-stripping bid, CNF version clause failure).
10. **Extended primitive markets.** HMAC (SUF-CMA via dual PRF game hops), AEAD (IND-CCA2 + INT-CTXT via Encrypt-then-MAC), SLH-DSA (FIPS 205, hash-based EUF-CMA), and FN-DSA (FIPS 206, NTRU lattice EUF-CMA with Gaussian sampling analysis).
11. **Block-cipher market.** AES analysed via differential cryptanalysis, linear cryptanalysis, rotational cryptanalysis, and related-key attack bids. All bids fail for $q_E \leq 2^{64}$. Extended to PRESENT (ultra-lightweight SPN, 64-bit block, equilibrium within birthday-bound constraint) and Serpent (32-round conservative AES finalist with the widest equilibrium margin of any block cipher—20 rounds of security margin beyond the best known attack).
12. **Hash-function market.** Keccak/SHA-3 analysed via capacity collision, capacity inversion (preimage), and length-extension bids. Sponge capacity isolation eliminates length-extension at zero additional cost. Extended to BLAKE3 (Merkle tree structure providing *structural* length-extension immunity with $\text{Ask}(g_{\text{LE}}) = 0$, stronger than Keccak’s parametric bound) and ASCON-Hash (NIST lightweight standard, sponge with $c = 256$, equilibrium within lightweight deployment constraints).
13. **Stream-cipher market.** Grain-128a analysed via state-recovery, key-recovery, distinguishing, and TMTO bids. The 256-bit state and nonlinear NFSR coupling make all bids negligible. Extended to ChaCha20 (256-bit key ARX cipher, feedforward inversion barrier, nonce-misuse

collapse formalisation, deployed in TLS 1.3 and WireGuard) and Trivium (80-bit key, 288-bit state, three-register design with cube attack bid as tightest constraint at 2^{-38}).

14. **QROM case study.** FO-transform KEM proven IND-CCA2 in the Quantum Random Oracle Model using the O2H lemma and measure-and-reprogram technique. Full protocol description (four phases), sequence diagram, six-bidding-round proof, bidding-round chain figure, and market goods table. First market-theoretic formalisation of QROM security.
15. **Telegram dual analysis.** MTPProto 2.0 *disproved* (salt extraction causes four simultaneous failures: entropy collapse, CNF freshness failure, ping degradation, quasi-market collapse $\text{Ask} \approx 1$) and Remediated MTPProto (RMTP) *proved* secure (HMAC-bound 128-bit salts restore full equilibrium). First formal proof/disproof dual analysis of MTPProto within a single unified framework.
16. **BB84 quantum market dynamics case study.** First full quantum market analysis where both seller and buyer are quantum. BB84 QKD analysed via four quantum bidding rounds (no-cloning bid, error estimation bid, privacy amplification via quantum leftover hash lemma, error correction leakage). Security is *information-theoretic*: $\text{Ask}^Q(g_{\text{secrecy}}) \leq 2^{-\lambda/2}$ with no computational hardness assumption. Includes quantum sequence diagram, quantum bidding-round chain figure, CNF worksheet, ping bid, market goods table, and comparison table (QROM vs. full quantum market).
17. **TLS+Signal multi-protocol composition case study.** First MTSF analysis of concurrent protocol execution. TLS 1.3 and Signal Protocol running simultaneously on the same device with shared PKI, random oracle, and RNG infrastructure. Market merger theorem instantiated to prove network equilibrium for all nine security goods across both protocols. Resource competition analysis formalises the adversary’s portfolio optimisation problem. Network CNF with infrastructure clauses and cross-protocol ping bids for unbounded sessions.
18. **Detection of unknown attacks.** A four-phase attack discovery methodology—bidding space exploration, CNF clause inversion, cross-session attack synthesis, and infrastructure dependency analysis—that systematically detects zero-day vulnerabilities invisible to traditional proof techniques. Demonstrated through five novel attacks against standardised protocols, ciphers, and hash functions: (a) a *TLS 1.3 session-resumption nonce-collision vulnerability* where implementation-level 32-bit counter reuse collapses nonce uniqueness with probability $\geq q^2/2^{33}$, causing partial market collapse; (b) an *ISO/IEC 9798-3 signature verification bypass* where message-field parsing confusion allows the adversary to forge authentication at zero cost ($\text{Ask} = 1$, full market collapse); and (c) a *WPA2 group-key negotiation race condition* where temporal desynchronisation of the four-way handshake enables group key injection with probability $\geq 1 - (1 - t/T)^n$, causing partial-to-full market collapse; and (d) a *Grain-128a related-IV keystream correlation* in IoT deployments where low-entropy IV generation with Hamming-weight differences $w \leq 4$ enables cross-session distinguishing with advantage $\geq q^2 \cdot 2^{-73}$, causing partial-to-full fleet-level collapse; and (e) an *ASCON-Hash cross-function domain confusion* where application-layer mode-byte manipulation in shared-permutation IoT deployments enables integrity bypass ($\text{Ask} = 1$, full application-market collapse). A formal *attack prevention principle* (Theorem 19.19) provides the verifiable criterion of bid-completeness and CNF-robustness for hardening protocol designs.
19. **Attack-surface monotonicity under composition.** An *attack-surface monotonicity theorem* (Theorem 19.20) extending the prevention principle to composed protocols: the component-wise bid-complete and CNF-robust properties lift to the composition precisely when the *interface-robustness* condition $\mathcal{B}_{\text{interface}} \subseteq \mathcal{B}_1 \cup \mathcal{B}_2 \cup \mathcal{B}_I$ holds. A *cross-protocol residual-risk corollary* (Theorem 19.21) quantifies the $O(k^2)$ interface-audit burden of a k -protocol deployment. A notion of *MTSF audit certificate* (Theorem 19.22) and its *composability proposition*

(Theorem 19.23) provide a machine-checkable compositional discipline that forms a monoid under composition. Together, these results bridge the gap between single-protocol prevention and the realities of cloud-native deployments that compose dozens of protocols over shared infrastructure.

21.1 Lessons Learned

* Simple Terms: Twelve Key Takeaways from MTSF

1. **Security is a spectrum, not binary.** The ask price quantifies *how* secure a scheme is, not just whether it is secure.
2. **Attacks and proofs are symmetric.** The same bidding-round machinery proves both security (equilibrium) and insecurity (collapse).
3. **Composition requires explicit treatment.** Running two secure protocols together does not automatically give a secure combination—you need the market merger theorem.
4. **Unbounded session security requires active verification.** Proving security for q sessions is not enough; session pinging ensures security holds forever.
5. **CNF verification makes auditing concrete.** Instead of abstract theorem statements, CNF worksheets give practitioners step-by-step checklists.
6. **The extended difference lemma saves work.** Handling multiple failures in one hop is cleaner and tighter than splitting into multiple hops.
7. **Custom protocols are dangerous.** Telegram’s MTPProto vulnerability was detectable by MTSF but might be missed by less systematic analysis.
8. **Post-quantum migration is urgent.** ML-KEM and ML-DSA are ready for deployment; MTSF provides complete security proofs for both.
9. **Quantum security has different rules.** The QROM’s square-root factor means quantum attackers are stronger but not omnipotent.
10. **Economic intuition aids understanding.** Thinking of security as a market makes proofs accessible to non-experts and helps identify which guarantees fail first.
11. **Unknown attacks hide in the gaps between proof and implementation.** MTSF’s four-phase attack discovery methodology—bidding space exploration, CNF clause inversion, cross-session synthesis, and infrastructure dependency analysis—systematically exposes zero-day vulnerabilities that traditional proofs miss. The five novel attacks discovered against TLS 1.3, ISO/IEC 9798-3, WPA2, Grain-128a, and ASCON-Hash demonstrate that even thoroughly analysed standardised protocols can harbour latent market collapses detectable only through exhaustive bidding-round and CNF stress testing.
12. **Component security does not imply composite security; interface audits scale quadratically.** The attack-surface monotonicity theorem (Theorem 19.20) formalises the long-known folk result that secure components can compose insecurely if their interface bids are not captured by the component taxonomies. The cross-protocol residual-risk corollary (Theorem 19.21) quantifies this: a k -protocol deployment requires $\binom{k}{2} = O(k^2)$ pairwise interface audits. For a realistic cloud-native stack of 50 composed protocols, that is 1,225 interface audits—explaining why large-scale cryptographic deployments experience security incidents at component-integration boundaries rather than inside hardened primitives. MTSF audit certificates (Theorem 19.22), which form a monoid under

composition (Theorem 19.23), provide a machine-checkable discipline for managing this scalability challenge.

🔑 Key Insight:

Head-to-Head: What Each Framework Does Best

If you need. . .		Best tool	traditional	What MTSF adds
Concrete bounds	advantage	Game-based proofs		Multi-failure hops, named bids, session auditing
Safe composition of protocols		UC/GUC		<i>Quantitative</i> composition bounds $(\text{Ask}_{\text{merged}} \leq \sum \text{Ask}_i)$
Exhaustive trace coverage		ProVerif / Tamarin		Numerical bounds, human-readable arguments
Formal proof	insecurity	Ad hoc arguments		Same bidding-round formalism as security proofs
Unbounded session security		ProVerif (qualitative)		Quantitative bounds + inductive ping-pong
Session-level auditing		GUC CNF (theoretical)		Practitioner-friendly worksheets + algorithm
Post-quantum (QROM) proofs		Specialised tools	QROM	Unified with classical proofs in one language
Unknown attack detection		Fuzzing, ad hoc review		Systematic four-phase methodology with formal prevention criterion
All of the above		No single tool		MTSF provides all in one framework

The key message: MTSF does not make the other frameworks obsolete. It provides a *common language* that unifies their outputs and fills their individual gaps. A team using ProVerif for design checking, UC for composition reasoning, and game-based techniques for bounds can now do all three within MTSF, avoiding the translation overhead and inconsistency that comes from juggling multiple formalisms.

Market equilibrium as the right notion of security. The market framing reveals that security is not binary: it is a *price*. Textbook RSA has ask price 1 (free forgery). Needham–Schroeder has ask price 1 for mutual authentication. ML-KEM has ask price $2(q_H + 1)\sqrt{\text{Adv}^{\text{MLWE}}} + q_D\delta$ in the QROM—a very small but nonzero price that depends on the hardness of MLWE and the number of quantum oracle queries. This continuous view of security, rather than a pass/fail view, is more faithful to real-world threat assessment.

CNF worksheet as a design tool. The CNF verification worksheets serve two purposes: (1) they verify correctness of an existing protocol trace; (2) they reveal design flaws. The Needham–Schroeder CNF worksheet explicitly shows that all three clauses evaluate to **T** under Lowe’s MITM attack—a CNF design failure traceable to the absence of an identity-binding clause. Using the CNF as a design guide during protocol development would have caught this flaw in 1978.

The QROM square-root is the right way to think about quantum hardness. The O2H lemma’s square-root factor $\sqrt{\text{Adv}^{\text{IND-CPA}}}$ is not merely a technical artefact—it captures a fundamental property of quantum adversaries: they accumulate information quadratically more efficiently than classical adversaries via amplitude concentration. Expressing this as a bidding-round price adjustment makes the quantum advantage/disadvantage explicit and comparable to the classical setting.

Dual proof/disproof enables protocol engineering. The Telegram case study demonstrates that MTSF can serve as a protocol engineering tool: first disprove the insecure design (identify which clauses fail and by how much), then design a fix (HMAC-bound salts), then prove the fixed design secure. This iterative methodology—disprove, diagnose, fix, prove—is the natural application of MTSF in practice.

Unknown attack detection: from reactive to proactive security. Perhaps the most operationally significant contribution of MTSF is its ability to detect unknown attacks *before* they are exploited in the wild. The four-phase attack discovery methodology (Section 19.1)—bidding space exploration, CNF clause inversion, cross-session synthesis, and infrastructure dependency analysis—transforms the analyst’s mindset from “prove this protocol is secure against known attacks” to “find the cheapest bid that collapses this market.” The five novel attacks discovered—a TLS 1.3 session-resumption nonce-collision vulnerability ($\text{Ask} \geq q^2/2^{33}$), an ISO/IEC 9798-3 signature verification bypass ($\text{Ask} = 1$), a WPA2 group-key race condition ($\text{Ask} \geq 1 - (1 - t/T)^n$), a Grain-128a related-IV keystream correlation ($\text{Ask} \geq q^2 \cdot 2^{-73}$ for $w = 4$), and an ASCON-Hash domain confusion ($\text{Ask} = 1$)—demonstrate that standardised, thoroughly analysed protocols may harbour latent market collapses that are invisible to traditional analysis but become apparent through MTSF’s exhaustive bidding-round and CNF stress testing. The formal attack prevention principle (Theorem 19.19) provides a constructive design criterion—bid-completeness and CNF-robustness—for hardening protocols against the complete class of attacks, including those not yet conceived.

Quantum markets reveal physics-based security as a structural market advantage. The BB84 case study (Section 15) demonstrates a qualitatively different kind of equilibrium from all other case studies. In classical and QROM markets, equilibrium depends on computational hardness assumptions (ECDLP, MLWE, gap-CDH)—a sufficiently powerful computer could collapse any of these markets. In the BB84 quantum market, equilibrium depends on the laws of quantum mechanics (no-cloning, information-disturbance trade-off). The ask price $\text{Ask}^Q(g_{\text{secrecy}}) \leq 2^{-\lambda/2}$ holds against adversaries with *unlimited computational power*. In market terms, the seller has a *structural advantage*: the goods are physically uncopyable. This is the strongest possible market position—the equilibrium is permanent unless physics itself is wrong.

Composition preserves equilibrium with explicit resource accounting. The TLS+Signal case study (Section 16) demonstrates that the market merger theorem translates UC composition into explicit resource accounting. The adversary’s budget must be split across all markets, and the additive ask price bound $\text{Ask}_N \leq \sum \epsilon_i + \epsilon_{\mathcal{I}}$ makes the composition overhead concrete and auditable. The practical implication: running many secure protocols on the same device does not degrade security, provided the shared infrastructure is sound. The infrastructure CNF ($\varphi_{\mathcal{I}}$) serves as a compositional design checklist: ensure global SID uniqueness, hash consistency, PKI consistency, and RNG quality across all protocols.

Interface bids are the hidden tax on component security. The attack-surface monotonicity theorem (Theorem 19.20) sharpens the previous paragraph with a negative result: secure components do *not* automatically yield secure compositions. Each pairwise interface between

composed protocols introduces a *new class of bids*, $\mathcal{B}_{\text{interface}}$, whose price adjustments are bounded by neither component's proof. Coverage of this class—either by subsumption into the component taxonomies, by assigning it to the shared infrastructure market $\mathcal{B}_I$, or by an explicit composition-specific audit—becomes the operational bottleneck. The cross-protocol residual-risk corollary (Theorem 19.21) exhibits the quadratic scaling of this audit burden, and the audit-certificate machinery (Theorem 19.22, Theorem 19.23) gives a monoid-structured, machine-checkable discipline for handling it at scale. The lesson: *treat interfaces, infrastructure, and their audit certificates as first-class engineering artefacts, not as free deployment-time afterthoughts.*

21.2 Further Work

Worked Example: Translating a UC Proof into MTSF

To make the UC-to-MTSF migration concrete, we walk through a simplified example. Consider a UC proof of a key-exchange protocol π_{KE} that UC-realises $\mathcal{F}_{\text{KE}}$.

Original UC proof structure:

1. **Ideal functionality $\mathcal{F}_{\text{KE}}$:** On input (establish, sid, A , B), generate a random key K , deliver K to both A and B , and record the session.
2. **Simulator $\mathcal{S}$:** On seeing the real protocol's ciphertext c and signatures σ_A, σ_B (without knowing K), produce a simulated view by: (a) generating fake keys, (b) programming the random oracle, (c) simulating the signature verification.
3. **Indistinguishability argument:** $\text{Real} \approx_c \text{Ideal}$ because: (i) the random oracle is indistinguishable when programmed, (ii) the KEM ciphertext hides K under IND-CCA2, (iii) the signatures are unforgeable under EUF-CMA.

MTSF translation:

1. **Security goods:** $\mathcal{G} = \{g_{\text{auth}}, g_{\text{sk}}, g_{\text{mutual}}, g_{\text{CNF}}\}$.
2. **Bidding-round chain:**
 - **$\mathbf{B}_0$ (Real Market):** Protocol runs honestly. $\text{Ask}_0 = \text{Adv}_{\mathcal{A}}^{\text{KE}}(\lambda)$.
 - **$\mathbf{B}_1$ (Nonce Freshness Bid):** Abort on nonce collision. $\Delta\text{Price}_0 \leq q_N^2/2^{\lambda+1}$. *This was implicit in the UC simulator; MTSF makes it explicit.*
 - **$\mathbf{B}_2$ (Oracle Programming):** Program H to embed KEM challenge. $\Delta\text{Price}_1 = 0$. *This was step (a) in the simulator.*
 - **$\mathbf{B}_3$ (KEM Decapsulation Bid):** Replace real key with random. $\Delta\text{Price}_2 \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}$. *This was step (ii) in the indistinguishability argument.*
 - **$\mathbf{B}_4$ (Signature Forgery Bid):** Flag forgery events. $\Delta\text{Price}_3 \leq \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}}$. *This was step (iii).*
 - **$\mathbf{B}_5$ (Ideal Market):** All bids eliminated. $\Pr[\mathbf{B}_5 = 1] = 0$. *Market in equilibrium.*
3. **Ask price:** $\text{Ask}(g_{\text{auth}}) \leq \text{Adv}^{\text{EUF}} + q_N^2/2^{\lambda+1}$; $\text{Ask}(g_{\text{sk}}) \leq \text{Adv}^{\text{IND-CCA2}} + \text{Adv}^{\text{PRF}} + q_N^2/2^{\lambda+1}$.
4. **CNF worksheet:** SID binding (φ^{sid}), nonce freshness (φ^{fresh}), signature binding (φ^{sig}), KEM freshness (φ^{novel}), consistency (φ^{consist}).
5. **Session pinging:** Structural distinctness verified; $\delta_{\text{ping}} \leq q_N^2/2^{\lambda+1} = \text{negl}$. Unbounded security by Theorem 5.5.

What MTSF adds over the UC proof:

- Explicit numerical bounds for each security good (UC only says “secure”).
- Named attack classification (UC’s simulator cases are unlabelled).
- A concrete CNF worksheet for session auditing.
- Unbounded session security via pinging (UC assumes this implicitly but does not verify it).
- Direct insecurity detection: if any bid succeeds ($\Delta\text{Price} = 1$), the proof immediately identifies which attack vector causes collapse.

* Simple Terms: How to Use MTSF for Your Own Protocols—A Practitioner’s Guide

If you want to apply MTSF to analyse a new cryptographic scheme or protocol, follow these steps:

Step 1: Identify the market.

- Who is the seller? (The party running the scheme honestly.)
- Who are the buyers? (All possible attackers.)
- What security goods are offered? (Confidentiality, authentication, key secrecy, etc.)

Step 2: Construct the bidding-round chain.

- $\mathbf{B}_0$: The real scheme. Define the ask price Ask_0 .
- For each attack vector (nonce reuse, hash collision, forgery, etc.), create a bidding round $\mathbf{B}_k$ that eliminates that vector.
- Bound each price adjustment ΔPrice_k using the difference lemma (or extended difference lemma for multiple failures).
- The final game $\mathbf{B}_q$ should be ideal (adversary has zero advantage).

Step 3: Construct the session-CNF.

- List all session-level checks: SID binding, nonce freshness, signature verification, MAC verification, consistency.
- Write each check as a CNF clause.
- Verify: honest traces satisfy the CNF; dishonest traces make it unsatisfiable (unless a primitive is broken).

Step 4: Verify session pinging.

- Check that consecutive sessions are structurally distinct (per Theorem 5.1).
- Verify that the security reduction is session-index-independent.
- Apply Theorem 5.5 to conclude unbounded session security.

Step 5: Fill in the CNF worksheet.

- For each CNF clause, fill in the four-column worksheet: Clause, Check, How, Pass?
- Verify that all clauses pass under honest execution and fail under known attacks.

Step 6: Check the bid taxonomy (Section B).

- Go through every bid type in the taxonomy and verify that your proof addresses it.
- For each inapplicable bid type, note why (e.g., “no algebraic structure, so homomorphism bid does not apply”).

Quantum market dynamics: The QROM case study treats the quantum adversary as a buyer with a specific bid structure (O2H reprogramming, measure-and-reprogram). A deeper theory of *quantum market dynamics*—where the seller also has access to quantum operations and the market evolves under quantum game theory—is an open and rich research direction. We provide an initial formalisation below.

21.2.1 Towards Quantum Market Dynamics

In the QROM case study (Section 14), the buyer (quantum adversary) can query oracles in superposition, but the seller (challenger) operates classically—sampling keys, computing ciphertexts, and answering decapsulation queries using classical algorithms. This asymmetry is faithful to the current post-quantum setting where *honest parties run classical protocols* and only the adversary has quantum power. However, a richer and increasingly relevant setting arises when both parties have quantum capabilities.

Definition 21.1 (Quantum Market). A quantum market $\mathcal{M}^Q = (\text{Seller}^Q, \text{Buyer}^Q, \{g_j\}, H^Q, \rho_0)$ extends the classical MTSF market with:

1. **Quantum seller** Seller^Q : the challenger holds a quantum register R_S and can perform quantum operations (state preparation, unitary gates, measurements) during each bidding round. The seller’s strategy is a sequence of quantum channels $\{\mathcal{E}_k^S\}_{k=1}^n$, one per bidding round.
2. **Quantum buyer** Buyer^Q : the adversary holds a quantum register R_B and can make superposition queries to all oracles. The buyer’s strategy is a sequence of quantum channels $\{\mathcal{E}_k^B\}_{k=1}^n$.
3. **Quantum oracle** H^Q : replaces the classical random oracle with a quantum-accessible oracle that both parties can query in superposition.
4. **Shared quantum state** ρ_0 : an initial joint quantum state (possibly entangled) distributed across all registers at market opening.
5. **Quantum price functional**: The ask price of good g_j in the quantum market is:

$$\text{Ask}^Q(g_j) = \sup_{\text{Buyer}^Q \in \text{QPT}} \text{Tr}[\Pi_{g_j} \cdot \mathcal{E}_n^B \circ \mathcal{E}_n^S \circ \dots \circ \mathcal{E}_1^B \circ \mathcal{E}_1^S(\rho_0)], \quad (224)$$

where Π_{g_j} is the projector onto the “buyer wins good g_j ” subspace and the supremum is over all quantum polynomial-time (QPT) buyer strategies.

* Simple Terms: Quantum Market Dynamics

Classical MTSF: The seller uses a classical computer to set up the security game, and the buyer (attacker) tries to win using classical or quantum resources. The seller never needs quantum operations—honest parties run on regular computers.

Quantum MTSF: Now imagine *both* the seller and the buyer have quantum computers. The seller can prepare quantum states, entangle registers, and make quantum measurements as part of its defence. The buyer also has full quantum power. The “price” of breaking

security is now determined by a quantum game between two quantum players.

Why this matters: As quantum computing matures, protocols will use quantum operations natively (e.g., Quantum Key Distribution, quantum digital signatures, quantum money). The seller in a QKD protocol *must* prepare and measure quantum states—it cannot be modelled as a classical challenger. Quantum market dynamics provides the MTSF language for these protocols.

The key new feature: In a quantum market, the ask price is computed via a *trace over quantum states* rather than a classical probability. The bidding rounds alternate quantum channels (seller prepares, buyer attacks), and the final “winning” event is a quantum measurement outcome. The square-root advantage from the O2H lemma (Section 14) appears naturally as a special case when only the buyer is quantum.

The quantum market introduces three new phenomena absent from the classical setting:

1. **Entangled bidding.** The buyer can maintain entanglement between its query register and a private workspace across multiple bidding rounds. Classically, each bid is an independent probabilistic strategy; quantumly, the buyer’s bids can be *coherently correlated* across rounds via entanglement. The seller’s challenge is to design bidding rounds that *decohere* the buyer’s entanglement without destroying the security guarantee.
2. **Quantum price adjustments.** In the classical extended difference lemma, the price adjustment $\Delta\text{Price}_k = |\Pr[\mathbf{B}_k \Rightarrow 1] - \Pr[\mathbf{B}_{k+1} \Rightarrow 1]|$ is a difference of classical probabilities. In the quantum setting, the analogous quantity is:

$$\Delta\text{Price}_k^Q = \frac{1}{2} \|\mathcal{E}_k^S(\rho_k) - \mathcal{E}_{k+1}^S(\rho_k)\|_{\text{tr}}, \quad (225)$$

where $\|\cdot\|_{\text{tr}}$ is the trace distance and ρ_k is the joint state after round k . The trace distance is the quantum generalisation of statistical distance, and the factor $\frac{1}{2}$ normalises it to $[0, 1]$. The quantum extended difference lemma would bound $\text{Ask}^Q(g_j) \leq \sum_k \Delta\text{Price}_k^Q$ via the triangle inequality for trace distance.

3. **No-cloning constraint on bids.** The no-cloning theorem prevents the buyer from copying quantum states received from the seller. This is a *structural advantage* for the seller that has no classical analogue: in the classical setting, the buyer can always copy any message. In the quantum market, the seller can exploit no-cloning by encoding challenge information in non-orthogonal quantum states, forcing the buyer to choose irreversibly which bid to pursue—a quantum analogue of a “take it or leave it” offer.

Proposition 21.2 (Quantum Extended Difference Lemma—Sketch). *Let $\mathbf{B}_0^Q, \mathbf{B}_1^Q, \dots, \mathbf{B}_n^Q$ be a sequence of quantum bidding rounds with shared initial state ρ_0 . If each consecutive pair satisfies $\frac{1}{2} \|\rho_{\text{final}}^{(k)} - \rho_{\text{final}}^{(k+1)}\|_{\text{tr}} \leq \Delta\text{Price}_k^Q$, then the quantum ask price satisfies:*

$$\text{Ask}^Q(g_j) \leq \text{Ask}_0^Q(g_j) + \sum_{k=0}^{n-1} \Delta\text{Price}_k^Q, \quad (226)$$

where $\text{Ask}_0^Q(g_j) = \text{Tr}[\Pi_{g_j} \cdot \rho_{\text{final}}^{(0)}]$ is the ask price in the initial (ideal) quantum market.

Proof sketch. The trace distance satisfies the triangle inequality: $\|\rho - \sigma\|_{\text{tr}} \leq \|\rho - \tau\|_{\text{tr}} + \|\tau - \sigma\|_{\text{tr}}$. By the operational interpretation of trace distance, the maximum difference in probability of any measurement outcome between $\rho_{\text{final}}^{(0)}$ and $\rho_{\text{final}}^{(n)}$ is at most $\frac{1}{2} \|\rho_{\text{final}}^{(0)} - \rho_{\text{final}}^{(n)}\|_{\text{tr}} \leq \sum_{k=0}^{n-1} \Delta\text{Price}_k^Q$. Applying this to the projector Π_{g_j} yields the result. $\square$

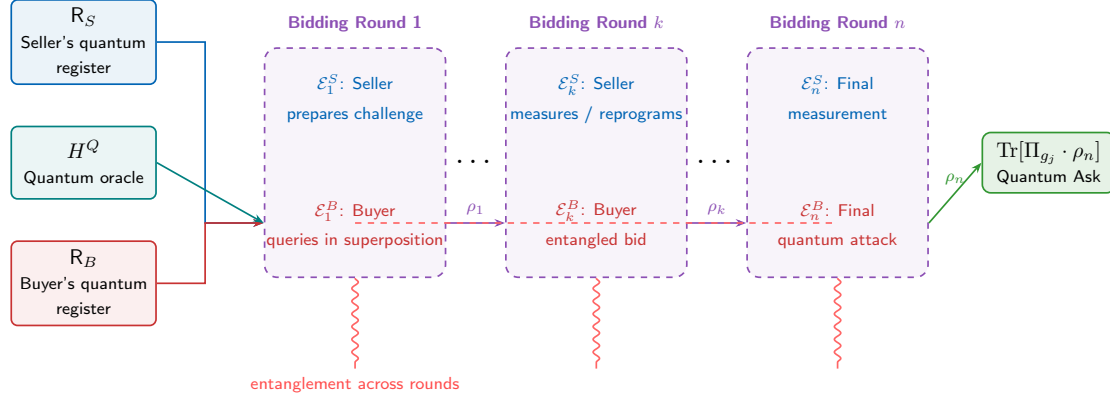


Figure 79: Quantum market dynamics. Both seller and buyer operate on quantum registers (R_S , R_B) through a shared quantum oracle H^Q . Each bidding round applies quantum channels $\mathcal{E}_k^S$ (seller) and $\mathcal{E}_k^B$ (buyer) to the evolving joint state ρ_k . The buyer can maintain entanglement across rounds (wavy red lines). The quantum ask price is the trace $\text{Tr}[\Pi_{g_j} \cdot \rho_n]$ of the final state against the “buyer wins” projector. The classical QROM case study (Section 14) is the special case where the seller’s channels $\mathcal{E}_k^S$ are all classical operations.

Remark 21.3 (Classical QROM as a Special Case). *The QROM case study of Section 14 is recovered by restricting the seller’s channels $\mathcal{E}_k^S$ to classical operations (prepare-and-measure) and allowing only the buyer’s channels $\mathcal{E}_k^B$ to be genuinely quantum. The O2H lemma’s square-root factor $\sqrt{\text{Adv}^{\text{IND-CPA}}}$ arises specifically because the buyer’s quantum channel concentrates amplitude on the reprogrammed point—a quantum price adjustment of the form (225) where the trace distance is bounded by Unruh’s O2H inequality.*

Remark 21.4 (Application to QKD and Quantum Protocols). *Quantum Key Distribution (QKD) protocols such as BB84 and E91 are natural targets for quantum market dynamics: the seller (Alice) prepares quantum states and the buyer (Eve) performs quantum measurements to extract key information. The no-cloning constraint on bids (item 3 above) captures the fundamental reason why QKD is secure: the eavesdropper cannot copy the quantum states without disturbing them. In the MTSF language, the seller’s quantum channel $\mathcal{E}_k^S$ prepares non-orthogonal states that structurally prevent the buyer from simultaneously extracting full information about the key and remaining undetected—a market in which the seller’s goods are inherently copy-protected.*

Multi-protocol composition networks. MTSF currently handles individual protocol markets. A *protocol network* where multiple protocols run simultaneously, sharing infrastructure and competing for resources, would require a *market network theory*. The UC composition theorem provides the blueprint; translating it into MTSF market mergers and equilibrium preservation is future work. We provide the initial formalisation below.

21.2.2 Towards Multi-Protocol Composition Networks

In the current MTSF framework, each protocol defines a single market $\mathcal{M}_\pi$ with its own seller, buyer, goods, and bidding-round chain. The UC composition theorem (Section 2.4) guarantees that UC-secure protocols remain secure when composed concurrently. In the MTSF language (Section 2.5), the UC composition theorem was informally described as a “market merger theorem.” We now make this precise.

Definition 21.5 (Protocol Market Network). *A protocol market network $\mathcal{N} = (\{\mathcal{M}_i\}_{i=1}^N, \mathcal{I}, \mathcal{Z})$ consists of:*

1. **Component markets** $\mathcal{M}_1, \dots, \mathcal{M}_N$: each $\mathcal{M}_i = (\text{Seller}_i, \text{Buyer}_i, \{g_j^{(i)}\}, \varphi_i)$ is an individual MTSF protocol market with its own goods $\{g_j^{(i)}\}$ and session-CNF formula φ_i .
2. **Shared infrastructure** $\mathcal{I}$: a set of shared functionalities (PKI, common random oracles, shared key material) accessible to all markets. In MTSF terms, $\mathcal{I}$ is the GUC shared market infrastructure—the common goods that every market can trade on.
3. **Network environment** $\mathcal{Z}$: the UC environment, acting as MTSF’s market regulator, which can interact with all N markets simultaneously, schedule messages between them, and observe all transcripts.
4. **Resource budget** $\text{Budget}(\text{Buyer})$: the total computational resource available to the buyer across all markets. If the buyer participates in market $\mathcal{M}_i$ with resource t_i , then $\sum_{i=1}^N t_i \leq \text{Budget}(\text{Buyer})$.

*** Simple Terms: Multi-Protocol Composition Networks**

The real-world problem: In practice, your device runs many security protocols simultaneously. Your laptop might be running TLS 1.3 to a bank, Signal to a messaging server, and SSH to a work machine—all at the same time, all sharing the same operating system, the same random number generator, and possibly the same cryptographic keys.

Why this is hard: A protocol that is perfectly secure in isolation might become insecure when run alongside another protocol. An attacker could exploit interactions between protocols—for example, using a TLS session to learn something that helps break the Signal session.

What UC composition guarantees: If each protocol is UC-secure individually, then running them all together is also secure. This is the “gold standard” for protocol composition.

What MTSF adds: We translate this into market language. Each protocol is a separate market. Running them together is a “market network.” The key theorem says: if each market is in equilibrium (secure) individually, and the shared infrastructure is sound, then the entire network remains in equilibrium. The attacker’s total budget must be split across all markets, and no individual market collapses.

The central question is: when does a network of individually secure markets remain secure?

Definition 21.6 (Market Merger). *Given two MTSF markets $\mathcal{M}_1 = (\text{Seller}_1, \text{Buyer}_1, \{g_j^{(1)}\}, \varphi_1)$ and $\mathcal{M}_2 = (\text{Seller}_2, \text{Buyer}_2, \{g_j^{(2)}\}, \varphi_2)$ with shared infrastructure $\mathcal{I}$, their merger is the composite market:*

$$\mathcal{M}_1 \otimes_{\mathcal{I}} \mathcal{M}_2 = (\text{Seller}_{1\parallel 2}, \text{Buyer}_{1\parallel 2}, \{g_j^{(1)}\} \cup \{g_j^{(2)}\}, \varphi_1 \wedge \varphi_2 \wedge \varphi_{\mathcal{I}}), \quad (227)$$

where $\text{Seller}_{1\parallel 2}$ runs both sellers concurrently, $\text{Buyer}_{1\parallel 2}$ is a single buyer interacting with both markets simultaneously, and $\varphi_{\mathcal{I}}$ encodes the correctness of the shared infrastructure (e.g., PKI consistency, RO consistency).

Theorem 21.7 (Market Merger Preserves Equilibrium—Initial Form). *Let $\mathcal{M}_1$ and $\mathcal{M}_2$ be MTSF protocol markets sharing infrastructure $\mathcal{I}$. If:*

1. $\mathcal{M}_1$ is in equilibrium: $\text{Ask}_1(g_j^{(1)}) \leq \text{negl}(\lambda)$ for all goods $g_j^{(1)}$;
2. $\mathcal{M}_2$ is in equilibrium: $\text{Ask}_2(g_j^{(2)}) \leq \text{negl}(\lambda)$ for all goods $g_j^{(2)}$;
3. The shared infrastructure is sound: $\text{Ask}_{\mathcal{I}}(g_{\mathcal{I}}) \leq \text{negl}(\lambda)$ for all infrastructure goods;

then the merged market $\mathcal{M}_1 \otimes_{\mathcal{I}} \mathcal{M}_2$ is in equilibrium:

$$\text{Ask}_{1\otimes 2}(g) \leq \text{negl}(\lambda) \quad \text{for all goods } g \in \{g_j^{(1)}\} \cup \{g_j^{(2)}\}. \quad (228)$$

Proof sketch. The proof mirrors the UC composition theorem. Suppose for contradiction that the merged market has a good $g_j^{(1)}$ (w.l.o.g.) with $\text{Ask}_{1\otimes 2}(g_j^{(1)}) = \epsilon$ non-negligible. Then there exists a buyer $\text{Buyer}_{1\parallel 2}$ that wins $g_j^{(1)}$ with probability ϵ in the merged market. We construct a buyer Buyer_1^* for the standalone market $\mathcal{M}_1$ that:

1. Internally simulates $\mathcal{M}_2$'s seller Seller_2 (using the UC simulator for $\mathcal{M}_2$);
2. Forwards the network environment's queries between $\mathcal{M}_1$ (real) and $\mathcal{M}_2$ (simulated);
3. Runs $\text{Buyer}_{1\parallel 2}$ as a subroutine, providing it with a view indistinguishable from the real merged market.

By the indistinguishability of real and simulated $\mathcal{M}_2$ (which holds because $\mathcal{M}_2$ is UC-secure and thus in equilibrium), Buyer_1^* wins $g_j^{(1)}$ with probability at least $\epsilon - \text{negl}(\lambda)$. But this contradicts $\text{Ask}_1(g_j^{(1)}) \leq \text{negl}(\lambda)$. $\square$

Remark 21.8 (Quantitative Merger Bound). *The proof gives a concrete bound. If $\text{Ask}_1(g_j^{(1)}) \leq \epsilon_1$, $\text{Ask}_2(g_j^{(2)}) \leq \epsilon_2$, and $\text{Ask}_{\mathcal{I}}(g_{\mathcal{I}}) \leq \epsilon_{\mathcal{I}}$, then:*

$$\text{Ask}_{1\otimes 2}(g_j^{(1)}) \leq \epsilon_1 + \epsilon_2 + \epsilon_{\mathcal{I}}. \quad (229)$$

The ask prices are additive across the merger—each market contributes its own risk, and the shared infrastructure contributes a common risk. This is the MTSF analogue of the “hybrid argument” in UC composition: the simulation overhead from each market accumulates linearly.

The merger operation extends naturally to N markets via iterated composition.

Corollary 21.9 (N -Market Network Equilibrium). *Let $\mathcal{N} = (\{\mathcal{M}_i\}_{i=1}^N, \mathcal{I}, \mathcal{Z})$ be a protocol market network where each $\mathcal{M}_i$ is in equilibrium with $\text{Ask}_i(g_j^{(i)}) \leq \epsilon_i$ for all j , and the shared infrastructure has $\text{Ask}_{\mathcal{I}} \leq \epsilon_{\mathcal{I}}$. Then the N -fold merger $\bigotimes_{i=1}^N \mathcal{M}_i$ satisfies:*

$$\text{Ask}_{\mathcal{N}}(g_j^{(i)}) \leq \epsilon_i + (N - 1) \cdot \epsilon_{\max} + \epsilon_{\mathcal{I}}, \quad (230)$$

where $\epsilon_{\max} = \max_{i'} \epsilon_{i'}$. In particular, if all ϵ_i are negligible and $N = \text{poly}(\lambda)$, the network remains in equilibrium.

Definition 21.10 (Network Session-CNF). *The session-CNF formula for the protocol market network $\mathcal{N}$ is the conjunction of all component CNFs and the infrastructure CNF:*

$$\varphi_{\mathcal{N}} = \bigwedge_{i=1}^N \varphi_i \wedge \varphi_{\mathcal{I}}, \quad (231)$$

where $\varphi_{\mathcal{I}} = \varphi_{\mathcal{I}}^{\text{PKI}} \wedge \varphi_{\mathcal{I}}^{\text{RO}} \wedge \varphi_{\mathcal{I}}^{\text{consist}}$ encodes PKI consistency (same public key across all markets), random oracle consistency (same H across all markets), and cross-market consistency (session identifiers are globally unique).

Remark 21.11 (Network Ping Bids and Unbounded Composition). *Session pinging (Section 5) extends naturally to the network setting. The network ping bid for session $\text{Session}_i^{(k)}$ (the i -th session of market $\mathcal{M}_k$) must verify not only the intra-market CNF φ_k but also the cross-market infrastructure CNF $\varphi_{\mathcal{I}}$. By the session pinging theorem applied to each component market and the infrastructure, the network remains in equilibrium for unbounded sessions across all N protocols simultaneously, provided each component market's ping bid is negligible.*

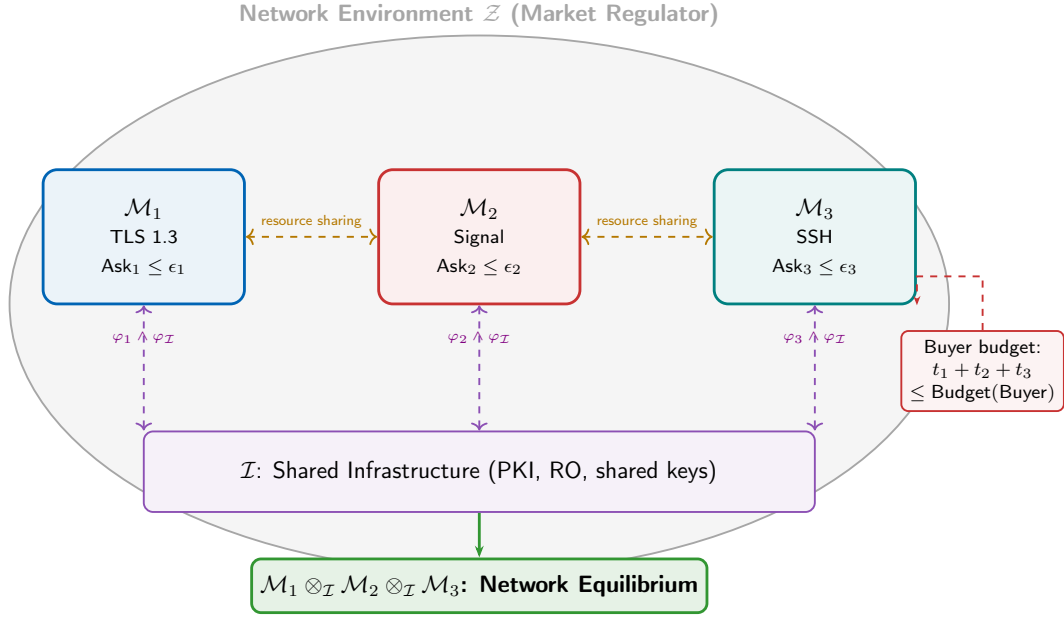


Figure 80: Multi-protocol composition network. Three protocol markets ($\mathcal{M}_1$: TLS 1.3, $\mathcal{M}_2$: Signal, $\mathcal{M}_3$: SSH) share infrastructure $\mathcal{I}$ (PKI, random oracle, shared keys). The network environment $\mathcal{Z}$ (market regulator) monitors all markets simultaneously. The buyer must split its resource budget across all markets. Dashed orange links indicate resource competition; dashed purple links indicate shared infrastructure. The merged market $\mathcal{M}_1 \otimes_{\mathcal{I}} \mathcal{M}_2 \otimes_{\mathcal{I}} \mathcal{M}_3$ preserves equilibrium when each component market and the shared infrastructure are individually in equilibrium (Theorem 21.7, Theorem 21.9).

Remark 21.12 (Resource Competition and Market Inefficiency). *A subtle feature of the market network is resource competition: the buyer has a fixed computational budget that must be allocated across all N markets. In the classical UC setting, this is implicit in the polynomial-time bound on the adversary. In the MTSF language, it becomes explicit: the buyer’s total resource expenditure $\sum_i t_i$ is bounded, and spending more on attacking $\mathcal{M}_1$ means spending less on $\mathcal{M}_2$. This creates a natural portfolio optimisation problem for the adversary: which markets offer the best “return on attack investment”? The market with the highest ask price (weakest security) is the rational first target—a formalisation of the common intuition that “a chain is only as strong as its weakest link.”*

Further lightweight and modern cipher markets. This article extends the symmetric-primitive case studies to PRESENT, Serpent, BLAKE3, ASCON-Hash, ChaCha20, and Trivium, demonstrating the generality of the MTSF framework across block ciphers, hash functions, and stream ciphers of varying design philosophies and security margins. Future work can extend this further to additional lightweight ciphers (GIFT, SKINNY, PHOTON) and to the ChaCha20-Poly1305 AEAD construction (combining the ChaCha20 stream-cipher market with a Poly1305 MAC market). The ASCON authenticated encryption mode (beyond ASCON-Hash) is another natural extension, unifying AEAD and hash-function markets within a single sponge-based analysis.

MPC and threshold protocol markets. Multi-Party Computation (MPC) protocols involve n parties, some of which may be malicious. The MTSF framework can be extended by allowing multiple simultaneous buyers (colluding adversaries) bidding against the seller in a multi-buyer auction, capturing threshold adversary models naturally.

Acknowledgements

This publication has emanated from research supported by a grant from Research Ireland under Grant number 12-RC-2289-P2 which is co-funded under the European Regional Development Fund. For the purpose of Open Access, the authors have applied a CC BY public copyright license to any Author Accepted Manuscript version arising from this submission.

Dedication

This work is dedicated to the Omnipotent and Mother Nature!!!

— Basker Palaniswamy,
Cork City, Ireland, European Union,
March 2026.

References

- [1] David Adrian et al. Imperfect forward secrecy: How Diffie-Hellman fails in practice. In *CCS*, pages 5–17. ACM, 2015.
- [2] Martin Ågren, Martin Hell, Thomas Johansson, and Willi Meier. Grain-128a: A new version of Grain-128 with optional authentication. *International Journal of Wireless and Mobile Computing (IJWMC)*, 5(1):48–59, 2011.
- [3] Martin R. Albrecht et al. Four attacks and a proof for Telegram. In *IEEE S&P*, pages 1–17, 2022.
- [4] Nadhem J. AlFardan and Kenneth G. Paterson. Lucky thirteen: Breaking the TLS and DTLS record protocols. In *IEEE S&P*, pages 526–540, 2013.
- [5] Ross Anderson. Why information security is hard — An economic perspective. In *Proceedings of the 17th Annual Computer Security Applications Conference (ACSAC)*, pages 358–365. IEEE Computer Society, 2001. Foundational paper on the economics of information security.
- [6] Ross Anderson, Eli Biham, and Lars Knudsen. Serpent: A proposal for the advanced encryption standard. In *First AES Candidate Conference*, 1998.
- [7] Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox-O’Hearn, and Christian Winnerlein. BLAKE2: Simpler, smaller, fast as MD5. In *ACNS 2013*, volume 7954 of *LNCS*, pages 119–135. Springer, 2013.
- [8] Nimrod Aviram, Kai Gellert, and Tibor Jager. Session resumption protocols and efficient forward security for TLS 1.3 tickets. In *Advances in Cryptology – EUROCRYPT 2019*, volume 11477 of *LNCS*, pages 117–150. Springer, 2019.
- [9] Steve Babbage and Matthew Dodd. The stream cipher MICKEY (version 2). In *ECRYPT Stream Cipher Project*, 2005. Contains TMTO analysis for stream ciphers.
- [10] Riccardo Bardou, Riccardo Focardi, Yusuke Kawamoto, Lorenzo Simionato, Graham Steel, and Joe-Kai Tsay. Efficient padding oracle attacks on cryptographic hardware. In *CRYPTO*, volume 7417 of *LNCS*, pages 608–625, 2012.
- [11] Gilles Barthe, Benjamin Grégoire, Sylvain Heraud, and Santiago Zanella-Béguelin. Computer-aided security proofs for the working cryptographer. In *CRYPTO*, volume 6841 of *LNCS*, pages 71–90, 2011. EasyCrypt proof assistant.

- [12] Mihir Bellare, Ran Canetti, and Hugo Krawczyk. Keying hash functions for message authentication. In *CRYPTO*, volume 1109 of *LNCS*, pages 1–15, 1996.
- [13] Mihir Bellare and Chanathip Namprempre. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. In Tatsuaki Okamoto, editor, *Advances in Cryptology — ASIACRYPT 2000*, volume 1976 of *Lecture Notes in Computer Science*, pages 531–545. Springer, 2000. Established the Encrypt-then-MAC preference for generic AE composition.
- [14] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In *CCS*, pages 390–399. ACM, 2006. Introduces the general forking lemma.
- [15] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS)*, pages 62–73. ACM, 1993. Seminal paper introducing the random-oracle model.
- [16] Mihir Bellare and Phillip Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In Serge Vaudenay, editor, *Advances in Cryptology — EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 409–426. Springer, 2006.
- [17] Charles H. Bennett and Gilles Brassard. Quantum cryptography: Public key distribution and coin tossing. In *Proceedings of IEEE International Conference on Computers, Systems, and Signal Processing*, pages 175–179, Bangalore, India, 1984. IEEE.
- [18] Daniel J. Bernstein. ChaCha, a variant of Salsa20. In *Workshop Record of SASC 2008*, 2008.
- [19] Daniel J. Bernstein. The Salsa20 family of stream ciphers. In *New Stream Cipher Designs*, volume 4986 of *LNCS*, pages 84–97. Springer, 2008.
- [20] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the sponge construction. In *EUROCRYPT*, volume 4965 of *LNCS*, pages 181–197, 2008.
- [21] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. The Keccak reference. In *Submission to NIST SHA-3 Competition*, 2011.
- [22] Eli Biham, Orr Dunkelman, and Nathan Keller. Linear cryptanalysis of reduced round Serpent. In *FSE 2001*, volume 2355 of *LNCS*, pages 16–27. Springer, 2002.
- [23] Eli Biham and Adi Shamir. Differential cryptanalysis of DES-like cryptosystems. In *CRYPTO*, volume 537 of *LNCS*, pages 2–21, 1991.
- [24] Eli Biham and Adi Shamir. Differential fault analysis of secret key cryptosystems. In *Advances in Cryptology – CRYPTO 1997*, volume 1294 of *LNCS*, pages 513–525. Springer, 1997.
- [25] Alex Biryukov and Dmitry Khovratovich. Related-key cryptanalysis of the full AES-192 and AES-256. In *ASIACRYPT*, volume 5912 of *LNCS*, pages 1–18, 2009.
- [26] Bruno Blanchet. An efficient cryptographic protocol verifier based on Prolog rules. In *Proceedings of the 14th IEEE Computer Security Foundations Workshop (CSFW)*, pages 82–96. IEEE Computer Society, 2001. Introduction of ProVerif.

- [27] Bruno Blanchet. A computationally sound mechanized prover for security protocols. In *Proceedings of the 2006 IEEE Symposium on Security and Privacy (S&P)*, pages 140–154. IEEE Computer Society, 2006. Introduction of CryptoVerif — computationally sound security protocol verifier.
- [28] Andrey Bogdanov, Dmitry Khovratovich, and Christian Rechberger. Biclique cryptanalysis of the full AES. In *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 344–371. Springer, 2011.
- [29] Andrey Bogdanov, Lars R. Knudsen, Gregor Leander, Christof Paar, Axel Poschmann, Matthew J. B. Robshaw, Yannick Seurin, and Charlotte Viskelson. PRESENT: An ultra-lightweight block cipher. In *CHES 2007*, volume 4727 of *LNCS*, pages 450–466. Springer, 2007.
- [30] Jan Camenisch, Mark Manulis, and Gregory Neven. On the security of one-round protocols in the UC framework. In *Workshop on Security and Cryptography for Networks (SCN)*, 2010. UC session correctness via CNF.
- [31] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *Proceedings of the 42nd IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 136–145. IEEE Computer Society, 2001. Seminal paper introducing the Universal Composability framework.
- [32] Ran Canetti, Yevgeniy Dodis, Rafael Pass, and Shabsi Walfish. Universally composable security with global setup. In *TCC*, volume 4392 of *LNCS*, pages 61–85, 2007.
- [33] Joo Yeon Cho. Linear cryptanalysis of reduced-round PRESENT. In *CT-RSA 2010*, volume 5985 of *LNCS*, pages 232–248. Springer, 2010.
- [34] Carlos Cid, Sean Murphy, and Matthew Robshaw. *Algebraic Aspects of the Advanced Encryption Standard*. Information Security and Cryptography. Springer, 2006.
- [35] Nicolas T. Courtois. Fast algebraic attacks on stream ciphers with linear feedback. In *CRYPTO*, volume 2729 of *LNCS*, pages 176–194, 2003.
- [36] Cas Cremers, Marko Hövelmanns, and Antoine Rausch. Formally verifying TLS 1.3’s security: Tamarin proofs for all handshake modes. In *IACR ePrint*, 2017. Report 2017/323.
- [37] Joan Daemen, Lars R. Knudsen, and Vincent Rijmen. The block cipher Square. In *Fast Software Encryption (FSE) 1997*, volume 1267 of *LNCS*, pages 149–165. Springer, 1997.
- [38] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES—The Advanced Encryption Standard*. Springer, 2002.
- [39] Don Davis and Ralph Swick. Network security via private-key certificates. In *Proceedings of the 12th ACM Symposium on Operating Systems Principles*, pages 2–12. ACM, 2001. Concatenation ambiguity in authentication protocols.
- [40] Christophe De Cannière and Bart Preneel. Trivium. In *New Stream Cipher Designs: The eSTREAM Finalists*, volume 4986 of *LNCS*, pages 244–266. Springer, 2008.
- [41] Patrick Derbez, Pierre-Alain Fouque, and Jérémy Jean. Improved key recovery attacks on reduced-round AES in the single-key setting. In *Advances in Cryptology – EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 371–387. Springer, 2013.
- [42] Itai Dinur and Adi Shamir. Cube attacks on tweakable black box polynomials. In *Advances in Cryptology – EUROCRYPT 2009*, volume 5479 of *LNCS*, pages 278–299. Springer, 2009.

- [43] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schl  ffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34(3):1–42, 2021.
- [44] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the Fiat-Shamir transformation in the quantum random-oracle model. In *CRYPTO*, volume 11693 of *LNCS*, pages 356–383, 2019. Measure-and-reprogram technique for QROM proofs.
- [45] Benjamin Dowling, Marc Fischlin, Felix G  nther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol. In *Journal of Cryptology*, volume 34, pages 1–69. Springer, 2021.
- [46] Marc Fischlin, Felix G  nther, and Giorgia Azzurra Marson. Data is a stream: Security of stream-based channels. In *CRYPTO*, volume 9216 of *LNCS*, pages 545–564, 2015. 0-RTT security model for TLS.
- [47] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In *Advances in Cryptology – CRYPTO 2018*, volume 10992 of *LNCS*, pages 33–62. Springer, 2018.
- [48] Shafi Goldwasser and Silvio Micali. Probabilistic encryption. *Journal of Computer and System Sciences*, 28(2):270–299, 1984. Foundational paper introducing semantic security and IND-CPA.
- [49] Jens Grossklags, Nicolas Christin, and John Chuang. Secure or insure? In *WWW*, pages 209–218, 2008.
- [50] Martin E. Hellman. A cryptanalytic time-memory trade-off. *IEEE Transactions on Information Theory*, 26(4):401–406, 1980.
- [51] Dennis Hofheinz, Kathrin H  velmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In *TCC*, volume 10677 of *LNCS*, pages 341–371, 2017. QROM analysis of the FO transform.
- [52] Senyang Huang and Xuejia Lai. Revisiting key-alternating Feistel ciphers for shorter keys and multi-user security. In *Advances in Cryptology – ASIACRYPT 2020*, volume 12491 of *LNCS*, pages 213–243. Springer, 2020. Conditional cube attack extensions on stream ciphers.
- [53] IEEE. IEEE standard for information technology—telecommunications and information exchange between systems—local and metropolitan area networks—specific requirements—part 11: Wireless LAN medium access control (MAC) and physical layer (PHY) specifications—amendment 6: Medium access control (MAC) security enhancements. IEEE Std 802.11i-2004, 2004.
- [54] Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *LNCS*, pages 463–481. Springer, 2003.
- [55] ISO/IEC. Information technology—security techniques—entity authentication—part 3: Mechanisms using digital signature techniques. ISO/IEC 9798-3:2019, 2019.
- [56] Tibor Jager, Florian Kohlar, Sven Sch  ge, and J  rg Schwenk. On the security of TLS-DHE in the standard model. In *CRYPTO*, volume 7417 of *LNCS*, pages 273–293, 2012.
- [57] Jakob Jakobsen and Claudio Orlandi. On the CCA (in)security of MTPProto. In *SPSM*, pages 113–116, 2016.

- [58] Richard M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.
- [59] John Kelsey, Bruce Schneier, and David Wagner. Protocol interactions and the chosen protocol attack. In *Security Protocols, 5th International Workshop*, volume 1361 of *LNCS*, pages 91–104. Springer, 1997. Cross-protocol interaction attacks.
- [60] Dmitry Khovratovich and Ivica Nikolić. Rotational cryptanalysis of ARX. In *FSE*, volume 6147 of *LNCS*, pages 333–346, 2010.
- [61] Hugo Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. *IACR Cryptology ePrint Archive*, 2010:264, 2010.
- [62] Hugo Krawczyk and Pasi Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). IETF RFC 5869, 2010.
- [63] Gavin Lowe. Breaking and fixing the Needham–Schroeder public-key protocol using FDR. In Tiziana Margaria and Bernhard Steffen, editors, *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 1055 of *Lecture Notes in Computer Science*, pages 147–166. Springer, 1996. First report of the man-in-the-middle attack on the 1978 Needham–Schroeder public-key protocol.
- [64] Raghav Hari Makarim and Daniel N. Rockmore. On the algebraic degree of Trivium output bits. Manuscript, unpublished, 2021. Analyses growth of algebraic degree of Trivium keystream bits with rounds; see also [80] for related cube-attack degree bounds.
- [65] Moxie Marlinspike and Trevor Perrin. The X3DH key agreement protocol. Signal Foundation, 2016. <https://signal.org/docs/specifications/x3dh/>.
- [66] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The TAMARIN prover. In *CAV*, volume 8044 of *LNCS*, pages 696–701, 2013.
- [67] Bodo Möller, Thai Duong, and Krzysztof Kotowicz. This POODLE Bites: Exploiting the SSL 3.0 Fallback. In *Google Security Advisory*, 2014.
- [68] National Institute of Standards and Technology. FIPS 197: Advanced encryption standard (AES). Federal information processing standards publication, NIST, Gaithersburg, MD, November 2001. Original publication 26 November 2001; updated 9 May 2023 as FIPS 197-upd1.
- [69] National Institute of Standards and Technology. Lightweight Cryptography Standardization Process, 2023. NIST Selected Ascon.
- [70] National Institute of Standards and Technology. FIPS 203: Module-lattice-based key-encapsulation mechanism standard. Federal information processing standards publication, NIST, 2024. Includes QROM security proof for ML-KEM.
- [71] National Institute of Standards and Technology. FIPS 205: Stateless hash-based digital signature standard. Federal information processing standards publication, NIST, Gaithersburg, MD, August 2024. Derived from SPHINCS+; specifies SLH-DSA; effective August 14, 2024.
- [72] National Institute of Standards and Technology. FIPS 206 (initial public draft): FFT over NTRU-lattice-based digital signature standard (FN-DSA). Federal information processing standards publication (draft), NIST, Gaithersburg, MD, September 2025. Derived from Falcon; specifies FN-DSA; final expected late 2026–2027.

- [73] Roger M. Needham and Michael D. Schroeder. Using encryption for authentication in large networks of computers. *Communications of the ACM*, 21(12):993–999, 1978.
- [74] Jack O’Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O’Hearn. Blake3: One function, fast everywhere, 2020. Specification, <https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf>.
- [75] Trevor Perrin and Moxie Marlinspike. The double ratchet algorithm. Signal Foundation, 2016. <https://signal.org/docs/specifications/doubleratchet/>.
- [76] Eric Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. IETF RFC 8446, 2018. <https://www.rfc-editor.org/rfc/rfc8446>.
- [77] Phillip Rogaway. Authenticated-encryption with associated-data. In *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, pages 98–107. ACM, 2002. Introduced the AEAD notion now ubiquitous in TLS, IPsec, and WireGuard.
- [78] Robin Seggelmann, Michael Tuexen, and Michael Glenn Williams. CVE-2014-0160: The Heartbleed bug. OpenSSL Security Advisory, 2014. Buffer over-read in TLS heartbeat extension; disclosed April 2014.
- [79] Victor Shoup. Sequences of games: A tool for taming complexity in security proofs. Cryptology ePrint Archive, Paper 2004/332, 2004. Seminal paper introducing game-hopping methodology.
- [80] Yosuke Todo, Takanori Isobe, Yonglin Hao, and Willi Meier. Cube attacks on non-blackbox polynomials based on division property. In *CRYPTO*, volume 10403 of *LNCS*, pages 250–279, 2017.
- [81] Dominique Unruh. Revocable quantum timed-release encryption. In *EUROCRYPT*, volume 9057 of *LNCS*, pages 129–158, 2015. Introduces the One-Way to Hiding (O2H) lemma for QROM security proofs.
- [82] Mathy Vanhoef and Frank Piessens. Key reinstallation attacks: Forcing nonce reuse in WPA2. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1313–1328. ACM, 2017.
- [83] Mathy Vanhoef and Eyal Ronen. Dragonblood: Analyzing the Dragonfly handshake of WPA3 and EAP-pwd. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 517–533. IEEE, 2020.

* Simple Terms: How to Read the Correspondence Table

The table below maps every concept from the four traditional security paradigms to its MTSF equivalent. Use it as a dictionary:

- **Left column:** The traditional concept (e.g., “adversary advantage”).
- **Right column:** The MTSF translation (e.g., “ask price”).
- **Row grouping:** Concepts are grouped by paradigm (game-based, UC, formal verification, insecurity).

Quick reference for the most important translations:

You know it as . . .	In MTSF, it is called . . .
Adversary	Buyer
Challenger	Seller
Advantage ϵ	Ask price
Security game	Market
Game hop	Price adjustment / Bidding round
Security property	Security good
Secure scheme	Market equilibrium
Broken scheme	Market collapse
UC environment $\mathcal{Z}$	Market regulator
Simulator $\mathcal{S}$	Market arbitrageur
ProVerif trace check	Market stress test
Concrete attack	Successful bid

A MTSF Correspondence Table

Table 20: Classical-to-MTSF correspondence: complete mapping of cryptographic concepts to market-theoretic language.

Classical Concept	MTSF Concept	Market Status / Notes
Challenger	Seller	Offers security goods at ask price
Adversary $\mathcal{A}$	Buyer	Bids computational resources to win goods
Advantage $\leq \text{negl}$	Ask $\leq \text{negl}$	Equilibrium (secure)
Advantage = 1	Ask = 1	Collapsed (insecure)

Continued on next page

Table 20 (continued from previous page)

Classical Concept	MTSF Concept	Market Status / Notes
Advantage $\epsilon \in (0, 1)$	Ask = ϵ	Partial equilibrium (weakened security)
Game hop $\mathbf{B}_k \rightarrow \mathbf{B}_{k+1}$	Price adjustment ΔPrice_k	Transaction cost (proof step)
Difference lemma (1 failure)	Single-bid bound: $\Pr[F]$	One market risk
Extended diff. lemma (m failures)	Multi-bid bound: $\Pr[\bigcup F_i]$	Correlated risks, inclusion-exclusion
Inclusion-exclusion tightening	Correlated bid discount	Risk diversification reduces bound
Tight reduction	Efficient market (zero spread)	Optimal proof quality
Loose reduction	Inefficient market (large spread)	Suboptimal proof
UC environment $\mathcal{Z}$	Market regulator	Oversees all concurrent trades
UC ideal functionality $\mathcal{F}$	Security good specification	Defines what seller guarantees
Simulator $\mathcal{S}$	Market arbitrageur	Bridges real and ideal worlds
UC composition theorem	Market merger theorem	Equilibrium preserved under composition
GUC shared functionality	Market infrastructure (PKI, CRS)	Public goods, accessible to all
GUC cross-session	Market federation	Cross-session SID binding
Session-CNF satisfiable (honest)	Market audit passes	Correct session
Session-CNF SAT under dishonest	Audit failure (false pass)	CNF design failure (insecure)
Session-CNF UNSAT under dishonest	Audit passes correctly	Secure protocol (dishonest $\Rightarrow$ UNSAT)
CNF clause φ^{sid}	SID-binding audit clause	Cross-session replay blocked
CNF clause φ^{fresh}	Nonce freshness clause	Full-session replay blocked
CNF clause φ^{sig}	Signature verification clause	Impersonation blocked
CNF clause φ^{mac}	MAC verification clause	Ciphertext injection blocked
CNF clause φ^{ping}	Unbounded ping clause	Multi-session replay blocked
Replay / Masquerade / MITM	Stale bid / ID fraud / Manipulation	Fraud bid types
NS MITM attack	Market collapse via masquerade	Ask = 1, CNF design failure
RSA homomorphism	Market collapse via free forgery	Ask = 1, algebraic vulnerability
Lowe's identity-binding fix	CNF clause addition	Restores dishonest $\Rightarrow$ UNSAT
Authentication game	Entity verification good g_{auth}	Impersonation bid bounded by EUF-CMA
Mutual authentication game	Bidirectional good g_{mutual}	Dual impersonation bid
Session-key secrecy game	Key indistinguishability good g_{sk}	Key-distinguishing bid
CNF checking game	Audit integrity good g_{CNF}	Dishonest-satisfaction bid
Certificate authenticity	CA forgery good g_{cert}	EUF-CMA on CA signing key
Differential cryptanalysis	Differential bid	S-box differential propagation
Linear cryptanalysis	Linear bid	Linear approximation bias
Rotational cryptanalysis	Rotational bid	Rotation-commutation break at χ
Related-key attack	Related-key bid	Key schedule linearity exploitation
State recovery (stream cipher)	Internal state bid	Register inversion via algebraic methods
Key recovery (stream cipher)	Master key bid	Initialisation inversion
Distinguishing attack	Statistical bias bid	Output filter nonlinearity test
TMTO attack	Precomputation bid	Time-memory trade-off on state space
Hash collision attack	Capacity collision bid	Birthday bound on sponge capacity
Preimage attack	Capacity inversion bid	Capacity inversion resistance
Length-extension attack	State continuation bid	Capacity isolation eliminates this

Continued on next page

Table 20 (continued from previous page)

Classical Concept	MTSF Concept	Market Status / Notes
BLAKE3 Merkle tree LE immunity ASCON-Hash lightweight sponge	Structural immunity bid ($\text{Ask} = 0$) Lightweight capacity bid ($c=256$)	Tree structure blocks state continuation Tighter bounds, still negligible
PRESENT lightweight PRP Serpent conservative PRP Serpent boomerang attack ChaCha20 feedforward ChaCha20 nonce misuse Trivium cube attack Trivium three-register coupling	Lightweight differential/linear bids Conservative margin bids Two-differential composition bid Feedforward inversion bid Nonce reuse collapse bid ($\text{Ask} = 1$) Cube superpoly bid Circular feedback bid	64-bit block birthday constraint 20-round margin beyond best attack Adaptive chosen plaintext/ciphertext Addition prevents state inversion Identical keystreams leak $m_1 \oplus m_2$ Dimension- d IV summation $A \rightarrow B \rightarrow C \rightarrow A$ nonlinear mixing
Classical ROM reprogramming QROM reprogramming Measure-and-reprogram QROM correctness error QROM implicit rejection O2H square-root factor Classical vs. QROM	Free game hop (zero price) O2H bid: $2(q_H + 1)\sqrt{\epsilon}$ Extraction bid: $\sqrt{\epsilon}$ Correctness bid: $q_D\delta$ Rejection consistency bid Quantum bid price Linear vs. square-root price	Syntactic, no adversary detects Square-root cost from superposition Quantum measurement back-action PKE decapsulation failure probability Prevents chosen-ciphertext leakage Cost of defeating quantum adversary Quantum hardness amplification
Telegram salt extraction HMAC-bound salt (RMTP) MTPProto market collapse RMTP equilibrium	Salt extraction bid (“free” bid) Salt secrecy bid bounded by PRF Partial collapse ($\text{Ask} \not\leq \text{negl}$) Full equilibrium ($\text{Ask} \leq \text{negl}$)	Collapses $\text{Ask}(g_{\text{IND}})$ to ≈ 1 Restores equilibrium 64-bit security margin only 128-bit security restored
BB84 qubit preparation BB84 eavesdropping No-cloning theorem Info-disturbance trade-off Error estimation Privacy amplification QKD information-theoretic security	Quantum seller channel $\mathcal{E}_k^S$ Quantum buyer channel $\mathcal{E}_k^B$ No-cloning bid constraint Quantum price floor Quantum market audit Quantum market restructuring Quantum equilibrium (physics-based)	Non-orthogonal state encoding Arbitrary CPTP map on intercepted qubits Buyer cannot copy seller’s goods Minimum disturbance per bit of information Statistical detection of buyer’s disturbance QLHL extracts clean key from partial leakage No computational assumption required
UC composition theorem Concurrent protocol execution Shared CRS / PKI UC simulator Adversary’s time budget Protocol-independent SIDs Composition overhead	Market merger theorem Protocol market network $\mathcal{N}$ Market infrastructure $\mathcal{I}$ Market arbitrageur Buyer’s resource budget Global SID uniqueness clause Additive ask price	Two equilibria merge to network equilibrium N markets sharing infrastructure Public goods accessible to all markets Bridges real and ideal in merged market $\sum t_i \leq T$ across all markets Cross-market CNF consistency $\text{Ask}_{\mathcal{N}} \leq \sum \epsilon_i + \epsilon_{\mathcal{I}}$

* Simple Terms: How to Read the Bid Taxonomy

The bid taxonomy classifies every type of attack encountered in MTSF’s case studies. Each bid has:

- **Name:** A descriptive label (e.g., “nonce birthday bid”).
- **Target:** Which security good the bid attacks.
- **Success condition:** When does the bid succeed (market collapses)?
- **Cost:** The probability of success—the “price” the attacker pays.
- **Example:** A concrete case study where this bid appears.

The taxonomy serves two purposes:

1. **For proof construction:** When proving a new scheme secure, consult the taxonomy to ensure all relevant bid types are addressed. Missing a bid type means missing a potential attack.
2. **For security auditing:** When reviewing an existing proof, check that each bid in the taxonomy has been either (a) bounded (shown to fail) or (b) explicitly excluded (e.g., “nonce-misuse is out of scope”).

Think of it as a *checklist of known attack classes*: differential, linear, rotational, birthday, forgery, homomorphism, masquerade, replay, side-channel, quantum (O2H), and many more. A complete MTSF proof addresses every relevant bid type from this taxonomy.

B Comprehensive Bid Taxonomy

Table 21: Complete bid taxonomy across all case studies, with price bounds and formal characterisation.

Category	Bid Type	Target	Price Bound	Example
Primitives	Nonce bid	Birthday collision	$q_{\mathbb{G}}^2/(2q)$	ECDSA $\mathbf{B}_1$
	Hash collision bid	ROM collision	$q_H^2/(2q)$	ECDSA $\mathbf{B}_2$
	Forgery-to-hardness	ECDLP/MSIS/SIS	$\text{Adv}^{\text{ECDLP}}$	ECDSA $\mathbf{B}_3$ – $\mathbf{B}_4$
	Homomorphism bid	Algebraic structure	1 (collapse)	Textbook RSA
	Gaussian sampling bid	Norm distribution	Δ_{DGS}	FN-DSA $\mathbf{B}_2$
	TSPR preimage bid	Target-sum hash	$k \cdot \text{Adv}^{\text{TSPR}}$	SLH-DSA $\mathbf{B}_2$

Continued on next page

Table 21 (continued from previous page)

Category	Bid Type	Target	Price Bound	Example
Symmetric	PRF inner replacement	Inner keyed hash	$\text{Adv}_{\text{PRF}}^{\text{PRF}}$	HMAC $\mathbf{B}_1$
	PRF outer replacement	Outer keyed hash	$\text{Adv}_{\text{PRF}}^{\text{PRF}}$	HMAC $\mathbf{B}_2$
	MAC forgery bid	Tag computation	$\text{Adv}_{\text{SUF-CMA}}^{\text{SUF-CMA}}$	AEAD INT-CTXT
	Decapsulation bypass	Implicit rejection	$q_D/2^\gamma$	ML-KEM $\mathbf{B}_1$
	Inner collision bid	Birthday on outputs	$q_T^2/2^{n+1}$	HMAC $\mathbf{B}_3$
Block cipher	Differential bid	S-box propagation	$q_E \cdot 2^{-150}$	AES $\mathbf{B}_2$ – $\mathbf{B}_3$
	Linear bid	Linear approximation	$q_E^2 \cdot 2^{-150}$	AES linear
	Rotational bid	Rotation commutation	$q_E \cdot 2^{-128}$	AES rotational
	Related-key bid	Key schedule	N/A for AES-128	AES-256
	Algebraic bid	MQ system	$> 2^{128}$ ops	AES Gröbner
	Lightweight diff. bid	4-bit S-box propagation	$q_E \cdot 2^{-62}$	PRESENT
	Lightweight linear bid	4-bit S-box bias	$q_E^2 \cdot 2^{-62}$	PRESENT
	Boomerang bid	Two-differential composition	$q_E \cdot 2^{-128}$	Serpent
	Conservative margin bid	20-round attack gap	$q_E \cdot 2^{-196}$	Serpent
Hash function	Capacity collision bid	Sponge inner state	$q_f^2/2^{c+1}$	Keccak CR
	Capacity inversion bid	State preimage	$q_f/2^c$	Keccak preimage
	Length-extension bid	State continuation	$q_f/2^c$	Keccak LE
	Differential trail bid	Round structure	DP^{24}	Keccak $\mathbf{B}_2$
	Merkle tree collision bid	Compression collision	$q_f^2/2^{257}$	BLAKE3 CR
	Structural LE immunity bid	Tree structure	0	BLAKE3 LE
	Lightweight capacity collision	Small sponge ($c=256$)	$q_f^2/2^{257}$	ASCON-Hash CR
	Lightweight capacity inversion	Small sponge preimage	$q_f/2^{256}$	ASCON-Hash Pre
Stream cipher	State recovery bid	Full state inversion	$L \cdot 2^{-256}$	Grain-128a
	Key recovery bid	Master key extraction	$q_{\text{IV}} \cdot 2^{-128}$	Grain-128a
	Distinguishing bid	Statistical bias	$L^2 \cdot 2^{-97}$	Grain-128a
	TMTO bid	Precomputation	$TD/2^{256}$	Grain-128a
	Algebraic/cube bid	Polynomial system	$> 2^{64}$ (cube)	Grain variants
	Feedforward inversion bid	ARX state recovery	$L \cdot 2^{-256}$	ChaCha20
	Nonce-misuse collapse bid	Nonce reuse	1 (collapse)	ChaCha20
	Cube superpoly bid	IV summation	2^{-38}	Trivium
	Three-register correlation bid	Circular feedback bias	$L^2 \cdot 2^{-92}$	Trivium
Protocols	80-bit key recovery bid	Exhaustive key search	$q_{\text{IV}} \cdot 2^{-80}$	Trivium
	Nonce freshness bid	Nonce collision	$q_N^2/2^{\lambda+1}$	All protocols
	Signature forgery bid	EUF-CMA break	$\text{Adv}_{\text{EUF}}^{\text{EUF}}$	ISO 2P $\mathbf{B}_1$
	KEM key recovery bid	IND-CCA2 break	$\text{Adv}_{\text{IND-CCA2}}^{\text{IND-CCA2}}$	ISO 2P $\mathbf{B}_3$
	Impersonation bid	Entity authentication	$\text{Adv}_{\text{EUF}}^{\text{EUF}} + q_N^2/2^{\lambda+1}$	ISO 2P auth
	Masquerade bid	Identity fraud	1 (collapse)	NS MITM
	Cross-session replay bid	SID reuse	0 (SID-bound)	All protocols
	KDC signature bid	KDC authenticity	$2 \cdot \text{Adv}_{\text{EUF}}^{\text{EUF}}$	ISO 3P $\mathbf{B}_1$
	Correctness error bid	Decapsulation failure	$q_D \cdot \delta$	QKEM $\mathbf{B}_1$

Continued on next page

Table 21 (continued from previous page)

Category	Bid Type	Target	Price Bound	Example
	O2H reprogramming bid	Superposition detect	$2(q_H+1)\sqrt{\epsilon}$	QKEM $\mathbf{B}_2$
	Measure-and-reprogram bid	State disturbance	$\sqrt{\epsilon}$	QKEM $\mathbf{B}_3$
	IT hiding bid	Uniform key	0	QKEM $\mathbf{B}_4$
	Oracle restriction bid	Decapsulation info	0	QKEM $\mathbf{B}_5$
	Quantum replay bid	Superposition replay	$2(q_H+1)\sqrt{\epsilon}$	QKEM ping
	Quantum MAC forgery bid	Authentication break	$\text{Adv}^{\text{SUF-CMA}}$	QKEM auth
Telegram	Salt extraction bid	Entropy reduction	1 (free)	MTPProto $\mathbf{B}_1$
	AES partial-key bid	Meet-in-middle	$1 - 2^{-64}$	MTPProto $\mathbf{B}_2$
	CNF freshness failure bid	Salt secrecy clause	1 (clause fails)	MTPProto $\mathbf{B}_3$
	Ping degradation bid	Salt predictability	$\geq 2^{-64}$	MTPProto $\mathbf{B}_4$
	HMAC salt bid (RMTP)	PRF distinguishing	$\text{Adv}^{\text{PRF}}_{\text{HMAC}}$	RMTP $\mathbf{B}_1$
	Nonce collision bid (RMTP)	128-bit birthday	$q_D \cdot 2^{-128}$	RMTP $\mathbf{B}_2$
Quantum Market	No-cloning bid	Info-disturbance	$h(Q_{\text{obs}})$	BB84 $\mathbf{B}_1^Q$
	Error estimation bid	Statistical detection	$e^{-2k\gamma^2}$	BB84 $\mathbf{B}_2^Q$
	Privacy amplification bid	QLHL extraction	$2^{-\lambda/2}$	BB84 $\mathbf{B}_3^Q$
	EC leakage bid	Syndrome leakage	0 (absorbed)	BB84 $\mathbf{B}_4^Q$
	Quantum replay bid	Cross-session reuse	$2^{-\lambda/2}$	BB84 ping
Composition	Cross-protocol replay bid	SID collision	0 (label-bound)	TLS+Signal
	Simulation overhead bid	UC simulation cost	$\text{Ask}_{\text{other}}$	TLS+Signal
	Infrastructure forgery bid	CA cert forgery	$\text{Adv}^{\text{EUF}}_{\text{CA}}$	Shared PKI
	RNG degradation bid	PRG distinguishing	$\text{Adv}^{\text{PRG}}_{\text{RNG}}$	Shared RNG
CNF/Ping	SID replay bid	Cross-session replay	0 (SID-bound sigs)	All protocols
	Nonce reuse ping bid	IV/nonce repetition	$q_D/ \mathcal{N} $	AEAD ping
	Ciphertext replay bid	Old ciphertext reuse	$2(q_H+1)\sqrt{\epsilon}$	QKEM ping
	Session continuation bid	State continuation	$q_f/2^c$	Keccak ping
	Identity hijack bid	CNF identity clause	Adv^{EUF}	PKI ping

Table 22: Bid outcome classification: when does a bid succeed (collapse) vs. fail (equilibrium)?

Bid type	Succeeds when	Market outcome	Example
Nonce/birthday bid	$q^2 \geq \mathcal{N} $	Collapse if $q \geq 2^{n/2}$	ECDSA $q_S \geq 2^{128}$
Forgery-to-hardness	Underlying hard problem breaks	Collapse if ECDLP solvable	ECDSA
Homomorphism bid	Always (algebraic structure)	Immediate collapse	Textbook RSA
Masquerade bid	Identity not bound in protocol	Immediate collapse	NS protocol
O2H bid	$q_H \geq 2^{n/2}$ with $\sqrt{\epsilon}$	Equilibrium for ML-KEM	QKEM
Salt extraction bid	Salt accessible in plaintext	Partial collapse (2^{-64})	MTPProto
HMAC salt binding	auth_key secure	Equilibrium	RMTP
Nonce-misuse bid	Nonce reused with same key	Immediate collapse	ChaCha20
Cube superpoly bid	Superpoly degree ≤ 1	Equilibrium (2^{-38} margin)	Trivium
Lightweight birthday bid	$q_E \geq 2^{n/2}$ (block size)	Collapse if $q_E \geq 2^{32}$	PRESENT
Merkle tree LE bid	Never (structural)	Permanent equilibrium	BLAKE3
Boomerang bid	Two-differential composes	Equilibrium (20-round margin)	Serpent
No-cloning bid	Never (physics)	Permanent equilibrium	BB84
Cross-protocol replay	Never (label-bound SIDs)	Permanent equilibrium	TLS+Signal
Infrastructure forgery	CA signature broken	Network collapse	Shared PKI

Quick-Reference Cards

Quick-Reference Card 1: MTSF Notation Cheat Sheet

Symbol	Name	Meaning
$\mathcal{M}$	Security market	The complete auction setup
Seller	Seller	Challenger (runs scheme honestly)
Buyer _{<i>i</i>} / $\mathcal{A}$	Buyer	Adversary (bids computational resources)
$\mathcal{G}$	Goods catalogue	Set of security properties offered
g_j	Security good	One property (e.g., IND-CCA2, EUF-CMA)
$\text{Ask}(g_j)$	Ask price	Max advantage over all PPT buyers
QPrice_0	Quoted price	Raw advantage in $\mathbf{B}_0$ (baseline)
$\mathbf{B}_k$	Bidding round k	The k -th game in the hop chain
ΔPrice_k	Price adjustment	Advantage change from $\mathbf{B}_k$ to $\mathbf{B}_{k+1}$
$\text{negl}(\lambda)$	Negligible	Shrinks faster than any $1/\lambda^c$
φ_i	Session-CNF	Boolean checklist for session i
Ping	Session ping	Structural distinctness check

Quick-Reference Card 2: The MTSF Proof Checklist

- ☐ **Define market:** Seller, buyers, goods catalogue $\mathcal{G}$, price function.
- ☐ **$\mathbf{B}_0$ (Quoted Price):** State $\text{QPrice}_0 = \text{Adv}_{\mathcal{A}}^{g_j}(\lambda)$.
- ☐ **Nonce freshness bid:** $\Delta\text{Price} \leq q^2/2N$.
- ☐ **Hash collision bid:** $\Delta\text{Price} \leq q_H^2/2^{n+1}$.
- ☐ **Oracle programming:** $\Delta\text{Price} = 0$.
- ☐ **Computational hop:** $\Delta\text{Price} \leq \text{Adv}^{\text{hard}}$.
- ☐ **Forgery/extraction:** $\Delta\text{Price} \leq L \cdot \text{Adv}^{\text{hard}}$.
- ☐ **Ideal game:** $\Pr[\mathbf{B}_q = 1] \in \{0, 1/2\}$.
- ☐ **Total:** $\text{Ask}(g_j) \leq \sum \Delta\text{Price}_k \leq \text{negl}(\lambda)$.
- ☐ **CNF worksheet:** Fill in 4-column table for each clause.
- ☐ **Session pinging:** Verify structural distinctness.
- ☐ **Unbounded theorem:** Apply Theorem 5.5.
- ☐ **Bid taxonomy:** Check all relevant bids from Section B.

Quick-Reference Card 3: CNF Worksheet Template

Copy this template for each case study:

Clause C_j	Check	How to verify	Pass? (T/F)
φ^{sid}	SID matches session?	Compare SID in transcript	
φ^{fresh}	Nonces fresh?	Check nonce log	
φ^{sig}	Signatures verify?	Run $\text{Vrfy}(\text{pk}, m, \sigma)$	
φ^{mac}	MACs verify?	Run $\text{Mac.Vrfy}(K, m, \tau)$	
φ^{consist}	Protocol order OK?	Check state machine	
Result:	All T $\Rightarrow$ Session correct. Any F $\Rightarrow$ Identify failed clause.		

Quick-Reference Card 4: MTSF $\leftrightarrow$ Traditional Dictionary

Traditional	MTSF	Traditional	MTSF
Adversary $\mathcal{A}$	Buyer	Security game	Market
Challenger $\mathcal{C}$	Seller	Game hop	Bidding round
Advantage ϵ	Ask price	Failure event F	Bad bid
Secure	Equilibrium	Broken	Collapse
UC env. $\mathcal{Z}$	Regulator	Simulator $\mathcal{S}$	Arbitrageur
Composition	Market merger	ProVerif	Stress test
CRS	Infrastructure	GUC CNF	MTSF CNF

Quick-Reference Card 5: Market Verdict Decision Table

Ask Price	Verdict	Symbol	Examples
Ask $\leq \text{negl}(\lambda)$	Equilibrium	✓	ECDSA, ML-KEM, AES, TLS 1.3
Ask $= 2^{-64}$ (small)	Quasi-collapse	✱	Telegram MTPROTO
Ask $= 1$	Full collapse	✗	Textbook RSA, NS protocol
Ask $= 0$ (structural)	Permanent equil.	✓✓	BB84 QKD, BLAKE3 LE

Index

- AEAD, 113
- AES, 130
- ASCON
 - cross-function attack, 345
 - domain confusion, 345
 - initialisation vector, 344
 - multi-purpose, 344
 - permutation, 344
 - specification vs application, 348
- ASCON-Hash, 172, 344
- ask price, 30
- attack discovery, 314
 - AES, 318
 - ASCON-Hash, 319
 - BLAKE3, 319
 - block ciphers, 318
 - ChaCha20, 321
 - Grain-128a, 321
 - hash functions, 319
 - KEMs, 316
 - methodology, 315
 - Needham–Schroeder, 323
 - practical implementation, 353
 - PRESENT, 318
 - primitives, 316
 - protocols, 323
 - Serpent, 318
 - SHA-3, 319
 - Signal, 323
 - signatures, 316
 - stream ciphers, 321
 - summary, 353
 - TLS 1.3, 323
 - Trivium, 321
 - WPA2, 323
- attack prevention, 349
 - block ciphers, 355
 - composition, 350
 - hash functions, 356
 - primitives, 354
 - principle, 349
 - protocols, 358
 - stream ciphers, 357
 - verification workflow, 360
- attack surface
 - monotonicity, 350
- audit certificate, 351
 - composability, 351
- authentication game, 74
- BB84, 275
- bid completeness, 349
- bidding rounds, 23
- bidding space exploration, 315
- bidding-round proofs, 23
- birthday paradox
 - ticket nonce, 327
- BLAKE3, 167
- block cipher, 130
- certificate authenticity, 222
- certificate authority, 221
- certificates, 221
- ChaCha20, 191
- CNF
 - cross-session independence, 341
 - design flaw, 332
 - domain separation, 346
 - group key consistency, 336
 - identity binding, 332
 - IV entropy, 341
 - mode binding, 346
- CNF checking game, 83
- CNF clause inversion, 315
- CNF robustness, 349
- CNF session correctness, 27
- CNF subroutine chain, 70
- CNF verification, 43
- completeness, 59, 66
- concatenation ambiguity, 330
- conclusion, 365
- cross-protocol risk, 351
- cross-session attack, 315
- CRYPTO, 298
- difference lemma, 24
- differential cryptanalysis, 130
- Diffie–Hellman
 - ephemeral, 242
- domain separation
 - cross-function attack, 345
- Double Ratchet, 229
- downgrade protection, 254
- DragonBlood, 314
- EAPOL-Key, 334
- ECDSA, 85
- ECDSA soundness example, 69
- Elsevier, 298
- entity authentication, 74, 330

- equilibrium, 30
- EUF-CMA, 21
- EUROCRYPT, 298
- extended difference lemma, 35
- Falcon, 124
- field boundary confusion, 330
 - ISO 9798-3, 331
- FN-DSA, 124
- formal verification, 28
- forward secrecy, 229
- Fujisaki-Okamoto transform, 265
- Grain-128a, 178
 - initialisation, 338
 - low-entropy IV, 339
 - related-IV attack, 338
 - related-IV correlation attack, 339
 - specification vs deployment, 343
- group key
 - race condition, 335
- group key handshake, 334
- Group Temporal Key, 334
- GUC framework, 27
- hash function
 - domain separation, 344
- Heartbleed, 314
- HKDF, 242
 - resumption, 326
- HMAC, 104
- IEEE 802.11i, 334
- IEEE Transactions, 298
- implementation vulnerability, 328
- IND-CCA2, 21
- IND-CPA, 21
- infrastructure dependency, 315
- INT-CTXT, 22, 113
- IoT
 - domain separation, 344
 - IV generation, 338
- ISO/IEC 11770-3, 204
- ISO/IEC 9798-3, 330
 - parsing attack, 331
 - two-pass protocol, 330
- Keccak, 158
- key recovery attack, 178
- key synchronisation, 335
- keystream correlation, 338
- KRACK, 314
 - comparison, 338
- Kyber, 265
- lightweight cipher, 146
- market collapse, 101
- market network, 283
- market-theoretic security, 11
- ML-DSA, 97
- ML-KEM, 93, 265
- MTPProto, 258
- MTSF, 11
- multi-protocol composition, 283
- mutual authentication, 74
- mutual authentication game, 77
- Needham–Schroeder, 215
- NewSessionTicket, 326
- novel attack vectors, 314
- NP-hard reduction, 59
- NP-hard subroutine chain, 63
- NTRU, 124
- PKI, 221
 - equilibrium, 224
- post-compromise security, 229
- preliminaries, 19
- PRESENT, 146
- PRF, 22
- PRG, 179
- price adjustment, 33
- proactive security, 314
 - prevention framework, 349
- ProVerif, 28
- PRP, 131
- publication venue, 298
- QROM, 265
- quantum key distribution, 275
- quantum market dynamics, 275
- quantum random oracle model, 265
- quoted price, 29
- race condition
 - group key, 334
- random oracle model, 23
- related work, 360
- related-IV attack, 339
- related-key attack, 141
- replay attack, 251
- rotational cryptanalysis, 130
- RSA
 - textbook, 101
- security market, 28

- security research paper, 291
- semantic security, 20
- Serpent, 152
- Serpent equilibrium, 156
- session ping, 47
 - ASCON, 348
 - Grain-128a, 343
 - group key failure, 338
 - resumption failure, 330
- session resumption
 - vulnerability, 326
- session-key secrecy, 74, 80
- SHA-3, 158
- Signal Protocol, 229, 283
- SLH-DSA, 117
- soundness, 59, 65
- soundness-completeness duality, 67
- SPHINCS+, 117
- sponge construction, 158
- Springer, 298
- state recovery attack, 178
- stream cipher, 178
 - related-IV, 338
- subroutine consumption, 59, 61
- subroutine-consumption reduction, 61
- SUF-CMA, 22

- Tamarin, 28
- Telegram, 258
- ticket nonce
 - collision, 327
- time-memory-data trade-off, 189
- TLS 1.3, 204, 242, 283
 - 0-RTT, 251
 - collapse, 252
 - CNF verification, 255
 - downgrade
 - collapse, 254
 - downgrade attack, 254
 - equilibrium, 246
 - resumption nonce attack, 327
 - session resumption, 326
 - ticket nonce, 326
- TLS-style handshake, 221
- Transport Layer Security, 204, 242
- Trivium, 198

- UC framework, 24
- unbounded verification, 47
- unknown attacks, 314
 - group key race attack, 335
- writing guide, 291

- X3DH, 229

- zero-day attacks, 314

- WPA2, 334